

Core Java Preparation

A Code-Review Interview Companion for Java 21+

MAHDI SADEGHI

Core Java Preparation

by Mahdi Sadeghi.

Typeset from the Markdown sources in `chapters/` on 08 August 2026: 22 chapters, roughly 157,778 words. Rebuild with `python3 tools/build_pdf.py`.

Body text is set in Erewhon, headings in Source Sans Pro, and code in Source Code Pro. Code listings are highlighted with Pygments and the book is typeset with XeLaTeX.

Examples target Java 21 and later. Code in this book is intended for study and may be used freely.

Contents

Preface	xiii
Conventions Used in This Book	xiii
1 Java Fundamentals	1
Primitive Data Types	1
Variables, Scope, and Lifetime	2
Operators and Expressions	4
Control Flow Statements	5
Methods and Parameter Passing	7
Arrays	9
Strings, StringBuilder, and StringBuffer	11
Autoboxing and Unboxing	13
Varargs	14
Local Variable Type Inference (var)	16
Common Code-Review Interview Pitfalls	17
2 Object-Oriented Programming	23
Classes and Objects	23
Constructors	24
Encapsulation	27
Inheritance	28
Polymorphism	30

Abstraction	32
Method Overloading	33
Method Overriding	35
Access Modifiers	37
Static Members	38
Final Keyword	40
This and Super	42
Nested, Inner, Local, and Anonymous Classes	44
Common Code-Review Interview Pitfalls	48
3 Modern Java Language Features	53
Enums	54
Records	55
Sealed Classes and Interfaces	57
Interfaces (default, static, private methods)	59
Abstract Classes	61
Pattern Matching (instanceof, switch)	62
Primitive Types in Patterns	64
Switch Expressions	65
Text Blocks	67
String Templates (Preview)	68
Unnamed Variables and Patterns	69
Value-Based Classes	70
Preview and Incubator Features	72
Deprecated and Removed Features Across Java Versions	73
Common Code-Review Interview Pitfalls	76

4	Generics and Type System	81
	Generics	82
	Type Erasure	89
	Wildcards (extends, super)	94
	Covariance and Contravariance	97
	Common Code-Review Interview Pitfalls	100
5	Object Class & Common APIs	105
	equals(), hashCode(), and toString()	105
	Object Cloning	112
	Optional	115
	Comparator and Comparable	118
	Immutability	121
	Defensive Copying	124
	Common Code-Review Interview Pitfalls	128
6	Exception Handling	133
	Exceptions	133
	Checked vs Unchecked Exceptions	136
	Exception Handling Best Practices	138
	Try-with-Resources	144
	Assertions	148
	Common Code-Review Interview Pitfalls	151
7	Collections Framework	157
	Collections Framework Overview	157
	Collection Interface	159
	List	159
	Set	160

Queue	160
Deque	161
Map	162
Master “Which Collection Should I Pick?” Table	163
ArrayList	164
LinkedList	165
HashMap	166
LinkedHashMap	167
TreeMap	168
HashSet	170
LinkedHashSet	170
TreeSet	171
PriorityQueue	171
ArrayDeque	173
IdentityHashMap	173
WeakHashMap	174
EnumMap	175
EnumSet	176
Immutable Collections (List.of())	176
Concurrent Collections	178
Common Code-Review Interview Pitfalls	182
8 Functional Programming	189
Functional Interfaces	189
Lambda Expressions	192
Method References	194
Streams API	196
Collectors	201

Spliterator	205
Stream Performance	207
Parallel Streams	210
Common Code-Review Interview Pitfalls	212
9 Date, Time & Localization	219
Date and Time API (java.time)	219
Formatting and Parsing	228
Internationalization (i18n)	235
Localization (l10n)	244
Common Code-Review Interview Pitfalls	248
10 File I/O	253
java.io (File I/O)	253
NIO and NIO.2	259
Paths and Files	263
Serialization	271
Externalizable	277
Common Code-Review Interview Pitfalls	279
11 JVM Fundamentals	285
JDK, JRE, JVM Architecture	285
Compilation, Class Loading, and Execution	288
Bytecode	290
JIT Compilation	293
Class Loaders	297
Java Memory Model (JMM)	301
Common Code-Review Interview Pitfalls	306

12 Memory Management	311
Garbage Collection	311
Serial GC	315
Parallel GC	316
G1 GC	317
ZGC	318
Shenandoah	319
Collector Comparison	319
Memory Leaks	320
Escape Analysis	325
JVM Flags and Diagnostics	326
Garbage Collection Tuning	328
JVM Performance Tuning	329
Profiling and Monitoring	331
JFR (JDK Flight Recorder)	332
JMC (JDK Mission Control)	333
jcmd	333
jmap	335
jstack	336
Worked Walkthrough: Debugging an OutOfMemoryError	337
Worked Walkthrough: Debugging a Deadlock with jstack	338
Common Code-Review Interview Pitfalls	340
13 Concurrency (Core)	345
Threads	345
Runnable and Callable	348
Thread Lifecycle	350
Synchronization	352

Intrinsic Locks	356
ReentrantLock	357
ReadWriteLock	360
StampedLock	361
Volatile	363
Atomic Classes	365
ThreadLocal	367
Happens-Before Relationship	369
Visibility, Atomicity, and Ordering	371
Common Code-Review Interview Pitfalls	376
14 Concurrency (Advanced)	383
Executors Framework	383
ForkJoinPool	390
CompletableFuture	393
BlockingQueue	397
ConcurrentHashMap Internals	400
CountDownLatch	403
CyclicBarrier	403
Phaser	404
Semaphore	406
Exchanger	407
Common Code-Review Interview Pitfalls	408
15 Modern Concurrency (Java 21+)	415
Virtual Threads (Project Loom)	415
Structured Concurrency	425
Scoped Values	430

Comparison Table: Platform Threads vs Virtual Threads	435
Worked Example: Fan-Out to Three Downstream Calls, Three Ways	436
Common Code-Review Interview Pitfalls	439
16 Reflection & Runtime Features	445
Reflection	445
Annotations	450
Annotation Processing	453
Dynamic Proxies	456
Method Handles	458
VarHandles	461
Unsafe Alternatives	463
Service Provider Interface (SPI)	465
Common Code-Review Interview Pitfalls	467
17 Modules & Native Interoperability	475
Packages	477
Java Platform Module System (JPMS)	485
Foreign Function & Memory API	497
Vector API (Incubator)	506
Common Code-Review Interview Pitfalls	511
18 Networking & Security	517
Networking (java.net)	517
HTTP Client API	523
Process API	528
Security Basics	532
Cryptography APIs	540
Common Code-Review Interview Pitfalls	546

19 Software Engineering Best Practices	553
Clean Code in Java	553
SOLID Principles	560
Effective Java Best Practices	566
Design Patterns in Java	575
API Design Best Practices	586
Common Code Smells	592
Common Code-Review Interview Pitfalls	600
20 Testing	605
Testing Fundamentals (JUnit 5)	605
Mocking (Mockito)	628
Common Code-Review Interview Pitfalls	643
21 Build & Documentation	649
Maven	649
Gradle	663
Javadoc	672
Common Code-Review Interview Pitfalls	681
22 Code-Review Interview Focused Topics	687
The Review Checklist	688
How to Communicate in a Code-Review Interview	690
Common Java Interview Pitfalls	691
Code Review Scenarios	693
Performance Optimization	721
Thread Safety Review	724
Collections Selection	726
Exception Design	728

API Design Review 731

Memory & GC Review 734

Concurrency Review 736

Modern Java Feature Usage 740

Common Code-Review Interview Pitfalls 744

Preface

Each topic in this book is a self-contained chapter. Every chapter explains its subtopics in plain language with runnable Java examples, and ends with a *Common Code-Review Interview Pitfalls* section that collects the mistakes reviewers actually catch.

The material is written for someone who already knows Java but wants to review it sharply — the kind of review you do before a code-review-style interview. Examples target Java 21 and later, with notes where older or newer releases differ.

Conventions Used in This Book

The following typographic conventions are used:

Italic

Indicates emphasis and new terms where they are first defined.

Constant width

Used for program listings, as well as within paragraphs to refer to program elements such as variable and method names, data types, statements, keywords, class names, and command-line tools.

Code listings are set on a tinted background and syntax-highlighted. Where a single line of code or shell output is too long for the measure, it wraps and the continuation is marked with a ↵ symbol in the left margin.



This element signifies a general note.



This element signifies a tip or suggestion.



This element indicates a warning or a caution.

Sections marked off by rules and a small heading — like THE REVIEW blocks in the final chapter — hold worked answers. Try the exercise before reading them.

Java Fundamentals

Every code review starts with the basics: how data is stored, how variables behave, and how control flow is structured. Get these wrong and everything built on top of them — collections, streams, concurrency — inherits the bug. This chapter is a fast refresher on the building blocks of Java, written for someone who already knows the language but wants to review it sharply before a code-review-style interview. Examples target Java 21+, with notes where older or newer versions differ.

Primitive Data Types

A **primitive type** is a basic building block for data — not an object, just a raw value stored directly in memory (on the stack, or inline inside an object). Java has eight of them. They’re fast and predictable because there’s no object overhead: no header, no pointer indirection, no garbage collection tracking.

Type	Size	Range	Default value
byte	8 bits	-128 to 127	0
short	16 bits	-32,768 to 32,767	0
int	32 bits	-2,147,483,648 to 2,147,483,647	0
long	64 bits	$\sim -9.2 \times 10^{18}$ to 9.2×10^{18}	0L
float	32 bits	$\sim \pm 3.4 \times 10^{38}$ (7 sig. digits)	0.0f
double	64 bits	$\sim \pm 1.8 \times 10^{308}$ (15 sig. digits)	0.0d
char	16 bits	0 to 65,535 (unsigned, UTF-16 code unit)	'\u0000'
boolean	JVM-dependent (not specified)	true / false	false

```

public class PrimitiveBasics {
    public static void main(String[] args) {
        byte smallCount = 100;
        int userId = 4_500_231;           // underscores improve
        ↪ readability
        long totalBytes = 9_000_000_000L; // 'L' suffix required beyond
        ↪ int range
        double price = 19.99;
        char grade = 'A';
        boolean isActive = true;

        System.out.println(userId);       // prints: 4500231
        System.out.println((int) grade);  // prints: 65 (ASCII code for
        ↪ 'A')
    }
}

```

Gotcha — integer overflow is silent. Primitives wrap around instead of throwing an exception:

```

int max = Integer.MAX_VALUE;
System.out.println(max + 1); // prints: -2147483648 (wraps around, no
↪ error!)

```

Gotcha — floating-point is not exact. float and double use binary fractions, so they can't represent every decimal value exactly. This matters a lot for money-related code.

```

double a = 0.1;
double b = 0.2;
System.out.println(a + b);           // prints: 0.30000000000000004
System.out.println(a + b == 0.3);    // prints: false

// Use BigDecimal for money or exact decimal math
java.math.BigDecimal x = new java.math.BigDecimal("0.1");
java.math.BigDecimal y = new java.math.BigDecimal("0.2");
System.out.println(x.add(y));         // prints: 0.3

```

Variables, Scope, and Lifetime

A **variable** is a named storage location. **Scope** is *where in the code* a variable's name is visible. **Lifetime** is *how long* the variable's storage actually exists in memory. These two often line up, but not always — that's where bugs hide.


```

public class ScopeExample {
    static int instanceCounter = 0; // class (static) scope: lives for the
    ↪ whole program

    public static void main(String[] args) {
        int outer = 10; // method scope: lives for the duration of main()

        if (outer > 5) {
            int inner = 20; // block scope: only visible inside this
            ↪ if-block
            System.out.println(outer + inner); // prints: 30
        }

        // System.out.println(inner); // COMPILE ERROR: inner is out of
        ↪ scope here

        for (int i = 0; i < 3; i++) {
            int loopLocal = i * i; // re-created fresh on every iteration
            System.out.println(loopLocal); // prints: 0, then 1, then 4
        }
    }
}

```

Local variables live on the **stack** (or are inlined by the JVM) and disappear when their block ends. Instance fields live as long as their object does, tracked by the **heap** and garbage collector. Static fields live as long as the class is loaded — effectively the whole program run.

Gotcha — shadowing. A variable declared in an inner scope can hide (shadow) one with the same name in an outer scope, which is legal for local variables but easy to misread:

```

public class Shadowing {
    static int value = 100;

    public static void main(String[] args) {
        int value = 5; // shadows the static field within main()
        System.out.println(value); // prints: 5
        System.out.println(Shadowing.value); // prints: 100 (explicit
        ↪ access to the field)
    }
}

```

Gotcha — variables must be definitely assigned before use. Java won't let you read a local variable that might not have been initialized on every path:

```

int result;
if (Math.random() > 0.5) {
    result = 1;
}
// System.out.println(result); // COMPILER ERROR: result might not have
↳ been initialized

```

Operators and Expressions

An **operator** combines values into an **expression** that produces a new value. Java groups them into arithmetic, relational, logical, bitwise, assignment, and ternary categories. Precedence and evaluation order matter for review because subtle mistakes often hide in dense one-liners.

```

public class Operators {
    public static void main(String[] args) {
        int a = 7, b = 2;

        System.out.println(a / b); // prints: 3 (integer division
↳ truncates)
        System.out.println(a % b); // prints: 1 (remainder)
        System.out.println((double) a / b); // prints: 3.5 (cast forces
↳ floating-point division)

        boolean eligible = (a > 5) && (b < 10); // logical AND
        System.out.println(eligible); // prints: true

        int shifted = 1 << 4; // bitwise left shift: 1 * 2^4
        System.out.println(shifted); // prints: 16

        String label = (a > b) ? "a wins" : "b wins"; // ternary operator
        System.out.println(label); // prints: a wins
    }
}

```

Short-circuit evaluation. `&&` and `||` stop evaluating as soon as the result is known. `&` and `|` (single character, also usable on booleans) always evaluate both sides. This distinction matters when the right-hand side has side effects or could throw:

```

String input = null;
if (input != null && input.length() > 0) { // safe: length() never called
↳ on null
    System.out.println("non-empty");
}

```

```

    }

    // if (input != null & input.length() > 0) // would throw
    ↪ NullPointerException!

```

Gotcha — integer division truncates toward zero.

```

System.out.println(-7 / 2); // prints: -3 (not -4; truncates toward zero)
System.out.println(-7 % 2); // prints: -1 (sign follows the dividend)

```

Gotcha — mixing ++/-- inside larger expressions is legal but hard to read.

```

int i = 5;
int j = i++ + ++i; // i++ uses 5, then i becomes 6; ++i makes it 7 and
↪ uses 7
System.out.println(j); // prints: 12
System.out.println(i); // prints: 7

```

Avoid writing code like this in production — it compiles fine but reviewers should flag it for clarity.

Control Flow Statements

Control flow statements decide which code runs, and how many times. Java's core set is `if/else`, `switch`, `for`, `while`, `do-while`, plus `break`, `continue`, and (since Java 14) the **`switch expression`**.

```

public class ControlFlow {
    public static void main(String[] args) {
        int score = 82;

        // Classic if/else chain
        String grade;
        if (score >= 90) {
            grade = "A";
        } else if (score >= 80) {
            grade = "B";
        } else {
            grade = "C";
        }
        System.out.println(grade); // prints: B
    }
}

```

```

// Traditional switch statement (fall-through by default)
switch (grade) {
    case "A":
        System.out.println("Excellent");
        break; // without break, execution falls into the next
↳ case!

    case "B":
        System.out.println("Good");
        break;
    default:
        System.out.println("Keep trying");
}

// Modern switch expression (Java 14+): no fall-through, returns a
↳ value
String message = switch (grade) {
    case "A" -> "Excellent";
    case "B" -> "Good";
    default -> "Keep trying";
};
System.out.println(message); // prints: Good

// Loops
for (int i = 0; i < 3; i++) {
    System.out.print(i + " "); // prints: 0 1 2
}
System.out.println();

int n = 3;
while (n > 0) {
    System.out.print(n + " "); // prints: 3 2 1
    n--;
}
System.out.println();

// do-while always runs the body at least once
int attempts = 0;
do {
    attempts++;
} while (attempts < 0); // condition false immediately, but body
↳ ran once
System.out.println(attempts); // prints: 1
}
}

```

Gotcha — switch fall-through. Forgetting break in a classic switch statement lets execution continue into the next case:

```

int day = 6;
switch (day) {
    case 6:
    case 7:
        System.out.println("Weekend");
        break;
    default:
        System.out.println("Weekday");
}
// prints: Weekend (cases 6 and 7 intentionally share the same code – this
→ is fine)

```

Prefer the arrow-style `switch` expression in new code: it removes fall-through entirely and forces every path to return a value (the compiler checks exhaustiveness for enums and sealed types).

Gotcha — labeled break/continue. Rarely used, but useful for breaking out of nested loops:

```

outer:
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
        if (j == 1) {
            continue outer; // skips to the next i, not just the next j
        }
        System.out.println(i + "," + j);
    }
}
// prints: 0,0 1,0 2,0

```

Methods and Parameter Passing

A **method** is a named, reusable block of code. Java is famous for one rule that trips up almost every interview candidate: **Java is always pass-by-value**. What gets copied differs for primitives versus objects, and that difference explains most of the confusion.

- For **primitives**, the value itself is copied. Changes inside the method never affect the caller's variable.
- For **objects**, the *reference* (the “address” pointing to the object) is copied — not the object itself. The method can use that reference to mutate the object's internal state, but it cannot make the caller's variable point to a different object.

```

public class ParameterPassing {

    static void incrementPrimitive(int value) {
        value = value + 1; // only modifies the local copy
    }

    static void appendToList(java.util.List<String> list) {
        list.add("added inside method"); // mutates the SAME object the
        ↪ caller sees
    }

    static void reassignReference(java.util.List<String> list) {
        list = new java.util.ArrayList<>(); // rebinds the LOCAL copy of
        ↪ the reference only
        list.add("this is invisible to the caller");
    }

    public static void main(String[] args) {
        int number = 5;
        incrementPrimitive(number);
        System.out.println(number); // prints: 5 (unchanged)

        java.util.List<String> names = new java.util.ArrayList<>();
        names.add("Alice");
        appendToList(names);
        System.out.println(names); // prints: [Alice, added inside method]

        reassignReference(names);
        System.out.println(names); // prints: [Alice, added inside method]
        ↪ (unchanged!)
    }
}

```

Think of a reference like a piece of paper with a house address written on it. Passing it to a method hands over a *photocopy* of that paper. The method can walk to the house and rearrange the furniture (mutate the object) — everyone with a copy of the address sees the new furniture. But if the method writes a *different* address on its photocopy, that only affects its own slip of paper; your original paper still points to the old house.

Gotcha — “pass by reference” is a myth in Java. There is no way to make a method change which object a caller’s variable points to, without returning a new value:

```

static void swap(String a, String b) {
    String temp = a;
    a = b;
    b = temp; // only swaps the local copies
}

```

```

}

String x = "first";
String y = "second";
swap(x, y);
System.out.println(x + " " + y); // prints: first second (NOT swapped)

```

Method overloading lets multiple methods share a name if their parameter lists differ. It's resolved at compile time based on the declared (static) types of the arguments.

```

static void log(int code) {
    System.out.println("int overload: " + code);
}

static void log(String message) {
    System.out.println("String overload: " + message);
}

log(404);           // prints: int overload: 404
log("Error");       // prints: String overload: Error

```

Arrays

An **array** is a fixed-size, ordered container of elements of the same type, stored contiguously in memory. Once created, its length can never change — that's the key trade-off against dynamic structures like `ArrayList`.

```

public class ArrayBasics {
    public static void main(String[] args) {
        int[] scores = new int[3];           // all elements default to 0
        scores[0] = 90;
        scores[1] = 75;
        scores[2] = 88;

        int[] literalScores = {90, 75, 88}; // shorthand array literal

        System.out.println(scores.length); // prints: 3
        System.out.println(java.util.Arrays.toString(scores)); // prints:
        ↪ [90, 75, 88]

        // 2D array: an array of arrays
        int[][] grid = {
            {1, 2, 3},
            {4, 5, 6}
        }
    }
}

```

```

    };
    System.out.println(grid[1][2]); // prints: 6

    // Iterating
    for (int score : literalScores) {
        System.out.print(score + " "); // prints: 90 75 88
    }
    System.out.println();
}
}

```

Gotcha — arrays of objects default to null, not empty objects.

```

String[] names = new String[3];
System.out.println(names[0]); // prints: null
// names[0].length(); // throws NullPointerException

```

Gotcha — `ArrayIndexOutOfBoundsException` is a runtime error, not a compile-time one.

```

int[] values = {1, 2, 3};
// System.out.println(values[3]); // throws:
↪ ArrayIndexOutOfBoundsException: Index 3 out of bounds for length 3

```

Gotcha — arrays are covariant, which can cause a runtime exception. `String[]` is a subtype of `Object[]`, so you can assign one to the other — but the JVM still remembers the real element type and enforces it at runtime:

```

Object[] objects = new String[3]; // legal: array covariance
// objects[0] = Integer.valueOf(42); // throws: ArrayStoreException at
↪ runtime

```

Gotcha — `==` compares array references, not contents; `equals()` on arrays does too. Use `Arrays.equals()` for content comparison:

```

int[] a = {1, 2, 3};
int[] b = {1, 2, 3};
System.out.println(a == b); // prints: false
System.out.println(a.equals(b)); // prints: false
↪ (Object.equals, reference check)
System.out.println(java.util.Arrays.equals(a, b)); // prints: true

```


Strings, StringBuilder, and StringBuffer

A `String` in Java is **immutable** — once created, its contents can never change. Every “modification” (concat, substring, replace, +) actually creates a brand-new `String` object. This is why heavy string manipulation should avoid `String` and use `StringBuilder` or `StringBuffer` instead — mutable, resizable character containers.

```
public class StringBasics {
    public static void main(String[] args) {
        String greeting = "Hello";
        String upper = greeting.toUpperCase(); // returns a NEW String
        System.out.println(greeting); // prints: Hello (unchanged)
        System.out.println(upper);     // prints: HELLO

        // StringBuilder: mutable, efficient for repeated concatenation
        StringBuilder sb = new StringBuilder();
        for (int i = 1; i <= 3; i++) {
            sb.append("item").append(i).append(",");
        }
        sb.deleteCharAt(sb.length() - 1); // remove trailing comma
        System.out.println(sb.toString()); // prints: item1,item2,item3
    }
}
```

The string pool. String literals are interned (cached and reused) in a special pool. Two literals with the same content share the same object; objects created with `new String(...)` do not.

```
String a = "cache";
String b = "cache";
System.out.println(a == b); // prints: true (same pooled literal)

String c = new String("cache");
System.out.println(a == c); // prints: false (different object on
→ the heap)
System.out.println(a.equals(c)); // prints: true (same content)
```

Rule of thumb: always compare strings with `.equals()`, never `==`, unless you specifically know you’re comparing interned constants.

Feature	String	StringBuilder	StringBuffer
Mutable?	No	Yes	Yes
Thread-safe?	Yes (immutable objects are inherently safe)	No	Yes (synchronized methods)
Performance	Slow for repeated edits (new object each time)	Fast	Slower than StringBuilder due to locking
Typical use	Fixed or rarely-changed text	Single-threaded string building	Shared mutable buffer across threads (rare today)
Introduced	Java 1.0	Java 1.5	Java 1.0

In modern code, `StringBuffer` is mostly legacy. If you don't need cross-thread safety (and usually you don't, since string building is typically local to one method), use `StringBuilder`.

Gotcha — + in a loop is quietly expensive. The compiler optimizes a single chained + expression into a `StringBuilder` automatically, but each iteration of a loop using += on a `String` creates a brand-new object every time:

```
String result = "";
for (int i = 0; i < 5; i++) {
    result += i; // creates a new String object on every iteration —
    ↪ O(n^2) overall
}
System.out.println(result); // prints: 01234
// Better: use a StringBuilder and call .append(i) in the loop
```

Gotcha — text blocks (Java 15+) make multi-line strings much cleaner, but indentation is significant:

```
String json = """
    {
        "name": "Ada"
    }
    """;
System.out.println(json.strip());
// prints:
// {
//     "name": "Ada"
```

```
// }
```

Autoboxing and Unboxing

Autoboxing is the automatic conversion of a primitive value into its corresponding **wrapper object** (e.g., `int` → `Integer`). **Unboxing** is the reverse. The compiler inserts these conversions for you, which is convenient — but it hides real object creation and real `NullPointerException` risk.

Primitive	Wrapper class
<code>byte</code>	<code>Byte</code>
<code>short</code>	<code>Short</code>
<code>int</code>	<code>Integer</code>
<code>long</code>	<code>Long</code>
<code>float</code>	<code>Float</code>
<code>double</code>	<code>Double</code>
<code>char</code>	<code>Character</code>
<code>boolean</code>	<code>Boolean</code>

```
public class BoxingBasics {
    public static void main(String[] args) {
        int primitiveValue = 42;
        Integer boxed = primitiveValue;    // autoboxing: int -> Integer
        int unboxed = boxed;               // unboxing: Integer -> int

        java.util.List<Integer> numbers = new java.util.ArrayList<>();
        numbers.add(10); // autoboxed to Integer before being stored
        int first = numbers.get(0); // unboxed back to int
        System.out.println(first); // prints: 10
    }
}
```

Gotcha — unboxing null throws `NullPointerException`. This is one of the most common review findings in real codebases:

```
Integer count = null;
// int total = count + 1; // throws NullPointerException during unboxing!

Map<String, Integer> inventory = new HashMap<>();
// int qty = inventory.get("widget"); // NPE if "widget" isn't in the map
↪ (get returns null)
int qty = inventory.getDefault("widget", 0); // safe
```

Gotcha — the Integer cache means == sometimes “works” and sometimes doesn’t. The JVM caches boxed Integer values from -128 to 127. Comparing cached values with == accidentally passes; comparing larger values fails:

```
Integer a = 100;
Integer b = 100;
System.out.println(a == b); // prints: true (both pulled from the internal
↪ cache)

Integer c = 200;
Integer d = 200;
System.out.println(c == d); // prints: false (outside cache range,
↪ different objects)

System.out.println(c.equals(d)); // prints: true (always correct – compare
↪ wrapper objects with equals)
```

Gotcha — mixing primitives and wrappers in arithmetic can silently unbox and throw. Also, autoboxing in tight loops (e.g., `Long sum = 0L; sum += i;` inside a million-iteration loop) creates a huge number of short-lived wrapper objects, hurting performance. Prefer primitive accumulator variables in hot loops.

Varargs

Varargs (variable-length arguments) let a method accept zero or more arguments of a type without the caller having to build an array explicitly. Internally, the compiler packages the arguments into an array. The syntax is `Type . . . name`, and it must be the *last* parameter in the method signature.

```
public class VarargsBasics {

    static int sum(int... numbers) {
        int total = 0;
        for (int n : numbers) {
```

```

        total += n;
    }
    return total;
}

public static void main(String[] args) {
    System.out.println(sum());           // prints: 0 (zero arguments
    ↪ is allowed)
    System.out.println(sum(5));           // prints: 5
    System.out.println(sum(1, 2, 3, 4)); // prints: 10

    int[] existingArray = {10, 20, 30};
    System.out.println(sum(existingArray)); // prints: 60 (an array
    ↪ can be passed directly)
}

```

You’ve already used varargs if you’ve called `String.format(...)`, `List.of(...)`, or `System.out.printf(...)`.

Gotcha — varargs and overload resolution can be ambiguous or surprising. A specific-arity overload is always preferred over the varargs version when both match:

```

static void report(int a, int b) {
    System.out.println("specific overload");
}

static void report(int... values) {
    System.out.println("varargs overload");
}

report(1, 2); // prints: specific overload (exact match wins over varargs)
report(1, 2, 3); // prints: varargs overload (no exact match exists)

```

Gotcha — mixing varargs with generics risks heap pollution. The compiler warns about “unchecked generics array creation” because a varargs array of a generic type can be assigned to `Object[]` and have incompatible elements inserted, breaking type safety at runtime. Annotate such methods with `@SafeVarargs` only if you’ve verified the method doesn’t misuse the array.

```

@SafeVarargs
static <T> java.util.List<T> listOf(T... items) {
    return java.util.Arrays.asList(items);
}

```

Gotcha — an ambiguous null argument. Passing `null` directly to a varargs method is ambiguous to a human reader (and sometimes to the compiler, if multiple overloads exist) — it's treated as a `null array`, not a single `null` element:

```
static void printAll(String... items) {
    System.out.println(items == null ? "null array" : items.length + "
↪ items");
}

printAll(null); // prints: null array (the whole array reference is null)
printAll((String) null); // prints: 1 items (a single-element array
↪ containing null)
```

Local Variable Type Inference (var)

Since Java 10, the `var` keyword lets the compiler infer a local variable's type from the right-hand side of its initialization. It is **not** dynamic typing — the type is fixed at compile time, just written implicitly. `var` only works for local variables with an initializer; it cannot be used for fields, method parameters, or return types.

```
public class VarBasics {
    public static void main(String[] args) {
        var name = "Alice"; // inferred as String
        var count = 10; // inferred as int
        var prices = new java.util.ArrayList<Double>(); // inferred as
↪ ArrayList<Double>
        prices.add(19.99);

        // name = 42; // COMPILE ERROR: name's type is fixed as String,
↪ not dynamic

        for (var price : prices) { // var also works in for-each loops
            System.out.println(price); // prints: 19.99
        }
    }
}
```

Gotcha — `var` requires an initializer, and can't infer from `null` alone.

```
// var x; // COMPILE ERROR: cannot infer type without an initializer
// var y = null; // COMPILE ERROR: cannot infer type from null
var z = (String) null; // legal: explicit cast tells the compiler the type
```

Gotcha — var can obscure the actual type in a review, hurting readability. This compiles but is a common review comment:

```
var result = service.process(request); // what type is "result"? Not
↳ obvious from this line alone.
```

var is best used when the type is already obvious from the right-hand side (e.g., `var list = new ArrayList<String>();`) and best avoided when it hides meaningful information (e.g., the return type of a method call the reader doesn't already know).

Gotcha — var with diamond operator loses generic type information if there's no explicit type argument.

```
var list = new ArrayList<>(); // inferred as ArrayList<Object>, NOT the
↳ type you might expect
list.add("text");
list.add(42); // compiles fine – the list is untyped (Object), losing
↳ compile-time safety
```

Common Code-Review Interview Pitfalls

1. **Using == to compare String or wrapper objects instead of .equals().** Why it matters: == compares references (or cached instances), not logical content, and fails unpredictably outside the small-integer cache or literal pool.

```
// Before
if (userInput == "admin") { ... }
// After
if ("admin".equals(userInput)) { ... }
```

2. **Unboxing a possibly-null wrapper type.** Why it matters: it throws `NullPointerException` at the unboxing point, often far from where the null originated, making the bug hard to trace.

```
// Before
int total = getCachedCount(id) + 1; // getCachedCount may return null
// After
int total = Optional.ofNullable(getCachedCount(id)).orElse(0) + 1;
```

3. **Building strings with += inside a loop.** Why it matters: each iteration allocates a new `String`, turning an $O(n)$ loop into $O(n^2)$ work.

```
// Before
String csv = "";
for (String s : items) csv += s + ",";
// After
StringBuilder csv = new StringBuilder();
for (String s : items) csv.append(s).append(",");
```

- 4. Forgetting break in a classic switch statement.** Why it matters: execution silently falls through into the next case, running unintended code.

```
// Before
switch (status) {
    case ACTIVE: enable(); // falls through to PENDING's code too!
    case PENDING: notify();
}
// After: add break, or better, use a switch expression with -> arrows
```

- 5. Assuming a method can reassign the caller's object reference.** Why it matters: Java is pass-by-value for references, so reassigning a parameter inside a method never affects the caller's variable — this is a frequent source of “why didn't my fix take effect” bugs.

```
// Before (does nothing useful)
static void reset(List<String> list) { list = new ArrayList<>(); }
// After
static List<String> reset(List<String> list) { return new
↳ ArrayList<>(); }
```

- 6. Relying on floating-point (double/float) for monetary or exact decimal calculations.** Why it matters: binary floating-point can't represent many decimal fractions exactly, causing rounding errors that compound over many operations.

```
// Before
double total = 0.1 + 0.2; // 0.30000000000000004
// After
BigDecimal total = new BigDecimal("0.1").add(new BigDecimal("0.2")); //
↳ 0.3
```

- 7. Ignoring silent integer overflow.** Why it matters: arithmetic that exceeds a type's range wraps around without any warning or exception, producing corrupted values that pass silently into further logic.


```
// Before
int total = bigValue1 * bigValue2; // may silently overflow
// After
long total = Math.multiplyExact((long) bigValue1, bigValue2); // throws
↳ on overflow
```

- 8. Using var where the inferred type isn't obvious to the reader.** Why it matters: it trades a small typing convenience for reduced readability, especially with method calls whose return type isn't well known.

```
// Before
var result = repository.fetch(id); // unclear type at a glance
// After
CustomerRecord result = repository.fetch(id);
```

- 9. Comparing or checking array contents with == or .equals().** Why it matters: arrays don't override equals(), so both operators check reference identity, not element-by-element equality — a classic false-negative bug.

```
// Before
if (expectedBytes.equals(actualBytes)) { ... } // always compares
↳ references
// After
if (Arrays.equals(expectedBytes, actualBytes)) { ... }
```

- 10. Passing a single null to a varargs parameter and expecting one null element.** Why it matters: an untyped null is interpreted as “no array at all,” not “an array containing one null,” which can cause a NullPointerException inside the method instead of the intended behavior.

```
// Before
printAll(null); // passes a null array reference
// After
printAll((String) null); // passes a one-element array: { null }
```

- 11. Not checking for ArrayIndexOutOfBoundsException risk on user-controlled indices.** Why it matters: array bounds are only checked at runtime, and code review should catch any index math derived from external input before it reaches the array access.

```
// Before
process(items[requestedIndex]); // requestedIndex comes straight from a
↪ request param
// After
if (requestedIndex >= 0 && requestedIndex < items.length)
↪ process(items[requestedIndex]);
```

- 12. Using StringBuffer by default instead of StringBuilder.** Why it matters: StringBuffer's synchronization overhead is wasted when the builder never leaves a single thread — which is the vast majority of real-world usage.

```
// Before
StringBuffer sb = new StringBuffer();
// After
StringBuilder sb = new StringBuilder(); // same API, no locking cost
```

- 13. Autoboxing wrapper types inside performance-critical loops.** Why it matters: each boxing operation allocates a short-lived object, adding GC pressure that's easy to miss because the code looks like plain arithmetic.

```
// Before
Long sum = 0L;
for (int i = 0; i < 1_000_000; i++) sum += i; // boxes/unboxes every
↪ iteration
// After
long sum = 0L;
for (int i = 0; i < 1_000_000; i++) sum += i; // stays primitive
```

- 14. Relying on operator precedence in dense expressions without parentheses.** Why it matters: mixing `&&/|`, bitwise operators, or chained `++/--` without parentheses makes intent ambiguous to a reviewer, even when the compiler's answer is well-defined.

```
// Before
if (a || b && c) { ... } // precedence is correct but not obvious to a
↪ reader
// After
if (a || (b && c)) { ... }
```

- 15. Declaring var with the diamond operator and no type witness.** Why it matters: `var list = new ArrayList<>();` infers `ArrayList<Object>`, silently discarding the compile-time type safety generics are supposed to provide.

```
// Before  
var names = new ArrayList<>(); // ArrayList<Object> – accepts anything  
// After  
var names = new ArrayList<String>(); // ArrayList<String> – type-safe
```


Object-Oriented Programming

Object-Oriented Programming (OOP) is a way of organizing code around “objects” instead of just functions and data. Java is built on OOP from the ground up, so almost every code review question about design quality traces back to these basics. This chapter walks through the core OOP building blocks in Java 21+, with small, realistic examples and the “gotchas” that trip people up in interviews and in real pull requests.

Classes and Objects

A **class** is a blueprint. It describes what data (fields) and behavior (methods) something has, but it is not itself a “thing” you can use directly. An **object** is an actual instance created from that blueprint, living in memory, with its own copy of the fields (unless they are `static`, see [Static Members](#)).

Think of `Account` as the blueprint for a bank account, and each customer’s actual account as an object built from that blueprint.

```
public class Account {
    private String ownerName;
    private double balance;

    public Account(String ownerName, double balance) {
        this.ownerName = ownerName;
        this.balance = balance;
    }

    public void deposit(double amount) {
        balance += amount;
    }

    public double getBalance() {
        return balance;
    }
}
```

```

    }

    public class Bank {
        public static void main(String[] args) {
            Account aliceAccount = new Account("Alice", 100.0);
            Account bobAccount = new Account("Bob", 50.0);

            aliceAccount.deposit(25.0);

            System.out.println(aliceAccount.getBalance()); // 125.0
            System.out.println(bobAccount.getBalance());   // 50.0
        }
    }
    ↪ (independent object)

```

Each new Account(...) call creates a separate object with its own balance. Changing aliceAccount never affects bobAccount. That independence is the whole point of objects: state is bundled with the behavior that operates on it.

Gotcha: object identity vs. object equality. Two different objects can have the same data but are still not “the same object” in memory.

```

Account a1 = new Account("Alice", 100.0);
Account a2 = new Account("Alice", 100.0);

System.out.println(a1 == a2);           // false: different objects in memory
System.out.println(a1.equals(a2));      // true only if equals() is
    ↪ overridden;                         // otherwise also false (default
    ↪ Object.equals uses ==)

```

== compares references (memory addresses), not content. Unless a class overrides equals(), .equals() behaves exactly like ==. This is covered more in the “Object Class & Common APIs” chapter, but it starts here with basic object identity.

Constructors

A **constructor** is a special method that runs when an object is created with new. It has the same name as the class, no return type (not even void), and its job is to put the object into a valid starting state.

```

public class Order {
    private final String orderId;
    private final List<String> items;
    private double total;

    // No-args constructor
    public Order() {
        this("ORD-0000"); // calls the other constructor below
    }

    // Constructor with a parameter
    public Order(String orderId) {
        this.orderId = orderId;
        this.items = new ArrayList<>();
        this.total = 0.0;
    }
}

```

If you don't write any constructor, Java gives you a free **default constructor** with no arguments. The moment you write any constructor yourself, that free one disappears.

```

public class Shape {
    private String color;
    // No constructor written here.
}

// Elsewhere:
Shape s = new Shape(); // Works: compiler-generated default constructor

```

```

public class Shape {
    private String color;

    public Shape(String color) {
        this.color = color;
    }
    // No no-args constructor written, and the default one is gone now.
}

// Elsewhere:
Shape s = new Shape(); // Compile error: no matching constructor

```

Constructor chaining means one constructor calls another using `this(...)`, so shared setup logic lives in one place. A constructor can also call a parent class constructor using `super(...)`. If you don't call `super(...)` explicitly, Java inserts an implicit no-argument `super()` call as the very first line.

```
public class Vehicle {
    protected String plateNumber;

    public Vehicle(String plateNumber) {
        this.plateNumber = plateNumber;
        System.out.println("Vehicle created: " + plateNumber);
    }
}

public class Car extends Vehicle {
    private int doors;

    public Car(String plateNumber, int doors) {
        super(plateNumber); // must be the first statement
        this.doors = doors;
        System.out.println("Car created with " + doors + " doors");
    }
}

// new Car("XYZ-123", 4);
// Output:
// Vehicle created: XYZ-123
// Car created with 4 doors
```

Gotcha: calling an overridable method from a constructor. If a constructor calls a method that a subclass overrides, the subclass version can run *before* the subclass has finished initializing its own fields.

```
public class Shape {
    public Shape() {
        draw(); // calling an overridable method during construction
    }

    public void draw() {
        System.out.println("Drawing a generic shape");
    }
}

public class Circle extends Shape {
    private int radius = 5;
```



```

@Override
public void draw() {
    // radius may not be assigned yet when this runs from Shape's
    ⇨ constructor!
    System.out.println("Drawing circle with radius " + radius);
}

// new Circle();
// Output: Drawing circle with radius 0
// (Shape's constructor runs first and calls draw(), but Circle's field
// initializer "radius = 5" hasn't executed yet.)

```

The fix: avoid calling overridable (non-private, non-final, non-static) methods from a constructor, or mark the method `final` if it must be called during construction.

Encapsulation

Encapsulation means hiding the internal state of an object and only exposing it through controlled methods. Fields are usually private, and public **getters/setters** (or no setters at all, for immutable data) control access. This protects invariants — rules that must always be true, like “balance can never go negative.”

```

public class Account {
    private double balance;

    public Account(double initialBalance) {
        if (initialBalance < 0) {
            throw new IllegalArgumentException("Initial balance cannot be
            ⇨ negative");
        }
        this.balance = initialBalance;
    }

    public double getBalance() {
        return balance;
    }

    public void withdraw(double amount) {
        if (amount > balance) {
            throw new IllegalStateException("Insufficient funds");
        }
        balance -= amount;
    }
}

```

```
}
```

Without encapsulation, any code could do `account.balance = -500`; directly and break the invariant. With a `private` field, the only way to change `balance` is through `withdraw()`, which enforces the rule.

Gotcha: a getter that leaks a mutable field breaks encapsulation even though the field is private.

```
public class Order {
    private final List<String> items = new ArrayList<>();

    public List<String> getItems() {
        return items; // returns the real internal list!
    }
}

Order order = new Order();
order.getItems().add("Hacked item"); // mutates internal state from
↳ outside
```

The fix is a **defensive copy**, or returning an unmodifiable view:

```
public List<String> getItems() {
    return List.copyOf(items); // caller gets a snapshot, can't mutate
↳ internals
}
```

Inheritance

Inheritance lets one class (the subclass) reuse and extend the fields and methods of another class (the superclass), using the `extends` keyword. It models an “is-a” relationship: a `Car` **is a** `Vehicle`.

```
public class Vehicle {
    protected String plateNumber;
    protected int speed;

    public Vehicle(String plateNumber) {
        this.plateNumber = plateNumber;
    }
}
```

```

    public void accelerate(int amount) {
        speed += amount;
    }

    public String describe() {
        return "Vehicle " + plateNumber + " at speed " + speed;
    }
}

public class Car extends Vehicle {
    private int doors;

    public Car(String plateNumber, int doors) {
        super(plateNumber);
        this.doors = doors;
    }

    // Inherits accelerate() and describe() for free
}

Car car = new Car("XYZ-123", 4);
car.accelerate(30);
System.out.println(car.describe()); // Vehicle XYZ-123 at speed 30

```

Java only allows **single inheritance** for classes: a class can extend only one other class. This avoids the “diamond problem” (ambiguity when two parent classes define the same method). Java does allow a class to implement multiple interfaces, since interfaces (mostly) don’t carry conflicting state.

```

public class Car extends Vehicle implements Insurable, Sellable {
    // one superclass, multiple interfaces – this is fine
}

```

Gotcha: field hiding. Unlike methods, fields in Java are not polymorphic. If a subclass declares a field with the same name as a superclass field, both fields exist — which one you see depends on the *declared type* of the reference, not the actual object type.

```

public class Vehicle {
    protected int speed = 10;
}

public class Car extends Vehicle {
    protected int speed = 20; // hides Vehicle's speed, does NOT override
    ↪ it
}

```

```

Vehicle v = new Car();
System.out.println(v.speed); // 10 (uses Vehicle's field, based on
↳ reference type!)

Car c = new Car();
System.out.println(c.speed); // 20

```

This is a classic code-review red flag: shadowing fields across a class hierarchy is confusing and error-prone. Prefer accessing state through methods, not fields, across inheritance boundaries.

Polymorphism

Polymorphism (“many forms”) means a single reference type can point to different actual object types, and calling a method on it runs the correct version for the *actual* object — decided at runtime. This is also called **dynamic dispatch** or **runtime polymorphism**.

```

public abstract class Shape {
    public abstract double area();
}

public class Circle extends Shape {
    private final double radius;
    public Circle(double radius) { this.radius = radius; }
    @Override
    public double area() { return Math.PI * radius * radius; }
}

public class Rectangle extends Shape {
    private final double width, height;
    public Rectangle(double width, double height) {
        this.width = width;
        this.height = height;
    }
    @Override
    public double area() { return width * height; }
}

public class AreaCalculator {
    public static void main(String[] args) {
        List<Shape> shapes = List.of(new Circle(2), new Rectangle(3, 4));
        for (Shape shape : shapes) {

```

```

        System.out.println(shape.area()); // correct area() runs for
↪ each real type
    }
    // Output:
    // 12.566370614359172
    // 12.0
}

```

The loop variable is declared as `Shape`, but each call to `area()` runs the actual subclass's implementation. The calling code doesn't need `if (shape instanceof Circle) ...` checks — that's the benefit of polymorphism.

There are two flavors people mention in interviews:

Type	Also called	Resolved when	Example
Compile-time polymorphism	Static binding	At compile time, by looking at parameter types	Method overloading
Runtime polymorphism	Dynamic binding	At runtime, by looking at the actual object	Method overriding

Gotcha: static methods are not polymorphic. They are resolved by the *reference type*, not the actual object — this is called **method hiding**, not overriding.

```

public class Vehicle {
    public static String category() { return "Generic Vehicle"; }
}

public class Car extends Vehicle {
    public static String category() { return "Car"; } // hides, not
↪ overrides
}

Vehicle v = new Car();
System.out.println(v.category()); // "Generic Vehicle" – decided by
↪ reference type!

```

Abstraction

Abstraction means exposing *what* something does while hiding *how* it does it. In Java this is done with abstract classes and interfaces. An abstract class can have some implemented methods and some unimplemented (abstract) ones; it cannot be instantiated directly.

```
public abstract class PaymentMethod {
    // Abstract method: no body, subclasses must implement it
    public abstract boolean authorize(double amount);

    // Concrete method: shared logic for all payment methods
    public final void pay(double amount) {
        if (authorize(amount)) {
            System.out.println("Paid " + amount);
        } else {
            System.out.println("Payment declined");
        }
    }
}

public class CreditCardPayment extends PaymentMethod {
    @Override
    public boolean authorize(double amount) {
        return amount <= 5000; // simplified rule
    }
}

PaymentMethod payment = new CreditCardPayment();
payment.pay(1200); // Paid 1200.0
```

```
PaymentMethod payment = new PaymentMethod(); // Compile error:
↳ PaymentMethod is abstract
```

Interfaces (covered fully in the “Modern Java Language Features” chapter) are a purer form of abstraction: historically 100% abstract, though modern Java interfaces can also have default, static, and private methods.

```
public interface Discountable {
    double applyDiscount(double price);
}

public class Order implements Discountable {
    @Override
```

```

    public double applyDiscount(double price) {
        return price * 0.9; // 10% off
    }
}

```

Rule of thumb for code review: use an abstract class when subclasses share state or common implementation; use an interface when you're describing a capability that unrelated classes might implement (Comparable, Discountable, Serializable).

Method Overloading

Method overloading means defining multiple methods with the *same name* but *different parameter lists* (different type, number, or order of parameters) in the same class. It is resolved at **compile time** based on the arguments' declared types — this is why it's called compile-time (static) polymorphism.

```

public class Invoice {
    public double calculateTotal(double price, int quantity) {
        return price * quantity;
    }

    public double calculateTotal(double price, int quantity, double
↪ discount) {
        return price * quantity * (1 - discount);
    }

    public double calculateTotal(double price, int quantity, double
↪ discount, double taxRate) {
        return price * quantity * (1 - discount) * (1 + taxRate);
    }
}

Invoice invoice = new Invoice();
System.out.println(invoice.calculateTotal(10.0, 3));           // 30.0
System.out.println(invoice.calculateTotal(10.0, 3, 0.1));      // 27.0

```

You cannot overload by return type alone — the parameter list must differ.

```

public int process(String data) { return 1; }
public double process(String data) { return 1.0; } // Compile error: same
↪ signature

```

Gotcha: overload resolution with autoboxing and varargs. Java picks the *most specific* match, and it prefers, in order: (1) exact match with widening of primitives, (2) autoboxing/unboxing, (3) varargs. This can surprise people.

```
public class Logger {
    public void log(int code) {
        System.out.println("int version: " + code);
    }

    public void log(Integer code) {
        System.out.println("Integer version: " + code);
    }

    public void log(Object... codes) {
        System.out.println("varargs version");
    }
}

Logger logger = new Logger();
logger.log(5); // "int version: 5"
// Java prefers the exact primitive match over autoboxing to Integer,
// and prefers both of those over varargs.
```

```
public class Calculator {
    public void print(long value) { System.out.println("long: " +
↪ value); }
    public void print(Integer value) { System.out.println("Integer: " +
↪ value); }

    public static void main(String[] args) {
        int x = 5;
        new Calculator().print(x);
        // Output: "long: 5"
        // Widening (int -> long) is preferred over autoboxing (int ->
↪ Integer)!
    }
}
```

This is a frequent code-review discussion point: overloads that mix primitives, wrapper types, and varargs are a maintenance hazard because the “obviously correct” overload isn’t always the one that’s chosen.

Method Overriding

Method overriding means a subclass provides its own implementation of a method already defined in its superclass, with the *same signature*. It is resolved at **runtime** based on the actual object type — runtime (dynamic) polymorphism. Always use the `@Override` annotation; it makes the compiler check that you're actually overriding something.

```
public class Shape {
    public double area() {
        return 0.0;
    }

    @Override
    public String toString() {
        return "Shape with area " + area();
    }
}

public class Square extends Shape {
    private final double side;
    public Square(double side) { this.side = side; }

    @Override
    public double area() {
        return side * side;
    }
}

Shape shape = new Square(4);
System.out.println(shape.area());           // 16.0
System.out.println(shape);                  // Shape with area 16.0 (toString
↳ overridden indirectly via area())
```

Overriding rules the compiler enforces:

- Same method name and parameter list.
- Return type must be the same or a **covariant** (narrower) subtype.
- Access modifier can stay the same or become *more* visible, never less visible.
- Cannot override a method marked `final` or `static` (that becomes hiding, not overriding — see [Static Members](#)).

	Method Overloading	Method Overriding
Same method name?	Yes	Yes
Parameter list	Must differ	Must be identical
Return type	Can differ freely	Must be same or covariant
Resolved	Compile time (static binding)	Runtime (dynamic binding)
Relationship	Same class (or same class hierarchy)	Superclass / subclass
Access modifier	Can differ freely	Cannot be more restrictive

Gotcha: narrowing the access modifier when overriding is a compile error.

```
public class Vehicle {
    public void start() {
        System.out.println("Vehicle starting");
    }
}

public class Car extends Vehicle {
    @Override
    protected void start() { // Compile error: cannot reduce visibility
        ↪ from public to protected
        System.out.println("Car starting");
    }
}
```

Gotcha: @Override catches signature mistakes early.

```
public class Shape {
    public double area() { return 0.0; }
}

public class Circle extends Shape {
    // Typo: overload instead of override, silently creates a NEW method
    public double area(double scale) {
        return 3.14 * scale;
    }
}

// Without @Override, this compiles but never overrides Shape.area().
// Adding @Override to area(double scale) would immediately fail to
↪ compile,
// revealing the mistake.
```

Access Modifiers

Access modifiers control which code can see a class, field, method, or constructor. Java has four levels:

Modifier	Same class	Same package	Subclass (different package)	Everywhere
private	Yes	No	No	No
(no modifier / package-private)	Yes	Yes	No	No
protected	Yes	Yes	Yes	No
public	Yes	Yes	Yes	Yes

```
package com.bank.account;

public class Account {
    private double balance;           // only inside Account itself
    String accountType;               // package-private: visible to classes
    ↪ in com.bank.account
    protected String branchCode;      // visible in package + subclasses
    ↪ anywhere
    public String ownerName;          // visible everywhere
}
```

```
package com.bank.account;

public class SavingsAccount extends Account {
    public void printBranch() {
        System.out.println(branchCode); // OK: protected, and this is a
    ↪ subclass
        // System.out.println(balance); // Compile error: balance is
    ↪ private to Account
    }
}
```

```
package com.bank.reporting;

public class ReportGenerator {
    public void printOwner(Account account) {
        System.out.println(account.ownerName); // OK: public
    }
}
```

```

        // System.out.println(account.branchCode); // Compile error:
        ↪ different package, not a subclass
    }
}

```

Gotcha: protected in a different package is only accessible through inheritance, not through an arbitrary reference.

```

package com.bank.reporting;

import com.bank.account.Account;

public class AuditTool extends Account {
    public void check(Account other) {
        this.branchCode = "B01";           // OK: accessing through "this"
        ↪ (the subclass instance)
        // other.branchCode = "B02";       // Compile error: accessing
        ↪ through another Account reference
    }
}

```

The rule of thumb code reviewers apply: **use the most restrictive modifier that still works**. Start with private, widen only when there's a real need. Overly public fields and methods make future refactoring risky because you don't know who depends on them.

Static Members

static members belong to the **class itself**, not to any particular object. There is exactly one copy, shared by all instances.

```

public class Account {
    private static int totalAccountsCreated = 0; // shared across all
    ↪ Account objects
    private final String ownerName;

    public Account(String ownerName) {
        this.ownerName = ownerName;
        totalAccountsCreated++;
    }

    public static int getTotalAccountsCreated() {
        return totalAccountsCreated;
    }
}

```

```

    }
}

new Account("Alice");
new Account("Bob");
new Account("Carol");

System.out.println(Account.getTotalAccountsCreated()); // 3

```

Common uses of static:

- **Static fields:** shared/global state per class, like counters or constants.
- **Static methods:** utility logic that doesn't need object state, e.g. `Math.sqrt(x)`.
- **Static nested classes:** see [Nested, Inner, Local, and Anonymous Classes](#).
- **Static initializer blocks:** run once when the class is first loaded.

```

public class Configuration {
    public static final String ENVIRONMENT;

    static {
        // Runs once, when the class is loaded – good for expensive
        ↪ one-time setup
        ENVIRONMENT = System.getenv().getOrDefault("APP_ENV",
        ↪ "development");
        System.out.println("Configuration loaded for: " + ENVIRONMENT);
    }
}

```

A static method cannot directly access instance (non-static) fields or methods, because there's no specific object to work on.

```

public class Account {
    private double balance = 100;

    public static void printBalance() {
        // System.out.println(balance); // Compile error: no instance to
        ↪ read balance from
    }
}

```

Gotcha: static methods are inherited but hidden, not overridden (also shown earlier in [Polymorphism](#)). Calling a static method through an instance reference is legal but misleading — code reviewers should flag it.

```

public class ReportUtil {
    public static void printHeader() {
        System.out.println("=== Report ===");
    }
}

ReportUtil util = new ReportUtil();
util.printHeader(); // Works, but should be called as
↳ ReportUtil.printHeader()

```

Gotcha: mutable static state is a shared, hidden dependency — dangerous in multi-threaded code and hard to test because it persists between tests.

```

public class IdGenerator {
    private static int nextId = 1; // shared mutable state!

    public static int next() {
        return nextId++; // not thread-safe: race condition under
↳ concurrent access
    }
}

```

Final Keyword

`final` means “cannot be changed again,” but what exactly it locks down depends on where it’s applied:

Applied to	Meaning
<code>final</code> variable/field	Value (or object reference) can only be assigned once
<code>final</code> method	Cannot be overridden by subclasses
<code>final</code> class	Cannot be extended (no subclasses at all)

```

public final class ImmutablePoint { // cannot be subclassed
    private final int x; // must be assigned exactly once (constructor or
↳ declaration)
    private final int y;

    public ImmutablePoint(int x, int y) {
        this.x = x;
    }
}

```

```

        this.y = y;
    }

    public final int getX() { // cannot be overridden, even though the
        ↪ class already forbids subclassing
        return x;
    }
}

```

```

public class Vehicle {
    protected final int maxSpeed;

    public Vehicle(int maxSpeed) {
        this.maxSpeed = maxSpeed; // OK: first (and only) assignment
    }

    public void boost() {
        // this.maxSpeed = maxSpeed + 10; // Compile error: final field
        ↪ can't be reassigned
    }
}

```

```

public class Vehicle {
    public final void identify() {
        System.out.println("I am a vehicle");
    }
}

public class Car extends Vehicle {
    @Override
    public void identify() { // Compile error: cannot override a final
        ↪ method
        System.out.println("I am a car");
    }
}

```

Gotcha: final on a reference only locks the reference, not the object's contents. A final field can still point to a mutable object whose internal state changes.

```

public class Order {
    private final List<String> items = new ArrayList<>();

    public void addItem(String item) {

```

```

        items.add(item); // legal! We're mutating the List, not
        ↪ reassigning "items"
    }

    public void replaceList() {
        // items = new ArrayList<>(); // Compile error: can't reassign a
        ↪ final field
    }
}

```

This is a common interview trap: `final` does **not** mean immutable. True immutability requires the class itself to prevent internal mutation too (see the “Immutability” topic in the “Object Class & Common APIs” chapter).

This and Super

`this` refers to the **current object** — the one whose method or constructor is currently executing. `super` refers to the **immediate superclass**, used to call its constructor or access members it defines.

```

public class Vehicle {
    protected String plateNumber;

    public Vehicle(String plateNumber) {
        this.plateNumber = plateNumber;
    }

    public String describe() {
        return "Vehicle " + plateNumber;
    }
}

public class Car extends Vehicle {
    private String plateNumber; // shadows Vehicle's field – usually a bad
    ↪ idea, shown here for illustration

    public Car(String plateNumber) {
        super(plateNumber); // calls Vehicle's constructor
        this.plateNumber = plateNumber; // "this" disambiguates the field
        ↪ from the parameter
    }

    @Override
    public String describe() {

```



```

        return super.describe() + " (Car version, plate=" +
↪ this.plateNumber + ")";
    }
}

Car car = new Car("ABC-999");
System.out.println(car.describe());
// Output: Vehicle ABC-999 (Car version, plate=ABC-999)

```

Common uses of this:

- Disambiguate a field from a constructor/method parameter with the same name (`this.balance = balance;`).
- Call another constructor in the same class (`this(...)` — constructor chaining, see [Constructors](#)).
- Pass the current object as an argument (`someMethod(this)`).
- Return the current object for method chaining (`return this;`).

```

public class InvoiceBuilder {
    private double amount;
    private String currency;

    public InvoiceBuilder withAmount(double amount) {
        this.amount = amount;
        return this; // enables chaining
    }

    public InvoiceBuilder withCurrency(String currency) {
        this.currency = currency;
        return this;
    }
}

InvoiceBuilder builder = new InvoiceBuilder()
    .withAmount(99.99)
    .withCurrency("EUR"); // fluent chaining thanks to "return this;"

```

Common uses of super:

- Call the parent constructor (`super(...)` — must be the first statement in a constructor).
- Call the parent's version of an overridden method (`super.describe()`).

- Access a parent field that's shadowed by a subclass field.

Gotcha: super only reaches one level up. There's no `super.super.method()` in Java — if `Car` extends `Vehicle` extends `Machine`, `Car` can call `super.describe()` to reach `Vehicle`'s version, but not `Machine`'s directly (unless `Vehicle` doesn't override it, in which case `super.describe()` naturally falls through to `Machine`'s implementation).

Nested, Inner, Local, and Anonymous Classes

Java allows classes defined inside other classes or even inside methods. They're grouped into four kinds:

Kind	Defined	Needs outer instance?	Typical use
Static nested class	Inside a class, marked <code>static</code>	No	Grouping a helper class that doesn't need outer state
Inner class	Inside a class, not <code>static</code>	Yes (holds implicit reference to outer object)	Tight coupling with an outer object's state
Local class	Inside a method body	Yes, if the method is an instance method	One-off helper used only within a method
Anonymous class	Inline, no name, usually implementing an interface or extending a class	Depends on context	Quick one-time implementation, e.g. a callback

Static nested class — behaves like a top-level class, just namespaced inside another for organization. Does not hold a reference to an outer instance.

```
public class Order {
    private final List<LineItem> lineItems = new ArrayList<>();

    public static class LineItem { // static nested class
        private final String productName;
        private final int quantity;

        public LineItem(String productName, int quantity) {
            this.productName = productName;
            this.quantity = quantity;
        }
    }
}
```

```

    }
}

public void addLineItem(String productName, int quantity) {
    lineItems.add(new LineItem(productName, quantity));
}
}

// Created without needing an Order instance:
Order.LineItem item = new Order.LineItem("Widget", 3);

```

Inner class (non-static)—tied to a specific outer object; it can access the outer object's fields directly, even private ones.

```

public class Order {
    private double total;
    private final List<String> discountLog = new ArrayList<>();

    public class DiscountApplier { // inner (non-static) class
        public void apply(double percentage) {
            total = total * (1 - percentage); // accesses outer field
            discountLog.add("Applied " + (percentage * 100) + "%
        }
    }
}

Order order = new Order();
Order.DiscountApplier applier = order.new DiscountApplier(); // needs an
applier.apply(0.1);

```

Local class—declared inside a method, visible only within that method. Useful when a helper type is needed just for one algorithm and shouldn't leak outside it.

```

public class ReportGenerator {
    public List<String> generateSummaries(List<Double> amounts) {
        class Formatter { // local class, scoped to this method
            String format(double amount) {
                return String.format("%.2f", amount);
            }
        }

        Formatter formatter = new Formatter();
        List<String> summaries = new ArrayList<>();
    }
}

```

```

        for (double amount : amounts) {
            summaries.add(formatter.format(amount));
        }
        return summaries;
    }
}

```

Anonymous class — a class with no name, defined and instantiated in a single expression. Common before lambdas existed, and still used for interfaces with more than one method, or when you need to keep extra state.

```

public interface DiscountRule {
    double apply(double price);
}

public class CheckoutService {
    public double checkout(double price, DiscountRule rule) {
        return rule.apply(price);
    }

    public void run() {
        double finalPrice = checkout(100.0, new DiscountRule() { //
↪ anonymous class
            @Override
            public double apply(double price) {
                return price * 0.85; // 15% off
            }
        });
        System.out.println(finalPrice); // 85.0
    }
}

```

For a single-method (functional) interface like `DiscountRule`, a **lambda expression** is usually preferred over an anonymous class in modern Java — it's shorter and equally clear:

```

double finalPrice = checkout(100.0, price -> price * 0.85);

```

Gotcha: a non-static inner class secretly holds a reference to its outer instance, which can cause **memory leaks** — the outer object can't be garbage collected while the inner object is still reachable (e.g., stored in a long-lived collection or passed to another thread).

```

public class Order {
    private final byte[] largePayload = new byte[10_000_000]; // 10 MB

    public class Listener { // inner class – implicitly holds a reference
        ↪ to the enclosing Order
        void onEvent() {
            System.out.println("Event for order with payload size " +
        ↪ largePayload.length);
        }
    }
}

// If a Listener instance is stored somewhere long-lived (e.g., a static
↪ list
// or an event bus), the entire enclosing Order – including its 10 MB
↪ payload –
// stays reachable and cannot be garbage collected, even if nobody needs
↪ the Order anymore.

```

The fix: prefer a static nested class (optionally taking any needed outer data explicitly as a constructor parameter) when you don't truly need access to the outer instance's state.

```

public class Order {
    private final byte[] largePayload = new byte[10_000_000];

    public static class Listener { // static: no hidden reference to Order
        void onEvent() {
            System.out.println("Event received");
        }
    }
}

```

Gotcha: local and anonymous classes can only capture effectively final local variables — variables that are never reassigned after initialization.

```

public List<Runnable> createTasks() {
    List<Runnable> tasks = new ArrayList<>();
    for (int i = 0; i < 3; i++) {
        ↪ iteration
        int taskNumber = i; // effectively final: assigned once per loop
        tasks.add(new Runnable() {
            @Override
            public void run() {

```

```

        System.out.println("Task " + taskNumber); // OK: captures
        ↪ effectively final variable
    }
    });
}
return tasks;
}

```

```

public Runnable createTask() {
    int counter = 0;
    Runnable task = () -> System.out.println(counter);
    counter = 5; // Compile error: "counter" is captured by the lambda, so
    ↪ it must stay effectively final
    return task;
}

```

Common Code-Review Interview Pitfalls

- 1. Public mutable fields instead of encapsulated access.** Why it matters: callers can corrupt an object's invariants directly, bypassing any validation.

```

// Before
public class Account { public double balance; }
// After
public class Account {
    private double balance;
    public void withdraw(double amount) { /* validate, then mutate */ }
}

```

- 2. Getters that return direct references to mutable internal collections/objects.** Why it matters: callers can mutate "private" state from outside, silently breaking encapsulation.

```

// Before
public List<String> getItems() { return items; }
// After
public List<String> getItems() { return List.copyOf(items); }

```

- 3. Calling an overridable instance method from a constructor.** Why it matters: the subclass's fields may not be initialized yet when the overridden method runs, producing wrong values (e.g., 0 or null).

```
// Before: Shape() calls draw(), Circle overrides draw() using a field
↳ set after super() runs
// After: mark draw() final, or move the call out of the constructor
↳ into an init method
```

4. **Confusing method overloading with overriding, especially with a missing `@Override`.** Why it matters: a typo in parameters silently creates a new overload instead of overriding, and the bug hides until runtime.

```
// Before (no annotation, silent overload)
public double area(double scale) { ... }
// After (compiler enforces correctness)
@Override
public double area() { ... }
```

5. **Field hiding across a class hierarchy.** Why it matters: which field you read depends on the *reference type*, not the actual object — a frequent source of confusing bugs.

```
// Before: Car declares its own "speed" field that shadows
↳ Vehicle.speed
// After: don't redeclare inherited fields; add new fields with
↳ distinct names, or use methods
```

6. **Treating static methods as polymorphic.** Why it matters: static methods are hidden, not overridden — calling them through a superclass-typed reference always uses the reference's declared type, not the runtime type.

```
Vehicle v = new Car();
v.category(); // resolves using Vehicle's static method, surprising
↳ reviewers expecting Car's
```

7. **Assuming final means immutable.** Why it matters: `final` only prevents reassigning the reference; the referenced object (e.g., a `List`) can still be mutated internally.

```
private final List<String> items = new ArrayList<>();
items.add("x"); // still allowed, even though "items" is final
```

8. **Overly broad access modifiers (public by default).** Why it matters: wide visibility becomes a long-term maintenance liability — every public member is part of your API contract and hard to change later.

```
// Before
public double balance;
// After
private double balance;
public double getBalance() { return balance; }
```

- 9. Non-static inner classes causing memory leaks.** Why it matters: an inner class instance implicitly holds a reference to its enclosing object, keeping it alive longer than expected if stored somewhere long-lived.

```
// Before
public class Listener { /* implicit outer reference */ }
// After
public static class Listener { /* no implicit outer reference */ }
```

- 10. Overload resolution ambiguity with autoboxing and varargs.** Why it matters: adding a new overload can silently change which overload existing calls resolve to, without any compile error.

```
void log(int code) { ... }
void log(Integer code) { ... }
void log(Object... codes) { ... }
// log(5) always prefers int over Integer over varargs – surprising to
→ reviewers unfamiliar with the resolution order
```

- 11. Narrowing visibility when overriding (or trying to).** Why it matters: it's a compile error, but the intent (restricting subclass behavior) usually signals a deeper design problem — maybe the method shouldn't be overridable at all.

```
// Before: attempting to override public start() as protected – fails
→ to compile
// After: keep the same or wider access, or reconsider whether the base
→ method should be public
```

- 12. Mutable static fields used as shared/global state.** Why it matters: they're a hidden dependency between unrelated pieces of code, cause race conditions in multi-threaded contexts, and leak state between unit tests.

```
// Before
private static int nextId = 1; // shared, unsynchronized
// After
```



```
private static final AtomicInteger nextId = new AtomicInteger(1);
```

13. No-args constructor silently disappearing once another constructor is added.

Why it matters: code (or a framework needing a default constructor, e.g. some serializers) that relied on `new Foo()` breaks at compile or runtime.

```
// Before: only Shape(String color) exists
// After: explicitly add Shape() if a no-args constructor is still
↳ needed
```

14. Using an abstract class where a simple interface would do (or vice versa). Why it matters: abstract classes force single inheritance and couple unrelated types; interfaces should be preferred for pure capability contracts, abstract classes for shared implementation/state.

```
// Before: abstract class Discountable with no shared state or
↳ implementation
// After: interface Discountable { double applyDiscount(double price);
↳ }
```

15. Anonymous classes used where a lambda would be clearer. Why it matters: for single-method functional interfaces, an anonymous class is verbose boilerplate compared to a lambda, and reviewers should flag it as an easy simplification.

```
// Before
new DiscountRule() {
    @Override
    public double apply(double price) { return price * 0.85; }
};
// After
(double price) -> price * 0.85;
```


Modern Java Language Features

Java has changed a lot since Java 8. Newer versions added features that make code shorter, safer, and easier to read. This chapter covers the features you are most likely to see in a modern codebase or be asked about in a code-review interview. Each section explains what the feature is, which JDK version it shipped in, and shows a small, realistic example.

- [Enums](#)
- [Records](#)
- [Sealed Classes and Interfaces](#)
- [Interfaces \(default, static, private methods\)](#)
- [Abstract Classes](#)
- [Pattern Matching \(instanceof, switch\)](#)
- [Primitive Types in Patterns](#)
- [Switch Expressions](#)
- [Text Blocks](#)
- [String Templates \(Preview\)](#)
- [Unnamed Variables and Patterns](#)
- [Value-Based Classes](#)
- [Preview and Incubator Features](#)
- [Deprecated and Removed Features Across Java Versions](#)
- [Common Code-Review Interview Pitfalls](#)

Enums

An **enum** (short for “enumeration”) is a type with a fixed set of constant values. Use an enum when a variable can only be one of a small, known list of options, like days of the week or order statuses.

Enums in Java are full classes. They can have fields, constructors, methods, and they can even implement interfaces. This is more powerful than enums in many other languages.

```
public enum OrderStatus {
    NEW,
    PAID,
    SHIPPED,
    DELIVERED,
    CANCELLED
}
```

Enums can carry data and behavior:

```
public enum Planet {
    MERCURY(3.303e+23, 2.4397e6),
    VENUS(4.869e+24, 6.0518e6),
    EARTH(5.976e+24, 6.37814e6);

    private final double mass; // in kilograms
    private final double radius; // in meters

    Planet(double mass, double radius) {
        this.mass = mass;
        this.radius = radius;
    }

    double surfaceGravity() {
        final double G = 6.67300E-11;
        return G * mass / (radius * radius);
    }
}

// Usage:
double g = Planet.EARTH.surfaceGravity(); // ~9.8
```

Enums can also have abstract methods, where each constant provides its own implementation:

```

public enum Operation {
    PLUS {
        public int apply(int a, int b) { return a + b; }
    },
    MINUS {
        public int apply(int a, int b) { return a - b; }
    };

    public abstract int apply(int a, int b);
}

// Usage:
int result = Operation.PLUS.apply(2, 3); // 5

```

Key facts every reviewer should know:

Fact	Detail
Since	Java 5
Base class	Implicitly extends <code>java.lang.Enum</code> (cannot extend anything else)
Interfaces	Can implement one or more interfaces
Comparison	Always safe to compare with <code>==</code> (constants are singletons)
Switch	Enums work great in <code>switch</code> statements and expressions
Serialization	Enum serialization is handled specially and is safe by default
<code>values()</code> / <code>valueOf()</code>	Auto-generated static methods to list constants or parse from a string

```

for (OrderStatus s : OrderStatus.values()) {
    System.out.println(s); // NEW, PAID, SHIPPED, DELIVERED, CANCELLED
}

OrderStatus s = OrderStatus.valueOf("PAID"); // throws
→ IllegalArgumentException if not found

```

Records

A **record** is a compact way to declare an immutable data class. Before records, developers had to hand-write (or generate) a constructor, getters, `equals()`, `hashCode()`,

and `toString()` for simple “data holder” classes. Records generate all of that automatically.

Records were previewed in Java 14/15 and finalized as a standard feature in **Java 16**.

```
public record Point(int x, int y) { }

// The compiler generates:
// - a constructor Point(int x, int y)
// - accessors x() and y() (note: no "get" prefix)
// - equals(), hashCode(), toString()

Point p1 = new Point(1, 2);
Point p2 = new Point(1, 2);

System.out.println(p1);           // Point[x=1, y=2]
System.out.println(p1.x());       // 1
System.out.println(p1.equals(p2)); // true
```

You can add validation with a **compact constructor**, plus extra methods and static factories:

```
public record Range(int min, int max) {

    // Compact constructor: runs before fields are assigned.
    public Range {
        if (min > max) {
            throw new IllegalArgumentException("min must be <= max");
        }
    }

    public int length() {
        return max - min;
    }

    public static Range of(int min, int max) {
        return new Range(min, max);
    }
}

Range r = Range.of(1, 10);
System.out.println(r.length()); // 9
```

Records vs. regular classes:

Aspect	Record	Regular class
Fields	All fields are <code>private final</code> automatically	You choose mutability
Accessors	Auto-generated (<code>x()</code> , not <code>getX()</code>)	You write them, often <code>getX()</code>
<code>equals/hashCode/toString</code>	Auto-generated from all fields	You write them (or use IDE/Lombok)
Inheritance	Cannot extend another class (implicitly extends <code>Record</code>)	Can extend any class
Can implement interfaces	Yes	Yes
Mutability	Always immutable (fields cannot change)	Can be mutable or immutable
Best use case	Simple immutable data carriers (DTOs, value objects)	Entities with behavior, mutable state, or inheritance

Records can implement interfaces, which is useful with sealed types (see next section):

```
public interface Shape {
    double area();
}

public record Circle(double radius) implements Shape {
    public double area() {
        return Math.PI * radius * radius;
    }
}
```

Records can also be **generic**:

```
public record Pair<A, B>(A first, B second) { }

Pair<String, Integer> pair = new Pair<>("age", 30);
```

Sealed Classes and Interfaces

A **sealed** class or interface restricts which other classes or interfaces may extend or implement it. This gives you controlled inheritance: you (the author) decide the exact, closed list of subtypes. Sealed types were finalized in **Java 17**.

Why this matters: with a sealed hierarchy, the compiler (and tools like `switch`) knows every possible subtype. This enables **exhaustiveness checking** — the compiler can tell you if you forgot to handle a case.

```
public sealed interface Shape permits Circle, Square, Rectangle { }

public record Circle(double radius) implements Shape { }
public record Square(double side) implements Shape { }
public record Rectangle(double width, double height) implements Shape { }
```

If the permitted subtypes are in the same file, you can drop the `permits` clause — the compiler figures it out:

```
public sealed interface Payment {
    record CreditCard(String number) implements Payment { }
    record BankTransfer(String iban) implements Payment { }
    record Cash() implements Payment { }
}
```

A permitted subclass must be declared as one of: `final`, `sealed` (with its own permitted subtypes), or `non-sealed` (reopens the hierarchy for further extension).

```
public sealed class Vehicle permits Car, Truck { }

public final class Car extends Vehicle { }           // no further
↳ subclassing
public non-sealed class Truck extends Vehicle { }    // anyone can extend
↳ Truck now
```

Sealed vs `final` vs open (normal) inheritance:

Modifier	Who can extend it	Typical use
(none, open)	Anyone	Flexible libraries, frameworks
<code>final</code>	No one	Utility classes, immutable value types
<code>sealed</code>	Only the classes listed in <code>permits</code> (or same file/package)	Closed hierarchies you want exhaustive <code>switch</code> over
<code>non-sealed</code>	Anyone (used to reopen a branch of a sealed hierarchy)	Escape hatch inside a sealed hierarchy

Sealed types shine with pattern matching in `switch` (covered below):

```
static double area(Shape shape) {
    return switch (shape) {
        case Circle c -> Math.PI * c.radius() * c.radius();
        case Square s -> s.side() * s.side();
        case Rectangle r -> r.width() * r.height();
        // no "default" needed: compiler knows these are ALL the cases
    };
}
```

Interfaces (default, static, private methods)

Classic interfaces (before Java 8) could only declare abstract methods — no bodies allowed. Modern interfaces can now include real code.

Method type	Since	Has a body?	Can be overridden?	Purpose
abstract (plain)	Java 1	No	Yes (must be implemented)	The contract
default	Java 8	Yes	Yes	Provide a default implementation so existing implementers don't break
static	Java 8	Yes	No (belongs to the interface itself)	Utility/helper methods related to the interface
private	Java 9	Yes	No	Share code between default/static methods without exposing it

```
public interface Vehicle {

    // Abstract method: every implementer must define this.
    void drive();

    // Default method: implementers get this for free, but may override
    ↪ it.
    default void honk() {
        System.out.println("Beep beep!");
    }

    // Static method: called on the interface itself, e.g.
    ↪ Vehicle.create(...)
```

```

    static Vehicle create(String type) {
        return switch (type) {
            case "car" -> new Car();
            default -> throw new IllegalArgumentException("Unknown type:
↪ " + type);
        };
    }

    // Private method (Java 9+): helper reused by default methods below.
    private void log(String action) {
        System.out.println("[Vehicle] " + action);
    }

    default void startTrip() {
        log("starting trip");
        drive();
    }
}

class Car implements Vehicle {
    public void drive() {
        System.out.println("Car is driving");
    }
}

```

Why this matters for code review: default methods let library authors add new methods to an interface **without breaking existing implementations** (this is exactly why `Collection` and `Map` gained many default methods over the years, like `forEach` and `getOrDefault`). Watch out, though — if a class implements two interfaces with the same default method signature, the class must override it to resolve the conflict, or the code will not compile.

```

interface A { default String name() { return "A"; } }
interface B { default String name() { return "B"; } }

class C implements A, B {
    // Must override: ambiguous otherwise.
    public String name() {
        return A.super.name() + B.super.name(); // "AB"
    }
}

```

Abstract Classes

An **abstract class** cannot be instantiated directly. It exists to be extended. It can mix fully implemented methods with abstract ones that subclasses must fill in. Abstract classes have existed since Java 1, but it's worth revisiting how they compare to modern interfaces since both can now hold real code.

```
public abstract class Employee {  
  
    protected final String name;  
  
    protected Employee(String name) {  
        this.name = name;  
    }  
  
    // Concrete method: shared by all subclasses.  
    public String greet() {  
        return "Hello, my name is " + name;  
    }  
  
    // Abstract method: each subclass must define its own pay calculation.  
    public abstract double calculatePay();  
}  
  
public class Manager extends Employee {  
    private final double baseSalary;  
    private final double bonus;  
  
    public Manager(String name, double baseSalary, double bonus) {  
        super(name);  
        this.baseSalary = baseSalary;  
        this.bonus = bonus;  
    }  
  
    @Override  
    public double calculatePay() {  
        return baseSalary + bonus;  
    }  
}
```

Abstract class vs. interface — the modern comparison:

Aspect	Abstract class	Interface
Instance fields	Yes, any visibility	No instance fields (only <code>static final</code> constants)
Constructors	Yes	No
Multiple inheritance	A class can extend only one abstract class	A class can implement many interfaces
Method bodies	Any method can have a body	default, static, private can have bodies; plain methods cannot
Access modifiers on methods	public, protected, private all allowed	Methods are implicitly public (except private helper methods)
State/encapsulation	Good fit — designed to hold and manage state	Poor fit — designed to describe capability/contract
When to use	“Is-a” relationship with shared state and shared code	“Can-do” capability, especially across unrelated classes

Rule of thumb for reviews: if two classes need to **share fields or a constructor**, lean toward an abstract class. If you’re just describing a **capability** (e.g. `Comparable`, `Runnable`, `Serializable`), use an interface.

Pattern Matching (`instanceof`, `switch`)

Pattern matching lets you check the shape or type of a value and extract its parts in one step, instead of checking the type and then manually casting.

`instanceof` pattern matching (Java 16, finalized)

Before:

```
if (obj instanceof String) {
    String s = (String) obj; // manual, error-prone cast
    System.out.println(s.length());
}
```

After — the cast is automatic, and the variable `s` is only in scope where it makes sense:

```
if (obj instanceof String s) {
    System.out.println(s.length()); // "s" is already a String here
}
```

You can combine it with extra conditions:

```
if (obj instanceof String s && !s.isBlank()) {
    System.out.println("Non-blank string: " + s);
}
```

switch pattern matching (Java 21, finalized)

Pattern matching for `switch` lets you match on type (and, with records, on structure) directly in a `switch`.

```
sealed interface Shape permits Circle, Square { }
record Circle(double radius) implements Shape { }
record Square(double side) implements Shape { }

static String describe(Shape shape) {
    return switch (shape) {
        case Circle c when c.radius() > 100 -> "huge circle"; // guarded
        case Circle c -> "circle r=" + c.radius();
        case Square s -> "square side=" + s.side();
    };
}
```

→ pattern

Record patterns (Java 21) let you destructure a record right in the pattern, pulling out its components:

```
record Point(int x, int y) { }
record Line(Point start, Point end) { }

static String describe(Object obj) {
    return switch (obj) {
        // Nested destructuring: pull x1,y1,x2,y2 straight out of the
        case Line(Point(var x1, var y1), Point(var x2, var y2)) ->
            "Line from (%d,%d) to (%d,%d)".formatted(x1, y1, x2, y2);
        case Point(var x, var y) -> "Point(%d,%d)".formatted(x, y);
        default -> "unknown";
    };
}
```

→ Line.

```
}
```

Timeline summary:

Feature	Preview since	Finalized in
<code>instanceof</code> pattern matching	Java 14	Java 16
<code>switch</code> pattern matching (type patterns, guards)	Java 17	Java 21
Record patterns (destructuring)	Java 19	Java 21

Primitive Types in Patterns

Historically, patterns (in `instanceof` and `switch`) only worked with reference types (objects), not primitives like `int` or `double`. **Primitive type patterns** extend pattern matching to primitives, including inside record patterns.

This feature is a **preview feature starting in JDK 23** (JEP 455, "Primitive Types in Patterns, `instanceof`, and `switch`"). It is still evolving – as of Java 25 it remains a preview feature (re-previewed as JEP 507). You must compile and run with `-enable-preview` to use it.

```
// Requires: javac --release 23 --enable-preview Main.java
//           java --enable-preview Main

Object obj = 42;

// instanceof with a primitive pattern (preview):
if (obj instanceof int i) {
    System.out.println("It's an int: " + i);
}
```

It also removes an old rough edge: `switch` on primitives couldn't easily be combined with ranges or exact numeric matching against boxed types. With primitive patterns, you get natural, type-safe matching:

```
static String classify(Object o) {
    return switch (o) {
        case int i when i < 0    -> "negative int";
        case int i               -> "non-negative int: " + i;
        case long l              -> "a long: " + l;
    }
}
```

```

        case double d          -> "a double: " + d;
        default                 -> "something else";
    };
}

```

Because this is preview, do not rely on it in production code yet; the syntax could still change before final release. Always check the JDK release notes before using it on a real project.

```

// Compiling without --enable-preview gives an error like:
// error: primitive patterns in switch and instanceof are a preview
↪ feature
//      and are disabled by default.

```

Switch Expressions

A traditional `switch` is a **statement**: it doesn't produce a value, and it needs `break` to avoid falling through to the next case. A **switch expression** (finalized in **Java 14**) produces a value directly, uses `->` (arrow) syntax, and has no fall-through.

```

// Old style: switch STATEMENT
int day = 3;
String name;
switch (day) {
    case 1:
        name = "Monday";
        break;
    case 2:
        name = "Tuesday";
        break;
    default:
        name = "Unknown";
        break;
}

```

```

// New style: switch EXPRESSION
int day = 3;
String name = switch (day) {
    case 1 -> "Monday";
    case 2 -> "Tuesday";
    default -> "Unknown";
};

```

Multiple case labels can share one arrow:

```
String category = switch (day) {
    case 1, 2, 3, 4, 5 -> "Weekday";
    case 6, 7 -> "Weekend";
    default -> "Invalid";
};
```

When a case needs more than one statement, use a block and `yield` to return the value:

```
int score = 85;
String grade = switch (score / 10) {
    case 10, 9 -> "A";
    case 8 -> {
        System.out.println("Good job!"); // side effect allowed
        yield "B";
    }
    default -> "C or below";
};
```

Switch statement vs. switch expression:

Aspect	switch statement	switch expression
Produces a value	No	Yes
Syntax	case X: with break	case X -> (arrow) or block with yield
Fall-through	Yes, unless you add break	No fall-through
Exhaustiveness check (enums/sealed types)	Not enforced	Compiler can enforce it — missing cases won't compile
Since	Java 1	Java 14

Exhaustiveness with enum or sealed types is a big win for reviewers: the compiler forces you to handle every case (or add a default), catching bugs when a new enum constant is added later but a switch elsewhere wasn't updated.

```
enum TrafficLight { RED, YELLOW, GREEN }

static String action(TrafficLight light) {
    // No default needed: compiler verifies all 3 enum values are covered.
}
```



```

    return switch (light) {
        case RED -> "Stop";
        case YELLOW -> "Slow down";
        case GREEN -> "Go";
    };
}

```

Text Blocks

A **text block** is a multi-line string literal. It removes the need for `\n` and string concatenation when writing multi-line text like JSON, SQL, or HTML. Text blocks were finalized in **Java 15**.

A text block starts and ends with `"""` (three double quotes), and the opening `"""` must be followed by a line break.

```

String json = """
    {
        "name": "Ada",
        "role": "Engineer"
    }
    """;

System.out.println(json);
// Output:
// {
//   "name": "Ada",
//   "role": "Engineer"
// }

```

Before text blocks, the same JSON needed manual escaping:

```

String jsonOld = "{\n" +
    "  \"name\": \"Ada\",\n" +
    "  \"role\": \"Engineer\"\n" +
    "}";

```

Indentation rules: the compiler looks at the **least-indented line** (including the closing `"""`) and strips that much leading whitespace from every line. This means the position of the closing `"""` controls the indentation of the output.

```
String sql = """
    SELECT id, name
    FROM employees
    WHERE department = 'Engineering'
    """;
```

You can still use escape sequences, and a few special ones help control whitespace:

```
String noTrailingNewline = """
    Line one
    Line two\
    """; // "\" at end of line joins it with the next line (no newline
↪ inserted)

String withTrailingSpaces = """
    Trailing spaces matter here.  \s
    """; // \s forces a space that would otherwise be trimmed
```

You can also combine text blocks with `String.format` or `formatted`:

```
String template = """
    Hello, %s!
    You have %d new messages.
    """;
String message = template.formatted("Ada", 5);
```

String Templates (Preview)

String templates were a proposed feature to embed expressions directly inside string literals, similar to JavaScript template literals or Kotlin string interpolation. They used the syntax `STR."Hello \{name}"`.

```
// Example of the PREVIEWED (now withdrawn) syntax – do not use in real
↪ code:
// String name = "Ada";
// int count = 5;
// String message = STR."Hello \{name}, you have \{count} messages.";
```

Important — be accurate about status: String templates were previewed in **Java 21** (JEP 430) and again, with refinements, in **Java 22** (JEP 459). However, based on developer feedback, the OpenJDK team decided the design needed more work, and the feature was **withdrawn** — it did not ship as a preview in Java 23, and there is no finalized version as of Java 25. If you see `STR. " . . . "` syntax in an article or an older tutorial, know that it will not compile on any released JDK.

Until a template mechanism (re-)ships, the standard ways to build formatted strings remain:

```
String name = "Ada";
int count = 5;

// String concatenation
String a = "Hello, " + name + ", you have " + count + " messages.";

// String.format / formatted (classic, since Java 5 / Java 15)
String b = "Hello, %s, you have %d messages.".formatted(name, count);

// Text block + formatted (Java 15+)
String c = """
    Hello, %s, you have %d messages.
    """.formatted(name, count);
```

Code-review takeaway: if a candidate claims string templates are “a stable Java feature,” that is a **red flag** — the honest answer is “previewed twice, then withdrawn; not currently available.”

Unnamed Variables and Patterns

Sometimes you must declare a variable (for a lambda parameter, a catch block, or a pattern component) that you never actually use. The **unnamed variable**, written as a single underscore `_`, lets you say “I don’t need this” explicitly. Finalized in **Java 22** (JEP 456).

```
map.forEach((key, _) -> System.out.println(key)); // value is unused
```

```
try {
    riskyOperation();
} catch (IOException _) {
    // We don't care about the exception details, just that it failed.
```

```
System.out.println("Operation failed");
}
```

Unnamed patterns work well with record deconstruction when you only care about some components:

```
record Point3D(int x, int y, int z) { }

static String flatten(Point3D p) {
    // We only care about x and y; z is discarded.
    if (p instanceof Point3D(var x, var y, var _)) {
        return "(" + x + ", " + y + ")";
    }
    return "invalid";
}
```

You can also use `_` for a loop variable you never read:

```
int count = 0;
for (var _ : someList) {
    count++; // we only care how many elements there are
}
```

Why this matters in review: unlike an unused variable named `ignored` or `e`, the unnamed variable `_` is a language-level signal — some static analysis tools and the compiler itself understand “this is intentionally unused,” reducing false-positive “unused variable” warnings and making intent explicit to the next reader.

Value-Based Classes

A **value-based class** is a class whose instances are treated as pure values, not as objects with identity. `Integer`, `LocalDate`, `Optional`, and (since Java 16) record types are all examples. The JDK documentation defines conventions for such classes; they’re a design pattern/contract, not a language keyword.

Rules for a value-based class (per the JDK’s `java.lang.doc-files/ValueBased.html` conventions):

1. Fields are `final`, and the object is immutable.
2. The class is `final` (or effectively behaves that way) — no subclassing that adds identity-sensitive behavior.

3. Instances are considered interchangeable if `equals()` says they're equal — you should never rely on `==` or synchronize on them.
4. The class does not expose an identity-sensitive API (no exposed lock, no `==`-based contracts).
5. Instances are typically created via static factory methods, not always via visible constructors (e.g. `Optional.of(...)`, `LocalDate.of(...)`).

```
// Integer is value-based: NEVER rely on identity ("==") for comparison.
Integer a = 100;
Integer b = 100;
System.out.println(a == b);      // true (small int cache, but DO NOT rely
↳ on this)

Integer c = 200;
Integer d = 200;
System.out.println(c == d);      // false! Outside the cached range
↳ (-128..127)
System.out.println(c.equals(d)); // true – this is the correct way to
↳ compare
```

```
// Records are value-based by nature.
public record Money(String currency, long cents) { }

Money price1 = new Money("EUR", 1999);
Money price2 = new Money("EUR", 1999);

System.out.println(price1 == price2);      // false – different instances
System.out.println(price1.equals(price2)); // true – same value
```

Because value-based instances may be interchangeable, the JDK explicitly warns against **synchronizing on them**:

```
Integer lock = 42;
synchronized (lock) { // BAD: value-based classes may be
↳ cached/shared/pooled
    // ...
}
```

Use a dedicated `Object` or `ReentrantLock` for locking instead:

```
private final Object lock = new Object();

synchronized (lock) {
    // safe: this object has real identity and is never shared
}
```

Preview and Incubator Features

The JDK ships experimental features in two ways, so developers can try them before they become permanent:

Mechanism	What it means	How to enable
Preview feature	A complete, fully-specified language or API feature, but not yet guaranteed stable. Might change or be withdrawn based on feedback.	Compile and run with <code>--enable-preview</code> (plus <code>--release <N></code> when compiling)
Incubator module	An API delivered in a <code>jdk.incubator.*</code> module, for APIs that need real-world testing before joining the standard <code>java.*</code> API.	Add <code>--add-modules jdk.incubator.<name></code>

Preview features must be explicitly enabled on **both** compilation and execution, and the produced `.class` files are marked so they cannot run on a JDK that isn't the exact same preview-enabled version.

```
# Compiling code that uses a preview feature (e.g. primitive patterns, JDK
↪ 23):
javac --release 23 --enable-preview Main.java

# Running it:
java --enable-preview Main
```

Examples of features that went through the preview process (with outcomes):

Feature	Preview JDKs	Outcome
Records	14, 15	Finalized in 16
Sealed classes	15, 16	Finalized in 17
Pattern matching for switch	17, 18, 19, 20	Finalized in 21
Record patterns	19, 20	Finalized in 21

Feature	Preview JDKs	Outcome
Virtual threads	19, 20	Finalized in 21
Structured concurrency	21, 22, 23, 24	Still preview as of Java 25
String templates	21, 22	Withdrawn — not shipped
Unnamed variables & patterns	21	Finalized in 22
Primitive types in patterns	23 (JEP 455), 24 (JEP 488)	Still preview as of Java 25 (re-previewed as JEP 507)

Incubator module examples:

```
// Vector API (jdk.incubator.vector) – SIMD-style computation, still
↪ incubating
// across many releases (first incubated in Java 16, still incubating in
↪ Java 25).
import jdk.incubator.vector.*;

// Foreign Function & Memory API started as incubator/preview (Java 17–21)
// and was finalized as a standard feature in Java 22.
```

Code-review guidance: preview and incubator features are **not recommended for production code** unless the team has explicitly decided to accept the risk (API changes, or the feature being withdrawn entirely, as happened with string templates). Flag any use of `--enable-preview` in a build file as something that needs a deliberate, documented decision.

Deprecated and Removed Features Across Java Versions

Java evolves by deprecating old APIs before removing them, usually giving developers multiple releases of warning. **Deprecated** means “still works, but don’t use it — it may be removed.” **Removed** means “it’s actually gone; the code won’t compile or run.”

```
@Deprecated(since = "9", forRemoval = true)
public void oldMethod() {
    // ...
}
```

```
// Calling a deprecated method produces a compiler warning:
obj.oldMethod();
// warning: [deprecation] oldMethod() in Foo has been deprecated and
↪ marked for removal
```

Notable deprecations and removals:

Feature	Status	Since / Removed in	Why
<code>Object.finalize()</code>	Deprecated for removal	Deprecated in Java 9, deprecated-for-removal in Java 18	Unpredictable timing, GC overhead, security issues. Use <code>try-with-resources</code> or <code>Cleaner</code> instead.
Security Manager (<code>SecurityManager</code> , <code>System.setSecurityManager</code>)	Removed	Deprecated for removal in Java 17, removed in Java 24	Rarely used correctly in practice, and complicated the JVM. No replacement — sandboxing should happen outside the JVM (containers, OS-level controls).
Applets (<code>java.applet</code>)	Removed	Deprecated in Java 9, removed in Java 17	Browsers dropped plugin support (NPAPI); applets became unusable.
<code>Thread.stop()</code> , <code>Thread.suspend()</code> , <code>Thread.resume()</code>	Deprecated for removal	Deprecated since Java 1.2, still present but strongly discouraged	Inherently unsafe — can leave shared state in a corrupted, half-updated condition. Use cooperative cancellation (<code>volatile</code> flags, <code>interrupt()</code>) instead.
<code>new Integer(...)</code> , <code>new Long(...)</code> , <code>new Boolean(...)</code> , etc. (boxed-type constructors)	Deprecated for removal	Deprecated in Java 9	Wastes memory by creating unnecessary objects; <code>valueOf(...)</code> (or autoboxing) reuses cached instances for common values.
Nashorn JavaScript engine (<code>javax.script</code> , <code>jjs</code>)	Removed	Deprecated in Java 11, removed in Java 15	Fell behind modern JavaScript (ES6+); GraalVM's JS engine is the modern replacement.
<code>Runtime.exec(String)</code> (single-string overload usage patterns)	Still present, but risky usage patterns discouraged	Ongoing guidance, not removed	Splitting a single command string on spaces is fragile and can be an injection risk; prefer the <code>String[]</code> / <code>ProcessBuilder</code> form with explicit arguments.
Java Web Start / <code>javaws</code>	Removed	Removed in Java 11 (was part of Oracle JDK)	Superseded by other deployment approaches; browser plugin model became obsolete.
CORBA and Java EE modules (<code>java.corba</code> , <code>java.xml.ws</code> , <code>java.xml.bind</code> , etc.)	Removed	Deprecated in Java 9, removed in Java 11	Moved out of the JDK; available as separate Maven/Gradle dependencies if still needed.

Feature	Status	Since / Removed in	Why
finalization mechanism as a whole (the Finalizable protocol)	Disabled by default, planned for removal	Disabled by default via <code>--finalization=disabled</code> option available since Java 18, deprecated for removal since Java 18	Same reasons as <code>finalize()</code> above — the whole mechanism is being phased out.
Biased Locking (JVM internal optimization)	Removed	Deprecated in Java 15, removed in Java 18	Modern GC and JIT improvements made the added complexity not worth it.
<code>com.sun.security.auth</code> classes / old JAAS internals	Deprecated	Various	Prefer supported <code>javax.security.auth</code> public APIs.

```
// BAD (deprecated boxed constructor):
Integer wasteful = new Integer(42); // creates a brand-new object every
→ time

// GOOD (uses the internal cache for common values):
Integer efficient = Integer.valueOf(42);
Integer autoboxed = 42; // compiler calls Integer.valueOf() automatically
```

```
// BAD (Thread.stop is unsafe – can corrupt shared state mid-update):
// worker.stop();

// GOOD (cooperative cancellation):
class Worker implements Runnable {
    private volatile boolean running = true;

    public void run() {
        while (running) {
            // do work
        }
    }

    public void shutdown() {
        running = false; // worker checks this flag and exits cleanly
    }
}
```

Code review tip: when you see `@Deprecated` in your own project's code (not a library), check the Javadoc for `forRemoval = true` and a `since` version. That tells you how

urgent the migration is — `forRemoval = true` means it is scheduled to actually disappear.

Common Code-Review Interview Pitfalls

1. **Using `==` to compare boxed types or records instead of `equals()`.** Why it matters: `Integer`, `Long`, and record instances are value-based; identity comparison is misleading and can pass by accident for small cached values, then fail in production for larger ones.

```
// Before (bug hiding in plain sight)
Integer a = 1000, b = 1000;
if (a == b) { /* false! outside Integer cache range */ }

// After
if (a.equals(b)) { /* correctly true */ }
```

2. **Forgetting permits doesn't limit exhaustiveness — missing a default on a non-sealed switch.** Why it matters: only sealed hierarchies and enums give you compiler-enforced exhaustiveness. A switch over a plain interface or class still needs a default, or it won't compile as an expression.

```
// Before: compile error, no default and Shape is not sealed
// String s = switch (shape) { case Circle c -> "circle"; };

// After: either seal Shape, or add a default
String s = switch (shape) {
    case Circle c -> "circle";
    default -> "other";
};
```

3. **Assuming records are always the right replacement for a class.** Why it matters: records cannot extend another class, and all fields are immutable. If a candidate needs inheritance or mutable state, a record is the wrong tool.

```
// Before: trying to add mutable state to a record — won't compile
// record Counter(int count) { void increment() { count++; } } // ERROR

// After: use a regular class for mutable state
class Counter {
    private int count;
    void increment() { count++; }
}
```

```
}
```

4. **Believing string templates (STR. "...") are usable today.** Why it matters: this feature was previewed in Java 21/22 and then withdrawn. Code using STR. "... " will not compile on any released JDK, including 21 through 25.

```
// Before (does not compile on any released JDK):
// String msg = STR."Hello \{name}";

// After (works today):
String msg = "Hello %s".formatted(name);
```

5. **Shipping preview-feature code without --enable-preview documented in the build.** Why it matters: preview class files are tagged and will fail to run on a JDK that isn't the exact matching preview-enabled version — this can break CI/CD or production silently.

```
// Before: build script has no mention of preview, but code uses
↳ primitive patterns
// (compiles locally with a preview-enabled IDE, fails in CI)

// After: explicit and documented
// javac --release 23 --enable-preview Main.java
// java --enable-preview Main
```

6. **Synchronizing on a value-based class instance (e.g. Integer, Long, boxed values).** Why it matters: value-based instances may be cached or otherwise shared; two “different” locks might secretly be the same object, or a future JDK might change caching behavior, causing subtle deadlocks or lost synchronization.

```
// Before
Integer lock = 1;
synchronized (lock) { /* risky: cached instance could be shared
↳ elsewhere */ }

// After
private final Object lock = new Object();
synchronized (lock) { /* safe: unique identity, never reused */ }
```

7. **Using Thread.stop() or relying on finalize() for cleanup.** Why it matters: both are deprecated for removal and unsafe/unreliable. Reviewers should flag any use immediately.

```
// Before
// worker.stop(); // can corrupt shared state
// protected void finalize() { resource.close(); } // timing not
↪ guaranteed

// After
worker.requestShutdown(); // cooperative flag
try (var resource = openResource()) { /* auto-closed */ }
```

- 8. Treating default interface methods as free multiple inheritance without checking for conflicts.** Why it matters: implementing two interfaces with clashing default methods forces the implementing class to resolve the conflict explicitly, or the code won't compile — this is easy to overlook when interfaces evolve independently.

```
// Before: compile error if both A and B declare default String name()
// class C implements A, B { }

// After: explicitly resolve
class C implements A, B {
    public String name() { return A.super.name(); }
}
```

- 9. Forgetting that text block indentation depends on the closing `"""` position.** Why it matters: moving the closing delimiter changes the output's leading whitespace — a common source of “why does my JSON/SQL have extra spaces” bugs.

```
// Before: closing """ at column 0 adds no stripping, keeping unwanted
↪ indentation
String s = """
    line1
""";

// After: align closing delimiter with the intended left margin
String s = """
    line1
    """;
```

- 10. Using an unnamed variable `_` and then trying to read it.** Why it matters: `_` is not a normal variable — the compiler forbids referencing it after declaration. Reusing multiple `_` in the same scope is fine (that's the point), but reading it is not.

```
// Before: compile error
// map.forEach((key, _) -> System.out.println(_)); // ERROR: cannot use
↳ _ as a value

// After
map.forEach((key, _) -> System.out.println(key));
```

- 11. Assuming an enum's ordinal() is a stable identifier for persistence.** Why it matters: ordinal() reflects declaration order. Reordering or inserting a new constant silently shifts every ordinal after it, corrupting any stored data (database rows, serialized files) that relied on the old numbering.

```
// Before: storing ordinal() as a DB value
// int code = status.ordinal(); // fragile: breaks if enum order
↳ changes

// After: store an explicit, stable code
enum OrderStatus {
    NEW(1), PAID(2), SHIPPED(3);
    final int code;
    OrderStatus(int code) { this.code = code; }
}
```

- 12. Marking a class sealed but forgetting permits subclasses must each declare final, sealed, or non-sealed.** Why it matters: this is a compile error waiting to happen for anyone extending a sealed type for the first time, and reviewers should know it's mandatory, not optional style.

```
// Before: compile error – Car doesn't declare its inheritance modifier
// sealed class Vehicle permits Car { }
// class Car extends Vehicle { } // ERROR

// After
sealed class Vehicle permits Car { }
final class Car extends Vehicle { }
```

- 13. Confusing “deprecated” with “removed” and assuming old code will “just still work.”** Why it matters: some features (Security Manager, Applets, Nashorn, CORBA modules) are fully removed, not just discouraged. Code or dependencies relying on them will fail to build or run on modern JDKs, which matters a lot when reviewing an upgrade PR.

```
// Before (Java 8-era code, will NOT run on Java 24+):
// System.setSecurityManager(new SecurityManager()); // REMOVED in Java
↪ 24

// After: use OS/container-level sandboxing instead
// (no direct JVM replacement – redesign the isolation strategy)
```

- 14. Using new Integer(...) / new Boolean(...) style boxed constructors in new code.** Why it matters: these constructors are deprecated for removal and always allocate a new object, defeating the JVM's integer cache and wasting memory for no benefit.

```
// Before
Boolean flag = new Boolean(true); // deprecated, wasteful

// After
Boolean flag = Boolean.TRUE; // or plain `true` with autoboxing
```

- 15. Overusing guarded patterns (case X when ...) instead of simple boolean logic, hurting readability.** Why it matters: pattern matching in switch is powerful, but stacking many when clauses with complex conditions can make a switch harder to read than a plain if/else chain — reviewers should ask whether the pattern actually clarifies intent.

```
// Before: hard to scan, conditions bury the intent
String result = switch (shape) {
    case Circle c when c.radius() > 100 && c.radius() < 1000 &&
↪ isValid(c) -> "big circle";
    default -> "other";
};

// After: extract the condition to a named method for clarity
String result = switch (shape) {
    case Circle c when isValidCircle(c) -> "big circle";
    default -> "other";
};
```

Generics and Type System

Generics let you write code that works with different types while keeping type safety at compile time. They are one of the most interview-heavy topics in Java because the compiler does a lot of invisible work — and that invisible work (erasure) causes surprising rules that trip up even experienced developers. This chapter builds up from basic generic classes to the trickiest interview questions: erasure, bridge methods, wildcards, and why arrays and generics disagree about variance.

By the end, you should be able to explain *why* a rule exists, not just recite it. That is what separates a strong code-review answer from a memorized one.

- Generics
 - Why generics exist
 - Generic classes
 - Generic methods
 - Bounded type parameters
 - Multiple bounds
 - Recursive generic bounds
 - Raw types and unchecked warnings
 - Generic type inference and the diamond operator
 - `var` with generics
 - Generics and `Optional` / streams
- Type Erasure
 - What erasure does at the bytecode level
 - Bridge methods

- Why you cannot do `new T[]`
- Why you cannot do `instanceof List<String>`
- Why you cannot have static generic fields
- Heap pollution and `@SafeVarargs`
- Wildcards (extends, super)
 - Unbounded wildcards `<?>`
 - Upper-bounded wildcards `<? extends T>`
 - Lower-bounded wildcards `<? super T>`
 - PECS: Producer Extends, Consumer Super
 - `List<?>` vs `List<Object>` vs raw `List`
- Covariance and Contravariance
 - Arrays are covariant
 - Generics are invariant
 - `ArrayStoreException` demo
 - Method return type covariance
- Common Code-Review Interview Pitfalls

Generics

Generics let a class, interface, or method work with a *type parameter* — a placeholder for a real type that gets filled in later. Think of a type parameter like a function argument, except it stands for a type (like `String` or `Integer`) instead of a value.

Why generics exist

Before Java 5 (2004), collections held plain `Object` references. You had to cast everything, and mistakes only showed up at runtime.

```
// Pre-generics style (Java 1.4 and earlier) – do not write this today
List oldList = new ArrayList();
oldList.add("hello");
oldList.add(42); // no compile-time check, mixed types allowed
```



```
String s = (String) oldList.get(1); // compiles fine...
// ...but throws java.lang.ClassCastException at runtime
```

Generics move that error from runtime to compile time:

```
List<String> names = new ArrayList<>();
names.add("Ada");
// names.add(42); // compile error: incompatible types: int cannot be
→ converted to String
String first = names.get(0); // no cast needed
```

That is the whole point of generics: **catch type mistakes earlier, and remove casts.**

Generic classes

A generic class declares one or more type parameters in angle brackets after the class name. By convention, type parameters are single, uppercase letters: T (Type), E (Element), K/V (Key/Value), R (Result).

```
public class Box<T> {
    private T content;

    public void set(T content) {
        this.content = content;
    }

    public T get() {
        return content;
    }
}
```

Usage:

```
Box<String> stringBox = new Box<>();
stringBox.set("hello");
String value = stringBox.get(); // no cast

Box<Integer> intBox = new Box<>();
intBox.set(5);
// intBox.set("oops"); // compile error: incompatible types: String cannot
→ be converted to Integer
```

A class can have multiple type parameters:

```
public class Pair<K, V> {
    private final K key;
    private final V value;

    public Pair(K key, V value) {
        this.key = key;
        this.value = value;
    }

    public K getKey() { return key; }
    public V getValue() { return value; }
}

Pair<String, Integer> ageOf = new Pair<>("Grace", 85);
```

Generic methods

A method can introduce its own type parameter, independent of the class it lives in. The type parameter is declared right before the return type.

```
public class Utils {
    // <T> here is the method's own type parameter
    public static <T> T firstNonNull(T a, T b) {
        return a != null ? a : b;
    }

    // Works with any two types that share a common supertype
    public static <T> List<T> listOf(T a, T b) {
        List<T> list = new ArrayList<>();
        list.add(a);
        list.add(b);
        return list;
    }
}

String result = Utils.firstNonNull(null, "fallback"); // T inferred as
↳ String
List<Integer> nums = Utils.listOf(1, 2);                // T inferred as
↳ Integer
```

You rarely need to spell out the type parameter explicitly — the compiler infers it from the arguments. But you can be explicit if inference fails:

```
List<String> empty = Utils.<String>listOf(null, null);
```

Bounded type parameters

Sometimes you want to restrict what types are allowed. An **upper bound** (extends) says “T must be this type or a subtype of it.” This lets you call methods from the bound inside the generic code.

```
// Without a bound, you cannot call numeric methods on T
public class NumberBox<T> {
    private T value;
    // double asDouble() { return value.doubleValue(); } // compile error:
    ↪ cannot find symbol
}

// With an upper bound, Number's methods become available
public class BoundedNumberBox<T extends Number> {
    private T value;

    public BoundedNumberBox(T value) {
        this.value = value;
    }

    public double asDouble() {
        return value.doubleValue(); // OK — every Number has doubleValue()
    }
}

BoundedNumberBox<Integer> b1 = new BoundedNumberBox<>(5);
BoundedNumberBox<Double> b2 = new BoundedNumberBox<>(5.5);
// BoundedNumberBox<String> b3 = new BoundedNumberBox<>("x");
// compile error: String is not a subtype of Number
```

Note that `extends` here means “extends or implements” — it works for both classes and interfaces.

Multiple bounds

A type parameter can have more than one bound: at most one class, plus any number of interfaces, joined with `&`. The class (if present) must come first.

```

interface Named {
    String name();
}

// T must be a Number AND implement Named
static <T extends Number & Named> void printLabeled(T value) {
    System.out.println(value.name() + " = " + value.doubleValue());
}

// class Weird<T extends Named & Number> {} // compile error: class types
↳ must come first

```

Recursive generic bounds (<T extends Comparable<T>>)

This pattern shows up constantly in interviews: a type parameter bounded by a generic type that refers back to itself. It says “T must be comparable to other T’s.”

```

public static <T extends Comparable<T>> T max(List<T> list) {
    T best = list.get(0);
    for (T item : list) {
        if (item.compareTo(best) > 0) {
            best = item;
        }
    }
    return best;
}

List<Integer> nums = List.of(3, 7, 2);
Integer top = max(nums); // works, Integer implements Comparable<Integer>

class Point { int x, y; } // does NOT implement Comparable
List<Point> points = List.of(new Point());
// max(points); // compile error: Point is not a valid substitute for T
↳ extends Comparable<T>

```

Why not just <T extends Comparable> (raw)? Because then `compareTo` would accept any `Object`, losing type safety. The recursive form `Comparable<T>` guarantees you can only compare `T` to other `T`. This exact signature is how `Collections.max` is declared in the JDK.

Raw types and unchecked warnings

A **raw type** is a generic class used without its type argument, e.g. `List` instead of `List<String>`. Raw types exist only for backward compatibility with pre-Java-5 code. Using them today throws away all compile-time type checking.

```
@SuppressWarnings("unchecked")
List raw = new ArrayList(); // raw type – legal but discouraged
raw.add("a string");
raw.add(99); // no error here, but it's a landmine

List<String> typed = raw; // unchecked warning: unchecked conversion
for (String s : typed) { // compiles, but...
    System.out.println(s.length());
}
// Throws java.lang.ClassCastException at the Integer 99,
// because the compiler inserts a hidden (String) cast on read.
```

The compiler warning here is `unchecked conversion` — it means “I cannot verify this is actually a `List<String>`; trust me.” Never silence these warnings without understanding why they are safe (see `@SafeVarargs` below for a legitimate case).

Generic type inference and the diamond operator

Before Java 7, you had to repeat the type argument on both sides:

```
Map<String, List<Integer>> map = new HashMap<String, List<Integer>>(); //
↪ verbose, pre-Java-7
```

The **diamond operator** `<>` (Java 7+) tells the compiler to infer the type argument from the left-hand side (target type):

```
Map<String, List<Integer>> map = new HashMap<>(); // compiler infers
↪ <String, List<Integer>>
```

Inference also works across method calls and generic methods, and Java 10+ can combine it with `var` for local variables (see below). Inference has limits — it works left-to-right and top-down, so it cannot look *forward* into how you use the result:

```
// Compiler cannot infer T just from an empty list with no target type
var list = List.of(); // inferred as List<Object>, probably not what you
↳ wanted

List<String> strings = List.of(); // OK: target type String is known here
```

var with generics

var (Java 10+) infers the *declared* type from the initializer — it does not create a raw or wildcard type, and it does not weaken generics. var is not “dynamic typing”; the type is fixed at compile time, just not written out.

```
var names = new ArrayList<String>(); // names has static type
↳ ArrayList<String>
names.add("Linus");
// names.add(42); // compile error: incompatible types: int cannot be
↳ converted to String
```

Caution: var needs a type on the right to infer from. You cannot use var with the diamond alone:

```
// var list = new ArrayList<>(); // compiles, but infers ArrayList<Object>
↳ — loses precision
var list = new ArrayList<String>(); // be explicit on one side
```

Rule of thumb for code review: if var combined with <> makes the element type unclear at the call site, prefer spelling out the generic type.

Generics and Optional / streams

Optional<T> and the Streams API (Stream<T>) are generic types that lean heavily on inference and bounded wildcards internally. They are covered fully in Chapter 5 and Chapter 8, but a couple of generics-specific points belong here.

```
Optional<String> maybeName = Optional.of("Ada");

// map() is a generic method: <U> Optional<U> map(Function<? super T, ?
↳ extends U> mapper)
Optional<Integer> length = maybeName.map(String::length);

List<String> names = List.of("Ada", "Grace", "Linus");
```

```
List<Integer> lengths = names.stream()
    .map(String::length)    // Stream<String> -> Stream<Integer>, T
    ↪ inferred per stage
    .toList();              // Java 16+ shorthand for
    ↪ collect(Collectors.toList())
```

Notice `Function<? super T, ? extends U>` in `Optional.map`'s real signature — that is the PECS wildcard pattern explained later in this chapter. Streams use the same pattern throughout (`Collector<? super T, A, R>`, etc.) so that a pipeline can accept producers and consumers of *related* types, not just exact matches.

Type Erasure

Type erasure is how the Java compiler implements generics: type parameters exist only in source code and are checked at compile time, then *erased* (removed) before bytecode is generated. At runtime, there is no `List<String>` — there is only `List`, with a compiler-inserted cast wherever you read an element.

Erase exists for one reason: **backward compatibility**. Java 5 needed generic collections to run on a JVM whose bytecode format did not change, and to interoperate with pre-generics `.class` files already in production. Erasure made that possible at the cost of some runtime type information.

What erasure does at the bytecode level

Given this source:

```
public class Box<T> {
    private T value;

    public void set(T value) {
        this.value = value;
    }

    public T get() {
        return value;
    }
}
```

The compiler erases `T` to its bound — `Object`, since `T` has no bound here — and produces bytecode roughly equivalent to:

```
// What the erased bytecode is equivalent to (not real source you write)
public class Box {
    private Object value;

    public void set(Object value) {
        this.value = value;
    }

    public Object get() {
        return value;
    }
}
```

Anywhere you call `box.get()` from code that expects `String`, the compiler inserts a cast:

```
Box<String> box = new Box<>();
box.set("hi");
String s = box.get();
// Compiled roughly as:
// String s = (String) box.get();
```

You can verify this yourself with `javap -c`:

```
javac Box.java
javap -c Box.class
# Look at the set(Ljava/lang/Object;)V and get()Ljava/lang/Object;
↪ signatures –
# there is no T anywhere in the compiled descriptor.
```

If `T` is bounded, e.g. `<T extends Number>`, erasure uses the bound instead of `Object`:

```
public class NumberBox<T extends Number> {
    private T value;
    public T get() { return value; }
}
// Erased: private Number value; public Number get() { return value; }
```

Bridge methods

Erasure creates a subtle problem with overriding. Consider a generic interface and a class that implements it with a concrete type:


```

interface Container<T> {
    void set(T value);
}

class StringContainer implements Container<String> {
    @Override
    public void set(String value) {
        System.out.println("Set: " + value);
    }
}

```

After erasure, `Container<T>.set` becomes `set(Object)`. But `StringContainer.set(String)` does not have the same erased signature as `set(Object)` — so it would not actually override the interface method at the bytecode level, breaking polymorphism.

The compiler fixes this by generating a hidden **bridge method**: a synthetic `set(Object)` method in `StringContainer` that casts and delegates to `set(String)`.

```

// What the compiler generates behind the scenes (not something you write)
class StringContainer implements Container<String> {
    public void set(String value) { /* real body */ }

    // synthetic bridge method, added automatically
    public void set(Object value) {
        set((String) value); // cast, then delegate
    }
}

```

You can see this with reflection — `StringContainer.class.getDeclaredMethods()` lists two `set` methods, and `Method.isBridge()` is true for the synthetic one. This is a classic “gotcha” interview question: “why does my class have two `set` methods when I only wrote one?”

]

Why you cannot do `new T[]` You cannot create an array of a generic type parameter directly, because array creation needs a real, runtime-known component type — and after erasure, `T` is gone.

```

public class Stack<T> {
    // private T[] items = new T[10];
    // compile error: generic array creation
}

```

```

    @SuppressWarnings("unchecked")
    private T[] items = (T[]) new Object[10]; // common workaround, still
    ↪ an unchecked cast
}

```

The workaround `((T[]) new Object[10])` compiles but is unsafe: the actual runtime array is `Object[]`, not `T[]`. If you ever leak that array reference out and someone assigns it to a more specific array variable, you get a `ClassCastException` later. The safer alternative, when you truly need a typed array, is to require a `Class<T>` token from the caller and use `java.util.reflect.Array.newInstance`:

```

    @SuppressWarnings("unchecked")
    public static <T> T[] newArray(Class<T> type, int size) {
        return (T[]) java.lang.reflect.Array.newInstance(type, size);
    }

    String[] strings = newArray(String.class, 5); // works, and is actually a
    ↪ String[] at runtime

```

Why you cannot do `instanceof List<String>`

`instanceof` is a runtime check, and after erasure there is no runtime distinction between `List<String>` and `List<Integer>` — both are just `List`.

```

List<String> list = new ArrayList<>();

// if (list instanceof List<String>) { } // compile error: illegal generic
    ↪ type for instanceof

if (list instanceof List<?>) { // OK – unbounded wildcard is allowed
    System.out.println("it's some kind of List");
}

if (list instanceof ArrayList<String> al) { // still compile error, same
    ↪ reason
        // ...
}

```

Only the unbounded wildcard form (`List<?>`) or the raw form (`List`) is legal with `instanceof`, because those do not claim any specific type argument that would need a runtime check.

Why you cannot have static generic fields

A static field belongs to the class itself, and a class exists once at runtime, regardless of how many different type arguments (`Box<String>`, `Box<Integer>`, ...) are used in source code. Since erasure removes `T` from the class entirely, there is nothing for a static field of type `T` to mean.

```
public class Box<T> {
    // private static T defaultValue; // compile error: non-static type
    ↪ variable T cannot be referenced from a static context

    private static Object defaultValue; // fine – just use Object, or a
    ↪ concrete type
}
```

Static generic *methods* are fine (`static <T> T identity(T t)`), because the method itself introduces a fresh type parameter each time it is called — it does not depend on the class’s erased type parameter.

Heap pollution and @SafeVarargs

Heap pollution happens when a variable of a parameterized type refers to an object that is not actually of that parameterized type — usually caused by mixing raw types, unchecked casts, or varargs with generics.

```
static void dangerous(List<String>... lists) { // varargs of a generic
    ↪ type – allowed, but risky
        Object[] array = lists;           // varargs is implemented as an array,
    ↪ arrays are covariant (see below)
        array[0] = List.of(42);           // legal at the array level:
    ↪ List<Integer> stored where List<String> expected
        String s = lists[0].get(0);       // compiles fine...
        // ...throws ClassCastException at runtime: Integer cannot be cast to
    ↪ String
    }
```

This compiles only with an unchecked warning at the declaration site, because `T...` varargs create a hidden array of a generic type — exactly the “arrays are covariant, generics are not” mismatch. If you are certain your varargs method never stores anything unsafe into that array (i.e., it only *reads* from it), you can suppress the warning with `@SafeVarargs`:

```

@SafeVarargs // a promise to the compiler AND to readers: this method is
↳ safe
static <T> List<T> listOf(T... items) {
    return Arrays.asList(items); // only reads from the array, never
↳ writes into it
}

```

`@SafeVarargs` is only allowed on static or final methods, private instance methods (Java 9+), and constructors — never on a regular overridable instance method, because a subclass could override it with unsafe behavior, breaking the promise.

```

// @SafeVarargs
// void notAllowed(List<String>... lists) { } // compile error if the
↳ method is a non-final, non-private instance method

```

Wildcards (extends, super)

A **wildcard** (?) represents an unknown type argument. Wildcards exist to make generic APIs more flexible when you do not need to know or care about the exact type argument — only what you can do with it (read vs. write).

Unbounded wildcards <?>

<?> means “a list of some type, I don’t know or care which.” Use it when your code only needs methods that do not depend on the type argument at all (e.g., `size()`, `clear()`).

```

static void printSize(List<?> list) {
    System.out.println("Size: " + list.size()); // fine, size() doesn't
↳ depend on T
}

static void printAll(List<?> list) {
    for (Object o : list) { // you can only read elements as Object
        System.out.println(o);
    }
    // list.add("x"); // compile error: no way to prove String matches the
↳ unknown type
}

```

Upper-bounded wildcards <? extends T>

<? extends Number> means “a list of some unknown type that is Number or a sub-type of Number.” You can safely **read** elements out as Number, but you cannot safely **add** to it (the compiler cannot know the exact subtype, so any insert could violate it).

```
static double sumAll(List<? extends Number> numbers) {
    double total = 0;
    for (Number n : numbers) { // reading is safe – every element IS-A
        ↪ Number
            total += n.doubleValue();
    }
    // numbers.add(1);           // compile error: cannot add to a List<?
    ↪ extends Number>
    // numbers.add(1.0);         // compile error, same reason
    return total;
}

List<Integer> ints = List.of(1, 2, 3);
List<Double> doubles = List.of(1.0, 2.0);
sumAll(ints);    // OK
sumAll(doubles); // OK – this flexibility is the whole point
```

Lower-bounded wildcards <? super T>

<? super Integer> means “a list of some unknown type that is Integer or a super-type of Integer.” You can safely **write** Integers into it, but reading only gives you Object (the compiler only knows the list holds *at least* Integer-compatible items, so it cannot promise a more specific type back).

```
static void addNumbers(List<? super Integer> list) {
    list.add(1); // safe – any supertype of Integer can hold an Integer
    list.add(2);
    // Integer first = list.get(0); // compile error: get() returns Object,
    ↪ not Integer
    Object first = list.get(0); // this is all you get back
}

List<Number> numbers = new ArrayList<>();
List<Object> objects = new ArrayList<>();
addNumbers(numbers); // OK, Number is a supertype of Integer
addNumbers(objects); // OK, Object is a supertype of Integer
```

PECS: Producer Extends, Consumer Super

PECS is the mnemonic for choosing the right wildcard: if a parameterized type *produces* (you read from it), use `extends`. If it *consumes* (you write into it), use `super`. If it does both, use no wildcard at all (an exact type).

| Role | Wildcard | You can read | You can write |
|----------------|----------------------------|----------------|---------------------------------|
| Producer only | ? extends T | Yes, as T | No (compile error) |
| Consumer only | ? super T | Only as Object | Yes, T and subtypes |
| Both | exact type T (no wildcard) | Yes, as T | Yes, as T |
| Neither (rare) | ? (unbounded) | Only as Object | No (compile error, except null) |

The JDK's own `Collections.copy` is the textbook PECS example:

```
// Real JDK signature:
// public static <T> void copy(List<? super T> dest, List<? extends T>
↪ src)
public static <T> void copy(List<? super T> dest, List<? extends T> src) {
    for (int i = 0; i < src.size(); i++) {
        dest.set(i, src.get(i)); // read (produce) from src, write
↪ (consume) into dest
    }
}

List<Object> dest = new ArrayList<>(List.of(0, 0, 0));
List<Integer> src = List.of(1, 2, 3);
copy(dest, src); // src produces Integers, dest consumes them as Objects –
↪ PECS in action
```

Without PECS, `copy` would need to be `copy(List<T> dest, List<T> src)`, forcing both lists to hold the *exact* same type — you could not copy `List<Integer>` into `List<Object>`, even though that is perfectly safe.

List<?> vs List<Object> vs raw List

These three look similar but behave very differently. This is a favorite whiteboard question.

Expression	Meaning	Can you add (String)?	Can you add (Object)?	Can you assign List<String> to it?
List<?>	list of <i>some</i> unknown, fixed type	No	No	Yes
List<Object>	list whose element type IS Object	Yes	Yes	No (compile error)
List (raw)	no compile-time type checking at all	Yes (unchecked)	Yes (unchecked)	Yes (with unchecked warning)

```

List<String> strings = new ArrayList<>(List.of("a", "b"));

List<?> wildcard = strings;           // OK, ? matches any type argument
// wildcard.add("c");                 // compile error: unknown type, can't
⇒ prove safety

List<Object> objects = strings; // compile error: incompatible types
// The compiler will not let this compile:
// error: incompatible types: List<String> cannot be converted to
⇒ List<Object>

List raw = strings;                   // compiles, with an "unchecked" nature –
⇒ no error, no warning here
raw.add(42);                         // compiles! but corrupts the underlying
⇒ List<String> at runtime
// strings.get(2); // throws ClassCastException – the "42" is really an
⇒ Integer, cast to String fails

```

That last example is exactly why raw types are dangerous: they let you silently break the type-safety promise that generics exist to enforce.

Covariance and Contravariance

Covariance means “a subtype relationship is preserved”: if `Cat` is a subtype of `Animal`, then `Cat[]` is treated as a subtype of `Animal[]`. **Contravariance** is the reverse direction (used for function parameters, e.g. `? super T`). **Invariance** means no such relationship exists at all — `List<Cat>` and `List<Animal>` are completely unrelated types, even though `Cat` and `Animal` are related.

Java arrays are covariant. Java generics are invariant. This mismatch is one of the most important things to understand for a code-review interview, because it explains half of the wildcard rules above.

Arrays are covariant

```
Object[] objects = new String[3]; // legal: String[] IS-A Object[]
objects[0] = "hello";             // fine
```

This is convenient, but it means the *compile-time* type of the array reference (`Object[]`) does not match its *actual runtime* type (`String[]`). The JVM has to check every array write at runtime to protect against violations.

Generics are invariant

```
// List<Object> objects = new ArrayList<String>();
// compile error: incompatible types: ArrayList<String> cannot be
//   ↪ converted to List<Object>
```

Even though `String` IS-A `Object`, `List<String>` is NOT a `List<Object>`. This is invariance, and it is deliberate — generics were designed to catch the exact mistake that array covariance allows to slip through to runtime (see next section). If you need covariant-like behavior for generics, that is exactly what `?` extends `T` wildcards are for.

ArrayStoreException demo

Because arrays check types at runtime, a covariant array write that violates the actual element type fails with `ArrayStoreException` — at runtime, not compile time.

```
Object[] objects = new String[3]; // compiles: array covariance
objects[0] = "safe";             // OK, actually a String
objects[1] = 42;                 // compiles (Integer IS-A Object)...
// ...but throws java.lang.ArrayStoreException at runtime,
// because the actual array is a String[] and 42 is not a String.
```

Generics avoid this entire class of bug at compile time:


```
List<String> list = new ArrayList<>();
list.add("safe");
// list.add(42); // compile error: incompatible types – caught here, never
→ reaches runtime
```

This is the single clearest argument for “generics over arrays” in a code review: **prefer `List<T>` to `T[]`** in APIs whenever possible, because the failure mode moves from a runtime exception to a compile error.

Aspect	Arrays	Generics
Variance	Covariant (<code>String[]</code> IS-A <code>Object[]</code>)	Invariant (<code>List<String></code> is NOT a <code>List<Object></code>)
Type check timing	Runtime (per-element store check)	Compile time (erased after that)
Failure mode on mismatch	<code>ArrayStoreException</code> at runtime	Compile error, or none if avoided by design
Runtime type info	Preserved (<code>array.getClass()</code> knows <code>String[]</code>)	Erased (no <code>List<String>.class</code>)

Method return type covariance

There is a second, unrelated meaning of “covariance” that shows up in overriding: an overriding method’s return type may be a *subtype* of the overridden method’s return type. This is legal and has nothing to do with type erasure — it is about the class hierarchy, not generics.

```
class Animal {
    Animal reproduce() { return new Animal(); }
}

class Cat extends Animal {
    @Override
    Cat reproduce() { return new Cat(); } // covariant return type – legal
→ since Java 5
}
```

Interview tip: do not confuse this (covariant return types, a class-hierarchy feature) with generic type variance (`List<Cat>` vs `List<Animal>`, which is invariant unless you use wildcards). They are two different “covariance” concepts that happen to share a name.

Common Code-Review Interview Pitfalls

1. **Using raw types “to make the compiler stop complaining.”** Why it matters: raw types disable all generic type checking, turning compile-time errors into runtime `ClassCastExceptions`.

```
// Before
List list = new ArrayList();
list.add("a");
list.add(1); // silently allowed

// After
List<String> list = new ArrayList<>();
list.add("a");
// list.add(1); // compile error – caught immediately
```

2. **Suppressing unchecked warnings without verifying safety.** Why it matters: `@SuppressWarnings("unchecked")` is a promise to the compiler that you manually verified type safety. Blind suppression hides real bugs.

```
// Before – suppressed without justification
@SuppressWarnings("unchecked")
List<String> list = (List<String>) someRawList;

// After – scoped narrowly, with a comment explaining why it's safe
@SuppressWarnings("unchecked") // safe: someRawList is populated
↳ exclusively by addString() above
List<String> list = (List<String>) someRawList;
```

3. **Designing an API with `T[]` instead of `List<T>`.** Why it matters: arrays are covariant and only check types at runtime (`ArrayStoreException`); `List<T>` is invariant and checked at compile time.

```
// Before
void process(Number[] values) { values[0] = 3.14; } //
↳ ArrayStoreException risk if called with Integer[]

// After
void process(List<? extends Number> values) { /* read-only access is
↳ compile-time safe */ }
```

4. **Using `? extends T` on a parameter you need to write into.** Why it matters: `extends` wildcards are producer-only; the compiler will reject `add()` calls, and “fixing” it by removing the wildcard silently loses flexibility.

```
// Before
void fill(List<? extends Number> list) {
    // list.add(1); // compile error
}

// After – use super if you need to write, or an exact type if you need
↳ both
void fill(List<? super Number> list) {
    list.add(1); // OK
}
```

5. **Assuming `List<String>` and `List<Object>` are interchangeable.** Why it matters: generics are invariant; this is one of the most common “why won’t this compile” confusions.

```
// Before
// List<Object> objs = new ArrayList<String>(); // compile error

// After – use a wildcard if you only need read access
List<? extends Object> objs = new ArrayList<String>(); // fine, though
↳ rarely useful since Object adds nothing
```

6. **Trying `instanceof List<String>` or catching a specific generic exception type per type argument.** Why it matters: type erasure removes the type argument at runtime; the JVM cannot check it.

```
// Before
// if (obj instanceof List<String>) { } // compile error

// After
if (obj instanceof List<?> list && !list.isEmpty() && list.get(0)
↳ instanceof String) {
    // manually verify the first element's runtime type instead
}
```

7. **Forgetting that overloaded generic methods can collide after erasure.** Why it matters: two overloads that differ only by type argument have the *same* erased signature and will not compile.

```

// Before
// void process(List<String> list) { }
// void process(List<Integer> list) { } // compile error: both erase to
↪ process(List)

// After – differentiate by method name or take a wrapper/marker type
void processStrings(List<String> list) { }
void processIntegers(List<Integer> list) { }

```

- 8. Writing an unsafe generic varargs method without @SafeVarargs, or worse, adding @SafeVarargs to an unsafe one.** Why it matters: @SafeVarargs is a promise; misusing it hides real heap pollution bugs, and omitting it on a genuinely safe method just produces noisy warnings for every caller.

```

// Before – unsafe, and no annotation to flag the risk
static <T> void addAll(List<T> list, T... items) {
    Object[] array = items;
    array[0] = "corrupt"; // writes an unrelated type into the array –
↪ heap pollution
}

// After – either make it actually safe (read-only) and annotate, or
↪ take a List parameter instead
@SafeVarargs
static <T> List<T> listOf(T... items) {
    return List.of(items); // read-only, never stores into items
}

```

- 9. Expecting a static field or method to “remember” the type argument per instance.** Why it matters: erasure means there is one class, one set of static members, shared across all parameterizations (Box<String>, Box<Integer>, ...).

```

// Before
class Box<T> {
    // static T lastCreated; // compile error
}

// After
class Box<T> {
    static Object lastCreated; // shared across all Box<...> instances,
↪ by design
}

```

- 10. Not recognizing bridge methods when reading decompiled/reflected code.** Why it matters: reviewers who see two overloaded-looking methods with the same name after implementing a generic interface may think it's a bug, when it's a compiler-generated bridge method required for correct polymorphism.

```
interface Container<T> { void set(T value); }
class StringContainer implements Container<String> {
    public void set(String value) { /* ... */ }
    // compiler silently also generates: public void set(Object value)
    ↪ { set((String) value); }
}
```

- 11. Using a recursive bound incorrectly, e.g. <T extends Comparable> (raw) instead of <T extends Comparable<T>>.** Why it matters: the raw bound loses type safety — `compareTo` accepts any `Object`, so mismatched comparisons compile and fail at runtime instead of compile time.

```
// Before
static <T extends Comparable> T max(List<T> list) { /* ... */ } // raw
↪ Comparable, weak guarantee

// After
static <T extends Comparable<T>> T max(List<T> list) { /* ... */ } // T
↪ can only compare to T
```

- 12. Confusing `var` with type erasure or loss of type safety.** Why it matters: `var` infers a concrete compile-time type from the initializer; it is not related to generics' runtime erasure and does not weaken checking.

```
// Before – reviewer flags this as "unsafe, dynamic typing"
var names = new ArrayList<String>();

// After – clarify: names has static type ArrayList<String>, exactly as
↪ if written out
ArrayList<String> names = new ArrayList<>(); // equivalent compile-time
↪ behavior
```

- 13. Relying on `List.of()` or an empty generic literal without a target type.** Why it matters: without a target type, the compiler infers the most general type (often `Object`), which silently defeats type checking downstream.

```
// Before
var empty = List.of(); // inferred as List<Object>

// After – give the compiler a target type
List<String> empty = List.of(); // inferred correctly as List<String>
```

- 14. Treating PECS as optional API polish instead of a correctness tool.** Why it matters: skipping extends/super on library-style methods forces callers into exact-type matches, breaking otherwise-valid calls and causing unnecessary casts or `List.copyOf()` calls at call sites.

```
// Before – overly strict, exact type required
static double sum(List<Double> values) { /* ... */ }
// sum(List.of(1, 2, 3)); // compile error: List<Integer> is not
↳ List<Double>

// After – PECS: this is a producer, so use extends
static double sum(List<? extends Number> values) { /* ... */ }
// sum(List.of(1, 2, 3)); // now compiles
```

- 15. Assuming array covariance is “safe” because it compiles.** Why it matters: covariant array writes that violate the actual runtime type compile cleanly but throw `ArrayStoreException` in production — often only under rarely-hit code paths, making it a nasty review-time-invisible bug.

```
// Before
Object[] items = new String[3];
items[0] = 42; // compiles, throws ArrayStoreException at runtime

// After – use a generic collection so the mistake is caught at compile
↳ time
List<String> items = new ArrayList<>();
// items.add(42); // compile error, caught immediately
```

Object Class & Common APIs

Every class in Java, whether you write it or not, is a child of `java.lang.Object`. That means every object you create already has a few methods built in: `equals()`, `hashCode()`, `toString()`, `clone()`, and others. In code review, bugs in these “invisible” methods are some of the most common findings — a broken `equals()` can silently corrupt a `HashMap`, and a missing defensive copy can let external code mutate your “immutable” class. This chapter walks through these APIs in depth, plus the related tools (`Optional`, `Comparator`, immutability patterns) that reviewers check every day.

`equals()`, `hashCode()`, and `toString()`

`Object` gives every class three methods for free:

- `equals(Object other)` — decides if two objects are “equal”.
- `hashCode()` — returns an `int` “fingerprint” used by hash-based collections (`HashMap`, `HashSet`).
- `toString()` — returns a human-readable `String` representation, used by logging, debuggers, and string concatenation.

The default implementations (inherited straight from `Object`) compare **identity** (are they the same object in memory?) and print the class name plus a memory-based hash, like `com.acme.User@1b6d3586`. That is almost never what you want for a data-carrying class.

```
class User {  
    private final String email;  
    User(String email) { this.email = email; }  
}
```

```
User a = new User("a@acme.com");
User b = new User("a@acme.com");

System.out.println(a.equals(b)); // false! Different objects, same data.
System.out.println(a);           // User@1b6d3586 - useless in logs.
```

The equals() contract

If you override `equals()`, you must honor a strict contract. It is defined in the Javadoc of `Object.equals` and every reviewer should be able to recite it:

Property	Meaning
Reflexive	<code>x.equals(x)</code> must be true.
Symmetric	If <code>x.equals(y)</code> is true, then <code>y.equals(x)</code> must also be true.
Transitive	If <code>x.equals(y)</code> and <code>y.equals(z)</code> are true, then <code>x.equals(z)</code> must be true.
Consistent	Calling <code>x.equals(y)</code> repeatedly returns the same result, as long as neither object changed.
Null comparison	<code>x.equals(null)</code> must return false — never throw a <code>NullPointerException</code> .

A typical, correct `equals()/hashCode()` pair:

```
import java.util.Objects;

final class Point {
    private final int x;
    private final int y;

    Point(int x, int y) {
        this.x = x;
        this.y = y;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;           // reflexive shortcut
        if (o == null || getClass() != o.getClass()) return false; //
        ↪ null-safe + type check
        Point other = (Point) o;
```



```

        return x == other.x && y == other.y;
    }

    @Override
    public int hashCode() {
        return Objects.hash(x, y);
    }

    @Override
    public String toString() {
        return "Point{x=" + x + ", y=" + y + "}";
    }
}

```

The hashCode() contract, and why it must match equals()

The rule is simple but easy to forget: **if two objects are equal according to equals(), they must have the same hashCode().** The reverse is not required — unequal objects are *allowed* to share a hash code (a “hash collision”), but equal objects can never disagree on their hash.

```

// BROKEN: overrides equals() but not hashCode()
class Bad {
    private final String id;
    Bad(String id) { this.id = id; }

    @Override
    public boolean equals(Object o) {
        return o instanceof Bad b && b.id.equals(id);
    }
    // hashCode() NOT overridden – still identity-based!
}

Set<Bad> set = new HashSet<>();
set.add(new Bad("x"));
System.out.println(set.contains(new Bad("x"))); // false! Different hash
→ buckets.

```

HashSet and HashMap first look at hashCode() to find the right “bucket,” then use equals() inside that bucket. If hashCode() is inconsistent with equals(), two “equal” objects can land in different buckets and the collection will never find one using the other. This is one of the most common code-review findings: **“you overrode equals() but not hashCode().”** Modern IDEs and static analyzers (and the compiler with certain lint flags) will warn about this.

Mutable fields as HashMap keys — a classic bug

Never use a mutable object as a HashMap key (or HashSet element) if its hashCode() depends on fields that can change after insertion.

```
class MutableKey {
    private int value;
    MutableKey(int value) { this.value = value; }
    void setValue(int value) { this.value = value; }

    @Override
    public boolean equals(Object o) {
        return o instanceof MutableKey k && k.value == value;
    }
    @Override
    public int hashCode() {
        return Integer.hashCode(value);
    }
}

Map<MutableKey, String> map = new HashMap<>();
MutableKey key = new MutableKey(1);
map.put(key, "hello");

key.setValue(2); // mutate the key AFTER insertion
System.out.println(map.get(key)); // null! It's stored in the bucket for
↳ hashCode(1), // but now key.hashCode() == 2.
```

Rule of thumb: keys used in hash-based collections should be immutable, or at least never mutated while stored as a key. This is why String, Integer, records, and enums are popular map keys — they are immutable.

getClass() vs instanceof — the symmetry trap with inheritance

There are two common ways to type-check inside equals(), and they behave differently with subclassing:

Approach	Behavior	Risk
getClass() != o.getClass() Class()	Only equal to objects of the exact same class.	Safe from symmetry bugs, but two logically-equal instances of different subclasses are never equal.

Approach	Behavior	Risk
o instanceof MyClass	Equal to any subclass instance too.	Can break symmetry if a subclass adds fields and also overrides equals().

```

class Point {
    int x, y;
    Point(int x, int y) { this.x = x; this.y = y; }

    @Override
    public boolean equals(Object o) {
        if (!(o instanceof Point p)) return false; // instanceof-based
        return x == p.x && y == p.y;
    }
}

class ColorPoint extends Point {
    String color;
    ColorPoint(int x, int y, String color) {
        super(x, y);
        this.color = color;
    }

    @Override
    public boolean equals(Object o) {
        if (!(o instanceof ColorPoint cp)) return false;
        return super.equals(cp) && color.equals(cp.color);
    }
}

Point p = new Point(1, 2);
ColorPoint cp = new ColorPoint(1, 2, "red");

System.out.println(p.equals(cp)); // true -> cp IS a Point with matching
    ↪ x,y
System.out.println(cp.equals(p)); // false -> p is not a ColorPoint
// SYMMETRY BROKEN: p.equals(cp) != cp.equals(p)

```

Effective Java's advice (Joshua Bloch): **favor composition over inheritance for value classes**, or use getClass() equality when subclassing is possible, or make the class final. If a class is designed to be extended, consider giving it no equals()/hashCode() at all, or documenting clearly that subclasses must not add fields relevant to equality.

```
// SAFER: getClass() check avoids the symmetry problem
@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (o == null || getClass() != o.getClass()) return false;
    Point p = (Point) o;
    return x == p.x && y == p.y;
}
```

Objects.equals and Objects.hash — null-safe helpers

`java.util.Objects` provides small utilities that remove a lot of boilerplate and, importantly, handle `null` correctly:

```
import java.util.Objects;

class Person {
    private final String name;    // could be null
    private final Integer age;    // could be null

    Person(String name, Integer age) {
        this.name = name;
        this.age = age;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Person p)) return false;
        return Objects.equals(name, p.name) && Objects.equals(age, p.age);
    }

    @Override
    public int hashCode() {
        return Objects.hash(name, age); // null-safe, handles any number
        ↪ of fields
    }
}
```

Without `Objects.equals`, comparing potentially-null fields with `name.equals(p.name)` risks a `NullPointerException`. `Objects.equals(a, b)` returns `true` if both are `null`, `false` if only one is `null`, and delegates to `a.equals(b)` otherwise.

Records give you equals/hashCode/toString for free

Since Java 16 (standard), record types auto-generate `equals()`, `hashCode()`, and `toString()` based on all their components, and the generated code is correct by construction (using `getClass()` semantics, so it is symmetric).

```
record Point(int x, int y) {}

Point a = new Point(1, 2);
Point b = new Point(1, 2);

System.out.println(a.equals(b)); // true
System.out.println(a);           // Point[x=1, y=2]
System.out.println(a.hashCode() == b.hashCode()); // true
```

In a code review, if you see a hand-written data class with manually written `equals()`/`hashCode()` and no extra behavior, a very reasonable comment is: **“could this be a record?”**

IDE-generated equals/hashCode

IntelliJ, Eclipse, and Lombok’s `@EqualsAndHashCode` all generate correct, contract-following code — but they generate it based on a snapshot of the fields *at generation time*. Common review issue: a field is added later and the developer forgets to regenerate `equals()`/`hashCode()`, so the new field is silently ignored in comparisons.

```
// Generated when the class had only `id`. Later, `status` was added but
↪ equals() was
// never regenerated:
class Order {
    private final String id;
    private String status; // added later, NOT included below

    @Override
    public boolean equals(Object o) {
        if (!(o instanceof Order order)) return false;
        return Objects.equals(id, order.id); // status is silently
↪ ignored!
    }
    @Override
    public int hashCode() {
        return Objects.hash(id);
    }
}
```

Whether that is a bug depends on intent — sometimes you *want* equality by ID only (e.g., entity semantics). But it must be a deliberate choice, documented in a comment, not an oversight.

Object Cloning

Object declares a protected method called `clone()` that is supposed to create a copy of an object. In practice, it is widely considered one of Java's worst-designed APIs, and most style guides (including Effective Java) recommend avoiding it.

Why Cloneable is broken

To use `clone()`, a class must implement the empty marker interface `Cloneable` — but `Cloneable` does not declare a `clone()` method itself. It just flips a hidden flag that `Object.clone()` checks at runtime. If you forget implements `Cloneable`, calling `clone()` throws `CloneNotSupportedException` — a checked exception, discovered only at runtime, not compile time.

```
class Box implements Cloneable {
    private int size;
    Box(int size) { this.size = size; }

    @Override
    public Box clone() {
        try {
            return (Box) super.clone(); // field-by-field shallow copy
        } catch (CloneNotSupportedException e) {
            throw new AssertionError(e); // "can't happen" since we
↪ implement Cloneable
        }
    }
}
```

Problems with this design:

- `Cloneable/clone()` bypass constructors entirely — `super.clone()` does a raw memory copy, so invariants enforced in your constructor are never re-checked.
- The contract is vague and unenforceable by the compiler (no `@Override`-style safety net for correctness).

- Subclassing is fragile: if a subclass adds fields, it must remember to also override `clone()` correctly, and `final` fields cannot be “fixed up” after `super.clone()`.
- Checked exception (`CloneNotSupportedException`) makes calling code messy even though, in a correct implementation, it never actually happens.

Shallow copy vs deep copy

`Object.clone()` (and any naive copy) performs a **shallow copy**: primitive fields are duplicated, but reference fields (objects, arrays, collections) still point to the *same* underlying object. A **deep copy** recursively copies referenced objects too, so the two copies share no mutable state.

```
class Team implements Cloneable {
    private String name;
    private List<String> members; // reference field

    Team(String name, List<String> members) {
        this.name = name;
        this.members = members;
    }

    @Override
    public Team clone() {
        try {
            Team copy = (Team) super.clone(); // shallow: `members` list
            ↪ is SHARED
            return copy;
        } catch (CloneNotSupportedException e) {
            throw new AssertionError(e);
        }
    }
}

List<String> members = new ArrayList<>(List.of("Alice", "Bob"));
Team original = new Team("Red", members);
Team shallow = original.clone();

shallow.members.add("Eve"); // mutates the list shared by BOTH teams!
System.out.println(original.members); // [Alice, Bob, Eve] – surprise
↪ mutation
```

A correct deep copy must clone (or defensively copy) every mutable reference field:

```

@Override
public Team clone() {
    try {
        Team copy = (Team) super.clone();
        copy.members = new ArrayList<>(this.members); // deep-ish copy of
↳ the list
        return copy;
    } catch (CloneNotSupportedException e) {
        throw new AssertionError(e);
    }
}

```

The better alternative: copy constructors and static factories

Most experienced reviewers push back on `clone()` in favor of a **copy constructor** or a **static factory method**. Both are simpler, do not throw checked exceptions, run through normal constructor logic (so invariants are validated), and work fine with `final` fields.

```

final class Team {
    private final String name;
    private final List<String> members;

    Team(String name, List<String> members) {
        this.name = name;
        this.members = new ArrayList<>(members); // defensive copy, see
↳ later section
    }

    // Copy constructor
    Team(Team other) {
        this(other.name, other.members);
    }

    // Static factory alternative
    static Team copyOf(Team other) {
        return new Team(other.name, other.members);
    }
}

Team original = new Team("Red", List.of("Alice", "Bob"));
Team copy = new Team(original); // clean, no checked exceptions, no
↳ Cloneable dance

```


Approach	Pros	Cons
Object. clone() / Cloneable	Built into the language	Bypasses constructors, checked exception, fragile with subclasses and final fields
Copy constructor	Simple, runs constructor logic, works with final fields	Must be written per class
Static factory (copyOf)	Same benefits, can return an interface type or cache instances	Same as above

Interview takeaway: if asked “how do you copy an object in Java,” the strong answer is “prefer a copy constructor or static factory; avoid Cloneable because it’s fundamentally broken.”

Optional

`java.util.Optional<T>` is a container that either holds a value or is empty. It was added in Java 8 specifically to make **absence of a value explicit in a method’s return type**, instead of relying on `null` and hoping callers remember to check.

```
import java.util.Optional;

Optional<String> findUserEmail(long userId) {
    User user = repository.findById(userId); // may not exist
    return user == null ? Optional.empty() : Optional.of(user.getEmail());
}
```

Callers are nudged (though not forced) to handle the empty case instead of risking a `NullPointerException`:

```
Optional<String> email = findUserEmail(42);

String display = email.orElse("no email on file");
```

map, flatMap, filter

`Optional` supports a small functional pipeline, similar to `Stream`:

```
Optional<String> email = findUserEmail(42);

// map: transform the value if present, otherwise stay empty
Optional<Integer> domainLength = email.map(e -> e.split("@")[1].length());

// filter: keep the value only if it matches a predicate
Optional<String> corporateEmail = email.filter(e ->
↳ e.endsWith("@acme.com"));

// flatMap: like map, but the function itself returns an Optional (avoids
↳ Optional<Optional<T>>)
Optional<User> findUserByEmail(String email) { /* ... */ return
↳ Optional.empty(); }

Optional<User> user = email.flatMap(this::findUserByEmail);
```

```
// Without flatMap, map would produce a nested Optional<Optional<User>> -
↳ awkward:
Optional<Optional<User>> nested = email.map(this::findUserByEmail); //
↳ wrong shape
```

orElse vs orElseGet vs orElseThrow

These three “unwrap” an `Optional`, but they differ in when their argument runs — a frequent code-review gotcha:

Method	Argument	When the argument runs
<code>orElse(T value)</code>	A plain value	Always evaluated , even if the <code>Optional</code> is present
<code>orElseGet(Supplier<T>)</code>	A lazy supplier	Only called if the <code>Optional</code> is empty
<code>orElseThrow(Supplier<X>)</code>	An exception supplier	Only called (and thrown) if the <code>Optional</code> is empty

```
// BROKEN-ish: orElse() always builds the fallback, even when not needed.
String email = findUserEmail(42).orElse(buildExpensiveDefault()); //
↳ buildExpensiveDefault() ALWAYS runs

// FIXED: orElseGet() only calls the supplier when the Optional is empty.
```

```
String email2 = findUserEmail(42).orElseGet(this::buildExpensiveDefault);
```

For a simple constant like "unknown", `orElse` is fine and slightly clearer. The rule: **use `orElse` for cheap constants, `orElseGet` for anything expensive or side-effecting.**

```
User user = repository.findById(42)
    .orElseThrow(() -> new NoSuchElementException("User 42 not
    ↪ found"));
```

`orElseThrow()` with no argument throws `NoSuchElementException`; the overload with a `Supplier<X>` lets you throw a domain-specific exception.

ifPresent, ifPresentOrElse, and Optional in a switch/pattern context

```
findUserEmail(42).ifPresent(email -> System.out.println("Found: " +
    ↪ email));

findUserEmail(42).ifPresentOrElse(
    email -> System.out.println("Found: " + email),
    () -> System.out.println("No email found")
);
```

What Optional is NOT for

This is the part reviewers check most: `Optional` was designed for **return types**, not for fields, parameters, or collection elements.

```
// BAD: Optional as a field – adds boxing overhead, complicates
↪ serialization,
// and Optional itself is not Serializable.
class User {
    private Optional<String> nickname; // avoid
}

// BAD: Optional as a method parameter – forces every caller to wrap
↪ values,
// and null is still possible anyway (Optional can itself be null!).
void updateNickname(Optional<String> nickname) { /* avoid */ }

// BAD: Optional inside a collection – just omit the entry, or use an
↪ empty value.
List<Optional<String>> names; // avoid
```

Better alternatives:

```
// GOOD: plain nullable field, document with @Nullable or just a comment.
class User {
    private String nickname; // may be null
}

// GOOD: overload instead of Optional parameter.
void updateNickname(String nickname) { /* ... */ }

// GOOD: absence in a collection is just "not in the list" or an empty
↪ String.
List<String> names; // simply don't add a null/absent entry
```

Interview one-liner: “Optional is for values a method *returns* that might not exist — never for fields, parameters, or collections. It’s a return-type tool, not a general null-replacement.”

Comparator and Comparable

Java has two interfaces for ordering objects, and mixing them up is a common review comment.

Interface	Method	Where the logic lives	How many orderings?
Comparable<T>	compareTo(T o)	Inside the class itself	One “natural order”
Comparator<T>	compare(T a, T b)	External, standalone	As many as you want

The compareTo contract

compareTo must return a negative number, zero, or a positive number, meaning “less than,” “equal to,” or “greater than” respectively. Like equals(), it has a formal contract:

- **Sign consistency:** `x.compareTo(y)` and `y.compareTo(x)` must have opposite signs (or both be zero).
- **Transitivity:** if `x.compareTo(y) > 0` and `y.compareTo(z) > 0`, then `x.compareTo(z) > 0`.

- **Consistency with equals (strongly recommended, not strictly required):** `x.compareTo(y) == 0` should imply `x.equals(y)`. If not, document it — `TreeSet/TreeMap` use `compareTo` for equality, *not* `equals()`, so violating this causes confusing duplicate-removal behavior.

```
final class Money implements Comparable<Money> {
    private final long cents;
    Money(long cents) { this.cents = cents; }

    @Override
    public int compareTo(Money other) {
        return Long.compare(this.cents, other.cents); // safe, no overflow
    }
}
```

The integer-subtraction overflow bug

A very common — and very wrong — shortcut is subtracting two numbers to get the comparison result:

```
// BROKEN: integer overflow can flip the sign!
class Score implements Comparable<Score> {
    private final int value;
    Score(int value) { this.value = value; }

    @Override
    public int compareTo(Score other) {
        return this.value - other.value; // overflow risk
    }
}

Score a = new Score(Integer.MAX_VALUE); // 2147483647
Score b = new Score(-1);
System.out.println(a.compareTo(b)); // expected positive, but overflows to
→ negative!
```

Because `Integer.MAX_VALUE - (-1)` overflows past `Integer.MAX_VALUE` and wraps around to a negative number, `compareTo` reports `a < b` when actually `a > b`. This breaks sorting and binary search silently, often only on large or negative inputs — exactly the kind of bug that slips past small tests.

```
// FIXED: use the boxed type's static compare method – never overflows.
@Override
public int compareTo(Score other) {
    return Integer.compare(this.value, other.value);
}
```

This applies to `Integer.compare`, `Long.compare`, `Double.compare`, etc. **Rule for reviewers: any `compareTo/compare` that uses `a - b` on `int/long` fields should be flagged.**

Comparator.comparing, thenComparing, reversed

`Comparator` has static and default methods (since Java 8) that make building complex orderings declarative instead of hand-written `if/else` chains:

```
record Employee(String name, String department, int salary) {}

List<Employee> employees = new ArrayList<>(List.of(
    new Employee("Alice", "Engineering", 90000),
    new Employee("Bob", "Engineering", 95000),
    new Employee("Carol", "Sales", 80000)
));

Comparator<Employee> byDeptThenSalaryDesc =
    Comparator.comparing(Employee::department)
    → .thenComparing(Comparator.comparing(Employee::salary).reversed());

employees.sort(byDeptThenSalaryDesc);
// Engineering: Bob (95000), Alice (90000); then Sales: Carol (80000)
```

Read it left to right: sort by department first; for ties, sort by salary, descending.

```
// Manual equivalent – verbose and error-prone by comparison:
Comparator<Employee> manual = (e1, e2) -> {
    int deptCompare = e1.department().compareTo(e2.department());
    if (deptCompare != 0) return deptCompare;
    return Integer.compare(e2.salary(), e1.salary()); // reversed manually
};
```

naturalOrder and nullsFirst / nullsLast

```
List<String> names = new ArrayList<>(List.of("Bob", "alice", "Carol"));
names.sort(Comparator.naturalOrder()); // uses String's own compareTo
↳ (case-sensitive)

// Null-safe sorting: nulls can crash a naive comparator with NPE.
List<String> withNulls = new ArrayList<>(List.of("Bob", null, "Alice"));

// BROKEN: NullPointerException on the null element.
// withNulls.sort(Comparator.naturalOrder());

// FIXED: push nulls to the front (or use nullsLast to push them to the
↳ end).
withNulls.sort(Comparator.nullsFirst(Comparator.naturalOrder()));
```

Comparable vs Comparator — when to use which

```
// Comparable: ONE natural ordering, baked into the class.
class Invoice implements Comparable<Invoice> {
    private final LocalDate dueDate;
    @Override
    public int compareTo(Invoice other) {
        return this.dueDate.compareTo(other.dueDate); // "natural" order =
↳ due date
    }
}

// Comparator: as many EXTRA orderings as you need, defined externally.
Comparator<Invoice> byAmountDesc =
↳ Comparator.comparing(Invoice::getAmount).reversed();
Comparator<Invoice> byCustomerName =
↳ Comparator.comparing(Invoice::getCustomerName);
```

Interview one-liner: “Comparable defines a type’s single natural order and lives inside the class; Comparator defines external, swappable orderings, and you can have as many as you like.”

Immutability

An **immutable** object is one whose observable state never changes after construction. Once built, it stays exactly as it is — no setters, no mutation, ever. Immutable objects

are automatically thread-safe (no synchronization needed), easier to reason about, and safe to share and cache. `String`, `Integer`, `LocalDate`, and records (when built carefully) are all immutable or close to it.

Recipe for a truly immutable class

1. Make the class `final` (or use only private constructors) so it cannot be subclassed to add mutable state or override behavior.
2. Make every field `private` and `final`.
3. Do not provide setters or any other mutator methods.
4. If a field is a mutable type (array, `List`, `Date`, mutable custom object), make a **defensive copy** on the way in (constructor) and on the way out (getter). Covered in detail in the next section.

```
import java.util.List;

final class ImmutableInvoice {
    private final String id;
    private final List<String> items;
    private final int totalCents;

    ImmutableInvoice(String id, List<String> items, int totalCents) {
        this.id = id;
        this.items = List.copyOf(items); // defensive copy + unmodifiable
        this.totalCents = totalCents;
    }

    String getId() { return id; }
    int getTotalCents() { return totalCents; }
    List<String> getItems() { return items; } // safe: List.copyOf is
↳ already immutable
}
```

```
// Usage: attempts to mutate the exposed list fail loudly, not silently.
ImmutableInvoice invoice = new ImmutableInvoice("INV-1",
↳ List.of("Widget"), 999);
invoice.getItems().add("Sneaky item"); // throws
↳ UnsupportedOperationException
```


Records and immutability

record gives you private `final` fields and no setters automatically — but it does **not** automatically give you deep immutability. If a record component is a mutable type, you still need to defend it yourself:

```
// Looks immutable, but ISN'T fully:
record Invoice(String id, List<String> items) {}

List<String> mutable = new ArrayList<>(List.of("Widget"));
Invoice invoice = new Invoice("INV-1", mutable);
mutable.add("Sneaky item"); // mutates the record's internal state from
↪ outside!
System.out.println(invoice.items()); // [Widget, Sneaky item]
```

```
// FIXED: use a compact constructor to defensively copy.
record Invoice(String id, List<String> items) {
    Invoice(String id, List<String> items) {
        this.id = id;
        this.items = List.copyOf(items); // defends against caller
    }
}
↪ mutation
```

Immutability and performance / thread-safety trade-offs

Benefit	Cost
Naturally thread-safe, no locks needed	Every “change” allocates a new object
Safe to cache, share, use as map keys	Can be wasteful for large objects mutated frequently in a loop
Simpler to reason about, fewer invariant bugs	Requires discipline (defensive copies) for every mutable field

For a value that changes frequently inside a hot loop (e.g., a running total), a mutable local variable or builder is usually fine — immutability shines for objects that are shared across threads or passed around as stable values (DTOs, config, keys, events).

Defensive Copying

Defensive copying means making an independent copy of a mutable object at a trust boundary, so that neither side can affect the other through shared references. Without it, “immutable” classes can be mutated indirectly by whoever handed you the object — or whoever you handed the object to.

The two places you need it: constructor AND getter

A very common code-review finding is defending only *one* side (usually the constructor) and forgetting the other (the getter).

```
import java.util.ArrayList;
import java.util.Date;
import java.util.List;

// BROKEN: no defensive copies anywhere.
final class Trip {
    private final Date startDate; // Date is mutable!
    private final List<String> stops;

    Trip(Date startDate, List<String> stops) {
        this.startDate = startDate; // caller's Date object is stored
        ↪ directly
        this.stops = stops; // caller's List is stored directly
    }

    Date getStartDate() { return startDate; } // returns the SAME mutable
    ↪ Date
    List<String> getStops() { return stops; } // returns the SAME mutable
    ↪ List
}

Date d = new Date();
List<String> stops = new ArrayList<>(List.of("Paris"));
Trip trip = new Trip(d, stops);

d.setTime(0); // mutates the Trip's startDate from OUTSIDE
stops.add("Berlin"); // mutates the Trip's stops from OUTSIDE

trip.getStartDate().setTime(123456789L); // mutates the Trip's internal
↪ Date via the GETTER
```

```
// FIXED: defend both directions.
final class Trip {
    private final Date startDate;
    private final List<String> stops;

    Trip(Date startDate, List<String> stops) {
        this.startDate = new Date(startDate.getTime()); // copy IN
        this.stops = new ArrayList<>(stops);             // copy IN
    }

    Date getStartDate() {
        return new Date(startDate.getTime()); // copy OUT
    }

    List<String> getStops() {
        return List.copyOf(stops); // copy OUT, and unmodifiable too
    }
}
```

Now, mutating the caller's original `Date/List` after construction — or mutating whatever the getter returns — has zero effect on the `Trip`'s internal state.

List.copyOf vs unmodifiable views vs true immutable copies

These three look similar but behave very differently, and mixing them up is a frequent bug:

Technique	Independent from source?	Can it be mutated directly?
<code>Collections.unmodifiableList(list)</code>	No — it's a view ; changes to the original list still show through	No (throws on direct mutation attempts)
<code>List.copyOf(list)</code>	Yes — makes an actual independent copy	No (immutable)
<code>new ArrayList<>(list)</code>	Yes — independent copy	Yes (still mutable)

```
List<String> source = new ArrayList<>(List.of("A", "B"));

List<String> view = Collections.unmodifiableList(source);
source.add("C");
```

```
System.out.println(view); // [A, B, C] – the view is NOT independent, it
↳ just blocks writes
```

```
List<String> copy = List.copyOf(source);
source.add("D");
System.out.println(copy); // [A, B, C] – the copy IS independent
```

```
// BROKEN: developer thinks unmodifiableList() protects against upstream
↳ mutation.
class Roster {
    private final List<String> names;
    Roster(List<String> names) {
        this.names = Collections.unmodifiableList(names); // still a VIEW
↳ over the caller's list!
    }
}
```

```
List<String> mutable = new ArrayList<>(List.of("Alice"));
Roster roster = new Roster(mutable);
mutable.add("Mallory"); // Roster's "immutable" view now includes
↳ "Mallory" too!
```

```
// FIXED: copy first, THEN wrap (or just use List.copyOf, which does
↳ both).
class Roster {
    private final List<String> names;
    Roster(List<String> names) {
        this.names = List.copyOf(names); // independent AND unmodifiable
    }
}
```

Rule of thumb: `unmodifiableXxx()` prevents *your* code (or the caller of your getter) from mutating a collection through that particular reference, but it does nothing to stop the *original owner* of the backing collection from mutating it. Use it for a getter when you already own an independent copy; use `copyOf` (or a manual copy) when you need true independence.

Arrays are always a leak vector

Arrays are mutable in Java, and unlike `List`, there is no built-in “unmodifiable array” wrapper. Any method that returns or accepts an array is a defensive-copying hotspot.

```

// BROKEN: exposes the internal array directly.
class PasswordPolicy {
    private final char[] allowedSymbols;
    PasswordPolicy(char[] allowedSymbols) {
        this.allowedSymbols = allowedSymbols; // stores caller's array
        ↪ directly
    }
    char[] getAllowedSymbols() { return allowedSymbols; } // returns
        ↪ internal array directly
}

char[] symbols = {'!', '@', '#'};
PasswordPolicy policy = new PasswordPolicy(symbols);
policy.getAllowedSymbols()[0] = 'X'; // mutates internal state via the
        ↪ returned array!
symbols[1] = 'Y'; // mutates internal state via the
        ↪ original reference!

```

```

// FIXED: clone() on the way in AND on the way out.
class PasswordPolicy {
    private final char[] allowedSymbols;
    PasswordPolicy(char[] allowedSymbols) {
        this.allowedSymbols = allowedSymbols.clone(); // array clone() is
        ↪ a fast shallow copy
    }
    char[] getAllowedSymbols() {
        return allowedSymbols.clone();
    }
}

```

(Note: `array.clone()` is one of the few reasonable uses of `clone()` in Java — arrays have well-defined, non-broken clone semantics, unlike ordinary objects.)

Dates are a classic leak vector too

`java.util.Date` is mutable (it has `setTime()`), which is one of the reasons the legacy date API is considered a design mistake. The modern `java.time` types (`LocalDate`, `Instant`, `LocalDateTime`, etc.) are all immutable, so **switching to `java.time` eliminates this entire category of bug** — no defensive copy is needed because there is nothing to mutate.

```
import java.time.LocalDate;

// GOOD: no defensive copying needed, because LocalDate is immutable.
final class Membership {
    private final LocalDate startDate;
    Membership(LocalDate startDate) {
        this.startDate = startDate; // safe: LocalDate can't be mutated by
        ↪ anyone
    }
    LocalDate getStartDate() {
        return startDate; // safe: caller can't mutate it either
    }
}
```

Interview one-liner: “Defensive copying is only necessary for mutable types. The best fix is often to avoid mutable types altogether — prefer `java.time` over `Date`, and `List`. `copyOf`/records over hand-rolled mutable collections.”

Common Code-Review Interview Pitfalls

1. **Overriding `equals()` without `hashCode()`.** Why it matters: breaks the `equals`/`hashCode` contract; the object silently “disappears” from `HashMap`/`HashSet` lookups.

```
// Before: equals() overridden, hashCode() left default
↪ (identity-based)
@Override public boolean equals(Object o) { return o instanceof Id i
↪ && i.value == value; }

// After: keep them in sync
@Override public int hashCode() { return Integer.hashCode(value); }
```

2. **Using a mutable field as a `HashMap`/`HashSet` key’s basis for `hashCode()`.** Why it matters: mutating the key after insertion makes it unfindable — a silent data-loss bug.

```
// Before: key.setStatus("DONE") after map.put(key, ...) ->
↪ map.get(key) returns null
// After: use an immutable id (String/UUID/record) as the key instead
↪ of the mutable entity
```

3. **instanceof-based equals() in a class hierarchy where subclasses add fields.** Why it matters: breaks symmetry (`a.equals(b) != b.equals(a)`), causing inconsistent behavior in collections.

```
// Before: if (!(o instanceof Point p)) ... // ColorPoint subclass
↳ breaks symmetry
// After: if (o == null || getClass() != o.getClass()) return false; //
↳ or make the class final
```

4. **Integer subtraction inside compareTo/compare.** Why it matters: overflows silently for large or negative values, corrupting sort order.

```
// Before: return this.value - other.value;
// After: return Integer.compare(this.value, other.value);
```

5. **Using Cloneable/Object.clone() for copying.** Why it matters: bypasses constructors (invariants unchecked), fragile with `final` fields and subclassing, throws a checked exception unnecessarily.

```
// Before: class Team implements Cloneable { ... super.clone() ... }
// After: Team(Team other) { this(other.name, other.members); } //
↳ copy constructor
```

6. **Shallow-copying an object that has mutable reference fields.** Why it matters: the “copy” secretly shares mutable state with the original — mutating one affects the other.

```
// Before: Team copy = (Team) super.clone(); // members list is SHARED
// After: copy.members = new ArrayList<>(this.members); // deep-ish
↳ copy
```

7. **Optional used as a field, parameter, or inside a collection.** Why it matters: adds overhead, is not `Serializable`, and `Optional` itself can be `null` — defeating its purpose.

```
// Before: void updateNickname(Optional<String> nickname)
// After: void updateNickname(String nickname) // nullable, documented
```

8. **Calling `orElse(expensiveCall())` instead of `orElseGet(...)`.** Why it matters: `orElse`’s argument is always evaluated eagerly, even when the `Optional` is present — wasted work or unwanted side effects.

```
// Before: opt.orElse(buildExpensiveDefault());
// After:  opt.orElseGet(this::buildExpensiveDefault);
```

- 9. Exposing an internal mutable collection or array directly from a getter.** Why it matters: callers can mutate “encapsulated” internal state from outside the class, breaking invariants silently.

```
// Before: List<String> getItems() { return items; }
// After:  List<String> getItems() { return List.copyOf(items); }
```

- 10. Storing a caller-provided mutable object directly in the constructor without copying.** Why it matters: the caller can mutate your object’s “immutable” internal state after construction, from their original reference.

```
// Before: this.startDate = startDate; // Date is mutable
// After:  this.startDate = new Date(startDate.getTime()); // defensive
↳ copy in
```

- 11. Confusing `Collections.unmodifiableList()` (a view) with a true immutable copy.** Why it matters: an unmodifiable *view* still reflects changes made to the original backing collection — it does not provide independence.

```
// Before: this.names = Collections.unmodifiableList(names); // still a
↳ view over caller's list
// After:  this.names = List.copyOf(names); // independent AND
↳ unmodifiable
```

- 12. `compareTo` that is inconsistent with `equals()`, used with `TreeSet/TreeMap`.** Why it matters: `TreeSet/TreeMap` use `compareTo` (not `equals()`) to decide duplicates, so “equal” elements by `equals()` can both end up in the set if `compareTo` disagrees — or vice versa, distinct elements silently vanish as “duplicates.”

```
// Before: compareTo compares only `lastName`, but equals() compares
↳ full name -> surprises in TreeSet
// After:  make compareTo and equals() agree, or clearly document the
↳ difference
```

- 13. Sorting a list that may contain null with `Comparator.naturalOrder()` directly.** Why it matters: throws `NullPointerException` at runtime instead of handling nulls predictably.


```
// Before: list.sort(Comparator.naturalOrder());  
// After:  list.sort(Comparator.nullsFirst(Comparator.naturalOrder()));
```

- 14. Forgetting to regenerate IDE/Lombok-generated equals()/hashCode() after adding a field.** Why it matters: the new field is silently excluded from equality checks, causing subtly wrong deduplication or map behavior.

```
// Before: `status` field added, but equals()/hashCode() still only  
↳ reference `id`  
// After:  regenerate, or intentionally document ID-only equality  
↳ semantics
```

- 15. Hand-writing a data class (fields, constructor, getters, equals/hashCode/toString) that could be a record.** Why it matters: more boilerplate to maintain, more surface area for the bugs above (missing hashCode, stale generated code); records get correct semantics for free.

```
// Before: class Point { private final int x, y; /* ~30 lines of  
↳ boilerplate */ }  
// After:  record Point(int x, int y) {}
```


Exception Handling

Things go wrong at runtime: files are missing, networks time out, arrays get the wrong index. Java uses **exceptions** to signal that something went wrong, separate from your normal return values. This chapter explains how exceptions work under the hood, how to handle them correctly, and the mistakes that reviewers flag most often in real code review. We target Java 21+ throughout.

Exceptions

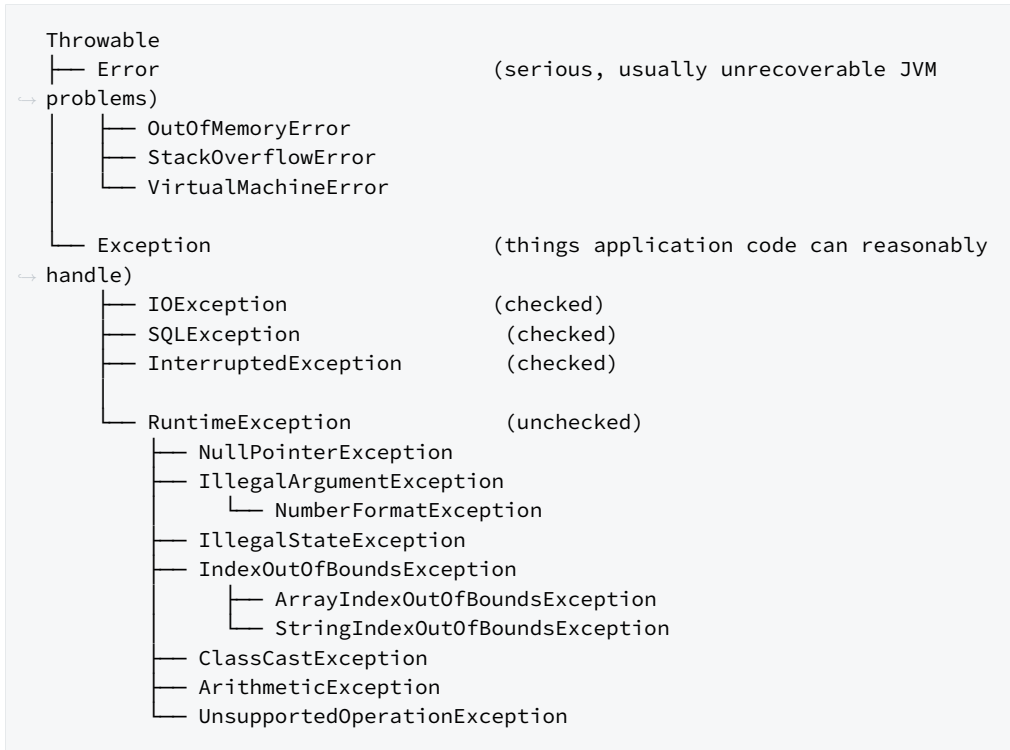
An **exception** is an object that describes an unexpected event. When something goes wrong, Java **throws** an exception object. If nothing handles it, the program (or thread) stops and prints a **stack trace**. If code **catches** it, the program can recover or fail gracefully instead of crashing.

```
public class Divider {
    public static int divide(int a, int b) {
        return a / b; // throws ArithmeticException if b == 0
    }

    public static void main(String[] args) {
        try {
            System.out.println(divide(10, 0));
        } catch (ArithmeticException e) {
            System.out.println("Cannot divide by zero: " +
        ↪ e.getMessage());
        }
    }
}
```

The Throwable Hierarchy

Every exception in Java is a subclass of `Throwable`. There are three important branches: `Error`, `Exception`, and inside `Exception`, `RuntimeException`.



- `Error` — things like `OutOfMemoryError` or `StackOverflowError`. These signal that the JVM itself is in trouble. Application code almost never catches these, because there is usually nothing useful you can do.
- `Exception` — problems your application should anticipate and possibly recover from, such as a missing file.
- `RuntimeException` — a subclass of `Exception` for programming errors, like calling a method on `null`. These are **unchecked** (explained below).

`Throwable` itself is rarely used directly. Catching `Throwable` is even broader than catching `Exception` — it also captures `Error`, which is almost never what you want.

```
// Avoid: this also swallows OutOfMemoryError and StackOverflowError
try {
    riskyOperation();
} catch (Throwable t) {
    log.error("Something went wrong", t);
}
```

Stack Traces and How to Read Them

A **stack trace** is a printed list of method calls that were active when the exception was created. It reads top to bottom: the top line is where the exception happened, and each line below is “called from here.”

```
public class Chain {
    public static void main(String[] args) {
        step1();
    }
    static void step1() { step2(); }
    static void step2() { step3(); }
    static void step3() { throw new IllegalStateException("boom"); }
}
```

Output:

```
Exception in thread "main" java.lang.IllegalStateException: boom
    at Chain.step3(Chain.java:7)
    at Chain.step2(Chain.java:6)
    at Chain.step1(Chain.java:5)
    at Chain.main(Chain.java:2)
```

How to read it:

1. **First line** — exception class and message. This tells you *what* went wrong.
2. **at lines** — the call stack at the moment of the throw, most recent call first. `Chain.step3(Chain.java:7)` means line 7 of `Chain.java`, inside method `step3`.
3. **Caused by** — if present, a chained exception (see below). Always read the *innermost* “Caused by” first; that is usually the real root cause.
4. **... N more** — Java collapses stack frames that are identical to a previous trace, to save space, when printing a chained exception.

```
try {
    loadConfig();
} catch (IOException e) {
    throw new RuntimeException("Failed to start application", e);
}
```

```
Exception in thread "main" java.lang.RuntimeException: Failed to start
↳ application
    at App.main(App.java:12)
    Caused by: java.io.FileNotFoundException: config.yml (No such file or
↳ directory)
        at java.base/java.io.FileInputStream.open0(Native Method)
        ... 5 more
```

Checked vs Unchecked Exceptions

Java splits exceptions into two categories based on whether the compiler forces you to deal with them.

	Checked	Unchecked
Base class	Exception (not RuntimeException)	RuntimeException or Error
Compiler enforcement	Must be caught or declared with throws	No compiler enforcement
Typical cause	External conditions (file missing, network down)	Programming bugs (null access, bad argument)
Examples	IOException, SQLException, InterruptedException	NullPointerException, IllegalArgumentException, IndexOutOfBoundsException
Should caller recover?	Often yes — retry, fallback, ask user again	Usually no — fix the bug instead

```
import java.io.*;

public class CheckedExample {
    // Checked exception: caller MUST catch it or declare "throws
↳ IOException"
    public static String readFirstLine(String path) throws IOException {
```

```

        try (BufferedReader reader = new BufferedReader(new
↪ FileReader(path))) {
            return reader.readLine();
        }
    }
}

```

```

public class UncheckedExample {
    // Unchecked: compiler does not force the caller to do anything
    public static int parsePositive(String value) {
        int n = Integer.parseInt(value);           // may throw
↪ NumberFormatException
        if (n < 0) {
            throw new IllegalArgumentException("Value must be positive: "
↪ + n);
        }
        return n;
    }
}

```

When to Use Which

Situation	Use
Caller can realistically recover (retry, prompt again, use a default)	Checked exception
The failure is a bug in the calling code (bad argument, wrong state)	Unchecked exception
Library/API boundary where you want to force callers to think about failure	Checked exception
Internal code where wrapping every call in try/catch would just add noise	Unchecked exception

Modern Java style (post Java 8, especially with streams and lambdas) leans toward **unchecked** exceptions, because checked exceptions do not compose well with functional interfaces (a `Function<T, R>` cannot declare `throws IOException`). This is why many modern libraries (e.g., `UncheckedIOException`) provide unchecked wrappers around checked ones.

```

import java.io.UncheckedIOException;
import java.io.IOException;
import java.nio.file.*;
import java.util.List;
import java.util.stream.Stream;

public class StreamsAndCheckedExceptions {
    public static List<String> linesOf(Path path) {
        try (Stream<String> lines = Files.lines(path)) {
            return lines.toList();
        } catch (IOException e) {
            // Files.lines already throws unchecked UncheckedIOException
            ↪ lazily during iteration,
            // but the try-with-resources open call itself is still
            ↪ checked, so we wrap it too.
            throw new UncheckedIOException(e);
        }
    }
}

```

Exception Handling Best Practices

Handling exceptions well is more about discipline than syntax. This section covers the patterns reviewers look for.

finally Semantics

Code in a `finally` block always runs, whether the `try` succeeded, threw, or returned. It is the right place for cleanup that must happen no matter what.

```

public class FinallyDemo {
    static int compute() {
        try {
            System.out.println("try");
            throw new RuntimeException("fail");
        } finally {
            System.out.println("finally always runs");
        }
    }
}

```


How return in finally Swallows Exceptions

This is a classic trap: if finally itself contains a return (or a throw), it **overrides** whatever the try or catch block was about to do — including silently discarding an in-flight exception.

```
// BROKEN: the exception disappears without a trace
public class SwallowedException {
    static int risky() {
        try {
            throw new IllegalStateException("real problem");
        } finally {
            return 42; // this return wins; the exception above is
↪ discarded
        }
    }

    public static void main(String[] args) {
        System.out.println(risky()); // prints 42, exception is gone
↪ forever
    }
}
```

```
// FIXED: never put a return (or throw) inside finally
public class NoSwallowedException {
    static int risky() {
        try {
            throw new IllegalStateException("real problem");
        } finally {
            System.out.println("cleanup only, no return here");
        }
    }
}
```

Rule of thumb: finally blocks should only do cleanup (closing resources, releasing locks, logging). They should never return, break, continue, or throw.

Multi-Catch

When several exception types need the same handling, catch can list them with `|` instead of duplicating the block.

```

public class MultiCatchDemo {
    static void parse(String input) {
        try {
            int value = Integer.parseInt(input);
            System.out.println(100 / value);
        } catch (NumberFormatException | ArithmeticException e) {
            System.out.println("Invalid input or division error: " +
↪ e.getMessage());
        }
    }
}

```

Note: the caught variable (*e* here) is effectively `final`, and its static type is the least upper bound of the listed types — you cannot reassign it, and you cannot catch a type and its subtype together in the same multi-catch (the compiler will reject that as redundant).

Rethrow with Precise Type Inference

Since Java 7, if a catch block only rethrows the caught exception (without reassigning it to a broader type), the compiler tracks the **precise** type that was actually thrown, not just the declared catch type. This lets a method declare `throws IOException, SQLException` instead of the broader `throws Exception`.

```

public class PreciseRethrow {
    static void doWork(boolean useDb) throws IOException, SQLException {
        try {
            if (useDb) {
                throw new SQLException("db error");
            } else {
                throw new IOException("io error");
            }
        } catch (Exception e) {
            // e is caught as Exception, but the compiler knows only
            // IOException or SQLException could actually reach here,
            // so this rethrow is allowed without "throws Exception".
            throw e;
        }
    }
}

```

If you reassign *e* inside the catch block, the compiler loses this precision and falls back to the declared catch type.

Exception Chaining (Cause)

When you catch a low-level exception and throw a higher-level one, always pass the original as the **cause**. This preserves the full stack trace instead of hiding the real reason.

```
// BROKEN: original cause is lost
public class LostCause {
    static void loadUser(String id) {
        try {
            fetchFromDatabase(id);
        } catch (SQLException e) {
            throw new RuntimeException("Could not load user " + id); //
            ↪ cause dropped!
        }
    }
    static void fetchFromDatabase(String id) throws SQLException {
        throw new SQLException("connection refused");
    }
}
```

```
// FIXED: chain the cause
public class PreservedCause {
    static void loadUser(String id) {
        try {
            fetchFromDatabase(id);
        } catch (SQLException e) {
            throw new RuntimeException("Could not load user " + id, e); //
            ↪ cause preserved
        }
    }
    static void fetchFromDatabase(String id) throws SQLException {
        throw new SQLException("connection refused");
    }
}
```

You can also inspect the cause chain programmatically with `getCause()`, and Java 9+ adds `Throwable.getSuppressed()` for suppressed exceptions (covered later).

```
try {
    PreservedCause.loadUser("42");
} catch (RuntimeException e) {
    Throwable cause = e.getCause();
    System.out.println("Root cause: " + cause);
}
```

Custom Exception Design

Design custom exceptions when the standard library types don't communicate enough domain meaning. Keep them simple, immutable, and always provide constructors that support chaining.

```
public class InsufficientFundsException extends Exception {
    private final BigDecimal shortfall;

    public InsufficientFundsException(String message, BigDecimal
↳ shortfall) {
        super(message);
        this.shortfall = shortfall;
    }

    public InsufficientFundsException(String message, BigDecimal
↳ shortfall, Throwable cause) {
        super(message, cause);
        this.shortfall = shortfall;
    }

    public BigDecimal getShortfall() {
        return shortfall;
    }
}
```

Guidelines for custom exceptions:

- Extend `Exception` for checked, recoverable business errors (e.g., `InsufficientFundsException`); extend `RuntimeException` for programming errors or when using functional/stream code.
- Always provide a `(String message, Throwable cause)` constructor for chaining.
- Add extra fields (like `shortfall` above) only when calling code actually needs them programmatically — don't just stuff everything into the message string.
- Avoid deep exception hierarchies. One or two custom types per module is usually enough.
- Don't make exceptions mutable; they should describe a moment in time.

NullPointerException Helpful Messages (JDK 14+)

Since JDK 14 (stabilized as a default from JDK 15), the JVM can generate a **helpful NPE message** that names exactly which variable or method call was null, instead of just a bare stack trace line.

```
public class HelpfulNpeDemo {
    record Address(String city) {}
    record Person(Address address) {}

    public static void main(String[] args) {
        Person person = new Person(null);
        System.out.println(person.address().city()); // NPE here
    }
}
```

Without the feature you'd just see `NullPointerException` with no detail. With helpful NPE messages enabled (default since Java 15), you get:

```
Exception in thread "main" java.lang.NullPointerException:
    Cannot invoke "HelpfulNpeDemo$Address.city()" because the return value
    ↪ of
        "HelpfulNpeDemo$Person.address()" is null
        at HelpfulNpeDemo.main(HelpfulNpeDemo.java:8)
```

This tells you precisely which call returned `null`, which is invaluable when a line has several chained calls. If you're on an older JVM or it's disabled, enable it with `-XX:+ShowCodeDetailsInExceptionMessages` (unnecessary on modern JDKs, where it's on by default).

Suppressed Exceptions

When a try-with-resources block throws an exception in the body *and* closing a resource also throws, Java doesn't discard either one. The body's exception is thrown as the primary exception, and the close-time exception is attached to it as a **suppressed exception**.

```
public class SuppressedDemo {
    static class Resource implements AutoCloseable {
        public void use() {
            throw new RuntimeException("failure during use");
        }
    }
}
```

```

        @Override
        public void close() {
            throw new RuntimeException("failure during close");
        }
    }

    public static void main(String[] args) {
        try (Resource r = new Resource()) {
            r.use();
        } catch (RuntimeException e) {
            System.out.println("Primary: " + e.getMessage());
            for (Throwable suppressed : e.getSuppressed()) {
                System.out.println("Suppressed: " +
↪ suppressed.getMessage());
            }
        }
    }
}

```

Output:

```

Primary: failure during use
Suppressed: failure during close

```

Without try-with-resources, if you manually close a resource inside `finally`, the close-time exception would simply *replace* the original one — losing the real cause. Suppressed exceptions solve exactly that problem.

Try-with-Resources

try-with-resources (introduced in Java 7) automatically closes resources for you, even if an exception is thrown, without needing a manual `finally` block.

```

// BROKEN: manual close, easy to get wrong, and swallows exceptions from
↪ use()
public class ManualClose {
    static void copy(String from, String to) throws IOException {
        FileInputStream in = null;
        FileOutputStream out = null;
        try {
            in = new FileInputStream(from);
            out = new FileOutputStream(to);
            in.transferTo(out);
        } finally {

```

```

        if (out != null) out.close(); // if this throws, exception
        ↪ from in.close() below is lost
        if (in != null) in.close();
    }
}

```

```

// FIXED: try-with-resources, both resources closed automatically, in
↪ reverse order
public class TryWithResourcesDemo {
    static void copy(String from, String to) throws IOException {
        try (FileInputStream in = new FileInputStream(from);
             FileOutputStream out = new FileOutputStream(to)) {
            in.transferTo(out);
        }
    }
}

```

Desugaring: What the Compiler Actually Generates

The compiler rewrites the try-with-resources block into ordinary try/finally code with null-checks and suppressed-exception wiring. Roughly, copy above desugars to something like this:

```

static void copy(String from, String to) throws IOException {
    FileInputStream in = new FileInputStream(from);
    try {
        FileOutputStream out = new FileOutputStream(to);
        try {
            in.transferTo(out);
        } finally {
            if (out != null) {
                out.close(); // any exception here becomes suppressed if
                ↪ 'try' already threw
            }
        }
    } finally {
        if (in != null) {
            in.close();
        }
    }
}

```

Key detail: this is why any exception thrown while closing becomes a **suppressed** exception rather than replacing the original — the generated code catches the primary exception, tries to close, and calls `addSuppressed` on failure before rethrowing.

Resource Close Order

Resources declared in a try-with-resources header are closed in the **reverse** order of declaration — last declared, first closed. This mirrors how you'd unwind nested `finally` blocks by hand.

```
public class CloseOrderDemo {
    record Loud(String name) implements AutoCloseable {
        Loud open() { System.out.println("open " + name); return this; }
        @Override public void close() { System.out.println("close " +
↵ name); }
    }

    public static void main(String[] args) {
        try (Loud a = new Loud("A").open();
            Loud b = new Loud("B").open();
            Loud c = new Loud("C").open()) {
            System.out.println("body");
        }
    }
}
```

Output:

```
open A
open B
open C
body
close C
close B
close A
```

Effectively-Final Resources (Java 9+)

Since Java 9, you can use an already-declared effectively-final variable directly in the try-with-resources header, without redeclaring it.


```

public class ExistingResource {
    static void process() throws IOException {
        BufferedReader reader = new BufferedReader(new
    ↪ FileReader("data.txt"));
        try (reader) { // just reference it, no re-declaration needed
            System.out.println(reader.readLine());
        }
    }
}

```

AutoCloseable vs Closeable

	AutoCloseable	Closeable
Introduced	Java 7	Java 5 (retrofitted to extend AutoCloseable in Java 7)
close() signature	void close() throws Exception	void close() throws IOException
Exception type	Any Exception	Only IOException (narrower)
Idempotent close required?	Not guaranteed by contract	Yes — calling close() more than once must be a no-op
Typical use	Any resource: locks, DB connections, custom types	I/O-specific types (streams, readers, writers)
Can be used in try-with-resources?	Yes	Yes (it extends AutoCloseable)

```

public class LockResource implements AutoCloseable {
    private final java.util.concurrent.locks.Lock lock;

    public LockResource(java.util.concurrent.locks.Lock lock) {
        this.lock = lock;
        lock.lock();
    }

    @Override
    public void close() { // narrows the throws clause to nothing, which
    ↪ is fine
        lock.unlock();
    }
}

```

```
}
```

```
public static void withLock(java.util.concurrent.locks.Lock lock,  
↳ Runnable action) {  
    try (LockResource guard = new LockResource(lock)) {  
        action.run();  
    }  
}
```

Prefer `Closeable` for I/O-flavored classes so callers get the narrower, more specific `IOException` instead of a generic `Exception`. Use `AutoCloseable` for everything else (locks, custom pooled resources, transactions).

Assertions

An **assertion** is a statement that checks something you believe must always be true. If it's false, the JVM throws an `AssertionError`. Assertions are for catching *your own bugs* during development and testing — they are not a validation mechanism for public APIs.

```
public class AssertionDemo {  
    static int half(int even) {  
        assert even % 2 == 0 : "Expected an even number but got " + even;  
        return even / 2;  
    }  
}
```

The `-ea` Flag

Assertions are **disabled by default** at runtime for performance reasons. You must explicitly enable them with `-ea` (or `-enableassertions`) when running the JVM:

```
java -ea com.example.AssertionDemo
```

Because assertions can be silently switched off, code must never depend on them running. If `-ea` is off, the `assert` line is skipped entirely — no side effect happens at all.

Why Assertions Must Not Have Side Effects

If an assertion is disabled, its expression is **never evaluated**. Any side effect inside it simply won't happen in production, creating a bug that only shows up when assertions are off — which is the default.

```
import java.util.List;

public class SideEffectAssertion {
    static void process(List<String> items) {
        // BROKEN: removeDuplicates() only runs when -ea is enabled!
        assert removeDuplicates(items);
        System.out.println(items);
    }

    static boolean removeDuplicates(List<String> items) {
        // pretend this mutates 'items' and returns true
        return true;
    }
}
```

```
import java.util.List;

public class NoSideEffectAssertion {
    static void process(List<String> items) {
        removeDuplicates(items); // always runs, regardless of -ea
        assert !hasDuplicates(items) : "Duplicates remain after cleanup";
        // pure check, no side effect
        System.out.println(items);
    }

    static void removeDuplicates(List<String> items) { /* mutates items */ }
    static boolean hasDuplicates(List<String> items) { return false; }
}
```

Why Assertions Must Not Validate Public API Arguments

Because assertions can be disabled, they are the wrong tool for checking arguments on a **public** method — anyone calling your API from outside might run with assertions off, and your validation would silently vanish, letting bad input through.

```

// BROKEN: public API validation via assert – disappears when -ea is off
public class AccountBroken {
    private double balance;

    public void withdraw(double amount) {
        assert amount > 0 : "amount must be positive"; // NOT enforced by
↪ default!
        balance -= amount;
    }
}

```

```

// FIXED: public API validation should throw a real, unconditional
↪ exception
public class AccountFixed {
    private double balance;

    public void withdraw(double amount) {
        if (amount <= 0) {
            throw new IllegalArgumentException("amount must be positive:
↪ " + amount);
        }
        balance -= amount;
    }
}

```

Where Assertions Belong

Use case	Right tool
Checking a public method's arguments	<code>if (...) throw new IllegalArgumentException(...)</code>
Checking internal invariants only your own code can violate (e.g., "this private helper should never see a negative index here")	<code>assert</code>
Documenting an assumption for future maintainers, checked only during testing	<code>assert</code>
Guarding against <code>null</code> from an external caller	Explicit <code>null</code> check + exception (e.g., <code>Objects.requireNonNull()</code>)

Use case	Right tool
Verifying test expectations	Use a test framework's assertions (e.g., JUnit's <code>assertEquals</code>), not the <code>assert</code> keyword

```

public class InvariantExample {
    private int index(int size, int position) {
        int result = position % size;
        // Internal invariant: our own arithmetic guarantees this is
        ↪ always >= 0.
        // Safe to use assert here because it's checking OUR logic, not
        ↪ caller input.
        assert result >= 0 : "computed negative index: " + result;
        return result;
    }
}

```

Common Code-Review Interview Pitfalls

1. **Catching Exception (or Throwable) and doing nothing (catch (Exception e) {}).** Why it matters: it hides every failure, including bugs, making the system silently wrong instead of loudly failing. It's one of the fastest ways to fail a code review.

```

// Before
try { save(record); } catch (Exception e) {}
// After
try { save(record); } catch (IOException e) { throw new
↪ UncheckedIOException("Failed to save record", e); }

```

2. **return inside a finally block, silently discarding an exception.** Why it matters: the caller thinks the operation succeeded and never learns about the real failure.

```

// Before
finally { return result; }
// After
finally { cleanup(); } // no return/throw in finally

```

- 3. Losing the cause when wrapping exceptions.** Why it matters: without the cause, the stack trace stops at the wrapping point and the real root cause is unrecoverable in logs.

```
// Before
throw new RuntimeException("failed");
// After
throw new RuntimeException("failed", e);
```

- 4. Logging an exception and then rethrowing it (double reporting).** Why it matters: the same error gets logged multiple times up the call stack, flooding logs and confusing on-call engineers about how many times it actually happened. Log at the boundary where you handle it, or rethrow — never both at every layer.

```
// Before
catch (IOException e) { log.error("failed", e); throw e; }
// After
catch (IOException e) { throw new ServiceException("Upload failed",
↳ e); } // log once, at the top-level handler
```

- 5. Catching InterruptedException and swallowing it without restoring the interrupt flag.** Why it matters: Thread.sleep/wait/blocking calls clear the interrupt flag when they throw; if you don't restore it, the thread's owning framework (e.g., an executor) can no longer tell the thread was asked to stop, breaking graceful shutdown.

```
// Before
try { Thread.sleep(1000); } catch (InterruptedException e) { /* ignored
↳ */ }
// After
try { Thread.sleep(1000); } catch (InterruptedException e) {
↳ Thread.currentThread().interrupt(); }
```

- 6. Using exceptions for normal control flow.** Why it matters: exceptions capture a full stack trace and are expensive; using them to signal ordinary conditions (like “not found”) hurts performance and readability.

```
// Before
try { return map.get(key); } catch (NullPointerException e) { return
↳ defaultValue; }
// After
return map.getDefault(key, defaultValue);
```

- 7. Throwing generic exceptions (throw new Exception("error") or RuntimeException) instead of a specific, meaningful type.** Why it matters: callers can't distinguish failure modes to handle them differently, and generic catches become tempting, widening the blast radius of catch blocks.

```
// Before
throw new RuntimeException("user not found");
// After
throw new NoSuchElementException("No user with id " + id);
```

- 8. Validating public API arguments with assert instead of a real check.** Why it matters: assertions are off by default in production (-ea not set), so the validation silently disappears, letting invalid state propagate deep into the system before it fails somewhere confusing.

```
// Before
assert amount > 0;
// After
if (amount <= 0) throw new IllegalArgumentException("amount must be
↳ positive: " + amount);
```

- 9. Putting side effects inside an assert expression.** Why it matters: when assertions are disabled, the expression is never evaluated, so that side effect silently stops happening — a correctness bug that appears only in production.

```
// Before
assert list.remove(item);
// After
boolean removed = list.remove(item);
assert removed : "expected item to be present";
```

- 10. Not closing resources, or closing them manually instead of using try-with-resources.** Why it matters: manual finally-based closing is verbose and easy to get subtly wrong (e.g., an exception from close() masking the real exception, or a missing null-check causing a NullPointerException during cleanup).

```
// Before
FileInputStream in = new FileInputStream(path);
try { use(in); } finally { in.close(); }
// After
try (FileInputStream in = new FileInputStream(path)) { use(in); }
```

- 11. Throwing checked exceptions from lambdas passed to standard functional interfaces, causing awkward wrapping or compile errors.** Why it matters: `Function`, `Consumer`, etc. don't declare `throws`, so checked exceptions inside lambdas must be caught locally or wrapped — forgetting this leads to compile errors or ugly ad hoc wrapping scattered across the codebase.

```
// Before (does not compile: readAllBytes throws IOException)
paths.stream().map(p -> Files.readAllBytes(p));
// After
paths.stream().map(p -> {
    try { return Files.readAllBytes(p); }
    catch (IOException e) { throw new UncheckedIOException(e); }
});
```

- 12. Catching a broad type just to rethrow a narrower, unrelated one, losing the original stack trace.** Why it matters: creating a brand-new exception without chaining the original throws away the actual failure location, making the bug much harder to diagnose from logs alone.

```
// Before
catch (Exception e) { throw new BusinessException("failed"); }
// After
catch (Exception e) { throw new BusinessException("failed", e); }
```

- 13. Declaring `throws Exception` on a method signature instead of the specific checked exceptions it actually throws.** Why it matters: it forces every caller to catch the overly broad `Exception`, defeating the whole purpose of checked exceptions — communicating precisely what can go wrong.

```
// Before
void process() throws Exception { ... }
// After
void process() throws IOException, SQLException { ... }
```

- 14. Comparing exception messages (`e.getMessage().contains(...)`) to detect a specific failure instead of catching a specific type.** Why it matters: messages are not part of the API contract and can change between JDK versions or library updates, silently breaking the check.


```

// Before
catch (Exception e) { if (e.getMessage().contains("not found")) { ...
↳ } }
// After
catch (NoSuchElementException e) { ... }

```

15. Ignoring suppressed exceptions when debugging try-with-resources failures.

Why it matters: if closing a resource also fails, that information is attached as a suppressed exception on the primary one; skipping `getSuppressed()` during triage can hide a second, independent root cause (e.g., a leaked connection on top of the original error).

```

// Before
catch (Exception e) { log.error("Failed", e); } // suppressed causes
↳ buried in nested trace
// After
catch (Exception e) {
    log.error("Failed", e);
    for (Throwable s : e.getSuppressed()) log.error("Also failed while
↳ closing", s);
}

```

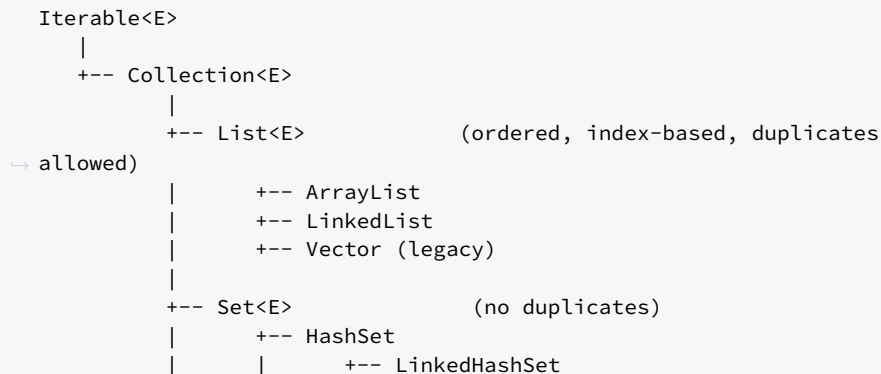

Collections Framework

The Collections Framework is Java’s standard toolkit for storing and processing groups of objects. It gives you ready-made data structures (lists, sets, maps, queues) so you don’t write your own array-resizing or hash-table code. This chapter walks through the interfaces, the concrete classes that implement them, and the mistakes that come up most often in code review. The goal is that you can pick the right collection quickly and explain *why* it is right.

Collections Framework Overview

A **collection** is an object that holds other objects, called *elements*. The Collections Framework is a set of interfaces (contracts) and classes (implementations) that all work together. Almost every Java program uses it, so interviewers expect you to know the shape of it cold.

Here is the interface hierarchy you need in your head. `Iterable` is the root because “something you can loop over” is the most basic idea. `Map` is drawn separately because a map does not extend `Collection` — it holds key-value pairs, not single elements.



```

|      +-- SortedSet<E>
|      |      +-- NavigableSet<E>
|      |      |      +-- TreeSet
|      |
+-- Queue<E>          (head/tail processing)
    +-- PriorityQueue
    +-- Deque<E>      (double-ended queue)
        +-- ArrayDeque
        +-- LinkedList (also a List)

Map<K,V>              (separate hierarchy, NOT a Collection)
+-- HashMap
|      +-- LinkedHashMap
+-- SortedMap<K,V>
|      +-- NavigableMap<K,V>
|      |      +-- TreeMap
+-- Hashtable (legacy)
+-- IdentityHashMap
+-- WeakHashMap
+-- EnumMap

```

Java 21 added two small but handy interfaces:

- **SequencedCollection<E>** — for collections that have a well-defined *first* and *last* element (like `List` and `Deque`). It adds `getFirst()`, `getLast()`, `addFirst()`, `addLast()`, `removeFirst()`, `removeLast()`, and `reversed()`. `List` and `Deque` now implement it, so you get these methods on `ArrayList`, `LinkedList`, and `ArrayDeque` for free.
- **SequencedMap<K,V>** — the map equivalent, with `firstEntry()`, `lastEntry()`, `putFirst()`, `putLast()`, and `reversed()`. `LinkedHashMap` and `TreeMap` implement it.

```

List<String> names = new ArrayList<>(List.of("Ann", "Bob", "Cara"));
System.out.println(names.getFirst()); // Ann
System.out.println(names.getLast());  // Cara
List<String> reversed = names.reversed();
System.out.println(reversed);          // [Cara, Bob, Ann]

```

Why the framework matters in a code review: picking the wrong collection is one of the most common review findings — using a `List` where a `Set` was meant, using `ArrayList` where a `Deque` fits better, or forgetting that `HashMap` iteration order is unspecified. Knowing the contracts lets you spot these fast.

Collection Interface

`Collection<E>` is the root interface for anything that is a “bunch of elements” (excluding maps). It defines the operations every collection must support: `add`, `remove`, `contains`, `size`, `isEmpty`, `iterator`, `stream`, `forEach`, and bulk operations like `addAll`, `removeAll`, `retainAll`.

```
Collection<String> col = new ArrayList<>();
col.add("apple");
col.add("banana");
System.out.println(col.contains("apple")); // true
System.out.println(col.size());           // 2
col.removeIf(s -> s.startsWith("b"));
System.out.println(col);                   // [apple]
```

Key things every `Collection` implementation must get right:

- **equals/hashCode contract:** `Collection.equals` is defined precisely for `List` and `Set` (element-by-element), so if you put custom objects in a collection, your `equals`/`hashCode` must be consistent (see Chapter 5).
- **Optional operations:** some collections (like the ones from `List.of()`) throw `UnsupportedOperationException` for `add`/`remove`. The interface documents these as “optional operations.”
- **Fail-fast iterators:** most `java.util` collections detect structural modification during iteration and throw `ConcurrentModificationException` (covered in detail below).

List

A `List<E>` is an **ordered** collection accessed by a zero-based index. It allows **duplicate** elements. Think of it as a resizable array with extra methods.

Key methods: `get(int)`, `set(int, E)`, `add(int, E)`, `remove(int)`, `indexOf(Object)`, `subList(from, to)`, plus everything from `Collection` and (since Java 21) `SequencedCollection`.

```
List<String> fruits = new ArrayList<>();
fruits.add("apple");
fruits.add("banana");
fruits.add(1, "kiwi");           // insert at index 1
```

```
System.out.println(fruits); // [apple, kiwi, banana]
System.out.println(fruits.get(0)); // apple
```

Main implementations: `ArrayList` (array-backed, default choice), `LinkedList` (doubly-linked list, rarely the right default), `Vector` (legacy, synchronized, avoid).

`List.equals` compares element order and values, so `[1, 2, 3]` and `[3, 2, 1]` are **not** equal even though they hold the same elements.

Set

A `Set<E>` is a collection with **no duplicate elements**, based on `equals`. It models mathematical sets. It has no index-based access.

Sub-interfaces:

- `SortedSet<E>` — elements kept in sorted order (by natural ordering or a `Comparator`).
- `NavigableSet<E>` — adds navigation methods like `floor`, `ceiling`, `higher`, `lower`.

```
Set<String> tags = new HashSet<>();
tags.add("java");
tags.add("java"); // ignored, duplicate
tags.add("spring");
System.out.println(tags.size()); // 2
System.out.println(tags.contains("java")); // true
```

Main implementations: `HashSet` (fast, no order), `LinkedHashSet` (insertion order), `TreeSet` (sorted order).

Queue

A `Queue<E>` models a line of elements processed in a specific order, typically **FIFO** (first-in-first-out). Core methods come in two flavors: ones that throw exceptions and ones that return a special value (`null` or `false`) on failure.

Operation	Throws exception	Returns special value
Insert	<code>add(e)</code>	<code>offer(e)</code>
Remove	<code>remove()</code>	<code>poll()</code>
Examine	<code>element()</code>	<code>peek()</code>

```
Queue<Integer> queue = new java.util.ArrayDeque<>();
queue.offer(1);
queue.offer(2);
queue.offer(3);
System.out.println(queue.poll()); // 1 (FIFO order)
System.out.println(queue.peek()); // 2, does not remove
```

Main implementations: `PriorityQueue` (elements ordered by priority, not insertion order), `ArrayDeque` (fast general-purpose queue/stack), `LinkedList` (also implements `Queue`, rarely the best choice).

Prefer the `offer/poll/peek` family in production code — they signal “empty” or “full” with a return value instead of an exception, which is usually easier to handle in a loop.

Deque

A `Deque<E>` (“deck”, **double-ended queue**) supports insertion and removal at **both ends**. It can act as a `Queue` (FIFO), a `Stack` (LIFO), or both.

```
Deque<Integer> stack = new java.util.ArrayDeque<>();
stack.push(1); // addFirst
stack.push(2);
stack.push(3);
System.out.println(stack.pop()); // 3 (LIFO, like a Stack)

Deque<Integer> dq = new java.util.ArrayDeque<>();
dq.addFirst(10);
dq.addLast(20);
System.out.println(dq); // [10, 20]
System.out.println(dq.pollFirst()); // 10
System.out.println(dq.pollLast()); // 20
```

Because `Deque` extends `SequencedCollection`, it also has `getFirst()`, `getLast()`, and `reversed()`.

Deque replaces both Stack and java.util.Stack/legacy usage. The official Javadoc recommends `ArrayDeque` over `java.util.Stack` (legacy, synchronized, extends `Vector`) for stack behavior, and over `LinkedList` for queue behavior, because `ArrayDeque` is faster and has no synchronization overhead.

Map

A `Map<K,V>` stores **key-value pairs**. Keys are unique; each key maps to exactly one value. Map does **not** extend `Collection` because its elements are pairs, not single objects — but it does expose collection views: `keySet()`, `values()`, and `entrySet()`.

```
Map<String, Integer> ages = new HashMap<>();
ages.put("Ann", 30);
ages.put("Bob", 25);
ages.put("Ann", 31); // overwrites, put returns old value (30)

for (Map.Entry<String, Integer> entry : ages.entrySet()) {
    System.out.println(entry.getKey() + " -> " + entry.getValue());
}
// Ann -> 31
// Bob -> 25 (order not guaranteed for plain HashMap)
```

Useful modern default methods (Java 8+), all worth knowing for reviews:

```
Map<String, Integer> scores = new HashMap<>();
scores.putIfAbsent("Ann", 10); // inserts only if key missing
scores.merge("Ann", 5, Integer::sum); // Ann -> 15
scores.computeIfAbsent("Bob", k -> 0); // Bob -> 0
scores.compute("Bob", (k, v) -> v + 1); // Bob -> 1
scores.getOrDefault("Cara", -1); // -1, no exception
```

Sub-interfaces:

- `SortedMap<K,V>` — keys kept sorted.
- `NavigableMap<K,V>` — adds `floorKey`, `ceilingKey`, `firstEntry`, `lastEntry`, etc.
- `SequencedMap<K,V>` (Java 21) — `putFirst`, `putLast`, `firstEntry`, `lastEntry`, `reversed()`. Implemented by `LinkedHashMap` and `TreeMap`.

Main implementations: `HashMap`, `LinkedHashMap`, `TreeMap`, plus special-purpose ones: `IdentityHashMap`, `WeakHashMap`, `EnumMap`.

Master “Which Collection Should I Pick?” Table

Use this as your mental cheat sheet in an interview. “Amortized” means the average cost over many operations; an occasional resize can cost more.

Collection	Get by index	Get by key	Add/remove at end	Add/remove at front	Contains	Ordering	Duplicate	Null elements
ArrayList	O(1)	—	O(1) amortized	O(n)	O(n)	insertion order	yes	yes
LinkedList	O(n)	—	O(1)	O(1)	O(n)	insertion order	yes	yes
HashMap	—	O(1) avg	—	—	O(1) avg (key)	unspecified	keys: no, values: yes	1 null key, many null values
LinkedHashMap	—	O(1) avg	—	—	O(1) avg	insertion (or access) order	keys: no	same as HashMap
TreeMap	—	O(log n)	—	—	O(log n)	sorted by key	keys: no	no null keys
HashSet	—	—	O(1) avg	—	O(1) avg	unspecified	no	one null
LinkedHashSet	—	—	O(1) avg	—	O(1) avg	insertion order	no	one null
TreeSet	—	—	O(log n)	—	O(log n)	sorted	no	no null
ArrayDeque	—	—	O(1)	O(1)	O(n)	insertion order	yes	no null
PriorityQueue	—	—	O(log n) insert	O(log n) poll head	O(n)	priority order (only head is guaranteed smallest)	yes	no null

Quick decision guide:

- Need index access, mostly append at the end → **ArrayList**.
- Need a stack or a queue, or fast add/remove at both ends → **ArrayDeque**.

- Need uniqueness, don't care about order → **HashSet**.
- Need uniqueness and insertion order preserved → **LinkedHashSet**.
- Need uniqueness and sorted order → **TreeSet**.
- Need key-value lookups, don't care about order → **HashMap**.
- Need key-value lookups with predictable iteration order → **LinkedHashMap**.
- Need key-value lookups sorted by key, or range queries → **TreeMap**.
- Need “always process smallest/largest next” → **PriorityQueue**.
- Multithreaded access → see [Concurrent Collections](#).

ArrayList

How it works internally: `ArrayList` wraps a plain `Object[]` array. When the array is full and you add another element, it allocates a new array (roughly 1.5x the old capacity) and copies everything over (`Arrays.copyOf`). This is why appends are “amortized” $O(1)$ — most calls are cheap, but occasionally one call pays for the copy.

```
List<Integer> list = new ArrayList<>();           // default capacity, grows
↪ as needed
List<Integer> sized = new ArrayList<>(1000);      // pre-size to avoid early
↪ resizes
for (int i = 0; i < 5; i++) list.add(i);
list.remove(Integer.valueOf(2)); // removes value 2, not index 2
list.remove(2);                 // removes index 2 (int overload)
System.out.println(list);       // [0, 1, 4] after both removals
```

Big-O:

- `get(index)` / `set(index, e)`: $O(1)$ — direct array access.
- `add(e)` (append at end): $O(1)$ amortized, $O(n)$ worst case (on resize).
- `add(index, e)` / `remove(index)` in the middle or front: $O(n)$ — must shift elements.
- `contains` / `indexOf`: $O(n)$ — linear scan.

Ordering: preserves insertion order (it's index-based).

Null handling: allows multiple `null` elements.

Thread safety: not thread-safe. Concurrent modification from multiple threads can corrupt internal state or throw `ConcurrentModificationException`.

When to choose it: default `List` choice. Use it when you mostly read by index or append at the end, and rarely insert/remove in the middle or at the front.

LinkedList

How it works internally: a doubly-linked list of nodes; each node holds the element plus references to the previous and next node. It implements both `List` and `Deque`.

```
LinkedList<String> chain = new LinkedList<>();
chain.addLast("b");
chain.addFirst("a");
chain.addLast("c");
System.out.println(chain); // [a, b, c]
chain.removeFirst();
System.out.println(chain); // [b, c]
```

Big-O:

- `addFirst` / `addLast` / `removeFirst` / `removeLast`: $O(1)$ — just pointer updates.
- `get(index)`: $O(n)$ — must walk the list from the nearer end.
- `add(index, e)` / `remove(index)`: $O(n)$ to find the position, $O(1)$ to splice.

Ordering: insertion order.

Null handling: allows null elements.

Thread safety: not thread-safe.

When to choose it: rarely the best default. Choose it only when you need frequent insert/remove at both ends *and* also need `List` semantics (index access) in the same structure. For pure queue/stack behavior, prefer `ArrayDeque` — it's faster and uses less memory (no per-node object overhead, no boxing of pointers).

```
// Common mistake: using LinkedList as a general-purpose List
List<Integer> bad = new LinkedList<>();
for (int i = 0; i < 100_000; i++) bad.add(i);
// bad.get(50_000) is O(n) -- walks half the list every time!
```

HashMap

HashMap is the workhorse map implementation. Understanding its internals is one of the most frequently asked collection topics in interviews.

How it works internally:

1. **Buckets:** internally, HashMap holds an array called the *table*. Each slot in that array is called a **bucket**. A bucket can hold zero, one, or many entries.
2. **Hashing:** when you call `put(key, value)`, the map computes `key.hashCode()`, then applies an internal “spread” function (XORs the hash with its own upper bits) to reduce collisions, then uses `hash & (table.length - 1)` to pick a bucket index. This is why table size is always a power of two — `& (length - 1)` is a fast way to do `% length` when `length` is a power of two.
3. **Collision handling:** if two keys land in the same bucket, they used to be stored as a **linked list** of entries in that bucket. You look up a key by finding its bucket, then walking the list and comparing with `equals()`.
4. **Treeification (JDK 8+):** if a single bucket’s linked list grows to **8 or more** entries (and the table has at least 64 buckets), that bucket is converted into a small **red-black tree** instead of a linked list. This turns worst-case lookup within a bucket from $O(n)$ into $O(\log n)$. This mostly protects against pathological hash collisions (e.g., poorly written `hashCode()` or adversarial input) — it is a safety net, not something you should rely on for normal use. If entries in a treeified bucket drop below 6, it converts back to a linked list.
5. **Resizing / load factor:** the map tracks a **load factor**, default `0.75`. When `size > capacity * loadFactor`, the table **doubles** in size and every entry is rehashed into the new, larger table (a “resize” or “rehash”). Default initial capacity is 16, so the first resize happens after the 13th entry ($16 * 0.75 = 12$). Resizing is $O(n)$ but happens rarely, so `put` is amortized $O(1)$.

```
Map<String, Integer> inventory = new HashMap<>(); // capacity 16, load
↪ factor 0.75
inventory.put("apples", 10);
inventory.put("bananas", 20);
inventory.put(null, 0);           // one null key is allowed
inventory.put("apples", 15);      // overwrites, returns old value 10

System.out.println(inventory.get("apples")); // 15
System.out.println(inventory.get("kiwi"));   // null: no such key
System.out.println(inventory.get(null));     // 0
```

```
// Pre-sizing avoids repeated resizes when you know the approximate size
Map<String, String> big = new HashMap<>(200); // avoids ~4 resize/rehash
↪ cycles
```

Big-O: get/put/remove/containsKey: $O(1)$ average, $O(\log n)$ worst case per bucket after treeification (was $O(n)$ before JDK 8). This assumes a good `hashCode()` distribution; a bad `hashCode()` (e.g., always returning 0) degrades performance because everything lands in one bucket.

Ordering: unspecified and not guaranteed to be stable across JDK versions or even across resizes of the same map. Never rely on `HashMap` iteration order.

Null handling: allows exactly **one** null key and any number of null values.

Thread safety: not thread-safe. Concurrent put calls from multiple threads can corrupt the internal structure (classic bug: infinite loop during resize under older JDKs, or lost updates). Use `ConcurrentHashMap` instead.

When to choose it: the default map. Use it whenever you need key-value lookups and don't care about iteration order.

```
// hashCode()/equals() contract matters for keys!
class BadKey {
    int id;
    BadKey(int id) { this.id = id; }
    // no equals()/hashCode() override -> uses Object identity
}
Map<BadKey, String> map = new HashMap<>();
map.put(new BadKey(1), "a");
System.out.println(map.get(new BadKey(1))); // null! different object,
↪ same "logical" key
```

LinkedHashMap

How it works internally: extends `HashMap` and adds a doubly-linked list threading through all entries, to remember order. Every `Map.Entry` also has before/after pointers.

```
Map<String, Integer> lru = new LinkedHashMap<>(16, 0.75f, true); //
↪ accessOrder = true
lru.put("a", 1);
lru.put("b", 2);
lru.put("c", 3);
```

```
lru.get("a"); // touches "a", moves it to the end in access-order mode
System.out.println(lru.keySet()); // [b, c, a]
```

Because it tracks access order, `LinkedHashMap` is the standard building block for a simple **LRU cache**:

```
class LruCache<K, V> extends LinkedHashMap<K, V> {
    private final int maxSize;
    LruCache(int maxSize) {
        super(16, 0.75f, true); // true = access-order
        this.maxSize = maxSize;
    }
    @Override
    protected boolean removeEldestEntry(Map.Entry<K, V> eldest) {
        return size() > maxSize; // evict the oldest entry once full
    }
}

LruCache<String, String> cache = new LruCache<>(2);
cache.put("a", "1");
cache.put("b", "2");
cache.get("a"); // "a" becomes most-recently-used
cache.put("c", "3"); // evicts "b" (least recently used)
System.out.println(cache.keySet()); // [a, c]
```

Big-O: same as `HashMap` — $O(1)$ average for `get`/`put`/`remove` — plus a small constant overhead to maintain the linked list.

Ordering: insertion order by default, or access order if constructed with `accessOrder = true`.

Null handling: same as `HashMap` (one null key, many null values).

Thread safety: not thread-safe.

When to choose it: when you need predictable, repeatable iteration order (e.g., for deterministic tests, or to preserve the order fields were added for serialization), or to build an LRU cache.

TreeMap

How it works internally: a **red-black tree** (a self-balancing binary search tree), keyed by `Comparable` order or a supplied `Comparator`. Every insert/lookup walks down the tree comparing keys.

```

TreeMap<String, Integer> sorted = new TreeMap<>();
sorted.put("banana", 2);
sorted.put("apple", 1);
sorted.put("cherry", 3);
System.out.println(sorted); // {apple=1, banana=2, cherry=3} -- sorted by
↳ key

System.out.println(sorted.firstKey());           // apple
System.out.println(sorted.lastKey());            // cherry
System.out.println(sorted.ceilingKey("b"));      // banana (smallest key >=
↳ "b")
System.out.println(sorted.floorKey("b"));        // apple (largest key <=
↳ "b")
System.out.println(sorted.headMap("banana"));    // {apple=1} -- keys
↳ strictly less than "banana"

```

Big-O: get/put/remove/containsKey: $O(\log n)$. Range operations (headMap, tailMap, subMap) are $O(\log n)$ to locate the boundary, and the returned view is backed by the original map.

Ordering: sorted by natural ordering of keys, or by the Comparator given at construction time.

Null handling: no null keys allowed (throws NullPointerException — comparisons need a real key). Null values are allowed if the comparator doesn't touch values.

Thread safety: not thread-safe.

When to choose it: when you need keys sorted, or range queries (floorKey, ceilingKey, subMap), or the smallest/largest key on demand.

```

// Custom key type needs a Comparator, or must implement Comparable
TreeMap<String, Integer> byLength = new
↳ TreeMap<>(Comparator.comparingInt(String::length));
byLength.put("kiwi", 1);
byLength.put("fig", 2);
System.out.println(byLength); // {fig=2, kiwi=1} -- sorted by string
↳ length

```

HashSet

How it works internally: backed internally by a `HashMap<E, Object>` where every element is stored as a key, and all values point to a shared dummy constant object. So everything said about `HashMap` buckets, hashing, treeification, and resizing applies here too.

```
Set<String> set = new HashSet<>();
set.add("x");
set.add("y");
set.add("x"); // ignored -- already present
System.out.println(set.size()); // 2
System.out.println(set);        // order unspecified, e.g. [x, y] or [y,
↪ x]
```

Big-O: add/remove/contains: $O(1)$ average, same worst-case caveats as `HashMap`.

Ordering: unspecified.

Null handling: allows one `null` element.

Thread safety: not thread-safe.

When to choose it: default `Set` choice, when you need fast membership tests and don't care about order.

LinkedHashSet

How it works internally: backed by a `LinkedHashMap` internally, so it has the same linked-list-through-buckets trick to preserve insertion order.

```
Set<String> visited = new LinkedHashSet<>();
visited.add("home");
visited.add("cart");
visited.add("home"); // ignored, but original position is kept
visited.add("checkout");
System.out.println(visited); // [home, cart, checkout]
```

Big-O: same as `HashSet` — $O(1)$ average.

Ordering: insertion order, preserved.

Null handling: one `null` allowed.

Thread safety: not thread-safe.

When to choose it: when you want set semantics (no duplicates) but also need to remember the order elements were added — e.g., de-duplicating a list while preserving first-seen order.

```
List<String> withDups = List.of("b", "a", "b", "c", "a");
Set<String> deduped = new LinkedHashSet<>(withDups);
System.out.println(deduped); // [b, a, c] -- de-duplicated, first-seen
→ order kept
```

TreeSet

How it works internally: backed by a `TreeMap<E, Object>`, so it's a red-black tree under the hood.

```
TreeSet<Integer> scores = new TreeSet<>(List.of(42, 7, 15, 4));
System.out.println(scores); // [4, 7, 15, 42] -- sorted
System.out.println(scores.first()); // 4
System.out.println(scores.last()); // 42
System.out.println(scores.higher(15)); // 42 (smallest element strictly
→ greater than 15)
System.out.println(scores.lower(15)); // 7 (largest element strictly
→ less than 15)
```

Big-O: add/remove/contains: $O(\log n)$.

Ordering: sorted, by natural order or supplied `Comparator`.

Null handling: no nulls (comparisons would throw `NullPointerException`).

Thread safety: not thread-safe.

When to choose it: when you need a de-duplicated, always-sorted collection, or navigation methods like `first()`, `last()`, `higher()`, `lower()`, `subSet()`.

PriorityQueue

How it works internally: a **binary heap** stored in an array. The heap property guarantees the smallest element (by natural order or comparator) is always at the root/head, so `peek()`/`poll()` are cheap. It is **not** a full sort — only the head is guaranteed to be the minimum; the rest of the internal array is *not* in sorted order.

```

PriorityQueue<Integer> pq = new PriorityQueue<>();
pq.offer(5);
pq.offer(1);
pq.offer(3);
System.out.println(pq.poll()); // 1 (smallest first)
System.out.println(pq.poll()); // 3
System.out.println(pq.poll()); // 5

// Max-heap using a reverse comparator
PriorityQueue<Integer> maxHeap = new
↳ PriorityQueue<>(Comparator.reverseOrder());
maxHeap.offer(5);
maxHeap.offer(1);
maxHeap.offer(3);
System.out.println(maxHeap.poll()); // 5 (largest first)

// Common use case: "top-k" or scheduling by priority
record Task(String name, int priority) {}
PriorityQueue<Task> tasks = new
↳ PriorityQueue<>(Comparator.comparingInt(Task::priority));
tasks.offer(new Task("cleanup", 3));
tasks.offer(new Task("urgent-fix", 1));
tasks.offer(new Task("report", 2));
System.out.println(tasks.poll().name()); // urgent-fix (lowest number =
↳ highest priority)

```

Big-O: offer/add (insert): $O(\log n)$. poll/remove head: $O(\log n)$. peek: $O(1)$. contains (arbitrary element): $O(n)$.

Ordering: only the head element order is guaranteed; iterating the queue directly does not yield sorted order (use repeated `poll()` for that).

Null handling: no nulls — throws `NullPointerException` (nulls can't be compared).

Thread safety: not thread-safe. For a concurrent priority queue, use `PriorityBlockingQueue`.

When to choose it: task scheduling, “always process the most urgent/smallest item next,” or top-k problems (e.g., “k largest elements in a stream”).

ArrayDeque

How it works internally: a resizable circular array (a ring buffer) with head and tail indices. Unlike `ArrayList`, it can grow efficiently at both ends because it wraps around the array instead of always shifting from index 0.

```
Deque<String> deque = new ArrayDeque<>();
deque.addFirst("b");
deque.addFirst("a");
deque.addLast("c");
System.out.println(deque); // [a, b, c]

// as a Stack (LIFO)
Deque<Integer> stack = new ArrayDeque<>();
stack.push(1); stack.push(2); stack.push(3);
System.out.println(stack.pop()); // 3

// as a Queue (FIFO)
Deque<Integer> queue = new ArrayDeque<>();
queue.offer(1); queue.offer(2); queue.offer(3);
System.out.println(queue.poll()); // 1
```

Big-O: `addFirst/addLast/removeFirst/removeLast/peekFirst/peekLast`: $O(1)$ amortized. `get(index)`-style access is not offered (it's not a `List`); contains: $O(n)$.

Ordering: insertion order (head to tail).

Null handling: no nulls — `null` is used internally as a sentinel to mean “empty,” so `add(null)` throws `NullPointerException`.

Thread safety: not thread-safe. For concurrent use, see `ConcurrentLinkedDeque` or `LinkedBlockingDeque`.

When to choose it: the default choice for both stacks and queues. Faster than `Stack` (legacy, synchronized) and faster than `LinkedList` for the same job, with less memory overhead.

IdentityHashMap

How it works internally: looks like `HashMap`, but uses **reference equality** (`==`) instead of `.equals()`, and `System.identityHashCode()` instead of `.hashCode()`, to compare and hash keys. Two distinct objects that are “equal” by `.equals()` are treated as **different** keys if they are not the same object in memory.

```
Map<String, String> identityMap = new IdentityHashMap<>();
String a = new String("key");
String b = new String("key"); // different object, same content
identityMap.put(a, "first");
identityMap.put(b, "second"); // treated as a DIFFERENT key
System.out.println(identityMap.size()); // 2, not 1!
System.out.println(identityMap.get(a)); // first
System.out.println(identityMap.get(new String("key"))); // null -- not the
↳ same reference
```

Big-O: $O(1)$ average for get/put — same style of hash table as `HashMap`, just with identity hashing.

Ordering: unspecified.

Null handling: allows a null key and null values.

Thread safety: not thread-safe.

When to choose it: rare. Useful for object-graph traversal algorithms (e.g., serialization, deep-copy, cycle detection) where you need to track “have I visited this exact object instance before,” regardless of its `.equals()` implementation. The JDK itself uses it in `ObjectOutputStream`.

WeakHashMap

How it works internally: stores keys as **weak references**. If nothing else in the program holds a strong reference to a key, the garbage collector is free to reclaim it. Once a key is collected, its entry is removed from the map automatically (lazily, on the next access or via an internal cleanup on the next operation).

```
Map<Object, String> cache = new WeakHashMap<>();
Object key = new Object();
cache.put(key, "metadata");
System.out.println(cache.size()); // 1

key = null; // drop the only strong reference
System.gc(); // suggest GC (not guaranteed, but likely in this
↳ simple example)
// after GC runs, the entry may be gone:
// System.out.println(cache.size()); // often 0, though timing isn't
↳ guaranteed
```

Big-O: $O(1)$ average for get/put, same as `HashMap`.

Ordering: unspecified.

Null handling: allows a null key/value (though a null key is a bit unusual conceptually).

Thread safety: not thread-safe.

When to choose it: caches or metadata maps keyed by objects you don't want to keep alive artificially — e.g., attaching extra data to objects managed elsewhere, such as classloader-scoped caches. Not a general-purpose cache eviction tool; for size/time-based eviction, prefer a real caching library (Caffeine, Guava) or LinkedHashMap-based LRU.

EnumMap

How it works internally: a specialized Map implementation for **enum keys only**, backed internally by a plain array indexed by the enum constant's `ordinal()`. No hashing needed at all.

```
enum Day { MON, TUE, WED, THU, FRI, SAT, SUN }

EnumMap<Day, String> schedule = new EnumMap<>(Day.class);
schedule.put(Day.MON, "Standup");
schedule.put(Day.FRI, "Retro");
System.out.println(schedule); // {MON=Standup, FRI=Retro} -- always in
→ enum declaration order
```

Big-O: get/put/remove: $O(1)$ — direct array indexing by ordinal, no hash collisions possible.

Ordering: always iterates in the natural order the enum constants are declared (i.e., ordinal order), regardless of insertion order.

Null handling: no null keys (throws `NullPointerException`); null values allowed.

Thread safety: not thread-safe.

When to choose it: whenever your map key is an enum. It is faster and more memory-efficient than `HashMap<EnumType, V>`, and its guaranteed ordering is a nice bonus.

EnumSet

How it works internally: a specialized Set for enum values, internally represented as a **bitmask** (a long, or an array of longs for enums with more than 64 constants). Membership tests and unions/intersections become simple bitwise operations.

```
enum Permission { READ, WRITE, EXECUTE, DELETE }

EnumSet<Permission> readWrite = EnumSet.of(Permission.READ,
↳ Permission.WRITE);
EnumSet<Permission> all = EnumSet.allOf(Permission.class);
EnumSet<Permission> none = EnumSet.noneOf(Permission.class);
EnumSet<Permission> everythingButDelete =
↳ EnumSet.complementOf(EnumSet.of(Permission.DELETE));

System.out.println(readWrite);           // [READ, WRITE]
System.out.println(everythingButDelete); // [READ, WRITE, EXECUTE]
System.out.println(readWrite.contains(Permission.READ)); // true
```

Big-O: add/remove/contains: $O(1)$ — bitmask operations.

Ordering: natural order of enum declaration (ordinal order).

Null handling: no nulls (throws `NullPointerException`).

Thread safety: not thread-safe (though because it's so compact, wrapping with `Collections.synchronizedSet` is cheap if needed).

When to choose it: whenever you have a set of flags/options drawn from a fixed enum — e.g., permissions, feature flags, days of the week. Much faster and more compact than `HashSet<EnumType>`.

Immutable Collections (List.of())

Since Java 9, the JDK ships **factory methods** that create truly immutable collections directly, without needing a mutable builder first.

```
List<String> list = List.of("a", "b", "c");
Set<String> set = Set.of("x", "y", "z");
Map<String, Integer> map = Map.of("a", 1, "b", 2);
Map<String, Integer> bigMap = Map.ofEntries(
    Map.entry("a", 1),
    Map.entry("b", 2),
    Map.entry("c", 3))
```

```
);
```

Key rules of these factories, all commonly tested in interviews:

1. **Null-hostile:** passing `null` as an element, key, or value throws `NullPointerException` immediately — even just constructing the list fails, it's not a lazy check.

```
List.of("a", null); // throws NullPointerException
```

2. **No duplicates in Set.of/Map.of:** passing a duplicate element or key throws `IllegalArgumentException` at creation time.

```
Set.of("a", "a"); // throws IllegalArgumentException: duplicate element
Map.of("a", 1, "a", 2); // throws IllegalArgumentException: duplicate
↳ key
```

3. **Truly unmodifiable:** any mutating call (add, remove, set, put, clear, ...) throws `UnsupportedOperationException`.

```
List<String> immutable = List.of("a", "b");
immutable.add("c"); // throws UnsupportedOperationException
```

List.copyOf / Set.copyOf / Map.copyOf: produce an immutable *copy* of an existing collection. If the source is already an immutable instance from these same factories, it may return the same instance instead of copying (an optimization, not something to rely on).

```
List<String> mutable = new ArrayList<>(List.of("a", "b"));
List<String> snapshot = List.copyOf(mutable); // independent, immutable
↳ snapshot
mutable.add("c");
System.out.println(snapshot); // [a, b] -- unaffected by later changes to
↳ "mutable"
```

Collectors.toUnmodifiableList/Set/Map: the Streams equivalent, producing an immutable result directly from a stream pipeline.

```
List<String> upper = java.util.stream.Stream.of("a", "b", "c")
    .map(String::toUpperCase)
    .collect(java.util.stream.Collectors.toUnmodifiableList());
```

```
// upper.add("D") would throw UnsupportedOperationException
```

Immutable factories vs Collections.unmodifiableList — not the same thing: Collections.unmodifiableXxx wraps an existing mutable collection in a **read-only view**. The view itself can't be modified directly, but if you keep a reference to the underlying mutable collection, changes there still show through the view.

```
List<String> backing = new ArrayList<>(List.of("a", "b"));
List<String> view = Collections.unmodifiableList(backing);

view.add("c"); // throws UnsupportedOperationException (the view is
↳ read-only)
backing.add("c"); // allowed! and it changes what "view" shows
System.out.println(view); // [a, b, c] -- the "immutable" view just
↳ changed
```

	Collections.unmodifiableList	List.of() / List.copyOf()
Backed by	the original mutable list (a view)	its own internal storage
Changes to source visible?	yes	no (copy)
Allows null elements?	depends on the backing list	never (throws NPE)
Created before Java 9?	yes (Java 1.2)	no (Java 9+)

Concurrent Collections

Plain java.util collections (ArrayList, HashMap, etc.) are **not** thread-safe. java.util.concurrent provides collections designed for multithreaded use, with better scalability than simply synchronizing everything.

Why Collections.synchronizedList is not enough

Collections.synchronizedXxx wraps a collection so each **individual method call** is synchronized (guarded by a single lock). That protects against corruption from concurrent single calls, but it does **not** make compound actions (check-then-act, like “if absent, then add”) atomic.


```

List<String> list = Collections.synchronizedList(new ArrayList<>());

// BUG: this is two separate synchronized calls, not one atomic operation.
// Between contains() and add(), another thread could add the same item.
if (!list.contains("item")) {
    list.add("item"); // race condition: might now add a duplicate
}

// Correct: manually synchronize the whole compound action on the list's
→ own lock
synchronized (list) {
    if (!list.contains("item")) {
        list.add("item");
    }
}

// Also: even a single iteration needs manual synchronization to avoid
// ConcurrentModificationException from another thread's concurrent write
synchronized (list) {
    for (String s : list) {
        System.out.println(s);
    }
}

```

This is exactly why `java.util.concurrent` collections exist: they provide atomic compound operations (like `putIfAbsent`) without you needing an external lock.

ConcurrentHashMap

A highly-scalable map for concurrent access. Internally (JDK 8+) it does **not** lock the whole map; it uses fine-grained locking per bin (bucket) plus CAS (compare-and-swap) operations for many updates, so multiple threads can write to different parts of the map at the same time.

```

ConcurrentHashMap<String, Integer> counts = new ConcurrentHashMap<>();
counts.putIfAbsent("errors", 0);
counts.compute("errors", (k, v) -> v + 1); // atomic read-modify-write
counts.merge("errors", 1, Integer::sum); // also atomic

// Safe iteration while other threads mutate the map -- no
→ ConcurrentModificationException,
// but the view may or may not reflect very recent concurrent updates
→ ("weakly consistent")
for (String key : counts.keySet()) {
    System.out.println(key + " = " + counts.get(key));
}

```

```
}
```

Notes:

- Does **not** allow null keys or values (unlike HashMap) — this is deliberate, because null would be ambiguous in a concurrent get (is it “not present” or “present with null value”?).
- Iterators are **weakly consistent**: they never throw ConcurrentModificationException, and they reflect the state of the map at some point during or after the iterator was created, but not necessarily every single concurrent update.
- size() is an estimate under heavy concurrent modification, not a hard snapshot.

CopyOnWriteArrayList

A List where every mutating operation (add, remove, set) makes a **fresh copy of the entire underlying array**. Reads never block and never see partial writes — they just read a stable snapshot array.

```
CopyOnWriteArrayList<String> listeners = new CopyOnWriteArrayList<>();
listeners.add("listenerA");
listeners.add("listenerB");

// Safe to iterate even if another thread adds/removes concurrently --
// the iterator works on the snapshot array taken when the loop started,
// so no ConcurrentModificationException, ever.
for (String listener : listeners) {
    System.out.println("Notifying " + listener);
    listeners.remove(listener); // does NOT affect this ongoing
    ↪ iteration's snapshot
}
```

When to use it: read-heavy, write-rare scenarios, like a list of event listeners/observers. **Avoid** it for write-heavy workloads — every single write copies the whole array, which is O(n) per write and creates a lot of garbage.

BlockingQueue family

BlockingQueue<E> extends Queue and adds methods that **block** the calling thread instead of failing, until space or an element becomes available. This is the standard building block for producer-consumer pipelines.

Method	On empty (poll side)	On full (offer side)
add/remove	throws exception	throws exception
offer/poll	returns null/false immediately	returns false immediately
put/take	blocks until available	blocks until space
offer(e, timeout) / poll(timeout)	blocks up to a timeout, then gives up	blocks up to a timeout, then gives up

```

BlockingQueue<String> jobs = new
↳ java.util.concurrent.LinkedBlockingQueue<>(100); // bounded capacity

// Producer thread
new Thread(() -> {
    try {
        jobs.put("job-1"); // blocks if the queue is full
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
}).start();

// Consumer thread
new Thread(() -> {
    try {
        String job = jobs.take(); // blocks until a job is available
        System.out.println("processing " + job);
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
}).start();

```

Common implementations:

- **ArrayBlockingQueue**: fixed-capacity, array-backed, FIFO.
- **LinkedBlockingQueue**: optionally bounded, linked-node-backed, FIFO. Generally higher throughput than **ArrayBlockingQueue** under contention.
- **PriorityBlockingQueue**: unbounded, orders elements by priority like **PriorityQueue**, but with blocking `take()`.
- **SynchronousQueue**: zero capacity — a put must wait for a matching take (a direct handoff). Used inside some **ExecutorService** configurations.

- **DelayQueue**: elements become available only after their delay expires — useful for scheduling/retry logic.

ConcurrentLinkedQueue / ConcurrentLinkedDeque

Unbounded, **non-blocking**, thread-safe queues/deques built with CAS (lock-free) operations instead of locks. They implement plain Queue/Deque, not BlockingQueue — there is no put/take that waits; offer/poll return immediately.

```
ConcurrentLinkedQueue<String> queue = new ConcurrentLinkedQueue<>();
queue.offer("task1");
queue.offer("task2");
System.out.println(queue.poll()); // task1, thread-safe, no blocking, no
↪ external lock needed
```

When to use it: high-throughput producer-consumer scenarios where you don't need threads to block and wait — e.g., a metrics collector where producers just fire-and-forget and a consumer periodically drains what's there.

Quick summary table

Class	Blocking?	Bounded?	Typical use
ConcurrentHashMap	no	no	general-purpose concurrent map
CopyOnWriteArrayList	no	no	read-heavy list, e.g. listener lists
ArrayBlockingQueue	yes	yes (fixed)	producer-consumer with backpressure
LinkedBlockingQueue	yes	optional	producer-consumer, general purpose
ConcurrentLinkedQueue	no	no	lock-free, high-throughput, non-blocking
SynchronousQueue	yes	zero capacity	direct handoff between threads

Common Code-Review Interview Pitfalls

1. **Using HashMap/HashSet iteration order as if it were guaranteed.** Why it matters: order can change between JDK versions, or even between runs, and silently breaks logic or tests that depend on it.

```
// Before: relies on unspecified order
Map<String, Integer> m = new HashMap<>();
// ... assumes keys always print in insertion order

// After: use LinkedHashMap if order must be predictable
Map<String, Integer> m = new LinkedHashMap<>();
```

- 2. Modifying a collection while iterating with a for-each loop.** Why it matters: fail-fast iterators detect structural changes and throw `ConcurrentModificationException` — this includes single-threaded code, not just concurrent access.

```
// Before: throws ConcurrentModificationException
for (String s : list) {
    if (s.isEmpty()) list.remove(s);
}

// After: use the iterator's own remove, or removeIf
list.removeIf(String::isEmpty);
// or:
Iterator<String> it = list.iterator();
while (it.hasNext()) {
    if (it.next().isEmpty()) it.remove();
}
```

- 3. Storing mutable objects as `HashMap/HashSet` keys and then mutating them.** Why it matters: the bucket a key lives in is chosen from its hash at insertion time; if the key's hash changes afterward, the entry becomes unreachable (“lost” in the map).

```
// Before: mutable key breaks lookup
List<Integer> key = new ArrayList<>(List.of(1, 2));
Set<List<Integer>> set = new HashSet<>();
set.add(key);
key.add(3); // hashCode changes!
set.contains(key); // often false -- looked up in the wrong
↳ bucket

// After: use immutable keys (e.g., List.copyOf, records, Strings)
List<Integer> key = List.of(1, 2);
```

- 4. Assuming `List.remove(int)` and `List.remove(Object)` do the same thing with `Integer` arguments.** Why it matters: `list.remove(2)` on a `List<Integer>` calls the `int` overload (removes by index), not the `Object` overload (removes by value) — a classic autoboxing trap.

```
List<Integer> nums = new ArrayList<>(List.of(10, 20, 30));
nums.remove(2); // removes INDEX 2 -> [10, 20]
nums.remove(Integer.valueOf(20)); // removes VALUE 20 -> [10]
```

- 5. Not overriding equals()/hashCode() on custom keys/elements.** Why it matters: without them, HashMap/HashSet fall back to identity comparison, so logically-equal objects are treated as distinct.

```
// Before: no equals()/hashCode() -> identity semantics
class Point { int x, y; }

// After: value semantics, works correctly as a map key / set element
record Point(int x, int y) {}
```

- 6. Choosing LinkedList as a default List implementation.** Why it matters: get(index) is O(n) on LinkedList, and it has higher memory overhead per element (node objects with two pointers each) than ArrayList's flat array.

```
// Before: slow random access
List<String> list = new LinkedList<>();

// After: ArrayList is the right default unless you specifically need
// O(1) insert/remove at both ends combined with List semantics
List<String> list = new ArrayList<>();
```

- 7. Using Vector or Stack in new code.** Why it matters: both are legacy, synchronized classes with unnecessary locking overhead for single-threaded use, and Stack extends Vector (an odd, dated design).

```
// Before: legacy, synchronized, slower
Stack<Integer> stack = new Stack<>();

// After: modern, faster, unsynchronized
Deque<Integer> stack = new ArrayDeque<>();
```

- 8. Passing an immutable collection (List.of(...)) somewhere it will be mutated later.** Why it matters: it throws UnsupportedOperationException at runtime, often far from where the immutable list was created, making the bug hard to trace.

```

// Before: crashes deep inside some other method
List<String> config = List.of("a", "b");
someLegacyCodeThatAddsDefaults(config); // throws
↳ UnsupportedOperationException

// After: wrap in a mutable copy if the caller will mutate it
List<String> config = new ArrayList<>(List.of("a", "b"));

```

- 9. Assuming Collections.synchronizedList/synchronizedMap makes compound operations atomic.** Why it matters: each individual call is synchronized, but a sequence like check-then-add is still a race condition across threads.

```

// Before: race condition between contains() and add()
if (!syncList.contains(x)) syncList.add(x);

// After: synchronize the whole compound action, or use a concurrent
↳ collection
synchronized (syncList) {
    if (!syncList.contains(x)) syncList.add(x);
}

```

- 10. Using HashMap/HashSet from multiple threads without synchronization.** Why it matters: concurrent structural modification can corrupt the internal bucket structure, causing silent data loss, infinite loops (older JDKs), or ConcurrentModificationException.

```

// Before: not thread-safe
Map<String, Integer> counters = new HashMap<>();

// After: use a collection designed for concurrency
Map<String, Integer> counters = new ConcurrentHashMap<>();

```

- 11. Putting null into a ConcurrentHashMap or a TreeMap/TreeSet.** Why it matters: ConcurrentHashMap rejects null keys/values outright (ambiguity with “absent”); TreeMap/TreeSet need to *compare* keys, and comparing against null throws NullPointerException.

```

// Before: throws NullPointerException at runtime
new ConcurrentHashMap<String, String>().put("key", null);
new TreeSet<String>().add(null);

// After: use Optional, a sentinel value, or filter out nulls before
↳ inserting

```

- 12. Forgetting that PriorityQueue iteration order is not sorted order.** Why it matters: only `peek()`/`poll()` guarantee the smallest element; iterating the queue (for `(var x : pq)`) or printing it shows heap-array order, which looks unsorted.

```
PriorityQueue<Integer> pq = new PriorityQueue<>(List.of(5, 1, 3));
System.out.println(pq); // NOT guaranteed to print [1, 3, 5]

// To get sorted output, drain with poll():
List<Integer> sorted = new ArrayList<>();
while (!pq.isEmpty()) sorted.add(pq.poll());
```

- 13. Using `Arrays.asList()` and expecting a fully mutable, independent list.** Why it matters: it returns a fixed-size list backed by the array — `add/remove` throw `UnsupportedOperationException`, and `set` writes through to the original array.

```
// Before: surprising behavior
Integer[] arr = {1, 2, 3};
List<Integer> list = Arrays.asList(arr);
list.add(4); // throws UnsupportedOperationException

// After: wrap in a real, resizable, independent list
List<Integer> list = new ArrayList<>(Arrays.asList(arr));
```

- 14. Not pre-sizing collections when the final size is known.** Why it matters: repeated resizing (`ArrayList` array copies, `HashMap` rehashing) wastes CPU and creates avoidable garbage under load.

```
// Before: several resize/copy operations as the list grows
List<String> results = new ArrayList<>();
for (int i = 0; i < 100_000; i++) results.add(fetch(i));

// After: allocate once
List<String> results = new ArrayList<>(100_000);
```

- 15. Assuming `TreeMap/TreeSet` ordering matches `equals()`.** Why it matters: `TreeSet/TreeMap` decide “duplicate” using the `Comparator/compareTo`, not `equals()`. Two elements that compare as 0 are treated as duplicates, even if `equals()` would say they differ.


```

// Before: silently drops "second" because comparator says they're
↳ "equal"
    TreeSet<String> set = new
↳ TreeSet<>(Comparator.comparingInt(String::length));
    set.add("cat");
    set.add("dog"); // same length as "cat" -> compareTo returns 0 ->
↳ treated as duplicate, ignored!
    System.out.println(set); // [cat]

```

- 16. Choosing CopyOnWriteArrayList for a write-heavy workload.** Why it matters: every mutation copies the entire backing array — $O(n)$ per write — which becomes a serious bottleneck as the list grows or writes become frequent.

```

// Before: O(n) per add(), terrible for a write-heavy queue-like list
CopyOnWriteArrayList<String> events = new CopyOnWriteArrayList<>();
for (int i = 0; i < 1_000_000; i++) events.add("event-" + i);

// After: use a concurrent queue built for frequent writes
ConcurrentLinkedQueue<String> events = new ConcurrentLinkedQueue<>();

```

- 17. Relying on Map.get() returning null to mean “absent” when null values are also stored.** Why it matters: you can’t tell “key missing” apart from “key present with a null value” using get() alone, which leads to subtle bugs.

```

// Before: ambiguous
if (map.get("key") == null) { /* missing, or present-with-null? unclear
↳ */ }

// After: use containsKey, or getOrDefault with a real sentinel, and
↳ avoid
// storing null values in the first place when possible
if (!map.containsKey("key")) { /* definitely missing */ }

```

- 18. Using a plain Iterator.remove() loop when removeIf says exactly what you mean.** Why it matters: not a correctness bug, but a clarity/maintainability one that reviewers flag — removeIf is shorter, less error-prone (no risk of forgetting to call it.remove() and hitting ConcurrentModificationException), and communicates intent directly.

```

// Before: verbose, easy to get wrong
Iterator<String> it = names.iterator();
while (it.hasNext()) {

```

```
    if (it.next().isBlank()) it.remove();  
}
```

```
// After: same result, clearer intent  
names.removeIf(String::isBlank);
```

Functional Programming

Java added lambdas and the Streams API in Java 8, and it has kept building on them ever since. These features let you write code that describes *what* you want to happen instead of *how* to loop and mutate to get there. This style is called “functional programming” — you pass behavior around as values, instead of only passing data. This chapter is a fast, practical tour of the pieces you need to read and write this kind of code confidently, and to catch the subtle mistakes reviewers are expected to flag.

All examples target Java 21+.

Functional Interfaces

A **functional interface** is an interface with exactly one abstract method (often called a “SAM” — single abstract method). It can have any number of default or static methods too — those don’t count. Java uses functional interfaces as the “shape” that a lambda expression or method reference plugs into.

The `@FunctionalInterface` annotation is optional, but you should always add it to interfaces you design for this purpose. It doesn’t change runtime behavior — it just tells the compiler “this interface must have exactly one abstract method.” If someone later adds a second abstract method, the build fails immediately instead of breaking lambdas elsewhere in the codebase at a mysterious call site.

```
@FunctionalInterface
interface Validator<T> {
    boolean isValid(T value);

    // default methods are allowed - they don't break the "single abstract
    ↪ method" rule
    default Validator<T> negate() {
        return value -> !isValid(value);
    }
}
```

```
}

```

If you try to add a second abstract method, the compiler rejects it:

```
@FunctionalInterface
interface Broken<T> {
    boolean isValid(T value);
    boolean isValid(T value); // compile error: Broken is not a
    ↪ functional interface
}
```

The java.util.function catalogue

Java ships a standard set of general-purpose functional interfaces in `java.util.function` so you rarely need to declare your own. Learning this table is worth it — reviewers expect you to reach for the right one instead of inventing a custom interface every time.

Interface	Abstract method	Signature (conceptually)	Typical use
Function<T, R>	apply	T -> R	Transform a value
BiFunction<T, U, R>	apply	(T, U) -> R	Combine two values into one
Supplier<T>	get	() -> T	Produce/lazily create a value
Consumer<T>	accept	T -> void	Do something with a value, no result
BiConsumer<T, U>	accept	(T, U) -> void	Do something with two values
Predicate<T>	test	T -> boolean	Yes/no check on a value
BiPredicate<T, U>	test	(T, U) -> boolean	Yes/no check on two values
UnaryOperator<T>	apply	T -> T	Function<T, T> specialization, same input/output type
BinaryOperator<T>	apply	(T, T) -> T	BiFunction<T, T, T> specialization, used in reduce

Boxing `int/long/double` into `Integer/Long/Double` costs memory and CPU (see [Stream Performance](#)). To avoid it, the JDK also provides primitive specializations:

Interface	Purpose
IntFunction<R>, LongFunction<R>, DoubleFunction<R>	int/long/double -> R
ToIntFunction<T>, ToLongFunction<T>, ToDoubleFunction<T>	T -> int/long/double
IntUnaryOperator, LongUnaryOperator, DoubleUnaryOperator	int -> int, etc.
IntBinaryOperator, LongBinaryOperator, DoubleBinaryOperator	(int, int) -> int, etc.
IntPredicate, LongPredicate, DoublePredicate	int -> boolean, etc.
IntConsumer, LongConsumer, DoubleConsumer	int -> void, etc.
IntSupplier, LongSupplier, DoubleSupplier	() -> int, etc.
ObjIntConsumer<T>, ObjLongConsumer<T>, ObjDoubleConsumer<T>	(T, int) -> void, etc.

```
import java.util.function.*;

Function<String, Integer> length = String::length;
BiFunction<Integer, Integer, Integer> add = Integer::sum;
Supplier<List<String>> newList = ArrayList::new;
Consumer<String> print = System.out::println;
Predicate<String> isBlank = String::isBlank;
UnaryOperator<String> upper = String::toUpperCase;
BinaryOperator<Integer> max = Integer::max;
IntPredicate isEven = n -> n % 2 == 0;

System.out.println(length.apply("hello")); // 5
System.out.println(add.apply(2, 3));      // 5
System.out.println(isEven.test(4));       // true
```

Writing your own

Reach for a custom functional interface only when none of the standard ones fit — usually because you need a *named*, self-documenting method, or you need to throw a checked exception.

```
@FunctionalInterface
interface RowMapper<T> {
```

```

    T map(ResultSet row) throws SQLException; // checked exception allowed
↪ here
    }

    RowMapper<String> nameMapper = row -> row.getString("name");

```

Lambda Expressions

A **lambda expression** is an anonymous, inline implementation of a functional interface's single method. Syntax ranges from terse to explicit:

```

    Runnable r1 = () -> System.out.println("run"); // no
↪ params
    Function<Integer, Integer> square = x -> x * x; // one
↪ param, inferred type
    Function<Integer, Integer> square2 = (Integer x) -> x * x; // one
↪ param, explicit type
    BiFunction<Integer, Integer, Integer> add = (a, b) -> a + b; // two
↪ params, block-less
    BiFunction<Integer, Integer, Integer> addBlock = (a, b) -> { // block
↪ body, needs return
        int sum = a + b;
        return sum;
    };

```

Rules of thumb:

- Parentheses around a single inferred parameter are optional: `x -> x * x` and `(x) -> x * x` are the same.
- If you type one parameter, type them all `((a, b)` or `(int a, int b)`, not `(a, int b)`).
- A block body `{ ... }` needs an explicit `return` (unless the return type is `void`); an expression body does not.

Effectively-final capture

A lambda can read local variables from its enclosing scope, but only if those variables are **effectively final** — assigned exactly once, even without the `final` keyword. This is called “capturing” a variable.

```
int factor = 10; // effectively final: never reassigned
Function<Integer, Integer> scale = x -> x * factor; // OK, captures factor

int counter = 0;
Runnable bad = () -> {
    // counter++; // compile error: counter is not effectively final
};
```

Why the restriction exists: the lambda might run later, or on another thread. Java captures a *snapshot* of the variable's value (for primitives/references) rather than a live reference to the variable itself, so it forbids anything that could make that snapshot stale or ambiguous. If you need a mutable counter across calls, wrap it in a container:

```
int[] counter = {0};
Runnable increment = () -> counter[0]++; // OK: counter (the array
↳ reference) never changes
increment.run();
increment.run();
System.out.println(counter[0]); // 2
```

Fields (instance or static) are exempt from this rule — only *local* variables and parameters must be effectively final.

this semantics vs anonymous classes

A lambda does **not** introduce its own `this`. Inside a lambda, `this` refers to the enclosing instance — exactly as if the lambda's body were pasted directly into the surrounding method. An anonymous inner class, by contrast, introduces a new `this` that refers to the anonymous class instance itself.

```
class Counter {
    int value = 0;

    Runnable lambdaVersion() {
        return () -> {
            this.value++; // 'this' is the Counter instance
            System.out.println(this.getClass()); // class Counter
        };
    }

    Runnable anonymousVersion() {
        return new Runnable() {
            int value = 100; // shadows Counter.value inside this class
        };
    }
}
```

```

        @Override
        public void run() {
            this.value++;           // 'this' is the anonymous Runnable
            System.out.println(this.getClass()); // class Counter$1
        }
    }
}

```

↪ (anonymous)

This matters in code review: if you convert an anonymous class to a lambda, double check any use of `this` — the meaning can change and silently start referring to a different object.

No checked exceptions

None of the standard `java.util.function` interfaces declare `throws`. If the method you're calling inside a lambda throws a checked exception, you must catch it, wrap it in an unchecked exception, or use a custom functional interface that declares `throws`.

```

List<String> paths = List.of("a.txt", "b.txt");

// Function<String, byte[]> reader = Files::readAllBytes; // won't compile:
// readAllBytes throws IOException

```

↪ `readAllBytes` throws `IOException`

```

Function<String, byte[]> reader = path -> {
    try {
        return Files.readAllBytes(Path.of(path));
    } catch (IOException e) {
        throw new UncheckedIOException(e); // wrap checked -> unchecked
    }
};

```

Method References

A **method reference** is shorthand for a lambda that just calls one existing method. It uses the `::` operator. It's not a separate concept from lambdas — the compiler converts it to the same kind of functional-interface implementation. Use it whenever a lambda would do nothing but forward its argument(s) to an existing method.

Kind	Syntax	Equivalent lambda	Example
Static method	ClassName:: staticMethod	(args) -> ClassName. staticMethod(args)	Integer:: parseInt
Bound instance method	instance::method	(args) -> instance. method(args)	myList::add
Unbound instance method	ClassName:: instanceMethod	(obj, args) -> obj. instanceMethod(args)	String:: toUpperCase
Constructor	ClassName::new	(args) -> new ClassName(args)	ArrayList:: new

```

// 1. Static method reference
Function<String, Integer> parse = Integer::parseInt;
System.out.println(parse.apply("42")); // 42

// 2. Bound instance method reference - the receiver is a specific,
→ already-existing object
List<String> names = new ArrayList<>();
Consumer<String> addToNames = names::add;
addToNames.accept("Ada");
System.out.println(names); // [Ada]

// 3. Unbound instance method reference - the receiver becomes the first
→ parameter
Function<String, String> toUpper = String::toUpperCase;
System.out.println(toUpper.apply("hi")); // HI

BiFunction<String, String, Boolean> startsWith = String::startsWith;
System.out.println(startsWith.apply("hello", "he")); // true

// 4. Constructor reference
Supplier<List<String>> listFactory = ArrayList::new;
List<String> fresh = listFactory.get();

Function<String, StringBuilder> sbFactory = StringBuilder::new;
System.out.println(sbFactory.apply("go").append("!")); // go!

```

A quick way to tell #2 from #3: if the part before `::` is a *variable* (an existing object), it's bound. If it's a *type name*, it's unbound and the receiver becomes the method's first parameter.

Streams API

A **stream** is a sequence of elements that supports functional-style operations — map, filter, reduce, and so on — computed in a pipeline. A stream is *not* a data structure; it doesn't store elements. It's a view over a source (a collection, an array, an I/O channel, a generator function) that computes results on demand.

Creating streams

```
Stream<String> fromValues = Stream.of("a", "b", "c");
Stream<String> fromList = List.of("a", "b", "c").stream();
Stream<String> fromArray = Arrays.stream(new String[]{"a", "b", "c"});
Stream<String> empty = Stream.empty();
IntStream fromRange = IntStream.range(0, 5);           // 0,1,2,3,4
IntStream fromRangeClosed = IntStream.rangeClosed(1, 5); // 1,2,3,4,5
```

Intermediate vs terminal operations

Intermediate operations (map, filter, sorted, ...) return a new stream and are *lazy* — they don't run anything by themselves. **Terminal operations** (collect, forEach, reduce, count, ...) trigger the pipeline to actually execute and produce a result or side effect. A stream pipeline with no terminal operation does nothing at all.

```
Stream<String> pipeline = List.of("banana", "apple", "cherry").stream()
    .filter(s -> s.length() > 5) // nothing happens yet
    .map(String::toUpperCase);    // still nothing happens

List<String> result = pipeline.collect(Collectors.toList()); // NOW it
↳ runs
System.out.println(result); // [BANANA, CHERRY]
```

Laziness and short-circuiting

Because intermediate ops are lazy, the pipeline processes one element at a time, end-to-end, rather than materializing an intermediate list at every stage. Combined with **short-circuiting** operations (findFirst, anyMatch, limit, ...), this means a stream can stop early without touching the rest of the source.

```
Optional<Integer> firstBig = Stream.of(1, 2, 3, 4, 5, 6)
    .peek(n -> System.out.println("checking " + n))
    .filter(n -> n > 3)
    .findFirst();
// Output:
// checking 1
// checking 2
// checking 3
// checking 4
// (stops here - findFirst is short-circuiting)
System.out.println(firstBig); // Optional[4]
```

map, filter, flatMap, mapMulti

- **map**: transform each element 1-to-1.
- **filter**: keep elements matching a Predicate.
- **flatMap**: transform each element into a *stream*, then flatten all those streams into one (1-to-many, or 1-to-zero).
- **mapMulti** (Java 16+): an imperative alternative to **flatMap** for cases where building an intermediate stream is wasteful — you push zero or more results into a consumer yourself.

```
List<List<Integer>> nested = List.of(List.of(1, 2), List.of(3, 4),
↳ List.of());

List<Integer> flat = nested.stream()
    .flatMap(List::stream)
    .toList();
System.out.println(flat); // [1, 2, 3, 4]

List<Integer> viaMapMulti = nested.stream()
    .<Integer>mapMulti((list, consumer) -> list.forEach(consumer))
    .toList();
System.out.println(viaMapMulti); // [1, 2, 3, 4]
```

sorted, distinct

```
List<String> words = List.of("pear", "fig", "apple", "fig", "date");

List<String> sortedDistinct = words.stream()
    .distinct() // relies on equals()/hashCode()
    .sorted(Comparator.comparing(String::length).thenComparing(Comparator_
↪ .naturalOrder()))
    .toList();
System.out.println(sortedDistinct); // [fig, date, pear, apple]
```

`sorted()` on a stream is a *stateful* intermediate op — it must buffer the whole stream before it can emit anything, so it doesn't play well with infinite streams unless combined with `limit` first.

limit/skip, takeWhile/dropWhile

`limit/skip` cut by *position*. `takeWhile/dropWhile` (Java 9+) cut by a *predicate*, and both stop evaluating the predicate as soon as it flips (they assume, but do not enforce, that the input is suitably ordered for the use case).

```
List<Integer> nums = List.of(1, 2, 3, 10, 4, 5);

System.out.println(nums.stream().skip(1).limit(2).toList()); // [2,
↪ 3]
System.out.println(nums.stream().takeWhile(n -> n < 5).toList()); // [1, 2,
↪ 3] - stops at 10
System.out.println(nums.stream().dropWhile(n -> n < 5).toList()); // [10,
↪ 4, 5] - drops until first >= 5, then keeps rest
```

peek

`peek` lets you look at each element as it flows through, without transforming it. It exists for debugging a pipeline — **not** for side effects that affect the program's result. Two reasons:

1. If the stream is short-circuited (e.g., a later `findFirst`), `peek` may not run for every element.
2. Implementations are free to skip calling `peek` at all if they can prove the pipeline's result doesn't depend on it (an optimization the JDK explicitly reserves the right to make).

```
// Fine: debugging only, no logic depends on it
long count = Stream.of("a", "bb", "ccc")
    .peek(s -> System.out.println("seen: " + s))
    .count(); // JDK may actually SKIP peek entirely here since count
↳ doesn't need the elements!
```

reduce

reduce folds a stream down to a single value using a combining function.

```
List<Integer> nums = List.of(1, 2, 3, 4, 5);

int sum = nums.stream().reduce(0, Integer::sum); // identity
↳ + accumulator
System.out.println(sum); // 15

Optional<Integer> product = nums.stream().reduce((a, b) -> a * b); // no
↳ identity -> Optional
System.out.println(product); // Optional[120]

// Three-arg form: identity, accumulator, combiner (combiner used only
↳ when running in parallel)
int totalLength = Stream.of("a", "bb", "ccc")
    .reduce(0, (acc, s) -> acc + s.length(), Integer::sum);
System.out.println(totalLength); // 6
```

findFirst vs findAny

Both are short-circuiting and return `Optional<T>`. `findFirst` always returns the first matching element in encounter order. `findAny` may return *any* matching element — it exists so that a parallel stream can return as soon as one thread finds a match, without waiting to determine which was truly “first.” On a sequential stream they usually behave the same, but only `findFirst` is guaranteed to.

```
List<Integer> nums = List.of(1, 2, 3, 4, 5);

Optional<Integer> first = nums.parallelStream().filter(n -> n % 2 ==
↳ 0).findFirst(); // always 2
Optional<Integer> any = nums.parallelStream().filter(n -> n % 2 ==
↳ 0).findAny(); // 2 or 4, whichever thread finished first
```

Primitive streams

`IntStream`, `LongStream`, `DoubleStream` avoid boxing overhead and add numeric operations (sum, average, max, min, summaryStatistics). Convert between object streams and primitive streams with `mapToInt`/`mapToObj`/`boxed`.

```
int[] values = {3, 1, 4, 1, 5, 9};

IntSummaryStatistics stats = IntStream.of(values).summaryStatistics();
System.out.println(stats.getSum());      // 23
System.out.println(stats.getAverage());  // 3.8333...
System.out.println(stats.getMax());      // 9

List<Integer> boxed = IntStream.of(values).boxed().toList(); // [3, 1, 4,
↳ 1, 5, 9]

List<String> words = List.of("a", "bb", "ccc");
int totalChars = words.stream().mapToInt(String::length).sum(); // 6
```

Stream.iterate/Stream.generate

Both produce potentially *infinite* streams — always pair them with `limit`, or with `takeWhile` on `iterate`, or you'll hang the program.

```
List<Integer> powersOfTwo = Stream.iterate(1, n -> n * 2)
    .limit(5)
    .toList();
System.out.println(powersOfTwo); // [1, 2, 4, 8, 16]

// Java 9+ three-arg iterate: seed, hasNext predicate (built-in
↳ short-circuit), next function
List<Integer> below100 = Stream.iterate(1, n -> n < 100, n -> n *
↳ 2).toList();
System.out.println(below100); // [1, 2, 4, 8, 16, 32, 64]

Stream<Double> randoms = Stream.generate(Math::random).limit(3);
```

Optional interop

Streams and `Optional` are designed to work together. `findFirst`, `reduce` (no identity), `max`, `min` all return `Optional`. `Optional` itself has a `stream()` method (0 or 1 element), handy for flattening a stream of `Optionals`.

```

List<Optional<String>> maybeNames = List.of(Optional.of("Ada"),
↳ Optional.empty(), Optional.of("Linus"));

List<String> presentOnly = maybeNames.stream()
    .flatMap(Optional::stream) // Optional.stream() turns Optional<T> into
↳ Stream<T> of 0 or 1 elements
    .toList();
System.out.println(presentOnly); // [Ada, Linus]

```

Streams are single-use

A stream can be consumed by a terminal operation exactly once. Reusing a stream throws `IllegalStateException`. If you need to run the pipeline twice, build it from the source again.

```

Stream<String> stream = Stream.of("a", "b", "c");
stream.forEach(System.out::println);
// stream.count(); // throws IllegalStateException: stream has already
↳ been operated upon or closed

// Correct: create a new stream from the source each time
List<String> source = List.of("a", "b", "c");
source.stream().forEach(System.out::println);
long count = source.stream().count();

```

Collectors

A **collector** is a recipe for how to accumulate the elements of a stream into a result — a List, a Map, a single summary number, a String, or anything else. You use one via the terminal `collect(...)` operation. `Collectors` is the factory class of built-in recipes.

toList/toSet/toMap

```

List<String> list = Stream.of("a", "b",
↳ "a").collect(Collectors.toList()); // mutable, allows duplicates
List<String> immutableList = Stream.of("a", "b", "a").toList();
↳ // Java 16+ shortcut, unmodifiable

Set<String> set = Stream.of("a", "b", "a").collect(Collectors.toSet()); //
↳ [a, b] - duplicates dropped

```

```
Map<String, Integer> byLength = Stream.of("fig", "date", "kiwi")
    .collect(Collectors.toMap(Function.identity(), String::length));
System.out.println(byLength); // {date=4, fig=3, kiwi=4} (order not
↳ guaranteed)
```

`Collectors.toMap` throws `IllegalStateException` if two elements produce the same key — because it has no idea how you'd like the conflict resolved. Fix it by supplying a merge function.

```
List<String> words = List.of("fig", "kiwi", "date", "pear"); // "fig" and
↳ "pear" both length 4? no - kiwi/date/pear length 4

// This throws: two words with the same length collide as keys
// Map<Integer, String> byLen = words.stream()
//     .collect(Collectors.toMap(String::length, w -> w)); //
↳ IllegalStateException: Duplicate key 4

// Fix: provide a merge function to resolve collisions
Map<Integer, String> byLenFixed = words.stream()
    .collect(Collectors.toMap(String::length, w -> w, (existing,
↳ incoming) -> existing + "," + incoming));
System.out.println(byLenFixed); // {3=fig, 4=kiwi,date,pear}
```

groupingBy with downstream collectors

`groupingBy` buckets elements by a classifier function into a `Map<K, List<T>>` by default. Pass a **downstream collector** as a second argument to summarize each bucket instead of just listing it.

```
record Employee(String department, String name, double salary) {}

List<Employee> employees = List.of(
    new Employee("Eng", "Ada", 9000),
    new Employee("Eng", "Linus", 9500),
    new Employee("Sales", "Grace", 7000)
);

Map<String, List<String>> namesByDept = employees.stream()
    .collect(Collectors.groupingBy(Employee::department,
↳ Collectors.mapping(Employee::name, Collectors.toList())));
System.out.println(namesByDept); // {Eng=[Ada, Linus], Sales=[Grace]}

Map<String, Double> totalSalaryByDept = employees.stream()
```



```

        .collect(Collectors.groupingBy(Employee::department,
    ↪ Collectors.summingDouble(Employee::salary)));
    System.out.println(totalSalaryByDept); // {Eng=18500.0, Sales=7000.0}

```

partitioningBy

A special case of grouping into exactly two buckets: true and false. Both keys always appear, even if a bucket is empty.

```

Map<Boolean, List<Integer>> evenOdd = IntStream.rangeClosed(1, 6).boxed()
    .collect(Collectors.partitioningBy(n -> n % 2 == 0));
System.out.println(evenOdd); // {false=[1, 3, 5], true=[2, 4, 6]}

```

joining, counting, summingInt, averagingDouble

```

String csv = Stream.of("a", "b", "c").collect(Collectors.joining(", ",
    ↪ "[", "]"));
System.out.println(csv); // [a, b, c]

long count = Stream.of("a", "b", "c").collect(Collectors.counting());
    ↪ // 3
int totalLen = Stream.of("aa", "b",
    ↪ "ccc").collect(Collectors.summingInt(String::length)); // 6
double avgLen = Stream.of("aa", "b",
    ↪ "ccc").collect(Collectors.averagingDouble(String::length)); // 2.0

```

mapping, flatMapping, filtering

These are **downstream collectors** — meant to be nested inside groupingBy/partitioningBy to reshape or pre-process each bucket before the final collector runs.

```

record Order(String customer, List<String> items) {}

List<Order> orders = List.of(
    new Order("Ada", List.of("pen", "paper")),
    new Order("Ada", List.of("stapler")),
    new Order("Grace", List.of("pen"))
);

// flatMapping: flatten each customer's nested item lists into one flat
    ↪ set

```

```

Map<String, Set<String>> itemsByCustomer = orders.stream()
    .collect(Collectors.groupingBy(Order::customer,
        Collectors.flatMapping(o -> o.items().stream(),
↳ Collectors.toSet())));
System.out.println(itemsByCustomer); // {Ada=[pen, paper, stapler],
↳ Grace=[pen]}

// filtering: keep only orders with more than one item, per customer,
↳ before collecting
Map<String, List<Order>> multiItemOrdersByCustomer = orders.stream()
    .collect(Collectors.groupingBy(Order::customer,
        Collectors.filtering(o -> o.items().size() > 1,
↳ Collectors.toList())));
System.out.println(multiItemOrdersByCustomer); // {Ada=[Order[customer=Ada,
↳ items=[pen, paper]]], Grace=[]}

```

Note the difference from `stream().filter(...)` *before* `groupingBy`: filtering before grouping can make a whole group disappear; `Collectors.filtering` keeps the group key present (as shown above, `Grace=[]` still shows up).

teeing

`teeing` (Java 12+) runs a stream through *two* collectors simultaneously and merges their results with a `BiFunction`. Useful when you need two different summaries from a single pass.

```

record MinMax(int min, int max) {}

MinMax result = Stream.of(4, 1, 7, 3, 9, 2)
    .collect(Collectors.teeing(
        Collectors.minBy(Integer::compareTo),
        Collectors.maxBy(Integer::compareTo),
        (min, max) -> new MinMax(min.orElseThrow(), max.orElseThrow())
    ));
System.out.println(result); // MinMax[min=1, max=9]

```

collectingAndThen

Wraps a collector and applies a finishing transformation to its result — commonly used to make a mutable result immutable.

```
List<String> immutableUpper = Stream.of("b", "a", "c")
    .collect(Collectors.collectingAndThen(Collectors.toList(),
    ↪ Collections.unmodifiableList));
    // immutableUpper.add("x"); // throws UnsupportedOperationException
```

Writing a custom Collector

`Collector<T, A, R>` needs four ingredients: a **supplier** (create the mutable accumulator), an **accumulator** (fold one element in), a **combiner** (merge two accumulators — used in parallel streams), and a **finisher** (convert the accumulator to the final result).

```
Collector<String, StringBuilder, String> customJoiner = Collector.of(
    StringBuilder::new,                // supplier
    (sb, s) -> sb.append(s).append('|'), // accumulator
    (sb1, sb2) -> sb1.append(sb2),     // combiner (for parallel
    ↪ merging)
    StringBuilder::toString            // finisher
);

String result = Stream.of("a", "b", "c").collect(customJoiner);
System.out.println(result); // a|b|c|
```

For most cases, though, composing the built-in collectors (as shown above) is clearer and less error-prone than hand-rolling one.

Splitterator

A **Splitterator** (“splittable iterator”) is the low-level engine that powers both sequential and parallel stream traversal. You rarely write one yourself, but understanding it explains *why* some sources parallelize well and others don’t.

- `tryAdvance(Consumer<? super T> action)`: process the next element if there is one and return true; return false if the source is exhausted. This is how sequential traversal works — one element at a time, like `Iterator.next()` but pushing the value into a callback instead of returning it.
- `trySplit()`: attempt to carve off a chunk of the remaining elements into a *new* Splitterator that a different thread can process, returning null if the source can’t (or won’t) be split further. This is the method parallel streams call to divide work across the common fork/join pool.

- **Characteristics:** a bitmask describing properties of the source — ORDERED, SORTED, SIZED (size known up front), SUBSIZED (splits also report exact size), DISTINCT, NONNULL, IMMUTABLE, CONCURRENT. Stream operations use these as hints to skip unnecessary work — e.g., `distinct()` on a stream whose spliterator reports DISTINCT is a no-op.

```
List<Integer> data = List.of(1, 2, 3, 4, 5, 6, 7, 8);
Spliterator<Integer> spliterator = data.spliterator();

System.out.println(spliterator.estimateSize()); // 8
System.out.println(spliterator.characteristics() & Spliterator.SIZED); //
↳ non-zero: SIZED is set

Spliterator<Integer> firstHalf = spliterator.trySplit(); // spliterator
↳ now covers the second half
firstHalf.forEachRemaining(n -> System.out.print(n + " ")); // 1 2 3 4
System.out.println();
spliterator.forEachRemaining(n -> System.out.print(n + " ")); // 5 6 7 8
```

When you'd implement one

You'd write a custom `Spliterator` when you're exposing a custom data source as a `Stream` — for example, streaming rows from a specialized data structure (a skip list, a memory-mapped file, a custom ring buffer) and you want that stream to parallelize efficiently. You'd implement `trySplit()` to divide the source in a way that respects its actual structure, and report accurate characteristics so downstream operations can optimize correctly. Getting SIZED/SUBSIZED wrong, for instance, can make parallel splitting produce badly unbalanced chunks.

```
class RangeSpliterator implements Spliterator<Integer> {
    private int current;
    private final int end;

    RangeSpliterator(int start, int end) {
        this.current = start;
        this.end = end;
    }

    @Override
    public boolean tryAdvance(Consumer<? super Integer> action) {
        if (current >= end) return false;
        action.accept(current++);
        return true;
    }
}
```

```

    }

    @Override
    public Spliterator<Integer> trySplit() {
        int remaining = end - current;
        if (remaining < 2) return null; // too small to split
        int mid = current + remaining / 2;
        Spliterator<Integer> firstHalf = new RangeSpliterator(current,
    ↪ mid);
        this.current = mid; // this spliterator keeps the second half
        return firstHalf;
    }

    @Override
    public long estimateSize() { return end - current; }

    @Override
    public int characteristics() {
        return ORDERED | SIZED | SUBSIZED | IMMUTABLE | NONNULL;
    }
}

Stream<Integer> customStream = StreamSupport.stream(new
    ↪ RangeSpliterator(0, 10), false);
System.out.println(customStream.mapToInt(Integer::intValue).sum()); // 45

```

Stream Performance

Streams read nicely, but they are not free. Every stage adds a virtual call, and for simple operations on already-in-memory data, a classic `for` loop is usually faster and, more importantly, has fewer surprising costs.

When streams are slower than loops

For small collections or trivial operations, the overhead of building the pipeline (lambda allocation, boxing, virtual dispatch through each stage) can outweigh the benefit of readability. In hot paths — code that runs millions of times per second — measure before assuming a stream refactor is free.

```

int[] data = {1, 2, 3, 4, 5};

// Loop: simple, no allocation beyond the primitive array
int sumLoop = 0;

```

```

    for (int n : data) sumLoop += n;

    // Stream: clean, but goes through IntStream machinery + a lambda for a
    ↪ trivial task
    int sumStream = IntStream.of(data).sum();

```

Both are fine here — the point is that for tiny, simple aggregations, a loop is not something to apologize for in review.

Boxing costs

`Stream<Integer>` stores boxed `Integer` objects; every `int` you put into it gets wrapped in an object, and every arithmetic operation on it may need to unbox, compute, and re-box. `IntStream/LongStream/DoubleStream` avoid this entirely by keeping primitives unboxed through the pipeline.

```

List<Integer> numbers = List.of(1, 2, 3, 4, 5);

// Boxes every element into Integer, unboxes for the sum
int slow = numbers.stream().reduce(0, Integer::sum);

// No boxing at all - stays as int the whole way
int fast = numbers.stream().mapToInt(Integer::intValue).sum();

```

Splittability of sources

Parallel streams rely on `trySplit()` dividing work into roughly equal, cheaply-computed chunks (see [Splititerator](#)). Not all sources split well:

- **Good:** `ArrayList`, arrays, `IntStream.range` — random access means splitting a range in half is $O(1)$ and produces balanced chunks.
- **Bad:** `LinkedList`, `Iterator`-based sources, `Stream.iterate` — no random access, so splitting requires walking elements one at a time, which defeats the purpose, or can't be split at all.

```

List<Integer> arrayList = new ArrayList<>(List.of(1, 2, 3, 4, 5, 6, 7,
↪ 8));
List<Integer> linkedList = new LinkedList<>(arrayList);

arrayList.parallelStream().forEach(n -> {}); // splits into balanced
↪ chunks cheaply - scales well

```

```

    linkedList.parallelStream().forEach(n -> {}); // trySplit degrades to
    ↪ linear traversal - little/no benefit

```

The common ForkJoinPool and why blocking is dangerous

`parallelStream()` doesn't spin up its own threads — by default it submits work to the JVM-wide **common ForkJoinPool**, sized to `Runtime.getRuntime().availableProcessors() - 1` worker threads. That pool is shared by *every* parallel stream in the whole application (and by anything else using `ForkJoinPool.commonPool()` directly, including `CompletableFuture`'s `async` methods).

If a task inside a parallel stream **blocks** — a network call, a JDBC query, a `Thread.sleep`, waiting on a lock — it occupies one of those few shared worker threads and can starve unrelated parallel work elsewhere in the app. Since the pool is small and shared, one badly-behaved parallel stream can quietly degrade performance for the whole application, not just itself.

```

    // Dangerous: blocking I/O inside a parallel stream ties up common pool
    ↪ threads
    List<String> urls = List.of("https://a", "https://b", "https://c");
    List<String> bodies = urls.parallelStream()
        .map(url -> httpClient.get(url)) // blocks a shared ForkJoinPool
    ↪ worker thread per call
        .toList();

    // Safer: use a dedicated executor for blocking work, keep the common pool
    ↪ free
    ExecutorService ioPool = Executors.newFixedThreadPool(10);
    List<CompletableFuture<String>> futures = urls.stream()
        .map(url -> CompletableFuture.supplyAsync(() -> httpClient.get(url),
    ↪ ioPool))
        .toList();
    List<String> results =
    ↪ futures.stream().map(CompletableFuture::join).toList();

```

Stateful lambdas and shared mutable state

Passing a lambda that mutates shared state into a stream operation — especially a parallel one — is a race condition waiting to happen. Streams give no guarantee about thread-safety of your own captured state; that's on you.

```

// BROKEN: ArrayList is not thread-safe, and multiple threads add()
↳ concurrently
List<Integer> results = new ArrayList<>();
IntStream.range(0, 10_000).parallel().forEach(results::add); // corrupts
↳ internal state, may throw or lose elements

// FIXED: let the stream own the accumulation via a proper collector
List<Integer> results2 = IntStream.range(0,
↳ 10_000).parallel().boxed().collect(Collectors.toList());

```

The rule: lambdas passed to stream operations should be **stateless** with respect to shared mutable data. Let collect/reduce do the accumulating — those are designed to combine partial results safely across threads.

forEach vs forEachOrdered

On a parallel stream, `forEach` makes **no promise about order** — elements are processed as threads happen to finish, which is faster but unpredictable. `forEachOrdered` forces processing back into encounter order, which reintroduces most of the coordination overhead you were trying to avoid by going parallel.

```

List<Integer> nums = List.of(1, 2, 3, 4, 5);

nums.parallelStream().forEach(System.out::println);           // order NOT
↳ guaranteed, e.g. 3 1 4 2 5
nums.parallelStream().forEachOrdered(System.out::println); // always 1 2 3
↳ 4 5, but loses most parallel speedup

```

If you need parallel speed and ordered output, prefer collecting to a `List` (which preserves encounter order) over forcing `forEachOrdered`.

Parallel Streams

A **parallel stream** splits the source, processes chunks on multiple threads from the common `ForkJoinPool`, then merges the partial results. `stream()` and `parallelStream()` build the *same* pipeline API — the parallel-ness is just a flag, toggled with `.parallel()` / `.sequential()` at any point in the chain (the *last* call before the terminal operation wins).


```
List<Integer> nums = IntStream.rangeClosed(1, 20).boxed().toList();

int sumSequential = nums.stream().mapToInt(Integer::intValue).sum();
int sumParallel = nums.parallelStream().mapToInt(Integer::intValue).sum();
System.out.println(sumSequential == sumParallel); // true - same result,
↳ different execution
```

When parallel actually pays off — the “N × Q” rule of thumb

Parallelism has fixed costs: splitting the source, scheduling tasks, merging results. It only pays off when the *total work* is large enough to amortize those costs. A widely used heuristic (from Brian Goetz and the JDK’s own stream design notes) is:



N × Q > 10,000 (roughly), where **N** = number of elements and **Q** = cost of processing one element.

- Large **N**, cheap **Q** (e.g., summing a million ints): parallel can help, but boxing and splitting overhead may eat the gain — measure.
- Small **N**, expensive **Q** (e.g., 50 items, each needing a slow computation): parallel can help a lot, since 50 splits handled by several threads still swamps the coordination cost.
- Small **N**, cheap **Q** (e.g., summing 10 numbers): parallel almost always loses — pure overhead, no upside.
- Large **N**, expensive **Q**: parallel usually wins clearly.

```
// Small N, cheap Q: parallel overhead dominates - don't bother
int tinySum = IntStream.rangeClosed(1, 10).parallel().sum(); // slower
↳ than sequential in practice

// Large N, expensive Q per element: parallel is likely to help
List<Long> primesNear = LongStream.rangeClosed(2, 2_000_000)
    .parallel()
    .filter(BigIntegerUtil::isPrimeSlowCheck) // pretend this is CPU-heavy
↳ per element
    .boxed()
    .toList();
```

Always benchmark with something like JMH before trusting intuition — modern JITs and cache effects can make the “obvious” answer wrong, especially at small N. Never guess in a code review either way; ask for a benchmark or a clear justification when a PR reaches for `.parallel()`.

Common Code-Review Interview Pitfalls

1. Missing `@FunctionalInterface`, so a second abstract method sneaks in later.

Why it matters: without the annotation, the compiler won’t stop a teammate from breaking every lambda that implements the interface.

```
// Before
interface Handler<T> { void handle(T t); }

// After
@FunctionalInterface
interface Handler<T> { void handle(T t); }
```

2. Capturing a mutable loop variable or field expecting per-iteration semantics.

Why it matters: lambdas capture variables, not values in the “live” sense for fields, so shared mutable state read later can be wrong or racy.

```
// Before - all callbacks share and read the mutable field at call time,
↳ likely all seeing the final value
for (int i = 0; i < tasks.size(); i++) {
    this.currentIndex = i;
    tasks.get(i).setCallback(() -> process(this.currentIndex));
}

// After - capture an effectively-final local per iteration
for (int i = 0; i < tasks.size(); i++) {
    int index = i; // effectively final, one per iteration
    tasks.get(i).setCallback(() -> process(index));
}
```

3. Using `peek` to drive actual program logic instead of debugging.

Why it matters: `peek` may be skipped by the JDK if the result doesn’t depend on it, and short-circuiting can stop it from ever running for all elements.

```
// Before - relies on peek to populate a list as a side effect
List<String> seen = new ArrayList<>();
```

```

    long count = names.stream().peek(seen::add).count(); // may not run
    ↪ peek at all!

    // After - do the real work in map/forEach, not peek
    List<String> seen2 = names.stream().collect(Collectors.toList());

```

- 4. Mutating shared state from inside a parallel stream lambda.** Why it matters: non-thread-safe collections corrupt under concurrent add, causing silent data loss or exceptions.

```

    // Before
    List<Integer> out = new ArrayList<>();
    data.parallelStream().forEach(out::add); // race condition

    // After
    List<Integer> out2 =
    ↪ data.parallelStream().collect(Collectors.toList());

```

- 5. Reusing a stream after a terminal operation.** Why it matters: streams are single-use; a second terminal call throws `IllegalStateException` at runtime, not compile time.

```

    // Before
    Stream<String> s = items.stream();
    s.forEach(System.out::println);
    long n = s.count(); // IllegalStateException

    // After
    long n2 = items.stream().count(); // build a fresh stream from the
    ↪ source

```

- 6. Calling `.parallel()` on a small or cheap workload “for speed.”** Why it matters: fork/join scheduling and merge overhead usually outweighs the gain unless $N \times Q$ is large; it can actually be slower and it steals threads from the shared pool.

```

    // Before
    int sum = IntStream.rangeClosed(1, 20).parallel().sum(); // pure
    ↪ overhead

    // After
    int sum2 = IntStream.rangeClosed(1, 20).sum();

```

- 7. Blocking I/O inside a parallel stream.** Why it matters: it occupies threads in the shared common `ForkJoinPool`, potentially starving unrelated parallel work elsewhere in the JVM.

```
// Before
List<String> bodies =
↳ urls.parallelStream().map(httpClient::get).toList();

// After
ExecutorService ioPool = Executors.newFixedThreadPool(10);
List<String> bodies2 = urls.stream()
    .map(u -> CompletableFuture.supplyAsync(() -> httpClient.get(u),
↳ ioPool))
    .toList().stream().map(CompletableFuture::join).toList();
```

- 8. Assuming Collectors.toMap silently overwrites on duplicate keys.** Why it matters: it throws `IllegalStateException` at runtime instead — a merge function is required if duplicates are possible.

```
// Before
Map<Integer, String> byLen =
↳ words.stream().collect(Collectors.toMap(String::length, w -> w));
// throws on duplicate length

// After
Map<Integer, String> byLen2 = words.stream()
    .collect(Collectors.toMap(String::length, w -> w, (a, b) -> a)); //
↳ keep first on conflict
```

- 9. Filtering before groupingBy when you meant to filter within each group.** Why it matters: filtering first can make a whole key disappear from the resulting map, changing downstream logic that expects every key to be present.

```
// Before - Sales might vanish entirely if no order has >1 item
Map<String, List<Order>> m = orders.stream()
    .filter(o -> o.items().size() > 1)
    .collect(Collectors.groupingBy(Order::customer));

// After - every customer key is still present, possibly with an empty
↳ list
Map<String, List<Order>> m2 = orders.stream()
    .collect(Collectors.groupingBy(Order::customer,
    Collectors.filtering(o -> o.items().size() > 1,
↳ Collectors.toList())));
```

- 10. Wrapping a checked exception badly, or swallowing it, inside a lambda.** Why it matters: standard functional interfaces don't declare throws; a careless catch can hide real failures instead of surfacing them.

```
// Before - swallows the real error and returns garbage
Function<String, byte[]> reader = path -> {
    try { return Files.readAllBytes(Path.of(path)); }
    catch (IOException e) { return new byte[0]; }
};

// After - preserve the failure, just change its exception type
Function<String, byte[]> reader2 = path -> {
    try { return Files.readAllBytes(Path.of(path)); }
    catch (IOException e) { throw new UncheckedIOException(e); }
};
```

- 11. Confusing findFirst and findAny in code that depends on order.** Why it matters: findAny gives no ordering guarantee, especially in parallel; using it where determinism matters introduces flaky behavior.

```
// Before - flaky in tests/parallel runs, expects the smallest even
↳ number
Optional<Integer> firstEven = nums.parallelStream().filter(n -> n % 2
↳ == 0).findAny();

// After
Optional<Integer> firstEven2 = nums.parallelStream().filter(n -> n % 2
↳ == 0).findFirst();
```

- 12. Using an infinite Stream. iterate/Stream.generate without a bound.** Why it matters: without limit or a takeWhile/predicate form, the pipeline never terminates and hangs the thread.

```
// Before
List<Integer> powers = Stream.iterate(1, n -> n * 2).toList(); // hangs
↳ forever

// After
List<Integer> powers2 = Stream.iterate(1, n -> n *
↳ 2).limit(10).toList();
```

- 13. Boxing unnecessarily in numeric pipelines.** Why it matters: Stream<Integer> allocates and unboxes on every element; IntStream avoids it and is measurably faster on large data.

```
// Before
int total = list.stream().reduce(0, Integer::sum);

// After
int total2 = list.stream().mapToInt(Integer::intValue).sum();
```

- 14. Confusing anonymous-class `this` with lambda `this` after a refactor.** Why it matters: converting an anonymous class to a lambda changes what `this` refers to, which can silently break field access or logging that relied on the anonymous class's own identity.

```
// Before - 'this' refers to the anonymous Runnable
Runnable r = new Runnable() {
    public void run() { log(this.getClass().getSimpleName()); }
};

// After (lambda) - 'this' now refers to the ENCLOSING instance, not
→ the lambda
Runnable r2 = () -> log(this.getClass().getSimpleName()); // meaning
→ changed - verify this is intended
```

- 15. Parallelizing a poorly-splittable source and expecting a speedup.** Why it matters: `LinkedList`/iterator-backed sources can't split efficiently, so `parallelStream()` on them often gives little to no benefit while still paying coordination overhead.

```
// Before
LinkedList<Integer> data = new LinkedList<>(bigList);
data.parallelStream().map(this::heavyWork).toList(); // splitting
→ degrades to near-linear walk

// After - use a splittable source
List<Integer> data2 = new ArrayList<>(bigList);
data2.parallelStream().map(this::heavyWork).toList();
```

- 16. Relying on `forEach` order in a parallel stream.** Why it matters: parallel `forEach` makes no ordering guarantee; code that assumes sequential-looking output will be flaky under load.

```
// Before - assumes output prints 1..N in order
nums.parallelStream().forEach(System.out::println);

// After - if order matters, either go sequential or collect then
→ iterate
```

```
List<Integer> ordered = nums.parallelStream().map(n -> n *
↳ 2).toList(); // preserves encounter order
ordered.forEach(System.out::println);
```

- 17. Writing an overly clever one-liner stream pipeline that's hard to review.** Why it matters: dense chained pipelines with side effects buried in map/peek are hard to test and hide bugs; reviewers should push back on readability, not just correctness.

```
// Before - hard to review, mixes transformation with hidden side
↳ effects
List<String> r = data.stream().peek(this::audit).map(this::transform).j
↳ filter(Objects::nonNull)
    .peek(this::cache).sorted().distinct().toList();

// After - explicit steps, side effects named and separated
List<String> transformed =
↳ data.stream().map(this::transform).filter(Objects::nonNull).toList();
transformed.forEach(this::audit);
List<String> r2 = transformed.stream().sorted().distinct().toList();
```

- 18. Treating a custom Collector's combiner as optional in a stream that might run in parallel.** Why it matters: the combiner is only exercised under parallel execution; a broken or missing combiner passes all sequential tests but produces wrong results the moment someone adds `.parallel()`.

```
// Before - combiner just picks one side, silently dropping data when
↳ split
Collector<String, StringBuilder, String> broken = Collector.of(
    StringBuilder::new,
    (sb, s) -> sb.append(s),
    (sb1, sb2) -> sb1, // BUG: drops sb2's contents entirely on
↳ merge
    StringBuilder::toString);

// After - combiner actually merges both partial results
Collector<String, StringBuilder, String> fixed = Collector.of(
    StringBuilder::new,
    (sb, s) -> sb.append(s),
    (sb1, sb2) -> sb1.append(sb2), // correct merge
    StringBuilder::toString);
```


Date, Time & Localization

Dates, times, and text that must work in more than one country trip up even experienced Java developers. This chapter covers the modern `java.time` API, how to format and parse dates safely, and how to write code that behaves correctly for users in different languages, countries, and time zones. By the end you should be able to spot the classic review-time mistakes: mutable date bugs, wrong time zone assumptions, locale-sensitive string comparisons, and formatter thread-safety issues.

Date and Time API (`java.time`)

Why the old `Date/Calendar/SimpleDateFormat` were replaced

Before Java 8, date and time handling used `java.util.Date`, `java.util.Calendar`, and `java.text.SimpleDateFormat`. These classes had deep design problems:

- **Mutable:** A `Date` object can be changed after creation. Passing one to another method risks it being silently mutated.
- **Confusing month numbering:** Months in `Calendar` are zero-based (January is 0), a constant source of off-by-one bugs.
- **Not thread-safe:** `SimpleDateFormat` is not safe to share across threads, but people often make it a static field “for performance” and get corrupted output under load.
- **Poor API design:** `Date` mixes up “a point in time” with “a calendar date,” which are conceptually different things.
- **No clear time zone model:** It was easy to write code that silently used the JVM’s default time zone.

`java.time` (JSR-310, led by Stephen Colebourne, the author of Joda-Time) fixed all of this: every type is immutable, thread-safe, and clearly named for what it represents.

```
import java.util.Date;
import java.util.Calendar;

public class LegacyProblems {
    public static void main(String[] args) {
        Date date = new Date();
        Calendar cal = Calendar.getInstance();
        cal.set(2026, Calendar.JANUARY, 15); // JANUARY == 0, easy to get
        ↪ wrong
        System.out.println(cal.getTime());
        // Output (example): Thu Jan 15 00:00:00 UTC 2026

        // Mutation trap: passing "date" elsewhere lets other code change
        ↪ it.
        Date copy = date;
        copy.setTime(0L); // also changes the original "date" reference!
        System.out.println(date.getTime());
        // Output: 0 <-- surprising, "date" was mutated through "copy"
    }
}
```

Legacy type	Modern replacement	Notes
java.util.Date	Instant (point in time) or LocalDateTime (no zone)	Date conflated “instant” and “calendar” concepts
java.util. Calendar	ZonedDateTime	Zero-based months gone; clear zone handling
java.util. Gregorian Calendar	ZonedDateTime / LocalDate	
java.text. SimpleDate Format	DateTimeFormatter	Immutable and thread-safe
java.sql.Date / Timestamp	LocalDate / Instant (via JDBC 4.2+ getObject)	Modern JDBC drivers support java. time directly
TimeZone	ZoneId / ZoneOffset	

The core types

java.time splits “date and time” into precise, single-purpose types. Picking the right one is a common review topic.

Type	Represents	Example use case
<code>LocalDate</code>	Date only, no time, no zone	Birthdate, invoice date
<code>LocalTime</code>	Time only, no date, no zone	Store opening time (09:00)
<code>LocalDateTime</code>	Date + time, no zone	“Meeting at 2026-08-07T14:00” in an unspecified zone
<code>ZonedDateTime</code>	Date + time + full time zone rules (ZoneId)	Flight departure time in Europe/Amsterdam
<code>OffsetDateTime</code>	Date + time + fixed UTC offset (no DST rules)	Timestamps in APIs / logs that need an offset but not a full zone
<code>Instant</code>	A point on the machine timeline (nanoseconds since epoch)	Event timestamps, “now” for logging
<code>Duration</code>	Amount of time in seconds/nanoseconds	“Timeout after 30 seconds”
<code>Period</code>	Amount of time in years/months/days	“Subscription lasts 1 year”
<code>Year</code>	A calendar year, e.g. 2026	Leap-year checks
<code>YearMonth</code>	Year + month, no day	Credit card expiry (MM/YYYY)
<code>MonthDay</code>	Month + day, no year	Recurring anniversary, without a year
<code>DayOfWeek</code>	Enum: MONDAY...SUNDAY	“Is this a business day?”
<code>ZoneId</code>	A geographical/political time zone, e.g. Europe/Berlin	Zone-aware scheduling
<code>ZoneOffset</code>	A fixed offset from UTC, e.g. +02:00	Offset math without DST rules

```

import java.time.*;

public class CoreTypesDemo {
    public static void main(String[] args) {
        LocalDate date = LocalDate.of(2026, 8, 7);
        LocalTime time = LocalTime.of(14, 30);
        LocalDateTime dateTime = LocalDateTime.of(date, time);
        ZonedDateTime zoned =
→ dateTime.atZone(ZoneId.of("Europe/Amsterdam"));
        OffsetDateTime offset = zoned.toOffsetDateTime();
        Instant instant = zoned.toInstant();

        System.out.println(date);           // 2026-08-07
        System.out.println(time);           // 14:30
        System.out.println(dateTime);       // 2026-08-07T14:30
        System.out.println(zoned);          //
→ 2026-08-07T14:30+02:00[Europe/Amsterdam]
        System.out.println(offset);         // 2026-08-07T14:30+02:00

```

```

        System.out.println(instant);    // 2026-08-07T12:30:00Z

        Duration meetingLength = Duration.ofMinutes(45);
        Period subscription = Period.ofYears(1);
        System.out.println(meetingLength); // PT45M
        System.out.println(subscription);  // P1Y

        YearMonth expiry = YearMonth.of(2027, 12);
        MonthDay anniversary = MonthDay.of(6, 15);
        DayOfWeek dow = date.getDayOfWeek();
        System.out.println(expiry);        // 2027-12
        System.out.println(anniversary);   // --06-15
        System.out.println(dow);           // FRIDAY
    }
}

```

Immutability and the fluent plus/minus/with API

Every `java.time` type is immutable: methods like `plusDays`, `minusMonths`, and `withYear` return a **new** object instead of mutating the original. This removes an entire category of bugs where a shared date gets silently changed. It also means you must always use the return value — a very common code-review pitfall is calling `date.plusDays(1)`; and discarding the result.

```

import java.time.LocalDate;

public class ImmutabilityDemo {
    public static void main(String[] args) {
        LocalDate original = LocalDate.of(2026, 8, 7);

        LocalDate nextWeek = original.plusWeeks(1);
        LocalDate lastYear = original.minusYears(1);
        LocalDate changedYear = original.withYear(2030);

        System.out.println(original);    // 2026-08-07 (unchanged!)
        System.out.println(nextWeek);    // 2026-08-14
        System.out.println(lastYear);    // 2025-08-07
        System.out.println(changedYear); // 2030-08-07

        // Fluent chaining - each call returns a new instance
        LocalDate result =
    ↪ original.plusMonths(2).minusDays(3).withDayOfMonth(1);
        System.out.println(result); // 2026-10-01
    }
}

```

TemporalAdjusters

`TemporalAdjusters` provides ready-made rules for common “find this special date” queries, like “next Monday” or “last day of the month.” You can also write your own adjuster as a lambda.

```
import java.time.DayOfWeek;
import java.time.LocalDate;
import java.time.temporal.TemporalAdjusters;

public class AdjustersDemo {
    public static void main(String[] args) {
        LocalDate date = LocalDate.of(2026, 8, 7); // a Friday

        LocalDate nextMonday =
        ↪ date.with(TemporalAdjusters.next(DayOfWeek.MONDAY));
        LocalDate lastDayOfMonth =
        ↪ date.with(TemporalAdjusters.lastDayOfMonth());
        LocalDate firstDayOfNextMonth =
        ↪ date.with(TemporalAdjusters.firstDayOfNextMonth());

        System.out.println(nextMonday);           // 2026-08-10
        System.out.println(lastDayOfMonth);        // 2026-08-31
        System.out.println(firstDayOfNextMonth);   // 2026-09-01

        // Custom adjuster: move to the next even day of the month
        LocalDate nextEvenDay = date.with(temporal -> {
            LocalDate d = LocalDate.from(temporal);
            do {
                d = d.plusDays(1);
            } while (d.getDayOfMonth() % 2 != 0);
            return d;
        });
        System.out.println(nextEvenDay); // 2026-08-08
    }
}
```

ChronoUnit.between

`ChronoUnit` lets you measure the difference between two temporal objects in a specific unit (days, hours, years, etc.). It is often clearer than manually computing `Period` or `Duration` when you just need a single number.

```

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.temporal.ChronoUnit;

public class ChronoUnitDemo {
    public static void main(String[] args) {
        LocalDate start = LocalDate.of(2026, 1, 1);
        LocalDate end = LocalDate.of(2026, 8, 7);

        long daysBetween = ChronoUnit.DAYS.between(start, end);
        long monthsBetween = ChronoUnit.MONTHS.between(start, end);

        System.out.println(daysBetween);    // 218
        System.out.println(monthsBetween);  // 7

        LocalDateTime t1 = LocalDateTime.of(2026, 8, 7, 9, 0);
        LocalDateTime t2 = LocalDateTime.of(2026, 8, 7, 17, 30);
        long minutesWorked = ChronoUnit.MINUTES.between(t1, t2);
        System.out.println(minutesWorked); // 510
    }
}

```

Clock and testable time

Calling `LocalDate.now()` or `Instant.now()` directly inside business logic makes that logic hard to unit test — the result changes every time you run it. `Clock` is an abstraction over “the current time” that you can inject and replace with a fixed clock in tests.

```

import java.time.Clock;
import java.time.Instant;
import java.time.LocalDate;
import java.time.ZoneOffset;

public class ClockDemo {

    // Production code depends on a Clock, not on LocalDate.now() directly
    static class InvoiceService {
        private final Clock clock;

        InvoiceService(Clock clock) {
            this.clock = clock;
        }

        LocalDate today() {

```

```

        return LocalDate.now(clock);
    }
}

public static void main(String[] args) {
    // Real usage: system clock
    InvoiceService realService = new
    ↪ InvoiceService(Clock.systemUTC());
    System.out.println(realService.today()); // e.g. 2026-08-07

    // Test usage: fixed clock, deterministic and repeatable
    Clock fixedClock = Clock.fixed(
        Instant.parse("2026-01-01T00:00:00Z"), ZoneOffset.UTC);
    InvoiceService testService = new InvoiceService(fixedClock);
    System.out.println(testService.today()); // 2026-01-01 (always, in
    ↪ tests)
}

```

DST transitions: gaps and overlaps

Daylight Saving Time (DST) creates two tricky situations in local time:

- **Gap:** Clocks jump forward, and some local times never occur (e.g. 02:30 does not exist the night clocks spring forward).
- **Overlap:** Clocks jump backward, and some local times occur twice (e.g. 02:30 happens twice the night clocks fall back).

ZonedDateTime resolves these automatically but silently — it is important to know the behavior instead of assuming “one hour is always one hour.”

```

import java.time.LocalDateTime;
import java.time.ZoneId;
import java.time.ZonedDateTime;

public class DstDemo {
    public static void main(String[] args) {
        ZoneId amsterdam = ZoneId.of("Europe/Amsterdam");

        // Gap: 2026-03-29 02:30 does not exist in Europe/Amsterdam
        ↪ (spring forward at 02:00 -> 03:00)
        LocalDateTime gapTime = LocalDateTime.of(2026, 3, 29, 2, 30);
        ZonedDateTime resolvedGap = gapTime.atZone(amsterdam);
        System.out.println(resolvedGap);
    }
}

```

```

        // Output: 2026-03-29T03:30+02:00[Europe/Amsterdam] <-- shifted
        ↪ forward, not what you typed!

        // Overlap: 2026-10-25 02:30 happens twice (fall back at 03:00 ->
        ↪ 02:00)
        LocalDateTime overlapTime = LocalDateTime.of(2026, 10, 25, 2, 30);
        ZonedDateTime firstOccurrence = overlapTime.atZone(amsterdam);
        System.out.println(firstOccurrence);
        // Output: 2026-10-25T02:30+02:00[Europe/Amsterdam] (the earlier
        ↪ of the two offsets, by default)

        // Adding a "day" across a DST boundary: still 24h wall-clock days,
        ↪ but may be 23 or 25 actual hours.
        ZonedDateTime beforeDst = ZonedDateTime.of(2026, 3, 28, 12, 0, 0,
        ↪ 0, amsterdam);
        ZonedDateTime afterOneDay = beforeDst.plusDays(1);
        System.out.println(afterOneDay); //
        ↪ 2026-03-29T12:00+02:00[Europe/Amsterdam] - only 23 real hours passed
    }
}

```

Comparing instants vs local times

Instant represents an absolute point on the universal timeline; two instants can always be safely compared regardless of where they came from. `LocalDateTime` has no time zone information at all, so comparing two `LocalDateTime` values only tells you which “wall clock reading” is earlier — it says nothing about which one happened first in reality if they came from different zones.

```

import java.time.*;

public class ComparisonDemo {
    public static void main(String[] args) {
        ZonedDateTime tokyoMeeting =
            ZonedDateTime.of(2026, 8, 7, 22, 0, 0, 0,
        ↪ ZoneId.of("Asia/Tokyo"));
        ZonedDateTime amsterdamMeeting =
            ZonedDateTime.of(2026, 8, 7, 16, 0, 0, 0,
        ↪ ZoneId.of("Europe/Amsterdam"));

        // Wrong-ish: comparing LocalDateTime ignores the zone entirely
        boolean localLooksLater = tokyoMeeting.toLocalDateTime()
            .isAfter(amsterdamMeeting.toLocalDateTime());
        System.out.println(localLooksLater); // true (22:00 > 16:00) -
        ↪ misleading!
    }
}

```



```

        // Correct: compare the actual instants
        boolean instantIsLater =
        ↪ tokyoMeeting.toInstant().isAfter(amsterdamMeeting.toInstant());
        System.out.println(instantIsLater); // false - Tokyo 22:00 JST
        ↪ happens BEFORE Amsterdam 16:00 CEST

        System.out.println(tokyoMeeting.toInstant()); //
        ↪ 2026-08-07T13:00:00Z
        System.out.println(amsterdamMeeting.toInstant()); //
        ↪ 2026-08-07T14:00:00Z
    }
}

```

Storing time in a database

The safest rule: **store instants in UTC, and store the original time zone separately if you need to display local wall-clock time later.** A raw `LocalDateTime` alone loses the information needed to know “when” that really was in absolute terms; a `ZonedDateTime`’s zone rules can even change in the future (governments do change DST rules).

What you need to store	Recommended column type(s)
“This event happened at an exact moment”	TIMESTAMP WITH TIME ZONE (stored internally as UTC), mapped to <code>Instant</code> or <code>OffsetDateTime</code>
“This event happened at an exact moment, and I must show the original local time later”	UTC instant column + a separate <code>zone_id</code> (String) column
“A calendar date with no time meaning” (birthday, deadline date)	DATE, mapped to <code>LocalDate</code>
“A recurring local time with no date” (store opens at 9am, always local)	TIME, mapped to <code>LocalTime</code>

```

import java.time.Instant;
import java.time.ZoneId;
import java.time.ZonedDateTime;

public class DbStorageExample {

```

```

// What you'd persist for an appointment booked by a user in a given
↪ zone
record AppointmentRecord(Instant utcInstant, String zoneId) {

    ZonedDateTime toLocalDisplayTime() {
        return utcInstant.atZone(ZoneId.of(zoneId));
    }
}

public static void main(String[] args) {
    ZonedDateTime bookedAt =
        ZonedDateTime.of(2026, 8, 7, 15, 0, 0, 0,
↪ ZoneId.of("America/New_York"));

    AppointmentRecord record = new
↪ AppointmentRecord(bookedAt.toInstant(), "America/New_York");

    System.out.println(record.utcInstant()); // 2026-08-07T19:00:00Z
    System.out.println(record.toLocalDisplayTime());
    // Output: 2026-08-07T15:00-04:00[America/New_York]
}
}

```

Formatting and Parsing

DateTimeFormatter predefined formats

`DateTimeFormatter` ships with several constants for standard formats, most notably the ISO family. These are good defaults for machine-to-machine communication (logs, APIs, file names).

```

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

public class PredefinedFormatsDemo {
    public static void main(String[] args) {
        LocalDate date = LocalDate.of(2026, 8, 7);
        LocalDateTime dateTime = LocalDateTime.of(2026, 8, 7, 14, 30, 15);

        System.out.println(date.format(DateTimeFormatter.ISO_LOCAL_DATE));
        // Output: 2026-08-07

        System.out.println(dateTime.format(DateTimeFormatter.
↪ ISO_LOCAL_DATE_TIME));
    }
}

```

```
// Output: 2026-08-07T14:30:15

↪ System.out.println(dateTime.format(DateTimeFormatter.ISO_DATE_TIME));
    // Output: 2026-08-07T14:30:15 (no offset here because
↪ LocalDateTime has no zone)

↪ System.out.println(dateTime.format(DateTimeFormatter.BASIC_ISO_DATE));
    // Output: 20260807
    }
}
```

ofPattern and custom patterns

`DateTimeFormatter.ofPattern` lets you define your own layout using pattern letters (borrowed conceptually from `SimpleDateFormat`, but not identical — see the `YYYY` pitfall below).

```
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.Locale;

public class OfPatternDemo {
    public static void main(String[] args) {
        LocalDateTime dt = LocalDateTime.of(2026, 8, 7, 14, 30, 15);

        DateTimeFormatter fmt1 = DateTimeFormatter.ofPattern("dd/MM/yyyy
↪ HH:mm:ss");
        DateTimeFormatter fmt2 = DateTimeFormatter.ofPattern("EEEE, MMMM d,
↪ yyyy", Locale.ENGLISH);

        System.out.println(dt.format(fmt1)); // 07/08/2026 14:30:15
        System.out.println(dt.format(fmt2)); // Friday, August 7, 2026
    }
}
```

Pattern letter	Meaning	Example
y	Year of era	2026
M	Month of year	08 or Aug (with MMM)
d	Day of month	07

Pattern letter	Meaning	Example
E	Day of week	Fri or Friday (with EEEE)
H	Hour (0-23)	14
h	Hour (1-12), needs a for AM/PM	02 PM
m	Minute	30
s	Second	15
S	Fraction of second	123
z	Time zone name	CEST
Z	Zone offset	+0200
X	ISO-8601 offset	+02:00

ofLocalizedDate and locale-aware styles

Instead of hand-rolling a pattern, `ofLocalizedDate`, `ofLocalizedTime`, and `ofLocalizedDateTime` produce formats that automatically match the conventions of a given `Locale` (order of day/month/year, separators, month names).

```
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.time.format.FormatStyle;
import java.util.Locale;

public class LocalizedFormatDemo {
    public static void main(String[] args) {
        LocalDate date = LocalDate.of(2026, 8, 7);

        DateTimeFormatter us =
        ↪ DateTimeFormatter.ofLocalizedDate(FormatStyle.LONG).withLocale(Locale.US);
        DateTimeFormatter de = DateTimeFormatter.ofLocalizedDate(
        ↪ FormatStyle.LONG).withLocale(Locale.GERMANY);
        DateTimeFormatter jp = DateTimeFormatter.ofLocalizedDate(
        ↪ FormatStyle.LONG).withLocale(Locale.JAPAN);

        System.out.println(date.format(us)); // August 7, 2026
        System.out.println(date.format(de)); // 7. August 2026
        System.out.println(date.format(jp)); // 2026年8月7日
    }
}
```

DateTimeFormatterBuilder

For complex or conditional formats — optional sections, case-insensitive parsing, custom text lookups — `DateTimeFormatterBuilder` gives fine-grained control beyond a simple pattern string.

```
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.time.format.DateTimeFormatterBuilder;
import java.time.temporal.ChronoField;
import java.util.Locale;

public class FormatterBuilderDemo {
    public static void main(String[] args) {
        DateTimeFormatter formatter = new DateTimeFormatterBuilder()
            .appendValue(ChronoField.YEAR, 4)
            .appendLiteral('-')
            .appendValue(ChronoField.MONTH_OF_YEAR, 2)
            .appendLiteral('-')
            .appendValue(ChronoField.DAY_OF_MONTH, 2)
            .optionalStart()
            .appendLiteral(" ")
            .appendText(ChronoField.DAY_OF_WEEK)
            .appendLiteral(' ')
            .optionalEnd()
            .toFormatter(Locale.ENGLISH);

        LocalDate date = LocalDate.of(2026, 8, 7);
        System.out.println(date.format(formatter));
        // Output: 2026-08-07 (Friday)
    }
}
```

Strict vs lenient ResolverStyle

`ResolverStyle` controls how aggressively the parser accepts out-of-range or ambiguous field values. `STRICT` rejects anything invalid, `SMART` (the default) applies sensible corrections, and `LENIENT` allows arithmetic overflow into adjacent fields.

```
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.time.format.ResolverStyle;

public class ResolverStyleDemo {
    public static void main(String[] args) {
```

```

        DateTimeFormatter strict =
↳ DateTimeFormatter.ofPattern("yyyy-MM-dd")
                .withResolverStyle(ResolverStyle.STRICT);
        DateTimeFormatter smart =
↳ DateTimeFormatter.ofPattern("yyyy-MM-dd")
                .withResolverStyle(ResolverStyle.SMART); // default
        DateTimeFormatter lenient =
↳ DateTimeFormatter.ofPattern("yyyy-MM-dd")
                .withResolverStyle(ResolverStyle.LENIENT);

        // "2026-02-30" - February never has 30 days
        try {
            LocalDate.parse("2026-02-30", strict);
        } catch (Exception e) {
            System.out.println("STRICT fails: " + e.getMessage());
            // Output: STRICT fails: Invalid date 'FEBRUARY 30'
        }

        try {
            LocalDate.parse("2026-02-30", smart);
        } catch (Exception e) {
            System.out.println("SMART fails: " + e.getMessage());
            // Output: SMART fails: Invalid date 'FEBRUARY 30'
        }

        LocalDate lenientResult = LocalDate.parse("2026-02-30", lenient);
        System.out.println(lenientResult);
        // Output: 2026-03-02 (overflowed into March)
    }
}

```

ISO-8601

ISO-8601 is the international standard for representing dates and times as text (2026-08-07T14:30:15+02:00). It is unambiguous, sortable as a plain string, and the default choice for APIs, logs, and config files. `java.time`'s `toString()` methods and `DateTimeFormatter.ISO_*` constants follow it by default.

```

import java.time.OffsetDateTime;
import java.time.ZoneOffset;

public class Iso8601Demo {
    public static void main(String[] args) {
        OffsetDateTime odt = OffsetDateTime.of(2026, 8, 7, 14, 30, 15, 0,
↳ ZoneOffset.ofHours(2));
    }
}

```

```

        System.out.println(odt); // 2026-08-07T14:30:15+02:00 (ISO-8601,
↪ no formatter needed)

        OffsetDateTime parsedBack =
↪ OffsetDateTime.parse("2026-08-07T14:30:15+02:00");
        System.out.println(parsedBack.equals(odt)); // true
    }
}

```

The YYYY vs yyyy week-year bug

This is a famous, easy-to-miss bug. Lowercase yyyy is the **calendar year**. Uppercase YYYY is the **week-based year** (ISO week-year, used together with week-of-year w). Near the start or end of a year, they can disagree, silently shifting dates by a year.

```

import java.time.LocalDate;
import java.time.format.DateTimeFormatter;

public class WeekYearBugDemo {
    public static void main(String[] args) {
        // 2027-01-01 is a Friday, and it belongs to ISO week 53 of year
↪ 2026
        LocalDate date = LocalDate.of(2027, 1, 1);

        DateTimeFormatter withLowercaseY =
↪ DateTimeFormatter.ofPattern("yyyy-MM-dd");
        DateTimeFormatter withUppercaseY =
↪ DateTimeFormatter.ofPattern("YYYY-MM-dd");

        System.out.println(date.format(withLowercaseY)); // 2027-01-01
↪ (correct calendar date)
        System.out.println(date.format(withUppercaseY)); // 2026-01-01 (!)
↪ week-year, NOT what most people expect

        // Rule of thumb for code review: always use lowercase yyyy unless
        // you are deliberately building an ISO week-date format (with
↪ 'w').
    }
}

```

Parsing failures and DateTimeParseException

Any failed parse throws an unchecked `DateTimeParseException`. Because it's unchecked, it is easy to forget to handle at the boundary where you accept external, untrusted date strings (user input, CSV files, third-party APIs).

```
import java.time.LocalDate;
import java.time.format.DateTimeParseException;

public class ParseFailureDemo {
    public static void main(String[] args) {
        String[] inputs = {"2026-08-07", "07/08/2026", "not-a-date"};

        for (String input : inputs) {
            try {
                LocalDate parsed = LocalDate.parse(input); // expects
↳ ISO_LOCAL_DATE by default
                System.out.println("Parsed: " + parsed);
            } catch (DateTimeParseException e) {
                System.out.println("Failed to parse '" + input + "': " +
↳ e.getMessage());
            }
        }
        // Output:
        // Parsed: 2026-08-07
        // Failed to parse '07/08/2026': Text '07/08/2026' could not be
↳ parsed at index 2
        // Failed to parse 'not-a-date': Text 'not-a-date' could not be
↳ parsed at index 0
    }
}
```

Thread safety: DateTimeFormatter vs SimpleDateFormat

`SimpleDateFormat` is **not thread-safe** — internally it uses a mutable `Calendar` field, so two threads calling `format()` on the same instance concurrently can corrupt each other's results. `DateTimeFormatter` is **immutable and thread-safe**, so it is safe (and recommended) to share a single instance as a `static final` constant.

```
import java.text.SimpleDateFormat;
import java.time.format.DateTimeFormatter;
import java.util.Date;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
```



```

public class ThreadSafetyDemo {

    // UNSAFE: sharing a SimpleDateFormat across threads can produce
    ↪ garbled output or exceptions
    private static final SimpleDateFormat UNSAFE_FORMAT = new
    ↪ SimpleDateFormat("yyyy-MM-dd HH:mm:ss");

    // SAFE: DateTimeFormatter is immutable, sharing it is fine and
    ↪ idiomatic
    private static final DateTimeFormatter SAFE_FORMAT =
        DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss");

    public static void main(String[] args) throws InterruptedException {
        ExecutorService pool = Executors.newFixedThreadPool(4);
        for (int i = 0; i < 4; i++) {
            pool.submit(() -> {
                // Calling UNSAFE_FORMAT.format(new Date()) here
                ↪ concurrently
                // may throw NumberFormatException or return a corrupted
                ↪ string.
                String safeResult =
                ↪ java.time.LocalDateTime.now().format(SAFE_FORMAT);
                System.out.println(Thread.currentThread().getName() + ":
                ↪ " + safeResult);
            });
        }
        pool.shutdown();
        // Fix for legacy code: wrap SimpleDateFormat in a ThreadLocal, or
        ↪ just switch to DateTimeFormatter.
    }
}

```

Internationalization (i18n)

Internationalization (“i18n” — 18 letters between “i” and “n”) is designing software so it *can* be adapted to different languages and regions without code changes. Localization (“l10n”) is the actual act of adapting it for one specific locale (translating text, formatting dates, etc.).

Locale

A `Locale` identifies a language, optionally a country/region, and optionally a variant. It drives almost every locale-sensitive API in the JDK.

```

import java.util.Locale;

public class LocaleDemo {
    public static void main(String[] args) {
        Locale usEnglish = Locale.US;           // language=en,
        ↪ country=US
        Locale german = Locale.GERMANY;         // language=de,
        ↪ country=DE
        Locale custom = new Locale("en", "IN");  // English as
        ↪ used in India
        Locale fromTag = Locale.forLanguageTag("pt-BR"); // Brazilian
        ↪ Portuguese, BCP 47 tag
        Locale root = Locale.ROOT;              // no
        ↪ language/country - a neutral, culture-independent locale

        System.out.println(usEnglish.getLanguage() + "-" +
        ↪ usEnglish.getCountry()); // en-US
        System.out.println(german.getDisplayName(Locale.US));
        ↪ // German (Germany)
        System.out.println(fromTag.getDisplayName());
        ↪ // Portuguese (Brazil)
        System.out.println(root);
        // Output: (empty string) - Locale.ROOT has no language, country,
        ↪ or variant

        // Locale.ROOT is the right choice for locale-independent
        ↪ operations,
        // like normalizing internal identifiers or keys, not user-facing
        ↪ text.
        System.out.println("HELLO".toLowerCase(Locale.ROOT)); // hello
    }
}

```

ResourceBundle and property files

ResourceBundle loads translated text from .properties files named by locale, such as Messages_en.properties, Messages_de.properties, and Messages.properties (the default fallback).

```

# Messages.properties (default/fallback)
greeting=Hello, {0}!

# Messages_de.properties
greeting=Hallo, {0}!

```

```
# Messages_fr.properties
greeting=Bonjour, {0} !
```

```
import java.text.MessageFormat;
import java.util.Locale;
import java.util.ResourceBundle;

public class ResourceBundleDemo {
    public static void main(String[] args) {
        ResourceBundle enBundle = ResourceBundle.getBundle("Messages",
↳ Locale.ENGLISH);
        ResourceBundle deBundle = ResourceBundle.getBundle("Messages",
↳ Locale.GERMAN);

        String enGreeting =
↳ MessageFormat.format(enBundle.getString("greeting"), "Alice");
        String deGreeting =
↳ MessageFormat.format(deBundle.getString("greeting"), "Alice");

        System.out.println(enGreeting); // Hello, Alice!
        System.out.println(deGreeting); // Hallo, Alice!

        // Requesting a locale with no matching file falls back to
↳ Messages.properties
        ResourceBundle jaBundle = ResourceBundle.getBundle("Messages",
↳ Locale.JAPANESE);
        System.out.println(jaBundle.getString("greeting")); // Hello, {0}!
↳ (fallback, untranslated placeholder)
    }
}
```

MessageFormat: plurals and argument indexes

MessageFormat supports positional arguments ({0}, {1}) that can be reordered per language, and a choice/plural-style syntax for picking text based on a number. This matters because different languages pluralize differently (some languages have more than two plural forms).

```
import java.text.MessageFormat;
import java.util.Locale;

public class MessageFormatDemo {
    public static void main(String[] args) {
```

```

        // Argument indexes let translators reorder words for grammar
↪ reasons
        String pattern = "{0} sent {1} a message.";           // English word
↪ order
        String patternGerman = "{1} bekam eine Nachricht von {0}."; //
↪ reordered for German

        System.out.println(MessageFormat.format(pattern, "Alice", "Bob"));
        // Output: Alice sent Bob a message.
        System.out.println(MessageFormat.format(patternGerman, "Alice",
↪ "Bob"));
        // Output: Bob bekam eine Nachricht von Alice.

        // Simple pluralization using choice format
        MessageFormat itemsFormat = new MessageFormat(
            "There {0,choice,0#are no items|1#is one item|1<are
↪ {0,number,integer} items}.",
            Locale.ENGLISH);

        System.out.println(itemsFormat.format(new Object[]{0})); // There
↪ are no items.
        System.out.println(itemsFormat.format(new Object[]{1})); // There
↪ is one item.
        System.out.println(itemsFormat.format(new Object[]{5})); // There
↪ are 5 items.
        // Note: for serious plural-rule handling (many languages have 3-6
↪ plural forms),
        // prefer a library that implements full CLDR plural rules (e.g.
↪ ICU4J).
    }
}

```

NumberFormat and Currency

Numbers and currency amounts are formatted very differently across locales — decimal separators, grouping separators, and currency symbol placement all vary.

```

import java.text.NumberFormat;
import java.util.Currency;
import java.util.Locale;

public class NumberFormatDemo {
    public static void main(String[] args) {
        double amount = 1234567.891;

        NumberFormat us = NumberFormat.getNumberInstance(Locale.US);

```

```

        NumberFormat de = NumberFormat.getNumberInstance(Locale.GERMANY);

        System.out.println(us.format(amount)); // 1,234,567.891
        System.out.println(de.format(amount)); // 1.234.567,891

        NumberFormat usCurrency =
    ↪ NumberFormat.getCurrencyInstance(Locale.US);
        NumberFormat jpCurrency =
    ↪ NumberFormat.getCurrencyInstance(Locale.JAPAN);
        System.out.println(usCurrency.format(1234.5)); // $1,234.50
        System.out.println(jpCurrency.format(1234.5)); // ¥1,235 (Yen has
    ↪ no minor unit, gets rounded)

        Currency eur = Currency.getInstance("EUR");
        System.out.println(eur.getSymbol(Locale.GERMANY)); // €
        System.out.println(eur.getDefaultFractionDigits()); // 2
    }
}

```

Collation with Collator

Sorting text alphabetically is locale-dependent. A plain `String.compareTo` sorts by raw UTF-16 code unit values, which does not match human alphabetical order in most languages. `Collator` sorts the way a native speaker of a given locale would expect.

```

import java.text.Collator;
import java.util.Arrays;
import java.util.Locale;

public class CollatorDemo {
    public static void main(String[] args) {
        String[] words = {"Apfel", "Äpfel", "banane", "Banane"}; // Ä is
    ↪ U+00C4

        String[] naturalSort = words.clone();
        Arrays.sort(naturalSort); // plain code-unit comparison

        String[] germanSort = words.clone();
        Collator germanCollator = Collator.getInstance(Locale.GERMANY);
        Arrays.sort(germanSort, germanCollator);

        System.out.println(Arrays.toString(naturalSort));
        // Output: [Apfel, Banane, banane, Äpfel] <-- uppercase/accents
    ↪ sort oddly by raw code points
        System.out.println(Arrays.toString(germanSort));
    }
}

```

```

        // Output: [Apfel, Äpfel, banane, Banane] <-- matches how a
        ↪ German speaker expects it sorted
    }
}

```

String.format with a Locale

`String.format` (and `Formatter/printf`) is locale-sensitive for things like decimal points and digit grouping. Always pass an explicit `Locale` in server-side or shared code — otherwise it silently uses the JVM's default locale, which can differ between environments.

```

import java.util.Locale;

public class StringFormatLocaleDemo {
    public static void main(String[] args) {
        double price = 1234.5;

        String defaultResult = String.format("%.2f", price);    // uses
        ↪ JVM default locale - risky!
        String usResult = String.format(Locale.US, "%.2f", price);
        String deResult = String.format(Locale.GERMANY, "%.2f", price);

        System.out.println(usResult); // 1,234.50
        System.out.println(deResult); // 1.234,50
        // "defaultResult" could be either of the above depending on the
        ↪ machine's default locale -
        // this is a classic source of "works on my machine" bugs in
        ↪ server-side code.
    }
}

```

toUpperCase() and the Turkish dotless-i problem

Calling `toUpperCase()` or `toLowerCase()` without a `Locale` uses the JVM's default locale. In Turkish (and Azerbaijani), the letter `i` uppercases to `İ` (dotted capital I), not the ASCII `I` most code expects — and lowercase `i` becomes `ı` (dotless). This breaks case-insensitive comparisons against constants like `"ID"` or `"HTTP"` when the default locale is Turkish.

```

import java.util.Locale;

public class TurkishLocaleBugDemo {

```

```

public static void main(String[] args) {
    String input = "id";

    String upperTurkish = input.toUpperCase(new Locale("tr", "TR"));
    String upperRoot = input.toUpperCase(Locale.ROOT);

    System.out.println(upperTurkish); // İD (dotted capital İ, not
    ↪ plain "I")
    System.out.println(upperRoot);    // ID (plain ASCII, as
    ↪ expected)

    System.out.println(upperTurkish.equals("ID")); // false - a real
    ↪ bug if this runs on a Turkish-locale server!
    System.out.println(upperRoot.equals("ID"));    // true

    // Rule for code review: for internal/programmatic comparisons
    ↪ (protocol keywords,
        // enum names, HTTP headers), always use toUpperCase(Locale.ROOT)
    ↪ / toLowerCase(Locale.ROOT),
        // or better, String.equalsIgnoreCase() when you just need a
    ↪ comparison.
    }
}

```

Character encoding and UTF-8 as the default charset

Historically, `String.getBytes()`, `new String(bytes)`, `FileReader`, and similar APIs used the **platform default charset** if none was specified — which could be UTF-8 on Linux but something else (like Windows-1252) on Windows, causing garbled text (“mojibake”) when code moved between environments. Since **JDK 18** (JEP 400), the JVM’s default charset is **always UTF-8**, regardless of OS or locale, unless explicitly overridden with the `file.encoding` system property.

```

import java.nio.charset.Charset;
import java.nio.charset.StandardCharsets;

public class DefaultCharsetDemo {
    public static void main(String[] args) {
        System.out.println(Charset.defaultCharset());
        // Output (JDK 18+): UTF-8 (was platform-dependent before JDK
    ↪ 18)

        String text = "café ☹";
        byte[] defaultBytes = text.getBytes(); // uses
    ↪ default charset - UTF-8 on JDK 18+
    }
}

```

```

        byte[] explicitBytes = text.getBytes(StandardCharsets.UTF_8); //
↳ always explicit, safest for review

        System.out.println(defaultBytes.length); // 8 (matches UTF-8
↳ encoding)
        System.out.println(explicitBytes.length); // 8

        // Best practice even on JDK 18+: always pass an explicit Charset
↳ for I/O boundaries
        // (files, network, serialization) so behavior never depends on
↳ JVM defaults or -Dfile.encoding.
    }
}

```

Normalizer for Unicode

The same visible character can sometimes be represented by different sequences of Unicode code points — for example, “é” can be one precomposed code point (U+00E9) or an “e” followed by a combining accent (U+0065 U+0301). These compare as unequal with plain `.equals()` even though they look identical. `java.text.Normalizer` converts text into a consistent form before comparing or storing it.

```

import java.text.Normalizer;

public class NormalizerDemo {
    public static void main(String[] args) {
        String precomposed = "café"; // "café" using single code
↳ point U+00E9
        String decomposed = "café"; // "café" using 'e' + combining
↳ acute accent U+0301

        System.out.println(precomposed.equals(decomposed)); // false -
↳ look identical, but different bytes!

        String normalizedA = Normalizer.normalize(precomposed,
↳ Normalizer.Form.NFC);
        String normalizedB = Normalizer.normalize(decomposed,
↳ Normalizer.Form.NFC);

        System.out.println(normalizedA.equals(normalizedB)); // true - now
↳ safe to compare

        // NFC = canonical composition (preferred for storage/display)
        // NFD = canonical decomposition (useful for stripping accents,
↳ searching)
    }
}

```



```

        String stripped = Normalizer.normalize(precomposed,
        ↪ Normalizer.Form.NFD)
            .replaceAll("\\p{M}", ""); // remove combining marks
        System.out.println(stripped); // cafe
    }
}

```

Right-to-left text, code points, and grapheme clusters

Some languages (Arabic, Hebrew) are written right-to-left (RTL). Java strings store text left-to-right in memory regardless of visual direction — rendering direction is a display concern (handled by UI toolkits, not the `String` class itself). A separate, very common bug source: `String.length()` counts UTF-16 *code units*, not visible characters. Emoji, some CJK (Chinese/Japanese/Korean) characters, and combining accents can each take more than one code unit, and a single visible “character” (grapheme cluster) can be made of multiple code points.

```

import java.text.BreakIterator;
import java.util.Locale;

public class TextLengthDemo {
    public static void main(String[] args) {
        String flagEmoji = "\uD83C\uDDF3\uD83C\uDDF1"; // the Netherlands
        ↪ flag emoji, one visible glyph
        String familyEmoji =
        ↪ "\uD83D\uDC68\u200D\uD83D\uDC69\u200D\uD83D\uDC67"; // family emoji, one
        ↪ glyph

        System.out.println(flagEmoji.length()); // 4 (UTF-16
        ↪ code units, NOT 1!)
        System.out.println(flagEmoji.codePointCount(0,
        ↪ flagEmoji.length())); // 2 (code points, still not 1)

        // BreakIterator finds actual user-perceived "characters"
        ↪ (grapheme clusters)
        BreakIterator iterator =
        ↪ BreakIterator.getInstance(Locale.ROOT);
        iterator.setText(familyEmoji);
        int graphemeCount = 0;
        int boundary = iterator.first();
        while (boundary != BreakIterator.DONE) {
            int next = iterator.next();
            if (next != BreakIterator.DONE) graphemeCount++;
            boundary = next;
        }
    }
}

```

```

        System.out.println(graphemeCount); // 1 (one visible "character"
↳ made of several code points)

        // Rule for code review: never assume String.length() == "number
↳ of visible characters"
        // when the input may contain emoji, combining marks, or
↳ characters outside the Basic
        // Multilingual Plane. Use codePointCount() or BreakIterator
↳ depending on what you truly need.
    }
}

```

Localization (l10n)

Organizing bundles and fallback rules

A typical project has one base bundle plus one file per supported locale, sometimes split by variant (language, then language+country). `ResourceBundle` looks up a candidate locale chain from most to least specific, and falls back to the base bundle if nothing else matches.

```

Messages.properties          <- default / fallback (usually English)
Messages_en.properties      <- English (falls back to default
↳ automatically if same content)
Messages_en_GB.properties   <- British English overrides (e.g. "colour"
↳ vs "color")
Messages_de.properties      <- German
Messages_de_CH.properties   <- Swiss German overrides
Messages_fr.properties      <- French

```

```

import java.util.Locale;
import java.util.ResourceBundle;

public class BundleFallbackDemo {
    public static void main(String[] args) {
        // Requesting en_GB will look for: Messages_en_GB -> Messages_en
↳ -> Messages (base)
        ResourceBundle bundle = ResourceBundle.getBundle("Messages", new
↳ Locale("en", "GB"));
        System.out.println(bundle.getLocale());
        // Output depends on which files actually exist; falls back
↳ gracefully instead of throwing.
    }
}

```

```

        // Requesting a totally unsupported locale (e.g. Icelandic) still
        ↪ works,
        // it just falls all the way back to the base Messages.properties
        ↪ file.
        ResourceBundle fallback = ResourceBundle.getBundle("Messages",
        ↪ new Locale("is", "IS"));
        System.out.println(fallback.getBaseBundleName()); // Messages
    }
}

```

Requested locale	Lookup order (most to least specific)
de_CH (German, Switzerland)	Messages_de_CH -> Messages_de -> Messages
en_GB (English, UK)	Messages_en_GB -> Messages_en -> Messages
is_IS (Icelandic, Iceland) - no matching files	Messages_is_IS -> Messages_is -> Messages (base)

Pseudo-localization

Pseudo-localization is a testing technique: you generate a fake “translation” that expands text length, adds accented characters, and wraps strings with markers, without doing real translation. It quickly reveals UI bugs (truncated text, hardcoded string concatenation, untranslated strings) before real translators get involved.

```

import java.util.Locale;
import java.util.ResourceBundle;

public class PseudoLocalizationDemo {

    // A crude pseudo-localizer: wraps text and pads length by ~40%, which
    ↪ is
    // roughly how much German/French text tends to expand versus English.
    static String pseudoLocalize(String text) {
        String expanded = text.replace("a", "àà").replace("e",
        ↪ "éé").replace("o", "òò");
        return "[" + expanded + "];"
    }

    public static void main(String[] args) {

```

```

        ResourceBundle bundle = ResourceBundle.getBundle("Messages",
↳ Locale.ENGLISH);
        String original = bundle.getString("greeting"); // "Hello, {0}!"

        System.out.println(pseudoLocalize(original));
        // Output: [Héééllòò, {0}!]

        // If the UI truncates this or breaks layout, it will likely break
↳ for real
        // translations too (German and Finnish strings are often 30-50%
↳ longer than English).
    }
}

```

Testing with -Duser.language

You can override the JVM's default locale from the command line without changing any code, which is very useful for manual and automated locale testing.

```

import java.util.Locale;

public class DefaultLocaleDemo {
    public static void main(String[] args) {
        System.out.println("Default locale: " + Locale.getDefault());
        // Run with: java -Duser.language=de -Duser.country=DE
↳ DefaultLocaleDemo
        // Output: Default locale: de_DE

        // Run with: java -Duser.language=tr -Duser.country=TR
↳ DefaultLocaleDemo
        // Output: Default locale: tr_TR
        // (useful to specifically reproduce the Turkish dotless-i bug
↳ shown earlier)
    }
}

```

```

// A small JUnit-style test that temporarily swaps the default locale to
↳ verify behavior.
import java.util.Locale;
import org.junit.jupiter.api.*;

class LocaleSensitiveTest {
    private Locale originalLocale;

```

```

@BeforeEach
void saveLocale() {
    originalLocale = Locale.getDefault();
}

@AfterEach
void restoreLocale() {
    Locale.setDefault(originalLocale); // always restore - global
    ↪ state affects other tests!
}

@Test
void formatsCorrectlyUnderGermanLocale() {
    Locale.setDefault(new Locale("de", "DE"));
    String result = String.format("%.2f", 1234.5);
    Assertions.assertEquals("1.234,50", result);
}
}

```

CLDR as the default locale data provider

CLDR (Unicode Common Locale Data Repository) is the industry-standard source of locale data — date/time patterns, number formats, currency symbols, plural rules, and more — maintained collaboratively (contributors include Google, Apple, Microsoft, IBM, and others). Since **JDK 9**, the JDK uses **CLDR** as the default locale data provider instead of its own older, less complete data (JRE-specific data, sometimes called “COMPAT”). This changed some formatting output compared to Java 8 — a real source of subtle behavior differences when upgrading old codebases.

```

import java.text.NumberFormat;
import java.util.Locale;

public class CldrProviderDemo {
    public static void main(String[] args) {
        System.out.println(System.getProperty("java.locale.providers"));
        // Output (JDK 9+): null by default (meaning CLDR is used
        ↪ implicitly), or "CLDR" if explicitly set

        NumberFormat currency =
        ↪ NumberFormat.getCurrencyInstance(Locale.US);
        System.out.println(currency.format(1234.5));
        // Output (CLDR, JDK 9+): $1,234.50
        // On JDK 8 with the old COMPAT provider, some locales format
        ↪ currency slightly
    }
}

```

```

        // differently (e.g. different symbol placement or spacing for
        ↪ certain locales).

        // You can force the old JDK 8 behavior (not recommended, mostly
        ↪ for migration debugging):
        // java -Djava.locale.providers=COMPAT,CLDR CldrProviderDemo
    }
}

```

Common Code-Review Interview Pitfalls

- 1. Discarding the result of an immutable plus/minus/with call.** Why it matters: `java.time` types are immutable — the call does nothing unless you capture the return value. This is the single most common `java.time` bug.

```

// Before (bug): does nothing, "deadline" is unchanged
deadline.plusDays(7);

// After (fixed): capture the new value
deadline = deadline.plusDays(7);

```

- 2. Sharing a mutable `SimpleDateFormat` as a static field across threads.** Why it matters: `SimpleDateFormat` is not thread-safe; concurrent `format()/parse()` calls can corrupt results or throw exceptions.

```

// Before (bug): shared mutable state
static final SimpleDateFormat FMT = new SimpleDateFormat("yyyy-MM-dd");

// After (fixed): DateTimeFormatter is immutable and thread-safe
static final DateTimeFormatter FMT =
    ↪ DateTimeFormatter.ofPattern("yyyy-MM-dd");

```

- 3. Storing `LocalDateTime` for events that need an absolute point in time.** Why it matters: `LocalDateTime` has no zone, so “2026-08-07T14:00” is ambiguous across regions and can’t be safely compared or converted.

```

// Before (bug): ambiguous, no zone info
LocalDateTime eventTime = LocalDateTime.now();

// After (fixed): store an absolute instant, plus the zone if you need
    ↪ to redisplay locally
Instant eventInstant = Instant.now();

```

```
String zoneId = "Europe/Amsterdam";
```

- 4. Using YYYY (week-year) instead of yyyy (calendar year) in a pattern.** Why it matters: near year boundaries, the week-based year can differ from the calendar year, silently shifting dates by one year.

```
// Before (bug): week-year, wrong for calendar dates
DateTimeFormatter.ofPattern("YYYY-MM-dd");

// After (fixed): calendar year
DateTimeFormatter.ofPattern("yyyy-MM-dd");
```

- 5. Calling toUpperCase()/toLowerCase() without a Locale for internal comparisons.** Why it matters: the Turkish locale (and others) can transform i/I unexpectedly, breaking comparisons against ASCII constants.

```
// Before (bug): depends on default locale
if (input.toUpperCase().equals("ID")) { ... }

// After (fixed): locale-independent
if (input.equalsIgnoreCase("ID")) { ... }
```

- 6. Calling .now() directly inside business logic instead of injecting a Clock.** Why it matters: hardcoded “now” calls make unit tests flaky or impossible to make deterministic.

```
// Before (hard to test)
LocalDate today = LocalDate.now();

// After (testable)
LocalDate today = LocalDate.now(clock); // clock injected,
↳ Clock.fixed(...) in tests
```

- 7. Assuming “add one day” always adds exactly 24 hours.** Why it matters: DST transitions mean a calendar day can be 23 or 25 real hours; use `Duration` for exact elapsed time, `Period`/`plusDays` for calendar-based reasoning.

```
// Before (bug): assumes 24 real hours always pass
Instant next = instant.plus(Duration.ofDays(1));

// After (fixed): use calendar semantics when you mean "the next
↳ calendar day"
```

```
ZonedDateTime next = zonedDateTime.plusDays(1);
```

8. Comparing LocalDateTime values that originated from different time zones.

Why it matters: `LocalDateTime` ignores zone offsets entirely, so “later on the clock” does not mean “later in reality.”

```
// Before (bug): compares wall-clock time, ignoring zone
boolean isLater = a.toLocalDateTime().isAfter(b.toLocalDateTime());

// After (fixed): compare actual instants
boolean isLater = a.toInstant().isAfter(b.toInstant());
```

9. Letting String.format/NumberFormat use the JVM default locale in shared/server code.

Why it matters: default locale can differ between machines and environments, causing inconsistent output (“works on my machine” bugs, wrong decimal separators sent to clients).

```
// Before (bug): implicit, environment-dependent
String.format("%.2f", price);

// After (fixed): explicit locale
String.format(Locale.US, "%.2f", price);
```

10. Not handling DateTimeParseException at boundaries that accept external date strings.

Why it matters: it’s an unchecked exception; forgetting to catch it lets malformed user/API input crash a request instead of returning a clean validation error.

```
// Before (bug): unhandled, crashes on bad input
LocalDate date = LocalDate.parse(userInput);

// After (fixed): handled explicitly
try {
    LocalDate date = LocalDate.parse(userInput);
} catch (DateTimeParseException e) {
    throw new InvalidRequestException("Invalid date: " + userInput);
}
```

11. Using String.length() to validate or truncate text that may contain emoji or combining characters.

Why it matters: `length()` counts UTF-16 code units, not visible characters, so limits can cut a string mid-character or reject valid short input.


```
// Before (bug): may split a surrogate pair or combining sequence
String truncated = input.substring(0, Math.min(input.length(), 10));

// After (better): use code point aware truncation, or a grapheme-aware
// BreakIterator for user-facing character counts
int end = input.offsetByCodePoints(0, Math.min(10,
↳ input.codePointCount(0, input.length())));
String truncated = input.substring(0, end);
```

- 12. Comparing Unicode text with `.equals()` without normalizing first.** Why it matters: visually identical strings can have different code point sequences (precomposed vs. decomposed accents), causing “duplicate” entries or failed lookups.

```
// Before (bug): may treat identical-looking text as different
if (nameFromUserA.equals(nameFromUserB)) { ... }

// After (fixed): normalize first
if (Normalizer.normalize(nameFromUserA, Normalizer.Form.NFC)
    .equals(Normalizer.normalize(nameFromUserB,
↳ Normalizer.Form.NFC))) { ... }
```

- 13. Assuming the platform default charset is always UTF-8 on any JDK version.** Why it matters: before JDK 18, the default charset was platform-dependent; code relying on the implicit default can misbehave when moved to another OS or an older JDK.

```
// Before (risky on JDK < 18 / non-UTF-8 platforms): implicit charset
byte[] bytes = text.getBytes();

// After (fixed): always explicit, safe on any JDK version
byte[] bytes = text.getBytes(StandardCharsets.UTF_8);
```

- 14. Hardcoding date/number formats instead of using locale-aware formatters for user-facing text.** Why it matters: users expect dates and numbers formatted the way their own locale/region does, not a fixed pattern chosen by the developer.

```
// Before (bug): fixed pattern regardless of user's locale
DateTimeFormatter.ofPattern("MM/dd/yyyy");

// After (fixed): adapts to the user's locale automatically
DateTimeFormatter.ofLocalizedDate(FormatStyle.MEDIUM).withLocale(
↳ userLocale);
```

- 15. Sorting user-facing text with `String.compareTo` instead of a locale-aware `Collator`.** Why it matters: plain code-unit comparison does not match human alphabetical order for accented letters, case, or non-Latin scripts, producing confusing sort order in UIs.

```
// Before (bug): raw code-unit sort, wrong for most human languages  
Arrays.sort(names);  
  
// After (fixed): locale-aware collation  
Arrays.sort(names, Collator.getInstance(userLocale));
```

File I/O

Every real program eventually has to read or write something outside its own memory: a config file, a log, a network payload, a saved object. Java gives you three overlapping toolkits for this — the old `java.io` streams, the newer NIO/NIO.2 (`java.nio`) APIs, and Java’s built-in object serialization. Each one solves a slightly different problem, and each one has sharp edges that show up constantly in code review. This chapter walks through all three, plus the modern `Path/Files` API that has mostly replaced `java.io.File`.

Table of Contents

- [java.io \(File I/O\)](#)
- [NIO and NIO.2](#)
- [Paths and Files](#)
- [Serialization](#)
- [Externalizable](#)
- [Common Code-Review Interview Pitfalls](#)

java.io (File I/O)

`java.io` is the original Java I/O library, dating back to Java 1.0. It is *stream-based*: you read or write one chunk at a time, in order, from start to end. There is no random access and no built-in buffering — you add both yourself.

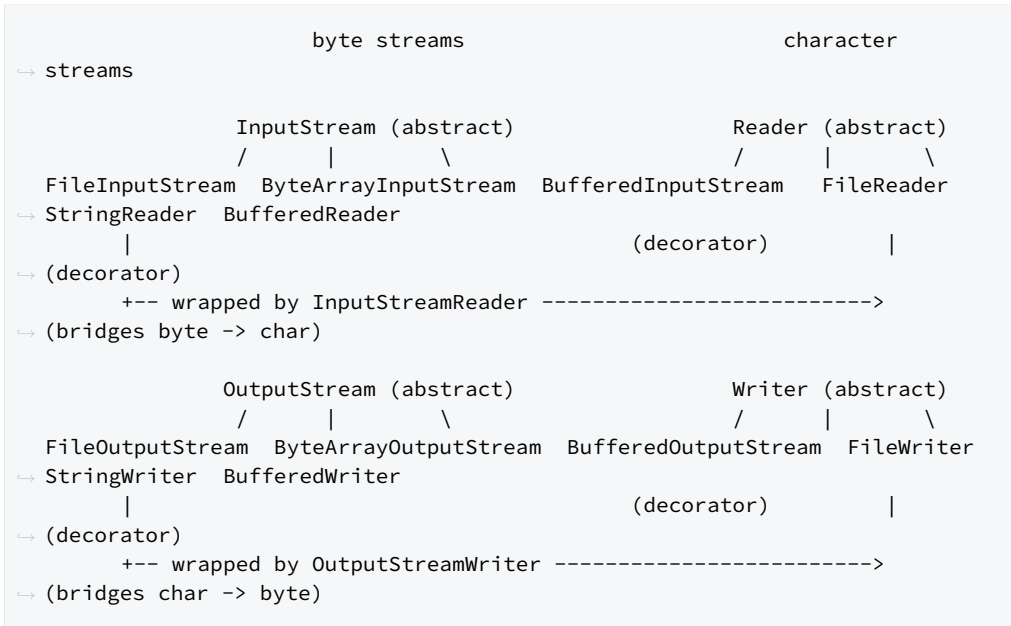
The most important split in `java.io` is **byte streams vs. character streams**.

- **Byte streams** (`InputStream / OutputStream`) move raw bytes (0–255). Use them for binary data: images, zip files, network sockets, serialized objects.

- **Character streams** (`Reader` / `Writer`) move `char` values (text). Internally they still read/write bytes, but they convert bytes to characters using a **charset** (a character encoding table, e.g. UTF-8). Use them for text files.

Mixing the two up is a classic bug: reading text with a byte stream and hand-rolling your own encoding logic, or reading binary data with a character stream and corrupting bytes during “encoding conversion” that binary data was never meant to go through.

The class hierarchy



`InputStreamReader` and `OutputStreamWriter` are the bridge classes: they wrap a byte stream and a charset, and expose it as a character stream. `FileReader` and `FileWriter` are just convenience subclasses of that bridge, hardwired to a `File` — historically they used the *platform default charset*, which is exactly why you should avoid their no-charset constructors (see below).

InputStream / OutputStream / Reader / Writer

These four abstract classes are the roots of everything in `java.io`.

```

import java.io.*;
import java.nio.charset.StandardCharsets;

public class BasicStreams {
    public static void main(String[] args) throws IOException {
        // Byte stream: read raw bytes from a file
        try (InputStream in = new FileInputStream("data.bin")) {
            int b;
            while ((b = in.read()) != -1) {    // read() returns -1 at end
                // process one byte at a time (slow without buffering, see
                // below)
            }

            // Character stream: read text, one char at a time
            try (Reader reader = new InputStreamReader(
                new FileInputStream("notes.txt"),
                StandardCharsets.UTF_8)) {
                int c;
                while ((c = reader.read()) != -1) {
                    // process one character
                }
            }
        }
    }
}

```

The decorator pattern

java.io is built almost entirely on the **decorator pattern**: small classes that wrap another stream and add one piece of behavior, without changing the type. You “stack” decorators to combine behaviors.

```

import java.io.*;
import java.nio.charset.StandardCharsets;

// FileInputStream: raw bytes from disk
// -> wrapped by BufferedInputStream: adds an internal buffer (fewer OS
// calls)
// -> wrapped by InputStreamReader: converts bytes to chars using UTF-8
// -> wrapped by BufferedReader: adds line-based reading (readLine())
try (BufferedReader reader = new BufferedReader(
    new InputStreamReader(
        new BufferedInputStream(new
        FileInputStream("report.txt")),

```

```

        StandardCharsets.UTF_8))) {

    String line;
    while ((line = reader.readLine()) != null) {
        System.out.println(line);
    }
}

```

Each layer only knows about the layer directly underneath it. That is the whole point of the pattern: `BufferedReader` does not care whether its underlying `Reader` is a file, a socket, or an in-memory string.

Why buffering matters

Every unbuffered `read()`/`write()` call on a `FileInputStream` or `FileOutputStream` typically triggers a system call to the OS. System calls are expensive compared to in-memory operations — doing one per byte is enormously wasteful.

```

import java.io.*;

// Bad: one system call per byte. On a 1 MB file, that's ~1,000,000
↪ syscalls.
try (InputStream in = new FileInputStream("big.dat")) {
    int b;
    while ((b = in.read()) != -1) {
        // ...
    }
}

// Good: BufferedInputStream reads a large chunk (default 8 KB) into
↪ memory once,
// then serves read() calls from that in-memory buffer. Far fewer
↪ syscalls.
try (InputStream in = new BufferedInputStream(new
↪ FileInputStream("big.dat"))) {
    int b;
    while ((b = in.read()) != -1) {
        // ...
    }
}

```

The rule of thumb in review: **any raw `FileInputStream`/`FileOutputStream`/`FileReader`/`FileWriter` that isn't wrapped in a `Buffered*` decorator is a performance smell**, unless the code already reads/writes in large chunks itself (e.g., `readAllBytes()`).

Always specify a charset

Text is just bytes plus an agreement on how to decode them. If you don't specify a charset, Java uses the **platform default charset**, which depends on the OS, locale, and JVM flags. That means the exact same code can produce different (or garbled) output on different machines — a Linux CI server, a developer's macOS laptop, and a Windows box might all disagree.

```
import java.io.*;
import java.nio.charset.StandardCharsets;

// Bad: charset is whatever the platform happens to default to (risky, not
→ portable)
Writer badWriter = new FileWriter("out.txt");

// Good: charset is explicit and reproducible everywhere
Writer goodWriter = new OutputStreamWriter(
    new FileOutputStream("out.txt"), StandardCharsets.UTF_8);

// Java 11+ convenience: FileReader/FileWriter overloads that accept a
→ Charset directly
Writer goodWriterJava11 = new FileWriter("out.txt",
→ StandardCharsets.UTF_8);
```

Since Java 18, the JVM's default charset for most contexts is UTF-8 (JEP 400), which fixes a lot of historical foot-guns. But relying on “the default happens to be UTF-8 now” is still fragile — a reviewer should still flag missing explicit charsets, since the JVM can still be launched with `-Dfile.encoding=...` overriding it, and library code should never assume the caller's environment.

File and its weak error reporting

`java.io.File` represents a path on the filesystem — but it is notorious for **silently returning false instead of throwing an exception** when something goes wrong. This makes bugs hard to diagnose because you get no information about *why* an operation failed.

```
import java.io.File;

File file = new File("/some/protected/path/data.txt");

boolean deleted = file.delete();
if (!deleted) {
```

```

        // Was it "file doesn't exist"? "Permission denied"? "Directory not
        ↪ empty"?
        // File.delete() gives you NO clue. You just get `false`.
        System.out.println("Delete failed, but we don't know why.");
    }

    boolean created = file.mkdir();
    if (!created) {
        // Same problem: could be "parent doesn't exist", "already exists",
        // "permission denied" – all collapse into a single boolean.
        System.out.println("mkdir failed, reason unknown.");
    }

```

This is one of the concrete reasons `java.nio.file.Files` (covered below) exists: `Files.delete()` and `Files.createDirectory()` throw specific, informative exceptions (`NoSuchFileException`, `FileAlreadyExistsException`, `AccessDeniedException`, `DirectoryNotEmptyException`) instead of a flat `boolean`.

try-with-resources for streams

All `java.io` streams implement `Closeable` (which extends `AutoCloseable`), so they work with `try-with-resources`. Forgetting to close a stream leaks file handles or sockets — on a long-running server this eventually exhausts OS file-descriptor limits.

```

import java.io.*;
import java.nio.charset.StandardCharsets;

// Bad: if readLine() throws, the stream never gets closed
BufferedReader reader = new BufferedReader(
    ↪ new InputStreamReader(new FileInputStream("data.txt"),
    StandardCharsets.UTF_8));
String line = reader.readLine();
reader.close(); // never reached if an exception is thrown above

// Good: try-with-resources guarantees close(), even on exception,
// and closes multiple resources in reverse order of declaration
try (FileInputStream fis = new FileInputStream("data.txt");
    InputStreamReader isr = new InputStreamReader(fis,
    ↪ StandardCharsets.UTF_8);
    BufferedReader reader2 = new BufferedReader(isr)) {

    String firstLine = reader2.readLine();
    System.out.println(firstLine);
}

```



```
} // fis, isr, and reader2 are all closed automatically here
```

NIO and NIO.2

“NIO” (New I/O, `java.nio`, added in Java 1.4) and “NIO.2” (the `java.nio.file` package, added in Java 7) are a different model from `java.io`. Instead of a one-directional stream of bytes, NIO works with **buffers** and **channels**, and supports non-blocking and memory-mapped I/O. NIO.2 additionally gave us the modern Path/Files API, covered in the next section.

Buffers and channels

A **channel** (`FileChannel`, `SocketChannel`, ...) is a connection to an I/O source — a file, a socket. Unlike a `Stream`, a channel can be read from *and* written to, and (for files) supports random access via `position()`.

A **buffer** (`ByteBuffer`, `CharBuffer`, ...) is a fixed-size block of memory that channels read into and write from. You don’t read byte-by-byte with NIO; you read a whole chunk into a buffer, then process the buffer.

```
import java.io.IOException;
import java.nio.ByteBuffer;
import java.nio.channels.FileChannel;
import java.nio.file.Path;
import java.nio.file.StandardOpenOption;

public class ChannelBufferExample {
    public static void main(String[] args) throws IOException {
        Path path = Path.of("data.bin");

        try (FileChannel channel = FileChannel.open(path,
            ↪ StandardOpenOption.READ)) {
            ↪ buffer
            ByteBuffer buffer = ByteBuffer.allocate(1024); // 1 KB heap

            int bytesRead = channel.read(buffer); // fills the buffer,
            ↪ returns count (-1 = EOF)
            System.out.println("Read " + bytesRead + " bytes");
        }
    }
}
```

ByteBuffer: position, limit, flip, clear

A `ByteBuffer` tracks three cursors you must understand to use it correctly:

- **capacity** — total size, fixed at creation.
- **position** — where the next read/write happens; advances as you go.
- **limit** — how far you're allowed to read/write; can't go past it.

```
import java.nio.ByteBuffer;
import java.nio.charset.StandardCharsets;

ByteBuffer buffer = ByteBuffer.allocate(128);

// --- WRITE mode ---
buffer.put("hello".getBytes(StandardCharsets.UTF_8));
// position = 5 (just wrote 5 bytes), limit = 128 (capacity)

// flip(): switch from writing to reading.
// Sets limit = current position, then resets position = 0.
buffer.flip();
// position = 0, limit = 5 -- ready to read exactly the 5 bytes we wrote

// --- READ mode ---
byte[] data = new byte[buffer.remaining()]; // remaining() = limit -
↳ position
buffer.get(data);
System.out.println(new String(data, StandardCharsets.UTF_8)); // "hello"
// position = 5, limit = 5

// clear(): reset for writing again.
// Sets position = 0, limit = capacity. Does NOT erase old data, just
↳ forgets it.
buffer.clear();
// position = 0, limit = 128 -- ready to write fresh data
```

Forgetting to call `flip()` before reading is one of the most common NIO bugs: you'll try to read starting at `position` (wherever writing left it) up to `limit` (still the full capacity), which usually reads garbage or nothing useful.

Direct vs. heap buffers

```
import java.nio.ByteBuffer;

ByteBuffer heapBuffer = ByteBuffer.allocate(1024);           // lives in JVM
↪ heap
ByteBuffer directBuffer = ByteBuffer.allocateDirect(1024); // lives
↪ outside the JVM heap
```

	Heap buffer (allocate)	Direct buffer (allocateDirect)
Memory location	JVM heap	Native (OS) memory, outside GC heap
Allocation cost	Cheap, fast	More expensive to allocate/free
I/O performance	JVM may copy to a temp native buffer before an OS call	Can be passed straight to the OS, no extra copy
Best for	Small, short-lived, frequently allocated buffers	Large, long-lived buffers reused for repeated I/O
GC impact	Collected normally	Freed via cleaner mechanisms, not standard GC

For most application code, heap buffers are fine and simpler. Direct buffers pay off for high-throughput I/O (e.g., network servers, bulk file copies) where you reuse the same buffer many times.

Memory-mapped files

`FileChannel.map()` maps a file's contents directly into memory, so you can treat file bytes like an in-memory array — the OS handles paging data in and out as you access it. This is great for large files you need random access into (e.g., a large binary index) without reading the whole thing.

```
import java.io.IOException;
import java.nio.MappedByteBuffer;
import java.nio.channels.FileChannel;
import java.nio.file.Path;
import java.nio.file.StandardOpenOption;
```

```

try (FileChannel channel = FileChannel.open(
    Path.of("large-file.dat"), StandardOpenOption.READ)) {

    MappedByteBuffer mapped = channel.map(
        FileChannel.MapMode.READ_ONLY, 0, channel.size());

    // Access any byte at any offset directly -- OS pages it in as needed
    byte firstByte = mapped.get(0);
    byte lastByte = mapped.get((int) channel.size() - 1);
}

```

Caveats: mapped regions are capped by addressable memory (practically, ~2 GB per mapping on many JVMs due to `int` indexing), and unmapping is not guaranteed to happen promptly — on some platforms the mapped file may stay locked until the buffer is garbage collected.

Selector and non-blocking I/O

Regular blocking I/O ties up one thread per connection: the thread calls `read()` and just sits there until data arrives. That doesn't scale to thousands of connections. NIO's `Selector` lets a **single thread monitor many channels at once**, and only wakes up to handle the channels that are actually ready (readable, writable, or have a new connection) — this is the classic “one thread, many sockets” event-loop model used by things like Netty. In modern Java, this specific problem is often solved more simply with **virtual threads** (Project Loom, Java 21) instead of hand-rolled `Selector` loops, but understanding `Selector` still matters for reading legacy networking code and frameworks built on it.

`FileChannel.transferTo` (zero-copy)

`transferTo` can copy data directly between channels, and on many operating systems this happens via a **zero-copy** OS call — the data never has to pass through a Java buffer in user space at all.

```

import java.io.IOException;
import java.nio.channels.FileChannel;
import java.nio.file.Path;
import java.nio.file.StandardOpenOption;

try (FileChannel source = FileChannel.open(Path.of("source.dat"),
    ↪ StandardOpenOption.READ);
    FileChannel destination = FileChannel.open(

```

```

        Path.of("destination.dat"),
        StandardOpenOption.CREATE, StandardOpenOption.WRITE)) {

    long transferred = 0;
    long size = source.size();
    while (transferred < size) {
        transferred += source.transferTo(transferred, size - transferred,
        ↪ destination);
    }
}

```

For plain file-to-file copies, though, `Files.copy(Path, Path, ...)` (NIO.2) is simpler and just as efficient in practice — reach for `transferTo` mainly when you're moving data channel-to-channel (e.g., file to socket) rather than file to file.

When NIO actually helps

Use case	Better fit
Simple file read/write in a small tool or CLI	<code>java.io / Files</code> (NIO.2) — simpler code
Large file, need random access to specific offsets	NIO <code>FileChannel</code> + memory mapping
High-throughput network server, many connections	NIO <code>Selector</code> , or virtual threads
Bulk copy between files or channels	<code>FileChannel.transferTo / Files.copy</code>
Everyday text/config file processing	<code>Files.readString, Files.lines</code> (NIO.2)

Reviewer's rule of thumb: don't reach for raw `ByteBuffer/Channel/Selector` code unless there's a real reason (performance-critical I/O, non-blocking networking, memory-mapped access). For ordinary file reading and writing, NIO.2's `Files` API (next section) is simpler, less error-prone, and just as fast.

Paths and Files

NIO.2 (Java 7+) introduced `java.nio.file.Path` and `java.nio.file.Files` to replace most uses of `java.io.File`. `Path` represents a filesystem location (it doesn't have to exist); `Files` is a utility class full of static methods that operate on `Paths`.

	<code>java.io.File</code>	NIO.2 (Path / Files)
Error reporting	Booleans (false on failure, no reason)	Specific exceptions (NoSuchFileException, etc.)
Symbolic link support	Poor / inconsistent	First-class (<code>Files.isSymbolicLink</code> , etc.)
File attributes	Limited (<code>lastModified</code> , <code>length</code>)	Rich (<code>BasicFileAttributes</code> , POSIX permissions, owner)
Directory walking	Manual recursion	<code>Files.walk</code> , <code>Files.walkFileTree</code>
Watching for changes	Not supported	<code>WatchService</code>
Reading whole file as text	Manual stream wiring	<code>Files.readString</code> (one line)
Extensibility	Tied to default filesystem	Pluggable filesystem providers (e.g., zip, in-memory)

Creating and manipulating Paths

```

import java.nio.file.Path;
import java.nio.file.Paths;

// Two equivalent ways to build a Path. Path.of() is preferred since Java
→ 11
// (Paths.get() still works and is common in older code / examples).
Path p1 = Paths.get("data", "input.csv");
Path p2 = Path.of("data", "input.csv");

System.out.println(p1.equals(p2)); // true

Path base = Path.of("/home/user/project");
Path resolved = base.resolve("src/Main.java");
System.out.println(resolved); // /home/user/project/src/Main.java

Path relative = base.relativeTo(Path.of("/home/user/other/lib.jar"));
System.out.println(relative); // ../other/lib.jar

Path messy = Path.of("/home/user/./project/../project/src");
System.out.println(messy.normalize()); // /home/user/project/src

Path relPath = Path.of("relative/path");
System.out.println(relPath.toAbsolutePath()); // resolved against the
→ current working directory

```

- `resolve` — joins a path onto another (like appending a subpath).
- `relativeTo` — computes the relative path needed to get from one path to another.
- `normalize` — removes redundant `.` and `..` segments *syntactically*, without touching the filesystem.
- `toAbsolutePath` — anchors a relative path to the current working directory (still doesn't check if it exists).

Reading and writing whole files

```

import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.List;
import java.util.stream.Stream;

Path file = Path.of("notes.txt");

```

```

// Read the entire file as one String (Java 11+)
String content = Files.readString(file, StandardCharsets.UTF_8);

// Write a String to a file, creating or truncating it (Java 11+)
Files.writeString(file, "Hello, NIO.2!\n", StandardCharsets.UTF_8);

// Read all lines into a List<String> -- fine for small/medium files
List<String> lines = Files.readAllLines(file, StandardCharsets.UTF_8);

// Stream lines lazily -- for large files, so you don't load everything
↳ into memory.
// IMPORTANT: Files.lines() opens a file handle that MUST be closed.
try (Stream<String> lineStream = Files.lines(file,
↳ StandardCharsets.UTF_8)) {
    long count = lineStream.filter(line -> !line.isBlank()).count();
    System.out.println("Non-blank lines: " + count);
} // stream (and its underlying file handle) closed here

```

`Files.lines()` returns a lazily-evaluated `Stream<String>` backed by an open file descriptor. If you don't close it (via `try-with-resources`, since `Stream` implements `AutoCloseable`), the file handle leaks — this is easy to miss because a `Stream` doesn't *look* like a resource the way an `InputStream` obviously does.

```

import java.io.BufferedReader;
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;

// Files.newBufferedReader gives you a fully-buffered Reader with a chosen
↳ charset,
// useful when you want line-by-line control instead of a Stream or a full
↳ readAllLines()
try (BufferedReader reader = Files.newBufferedReader(Path.of("big.log"),
↳ StandardCharsets.UTF_8)) {
    String line;
    while ((line = reader.readLine()) != null) {
        // process one line at a time, low memory overhead
    }
}

```


Copy, move, delete

```
import java.io.IOException;
import java.nio.file.CopyOption;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.StandardCopyOption;

Path source = Path.of("report.pdf");
Path target = Path.of("backup/report.pdf");

// Copy, overwriting the target if it already exists
Files.copy(source, target, StandardCopyOption.REPLACE_EXISTING);

// Move (rename) a file; ATOMIC_MOVE guarantees no partial/half-written
↪ result is ever visible
Files.move(source, target, StandardCopyOption.ATOMIC_MOVE);

// Delete: throws NoSuchFileException if missing (unlike File.delete()'s
↪ silent 'false')
Files.delete(target);

// deleteIfExists: same, but does nothing (no exception) if the file is
↪ already gone
Files.deleteIfExists(target);
```

Files.walk vs. Files.walkFileTree

Both traverse a directory tree, but they differ in flexibility and resource handling.

```
import java.io.IOException;
import java.nio.file.*;
import java.nio.file.attribute.BasicFileAttributes;
import java.util.stream.Stream;

// Files.walk: returns a lazy Stream<Path> of every file/dir under root.
// Simple for filtering/collecting, but MUST be closed (it holds an open
↪ directory handle).
try (Stream<Path> paths = Files.walk(Path.of("src"))) {
    paths.filter(p -> p.toString().endsWith(".java"))
        .forEach(System.out::println);
}

// Files.walkFileTree: callback-based (visitor pattern), more control,
```

```

// e.g. skipping whole subtrees, handling errors per-file, computing
↪ directory sizes.
Files.walkFileTree(Path.of("src"), new SimpleFileVisitor<Path>() {
    @Override
    public FileVisitResult visitFile(Path file, BasicFileAttributes
↪ attrs) {
        if (file.toString().endsWith(".class")) {
            return FileVisitResult.CONTINUE; // could delete, count, etc.
        }
        return FileVisitResult.CONTINUE;
    }

    @Override
    public FileVisitResult visitFileFailed(Path file, IOException exc) {
        System.err.println("Could not visit: " + file + " (" + exc + ")");
        return FileVisitResult.CONTINUE; // don't abort the whole walk on
↪ one bad file
    }
});

```

Use `Files.walk` for quick stream-style filtering/collecting. Use `Files.walkFileTree` when you need fine-grained control (skip directories, react to errors per file, avoid loading the whole path list into a stream pipeline first).

Temp files, DirectoryStream, WatchService, attributes

```

import java.io.IOException;
import java.nio.file.DirectoryStream;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.attribute.FileAttribute;
import java.nio.file.attribute.PosixFilePermission;
import java.nio.file.attribute.PosixFilePermissions;
import java.util.Set;

// Secure temp file: created with a random name, and on POSIX systems
↪ Files.createTempFile
// sets permissions so only the owner can read/write it (0600) -- avoids
↪ TOCTOU race
// conditions and world-readable secrets that java.io.File.createTempFile
↪ historically allowed.
Path tempFile = Files.createTempFile("upload-", ".tmp");
System.out.println(tempFile); // e.g. /tmp/upload-1234567890.tmp

```

```

    // DirectoryStream: lazily iterate direct children of a directory (not
    ↪ recursive),
    // with an optional glob filter. Must be closed -- holds a directory
    ↪ handle.
    try (DirectoryStream<Path> stream =
    ↪ Files.newDirectoryStream(Path.of("."), "*.java")) {
        for (Path entry : stream) {
            System.out.println(entry.getFileName());
        }
    }

    // PosixFilePermissions: set exact rwx bits when creating a file, useful
    ↪ for
    // "this file must only be readable by its owner" security requirements
    Set<PosixFilePermission> perms =
    ↪ PosixFilePermissions.fromString("rw-----");
    FileAttribute<Set<PosixFilePermission>> attr =
    ↪ PosixFilePermissions.asFileAttribute(perms);
    Path secretFile = Files.createFile(Path.of("secret.key"), attr);

```

```

import java.io.IOException;
import java.nio.file.*;

    // WatchService: get notified when files in a directory change, without
    ↪ polling.
    // Common use: reload config when a file on disk is edited.
    try (WatchService watchService =
    ↪ FileSystems.getDefault().newWatchService()) {
        Path dir = Path.of("config");
        dir.register(watchService,
            StandardWatchEventKinds.ENTRY_CREATE,
            StandardWatchEventKinds.ENTRY_MODIFY,
            StandardWatchEventKinds.ENTRY_DELETE);

        WatchKey key = watchService.take(); // blocks until an event occurs
        for (WatchEvent<?> event : key.pollEvents()) {
            System.out.println(event.kind() + ": " + event.context());
        }
        key.reset(); // must call this to keep receiving events on this key
    }

```

Atomic writes: temp file + ATOMIC_MOVE

A common bug: writing directly to a target file. If the process crashes mid-write, readers can see a half-written, corrupted file. The safe pattern is to write to a temp file first, then

atomically rename it into place — readers only ever see the old complete file or the new complete file, never a partial one.

```
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.StandardCopyOption;

public class AtomicWrite {
    public static void writeAtomically(Path target, String content)
↳ throws IOException {
        // temp file in the SAME directory as target (atomic move requires
↳ same filesystem)
        Path tempFile = Files.createTempFile(target.getParent(), "tmp-",
↳ ".swap");
        try {
            Files.writeString(tempFile, content, StandardCharsets.UTF_8);
            // ATOMIC_MOVE: the OS guarantees this rename is indivisible
↳ --
            // any reader sees either the fully-old or fully-new file,
↳ never a mix
            Files.move(tempFile, target,
                StandardCopyOption.ATOMIC_MOVE,
↳ StandardCopyOption.REPLACE_EXISTING);
        } finally {
            Files.deleteIfExists(tempFile); // cleanup if the move never
↳ happened
        }
    }
}
```

Path traversal security (.. /)

If any part of a file path comes from user input, an attacker can inject `../` segments to escape the intended directory and read or write files elsewhere on the system — a classic **path traversal** vulnerability.

```
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;

// Bad: naively trusts user input, allows "../../etc/passwd" style
↳ attacks
```

```

    public byte[] readUserFileUnsafe(String userSuppliedName) throws
↳ IOException {
        Path path = Path.of("/var/app/uploads").resolve(userSuppliedName);
        return Files.readAllBytes(path); // could read ANY file the process
↳ can access
    }

    // Good: resolve, normalize, then verify the result is still inside the
↳ allowed root
    public byte[] readUserFileSafe(String userSuppliedName) throws
↳ IOException {
        Path root = Path.of("/var/app/uploads").toRealPath(); // resolves
↳ symlinks too
        Path requested = root.resolve(userSuppliedName).normalize();

        if (!requested.startsWith(root)) {
            throw new SecurityException("Path traversal attempt: " +
↳ userSuppliedName);
        }
        return Files.readAllBytes(requested);
    }

```

The check must happen **after** `resolve` + `normalize` (and ideally after resolving symlinks with `toRealPath()`), never before — normalizing first and checking after is the part reviewers most often see done wrong or skipped entirely.

Serialization

Serialization converts an object (and everything it references — its **object graph**) into a byte stream, so it can be saved to disk or sent over a network. **Deserialization** reverses the process, rebuilding the object graph from bytes.

```

import java.io.*;

record Point(int x, int y) implements Serializable {}

public class SerializationBasics {
    public static void main(String[] args) throws IOException,
↳ ClassNotFoundException {
        Point point = new Point(3, 4);

        // Serialize to a byte array (could just as easily be a
↳ FileOutputStream)
        byte[] bytes;

```

```

        try (ByteArrayOutputStream baos = new ByteArrayOutputStream();
             ObjectOutputStream oos = new ObjectOutputStream(baos)) {
            oos.writeObject(point);
            bytes = baos.toByteArray();
        }

        // Deserialize back into an object
        try (ObjectInputStream ois = new ObjectInputStream(new
↪ ByteArrayInputStream(bytes))) {
            Point restored = (Point) ois.readObject();
            System.out.println(restored); // Point[x=3, y=4]
        }
    }
}

```

To be serializable, a class just implements the empty marker interface `Serializable` — there are no methods to implement. The JVM uses reflection to walk the object’s fields and write them out.

serialVersionUID

Every `Serializable` class has an implicit **version ID** used to check that a serialized object matches the class trying to deserialize it. If you don’t declare it explicitly, the JVM computes one automatically from the class’s structure (fields, methods, etc.) — which means **any change to the class**, even something as harmless as adding a method, can change the computed ID and break deserialization of old data with an `InvalidClassException`.

```

import java.io.Serializable;

public class User implements Serializable {
    // Always declare this explicitly. It lets you evolve the class
    // (add fields, add methods) without silently breaking old serialized
↪ data.
    private static final long serialVersionUID = 1L;

    private String username;
    private int age;

    // getters/setters omitted
}

```

Rule for review: any `Serializable` class without an explicit `serialVersionUID` is a red flag — it works today but is a landmine for future changes.

transient

Marking a field `transient` excludes it from serialization. Use it for fields that shouldn't (or can't) be persisted: caches, derived/computed values, `Thread/Socket/Connection` handles, secrets that shouldn't be written to disk.

```
import java.io.Serializable;

public class Session implements Serializable {
    private static final long serialVersionUID = 1L;

    private String sessionId;
    private transient String cachedDisplayName; // recomputed on demand,
    ↪ not persisted
    private transient char[] temporaryToken;    // sensitive; must never
    ↪ hit disk

    // after deserialization, cachedDisplayName and temporaryToken are
    ↪ null/zero-valued,
    // NOT whatever they held before serialization
}
```

After deserialization, `transient` fields are reset to their default value (`null`, `0`, `false`, etc.) — the constructor is **not** called during deserialization, so you can't rely on constructor logic to re-populate them; use `readObject` (below) if you need to recompute them.

Custom writeObject / readObject

You can hook into the serialization process by declaring `private` methods with these exact signatures. The JVM calls them via reflection if present.

```
import java.io.*;

public class Account implements Serializable {
    private static final long serialVersionUID = 1L;

    private String owner;
    private transient double cachedBalance; // recomputed after load, not
    ↪ stored directly

    private void writeObject(ObjectOutputStream out) throws IOException {
        out.defaultWriteObject(); // writes all non-transient fields
    }
    ↪ normally
}
```

```

        out.writeDouble(computeAuditedBalance()); // then write something
↪ extra
    }

    private void readObject(ObjectInputStream in) throws IOException,
↪ ClassNotFoundException {
        in.defaultReadObject(); // reads all non-transient fields normally
        this.cachedBalance = in.readDouble(); // then read the extra value
↪ back
    }

    private double computeAuditedBalance() {
        return 0.0; // placeholder for real logic
    }
}

```

Typical uses: encrypting a sensitive field before writing it out (and decrypting on read), validating invariants after loading untrusted data, or handling fields that aren't naturally serializable (e.g., writing a Map's contents manually instead of the map object itself).

writeReplace and readResolve

These let a class substitute a *different* object during serialization (`writeReplace`) or deserialization (`readResolve`) — commonly used to enforce singletons or to serialize a stable “proxy” representation instead of the real object.

```

import java.io.*;

public class Singleton implements Serializable {
    private static final long serialVersionUID = 1L;

    public static final Singleton INSTANCE = new Singleton();

    private Singleton() {}

    // Without readResolve, deserialization would create a SECOND,
↪ distinct
    // instance via reflection, breaking the singleton guarantee.
    protected Object readResolve() {
        return INSTANCE; // replace the deserialized instance with the
↪ canonical one
    }
}

```


enum types are serialization-safe singletons for free (the JVM guarantees enum constants deserialize to the same instance) — that's one more reason the enum singleton pattern is generally preferred over a hand-rolled one.

Why Java serialization is a security disaster

`ObjectInputStream.readObject()` does something dangerous by design: it instantiates classes and calls code (`readObject`, constructors of superclasses in some cases, `readResolve`, etc.) **based purely on bytes that came from the input** — before your application logic gets any say in whether that data should be trusted.

If an attacker controls the byte stream, they can craft input referencing classes already on your classpath whose `readObject`/`finalize`/other methods have *side effects* — chaining several such classes together forms a **gadget chain** that can lead to arbitrary code execution, denial of service, or data exfiltration, all before your code ever sees the resulting object. This class of vulnerability caused real, high-severity CVEs in major frameworks throughout the 2010s (e.g., in Apache Commons Collections, WebLogic, Jenkins) and is why security guidance now generally treats **deserializing untrustworthy data with native Java serialization as unsafe by default**.

```
import java.io.*;

// DANGEROUS: never deserialize bytes from an untrusted source (network
↳ request,
// user upload, unauthenticated input) with plain ObjectInputStream.
public Object deserializeUntrusted(byte[] attackerControlledBytes) throws
↳ Exception {
    try (ObjectInputStream ois = new ObjectInputStream(
        new ByteArrayInputStream(attackerControlledBytes))) {
        return ois.readObject(); // could trigger a gadget chain before
↳ you ever inspect it
    }
}
```

Serialization filters (ObjectInputFilter, JDK 9+)

If you must use Java serialization (e.g., legacy systems), `ObjectInputFilter` lets you restrict *which classes* are allowed to be deserialized, rejecting everything else before it's fully constructed.

```

import java.io.*;

ObjectInputFilter filter = ObjectInputFilter.Config.createFilter(
    "com.example.model.*;java.base/*;!*"); // allow our package + JDK
↳ basics, reject all else

try (ObjectInputStream ois = new ObjectInputStream(new
↳ FileInputStream("data.ser"))) {
    ois.setObjectInputFilter(filter);
    Object result = ois.readObject(); // throws InvalidClassException if a
↳ disallowed class appears
}

```

You can also set a process-wide default filter via the `jdk.serialFilter` system property or security property, so every `ObjectInputStream` in the JVM is protected even in code you don't control.

The modern recommendation

Given the risks and the format's other downsides (JVM-only, brittle across class changes, opaque binary format that's hard to debug or audit), the strong modern recommendation is:

Avoid Java's built-in `Serializable` for anything crossing a trust boundary (network, files from outside your process, another service). Prefer JSON (Jackson, Gson) or a schema-based binary format (Protocol Buffers, Avro) instead.

```

// Instead of: implements Serializable +
↳ ObjectOutputStream/ObjectInputStream
// Prefer something like (using a JSON library):
//
// String json = objectMapper.writeValueAsString(user);
// User user = objectMapper.readValue(json, User.class);
//
// JSON/protobuf are language-agnostic, human-inspectable (JSON) or
// schema-validated (protobuf), and don't execute arbitrary code on parse.

```

`Serializable` still shows up legitimately for things that never leave a single trusted JVM process — e.g., objects passed between nodes in a cluster you fully control, or short-lived in-memory caching frameworks — but even there, many teams now default to a safer format out of caution and long-term maintainability.

Externalizable

Externalizable is an alternative to Serializable that extends it and hands you **full manual control** over the byte format — you write every field yourself, in `writeExternal`, and read every field yourself, in `readExternal`.

```
import java.io.*;

public class Product implements Externalizable {
    private String name;
    private double price;

    // Externalizable REQUIRES a public no-arg constructor.
    // The JVM calls this constructor first, THEN calls readExternal()
    // to populate the fields -- unlike Serializable, which uses
    ↪ reflection
    // to bypass the constructor entirely.
    public Product() {}

    public Product(String name, double price) {
        this.name = name;
        this.price = price;
    }

    @Override
    public void writeExternal(ObjectOutput out) throws IOException {
        out.writeUTF(name);
        out.writeDouble(price);
    }

    @Override
    public void readExternal(ObjectInput in) throws IOException {
        name = in.readUTF();
        price = in.readDouble();
    }

    @Override
    public String toString() {
        return "Product{name='" + name + "', price=" + price + "'}";
    }
}
```

```
import java.io.*;

public class ExternalizableDemo {
```

```

    public static void main(String[] args) throws IOException,
↳ ClassNotFoundException {
        Product product = new Product("Widget", 9.99);

        byte[] bytes;
        try (ByteArrayOutputStream baos = new ByteArrayOutputStream();
             ObjectOutputStream oos = new ObjectOutputStream(baos)) {
            oos.writeObject(product);
            bytes = baos.toByteArray();
        }

        try (ObjectInputStream ois = new ObjectInputStream(new
↳ ByteArrayInputStream(bytes))) {
            Product restored = (Product) ois.readObject(); // calls
↳ Product() then readExternal()
            System.out.println(restored); // Product{name='Widget',
↳ price=9.99}
        }
    }
}

```

Serializable vs. Externalizable

	Serializable	Externalizable
Implementation effort	None (marker interface) — JVM uses reflection	Must implement <code>writeExternal/readExternal</code> yourself
No-arg constructor	Not required (fields set via reflection, bypassing constructors)	Required and must be public
Performance	Slower — reflection over all fields	Can be faster — you control exactly what's written, no reflection over field metadata
Format control	Default format mirrors class layout; can customize via <code>writeObject/readObject</code>	Total control — you decide every byte
Versioning safety	<code>serialVersionUID</code> + default mechanism handles a lot automatically	You must handle format evolution entirely yourself
Risk of mistakes	Lower — JVM does the heavy lifting	Higher — a forgotten field, wrong read order, or missing no-arg constructor breaks everything silently or loudly
Typical use	Most day-to-day cases (when serialization is genuinely needed)	Performance-critical or space-constrained scenarios, or when you need a stable, hand-controlled wire format

The trade-off in one sentence: `Externalizable` gives you speed and control, but takes away every safety net — get the field order wrong between `writeExternal` and `readExternal`, and you'll silently read corrupted data with no compiler or JVM warning.

Common Code-Review Interview Pitfalls

1. **Not closing streams / using try-with-resources.** *Why it matters:* leaked file handles or sockets eventually exhaust OS limits, causing mysterious failures under load.

```
// Before
FileInputStream in = new FileInputStream("data.txt");
// ... in.close() forgotten, or skipped if an exception is thrown above
↳ it

// After
try (FileInputStream in = new FileInputStream("data.txt")) {
    // ...
}
```

2. **Using platform-default charset instead of an explicit one.** *Why it matters:* the exact same code produces different bytes/text on different machines or JVM configurations.

```
// Before
Writer w = new FileWriter("out.txt");

// After
Writer w = new FileWriter("out.txt", StandardCharsets.UTF_8);
```

3. **Reading byte-by-byte or line-by-line without buffering.** *Why it matters:* each unbuffered call can trigger a system call — brutal performance hit on large files.

```
// Before
InputStream in = new FileInputStream("big.dat");

// After
InputStream in = new BufferedInputStream(new
↳ FileInputStream("big.dat"));
```

- 4. Trusting `File.delete()` / `File.mkdir()` booleans without understanding they hide the failure reason.** *Why it matters:* silent false gives no diagnostic information; bugs become “it just doesn’t work” reports.

```
// Before
boolean ok = file.delete();

// After
Files.delete(path); // throws NoSuchFileException,
↳ AccessDeniedException, etc.
```

- 5. Not closing `Files.lines()` or `Files.walk()` streams.** *Why it matters:* these hold an open file/directory handle under the hood even though they “look like” a plain in-memory Stream.

```
// Before
Files.lines(path).forEach(System.out::println); // handle never
↳ released

// After
try (Stream<String> lines = Files.lines(path)) {
    lines.forEach(System.out::println);
}
```

- 6. Building file paths from user input without validating against path traversal (`..`/`./`).** *Why it matters:* lets attackers read/write arbitrary files outside the intended directory.

```
// Before
Path p = Path.of("/uploads").resolve(userInput);

// After
Path root = Path.of("/uploads").toRealPath();
Path p = root.resolve(userInput).normalize();
if (!p.startsWith(root)) throw new SecurityException("Traversal
↳ attempt");
```

- 7. Writing directly to a target file instead of write-temp-then-atomic-move.** *Why it matters:* a crash mid-write leaves a corrupted, half-written file that readers may see.

```
// Before
Files.writeString(target, content);
```

```
// After
Path tmp = Files.createTempFile(target.getParent(), "tmp-", ".swap");
Files.writeString(tmp, content);
Files.move(tmp, target, StandardCopyOption.ATOMIC_MOVE,
↳ StandardCopyOption.REPLACE_EXISTING);
```

- 8. Serializable class with no explicit serialVersionUID.** *Why it matters:* any future change to the class can change the auto-computed ID, breaking deserialization of previously-saved data.

```
// Before
class User implements Serializable { String name; }

// After
class User implements Serializable {
    private static final long serialVersionUID = 1L;
    String name;
}
```

- 9. Deserializing untrusted input with plain ObjectInputStream.** *Why it matters:* this is the entry point for deserialization gadget-chain attacks — potential remote code execution.

```
// Before
Object obj = new ObjectInputStream(untrustedInputStream).readObject();

// After: use a JSON/protobuf library instead, or at minimum add an
↳ ObjectInputFilter
ObjectInputStream ois = new ObjectInputStream(untrustedInputStream);
ois.setObjectInputFilter(ObjectInputFilter.Config.createFilter("j
↳ com.example.*;!*"));
Object obj = ois.readObject();
```

- 10. Forgetting sensitive fields need transient.** *Why it matters:* secrets, tokens, or passwords silently end up serialized to disk or over the wire.

```
// Before
class Session implements Serializable { String authToken; }

// After
class Session implements Serializable { transient String authToken; }
```

- 11. Externalizable class missing the required public no-arg constructor.** *Why it matters:* deserialization throws `InvalidClassException` at runtime — the class looks correct until you actually try to deserialize it.

```
// Before
class Product implements Externalizable {
    Product(String name) { this.name = name; } // only constructor

// After
public Product() {} // required
Product(String name) { this.name = name; }
```

- 12. Mismatched field order between `writeExternal` and `readExternal`.** *Why it matters:* `Externalizable` has zero safety net — a swapped read order silently produces corrupted objects with no exception.

```
// Before: writeExternal writes name then price, readExternal reads
↳ price then name
out.writeUTF(name); out.writeDouble(price);
price = in.readDouble(); name = in.readUTF(); // WRONG ORDER

// After: keep read/write order identical, ideally documented
out.writeUTF(name); out.writeDouble(price);
name = in.readUTF(); price = in.readDouble();
```

- 13. Using `ByteBuffer` without calling `flip()` before reading.** *Why it matters:* reading resumes from wherever position was left after writing, producing empty or garbage results instead of the intended data.

```
// Before
buffer.put(data);
buffer.get(result); // reads from wrong position, likely returns
↳ nothing useful

// After
buffer.put(data);
buffer.flip(); // limit = position, position = 0
buffer.get(result);
```

- 14. Using `java.io.File.createTempFile` for sensitive data instead of NIO.2's secure temp file creation.** *Why it matters:* `File.createTempFile` predictable/world-readable permissions on some platforms invite race conditions and information

leaks; `Files.createTempFile` sets owner-only permissions on POSIX systems by default.

```
// Before
File temp = File.createTempFile("upload", ".tmp");

// After
Path temp = Files.createTempFile("upload-", ".tmp");
```

- 15. Choosing `Files.walk` (or recursive File listing) when directory traversal needs error handling or early termination per subtree.** *Why it matters:* a single unreadable subdirectory can blow up an entire `Stream` pipeline with no way to skip just that branch; `walkFileTree` lets you react per-file/per-directory and continue.

```
// Before
Files.walk(root).forEach(this::process); // one bad file aborts
↳ everything

// After
Files.walkFileTree(root, new SimpleFileVisitor<Path>() {
    @Override
    public FileVisitResult visitFileFailed(Path file, IOException exc)
↳ {
        log.warn("Skipping unreadable file: {}", file);
        return FileVisitResult.CONTINUE;
    }
    @Override
    public FileVisitResult visitFile(Path file, BasicFileAttributes
↳ attrs) {
        process(file);
        return FileVisitResult.CONTINUE;
    }
});
```


JVM Fundamentals

Every Java program you review eventually runs on the JVM (Java Virtual Machine), so understanding what happens between `javac` and a running thread is essential for spotting real bugs — not just style issues. This chapter walks through the JDK/JRE/JVM split, how source becomes bytecode and gets loaded and run, how the JIT (Just-In-Time compiler) makes hot code fast, how class loaders isolate and share code, and what the Java Memory Model actually guarantees about threads seeing each other’s writes. All examples target Java 21+ on HotSpot, the default JVM shipped in OpenJDK.

JDK, JRE, JVM Architecture

Three acronyms get thrown around loosely. Here is what each one actually is.

Term	Full name	What it contains
JVM	Java Virtual Machine	The engine that executes bytecode: class loader, runtime memory areas, interpreter, JIT compiler, garbage collector.
JRE	Java Runtime Environment	JVM + core class libraries (<code>java.lang</code> , <code>java.util</code> , ...) needed to <i>run</i> compiled programs. No compiler.
JDK	Java Development Kit	JRE + development tools: <code>javac</code> (compiler), <code>javap</code> (disassembler), <code>jar</code> , <code>jlink</code> , <code>jshell</code> , debuggers, etc. Needed to <i>build</i> programs.

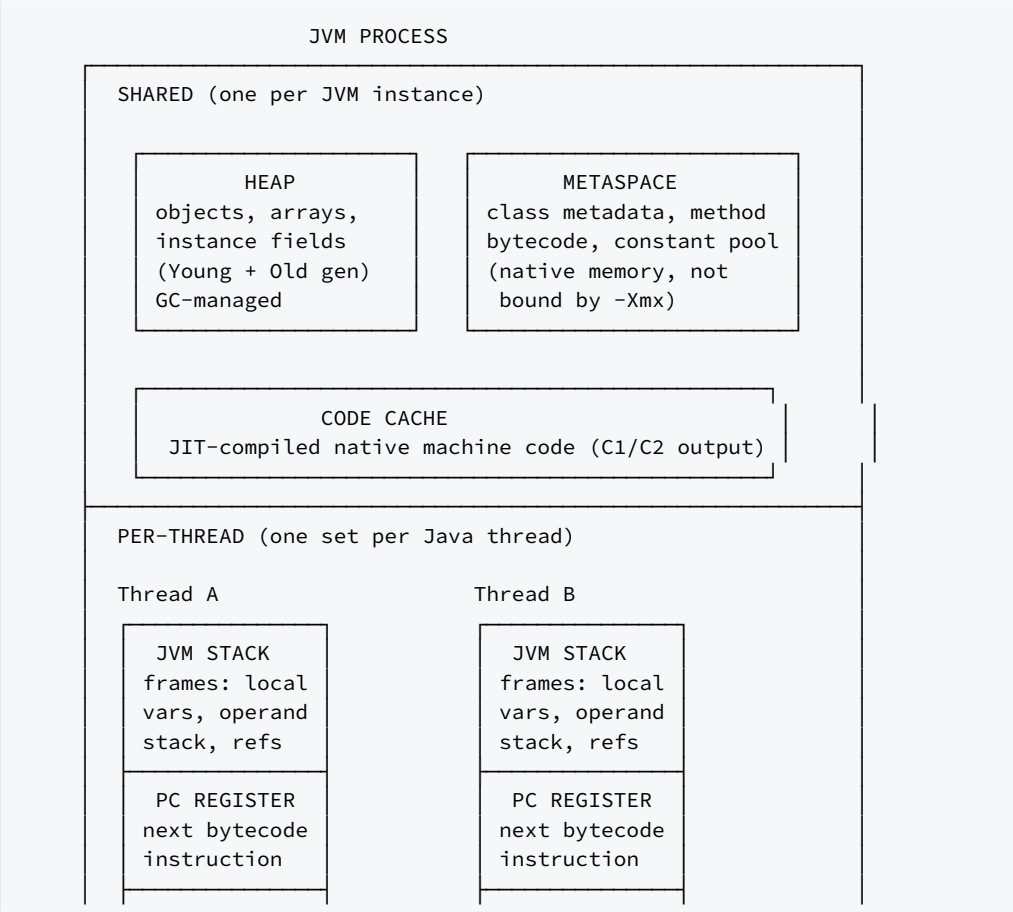
Important nuance for Java 11+: the JRE as a separately downloadable, separately installed artifact was removed starting with Java 11. Oracle and OpenJDK now ship only the JDK. If you need a smaller, runtime-only image (the old JRE’s job), you build one yourself with `jlink`, which assembles a custom minimal runtime image containing only the modules your application actually uses.

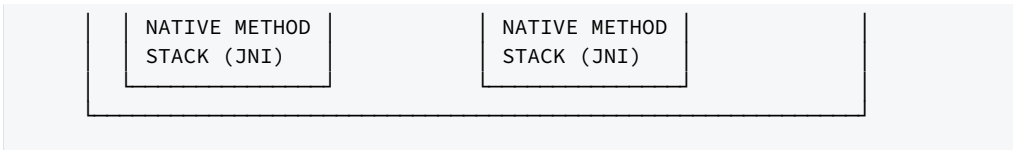
```
# Build a custom "JRE-like" runtime image containing only java.base and
↪ java.sql
jlink --add-modules java.base,java.sql \
      --output my-custom-runtime \
      --strip-debug --no-header-files --no-man-pages

# Run your app with it instead of a full JDK
my-custom-runtime/bin/java -jar app.jar
```

JVM runtime data areas

When the JVM starts, it carves up memory into distinct regions. Some are shared by all threads, some exist one-per-thread. Getting this picture straight is the foundation for understanding OutOfMemoryError variants, thread-safety, and GC behavior later.





Region	Shared or per-thread	Holds	Overflow error
Heap	Shared	All objects and arrays, instance fields	OutOfMemoryError: Java heap space
Metaspace	Shared	Class metadata, method bytecode, constant pools (replaced PermGen since Java 8)	OutOfMemoryError: Metaspace
Code Cache	Shared	Native machine code produced by the JIT	OutOfMemoryError: CodeCache (rare, tune -XX:ReservedCodeCacheSize)
JVM Stack	Per-thread	Stack frames: local variables, operand stack, return addresses	StackOverflowError
PC Register	Per-thread	Address of the currently executing bytecode instruction	n/a
Native Method Stack	Per-thread	State for native (JNI) method calls	StackOverflowError (native)

A quick mental model: **heap is for objects, stacks are for method calls, metaspace is for the classes themselves**. Local primitive variables and object *references* live on the stack; the objects they point to live on the heap.

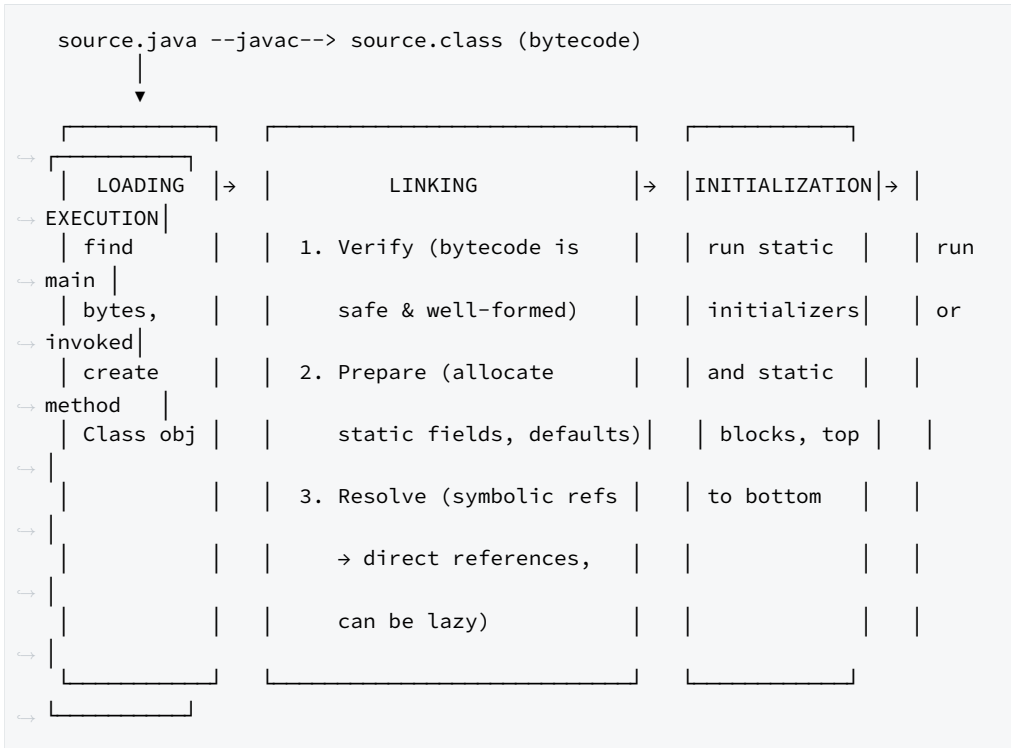
```

void example() {
    int x = 42;           // primitive lives on this thread's stack
    ↪ frame
    StringBuilder sb = new StringBuilder(); // "sb" reference on stack,
                                           // the StringBuilder object
    ↪ itself on the heap
}

```

Compilation, Class Loading, and Execution

Getting from a `.java` file to running code has five distinct stages. Interview reviewers care whether you know *when* things like static initializers actually fire, because that is where subtle bugs (and infinite loops, and `NoClassDefFoundErrors` from failed static blocks) hide.



- 1. Compilation (javac):** Human-readable `.java` source is compiled into platform-independent bytecode stored in a `.class` file.
- 2. Loading:** A class loader locates the bytes (from disk, a JAR, or the network) and creates a `java.lang.Class` object representing the type in the JVM.
- 3. Linking,** which has three sub-steps:
 - **Verify** — the bytecode verifier checks the class file is structurally valid and doesn't violate JVM safety rules (no stack underflows, no illegal casts, etc). This is what stops hand-crafted or corrupted bytecode from crashing the JVM.
 - **Prepare** — static fields are allocated and set to default values (`0`, `null`, `false`) — *not* their initializer expressions yet.

- **Resolve** — symbolic references (like "com/foo/Bar") in the constant pool are resolved to actual memory addresses/direct references. HotSpot resolves most of this lazily, on first use.
4. **Initialization:** Static initializer blocks and static field initializer expressions run, **in textual order, top to bottom**, exactly once per class, the first time the class is “actively used.”
 5. **Execution:** The JVM starts interpreting (and later JIT-compiling) the bytecode of the entry point (main, or whatever invoked the class).

When does initialization actually trigger?

A class is only initialized on first **active use** — not simply on loading. Active use includes:

- Creating an instance (new).
- Calling a static method.
- Accessing/assigning a static field (except a `static final` compile-time constant).
- Reflection (`Class.forName(name)` with default `initialize=true`).
- Initializing a subclass (forces superclass init first).

```
class Parent {
    static { System.out.println("Parent init"); }
}

class Child extends Parent {
    static { System.out.println("Child init"); }
    static int VALUE = 10;
}

public class Demo {
    public static void main(String[] args) {
        System.out.println("before");
        int v = Child.VALUE;    // triggers Parent init, THEN Child init
        System.out.println("after: " + v);
    }
}
```

Output:

```
before
Parent init
Child init
after: 10
```

Notice: merely referencing `Child` in the source doesn't run anything. Initialization is deferred until `Child.VALUE` is actually touched, and superclasses always initialize before subclasses.

```
# See the class-loading and initialization events for a run
java -Xlog:class+init=info Demo
```

Bytecode

Bytecode is the JVM's instruction set — a compact, platform-independent set of opcodes that `javac` emits instead of native machine code. The JVM's execution model is a **stack machine**: most instructions pop operands off an “operand stack” (part of the current stack frame) and push results back, rather than working with named CPU registers.

A small example

```
public class MathOps {
    public int add(int a, int b) {
        return a + b;
    }

    public static int square(int x) {
        return x * x;
    }
}
```

Compile it and disassemble with `javap`:

```
javac MathOps.java
javap -c MathOps.class
```

Output (trimmed to the two methods):


```

public int add(int, int);
    Code:
        0: iload_1          // push local var 1 (a) onto operand stack
        1: iload_2          // push local var 2 (b) onto operand stack
        2: iadd           // pop both, push their sum
        3: ireturn       // pop and return the top of the operand stack

public static int square(int);
    Code:
        0: iload_0          // push local var 0 (x) — note: no "this" slot
    ↪ for static methods
        1: iload_0
        2: imul
        3: ireturn

```

The operand stack model

Each method invocation gets its own **stack frame**, containing:

- **Local variable slots** (local var 0, 1, 2, ...) — parameters and local variables. For instance methods, slot 0 is always `this`; static methods have no such slot, which is why `square`'s `x` is slot 0 while `add`'s `a/b` are slots 1/2 (slot 0 is `this`).
- **The operand stack** — a small scratch stack instructions push to and pop from. `iload_1` pushes; `iadd` pops two ints and pushes their sum; `ireturn` pops the return value.

Common opcodes

Opcode	Meaning
<code>iload_n / aload_n</code>	Push int / object-reference local var <code>n</code> onto operand stack
<code>istore_n / astore_n</code>	Pop top of stack into int / object-reference local var <code>n</code>
<code>iadd, imul, isub</code>	Pop two ints, push arithmetic result
<code>getfield / putfield</code>	Read/write an instance field
<code>getstatic / putstatic</code>	Read/write a static field
<code>invokevirtual</code>	Call an instance method using dynamic dispatch (normal polymorphic call — most common)

Opcode	Meaning
<code>invokestatic</code>	Call a static method (no receiver, resolved at compile time)
<code>invokespecial</code>	Call a constructor, a private method, or a superclass method via <code>super.foo()</code> — non-virtual, resolved statically
<code>invokeinterface</code>	Call a method through an interface reference (needs extra runtime lookup vs <code>invokevirtual</code>)
<code>invokedynamic</code>	Defer the call-site linking logic to a “bootstrap method” resolved at first execution — powers lambdas, method references, and string concatenation

```
# Disassemble with line numbers and the constant pool too
javap -c -p -v MathOps.class
```

The constant pool

Every `.class` file has a **constant pool**: a table of literals, class/method/field names, and symbolic references that the bytecode indexes into instead of embedding raw strings and numbers inline. `javap -v` prints it:

```
Constant pool:
  #1 = Methodref      #6.#15      // java/lang/Object."<init>":()V
  #2 = Class          #16          // MathOps
  #3 = Utf8           add
  ...
```

An instruction like `invokevirtual #7` doesn’t carry a class/method name directly — it carries an index into this table, which is resolved to an actual method address during linking (the “Resolve” step described earlier).

`invokedynamic`: lambdas and string concatenation

Before Java 8, every method call site was one of the four `invoke*` opcodes above, each resolved to a fixed target. `invokedynamic` (added in Java 7, exploited heavily from Java 8 onward) instead calls a **bootstrap method** the first time a call site executes; that bootstrap method decides — and caches — how the call should actually be linked. This lets `javac` avoid generating a synthetic inner class for every single lambda.

```
Runnable r = () -> System.out.println("hi");
```

```
javap -c -p Demo.class
```

```
0: invokedynamic #7, 0    // InvokeDynamic
↳ #0:run():Ljava/lang/Runnable;
5: astore_1
```

The actual lambda body is compiled into a private synthetic method, and `invokedynamic` + `LambdaMetafactory` wire up a `Runnable` instance pointing at it lazily, at first call.

Since Java 9, string concatenation (`"a" + b + "c"`) also compiles to `invokedynamic` calling `StringConcatFactory`, instead of the old pattern of allocating a `StringBuilder` and chaining `.append()` calls:

```
String s = "Count: " + count;
```

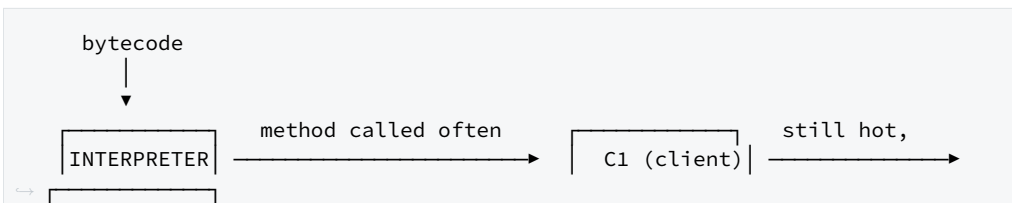
```
invokedynamic #2, 0    // InvokeDynamic
↳ #0:makeConcatWithConstants:(I)Ljava/lang/String;
```

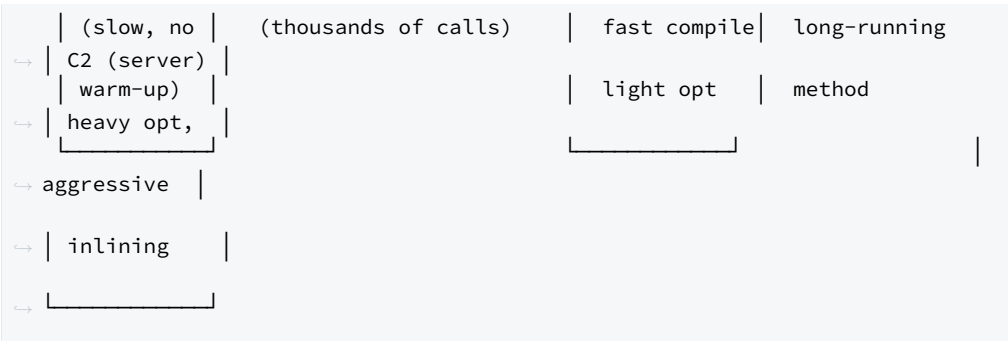
```
# Try it yourself
echo 'public class Cat { String go(int n){ return "n=" + n; } }' >
↳ Cat.java
javac Cat.java && javap -c -p Cat.java
```

JIT Compilation

Bytecode is portable but interpreting it instruction-by-instruction is slow. HotSpot (the default JVM) starts by **interpreting** bytecode, then compiles the “hot” (frequently executed) parts to native machine code on the fly — hence *Just-In-Time*.

Tiered compilation: interpreter → C1 → C2





- **Interpreter** — executes bytecode directly, one instruction at a time. Starts instantly, no compile delay, but slow per-iteration.
- **C1 (“client compiler”)** — compiles quickly with light optimization. Good for short-lived or moderately-hot code; gets you off the interpreter fast.
- **C2 (“server compiler”)** — compiles slowly but produces highly optimized native code: aggressive inlining, loop unrolling, escape analysis, etc. Reserved for genuinely hot methods.

Modern HotSpot uses **tiered compilation** by default: code moves interpreter → C1 (with several sub-tiers of profiling) → C2, escalating only if a method really is called a lot.

Hot spots and OSR

The JVM counts method invocations and *loop back-edges* (loop iterations). Once a counter crosses a threshold, the method is queued for JIT compilation. If a **single long-running loop** inside a method gets hot before the method itself is recompiled, HotSpot can do **On-Stack Replacement (OSR)**: it JIT-compiles just that loop and swaps the interpreter’s execution *in place*, mid-method, onto the compiled version — without waiting for the method to be called again.

```

long sum(int n) {
    long s = 0;
    for (int i = 0; i < n; i++) {    // if n is huge, this loop alone can
↔ trigger OSR
        s += i;
    }
    return s;
}

```

Inlining and deoptimization

Inlining replaces a call site with the callee’s body, eliminating call overhead and opening the door to further optimization (e.g., dead-code elimination once constants propagate through). C2 inlines aggressively, including through virtual calls when it can prove (via profiling) there’s effectively only one implementation in practice.

Deoptimization is the safety valve: if an assumption the JIT baked in turns out false at runtime (e.g., a second implementation of an interface finally shows up, invalidating a “monomorphic” inlining bet), the JVM discards the compiled code for that method, falls back to the interpreter, and re-profiles. This is invisible in correctness terms — Java’s semantics never change — but it does explain sudden latency blips in production right after a “cold” code path executes for the first time.

```
interface Shape { double area(); }
class Circle implements Shape { public double area() { return 3.14; } }
class Square implements Shape { public double area() { return 4.0; } }

// If callers only ever pass Circle for a long time, C2 may speculate
// "this call site is monomorphic" and inline Circle.area() directly.
// The moment a Square shows up, that assumption breaks -> deoptimization.
double totalArea(List<Shape> shapes) {
    double total = 0;
    for (Shape s : shapes) total += s.area();
    return total;
}
```

Why microbenchmarks need JMH and warm-up

A naive “time it in a loop with `System.nanoTime()`” benchmark is almost always wrong, because:

- The **interpreter** runs the first iterations, which are much slower than the eventual steady state.
- **JIT compilation happens on background threads mid-benchmark**, so timings from early iterations mix cold and hot code.
- **Dead-code elimination** can let the JIT discard a whole “benchmarked” computation if its result is never used.

JMH (Java Microbenchmark Harness) solves this by running explicit **warm-up iterations** (to let JIT compilation stabilize) before measuring, and by using tricks (blackholes,

forcing consumption of results) to prevent dead-code elimination from making your “fast” code fast simply because it does nothing.

```
@BenchmarkMode(Mode.AverageTime)
@OutputTimeUnit(TimeUnit.NANOSECONDS)
@Warmup(iterations = 5, time = 1)
@Measurement(iterations = 5, time = 1)
@Fork(1)
public class StringBenchmark {
    @Benchmark
    public String concat() {
        return "a" + "b" + "c";
    }
}
```

Observing the JIT

```
# Print every JIT compilation event as it happens (method, tier, time)
java -XX:+PrintCompilation MyApp

# Typical lines:
#   1      3      3      java.lang.String::hashCode (60 bytes)
#  100    45      4      MyApp::hotLoop (120 bytes)   made not entrant
↪ <- deopt!
```

made not entrant in the output is the visible sign of a deoptimization — that compiled version was thrown away.

AOT and CDS in one paragraph

The JIT’s warm-up cost is a real problem for short-lived processes (CLIs, serverless functions, fast-restarting microservices). Two HotSpot features help: **CDS (Class Data Sharing)**, which pre-parses and serializes class metadata into a shared archive (`-Xshare:dump` / `-XX:SharedArchiveFile`) so subsequent JVM startups skip re-parsing those classes, and its evolution **AppCDS / dynamic CDS archives**, which extend this to application classes, not just JDK classes. Separately, **Project Leyden** and tools like GraalVM Native Image push further into full **AOT (Ahead-Of-Time) compilation**, compiling to native machine code before the process even starts, trading some peak-throughput JIT optimizations for near-instant startup — a trade-off worth naming in an interview when startup latency comes up.

```
# Create and use an AppCDS archive
java -Xshare:off -XX:ArchiveClassesAtExit=app.jsa -jar app.jar
java -XX:SharedArchiveFile=app.jsa -jar app.jar
```

Class Loaders

A **class loader** is the object responsible for turning a class name into a `Class` object — locating the bytecode and defining it into the JVM. HotSpot has a built-in hierarchy of three:

```
Bootstrap ClassLoader    (native, written in C++, no Java superclass)
    loads: java.base and other core JDK modules (java.lang.*,
↳ java.util.*)
    |
    ▼ parent
Platform ClassLoader      (was "Extension" loader before Java 9)
    loads: other JDK-provided modules not in java.base
    |
    ▼ parent
Application ClassLoader   (a.k.a. "system" class loader)
    loads: your application's classes from the classpath / module path
```

Parent delegation

By default, when a class loader is asked to load a class, it **first asks its parent** to try, and only attempts to load it itself if the parent fails (`ClassNotFoundException`). This means core JDK classes like `java.lang.String` are always loaded by the bootstrap loader — even if you name your own class `java.lang.String`, the delegation model prevents your version from shadowing the real one (and the JVM outright rejects defining classes in the `java.*` package from a non-bootstrap loader).

```
public class WhichLoader {
    public static void main(String[] args) {
        System.out.println(String.class.getClassLoader());           //
↳ null (bootstrap)
        System.out.println(WhichLoader.class.getClassLoader());     //
↳ AppClassLoader

↳ System.out.println(WhichLoader.class.getClassLoader().getParent()); //
↳ PlatformClassLoader
    }
```

```
}

```

Custom class loaders

Frameworks that need to load code dynamically — plugin systems, application servers deploying multiple WARs, dependency-shading/isolation tools — write custom class loaders, usually by extending `ClassLoader` or `URLClassLoader`.

```
public class PluginClassLoader extends URLClassLoader {
    public PluginClassLoader(URL[] pluginJars, ClassLoader parent) {
        super(pluginJars, parent);
    }
    // Override findClass()/loadClass() to change lookup order,
    // e.g. "child-first" delegation for plugin isolation (as Tomcat's
    // webapp classloader does, breaking strict parent-first order
    // deliberately, on purpose, for isolation).
}
```

```
# Load and run a class from an arbitrary jar at runtime, illustrating
↳ dynamic loading
java -cp plugin.jar
↳ -Djava.system.class.loader=com.example.PluginClassLoader Main
```

ClassNotFoundException vs NoClassDefFoundError

These are commonly confused in code review and interviews:

	ClassNotFoundException	NoClassDefFoundError
Type	Checked Exception	Error (subclass of <code>LinkageError</code>)
When	You explicitly ask to load a class by name — <code>Class.forName(...)</code> , or a custom class loader's <code>loadClass()</code> — and it isn't found.	A class was available and successfully compiled against , but is missing at runtime , OR the class was found once but its static initializer threw , poisoning it for all future references.
Typical cause	Wrong string in <code>Class.forName</code> , missing JDBC driver, typo'd reflection lookup.	Jar removed from classpath after compiling against it; a previous load attempt failed (e.g. static init threw an exception) so the JVM marks the class permanently unusable.


```

try {
    Class.forName("com.example.MissingDriver"); // ->
    ↪ ClassNotFoundException if absent
} catch (ClassNotFoundException e) {
    // handle: driver jar not on classpath
}

```

```

class Broken {
    static int x = 1 / 0;    // ArithmeticException during static init
}

class UsesBroken {
    void run() {
        new Broken(); // first call: ExceptionInInitializerError
        new Broken(); // second call in this or another method:
        ↪ NoClassDefFoundError
                        // (class is now permanently "erroneous")
    }
}

```

Class identity = (name, loader)

The JVM treats two classes as **the same type only if both the fully-qualified name AND the defining class loader match**. Load the “same” .class bytes through two different loaders and you get two distinct, mutually incompatible Class objects — a frequent source of baffling ClassCastException with messages like com.example.Foo cannot be cast to com.example.Foo.

```

Class<?> a = loader1.loadClass("com.example.Foo");
Class<?> b = loader2.loadClass("com.example.Foo");
System.out.println(a == b);           // false!
System.out.println(a.equals(b));      // false!
// Casting an instance loaded by loader1 to the type loaded by loader2
↪ throws
// ClassCastException even though the source code is byte-for-byte
↪ identical.

```

ClassLoader leaks in application servers

Servlet containers like Tomcat give **each deployed webapp its own class loader**, so redeploying a WAR without restarting the whole server means loading a fresh set of

classes. If anything outside the webapp — a thread the app started, a static field in a JDK class, a `ThreadLocal`, JDBC driver registries — keeps a reference to a class (or an instance of a class) from the old webapp's loader, **the entire old class loader, and every class and object it ever loaded, cannot be garbage collected**. Symptoms: Metaspace usage climbing on every redeploy, eventually `OutOfMemoryError: Metaspace`. Common causes and fixes:

```
// LEAK: a thread started by the webapp keeps running after undeploy,
// its Thread object (loaded by the webapp's classloader) stays reachable
// from the JVM's global thread list -> webapp classloader can never be
↳ freed.
Thread worker = new Thread(this::pollForever);
worker.setDaemon(true);
worker.start();
// Fix: register a ServletContextListener.contextDestroyed() hook that
// signals the thread to stop and joins it before the webapp is unloaded.
```

JPMS impact (the module system)

Since Java 9, the **Java Platform Module System (JPMS)** adds a layer on top of class-loaders: modules declare explicit dependencies (requires) and exposed packages (exports) in a `module-info.java`. This affects class loading in two practical ways reviewers should know: (1) **strong encapsulation** — internal packages that aren't exported are inaccessible via reflection from outside the module by default, even if they were public, breaking some old reflection-heavy frameworks; (2) the platform's own bootstrap/platform/application loaders were re-organized around modules (e.g. `java.base` is the module the bootstrap loader owns), and each named module effectively gets a layer that still resolves through the same parent-delegation-style rules for unnamed/automatic modules on the classpath.

```
// module-info.java
module com.example.app {
    requires java.sql;
    exports com.example.app.api;    // only this package is visible to
↳ other modules
    // com.example.app.internal is NOT exported -> inaccessible outside
↳ the module,
    // even via setAccessible(true) reflection, unless "opens" is
↳ declared.
}
```

Java Memory Model (JMM)

The Java Memory Model is the specification (JLS Chapter 17) that answers a deceptively hard question: **when a thread writes a value, when is another thread guaranteed to see it?** Without a memory model, compilers and CPUs are free to reorder, cache, and optimize your code in ways that are safe for a single thread but disastrous once multiple threads are involved.

Reordering by compiler and CPU

Both the JIT compiler and the CPU are allowed to reorder instructions and cache values in registers, as long as a **single thread** can't observe the difference. But another thread, watching from the outside, absolutely can.

```
class Reorder {
    int a = 0;
    boolean flag = false;

    void writer() {
        a = 42;           // (1)
        flag = true;      // (2) - compiler/CPU may reorder (1) and (2)!
    }

    void reader() {
        if (flag) {
            System.out.println(a); // could print 0, not 42, without
    ↪ synchronization
        }
    }
}
```

Each CPU core also has its own **cache**, separate from main memory. A write by one thread might sit in that core's cache/store-buffer without being flushed to main memory (or the shared cache level other cores see) for an unbounded amount of time, from the other thread's perspective, unless something forces synchronization.

Happens-before

The JMM defines correctness in terms of a **happens-before** relationship: if action X happens-before action Y, every effect of X (every write) is guaranteed visible to Y. Without an established happens-before edge between two threads' actions, there is **no guarantee** the second thread ever sees the first thread's writes — not “might be slow,” but genuinely unspecified/broken behavior.

Key sources of happens-before edges:

- A synchronized block's unlock happens-before a later thread's lock on the *same monitor*.
- A `volatile` field write happens-before every subsequent `volatile` read of that same field.
- `Thread.start()` happens-before anything the started thread does.
- Everything a thread does happens-before another thread observes it has finished, via `Thread.join()`.
- Writes to `final` fields performed in a constructor happen-before any thread that gets a correctly-published reference to that object.

volatile

`volatile` gives you **visibility** (every read sees the latest write, no stale caching) and prevents certain compiler/CPU reorderings around that field, but it does **not** give you atomicity or mutual exclusion for compound operations.

```
class FlagExample {
    private volatile boolean running = true;

    void stop() { running = false; }           // thread A

    void loop() {
        while (running) {                     // thread B
            // do work
        }
    }
}
```

Without `volatile` here, the JIT is legally allowed to hoist `running` into a register in `loop()` (since single-threaded reasoning says it never changes inside the loop body),

producing an **infinite loop that never sees stop()'s write** — this is the concrete broken-visibility bug interviewers love to ask about.

```
// BROKEN: no volatile, no synchronization -> may spin forever on some
↳ JVMs/JITs
class BrokenFlag {
    private boolean running = true;
    void stop() { running = false; }
    void loop() { while (running) { /* spin */ } }
}
```

volatile does **not** make increments atomic:

```
private volatile int counter = 0;
void increment() { counter++; } // STILL a race: read, add, write are 3
↳ separate steps
// Fix: use AtomicInteger, or synchronize the whole read-modify-write.
```

final field freeze semantics

If an object is constructed with a `final` field, and the reference to that object is not leaked out of the constructor before it finishes, then **any thread that later obtains a reference to that object is guaranteed to see the fully-initialized value of the final field** — without needing any additional synchronization. This is the JMM’s “safe construction via final fields” guarantee.

```
final class Point {
    final int x, y;
    Point(int x, int y) { this.x = x; this.y = y; }
}
// Any thread that receives a Point reference through a properly published
// channel (see below) will see correct, fully-initialized x and y –
// guaranteed by final field "freeze" semantics, even with zero
↳ synchronization
// on the reading side.
```

Safe publication

“Safe publication” means making an object visible to other threads in a way the JMM actually guarantees is safe — not just “it usually works.” Common safe publication mechanisms:

- Assigning the reference to a `volatile` or `static final` field.
- Storing it into a properly locked field (synchronized).
- Putting it into a thread-safe collection like `ConcurrentHashMap` or `BlockingQueue`, whose own internal synchronization carries the happens-before edge.
- Initializing it as a `static` field during class initialization (class-init has its own locking guarantees).

```
// UNSAFE publication: another thread might see a half-constructed object
public class Holder {
    public static Helper helper;
    public static void init() {
        helper = new Helper(); // another thread reading `helper` non-null
                                // isn't guaranteed to see a fully built
↪ Helper                                // unless Helper's fields are all final,
↪ or                                    // helper is volatile/static-final.
    }
}
```

```
// SAFE publication via volatile
public class Holder {
    private static volatile Helper helper;
    public static Helper getHelper() {
        if (helper == null) {
            synchronized (Holder.class) {
                if (helper == null) helper = new Helper();
            }
        }
        return helper;
    }
}
```

Data races vs race conditions

These terms are related but not identical, and mixing them up in review comments is a common tell:

	Data race	Race condition
Definition	Two threads access the same memory location, at least one write, with no happens-before ordering between them.	Program's correctness depends on timing/interleaving of operations, regardless of memory visibility.
Scope	A specific JMM-defined term about memory access without synchronization.	A general concurrency term about outcome depending on execution order.
Example	Non-volatile boolean running read/written by two threads.	Two threads both doing “check-then-act” on a bank balance with a lock each holds separately — well-synchronized memory-wise but still logically racy (TOCTOU).

A program can have a race condition without a data race (properly synchronized but logically wrong ordering), and a data race almost always implies undefined/unreliable behavior even if it “seems to work” in testing.

synchronized: two jobs, not one

`synchronized` is often described only as “locking,” but it actually does **two separate jobs** at once, and code reviewers should check both are needed (or that neither is skip-pable):

1. **Mutual exclusion** — only one thread can hold a given monitor at a time, preventing concurrent execution of the guarded block.
2. **Visibility** — entering a `synchronized` block establishes a happens-before edge with the *previous* thread that held (and released) the same lock, guaranteeing you see all of its writes made inside that lock, not just the fact that it finished.

```
class Counter {
    private int count = 0;
    private final Object lock = new Object();

    void increment() {
        synchronized (lock) {
            count++;
        }
    }

    int get() {
```

```

        synchronized (lock) {           // visibility: guaranteed to see latest
↪ count,
            return count;               // even without this being volatile
        }
    }
}

```

A frequent review mistake: locking on `increment()` but reading `count` directly without synchronization elsewhere in `get()`. That reintroduces the visibility problem even though updates themselves are safely serialized — the lock’s visibility guarantee only applies to threads that *also* go through the same lock.

```

# See lock contention / thread states live, useful when reviewing
↪ concurrency claims
jcmd <pid> Thread.print | grep -A3 "waiting to lock"

```

Common Code-Review Interview Pitfalls

1. **Assuming the JRE still exists as a separate download.** Since Java 11, there is no standalone JRE artifact; reviewers sometimes still ask candidates to “install the JRE” — the correct modern answer is a full JDK, or a `jlink`-built custom runtime image.

```
jlink --add-modules java.base --output tiny-runtime
```

2. **Confusing heap `OutOfMemoryError` with metaspace/native exhaustion.** `-Xmx` only bounds the heap; unbounded class generation (common with proxy-heavy frameworks, or dynamically generated classes never unloaded) exhausts Metaspace instead, which needs `-XX:MaxMetaspaceSize` to even surface as a clean error instead of eating all system memory.

```
java -Xmx512m -XX:MaxMetaspaceSize=256m -jar app.jar
```

3. **Believing static blocks run at class-load time.** They run at class **initialization**, which is deferred to first active use — code that assumes a static block has “already run” just because a class was referenced (e.g. in a type declaration or import) is wrong.

```
static { System.out.println("I might run much later than you think"); }
```


- 4. Treating `invokevirtual` and `invokestatic` as interchangeable in reasoning about polymorphism.** A private or static “helper” method resolved via `invokestatic`/`invokespecial` cannot be overridden — reviewers should flag any comment claiming a private method “overrides” a superclass method; it just shadows/hides it.

```
class Base { private void helper() {} } // invokespecial, not
↳ virtual dispatch
class Sub extends Base { private void helper() {} } // unrelated method,
↳ NOT an override
```

- 5. Expecting `+` string concatenation to always allocate a `StringBuilder`.** Since Java 9, `javac` lowers concatenation to `invokedynamic` `StringConcatFactory`; reviewers shouldn’t ding code for “missing an explicit `StringBuilder`” in simple concatenation — the compiler already optimizes it, though a raw loop with `+=` inside still has quadratic-cost risk and *should* use an explicit `StringBuilder`.

```
String s = "";
for (String part : parts) s += part; // still O(n^2) – use
↳ StringBuilder here
```

- 6. Trusting raw loop-based microbenchmarks in a PR’s “performance proof.”** Without JIT warm-up and dead-code-elimination guards, a `System.nanoTime()` loop can report numbers dominated by interpreter overhead or optimized-away code. Ask for JMH results instead.

```
@Warmup(iterations = 5) @Measurement(iterations = 5)
public class MyBenchmark { @Benchmark public int compute() { return
↳ heavy(); } }
```

- 7. Assuming a hot method is always running JIT-compiled code.** A method can look identical in two profiling runs yet perform very differently if one run deoptimized it (e.g. a newly-loaded subclass broke a monomorphic call-site assumption). Look for `made not entrant in -XX:+PrintCompilation` output before blaming “the algorithm.”
- 8. Mixing up `ClassNotFoundException` and `NoClassDefFoundError` in exception-handling review.** Catching `ClassNotFoundException` around `Class.forName` won’t help if the real problem is a static initializer that already threw once — that manifests later as `NoClassDefFoundError`, which is an `Error`, not caught by typical `Exception` handlers.

```
try { Class.forName("x.Y"); }
catch (ClassNotFoundException e) { /* won't catch a poisoned class! */
↳ }
```

- 9. Comparing classes loaded by different class loaders with == or instanceof and being surprised by failures.** In app-server or plugin architectures, “the same” class loaded twice by different loaders are different types to the JVM. A `ClassCastException` with identical-looking type names on both sides is the signature symptom — check for multiple loaders before assuming a bug in the cast logic itself.
- 10. Overlooking classloader leaks from long-lived threads or `ThreadLocals` in app-server deployments.** A background thread or `ThreadLocal` value that outlives a hot-redeploy pins the entire old webapp class loader in Metaspace. Reviewers should ask: “who stops this thread / clears this `ThreadLocal` on shutdown?”

```
contextDestroyed(ServletContextEvent e) { worker.interrupt();
↳ threadLocal.remove(); }
```

- 11. Reading or writing a shared flag/field across threads with no `volatile`, lock, or other happens-before edge.** This is the textbook broken-visibility bug — it can compile, pass single-threaded tests, and still spin forever or read stale values in production, because nothing in the JMM obligates the writer’s update to ever become visible to the reader.

```
boolean running = true;           // missing volatile
void loop() { while (running) {} } // may never see stop()'s write
```

- 12. Believing `volatile` makes compound operations atomic.** `volatile int counter; counter++;` is still a three-step read-modify-write race between threads — visibility is guaranteed, atomicity is not. Look for `AtomicInteger`/`AtomicLong` or a lock instead.
- 13. Publishing a partially-constructed object reference without a `final`/`volatile`/lock-based safe-publication path.** The classic double-checked-locking-without-`volatile` singleton bug: another thread can see a non-null but not-yet-fully-initialized instance.

```
private static Helper instance; // missing volatile – double-checked
↳ locking is broken here
```

14. **Conflating “synchronized gives mutual exclusion” with “synchronized on this block automatically makes everything else visible too.”** Only accesses that go through the *same* lock get the happens-before guarantee. A synchronized setter paired with an unsynchronized getter on the same field is a subtle, easy-to-miss review flag — it looks safe but isn't.
15. **Calling something a “data race” when it’s really a logic-level “race condition” (or vice-versa) in review comments.** A well-synchronized (no data race) check-then-act sequence can still be logically racy (TOCTOU) if the check and act aren't combined atomically — that's a race condition, not a JMM data race, and the fix (atomic compound operations / higher-level locking) is different from “just add `volatile`.”

```
if (!map.containsKey(k)) map.put(k, v); // race condition even if map
↳ is a ConcurrentHashMap
// Fix: map.putIfAbsent(k, v);
```


Memory Management

Memory management is where “the code compiles” and “the code survives production” part ways. A code reviewer who understands garbage collection, common leak patterns, and the diagnostic tools can catch problems that unit tests never will — a slowly growing heap, a rogue `ThreadLocal`, a GC pause that violates an SLA. This chapter covers how the JVM’s garbage collectors work, how to spot and debug memory leaks, what escape analysis actually does (and doesn’t) for you, and the real commands you’ll reach for when production memory goes wrong. Examples target Java 21+ HotSpot, since that is the current LTS baseline for interviews and production.

Garbage Collection

Garbage collection (GC) is the JVM automatically finding objects nobody can reach anymore and freeing their memory. You never call `free()` like in C — the collector does it for you. That’s a huge productivity win, but it means understanding *how* it decides what’s garbage is part of writing (and reviewing) correct, fast Java.

Reachability and GC roots. An object is “alive” if it’s *reachable* — if you can follow a chain of references to it starting from a **GC root**. GC roots are the starting points the collector trusts by definition: local variables and parameters on any thread’s stack, active static fields, JNI references, and a few JVM-internal references (like classes loaded by the bootstrap classloader). If an object can’t be reached by following references from any GC root, it’s garbage — even if it still has a valid memory address and even if two garbage objects point at each other (a reference cycle does **not** keep objects alive in Java, unlike naive reference counting).

```
class Node {  
    Node next;  
}
```

```

Node a = new Node();           // 'a' is a GC root (local variable) -> object A
↪ is reachable
Node b = new Node();
a.next = b;                     // B is reachable through A
a = null;                       // A is no longer reachable from any root...
// ...so A AND B are both garbage now, even though B still exists in
↪ memory,
// because nothing reachable from a GC root points to A anymore.

```

Mark-sweep-compact. This is the classic three-phase algorithm most collectors build on:

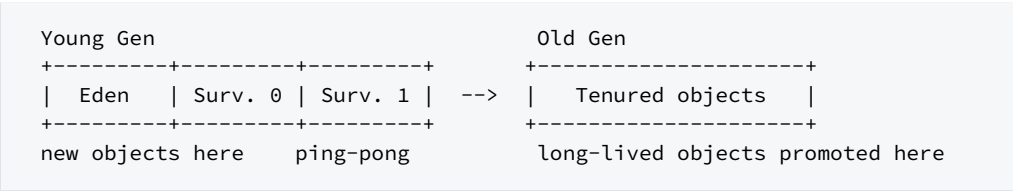
1. **Mark** — walk from GC roots and flag every reachable object.
2. **Sweep** — reclaim the memory used by everything unmarked.
3. **Compact** — slide the surviving objects together to remove gaps, so the heap has one large free region instead of fragmented holes.

Compaction matters because fragmented free memory can make a large allocation fail even though the *total* free space is enough — like having plenty of parking but no single spot big enough for a bus.

Copying collectors. Instead of sweeping in place, a copying collector moves all live objects from one memory region (“from-space”) to another (“to-space”) and then treats the entire from-space as free. This is very fast when most objects are garbage (which is typical for young objects) because it only touches *live* objects, not dead ones — cost is proportional to what survives, not to heap size.

The generational hypothesis. Empirically, most objects die young — a request-scoped object, a loop temporary, a `String` built for logging. Few objects live a long time (caches, singletons, connection pools). Collectors exploit this by splitting the heap into generations:

- **Young generation** — where new objects are born. Collected frequently and cheaply with a copying collector. A young-gen collection is called a **minor GC**.
- **Old (tenured) generation** — where objects that survive several young collections get promoted (“tenured”). Collected less often but more expensively, since it’s usually larger and mark-sweep-compact style. Collecting the old generation (or the whole heap) is a **major GC**; collecting the *entire* heap including metaspace is often called a **full GC**. Full GCs are the ones most likely to cause a noticeable pause.



New objects are allocated in **Eden**. When Eden fills up, a minor GC copies survivors into a **survivor space**, ping-ponging between two survivor spaces on each collection, and objects that survive enough cycles get promoted to the old generation.

TLABs (Thread-Local Allocation Buffers). To avoid every thread contending on a lock just to bump a heap pointer, each thread gets its own small private chunk of Eden (a TLAB) to allocate into without synchronization. Allocation becomes “bump a pointer in my own buffer” — extremely fast — until the TLAB is full, at which point the thread grabs a new one. This is why object allocation in Java is often faster than a naive C `malloc`.

Stop-the-world (STW) pauses. Most collectors need to pause all application threads at some point — for example, to safely walk thread stacks for GC roots without them changing underneath the collector. This pause is called **stop-the-world**. The whole story of modern collector design (G1, ZGC, Shenandoah) is about shrinking STW pauses from “proportional to heap size” down to a few milliseconds, regardless of how big the heap is.

Minor, major, and full GC — a quick disambiguation. Interviewers like this because the terms are used loosely in the wild:

Term	What it collects	Typical cost	Triggered by
Minor GC	Young generation only (Eden + survivor spaces)	Cheap, frequent	Eden fills up
Major GC	Old generation (definitions vary slightly by collector)	Expensive, less frequent	Old gen fills up / promotion pressure
Full GC	Entire heap, and usually metaspace too	Most expensive, rarest	Old gen exhausted, explicit <code>System.gc()</code> , metaspace pressure

In G1, the line between “major” and “full” is blurrier than in Parallel GC, because G1 does most old-gen work incrementally and concurrently — a true G1 full GC (falling back to single-threaded mark-sweep-compact of the whole heap) is meant to be a rare emergency fallback, and seeing one repeatedly in a log is itself a red flag that the collector is under more pressure than it’s designed to handle.

Allocation rate and promotion rate. Two numbers worth knowing by name because tools report them directly: **allocation rate** is how many bytes per second the application creates (mostly in Eden); **promotion rate** is how many bytes per second survive long enough to be pushed into the old generation. A high allocation rate with a *low* promotion rate is healthy (objects are dying young, exactly as the generational hypothesis predicts). A high promotion rate is a warning sign — it means minor GCs are working harder to fill up the old generation faster, dragging major/full GCs closer together.

Weak, soft, and phantom references. Normal (“strong”) references keep an object alive. Java also has three weaker kinds, all in `java.lang.ref`:

Reference type	Collected when	Typical use
Strong (normal)	Never, while reachable	Everyday references
Soft (<code>SoftReference</code>)	GC <i>may</i> clear it, typically only under memory pressure	Memory-sensitive caches
Weak (<code>WeakReference</code>)	GC clears it as soon as nothing else strongly reaches it	<code>WeakHashMap</code> keys, avoiding leaks in listener maps
Phantom (<code>PhantomReference</code>)	After the object is finalized/unreachable, but before memory is reclaimed; <code>get()</code> always returns null	Precise cleanup via <code>Cleaner</code>

```
Map<Key, Value> cache = new java.util.WeakHashMap<>();
// entries disappear automatically once 'Key' has no other strong
↪ references —
// useful for caches keyed by objects you don't own the lifecycle of
```

Why `System.gc()` is wrong. Calling `System.gc()` only *suggests* a full GC to the JVM — it does not force one, and on some collectors it’s an expensive full GC that pauses the whole application. Application code should never call it: it’s the collector’s job to decide when collection is worthwhile, and it usually knows better than a hardcoded call buried in business logic. (A code reviewer should flag any `System.gc()` call outside of a benchmark harness or manual profiling session.)

Why `finalize()` is wrong. `Object.finalize()` was Java’s original hook to run cleanup code before an object is reclaimed. It’s a bad tool: it runs on an unpredictable finalizer thread at an unpredictable time (or never, if the JVM exits first), it can resurrect objects, it slows GC down, and a throwing finalizer silently swallows the exception. **`finalize()` has been deprecated for removal since Java 9** — don’t use it in new code.


```

// Wrong – deprecated for removal, unpredictable timing
@Override
protected void finalize() {
    connection.close();
}

// Right – deterministic, explicit
try (var connection = openConnection()) {
    // use connection
} // .close() called here, guaranteed

// If you truly need a GC-triggered safety net (not the primary cleanup
→ path):
import java.lang.ref.Cleaner;

class ResourceHolder implements AutoCloseable {
    private static final Cleaner CLEANER = Cleaner.create();
    private final Cleaner.Cleanable cleanable;
    private final State state;

    private static class State implements Runnable {
        Connection connection;
        @Override public void run() { connection.close(); } // no
→ reference back to ResourceHolder!
    }

    ResourceHolder(Connection c) {
        state = new State();
        state.connection = c;
        cleanable = CLEANER.register(this, state);
    }

    @Override public void close() { cleanable.clean(); }
}

```

Cleaner is the modern replacement — it still shouldn't be your *primary* cleanup strategy (try-with-resources / explicit `close()` should be), but it's a safe backstop, unlike `finalize()`, because its cleanup action must not hold a reference back to the object being cleaned, avoiding the resurrection problem.

Serial GC

How it works. The simplest collector. One single thread does all the work — marking, sweeping, compacting — while every application thread is stopped. No parallelism, no concurrency.

Pause characteristics. Pauses are proportional to heap size and can be long, but there's zero coordination overhead between GC threads, so for a *small* heap the pause is small too.

Heap sweet spot. Small heaps — typically under a few hundred MB. Think embedded devices, small CLI tools, containers with 1-2 CPUs and constrained memory.

Flag to enable: `-XX:+UseSerialGC`

When to pick it. Single-core or memory-constrained environments where the simplicity and low overhead outweigh the fact that pauses aren't parallelized. Rarely the right choice for a server with multiple cores and a heap above a few hundred MB.

Example log line — notice the pause scales with the *whole* young generation size, with no “(Normal)”/“(Concurrent Start)” qualifiers because there's only one kind of pause:

```
[gc] GC(5) Pause Young (Allocation Failure) 96M->12M(128M) 45.112ms
```

Parallel GC

How it works. Same generational, mark-sweep-compact/copying design as Serial GC, but both the young and old generation collections use **multiple threads** to do the marking, copying, and compacting work. The application is still fully stopped during a collection (it's still stop-the-world) — the parallelism just makes that pause shorter by spreading the work across cores.

Pause characteristics. Optimizes for **throughput** (maximum total work done by the application over time), not for pause length. Pauses can still be noticeable, especially full GCs on a large old generation, but total CPU spent on GC (versus useful work) is low.

Heap sweet spot. Medium to large heaps on multi-core machines, for batch jobs and throughput-sensitive workloads that don't have a strict pause-time SLA.

Flag to enable: `-XX:+UseParallelGC`

When to pick it. Batch processing, offline data pipelines, scientific computing — anything where total job completion time matters more than any individual pause, and there's no interactive user waiting on a response.

Example log line — same shape as Serial's, but the wall-clock duration is shorter for a similarly sized young generation because multiple GC threads did the copying in parallel:

```
[gc] GC(5) Pause Young (Allocation Failure) 96M->10M(128M) 12.847ms
```

G1 GC

How it works. **G1 (Garbage-First)** divides the heap into many equally sized **regions** (typically 1-32 MB each) rather than into two contiguous young/old blocks. Each region can play the role of Eden, survivor, or old space, and this assignment can change over time. G1 tracks how much garbage is in each region and, true to its name, collects the regions with the *most garbage first* — maximizing reclaimed memory per unit of pause time. Most of its work (finding live objects, some copying) happens concurrently with the application; only short phases are stop-the-world.

Pause characteristics. Designed around a **pause-time goal** you set, not a fixed algorithm — G1 tries to keep pauses under a target (default 200 ms) by choosing how many regions to collect in each cycle. It trades a bit of throughput for much more predictable pauses than Parallel GC.

Heap sweet spot. Medium to large heaps (multi-GB, up into hundreds of GB) on multi-core machines — this is the general-purpose “good enough for almost everything” choice.

Flag to enable: `-XX:+UseG1GC` (this is also the **default collector since JDK 9**, so usually no flag is needed at all)

When to pick it. The default choice for most server applications: web services, microservices, anything wanting a balance of throughput and bounded pauses without special tuning. Tune the target with `-XX:MaxGCPauseMillis=<ms>`.

Humongous objects. Any object at least half the size of a single G1 region (region size defaults based on heap size, typically 1-32 MB) is classified as **humongous** and allocated directly into one or more dedicated regions, bypassing the normal young-gen path entirely. Humongous objects are never copied during young collections (too expensive to move), so they can only be reclaimed by an old-gen/full collection — a service that allocates lots of large arrays or buffers (e.g. `big byte[]` responses, large `ArrayList` backing arrays) can suffer humongous-allocation-driven full GCs even though the heap “looks” mostly empty. Symptom in the log: frequent `Pause Young (Concurrent Start) (G1 Humongous Allocation)` entries. Fix by increasing `-XX:G1HeapRegionSize` so fewer objects cross the humongous threshold, or by reducing how often the code allocates very large single objects (e.g. streaming/chunking instead of buffering the whole payload).

Example log line — G1 pauses are qualified with *why* the pause happened:

```
[gc] GC(41) Pause Young (Concurrent Start) (G1 Evacuation Pause)
↳ 612M->140M(2048M) 9.774ms
[gc] GC(42) Pause Remark 2.011ms
[gc] GC(43) Pause Cleanup 0.415ms
```

Concurrent Start kicks off a background marking cycle for the old generation; Remark and Cleanup are the short stop-the-world phases that bookend that concurrent marking work.

ZGC

How it works. **ZGC (Z Garbage Collector)** is designed for extremely low pause times, largely independent of heap size. It does almost all of its work — marking, relocating objects, remapping references — concurrently with the running application, using **colored pointers** (metadata bits embedded in the reference itself) and **load barriers** (a small check the JVM inserts on reference reads) to let it move objects while the application keeps running, safely redirecting any thread that reads a stale reference. Since JDK 21, ZGC has a **generational mode** that adds young/old generations (like G1) for much better throughput, because it can collect young garbage cheaply instead of scanning the whole heap every cycle.

Pause characteristics. Sub-millisecond stop-the-world pauses, and — this is the headline feature — pause time does **not** scale with heap size. A 10 GB heap and a 10 TB heap have similarly tiny pauses.

Heap sweet spot. Large to very large heaps (multi-GB to multi-TB) where minimizing worst-case pause latency matters more than raw throughput or memory footprint (ZGC uses more memory and CPU overhead for its barriers and metadata than G1).

Flag to enable: `-XX:+UseZGC` (generational mode is the default *shape* of ZGC as of JDK 21+; explicitly request it with `-XX:+UseZGC -XX:+ZGenerational` on JDK 21, and it's simply the only mode from JDK 23 onward)

When to pick it. Latency-critical services with strict pause-time SLAs (trading systems, real-time bidding, anything where a 500 ms pause is a user-visible incident) and large heaps. **Non-generational ZGC is deprecated for removal starting JDK 23** — generational ZGC is the production-ready, actively developed mode going forward.

Example log line — note the tiny STW pauses; almost everything else in a ZGC cycle is logged as Concurrent, not Pause:

```
[gc] GC(88) Pause Mark Start 0.012ms
[gc] GC(88) Pause Mark End 0.031ms
[gc] GC(88) Pause Relocate Start 0.028ms
[gc] GC(88) Garbage Collection (Allocation Rate) 8192M(50%)->6144M(37%)
```

Shenandoah

How it works. Shenandoah (developed by Red Hat) has a similar goal to ZGC — pause times independent of heap size — but a different mechanism: it uses **Brooks pointers** (an extra forwarding pointer stored with each object) and concurrent evacuation to move objects while the application runs, along with concurrent marking and reference updating. Conceptually it's a sibling approach to ZGC solving the same problem with different bookkeeping.

Pause characteristics. Very low, largely heap-size-independent pauses, comparable in spirit to ZGC, though the two have different throughput/footprint trade-offs depending on workload — benchmark both if latency is critical.

Heap sweet spot. Same territory as ZGC: large heaps, latency-sensitive workloads.

Flag to enable: `-XX:+UseShenandoahGC`

When to pick it. Available in most OpenJDK builds (Red Hat, Eclipse Temurin/Adoptium, Amazon Corretto) but notably **not shipped in Oracle's own JDK builds**. Pick it when you're already on a distribution that includes it and want ZGC-like pause behavior, or want to compare it against ZGC for your specific workload.

Collector Comparison

Collector	Pause goal	Throughput	Heap size sweet spot	Enable flag	Default since
Serial	Long, but simple	Low (single-threaded)	Small (< few hundred MB)	<code>-XX:+UseSerialGC</code>	Never default on Server VMs
Parallel	Long, throughput-first	Highest	Medium-large, batch jobs	<code>-XX:+UseParallelGC</code>	Default in JDK 8
G1	Configurable target (default 200 ms)	High	Medium-large, general purpose	<code>-XX:+UseG1GC</code>	Default since JDK 9

Collector	Pause goal	Throughput	Heap size sweet spot	Enable flag	Default since
ZGC	Sub-millisecond, size-independent	Good (generational mode)	Large-huge, latency-critical	-XX:+UseZGC	Not default; opt-in
Shenandoah	Sub-millisecond, size-independent	Good	Large, latency-critical	-XX:+UseShenandoahGC	Not default; opt-in, OpenJDK-only

Rule of thumb for an interview: **G1 unless you have a specific reason not to**. Reach for ZGC or Shenandoah when pause time is the dominant concern and you have the memory/CPU budget to spend on it; reach for Parallel for pure batch throughput; reach for Serial only in tiny, constrained environments.

Memory Leaks

A Java “memory leak” doesn’t mean memory vanishes — it means objects stay **reachable** long after they’re logically dead, so the GC can never reclaim them. The heap slowly grows until an `OutOfMemoryError`. Here are the classic causes.

Static collections that only grow.

```
public class SessionRegistry {
    // Wrong: nothing ever removes entries, so this grows forever
    private static final Map<String, Session> SESSIONS = new HashMap<>();

    public static void register(String id, Session s) {
        SESSIONS.put(id, s);
    }
    // no remove() is ever called
}
```

A static field lives as long as the class is loaded — effectively forever in most applications. If nothing ever calls `remove`, this is an unbounded leak in plain sight.

Unbounded caches.

```
// Wrong: a plain HashMap used as a cache has no eviction policy
private final Map<Long, Report> cache = new HashMap<>();
```

```
public Report get(Long id) {
    return cache.computeIfAbsent(id, this::loadReport); // grows forever
}
```

```
// Right: bound it, with size or time-based eviction
private final Cache<Long, Report> cache = Caffeine.newBuilder()
    .maximumSize(10_000)
    .expireAfterAccess(Duration.ofMinutes(30))
    .build();
```

Listeners/callbacks never unregistered.

```
public class EventBus {
    private final List<Listener> listeners = new ArrayList<>();
    public void subscribe(Listener l) { listeners.add(l); }
    // if callers subscribe but never call a matching unsubscribe(l),
    // every subscriber (and everything it references) leaks forever
}
```

This is especially common in UI frameworks and long-lived event buses. Always pair subscribe/addListener with a matching unsubscribe/removeListener, ideally enforced with try-finally or AutoCloseable.

ThreadLocal in pooled threads.

```
// Wrong: thread pools reuse threads, so a ThreadLocal that's set but
// never removed keeps its value (and everything it references) alive
// for the LIFE OF THE POOLED THREAD, across unrelated tasks
private static final ThreadLocal<UserContext> CONTEXT = new
→ ThreadLocal<>();

void handleRequest(Request r) {
    CONTEXT.set(loadUserContext(r));
    process(r);
    // missing CONTEXT.remove() – the UserContext (and its whole reference
    // graph) stays attached to this pool thread until it's overwritten
}
```

```
// Right
void handleRequest(Request r) {
    CONTEXT.set(loadUserContext(r));
    try {
        process(r);
    }
```

```

    } finally {
        CONTEXT.remove(); // always clean up, even on exception
    }
}

```

ClassLoader leaks. Common in application servers that redeploy webapps without restarting the JVM. If any object (a thread, a static reference in a shared library, a JDBC driver registered in `DriverManager`) keeps a reference into classes loaded by the *old* deployment's classloader, that entire classloader — and every class and static field it loaded — cannot be garbage collected, even after redeploy. Symptom: `OutOfMemoryError: Metaspace` (or the older `PermGen` space) that grows with every redeploy.

substring history (old JDKs). Before Java 7u6, `String.substring()` shared the backing `char[]` array with the original string — a small substring of a huge string kept the *entire* huge array alive.

```

// Only a real leak on JDK <= 6 / early 7 (fixed since 7u6 – substring
↳ copies now)
String hugeLine = readEntireFileAsOneLine(); // 10 MB string
String tinyPart = hugeLine.substring(0, 5); // pre-fix: still references
↳ the 10 MB array!

```

On modern JDKs (7u6+) substring always copies, so this specific leak is historical — but it's still a favorite interview question, and the general lesson (watch what a “small” object secretly references) still applies to things like `ByteBuffer` slices.

Non-static inner classes holding outer references.

```

public class ReportGenerator {                // large object with lots of
↳ state
    private byte[] hugeBuffer = new byte[100_000_000];

    class ProgressListener {                  // non-static inner class!
        void onProgress(int percent) { System.out.println(percent); }
    }

    ProgressListener newListener() {
        return new ProgressListener();      // implicitly holds
↳ ReportGenerator.this
    }
}

```

Every instance of a non-static inner class carries a hidden reference to its enclosing instance. If that listener is registered somewhere long-lived (a static event bus, a cache),

the *entire* outer ReportGenerator — including its 100 MB buffer — leaks with it. Fix: make the inner class static (or a top-level class) if it doesn't need the outer instance, and pass in only the specific data it needs.

Unclosed resources.

```
// Wrong: if readAll() throws, the stream is never closed – leaks a file
↪ handle
// and, since streams often hold internal buffers, JVM-heap memory too
FileInputStream in = new FileInputStream(path);
byte[] data = in.readAllBytes();
in.close();

// Right
try (FileInputStream in = new FileInputStream(path)) {
    byte[] data = in.readAllBytes();
} // closed automatically, even on exception
```

Direct (off-heap) buffer leaks. `ByteBuffer.allocateDirect()` and memory-mapped files allocate native memory *outside* the Java heap, freed only when the buffer object itself is garbage collected and its cleaner runs. Holding on to direct buffers longer than necessary (e.g., caching them the same way you'd cache a normal object) leaks native memory that **won't show up in a heap dump or -Xmx usage at all** — it shows up as the process's resident memory (RSS) growing past what the heap accounts for, which is a common source of confusing container OOM-kills where “the heap looks fine.”

```
// Wrong: direct buffers cached indefinitely, never explicitly released
private static final Map<String, ByteBuffer> BUFFER_CACHE = new
↪ HashMap<>();
ByteBuffer buf = ByteBuffer.allocateDirect(64 * 1024 * 1024); // 64 MB
↪ native memory
BUFFER_CACHE.put(key, buf); // never removed – native memory leak,
↪ invisible on the heap

// Right: bound the cache and/or size direct memory explicitly with
// -XX:MaxDirectMemorySize=<size> so a leak throws an OutOfMemoryError
↪ instead
// of silently exhausting container memory
```

Summary of causes and fixes:

Cause	Why it leaks	Fix
Static collection that only grows	Static fields live for the program's lifetime	Bound size, add explicit removal
Unbounded cache	No eviction policy	Use a cache library with <code>maximumSize/TTL</code>
Listener never unregistered	Event source outlives the “logical” subscriber	Pair every <code>subscribe</code> with <code>unsubscribe</code>
<code>ThreadLocal</code> in a pool	Pooled threads outlive individual tasks	Always call <code>remove()</code> in a <code>finally</code>
ClassLoader leak	A live reference reaches into an “unloaded” deployment's classes	Deregister drivers/threads on shutdown; restart JVM on redeploy if needed
substring history	Pre-7u6 shared backing array	N/A on modern JDKs; still watch buffer slices
Non-static inner class	Hidden reference to outer instance	Make it <code>static</code> unless it needs the outer instance
Unclosed resource	Native/buffered state never released	<code>try-with-resources</code>
Direct buffer left cached	Off-heap memory tied to buffer object's lifecycle	Bound cache; set <code>-XX:MaxDirectMemorySize</code>

How to find a leak: heap dump + dominator tree.

1. Capture a heap dump while the process is exhibiting the growth (or trigger one automatically on OOM — see [JVM Flags and Diagnostics](#)):

```
jcrcmd <pid> GC.heap_dump /tmp/heap.hprof
```

2. Open it in a heap dump analyzer (Eclipse MAT, or JFR/JMC's heap dump view, or VisualVM).
3. Look at the **dominator tree**, not just “biggest objects.” An object *X* *dominates* object *Y* if every path from a GC root to *Y* passes through *X* — meaning if *X* were freed, *Y* would become garbage too. The dominator tree groups the *retained* size (an object plus everything it exclusively keeps alive) under its dominator, so the true “root cause” object floats to the top instead of being hidden among thousands of small leaked instances.
4. Compare two heap dumps taken minutes apart (MAT's “compare” feature) to see which object counts are *still climbing* — that's a much stronger leak signal than a single snapshot, since some growth is normal caching, not a leak.

5. Once you've found the dominating object (say, a `HashMap` field on some `SessionRegistry`), trace back through its “path to GC roots” to find *why* it's still reachable, then fix the reachability (add eviction, call `remove`, unregister the listener, etc.).

Escape Analysis

What it is. Escape analysis is a JIT compiler optimization (see Chapter 11) that determines whether an object created inside a method can be observed — can “escape” — outside that method (returned, stored in a field, passed to another thread, etc.). If the JIT can *prove* an object never escapes its allocating method or thread, it can apply optimizations that would be unsafe for an object visible elsewhere.

Scalar replacement. If an object provably never escapes, the JIT can skip allocating it as an object on the heap at all and instead break it into its individual primitive fields (“scalars”), keeping those in registers or on the stack — exactly as if you'd hand-written the method using separate local variables instead of a wrapper object.

```
class Point {
    int x, y;
    Point(int x, int y) { this.x = x; this.y = y; }
}

int distanceSquared(int x1, int y1, int x2, int y2) {
    Point p1 = new Point(x1, y1); // may never actually be allocated on
    ↪ the heap
    Point p2 = new Point(x2, y2); // if the JIT proves neither escapes
    ↪ this method
    int dx = p1.x - p2.x;
    int dy = p1.y - p2.y;
    return dx * dx + dy * dy;
}
```

Lock elision. Similarly, if the JIT proves an object is only ever visible to one thread (it never escapes to another thread), any synchronized block guarding that object is provably uncontended — no other thread could possibly be waiting on that monitor — so the JIT can remove the locking overhead entirely.

```
void process() {
    StringBuilder sb = new StringBuilder(); // local, never escapes this
    ↪ method
    synchronized (sb) {                               // lock is pointless —
    ↪ nothing else can see 'sb'
```

```
        sb.append("data");  
    }  
    // JIT may elide the lock/unlock entirely  
}
```

Why “objects are always on the heap” is not strictly true. It’s a common simplification for beginners, and it’s *usually* true — but escape analysis is a real, everyday counterexample. A non-escaping object can live entirely in registers/stack, never touching the heap and never costing the GC anything. This is why microbenchmarks that appear to allocate millions of objects sometimes show suspiciously low GC activity — the JIT proved the objects never escaped and optimized the allocation away (this is also a classic JMH benchmarking pitfall — see Chapter 20).

Why you should NOT design code around it. Escape analysis is a best-effort, unguaranteed, JIT-tier-dependent optimization:

- It only kicks in after the JIT has profiled the method enough to compile it at a high tier (see Chapter 11 on JIT compilation) — cold code gets no benefit.
- Any change that makes escape harder to prove — passing the object to an interface method the JIT can’t fully inline, storing it in a collection “just for logging,” returning it conditionally — silently disables the optimization, with no compiler warning.
- It’s a JVM/version-specific implementation detail, not part of the Java Language Specification. Different JVMs (or the same JVM with different flags) may or may not apply it.

Write clear code — small objects, minimal scope, no unnecessary escapes — and let escape analysis take advantage of it *if* it can. Never rely on it being active as a correctness or capacity-planning assumption.

JVM Flags and Diagnostics

Flags are grouped into standard (`-X...`), and the vast “extra” (`-XX:...`) category, which itself splits into product flags (safe, documented) and experimental/diagnostic flags (need `-XX:+UnlockExperimentalVMOptions` or `-XX:+UnlockDiagnosticVMOptions`).

Commonly needed flags for memory work:

Flag	Purpose
<code>-Xms<size></code>	Initial heap size (e.g. <code>-Xms512m</code>)
<code>-Xmx<size></code>	Maximum heap size (e.g. <code>-Xmx4g</code>)
<code>-Xss<size></code>	Thread stack size
<code>-XX:MetaspaceSize=<size></code>	Initial metaspace size before triggering a GC to expand it
<code>-XX:MaxMetaspaceSize=<size></code>	Cap on metaspace (class metadata) — unbounded by default!
<code>-XX:+HeapDumpOnOutOfMemoryError</code>	Automatically write a .hprof heap dump when an <code>OutOfMemoryError</code> is thrown
<code>-XX:HeapDumpPath=<path></code>	Where to write that automatic heap dump
<code>-XX:+PrintFlagsFinal</code>	Print the effective value of every JVM flag (great for checking what a collector chose automatically)
<code>-Xlog:gc*:file=gc.log:time,uptime,level,tags</code>	Unified logging: write detailed GC logs with timestamps to a file
<code>-XX:MaxDirectMemorySize=<size></code>	Cap on off-heap <code>ByteBuffer.allocateDirect()</code> memory — unbounded by default
<code>-XX:+ExitOnOutOfMemoryError</code>	Kill the JVM immediately on any <code>OutOfMemoryError</code> , instead of limping along in a corrupted state (useful under an orchestrator that will restart the pod/container)
<code>-XX:+UseStringDeduplication</code>	(G1 only) Collapse duplicate <code>String</code> backing <code>char[]/byte[]</code> arrays during GC to save heap — helpful for apps that hold many identical strings (e.g. parsed JSON keys)
<code>-XX:+AlwaysPreTouch</code>	Commit and zero all heap pages at startup instead of lazily — trades slower startup for eliminating page-fault pauses later, useful with <code>-Xms == -Xmx</code>
<code>-XX:NativeMemoryTracking=summary</code>	Turn on Native Memory Tracking (NMT) to account for <i>non-heap</i> JVM memory (thread stacks, GC structures, code cache, metaspace)

```
# See exactly what heap size G1 picked by default on this machine
java -XX:+PrintFlagsFinal -version | grep -i heapsize

# Turn on detailed GC logging to a rotating file
java
↪ -Xlog:gc*:file=gc.log:time,uptime,level,tags:filecount=5,filesize=10M
↪ -jar app.jar
```

Native Memory Tracking (NMT). The heap is only part of a JVM process's memory footprint. NMT breaks down exactly where the *rest* went — thread stacks, GC bookkeeping, the JIT code cache, metaspace, internal buffers — which is exactly what you need when

RSS (resident memory) is much bigger than `-Xmx` would suggest (a classic container OOM-kill mystery).

```
# Enable at startup (small, constant overhead – safe for production)
java -XX:NativeMemoryTracking=summary -jar app.jar

# Query it later with jcmd
jcmd <pid> VM.native_memory summary
```

```
Total: reserved=5324912KB, committed=2109344KB
-           Java Heap (reserved=4194304KB, committed=1048576KB)
-           Class (reserved=1069056KB, committed=45120KB)
-           Thread (reserved=32784KB, committed=32784KB)
-           GC (reserved=25368KB, committed=25368KB)
-           Symbol (reserved=15236KB, committed=15236KB)
```

If “Thread” is unexpectedly huge, that’s often thousands of leaked/unbounded threads each holding a full stack (`-Xss`, default ~512KB-1MB each) — a different flavor of “memory leak” than a heap leak, but just as real.

Reading a GC log line (unified logging, JDK 9+):

```
[2026-08-07T10:15:22.123+0000][3.456s][info][gc] GC(12) Pause Young
↳ (Normal) (G1 Evacuation Pause) 512M->128M(1024M) 8.213ms
```

Read this as: at wall-clock time 10:15:22, 3.456s after JVM start, GC cycle #12 was a young “Normal” pause caused by a G1 evacuation. Heap usage went from 512 MB *before* the collection to 128 MB *after*, out of a current heap capacity of 1024 MB, and the whole pause took 8.213 ms. Watch three things across many lines: pause **duration** (spikes = latency problem), the **before/after gap** (how much was actually reclaimed — if it’s small and shrinking over time, that’s a leak signature), and **frequency** (collections getting closer together = allocation rate is outrunning what the heap can absorb).

Garbage Collection Tuning

Tuning a collector means adjusting a handful of knobs, always **after** you’ve measured, never before:

- **Pick the pause-time target, not a lower-level knob, first.** For G1: `-XX:MaxGCPauseMillis=100`. G1 will then choose region counts and collection sets to try to hit that target — you’re telling it *what you want*, not *how to get there*.
- **Heap sizing.** A too-small heap causes frequent GCs; a too-large heap causes rare but longer full GCs and wastes memory. Start with `-Xms` and `-Xmx` set to the *same* value in server/container deployments (see [JVM Performance Tuning](#)) so the heap doesn’t spend time resizing itself under load.
- **New (young) generation size.** `-XX:NewRatio=<n>` (old:young ratio) or explicit `-XX:NewSize=<size>` — a larger young gen means fewer, but bigger, minor GCs; too large and minor GC pauses grow.
- **Survivor space tuning.** `-XX:SurvivorRatio=<n>` — rarely needed unless promotion is happening too eagerly or too late (check with `-Xlog:gc+age=trace`).
- **Concurrent GC threads.** `-XX:ConcGCThreads=<n>` for G1/ZGC/Shenandoah’s background work; `-XX:ParallelGCThreads=<n>` for the STW parallel phases. Defaults scale with CPU count and are usually fine.
- **Watch for promotion failure / to-space exhaustion.** If the old generation doesn’t have enough free space to accept everything a minor GC needs to promote, the collector falls back to a full GC to make room — an expensive, avoidable pause. The GC log shows this explicitly:

```
[gc] GC(77) To-space exhausted
[gc] GC(78) Pause Full (G1 Evacuation Pause) 3800M->3750M(4096M)
↳ 1044.221ms
```

Fixes: grow the heap, grow the young generation’s headroom, or address whatever is driving the promotion rate up (see the leak-hunting steps above — a real leak often first shows up as promotion failures before it shows up as an outright OOM).

- **One change at a time, measured.** Change a single flag, run a representative load test, compare GC logs before/after. Changing three flags simultaneously and seeing an improvement tells you nothing about *which* flag helped (or which one is secretly hurting and being masked by the others).

JVM Performance Tuning

Memory tuning is one piece of the larger performance-tuning picture:

- **-Xms == -Xmx in containers.** Setting the initial and maximum heap to the *same* value avoids runtime heap-resizing pauses and — critically in Kubernetes/Docker — makes memory usage **predictable**, which matters because the container's memory limit will hard-kill (OOMKilled) the process if the JVM heap plus off-heap usage exceeds it, regardless of what the JVM *thinks* its budget is.
- **-XX:MaxRAMPercentage=<pct>** — instead of a hardcoded -Xmx, tell the JVM to use a *percentage* of the container's memory limit (default up to 25% historically; explicitly set higher, e.g. 75.0, for a heap-heavy service). This is the recommended approach for containers because it automatically adapts if the container's memory limit changes (e.g. between environments) without editing JVM flags.

```
java -XX:MaxRAMPercentage=75.0 -XX:InitialRAMPercentage=75.0 -jar
→ app.jar
```

- **Container awareness.** Modern JVMs (since JDK 10, mature by JDK 17+) read cgroup limits automatically — `Runtime.availableProcessors()` and `MaxRAMPercentage` calculations respect the container's *actual* CPU and memory limits, not the host machine's. Verify with `-XX:+PrintFlagsFinal` inside the container, not on your laptop — a value that looks right locally can be wrong inside a 512 MB container.
- **CPU quota affects GC thread count too.** `-XX:ParallelGCThreads` and `-XX:ConcGCThreads` default based on the *visible* CPU count. In a container with a low CPU quota (e.g. `limits.cpu: "2"` on a 32-core host), an old or misconfigured JVM/runtime combination that doesn't honor cgroup CPU quotas correctly can size GC thread pools for 32 cores, causing them to thrash for CPU time against the application's own threads — worth checking explicitly with `jcmd <pid> VM.flags | grep GCThreads` rather than assuming it's right.
- **Off-heap memory needs its own budget.** Direct buffers (`-XX:MaxDirectMemorySize`), the JIT code cache, metaspace, and thread stacks are all outside -Xmx — when sizing a container's memory limit, budget for heap **plus** all of these, not just the heap. A common rule of thumb is `heap = ~70-75%` of the container limit, leaving room for everything else; verify with NMT rather than guessing.
- **Metaspace sizing.** Metaspace holds class metadata (not object instances) and is off-heap by default with **no cap** unless you set one — a classloader leak (see above) can silently grow metaspace until the *whole container* runs out of memory, not just “the heap.” Set `-XX:MaxMetaspaceSize` explicitly in production so a metaspace leak becomes a visible, boring `OutOfMemoryError: Metaspace` instead of an unpredictable OS-level OOM kill.

- **Choosing a pause target.** Ask “what does the SLA actually require?” before picking a collector or a `MaxGCPauseMillis` value. A batch job with no user waiting has no reason to pay ZGC’s throughput/memory tax; a p99-latency-sensitive API absolutely does.
- **The golden rule: measure first, one change at a time.** Get a baseline (GC logs, JFR recording, actual latency/throughput numbers) *before* touching any flag. Change one thing. Re-measure under the same load. Keep the change only if it measurably helps your actual metric (not a synthetic microbenchmark). Tuning by folklore (“everyone sets `-XX:NewRatio=2`”) without measurement on *your* workload is a code smell in itself.

Profiling and Monitoring

Profiling answers “where is time/memory actually going” with low enough overhead to run in production; **monitoring** answers “is anything wrong right now” continuously over time. For memory specifically, you want to watch: heap usage over time (saw-tooth is healthy — a rising floor is a leak), GC pause frequency/duration, allocation rate (bytes/sec), and promotion rate (how much survives into old gen per minor GC). The tools below (JFR, JMC, `jcmd`, `jmap`, `jstack`) are the practical toolbox; a good code reviewer should be able to name which tool answers which question without reaching for a web search.

Question	Reach for
“What’s the effective value of every JVM flag right now?”	<code>jcmd <pid> VM.flags</code>
“What’s happening in this app over the last hour, in production, with low overhead?”	JFR recording, analyzed in JMC
“Why did we just get an <code>OutOfMemoryError</code> ?”	Heap dump (<code>jmap/jcmd</code>) + Eclipse MAT or JMC
“Are threads stuck / is there a deadlock right now?”	<code>jstack / jcmd Thread.print</code>
“What is allocating the most garbage?”	JFR allocation profiling, viewed in JMC
“Is memory usage trending up or holding steady over days?”	A metrics pipeline (Micrometer/Prometheus scraping JMX GC/heap metrics) plus dashboards, not one-off tool runs
“Quick, lightweight look at live heap/GC/thread stats without a full recording?”	<code>jstat</code> (e.g. <code>jstat -gcutil <pid> 1000</code>) or VisualVM’s live view
“Where is CPU time going, down to the line, with minimal overhead?”	<code>async-profiler</code> (third-party, flame graphs) or JFR’s method-profiling events

For continuous production monitoring (as opposed to point-in-time debugging), most teams expose JVM metrics through Micrometer or the JMX GC/heap/thread MBeans into Prometheus/Grafana or an APM tool, and alert on trends (heap floor rising over days, GC pause p99 crossing a threshold) rather than manually running these tools reactively. The manual tools in this chapter are for *investigating* an alert or incident, not replacing that baseline monitoring.

JFR (JDK Flight Recorder)

JFR is a low-overhead (typically <1-2%) profiling and event-recording engine built into the JDK itself — no external agent needed. It records hundreds of event types: object allocations (sampled), GC pauses, thread state changes, lock contention, exceptions, class loading, and more, all with timestamps, into a compact `.jfr` binary file you analyze after the fact (or stream live).

```
# Start a 60-second recording on a running process, using the built-in
# "profile" settings (more detail, still low overhead) and write it to
↪ rec.jfr
jcmd <pid> JFR.start settings=profile duration=60s filename=rec.jfr

# Start recording from the moment the JVM launches (captures startup too)
java -XX:StartFlightRecording=filename=startup.jfr,duration=60s -jar
↪ app.jar

# Check on / stop a running recording
jcmd <pid> JFR.check
jcmd <pid> JFR.stop name=1 filename=final.jfr
```

Because overhead is low enough for production, JFR is the tool of choice when you can't reproduce an issue locally and need to capture “what actually happened” on the live system. The resulting `.jfr` file is best read with JMC.

Some of the event categories most relevant to memory work:

Event category	What it tells you
jdk.ObjectAllocationSample / jdk.ObjectAllocationInNewTLAB	Which classes/call sites are allocating the most, sampled cheaply

Event category	What it tells you
<code>jdk.GarbageCollection / jdk.GCPhase*</code>	Every GC pause, its cause, duration, and phase breakdown
<code>jdk.OLDObjectSample</code>	Objects that have survived a long time — a direct signal for leak-hunting without a full heap dump
<code>jdk.ThreadPark / jdk.JavaMonitorWait</code>	Lock contention and thread blocking, useful alongside <code>jstack</code> for deadlock/contention analysis
<code>jdk.ExecutionSample</code>	Periodic stack-trace sampling for CPU hot-spot / “hot method” analysis

The `jdk.OLDObjectSample` event deserves a callout: it’s JFR’s built-in, low-overhead leak detector — it tracks a sample of objects that have lived unusually long and shows their allocation stack trace and retained size, often letting you spot a leak from a *live* recording without ever needing to pull a full heap dump.

JMC (JDK Mission Control)

JMC (JDK Mission Control) is the GUI tool for *analyzing* JFR recordings (and for live-monitoring a running JVM via JMX). Open a `.jfr` file in JMC and you get: a memory tab showing heap usage and GC pause timelines, an allocation tab ranking classes by bytes allocated (often the fastest way to spot “why is GC busy” — a class allocating way more than expected jumps out immediately), a “hot methods” view from sampled stack traces, and a thread/lock-contention view. JMC ships separately from the JDK (download from Adoptium/Eclipse) but reads recordings from any JDK 11+ vendor. For a code review context: if someone claims “this change reduces allocations,” JMC’s allocation-by-class view is the objective way to check.

jcmm

jcmm is the modern, unified command-line tool for talking to a running JVM — it has mostly superseded the older single-purpose tools (`jstack`, older `jmap` use cases) because it exposes the same diagnostic commands through one consistent interface, and it works reliably even in JVMs where the legacy tools are flaky (e.g. minimal container base images). List a process’s PIDs and available commands first:

```
# List all running JVMs and their PIDs (like a Java-aware 'ps')
jcmd -l

# List every diagnostic command a specific JVM supports
jcmd <pid> help
```

Common diagnostic commands:

```
# Heap summary: generation sizes, usage, GC algorithm in use
jcmd <pid> GC.heap_info

# Show every JVM flag and its current effective value
jcmd <pid> VM.flags

# Dump all thread stacks (equivalent to jstack, but built-in)
jcmd <pid> Thread.print

# Trigger a heap dump without waiting for OOM
jcmd <pid> GC.heap_dump /tmp/heap.hprof

# Start a JFR profiling recording for 60 seconds
jcmd <pid> JFR.start settings=profile duration=60s filename=rec.jfr

# Force a full GC right now (diagnostic use only – never in app code!)
jcmd <pid> GC.run

# Class-level allocation histogram (like jmap -histo:live, built into
↪ jcmd)
jcmd <pid> GC.class_histogram

# Native (off-heap) memory breakdown, if NMT was enabled at startup
jcmd <pid> VM.native_memory summary

# Print system properties / general JVM version and uptime info
jcmd <pid> VM.system_properties
jcmd <pid> VM.uptime
```

A quick reference of the commands most relevant to memory and thread debugging:

Command	What it does
GC.heap_info	Per-generation heap usage summary
GC.heap_dump <path>	Write a full heap dump (.hprof)
GC.class_histogram	Live-object counts and bytes by class

Command	What it does
<code>GC.run</code>	Request a full GC (diagnostic use only)
<code>VM.flags</code>	Effective value of every JVM flag
<code>VM.native_memory summary</code>	Off-heap memory breakdown (requires NMT enabled)
<code>Thread.print</code>	Full thread dump, including deadlock detection
<code>JFR.start / JFR.stop / JFR.check</code>	Control a Flight Recorder recording

`GC.heap_info` output looks like this — read the “used” vs “capacity” per generation to see how full each region is and whether the heap has room to grow:

```
garbage-first heap  total 1048576K, used 402384K [0x...]
  region size 1024K, 1024 total, 84 available
Metaspace          used 45231K, committed 46080K, reserved 1114112K
```

jmap

jmap is the older, more narrowly-focused memory tool. Its two most useful invocations:

```
# Live-object histogram: class name, instance count, total bytes – sorted
↪ by size.
# "live" forces a GC first so only reachable objects are counted (skip it
↪ and you
# also count garbage waiting to be collected, which inflates the numbers).
jmap -histo:live <pid>

# Full heap dump in the standard .hprof binary format, for MAT / JMC /
↪ VisualVM
jmap -dump:live,format=b,file=heap.hprof <pid>
```

Histogram output looks like:

num	#instances	#bytes	class name
1:	842,113	67,369,040	[C (char[])
2:	412,904	23,942,432	java.lang.String
3:	88,213	21,171,120	com.example.model.OrderLine

Read it top-to-bottom by bytes: if `OrderLine` has far more instances than the number of orders you'd expect to have in memory, that's your leak candidate — cross-reference with a heap dump's dominator tree to find *why* they're still reachable.

jmap is largely superseded by jcmd. `jcmd <pid> GC.heap_dump` and `jcmd <pid> GC.class_histogram` do the same jobs, plus `jcmd` is maintained more actively and is more reliable across JVM versions and restricted environments. New tooling knowledge should default to `jcmd`; know `jmap`'s syntax mainly because it still appears in older runbooks, documentation, and — often — interview questions.

jstack

jstack dumps the stack trace of every thread in a running JVM — essential for diagnosing hangs, deadlocks, and high-CPU-but-stuck-somewhere issues.

```
jstack <pid>
# or, if the process is unresponsive to normal signals:
jstack -F <pid>
```

Each thread entry shows its name, state, and a Java-level stack trace, e.g.:

```
"pool-1-thread-3" #15 prio=5 os_prio=0 tid=0x... nid=0x1a2b waiting on
↳ condition [0x...]
  java.lang.Thread.State: WAITING (parking)
    at jdk.internal.misc.Unsafe.park(Native Method)
    at java.util.concurrent.locks.AbstractQueuedSynchronizer.parkAndCj
↳ heckInterrupt
    at
↳ com.example.OrderProcessor.awaitCompletion(OrderProcessor.java:88)
```

Taking two or three `jstack` dumps a few seconds apart and comparing them is a classic trick: a thread stuck at the *exact same line* across all of them is a strong signal of a genuine hang (deadlock, infinite loop, or blocked I/O), whereas a thread that's at a *different* line each time is simply doing normal work.

Thread states you'll see in a dump:

State	Meaning
NEW	Thread object created, <code>start()</code> not yet called

State	Meaning
RUNNABLE	Executing, or ready and waiting for CPU — includes threads doing blocking I/O in native code
BLOCKED	Waiting to enter a synchronized block/method held by another thread
WAITING	Waiting indefinitely for another thread (<code>Object.wait()</code> , <code>Thread.join()</code> , <code>LockSupport.park()</code> with no timeout)
TIMED_WAITING	Same as WAITING but with a timeout (<code>Thread.sleep()</code> , <code>wait(ms)</code> , <code>park(nanos)</code>)
TERMINATED	Run to completion

A thread pool full of BLOCKED threads all waiting on the *same* lock, with one thread holding it and itself stuck elsewhere (often in WAITING on I/O or another lock), is the classic “everything is stuck behind one slow/stuck thread” pattern — look for the single thread that’s the odd one out, not the many that are merely blocked on it.

jstack vs jcmd Thread.print. They produce essentially the same information; `jcmd <pid> Thread.print` is the more modern, actively maintained path (same rationale as `jmap` vs `jcmd` for heap dumps), and it works over the same attach mechanism, so prefer it when starting fresh, while recognizing `jstack` output in existing runbooks and tickets.

Worked Walkthrough: Debugging an OutOfMemoryError

Scenario: a service occasionally crashes with `java.lang.OutOfMemoryError: Java heap space` in production.

Step 1 — make sure the next crash captures evidence. Add this flag (ideally it’s *always* on in production, before the incident even happens):

```
java -XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/var/dumps/ -jar
  ↪ app.jar
```

Step 2 — while the process is still alive but suspected of leaking, capture a live heap dump too (don’t wait for the crash if you can catch it early):

```
jcmd <pid> GC.heap_dump /var/dumps/pre-crash.hprof
```

Step 3 — check the GC log for the shape of the problem. If GC logging was already enabled (`-Xlog:gc*:file=gc.log:...`), look at the tail of the log right before the crash:

```
[gc] GC(430) Pause Full (G1 Evacuation Pause) 3980M->3971M(4096M)
↳ 812.442ms
[gc] GC(431) Pause Full (G1 Evacuation Pause) 3985M->3978M(4096M)
↳ 799.113ms
[gc] GC(432) Pause Full (G1 Evacuation Pause) 3990M->3982M(4096M)
↳ 820.977ms
```

This pattern — heap usage barely drops after each *full* GC, and full GCs are happening back-to-back — is the signature of a real leak (versus normal pressure, where after-GC usage would drop substantially).

Step 4 — open the heap dump in Eclipse MAT (or JMC). Run MAT’s built-in “Leak Suspects” report, or manually open the dominator tree, sort by retained size, and look at the top entries.

Step 5 — read the dominator tree result, e.g.:

Class Name	Shallow Heap	Retained
↳ Heap		
com.example.cache.ReportCache @ 0x7f2a...	48	
↳ 3,842,199,120		
-> java.util.HashMap\$Node[] entries	524,288	
↳ 3,842,190,000		

The ReportCache instance alone retains ~3.8 GB — almost the entire heap. That’s the leak. Trace its “path to GC roots” in MAT to see it’s a static field with no eviction policy.

Step 6 — fix and verify. Add a bounded eviction policy (e.g. switch to Caffeine with `maximumSize`), redeploy, and confirm with a fresh GC log that full-GC after-usage now drops back down close to the old-gen baseline instead of climbing indefinitely.

Worked Walkthrough: Debugging a Deadlock with jstack

Scenario: a service’s health check starts timing out; CPU is idle (not spinning), suggesting threads are stuck waiting, not busy-looping.

Step 1 — take a thread dump:


```
jstack <pid> > threads.txt
# or: jcmd <pid> Thread.print > threads.txt
```

Step 2 — search for the deadlock report. jstack detects classic lock-ordering deadlocks automatically and prints a dedicated section:

```
Found one Java-level deadlock:
=====
"Thread-A":
  waiting to lock monitor 0x00007f... (object 0x000000070a, a
↪ java.lang.Object),
  which is held by "Thread-B"
"Thread-B":
  waiting to lock monitor 0x00007f... (object 0x000000070b, a
↪ java.lang.Object),
  which is held by "Thread-A"

Java stack information for the threads listed above:
=====
"Thread-A":
  at com.example.TransferService.debit(TransferService.java:42)
  - waiting to lock <0x000000070a> (a java.lang.Object)
  - locked <0x000000070b> (a java.lang.Object)
"Thread-B":
  at com.example.TransferService.credit(TransferService.java:58)
  - waiting to lock <0x000000070b> (a java.lang.Object)
  - locked <0x000000070a> (a java.lang.Object)
```

Step 3 — read it as a cycle. Thread-A holds lock B's target and wants lock A's target; Thread-B holds the opposite. Neither can proceed — classic circular wait, the textbook deadlock condition.

Step 4 — trace it back to the code. Both `debit()` and `credit()` lock two account objects in *opposite order* depending on which account is the caller vs. the target:

```
// Wrong — lock order depends on argument order, so A->B and B->A both
↪ happen
synchronized (fromAccount) {
  synchronized (toAccount) {
    // transfer logic
  }
}
```

Step 5 — fix with a consistent lock order (e.g., always lock the account with the smaller ID first), or replace with a higher-level concurrency utility that avoids manual nested locking entirely (`java.util.concurrent` structures, or `ReentrantLock.tryLock` with a timeout so the app can recover instead of hanging forever). See Chapter 13 for lock-ordering strategies in depth.

Step 6 — verify. Under the same load test that reproduced the hang, confirm the health check stays responsive and no deadlock section appears in a follow-up `jstack` dump.

Common Code-Review Interview Pitfalls

- 1. Calling `System.gc()` in application code.** Why it matters: it's only a hint the JVM may ignore, and on some collectors it can trigger a genuinely expensive full GC pause in production for no real benefit.

```
// Before
cache.clear();
System.gc(); // "just to be safe" – actually just a pause risk
// After
cache.clear(); // trust the collector; profile if there's an actual
→ problem
```

- 2. Using `finalize()` for cleanup.** Why it matters: it's deprecated for removal, runs at an unpredictable time (or never), and can silently swallow exceptions.

```
// Before
@Override protected void finalize() { socket.close(); }
// After
try (var socket = openSocket()) { /* use it */ } // deterministic close
```

- 3. Not calling `ThreadLocal.remove()` in pooled-thread code.** Why it matters: pooled threads outlive any single task, so a forgotten value (and its whole reference graph) leaks for the pool's lifetime and can even leak data between unrelated requests.

```
// Before
CONTEXT.set(userContext); process(request);
// After
CONTEXT.set(userContext);
try { process(request); } finally { CONTEXT.remove(); }
```

- 4. Using an unbounded HashMap as a cache.** Why it matters: with no eviction policy, it grows without limit and is a slow-motion `OutOfMemoryError` waiting to happen.

```
// Before
private final Map<Long, Report> cache = new HashMap<>();
// After
private final Cache<Long, Report> cache =
    Caffeine.newBuilder().maximumSize(10_000).build();
```

- 5. Registering listeners without a matching unregister.** Why it matters: the listener (and everything it closes over) stays reachable from the event source forever, even after the “subscriber” logically should be gone.

```
// Before
eventBus.subscribe(listener); // no corresponding unsubscribe anywhere
// After
eventBus.subscribe(listener);
// ... and on shutdown/cleanup:
eventBus.unsubscribe(listener);
```

- 6. Making a class a non-static inner class when it doesn't need the outer instance.** Why it matters: every instance silently holds a hidden reference to its enclosing object, which can keep a large outer object (and its state) alive far longer than intended.

```
// Before
class Listener { /* implicitly holds Outer.this, unused */ }
// After
static class Listener { /* no hidden outer reference */ }
```

- 7. Assuming -Xmx alone controls container memory usage.** Why it matters: metaspace, thread stacks, direct buffers, and JIT code cache are all off-heap and can push total memory past the container limit even when the heap itself looks fine, causing an OOM-kill with no Java-level stack trace.

```
# Before: heap-only thinking
java -Xmx3800m -jar app.jar # in a 4 GB container — no room for
↳ anything else
# After: leave headroom, or size as a percentage of the container limit
java -XX:MaxRAMPercentage=70.0 -jar app.jar
```

- 8. Setting `-Xms` far below `-Xmx` in a server/container workload.** Why it matters: the JVM has to repeatedly resize the heap under load, causing extra pauses and unpredictable memory footprint — exactly the opposite of what you want in a container with a hard memory limit.

```
# Before
java -Xms256m -Xmx4g -jar app.jar
# After
java -Xms4g -Xmx4g -jar app.jar
```

- 9. Leaving metaspace uncapped in a service that dynamically loads classes (or gets redeployed).** Why it matters: a classloader leak silently grows metaspace with no upper bound until the JVM (or container) runs out of memory in a confusing way.

```
# Before: no cap at all
java -jar app.jar
# After: explicit cap turns a silent leak into a clear, diagnosable OOM
java -XX:MaxMetaspaceSize=512m -jar app.jar
```

- 10. Writing code that depends on escape analysis for correctness or guaranteed performance.** Why it matters: it's a best-effort, JIT-tier-dependent optimization with no language-level guarantee — a minor refactor can silently disable it with no warning.

```
// Before — relying on "the JIT will optimize this away" as a design
→ assumption
Point p = new Point(x, y); // assumed to never really allocate
// After — write clear code; treat escape analysis as a bonus, not a
→ plan
int dx = x2 - x1, dy = y2 - y1; // if it matters, measure, don't assume
```

- 11. Reaching for `jmap -dump` under time pressure without knowing `jcmd` exists.** Why it matters: `jcmd` is the actively maintained, more reliable tool for the same job, and knowing it signals up-to-date operational knowledge in a review.

```
# Older habit
jmap -dump:live,format=b,file=heap.hprof <pid>
# Preferred
jcmd <pid> GC.heap_dump /tmp/heap.hprof
```

- 12. Choosing ZGC/Shenandoah “because it’s the newest” without a pause-time requirement.** Why it matters: both trade extra memory and CPU overhead for lower

pauses — paying that cost with no latency SLA to justify it is a wasted resource, and G1's balance is right for most services.

```
# Before: reflexively latency-optimizing an offline batch job
-XX:+UseZGC
# After: throughput-first collector for a job nobody is waiting on
↳ interactively
-XX:+UseParallelGC
```

- 13. Treating a single heap-usage snapshot as proof of (or against) a leak.** Why it matters: normal caching and generational promotion both look like “heap usage went up” in one snapshot; only a trend across multiple full-GC cycles (usage not dropping back down) distinguishes a leak from healthy behavior.

```
# Before: "heap usage is 80% full, must be a leak!"
# After: compare after-full-GC usage across several cycles over time;
# a leak shows a rising floor, not just a high peak.
```

- 14. Manually locking two objects in an order that depends on caller-supplied argument order.** Why it matters: it creates a classic deadlock the moment two threads call the method with the arguments swapped — exactly the bug `jstack`'s deadlock detector exists to catch.

```
// Before
synchronized (fromAccount) { synchronized (toAccount) { /* ... */ } }
// After: always lock in a fixed, consistent order regardless of
↳ arguments
Account first = fromAccount.id() < toAccount.id() ? fromAccount :
↳ toAccount;
Account second = fromAccount.id() < toAccount.id() ? toAccount :
↳ fromAccount;
synchronized (first) { synchronized (second) { /* ... */ } }
```

- 15. Not enabling `-XX:+HeapDumpOnOutOfMemoryError` in production.** Why it matters: without it, an `OutOfMemoryError` crash leaves no artifact to analyze — the exact evidence needed to diagnose the leak disappears along with the crashed process.

```
# Before
java -jar app.jar
# After
```

```
java -XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/var/dumps/ -jar  
↳ app.jar
```

Concurrency (Core)

Concurrency is what happens when more than one thread touches your program's state at the same time. Get it wrong and bugs show up rarely, non-deterministically, and usually in production under load — which makes concurrency one of the highest-value topics in a code-review interview. This chapter covers the classic “platform threads” toolkit: threads themselves, locks, `volatile`, atomics, `ThreadLocal`, and the memory-model rules (happens-before) that explain *why* all of it works. Examples target Java 21+; virtual threads and structured concurrency get their own treatment in Chapter 15.

Threads

A **thread** is an independent path of execution inside a program. A single Java process can run many threads at once, each with its own call stack, but all of them share the same heap memory — the same objects, the same static fields. That sharing is exactly what makes concurrency both powerful and dangerous.

The classic (pre-virtual-thread) way to create one is `Thread` backed directly by an OS thread — a “platform thread.” You can subclass `Thread` and override `run()`, or (preferred) pass it a `Runnable`:

```
public class ThreadBasics {
    public static void main(String[] args) throws InterruptedException {
        Thread worker = new Thread(() -> {
            System.out.println("Running on: " +
↳ Thread.currentThread().getName());
            }, "worker-1"); // name the thread – huge help in thread dumps and
↳ logs

        worker.start(); // schedules the thread to run; never call run()
↳ directly
        worker.join(); // wait for it to finish before continuing
        System.out.println("Done");
    }
}
```

```
    }
}
```

start() vs run(). Calling `start()` asks the JVM to create a new call stack and schedule execution. Calling `run()` directly just executes the code on the *current* thread — no concurrency happens at all. This is a common interview trick question.

Daemon vs user (non-daemon) threads. A **user thread** keeps the JVM alive — the process won't exit while any user thread is still running. A **daemon thread** is a “background helper” thread; the JVM will exit even if daemon threads are still running, and it won't wait for them. Garbage collection and JIT compilation run on daemon threads internally.

```
Thread heartbeat = new Thread(() -> {
    while (true) {
        System.out.println("tick");
        try { Thread.sleep(1000); } catch (InterruptedException e) {
            break; }
    }
});
heartbeat.setDaemon(true); // must be called BEFORE start()
heartbeat.start();
// If main() returns here, the JVM exits and heartbeat is killed mid-loop
// no cleanup code in it is guaranteed to run.
```

Thread priorities. Every thread has a priority from `Thread.MIN_PRIORITY` (1) to `Thread.MAX_PRIORITY` (10), default `Thread.NORM_PRIORITY` (5). This is only a **hint** to the OS scheduler about relative importance — the JVM spec doesn't guarantee any particular scheduling behavior, and most OSes map Java's ten levels onto a much coarser native scale. Don't design correctness around priorities; use them only as a mild tuning hint, if at all.

```
Thread background = new Thread(() -> doLowPriorityWork());
background.setPriority(Thread.MIN_PRIORITY); // hint: "less urgent," not a
// guarantee
background.start();
```

Naming threads. Give threads meaningful names. It costs nothing and it's the difference between a readable thread dump and a wall of `Thread-0`, `Thread-1`, `Thread-2`.

UncaughtExceptionHandler. If a thread's `run()` throws an unchecked exception and nobody catches it, the thread just dies silently by default — the exception goes to `System.err`, but nothing else happens, and no other thread is notified. Set a handler to observe or react to that:

```
Thread risky = new Thread(() -> {
    throw new IllegalStateException("boom");
});
risky.setUncaughtExceptionHandler((t, e) ->
    System.err.println("Thread " + t.getName() + " died: " + e));
risky.start();
// Prints: Thread Thread-0 died: java.lang.IllegalStateException: boom
// Without the handler, this exception would be easy to miss in production
→ logs.
```

You can also set a JVM-wide default via `Thread.setDefaultUncaughtExceptionHandler(...)`, useful for centralized alerting/metrics.

Interruption is cooperative cancellation. `Thread.interrupt()` does not forcibly stop a thread. It just sets an internal “interrupted” flag and, if the thread is blocked in something interruptible (`sleep`, `wait`, `join`, many blocking I/O and concurrency calls), wakes it up early with an `InterruptedException`. The target thread has to *check* for interruption and decide to stop. This is why it’s called cooperative — both sides have to participate.

```
public class CooperativeCancellation {
    public static void main(String[] args) throws InterruptedException {
        Thread worker = new Thread(() -> {
            while (!Thread.currentThread().isInterrupted()) {
                // do a unit of work
            }
            System.out.println("Worker noticed the interrupt and is
→ exiting cleanly");
        });
        worker.start();
        Thread.sleep(50);
        worker.interrupt(); // politely ask it to stop
        worker.join();
    }
}
```

If your loop calls a blocking method, catch `InterruptedException` and either exit or **re-interrupt and rethrow** — never swallow it silently, because that erases the cancellation signal for anyone further up the call stack:

```
void pollUntilCancelled() {
    while (true) {
        try {
            Thread.sleep(500);
            // do work
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt(); // restore the flag for
↪ callers
            return; // stop the loop
        }
    }
}
```

Why `Thread.stop()` and `Thread.suspend()` are dead. Both are deprecated for removal. `stop()` kills a thread instantly, mid-operation, which can leave shared objects in a half-updated, permanently broken state (it releases locks it holds without any guarantee the protected data is consistent). `suspend()` freezes a thread while it may be holding a lock, which can deadlock every other thread that needs that lock. Neither has a safe way to guarantee the target thread cleans up. **Cooperative interruption (or a `volatile cancel flag`) is the only sound replacement.**

Virtual threads. Java 21 introduced lightweight, JVM-managed threads that let you write blocking-style code cheaply at massive scale. They’re built on the same `Thread` API you just saw, but are covered in full in Chapter 15 (Structured Concurrency / Modern Concurrency), since they change a lot of the cost-benefit calculus for the patterns in this chapter.

Runnable and Callable

`Runnable` and `Callable<V>` are the two “unit of work” interfaces you hand to a thread or an executor.

	Runnable	Callable<V>
Method	<code>void run()</code>	<code>V call()</code> throws <code>Exception</code>
Returns a result?	No	Yes, typed <code>V</code>
Can throw checked exceptions?	No	Yes
Introduced	Java 1.0	Java 5 (<code>java.util.concurrent</code>)

	Runnable	Callable<V>
Typical use	new Thread(...), fire-and-forget tasks	ExecutorService.submit(...) when you need a Future<V>

```
import java.util.concurrent.*;

public class RunnableVsCallable {
    public static void main(String[] args) throws Exception {
        Runnable fireAndForget = () -> System.out.println("logged, no
        ↪ result");

        Callable<Integer> withResult = () -> {
            Thread.sleep(10);
            return 42;
        };

        ExecutorService pool = Executors.newFixedThreadPool(2);
        pool.execute(fireAndForget);           // Runnable: no Future
        Future<Integer> future = pool.submit(withResult); // Callable: get
        ↪ a Future
        System.out.println("Result: " + future.get()); // blocks until
        ↪ done: 42
        pool.shutdown();
    }
}
```

Thread.sleep() vs **Object.wait()**. Both pause a thread, but they're built for different purposes:

	Thread.sleep(ms)	Object.wait()
Holds locks while paused?	Yes — keeps any locks it holds	No — releases the monitor lock it's waiting on
Needs a lock to call?	No	Yes — must be called while holding the object's monitor
How it wakes up	Timeout elapses, or interrupted	Another thread calls notify()/notifyAll(), timeout (if given), or interrupted

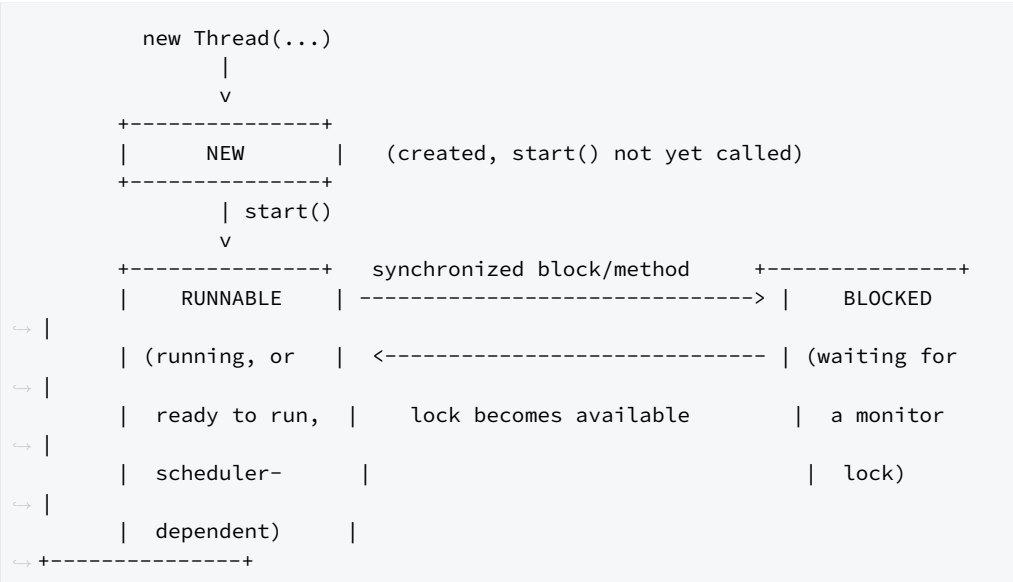
	Thread.sleep(ms)	Object.wait()
Typical use	“Pause for a fixed duration”	“Wait until some condition, controlled by another thread, becomes true”

```
synchronized (lock) {
    while (!conditionIsTrue) {
        lock.wait(); // releases 'lock' while waiting – other threads can
        ↪ acquire it
    }
}
// vs:
Thread.sleep(1000); // holds every lock this thread already has for the
↪ full second
```

Sleeping while holding a lock is a common cause of accidental contention — a slow, sleeping thread blocks everyone else waiting on that same lock.

Thread Lifecycle

Every Thread is always in exactly one of six states, defined by the Thread.State enum: NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, TERMINATED.




```

public class LifecycleDemo {
    public static void main(String[] args) throws InterruptedException {
        Object lock = new Object();
        Thread t = new Thread(() -> {
            synchronized (lock) {
                try { Thread.sleep(200); } catch (InterruptedException
↳ ignored) {}
            }
        });
        System.out.println(t.getState()); // NEW
        t.start();
        Thread.sleep(50);
        System.out.println(t.getState()); // TIMED_WAITING (inside
↳ Thread.sleep)
        t.join();
        System.out.println(t.getState()); // TERMINATED
    }
}

```

Synchronization

Synchronization means controlling access to shared, mutable state so that only one thread modifies it at a time, and every thread sees the latest value. In Java, the primary built-in tool is the `synchronized` keyword, which uses every object's built-in **monitor lock** (also called its intrinsic lock — see next section).

Synchronized methods vs synchronized blocks.

```

public class Counter {
    private int count = 0;

    // Synchronized instance method: locks 'this'
    public synchronized void incrementWhole() {
        count++;
    }

    // Synchronized block: locks an explicit object, narrower scope
    private final Object lock = new Object();
    public void incrementBlock() {
        int expensive = computeSomethingUnrelated(); // not protected –
↳ doesn't need to be
        synchronized (lock) {
            count++; // only the truly shared part is inside the lock
        }
    }
}

```

```

    private int computeSomethingUnrelated() { return 1; }

    public synchronized int getCount() {
        return count; // reads must be synchronized too, or you can see a
        ↪ stale value
    }
}

```

What object gets locked?

Declaration	Lock acquired
synchronized void method()	the instance (this)
static synchronized void method()	the Class object (one lock shared by <i>all</i> instances)
synchronized (obj) { ... }	whatever object obj refers to

Mixing an instance-level lock with a static-level lock is a classic bug: two threads think they're synchronizing on “the same thing” but are actually holding two different locks, so no mutual exclusion happens at all.

Reentrancy. Intrinsic locks are **reentrant**: a thread that already holds a lock can acquire it again (e.g., calling another synchronized method on the same object from inside a synchronized method) without blocking itself. The JVM tracks a hold count per thread.

```

    public synchronized void outer() {
        inner(); // same thread re-acquiring the same lock – allowed, does not
        ↪ deadlock
    }
    public synchronized void inner() {
        // ...
    }
}

```

wait() / notify() / notifyAll() — and the mandatory while loop. These three methods coordinate threads around a condition, and must be called while holding the object's monitor. `wait()` releases the lock and parks the thread; `notify()` wakes *one* waiting thread; `notifyAll()` wakes *all* of them. Always check the condition in a while loop, never an `if`:

```

public class BoundedBuffer<T> {
    private final Queue<T> queue = new LinkedList<>();
    private final int capacity;

    public BoundedBuffer(int capacity) { this.capacity = capacity; }

    public synchronized void put(T item) throws InterruptedException {
        while (queue.size() == capacity) { // WHILE, not if – re-check
        ↪ after waking up
            wait(); // releases the lock, parks this thread
        }
        queue.add(item);
        notifyAll(); // wake up any waiting consumers
    }

    public synchronized T take() throws InterruptedException {
        while (queue.isEmpty()) { // WHILE – guard condition must be
        ↪ re-checked
            wait();
        }
        T item = queue.remove();
        notifyAll(); // wake up any waiting producers
        return item;
    }
}

```

Why while, not if:

- **Spurious wakeups.** The JVM spec permits `wait()` to return without any `notify()` ever being called. Rare, but legal — so the condition must be re-checked regardless.
- **Multiple waiters.** With `notifyAll()`, several threads wake up but only one may get to act before the condition becomes false again (e.g., two consumers wake up but only one item was added). Each thread must re-verify before proceeding.

Lost wakeup. This happens when a thread calls `notify()`/`notifyAll()` *before* another thread starts waiting, and the signal is lost forever because there was nobody listening yet.

```

// BROKEN: checking the condition outside the lock creates a window for a
↪ lost wakeup
public void putBroken(T item) throws InterruptedException {
    if (isFull()) { // check happens OUTSIDE the lock
        synchronized (this) {

```



```

        wait();           // another thread could notify() in between
    ↪ check and wait()
        }                 // - that notify is lost, and this thread
    ↪ waits forever
        }
        // ...
    }

```

The fix is what `BoundedBuffer` already does above: the `check` and the `wait()` call happen **inside the same synchronized block**, so no notification can slip through the gap.

Lock granularity. A single, coarse lock around an entire large object is simple and safe, but it forces every operation to queue up behind every other, even unrelated ones — that's a **throughput** cost. Splitting into several fine-grained locks (e.g., one lock per shard/bucket) improves parallelism but is more complex and raises the risk of deadlock when a thread needs more than one lock at a time.

Deadlock. Two (or more) threads each hold a lock the other needs, and neither can proceed. Classic two-lock example:

```

public class DeadlockDemo {
    private static final Object lockA = new Object();
    private static final Object lockB = new Object();

    public static void main(String[] args) {
        Thread t1 = new Thread(() -> {
            synchronized (lockA) {
                sleepQuiet(50);
                synchronized (lockB) { System.out.println("t1 got both");
            }
        });
        Thread t2 = new Thread(() -> {
            synchronized (lockB) {           // t2 grabs B first...
                sleepQuiet(50);
                synchronized (lockA) { System.out.println("t2 got both");
            } // ...then wants A
        });
        t1.start();
        t2.start();
        // t1 holds A, waits for B. t2 holds B, waits for A. Neither ever
    ↪ finishes.
    }

```

```
private static void sleepQuiet(long ms) {
    try { Thread.sleep(ms); } catch (InterruptedException ignored) {}
}
```

The fix: consistent lock ordering. If every thread in the system always acquires locks in the same global order, a cycle can never form:

```
// FIXED: every thread acquires lockA before lockB, no exceptions.
Thread t1 = new Thread(() -> {
    synchronized (lockA) {
        synchronized (lockB) { System.out.println("t1 got both"); }
    }
});
Thread t2 = new Thread(() -> {
    synchronized (lockA) { // t2 also takes A first now, matching t1's
        ↪ order
        synchronized (lockB) { System.out.println("t2 got both"); }
    }
});
```

Other standard deadlock fixes: use `tryLock` with a timeout instead of a blocking acquire (covered under `ReentrantLock`), or acquire all needed locks up front and back out entirely if any single one isn't available.

Livelock. Threads aren't blocked — they're both actively running — but they keep responding to each other in a way that stops either from making progress. A common example is two threads that each politely “back off” and retry when they detect contention, but they back off in lockstep forever and never actually get through.

Starvation. A thread is perpetually denied access to a resource because other threads keep getting priority — e.g., low-priority threads never running because high-priority ones keep flooding the scheduler, or a “fair” resource being repeatedly grabbed by newcomers ahead of a long-waiting thread. Using a **fair lock** (see `ReentrantLock` below) is one direct mitigation.

Intrinsic Locks

Every Java object has a built-in lock, historically called its **monitor** or **intrinsic lock**. It's what `synchronized` uses under the hood — there's no separate “lock object” you create; the object *is* the lock. Only one thread can hold a given object's monitor at a time.

```

public class IntrinsicLockExample {
    private final Object monitor = new Object();
    private int balance = 0;

    public void deposit(int amount) {
        synchronized (monitor) { // acquire monitor's intrinsic lock
            balance += amount;
        } // lock is automatically released here, even if an exception is
        ↪ thrown
    }
}

```

Intrinsic locks have three defining properties, all already demonstrated above:

- **Mutual exclusion** — only one thread inside the block/method at a time.
- **Reentrancy** — the holding thread can re-enter without blocking itself.
- **Automatic release** — released when the block exits, normally or via exception. You never write an explicit “unlock” call, which avoids one entire class of bugs (forgetting to unlock) — but it also means you can’t do things like acquire in one method and release in another, or time out on acquisition. That’s exactly the gap `ReentrantLock` fills.

Intrinsic locks are **not fair** by default — the JVM makes no promise about which waiting thread gets the lock next — and they offer no way to interrupt a thread that’s blocked waiting to enter a `synchronized` block, and no way to check “is this lock free?” without blocking. Those limitations are the whole reason `java.util.concurrent.locks` exists.

ReentrantLock

`ReentrantLock` is an explicit, class-based lock with the same reentrant mutual-exclusion guarantee as `synchronized`, plus extra capabilities `synchronized` doesn’t offer: fairness policies, timed/non-blocking acquisition, interruptible acquisition, and multiple wait-conditions per lock.

The mandatory pattern: acquire, then `try/finally` to release. Unlike `synchronized`, nothing releases the lock for you automatically.

```
import java.util.concurrent.locks.ReentrantLock;

public class Account {
    private final ReentrantLock lock = new ReentrantLock();
    private int balance = 0;

    public void withdraw(int amount) {
        lock.lock();
        try {
            if (amount > balance) throw new
↪ IllegalStateException("insufficient funds");
            balance -= amount;
        } finally {
            lock.unlock(); // MUST be in finally – otherwise an exception
↪ leaks the lock forever
        }
    }
}
```

Fairness. The no-arg constructor gives a non-fair lock (higher throughput, but a thread can theoretically be overtaken repeatedly). `new ReentrantLock(true)` gives a fair lock — the longest-waiting thread goes next, which trades some throughput for protection against starvation.

```
ReentrantLock fair = new ReentrantLock(true); // FIFO-ish ordering,
↪ reduces starvation risk
```

tryLock with a timeout. Instead of blocking forever, ask for the lock and give up after a bound — a direct deadlock-avoidance tool:

```
import java.util.concurrent.TimeUnit;

public boolean tryWithdraw(int amount) throws InterruptedException {
    if (!lock.tryLock(200, TimeUnit.MILLISECONDS)) {
        return false; // couldn't get the lock in time – back off instead
↪ of deadlocking
    }
    try {
        balance -= amount;
        return true;
    } finally {
        lock.unlock();
    }
}
```

`lockInterruptibly()`. Acquire the lock, but allow this thread to be interrupted while it's waiting — useful when a blocked-on-lock thread still needs to respond to cancellation:

```
public void withdrawOrCancel(int amount) throws InterruptedException {
    lock.lockInterruptibly(); // throws InterruptedException instead of
    ↪ blocking forever
    try {
        balance -= amount;
    } finally {
        lock.unlock();
    }
}
```

Condition — like wait/notify, but per-lock and multiple per lock. `synchronized` gives every object exactly one implicit wait-set. A `ReentrantLock` can have several independent `Conditions`, which is handy for producer/consumer designs where “buffer is full” and “buffer is empty” are logically distinct waits:

```
import java.util.concurrent.locks.*;
import java.util.*;

public class BoundedBufferWithLock<T> {
    private final ReentrantLock lock = new ReentrantLock();
    private final Condition notFull = lock.newCondition();
    private final Condition notEmpty = lock.newCondition();
    private final Queue<T> queue = new LinkedList<>();
    private final int capacity;

    public BoundedBufferWithLock(int capacity) { this.capacity =
    ↪ capacity; }

    public void put(T item) throws InterruptedException {
        lock.lock();
        try {
            while (queue.size() == capacity) notFull.await(); // like
            ↪ wait(), but on this Condition
            queue.add(item);
            notEmpty.signal(); // like notify(), but only wakes threads
            ↪ waiting on notEmpty
        } finally {
            lock.unlock();
        }
    }

    public T take() throws InterruptedException {
```

```

        lock.lock();
        try {
            while (queue.isEmpty()) notEmpty.await();
            T item = queue.remove();
            notFull.signal();
            return item;
        } finally {
            lock.unlock();
        }
    }
}

```

The same while-loop rule from intrinsic locks applies here too: always re-check the condition after `await()` returns.

ReadWriteLock

Many workloads are read-heavy: lots of threads just want to read shared state, and only occasionally does a thread need to write it. A plain mutual-exclusion lock forces even simultaneous *reads* to queue up one at a time, which wastes parallelism. `ReadWriteLock` splits the lock into two: any number of readers can hold the read lock at once, but a writer needs exclusive access — no readers and no other writer at the same time.

```

import java.util.concurrent.locks.*;
import java.util.*;

public class Cache<K, V> {
    private final Map<K, V> map = new HashMap<>();
    private final ReadWriteLock rwLock = new ReentrantReadWriteLock();
    private final Lock readLock = rwLock.readLock();
    private final Lock writeLock = rwLock.writeLock();

    public V get(K key) {
        readLock.lock(); // many threads can hold this simultaneously
        try {
            return map.get(key);
        } finally {
            readLock.unlock();
        }
    }

    public void put(K key, V value) {

```

```

        writeLock.lock(); // exclusive: blocks all readers and other
↪ writers
        try {
            map.put(key, value);
        } finally {
            writeLock.unlock();
        }
    }
}

```

When it helps: read-mostly data structures — caches, configuration snapshots, lookup tables — where writes are rare and reads vastly outnumber them. **When it doesn't:** if writes are frequent, or reads are extremely short (a few nanoseconds of work), the book-keeping overhead of tracking readers/writers can cost more than the parallelism gains — a plain `ReentrantLock` or even `synchronized` may be faster in practice. Always measure before reaching for this.

StampedLock

`StampedLock` (Java 8+) is a further evolution: it supports the same read/write split as `ReadWriteLock`, plus a third mode — **optimistic reads** — that don't block writers at all and cost almost nothing when there's no contention. Instead of an object-based lock, it hands back a long **stamp** that you must validate afterward.

```

import java.util.concurrent.locks.StampedLock;

public class Point {
    private final StampedLock stampedLock = new StampedLock();
    private double x, y;

    public void move(double dx, double dy) {
        long stamp = stampedLock.writeLock(); // exclusive, like a normal
↪ write lock
        try {
            x += dx;
            y += dy;
        } finally {
            stampedLock.unlockWrite(stamp);
        }
    }

    public double distanceFromOrigin() {

```

```

        long stamp = stampedLock.tryOptimisticRead(); // does NOT block,
        ↪ does NOT take a real lock
        double currentX = x, currentY = y;           // read the fields
        ↪ hopefully
        if (!stampedLock.validate(stamp)) {           // did a writer
        ↪ sneak in while we read?
            stamp = stampedLock.readLock();           // fall back to a
        ↪ real, blocking read lock
            try {
                currentX = x;
                currentY = y;
            } finally {
                stampedLock.unlockRead(stamp);
            }
        }
        return Math.sqrt(currentX * currentX + currentY * currentY);
    }
}

```

The optimistic-read pattern is always: **grab a stamp** ✕ **read the data** ✕ **validate the stamp** ✕ **if invalid, retry with a real lock**. Skipping the validation step is a serious bug — it silently allows reading data that a writer changed mid-read.

Critical gotcha: StampedLock is NOT reentrant. Unlike ReentrantLock and intrinsic locks, calling `writeLock()` again from the same thread that already holds it will simply deadlock the thread against itself:

```

    long stamp = stampedLock.writeLock();
    try {
        // BUG: if this method (directly or indirectly) tries to acquire
        // stampedLock.writeLock() again on the SAME thread, it blocks forever
        ↪ —
        // StampedLock does not track "who already owns this."
        doSomethingThatAlsoLocks();
    } finally {
        stampedLock.unlockWrite(stamp);
    }
}

```

Because of this, be very careful using `StampedLock` in any code path that might recurse or call back into itself. It also has no `Condition` support and doesn't play with synchronized-style monitor semantics — it's a specialized tool for exactly the read-heavy, low-contention scenario it's built for.

Volatile

`volatile` is a field modifier that gives two guarantees, and only two: **visibility** and a limited form of **ordering**. It does **not** give atomicity.

- **Visibility:** a write to a `volatile` field by one thread is guaranteed to be immediately visible to any thread that subsequently reads it. Without `volatile`, a reading thread might keep seeing a stale, cached value indefinitely (in theory — real JITs/CPU's vary, but the JMM makes no promise otherwise).
- **Ordering:** the compiler and CPU cannot reorder other instructions across a `volatile` read/write in ways that would break the visible sequence (this feeds directly into the happens-before rules below).
- **What it does NOT give you: atomicity.** `count++` on a `volatile int` is still read-modify-write — three separate steps — and two threads can interleave those steps and lose an update, exactly like a non-volatile field.

```
public class VolatileIsNotAtomic {
    private volatile int count = 0;

    public void increment() {
        count++; // READS count, ADDS 1, WRITES count — three steps, not
        ↪ one
    }
}

// With 2 threads each calling increment() 100,000 times, the final count
// is very likely LESS than 200,000 — some increments are silently lost,
// even though 'volatile' is present.
```

The classic correct use case: a stop flag.

```
public class StopFlagExample {
    private volatile boolean running = true; // visibility is ALL this
    ↪ needs

    public void doWork() {
        while (running) {
            // do a unit of work
        }
        System.out.println("Stopped cleanly");
    }

    public void stop() {
```

```

        running = false; // guaranteed to become visible to the worker
↪ thread promptly
    }
}

```

This works with plain `volatile` because it's a single write and a single read of one variable — no read-modify-write, so no lost-update risk. Swap `volatile boolean` for a plain `boolean` here and the worker thread might loop forever, never observing the change (a real, documented class of bug).

Double-checked locking, done right. A common (broken) attempt at lazy, thread-safe singleton initialization:

```

// BROKEN: without 'volatile', another thread can see a half-constructed
↪ object
public class BrokenSingleton {
    private static BrokenSingleton instance;

    public static BrokenSingleton getInstance() {
        if (instance == null) { // first check, no lock –
↪ fast path
            synchronized (BrokenSingleton.class) {
                if (instance == null) { // second check, with
↪ lock
                    instance = new BrokenSingleton(); // NOT guaranteed
↪ fully-constructed
                    // before this reference becomes visible to other
↪ threads without 'volatile'
                }
            }
        }
        return instance;
    }
}

```

The problem: `new BrokenSingleton()` isn't a single atomic step from the JMM's point of view — it involves allocating memory, running the constructor, and assigning the reference. Without `volatile`, those steps can appear reordered to another thread, which could see a non-null `instance` reference that points to a not-yet-fully-initialized object.

```

// FIXED: 'volatile' prevents the reordering and publishes a fully-built
↪ object
public class CorrectSingleton {

```

```

private static volatile CorrectSingleton instance;

public static CorrectSingleton getInstance() {
    if (instance == null) {
        synchronized (CorrectSingleton.class) {
            if (instance == null) {
                instance = new CorrectSingleton();
            }
        }
    }
    return instance;
}
}

```

(In modern Java, the simplest and safest fix for a plain singleton is usually a static holder class, which relies on class-initialization guarantees instead of double-checked locking at all — but double-checked locking remains a fair-game interview topic.)

Atomic Classes

The `java.util.concurrent.atomic` package gives you classes that perform single-variable read-modify-write operations **atomically**, using hardware-level compare-and-swap (CAS) instructions instead of locks. That makes them faster than a lock under light-to-moderate contention, with no risk of deadlock (there's nothing to hold across a blocking call).

```

import java.util.concurrent.atomic.*;

public class AtomicBasics {
    public static void main(String[] args) {
        AtomicInteger counter = new AtomicInteger(0);
        counter.incrementAndGet();           // atomic ++counter, returns
        ↪ new value
        counter.getAndIncrement();           // atomic counter++, returns
        ↪ old value
        counter.addAndGet(5);                // atomic counter += 5

        AtomicLong bigCounter = new AtomicLong(0L);
        bigCounter.incrementAndGet();

        AtomicReference<String> ref = new AtomicReference<>("initial");
        boolean swapped = ref.compareAndSet("initial", "updated"); // CAS:
        ↪ only if still "initial"
        System.out.println(swapped);         // true
    }
}

```

```

        System.out.println(ref.get());           // "updated"

        // updateAndGet: apply a function atomically, retrying under
        ↪ contention
        AtomicInteger price = new AtomicInteger(100);
        price.updateAndGet(current -> current + 10); // atomic "price +=
        ↪ 10", CAS-retry loop internally
    }
}

```

compareAndSet (CAS) is the primitive underneath almost everything here: “set this value to X, but only if it’s still currently Y.” If another thread changed it in between, the CAS fails and the caller (or the atomic class internally, for methods like `updateAndGet`) retries.

The ABA problem. CAS only checks “is the value still what I last saw?” — it can’t tell if the value was changed to something else and then changed *back* to the original value while you weren’t looking. If your logic assumes “unchanged” means “nothing happened in between,” ABA breaks that assumption.

```

// Thread 1 reads reference, sees "A", plans to CAS "A" -> "C"
// Thread 2 (in between) CAS "A" -> "B", then CAS "B" -> "A"
// Thread 1's CAS("A", "C") now succeeds – it thinks nothing changed,
// but state actually went A -> B -> A, and any side effects tied to
// that B intermediate state have already happened invisibly.
AtomicReference<String> state = new AtomicReference<>("A");
// state.compareAndSet("A", "C") succeeds even though "B" happened in
↪ between.

```

Mitigation: use a version/stamp alongside the value (`AtomicStampedReference`) so a CAS also checks that the “version” hasn’t moved, not just the value.

LongAdder vs AtomicLong under contention. `AtomicLong` funnels every thread’s CAS attempt at the *same single memory location* — under heavy contention (many threads incrementing constantly), that creates a hot spot where CAS attempts keep failing and retrying. `LongAdder` (Java 8+) spreads the count across multiple internal cells and only combines them when you call `sum()`, dramatically reducing contention at the cost of `sum()` being an eventually-consistent snapshot rather than a single atomic memory location.

```

import java.util.concurrent.atomic.LongAdder;

LongAdder hits = new LongAdder();

```

```
// Many threads, high contention:
hits.increment(); // internally spreads updates across cells – much less
↳ CAS contention
long total = hits.sum(); // combines cells; fine for metrics/counters, not
↳ for tight CAS logic
```

Rule of thumb: use `AtomicLong` when you need the exact current value read consistently (e.g., you're going to `compareAndSet` against it). Use `LongAdder` for pure counters/metrics under high contention where you only need the total occasionally.

AtomicIntegerFieldUpdater (briefly). Lets you get atomic CAS-style updates on a plain `volatile int` field of an *existing* class, without changing its declared type to `AtomicInteger` — useful when you can't change a field's type (e.g., it's part of a serialized format or an inherited class) but still need atomic updates.

```
import java.util.concurrent.atomic.AtomicIntegerFieldUpdater;

class Node {
    volatile int visits = 0; // must be volatile, not private, and not
↳ final
}

AtomicIntegerFieldUpdater<Node> updater =
    AtomicIntegerFieldUpdater.newUpdater(Node.class, "visits");
Node node = new Node();
updater.incrementAndGet(node); // atomic increment on node.visits without
↳ an AtomicInteger field
```

ThreadLocal

`ThreadLocal<T>` gives each thread its own independent copy of a variable. Two threads reading/writing the “same” `ThreadLocal` never see each other's values — it's per-thread storage, not shared state, so it needs no locking at all.

```
public class ThreadLocalBasics {
    private static final ThreadLocal<Integer> requestId =
        ThreadLocal.withInitial(() -> 0);

    public static void main(String[] args) throws InterruptedException {
        Runnable task = () -> {
            requestId.set((int) (Math.random() * 1000));
            System.out.println(Thread.currentThread().getName() + " sees
↳ " + requestId.get());
        };
    }
}
```

```

    };
    Thread t1 = new Thread(task, "t1");
    Thread t2 = new Thread(task, "t2");
    t1.start(); t2.start();
    t1.join(); t2.join();
    // t1 and t2 print different values – each has its own independent
↪ slot
    }
}

```

Common use cases: per-request context (user id, transaction id, locale) in web servers, SimpleDateFormat/DecimalFormat instances (historically not thread-safe, so one-per-thread avoided sharing), database connections or transaction handles tied to “whatever thread is currently servicing this request.”

remove() in thread pools, or you leak. A ThreadLocal’s value is stored in a map owned by the Thread object itself. In a thread pool, threads are *reused* across many tasks/requests — if you never call remove(), the value from request #1 is still sitting there when the same pooled thread later handles request #47, which is both a memory leak (values pile up and are never GC’d while the thread lives) and a correctness bug (stale data leaking across unrelated requests).

```

// BROKEN in a pooled environment: value from a previous task can leak
↪ into the next one
private static final ThreadLocal<User> currentUser = new ThreadLocal<>();

void handleRequest(User user) {
    currentUser.set(user);
    process();
    // no cleanup – the next task run on this SAME pooled thread inherits
↪ 'user' by accident
}

```

```

// FIXED: always clean up, in a finally block, regardless of success or
↪ failure
void handleRequest(User user) {
    currentUser.set(user);
    try {
        process();
    } finally {
        currentUser.remove(); // returns the thread's slot to its initial,
↪ empty state
    }
}

```

InheritableThreadLocal. A variant that copies the parent thread's value into any *child* thread it creates at the moment the child is constructed (via an overridable `childValue()` hook). Useful for propagating context (like a trace id) into worker threads spawned by a request handler — but it only captures a snapshot at thread-creation time, and it doesn't work at all with pooled worker threads that were created long before the current task existed.

```
public class InheritableExample {
    static final InheritableThreadLocal<String> traceId = new
    ↪ InheritableThreadLocal<>();

    public static void main(String[] args) throws InterruptedException {
        traceId.set("trace-123");
        Thread child = new Thread(() -> {
            System.out.println(traceId.get()); // prints "trace-123" —
            ↪ inherited at creation time
        });
        child.start();
        child.join();
    }
}
```

Looking ahead. Java 21 introduced (as a preview, stabilizing later) **Scoped Values** as a safer, immutable alternative to `ThreadLocal` for propagating context — especially well-suited to virtual threads and structured concurrency, where `ThreadLocal`'s mutability and lifecycle can be awkward. Scoped Values are covered in Chapter 15.

Happens-Before Relationship

The **Java Memory Model (JMM)** defines exactly when one thread's actions are *guaranteed* to be visible, in order, to another thread. That guarantee is called **happens-before**: if action A happens-before action B, every effect of A (every write to memory) is visible to whatever executes B. Without an explicit happens-before edge between two threads' operations, the JMM makes **no promise at all** about ordering or visibility — the reordering and staleness bugs seen earlier in this chapter are exactly what happens in that gap.

The core rules:

- 1. Program order rule.** Within a single thread, each action happens-before every subsequent action in that thread's own code, in the order the code specifies (even though the JIT/CPU may reorder instructions *invisibly* to that thread — the thread itself always observes its own actions in order).

```

int a = 1;
int b = 2;
int c = a + b; // program order guarantees 'a' and 'b' are both assigned
↳ before this line runs

```

2. Monitor lock rule. Unlocking a monitor happens-before every subsequent lock of that *same* monitor by any thread. This is what makes synchronized work: everything a thread wrote before releasing a lock is visible to the next thread that acquires that same lock.

```

private final Object lock = new Object();
private int sharedValue = 0;

void writer() {
    synchronized (lock) {
        sharedValue = 42; // write happens before this unlock
    } // unlock here
}

void reader() {
    synchronized (lock) { // this lock happens-after the writer's unlock
        System.out.println(sharedValue); // guaranteed to see 42, not 0
    }
}

```

3. Volatile variable rule. A write to a `volatile` field happens-before every subsequent read of that *same* field by any thread. This is exactly why the stop-flag example above is correct.

```

private volatile boolean ready = false;
private int data = 0;

void writer() {
    data = 100;           // (a) plain write
    ready = true;         // (b) volatile write – happens-before any later
↳ read of 'ready'
}

void reader() {
    if (ready) {           // (c) volatile read
        System.out.println(data); // guaranteed to see 100, NOT 0 –
        // because (a) happens-before (b)
    }
↳ [program order],
}                               // and (b) happens-before (c) [volatile
↳ rule],

```



```

// so by transitivity (a)
↪ happens-before (c)

```

4. Thread start/join rules. A call to `Thread.start()` happens-before any action inside the started thread's `run()`. And every action inside a thread happens-before another thread's successful `join()` on it.

```

int[] result = new int[1];
Thread worker = new Thread(() -> result[0] = 99); // write happens inside
↪ worker's run()
worker.start(); // happens-before every action inside worker.run()
worker.join(); // worker's actions happen-before this returning
System.out.println(result[0]); // guaranteed to see 99, safely, no lock or
↪ volatile needed

```

5. Final field rule. If an object is properly constructed (its constructor doesn't leak `this` before finishing), the values of its `final` fields are guaranteed visible to any thread that gets a reference to that object after construction — no synchronization needed for those fields specifically.

```

public final class ImmutablePoint {
    private final int x, y; // final fields
    public ImmutablePoint(int x, int y) { this.x = x; this.y = y; }
    public int getX() { return x; }
    public int getY() { return y; }
}
// Any thread that receives a reference to a fully-constructed
↪ ImmutablePoint
// is guaranteed to see the correct x and y, even with zero
↪ synchronization –
// as long as the reference wasn't published before the constructor
↪ finished.

```

6. Transitivity. If A happens-before B, and B happens-before C, then A happens-before C — even if A and C are actions on different threads that never directly synchronize with each other. This is what let the volatile example above chain a plain write through a volatile write/read pair into a guaranteed-visible result.

Visibility, Atomicity, and Ordering

Nearly every concurrency bug is one (or more) of exactly three failures: **visibility** (a thread doesn't see another thread's latest write), **atomicity** (an operation that looks like

one step is actually several, and gets interrupted mid-way), and **ordering** (instructions execute or become visible in a different sequence than the source code implies). Below: three broken examples, each fixed three different ways, with trade-offs.

Broken counter (atomicity failure)

```
// BROKEN: count++ is read-modify-write, not atomic – updates get lost
↳ under contention
public class BrokenCounter {
    private int count = 0;
    public void increment() { count++; }
    public int get() { return count; }
}
// 10 threads x 100,000 increments each: final count is reliably LESS than
↳ 1,000,000
```

Fix 1 — synchronized:

```
public class SynchronizedCounter {
    private int count = 0;
    public synchronized void increment() { count++; }
    public synchronized int get() { return count; }
}
```

Fix 2 — volatile alone does NOT work here (shown earlier) — you’d need `volatile` *plus* some other mechanism to make the read-modify-write atomic, so `volatile` by itself is not a valid fix for a counter.

Fix 3 — atomic class:

```
import java.util.concurrent.atomic.AtomicInteger;

public class AtomicCounter {
    private final AtomicInteger count = new AtomicInteger(0);
    public void increment() { count.incrementAndGet(); }
    public int get() { return count.get(); }
}
```

Broken flag (visibility failure)

```
// BROKEN: no visibility guarantee – the reading thread may cache
↳ 'running' forever
public class BrokenFlag {
    private boolean running = true;
    public void stop() { running = false; }
    public void run() {
        while (running) { /* busy loop, may never see 'false' */ }
    }
}
```

Fix 1 — synchronized (works, but heavier than needed for a single boolean):

```
public class SynchronizedFlag {
    private boolean running = true;
    public synchronized void stop() { running = false; }
    public synchronized boolean isRunning() { return running; }
    public void run() {
        while (isRunning()) { /* every check takes a lock */ }
    }
}
```

Fix 2 — volatile (the idiomatic fix — single variable, single reads/writes, no atomicity needed):

```
public class VolatileFlag {
    private volatile boolean running = true;
    public void stop() { running = false; }
    public void run() {
        while (running) { /* cheap, correct, no locking */ }
    }
}
```

Fix 3 — atomic class (works, but overkill — you're not doing CAS logic on a plain flag):

```
import java.util.concurrent.atomic.AtomicBoolean;

public class AtomicFlag {
    private final AtomicBoolean running = new AtomicBoolean(true);
    public void stop() { running.set(false); }
    public void run() {
        while (running.get()) { /* correct, but heavier machinery than
↳ needed */ }
    }
}
```

```
    }
}
```

Broken publication (reordering failure)

```
// BROKEN: reader thread might see 'configured == true' before seeing the
↪ fully-set config fields,
// because there's no happens-before edge forcing the writes to become
↪ visible in order.
public class BrokenPublication {
    private int timeout;
    private String host;
    private boolean configured = false;

    public void configure(int timeout, String host) {
        this.timeout = timeout; // (1)
        this.host = host;       // (2)
        this.configured = true; // (3) - nothing stops (3) being seen
↪ before (1)/(2) by another thread
    }

    public void useConfig() {
        if (configured) {
            System.out.println(host + ":" + timeout); // could print a
↪ stale/partial host or 0 timeout
        }
    }
}
```

Fix 1 — `synchronized` (both methods share a lock, establishing happens-before via the monitor rule):

```
public class SynchronizedPublication {
    private int timeout;
    private String host;
    private boolean configured = false;
    private final Object lock = new Object();

    public void configure(int timeout, String host) {
        synchronized (lock) {
            this.timeout = timeout;
            this.host = host;
            this.configured = true;
        }
    }
}
```

```

    }

    public void useConfig() {
        synchronized (lock) {
            if (configured) System.out.println(host + ":" + timeout); //
            ↪ always consistent
        }
    }
}

```

Fix 2 — `volatile` on the guard flag (relies on the `volatile` happens-before rule to publish everything written before it):

```

public class VolatilePublication {
    private int timeout;
    private String host;
    private volatile boolean configured = false;

    public void configure(int timeout, String host) {
        this.timeout = timeout;
        this.host = host;
        this.configured = true; // volatile write: everything above is now
        ↪ visible to any reader
    }

    public void useConfig() {
        if (configured) { // volatile read
            System.out.println(host + ":" + timeout); // guaranteed
            ↪ correct, by the volatile rule
        }
    }
}

```

Fix 3 — `atomic reference to an immutable snapshot` (publish one object atomically instead of three separate fields):

```

import java.util.concurrent.atomic.AtomicReference;

public class AtomicPublication {
    private record Config(int timeout, String host) {}
    private final AtomicReference<Config> config = new
    ↪ AtomicReference<>();

    public void configure(int timeout, String host) {

```

```
        config.set(new Config(timeout, host)); // single atomic reference
↪ swap, fully-built object
    }

    public void useConfig() {
        Config c = config.get();
        if (c != null) System.out.println(c.host() + ":" + c.timeout());
↪ // always consistent
    }
}
```

Trade-offs at a glance

Approach	Visibility	Atomicity	Ordering	Performance	Best for
synchronized	Yes (via monitor rule)	Yes (whole block)	Yes	Lock overhead; blocks other threads	Multi-field invariants, compound operations
volatile	Yes	No (single read/write only)	Yes (for that field)	Very cheap, no blocking	Single flags, single published references
Atomic classes (Atomic*, LongAdder)	Yes	Yes (single variable)	Yes	CAS retry cost under heavy contention; LongAdder reduces this	Counters, single-variable compare-and-update logic
ReentrantLock / ReadWriteLock / StampedLock	Yes	Yes (whole critical section)	Yes	More features than synchronized, similar or better cost	Timed/interruptible acquisition, read-heavy workloads, multiple conditions

Common Code-Review Interview Pitfalls

- 1. **Calling run() instead of start().** Why it matters: run() executes on the calling thread — no new thread, no concurrency, and the bug is easy to miss because the code still “works,” just without any parallelism.

```
// Before
Thread t = new Thread(task);
t.run(); // runs on the CURRENT thread
// After
t.start(); // actually schedules a new thread
```

- 2. Swallowing InterruptedException without restoring the interrupt flag.** Why it matters: catching and ignoring it erases the cancellation signal, so any outer loop or framework relying on interruption to stop the thread never finds out.

```
// Before
try { Thread.sleep(100); } catch (InterruptedException e) { /* ignored
↳ */ }
// After
try { Thread.sleep(100); } catch (InterruptedException e) {
↳ Thread.currentThread().interrupt(); return; }
```

- 3. Using wait()/notify() (or a Condition.await()) with if instead of while.** Why it matters: spurious wakeups and multiple competing waiters mean the condition must be re-checked after waking — an if proceeds on a stale assumption.

```
// Before
if (queue.isEmpty()) wait();
// After
while (queue.isEmpty()) wait();
```

- 4. Checking a condition outside the lock, then locking separately (lost wakeup / TOCTOU).** Why it matters: another thread can change the condition and signal in the gap between the check and the lock, and that signal is lost forever.

```
// Before
if (isEmpty()) { synchronized (this) { wait(); } }
// After
synchronized (this) { while (isEmpty()) wait(); }
```

- 5. Acquiring locks in inconsistent order across different code paths (deadlock risk).** Why it matters: two threads taking the same two locks in opposite order is the textbook deadlock; it often only shows up under real production load, not in tests.

```
// Before: thread A locks(a,b); thread B locks(b,a);
// After: every thread locks in the same global order, e.g. always a
↳ before b
```

- 6. Using ReentrantLock without try/finally.** Why it matters: unlike synchronized, nothing releases the lock automatically — an exception between lock() and unlock() leaks the lock permanently, freezing every other thread that needs it.

```
// Before
lock.lock();
doWork(); // if this throws, the lock is NEVER released
lock.unlock();
// After
lock.lock();
try { doWork(); } finally { lock.unlock(); }
```

7. **Assuming `volatile` makes compound operations like `count++` atomic.** Why it matters: `volatile` only guarantees visibility and ordering for single reads/writes; a read-modify-write is still three separate steps that can interleave across threads.

```
// Before
private volatile int count = 0;
void increment() { count++; } // still lossy under contention
// After
private final AtomicInteger count = new AtomicInteger(0);
void increment() { count.incrementAndGet(); }
```

8. **Double-checked-locking singleton missing `volatile` on the instance field.** Why it matters: without `volatile`, another thread can observe a non-null reference to an object whose constructor hasn't finished running, i.e., a partially-built object.

```
// Before
private static Singleton instance;
// After
private static volatile Singleton instance;
```

9. **Forgetting `remove()` on a `ThreadLocal` used inside a pooled executor.** Why it matters: pooled threads are reused across tasks, so stale values silently leak from one task/request into the next — both a memory leak and a data-isolation bug.

```
// Before
currentUser.set(user);
process();
// After
currentUser.set(user);
try { process(); } finally { currentUser.remove(); }
```

10. **Using `StampedLock` recursively, assuming it behaves like `ReentrantLock`.** Why it matters: `StampedLock` is not reentrant — a thread re-acquiring its own write lock deadlocks against itself, with no exception to warn you.


```
// Before
long s = lock.writeLock();
recursiveCallThatAlsoLocks(); // deadlocks if it calls lock.writeLock()
↳ again on this thread
// After: restructure so the lock is acquired exactly once per call
↳ chain, or use ReentrantLock
```

- 11. Skipping `validate()` after a `StampedLock.tryOptimisticRead()`.** Why it matters: without validation, the “optimistic” read might have raced with a concurrent writer and returned torn or inconsistent data with no error at all.

```
// Before
long stamp = lock.tryOptimisticRead();
double result = x + y; // used without checking if a writer interleaved
// After
long stamp = lock.tryOptimisticRead();
double result = x + y;
if (!lock.validate(stamp)) { /* retry with lock.readLock() */ }
```

- 12. Relying on thread priority for correctness or scheduling guarantees.** Why it matters: priority is only a scheduling hint with JVM/OS-dependent behavior — no ordering or fairness guarantee exists, so “fixing” a race by tweaking priorities is not a real fix.

```
// Before: "the writer thread has higher priority, so it always runs
↳ first"
writerThread.setPriority(Thread.MAX_PRIORITY); // not a synchronization
↳ mechanism
// After: use an actual ordering guarantee — a lock, a latch, or
↳ happens-before via volatile/join
```

- 13. Using `Thread.stop()` or `Thread.suspend()` to cancel work.** Why it matters: both are deprecated for removal because they can kill a thread mid-update, leaving shared state permanently corrupted, or freeze a thread while it holds a lock other threads need.

```
// Before
worker.stop(); // can leave shared state half-updated
// After
volatile boolean cancelled; // or Thread.interrupt() + cooperative
↳ checks
void run() { while (!cancelled) { doUnitOfWork(); } }
```

- 14. Mixing `synchronized(this)` in one method with `synchronized(SomeClass.class)` in another for the same shared state.** Why it matters: these are two different locks, so “synchronizing” both methods provides zero actual mutual exclusion between them.

```
// Before
public synchronized void instanceMethod() { sharedStatic++; } //
↳ locks 'this'
public static synchronized void staticMethod() { sharedStatic++; } //
↳ locks the Class object
// After: pick one lock and use it consistently for all access to
↳ sharedStatic
```

- 15. Holding a lock while doing slow I/O, network calls, or `Thread.sleep()`.** Why it matters: every other thread waiting on that lock is blocked for the full duration of the slow operation, turning a local slowdown into a system-wide bottleneck.

```
// Before
synchronized (lock) { callSlowRemoteService(); updateSharedState(); }
// After
Result r = callSlowRemoteService(); // do the slow part outside the
↳ lock
synchronized (lock) { updateSharedState(r); } // hold the lock only for
↳ the fast part
```

- 16. Publishing a partially-constructed object by leaking `this` from inside a constructor.** Why it matters: this breaks the final-field happens-before guarantee — other threads can see the object before construction finishes, e.g., via a listener registration or a static collection add inside the constructor.

```
// Before
public Widget() { registry.add(this); this.id = computeId(); } //
↳ 'this' escapes before id is set
// After
public Widget() { this.id = computeId(); }
// then, after construction completes: registry.add(widget);
```

- 17. Choosing `AtomicLong` for a hot counter under heavy multi-threaded contention.** Why it matters: every thread’s CAS attempt targets the same memory location, so under high contention most CAS attempts fail and retry, wasting CPU; `LongAdder` spreads updates across cells for far better throughput.

```

// Before
private final AtomicLong hits = new AtomicLong();
void recordHit() { hits.incrementAndGet(); } // hot spot under heavy
↳ contention
// After
private final LongAdder hits = new LongAdder();
void recordHit() { hits.increment(); } // much less contention; sum()
↳ when you need the total

```

- 18. Treating ReadWriteLock/StampedLock as a free performance win without measuring.** Why it matters: for short critical sections or write-heavy workloads, the extra bookkeeping of tracking readers/writers can be slower than a plain synchronized block or ReentrantLock — reviewers should ask “was this actually measured?”

```

// Before: reflexively reaching for ReentrantReadWriteLock on a tiny,
↳ write-heavy map
// After: benchmark synchronized/ReentrantLock vs ReadWriteLock under
↳ the REAL read/write ratio
//           before committing to the more complex API

```


Concurrency (Advanced)

Chapter 13 covered the low-level tools: threads, locks, `volatile`, and atomics. Those are the building blocks, but almost nobody writes raw `Thread` code in production anymore. This chapter covers the higher-level tools built on top of them — thread pools that run tasks for you, composable async pipelines, queues that coordinate producers and consumers, a map built for concurrent access, and synchronizers that make threads wait for each other in specific patterns. These are the tools that actually show up in day-to-day code and in code review. Examples target Java 21+; where a virtual-thread (Chapter 15) note changes the advice, it’s called out.

Executors Framework

Creating a new `Thread` for every unit of work is expensive and unbounded — nothing stops you from creating ten thousand threads and crashing the process. A **thread pool** is a manager that keeps a fixed set of reusable worker threads and hands them tasks from a queue. The **Executors Framework** is Java’s standard toolkit for thread pools.

Three interfaces matter:

Interface	Adds
<code>Executor</code>	Just one method: <code>execute(Runnable)</code> . Fire-and-forget.
<code>ExecutorService</code>	Adds <code>submit()</code> (returns a <code>Future</code>), <code>lifecycle(shutdown())</code> , and batch methods (<code>invokeAll/invokeAny</code>).
<code>ScheduledExecutorService</code>	Adds <code>schedule()</code> , <code>scheduleAtFixedRate()</code> , <code>scheduleWithFixedDelay()</code> — run tasks later or repeatedly.

```
import java.util.concurrent.*;

public class ExecutorBasics {
    public static void main(String[] args) throws InterruptedException {
        ExecutorService pool = Executors.newFixedThreadPool(4);
        pool.execute(() -> System.out.println("running on: " +
        ↪ Thread.currentThread().getName()));
        pool.shutdown();
    }
}
```

Executors factory methods — and why two of them are risky

Factory method	Pool shape	Queue	Risk
<code>newFixedThreadPool(n)</code>	<code>n</code> threads	Unbounded LinkedBlockingQueue	Tasks pile up forever if producers outrun consumers. No backpressure, memory grows until OOM.
<code>newCachedThreadPool()</code>	unbounded max threads	SynchronousQueue (no storage)	No queueing at all — instead it just creates a new thread for every task that arrives while all existing threads are busy. Under load this can spawn thousands of threads, each with its own stack (~512KB–1MB), and the process runs out of memory or the OS runs out of native threads.
<code>newSingleThreadExecutor()</code>	1 thread	Unbounded queue	Same unbounded-queue risk as fixed pool, just with one worker.
<code>newScheduledThreadPool(n)</code>	<code>n</code> threads	Unbounded delayed queue	Fine for scheduling, same queue caveat applies to backlog.
<code>newVirtualThreadPerTaskExecutor()</code> (Java 21)	One virtual thread per task, no pooling	N/A	Different tradeoffs entirely — see Chapter 15.

The common thread: **both risky factories hide an unbounded resource** (queue or thread count) behind a friendly one-line call. In an interview or review, treat `Executors.newFixedThreadPool` and `Executors.newCachedThreadPool` as a yellow flag, not an automatic pass — ask “what happens when load exceeds capacity?”

ScheduledExecutorService

Use this when work needs to run later, or repeatedly, instead of right now:

```
import java.util.concurrent.*;

public class ScheduledJobs {
    public static void main(String[] args) {
        ScheduledExecutorService scheduler =
        ↪ Executors.newScheduledThreadPool(2);

        // run once, after a delay
        scheduler.schedule(() -> System.out.println("ran once after
        ↪ delay"), 5, TimeUnit.SECONDS);

        // run repeatedly, every 10s, measured from the START of each run
        // (if a run takes longer than the period, the next run starts
        ↪ immediately after it finishes)
        scheduler.scheduleAtFixedRate(() -> pollForUpdates(), 0, 10,
        ↪ TimeUnit.SECONDS);

        // run repeatedly, with 10s of rest AFTER each run finishes before
        ↪ the next one starts
        scheduler.scheduleWithFixedDelay(() -> pollForUpdates(), 0, 10,
        ↪ TimeUnit.SECONDS);
    }

    private static void pollForUpdates() {
        System.out.println("polling at " + java.time.Instant.now());
    }
}
```

The difference between `scheduleAtFixedRate` and `scheduleWithFixedDelay` trips people up: fixed-rate measures the interval from the *start* of one run to the *start* of the next (so a slow task can cause back-to-back runs with no gap, or even overlap being queued up), while fixed-delay measures the interval from the *end* of one run to the *start* of the next (always leaves a real gap). If a scheduled task throws an uncaught exception, that task's future scheduled runs **stop silently** — always wrap the body in a try/catch that logs, so one bad run doesn't quietly kill all future polling.

Building a ThreadPoolExecutor by hand

For anything that matters in production, construct a `ThreadPoolExecutor` explicitly so every limit is visible:

```

import java.util.concurrent.*;
import java.util.concurrent.atomic.AtomicInteger;

public class BoundedPoolExample {
    public static void main(String[] args) {
        BlockingQueue<Runnable> boundedQueue = new
↳ ArrayBlockingQueue<>(200); // bounded! not unbounded

        ThreadFactory namedThreads = new ThreadFactory() {
            private final AtomicInteger counter = new AtomicInteger(1);
            @Override
            public Thread newThread(Runnable r) {
                Thread t = new Thread(r, "order-worker-" +
↳ counter.getAndIncrement());
                t.setDaemon(false);
                return t;
            }
        };

        ThreadPoolExecutor executor = new ThreadPoolExecutor(
            4, // core pool size
            8, // max pool size
            60L, TimeUnit.SECONDS, // keep-alive for
↳ extra (non-core) threads
            boundedQueue,
            namedThreads,
            new ThreadPoolExecutor.CallerRunsPolicy() // rejection
↳ handler = backpressure
        );

        executor.execute(() -> System.out.println("handled by " +
↳ Thread.currentThread().getName()));
        executor.shutdown();
    }
}

```

How core size, max size, and the queue interact

This order surprises a lot of people, so it's worth memorizing:

1. If fewer than `corePoolSize` threads exist, **a new thread is created** for the task — even if other core threads happen to be idle.
2. Once `corePoolSize` threads exist, new tasks go into the **queue** instead.
3. Only when the **queue is full** does the pool create additional threads, up to `maximumPoolSize`.

4. If the queue is full **and** `maximumPoolSize` is reached, the task is **rejected** — handled by the `RejectedExecutionHandler`.

This means with an *unbounded* queue, step 3 never triggers — `maximumPoolSize` is dead code, because the queue absorbs everything first. That's exactly why `newFixedThreadPool`'s extra threads (beyond the fixed count, since `core == max` there) never get created, and why `newCachedThreadPool`'s `SynchronousQueue` never queues — it forces step 3 immediately for every task past the core threads (`core` is 0), maximizing thread creation instead.

RejectedExecutionHandler policies

Policy	Behavior
<code>AbortPolicy</code> (default)	Throws <code>RejectedExecutionException</code> .
<code>CallerRunsPolicy</code>	Runs the task on the calling thread instead — naturally throttles the producer since it's now busy running the task instead of submitting more.
<code>DiscardPolicy</code>	Silently drops the task. Dangerous — hides failures.
<code>DiscardOldestPolicy</code>	Drops the oldest queued task, then retries submission.

You can also implement your own `RejectedExecutionHandler` — for example, to log a metric before falling back to `CallerRunsPolicy`.

submit() vs execute() — and the exception trap

```

ExecutorService pool = Executors.newFixedThreadPool(2);

// execute(): fire-and-forget. An uncaught exception goes to the thread's
// UncaughtExceptionHandler (usually prints to stderr) — at least it's
↪ visible.
pool.execute(() -> { throw new RuntimeException("boom (execute)"); });

// submit(): returns a Future. The exception is CAPTURED inside the Future
// and only rethrown (wrapped in ExecutionException) when you call get().
Future<?> future = pool.submit(() -> { throw new RuntimeException("boom
↪ (submit)"); });
// If nobody ever calls future.get(), this exception disappears silently!

try {

```

```

        future.get();
    } catch (ExecutionException e) {
        System.out.println("caught: " + e.getCause()); // this is how you
↪ actually find out
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }

```

Rule of thumb: if you use `submit()`, you must eventually call `get()` (or check `isDone()/isCancelled()`) or handle the exception some other way — otherwise failures vanish.

Two-phase shutdown

`shutdown()` is graceful: it stops accepting new tasks but lets queued and running tasks finish. `shutdownNow()` is aggressive: it interrupts running tasks and returns the list of tasks that were still waiting in the queue, so you can decide what to do with the lost work. The standard idiom combines both:

```

public void shutdownAndAwait(ExecutorService pool) {
    pool.shutdown(); // phase 1: stop accepting new work, let existing
↪ work finish
    try {
        if (!pool.awaitTermination(30, TimeUnit.SECONDS)) {
            pool.shutdownNow(); // phase 2: didn't finish in time, force
↪ it
            if (!pool.awaitTermination(10, TimeUnit.SECONDS)) {
                System.err.println("pool did not terminate");
            }
        }
    } catch (InterruptedException e) {
        pool.shutdownNow(); // this thread was interrupted while waiting;
↪ force shutdown too
        Thread.currentThread().interrupt();
    }
}

```

invokeAll and invokeAny

```

ExecutorService pool = Executors.newFixedThreadPool(3);

List<Callable<Integer>> tasks = List.of(
    () -> { Thread.sleep(100); return 1; },
    () -> { Thread.sleep(50); return 2; },
    () -> { Thread.sleep(200); return 3; }
);

// invokeAll: blocks until ALL tasks finish (or the timeout expires),
↳ returns a Future per task
List<Future<Integer>> results = pool.invokeAll(tasks, 5,
↳ TimeUnit.SECONDS);

// invokeAny: blocks until the FIRST task succeeds, cancels the rest,
↳ returns that value directly
Integer fastest = pool.invokeAny(tasks);

```

ExecutorService as AutoCloseable (Java 19+)

Since JDK19, `ExecutorService` extends `AutoCloseable`. `close()` calls `shutdown()`, waits, and escalates to `shutdownNow()` if needed — so `try-with-resources` gives you the two-phase idiom for free:

```

try (ExecutorService pool = Executors.newFixedThreadPool(4)) {
    pool.submit(() -> System.out.println("work"));
} // close() called automatically: shutdown() + await, escalating to
↳ shutdownNow() if it hangs

```

Sizing a pool: CPU-bound vs IO-bound

- **CPU-bound work** (tight loops, computation, no blocking): size the pool close to the number of CPU cores (`Runtime.getRuntime().availableProcessors()`, maybe +1). More threads than cores just adds context-switch overhead — the CPU is already saturated.
- **IO-bound work** (waiting on network calls, disk, database): threads spend most of their time *blocked*, not computing, so you need many more of them to keep the CPU busy while some threads wait. A common formula:

```
threads = cores * (1 + waitTime / computeTime)
```

If a task waits 90ms for a network call and computes for 10ms, that's a wait/compute ratio of 9, so on an 8-core box you'd want roughly $8 * (1 + 9) = 80$ threads.

Note for Chapter 15: with **virtual threads**, this whole calculation mostly disappears for IO-bound work — you can have millions of cheap virtual threads blocked on I/O without tying up an OS thread each. The pool-sizing math above is still exactly right for CPU-bound work and for platform-thread pools.

ForkJoinPool

A `ForkJoinPool` is a thread pool optimized for **divide-and-conquer** work: split a big task into smaller subtasks, run them in parallel, combine the results. It's the engine behind parallel streams and `CompletableFuture`'s default async methods.

Work stealing

Each worker thread has its own double-ended queue (deque) of tasks. A thread pushes/pops its *own* tasks from one end (like a normal stack, cheap, no contention). When a thread runs out of work, it “steals” a task from the *other end* of some other busy thread's deque. This keeps all cores busy without a central lock that everyone contends on — the key reason `ForkJoinPool` scales better than a naive shared-queue pool for recursive work.

RecursiveTask vs RecursiveAction

Both extend `ForkJoinTask` and require implementing `compute()`. `RecursiveTask<V>` returns a value; `RecursiveAction` returns nothing (pure side effect).

```
import java.util.concurrent.RecursiveTask;
import java.util.concurrent.ForkJoinPool;

public class ParallelSum extends RecursiveTask<Long> {
    private static final int THRESHOLD = 10_000; // tune based on
    ↪ benchmarking
    private final long[] array;
    private final int start, end;

    ParallelSum(long[] array, int start, int end) {
```

```

        this.array = array; this.start = start; this.end = end;
    }

    @Override
    protected Long compute() {
        int length = end - start;
        if (length <= THRESHOLD) {
            long sum = 0;
            for (int i = start; i < end; i++) sum += array[i];
            return sum; // small enough: just compute directly, don't fork
        }
        // further
        int mid = start + length / 2;
        ParallelSum left = new ParallelSum(array, start, mid);
        ParallelSum right = new ParallelSum(array, mid, end);
        left.fork(); // run left asynchronously on another
        // worker
        long rightResult = right.compute(); // compute right on THIS
        // thread (avoid an extra fork)
        long leftResult = left.join(); // wait for left, combine
        return leftResult + rightResult;
    }

    public static void main(String[] args) {
        long[] data = new long[1_000_000];
        java.util.Arrays.fill(data, 1);
        ForkJoinPool pool = new ForkJoinPool(); // defaults to
        // availableProcessors()
        long total = pool.invoke(new ParallelSum(data, 0, data.length));
        System.out.println(total); // 10000000
    }
}

```

Picking the threshold matters. Fork too aggressively (tiny tasks) and the overhead of creating/scheduling tasks dwarfs the actual work — slower than sequential. Fork too little (huge tasks) and you don't get enough parallelism. There's no universal number; a common starting point is “a few thousand basic operations per leaf task,” then benchmark and adjust.

The common pool

`ForkJoinPool.commonPool()` is a shared, JVM-wide pool created lazily on first use, sized by default to `availableProcessors() - 1`. Two big consumers use it *by default*:

- **Parallel streams** (`.parallelStream()`, `.parallel()`).
- **CompletableFuture**'s `*Async` methods, when you don't pass an explicit `Executor`.

Because it's shared, **blocking a common-pool thread hurts everyone** — an unrelated parallel stream elsewhere in the same JVM can stall because there simply aren't enough worker threads left. This is a real production hazard: a slow blocking call buried inside a `.parallelStream()` or a `thenApplyAsync` without an executor can silently degrade throughput app-wide.

You can size the common pool with the system property `-Djava.util.concurrent.ForkJoinPool.common.parallelism=N` at JVM startup, or avoid the shared pool entirely by creating your own new `ForkJoinPool(parallelism)` for work you want isolated from parallel streams and `CompletableFuture` defaults:

```
ForkJoinPool dedicatedPool = new ForkJoinPool(4); // isolated from the
↳ common pool
dedicatedPool.submit(new ParallelSum(data, 0, data.length));
```

ManagedBlocker

If a fork/join task genuinely must block (e.g., waiting on I/O), it should announce that via `ForkJoinPool.ManagedBlocker` so the pool can compensate by spinning up a replacement worker, keeping parallelism intact instead of just losing a thread to the block:

```
import java.util.concurrent.ForkJoinPool;
import java.util.concurrent.locks.Lock;

public class LockBlocker implements ForkJoinPool.ManagedBlocker {
    private final Lock lock;
    private boolean acquired = false;

    LockBlocker(Lock lock) { this.lock = lock; }

    @Override
    public boolean block() {
        if (!acquired) acquired = lock.tryLock();
        return acquired;
    }

    @Override
    public boolean isReleasable() {
```

```

        return acquired || (acquired = lock.tryLock());
    }

    public static void useLock(Lock lock) throws InterruptedException {
        ForkJoinPool.managedBlock(new LockBlocker(lock));
        try {
            // critical section
        } finally {
            lock.unlock();
        }
    }
}

```

CompletableFuture

`CompletableFuture<T>` is a `Future` you can **compose**: chain transformations, combine multiple futures, and react to completion or failure, all without manually calling blocking `get()` everywhere.

Creating one

```

import java.util.concurrent.*;

ExecutorService executor = Executors.newFixedThreadPool(4);

CompletableFuture<String> fromSupplier = CompletableFuture.supplyAsync(
    () -> fetchUserName(42), executor); // runs on the given executor,
↳ returns a value

CompletableFuture<Void> fromRunnable = CompletableFuture.runAsync(
    () -> System.out.println("side effect"), executor); // no return
↳ value

CompletableFuture<String> already =
↳ CompletableFuture.completedFuture("cached"); // no async work at all

```

thenApply vs thenCompose vs thenCombine

```
// thenApply: transform the result, like Optional.map / Stream.map
CompletableFuture<Integer> length =
↳ fromSupplier.thenApply(String::length);

// thenCompose: chain to ANOTHER CompletableFuture-returning step, like
↳ Optional.flatMap.
// Using thenApply here would give you a
↳ CompletableFuture<CompletableFuture<Address>> – wrong shape.
CompletableFuture<Address> address = fromSupplier.thenCompose(name ->
↳ lookupAddressAsync(name));

// thenCombine: combine two INDEPENDENT futures once both are done
CompletableFuture<String> userName = CompletableFuture.supplyAsync(() ->
↳ "Alice");
CompletableFuture<Integer> userAge = CompletableFuture.supplyAsync(() ->
↳ 30);
CompletableFuture<String> combined = userName.thenCombine(userAge,
    (name, age) -> name + " is " + age + " years old");
```

thenAccept / thenRun

```
fromSupplier.thenAccept(name -> System.out.println("got: " + name)); //
↳ consume result, no return value
fromSupplier.thenRun(() -> System.out.println("done")); //
↳ ignore the result entirely
```

The *Async variants — which thread runs what

Every `then*` method has a plain version and an `*Async` version (`thenApplyAsync`, `thenComposeAsync`, ...):

- **Plain** (`thenApply`): the continuation runs on whichever thread completed the *previous* stage. If the previous stage was already complete when you attach the continuation, it may even run on the **calling thread**, synchronously. This is subtle and easy to get wrong in reasoning about which thread does what.
- ***Async without an executor**: runs on `ForkJoinPool.commonPool()`.
- ***Async with an executor**: runs on the executor you pass — the version you want for anything that matters.


```
CompletableFuture.supplyAsync(() -> "raw", executor)
    .thenApplyAsync(String::toUpperCase, executor) // explicit: runs
↳ on our pool, not the common pool
    .thenAccept(System.out::println);
```

allOf / anyOf

```
CompletableFuture<String> f1 = CompletableFuture.supplyAsync(() -> "a",
↳ executor);
CompletableFuture<String> f2 = CompletableFuture.supplyAsync(() -> "b",
↳ executor);
CompletableFuture<String> f3 = CompletableFuture.supplyAsync(() -> "c",
↳ executor);

// allOf: waits for every future; itself completes with Void – you collect
↳ results manually
CompletableFuture<Void> all = CompletableFuture.allOf(f1, f2, f3);
CompletableFuture<List<String>> collected = all.thenApply(v ->
↳ List.of(f1.join(), f2.join(), f3.join()));

// anyOf: completes as soon as ANY one completes; type is Object (loses
↳ specific type info)
CompletableFuture<Object> first = CompletableFuture.anyOf(f1, f2, f3);
```

Error handling: exceptionally / handle / whenComplete

```
CompletableFuture<Integer> risky = CompletableFuture.supplyAsync(() -> {
    if (Math.random() < 0.5) throw new RuntimeException("failed");
    return 42;
}, executor);

// exceptionally: only runs on failure, supplies a fallback VALUE,
↳ swallows the exception
risky.exceptionally(ex -> {
    System.out.println("recovered from: " + ex.getMessage());
    return -1;
});

// handle: runs on EITHER success or failure, gets both (result,
↳ exception) – one is always null
risky.handle((result, ex) -> ex != null ? -1 : result);
```

```
// whenComplete: side effect only, does NOT change the result, and
↳ re-throws the original exception
risky.whenComplete((result, ex) -> {
    if (ex != null) System.out.println("logging failure: " + ex);
    else System.out.println("logging success: " + result);
});
```

Timeouts (Java 9+)

```
CompletableFuture<String> slow = CompletableFuture.supplyAsync(() -> {
    sleepUninterruptibly(5000);
    return "late";
}, executor);

// orTimeout: completes EXCEPTIONALLY with a TimeoutException if not done
↳ in time
slow.orTimeout(1, TimeUnit.SECONDS);

// completeOnTimeout: completes with a FALLBACK VALUE instead of failing
slow.completeOnTimeout("default value", 1, TimeUnit.SECONDS);
```

Always pass an explicit executor

Relying on the default (common pool) mixes your application's async work with parallel streams and any other library using the common pool. It also makes tests slower/flakier (shared global state) and hides real capacity limits. Prefer:

```
CompletableFuture.supplyAsync(this::loadFromDatabase, ioExecutor)
    .thenApplyAsync(this::transform, cpuExecutor)
    .thenAcceptAsync(this::publish, ioExecutor);
```

Why `.join()` inside a callback is dangerous

`.join()` (like `.get()`) **blocks the current thread**. If that current thread is itself a worker thread from a *bounded* pool (the common pool, or any fixed executor), and the future you're joining needs a worker thread from that *same* pool to complete, you can starve or deadlock the pool: all workers end up blocked waiting on each other, and none are left to actually run the work that would unblock them.

```

    // DANGEROUS: runs on the common pool, then blocks a common-pool thread
    ↪ waiting on
    // another common-pool computation. Under load, this can starve the whole
    ↪ pool.
    CompletableFuture.supplyAsync(() -> {
        CompletableFuture<String> inner = CompletableFuture.supplyAsync(() ->
    ↪ fetchRemote());
        return inner.join(); // blocking a pool worker while waiting for
    ↪ another pool worker
    });

    // BETTER: compose instead of blocking – no thread sits idle waiting
    CompletableFuture.supplyAsync(() -> prepareRequest())
        .thenCompose(req -> fetchRemoteAsync(req)); // chained, nobody
    ↪ blocks

```

If you must block, use a separate, appropriately-sized executor dedicated to blocking work — never the common pool.

BlockingQueue

A `BlockingQueue<E>` is a queue that can make the caller **wait**: `put()` waits if the queue is full, `take()` waits if it's empty. It's the standard tool for producer-consumer pipelines.

The family

Implementation	Bounded?	Ordering	Notes
<code>ArrayBlockingQueue</code>	Yes, fixed at creation	FIFO	Backed by an array; capacity can't change.
<code>LinkedBlockingQueue</code>	Optional (default unbounded = <code>Integer.MAX_VALUE</code>)	FIFO	Watch out — “unbounded by default” is a common trap.
<code>SynchronousQueue</code>	Zero capacity	N/A	No storage at all — a <code>put()</code> blocks until a <code>take()</code> is ready to receive it, and vice versa. Pure hand-off.
<code>PriorityBlockingQueue</code>	Unbounded	Priority order (<code>Comparable/Comparator</code>)	<code>put()</code> never blocks (unbounded); <code>take()</code> blocks if empty.

Implementation	Bounded?	Ordering	Notes
DelayQueue	Unbounded	By delay expiry	Elements implement Delayed; take() only returns an element once its delay has elapsed.
LinkedTransferQueue	Bounded	FIFO	Adds transfer(): like put(), but waits until a consumer actually receives the element — combines queue and synchronous hand-off.

put/take vs offer/poll (with timeout) vs add/remove

```

BlockingQueue<String> queue = new ArrayBlockingQueue<>(10);

queue.put("x");           // blocks forever if full
String taken = queue.take(); // blocks forever if empty

boolean added = queue.offer("y", 500, TimeUnit.MILLISECONDS); // waits up
↳ to 500ms, then gives up
String polled = queue.poll(500, TimeUnit.MILLISECONDS);        // same, for
↳ reading

queue.add("z");           // throws IllegalStateException immediately if full
↳ — no waiting
String r = queue.remove(); // throws NoSuchElementException immediately if
↳ empty — no waiting

```

Use put/take when waiting is fine (typical producer-consumer). Use offer/poll with a timeout when you need bounded patience (e.g., don't wait more than a second). Use add/remove only when you've already guaranteed capacity/non-emptiness and truly want a hard failure otherwise — rare in practice.

Producer-consumer example

```

import java.util.concurrent.*;

public class ProducerConsumer {
    private static final String POISON_PILL = "__STOP__";

    public static void main(String[] args) throws InterruptedException {
        BlockingQueue<String> queue = new ArrayBlockingQueue<>(50); //
↳ bounded = backpressure
        ExecutorService pool = Executors.newFixedThreadPool(2);

```

```

        Runnable producer = () -> {
            try {
                for (int i = 0; i < 100; i++) {
                    queue.put("job-" + i); // blocks if consumer falls
                    ↪ behind – natural throttling
                }
                queue.put(POISON_PILL); // tell the consumer to stop
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        };

        Runnable consumer = () -> {
            try {
                while (true) {
                    String job = queue.take();
                    if (job.equals(POISON_PILL)) break; // sentinel: no
                    ↪ more work coming
                    System.out.println("processing " + job);
                }
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        };

        pool.submit(producer);
        pool.submit(consumer);
        pool.shutdown();
        pool.awaitTermination(1, TimeUnit.MINUTES);
    }
}

```

Poison pills are sentinel values that mean “stop, there’s no more work.” With multiple consumers, you typically send one pill per consumer (or use a shared atomic “shutting down” flag) so every consumer thread gets the signal to exit.

Bounded queues are backpressure

A bounded `BlockingQueue` is a **feedback mechanism**. When the queue fills up, `put()` blocks the producer — which slows it down to match the consumer’s real speed. An unbounded queue removes that feedback: the producer never feels resistance, memory quietly grows, and the failure shows up later as an `OutOfMemoryError` far from its actual cause. Choosing a bounded queue is a deliberate design decision to fail fast (or throttle) instead of failing silently later.

ConcurrentHashMap Internals

`ConcurrentHashMap` is a thread-safe map designed for **high concurrency**, not just thread safety. Unlike a synchronized `HashMap` (one lock for everything), it splits work so many threads can operate on different parts of the map at the same time.

No full-table lock

There is no single lock guarding the whole map. Reads generally don't lock at all (volatile reads of table slots). Writes lock only what they need to.

CAS on an empty bin

Inserting into an **empty bin** (bucket) uses a **CAS** (compare-and-swap) — a lock-free atomic operation — instead of taking a lock. This is the fast path: most insertions into a well-sized map hit empty or nearly-empty bins.

Per-bin synchronization

When a bin already has entries (a collision, or an update to an existing key), the map synchronizes on that bin's **first node** only — not the whole table. Two threads working on different bins never contend with each other at all.

Treeification

If a single bin's linked list grows past a threshold (8 nodes, and the table itself has at least 64 slots), that bin is converted into a small **red-black tree**, turning worst-case lookup from $O(n)$ into $O(\log n)$ for that bin. It converts back to a linked list if the bin shrinks below 6 nodes. This is an internal defense against pathological hash collisions (e.g., poor `hashCode()` implementations or deliberately crafted keys) — you never interact with it directly, but it's worth knowing it exists.

`size()` / `mappingCount()` are estimates

Because there's no global lock, there's no way to get a perfectly exact count under concurrent modification without stopping the world. `size()` and `mappingCount()` (the long-returning, Java 8+ preferred alternative) use a scalable striped counter internally and sum it up on demand — accurate as of *some* moment, but another thread could be

mid-insert at the exact time you read it. Treat both as approximate under concurrent writes; don't use them for correctness-critical logic (e.g., "if size == 0 then...").

Atomic compute / computeIfAbsent / merge / putIfAbsent

These perform an atomic read-modify-write for a single key, holding that key's bin lock for the duration of the supplied function:

```
ConcurrentHashMap<String, Integer> counts = new ConcurrentHashMap<>();

// atomic increment – no lost updates even with many threads calling this
↪ concurrently
counts.compute("clicks", (key, current) -> current == null ? 1 : current
↪ + 1);

// only computes the value if the key is absent – common for caches
counts.computeIfAbsent("views", key -> expensiveInitialValue());

// merge: combine an existing value with a new one, or just insert if
↪ absent
counts.merge("clicks", 1, Integer::sum);

// putIfAbsent: insert only if key is not already present, atomically
counts.putIfAbsent("clicks", 0);
```

The recursive-update deadlock hazard

The function you pass to `computeIfAbsent`/`compute`/`merge` runs **while the bin's lock is held**. If that function calls back into the *same map* — especially for a key that could hash to the same bin, or worse, the same key — you can deadlock (the thread tries to re-acquire a lock it already holds in a way that isn't reentrant for this purpose) or, since Java 9+, the map explicitly detects some of these cases and throws `IllegalStateException: Recursive update`.

```
ConcurrentHashMap<String, Integer> map = new ConcurrentHashMap<>();

// DANGEROUS: the remapping function touches the SAME map it's being
↪ called from
map.computeIfAbsent("a", key -> {
    return map.computeIfAbsent("b", k2 -> 1); // may deadlock or throw
↪ IllegalStateException
});
```

```
// SAFER: never call back into the same map from inside
↳ compute/computeIfAbsent/merge.
// Compute the value independently first, then insert.
Integer bValue = map.computeIfAbsent("b", k2 -> 1);
map.computeIfAbsent("a", key -> bValue);
```

Weakly-consistent iterators

Iterators over a `ConcurrentHashMap` (keys, values, entries) are **weakly consistent**: they never throw `ConcurrentModificationException`, they reflect the state of the map at some point at or since the iterator was created, and they're guaranteed not to return the same mapping twice — but they might or might not reflect an update made by another thread mid-iteration. This is different from `HashMap`'s fail-fast iterators, and it's exactly what you want for a map that's actively being written to by other threads while you iterate.

Bulk operations: `forEach` / `search` / `reduce`

```
ConcurrentHashMap<String, Integer> scores = new ConcurrentHashMap<>();
scores.put("alice", 90);
scores.put("bob", 75);
scores.put("carol", 88);

// parallelismThreshold: below this element count, runs sequentially;
↳ above it, may run in parallel
scores.forEach(1, (key, value) -> System.out.println(key + "=" + value));

String highScorer = scores.search(1, (key, value) -> value > 85 ? key :
↳ null);

int total = scores.reduce(1, (key, value) -> value, Integer::sum);
```

`ConcurrentHashMap.newKeySet()`

A quick way to get a thread-safe `Set<E>` backed by a `ConcurrentHashMap<E, Boolean>` internally — simpler than wrapping with `Collections.newSetFromMap`:

```
Set<String> activeSessions = ConcurrentHashMap.newKeySet();
activeSessions.add("session-123");
activeSessions.remove("session-456");
```


CountDownLatch

A `CountDownLatch` lets one or more threads wait until a set of events has happened. It's initialized with a count; each event calls `countDown()`; waiting threads call `await()` and are released once the count hits zero. **It's one-shot** — once it reaches zero, it stays there forever. It cannot be reset or reused.

Typical use: a main thread waits for several background services or worker threads to finish starting up before continuing.

```
import java.util.concurrent.CountDownLatch;

public class ServiceStartup {
    public static void main(String[] args) throws InterruptedException {
        int serviceCount = 3;
        CountDownLatch startupLatch = new CountDownLatch(serviceCount);

        for (int i = 1; i <= serviceCount; i++) {
            int id = i;
            new Thread(() -> {
                initializeService(id);
                System.out.println("service " + id + " ready");
                startupLatch.countDown(); // one fewer thing to wait for
            }).start();
        }

        startupLatch.await(); // blocks until all 3 have called
        ↪ countDown()
        System.out.println("all services ready, starting server");
    }

    private static void initializeService(int id) {
        try { Thread.sleep(100L * id); } catch (InterruptedException e) {
        ↪ Thread.currentThread().interrupt(); }
    }
}
```

CyclicBarrier

A `CyclicBarrier` makes a fixed number of threads (**parties**) all wait at a point until every one of them has arrived — then releases all of them at once. Unlike `CountDownLatch`, it's **reusable**: once everyone passes through, it automatically resets for the next round. It also supports an optional **barrier action** — a `Runnable` that runs once, on one of the arriving threads, right before the barrier releases everyone.

Typical use: a simulation or algorithm that runs in rounds, where every worker thread must finish round N before any of them can start round $N+1$.

```
import java.util.concurrent.BrokenBarrierException;
import java.util.concurrent.CyclicBarrier;

public class SimulationRounds {
    public static void main(String[] args) {
        int workerCount = 4;
        CyclicBarrier barrier = new CyclicBarrier(workerCount,
            () -> System.out.println("--- round complete, advancing
↳ ---")); // barrier action

        for (int i = 1; i <= workerCount; i++) {
            int id = i;
            new Thread(() -> {
                for (int round = 1; round <= 3; round++) {
                    doRoundOfWork(id, round);
                    try {
                        barrier.await(); // wait here until all 4 workers
↳ finish this round
                    } catch (InterruptedException |
↳ BrokenBarrierException e) {
                        Thread.currentThread().interrupt();
                        return;
                    }
                }
            }).start();
        }

        private static void doRoundOfWork(int workerId, int round) {
            System.out.println("worker " + workerId + " doing round " +
↳ round);
        }
    }
}
```

If a thread times out or is interrupted while waiting at the barrier, the barrier becomes **broken**: it throws `BrokenBarrierException` for every other thread currently waiting or that arrives later, and it stays broken until explicitly reset via `barrier.reset()`.

Phaser

A `Phaser` is a more flexible generalization of both `CountDownLatch` and `CyclicBarrier`. The key difference: the number of parties can change dynamically at runtime — threads

can `register()` to join and `arriveAndDeregister()` to leave, even mid-run. Each round is called a **phase**, and phases increment automatically as parties arrive.

Typical use: anything where the set of participants isn't fixed up front — e.g., a pool of workers that grows or shrinks between phases, or a multi-stage pipeline where different stages have different party counts.

```
import java.util.concurrent.Phaser;

public class DynamicWorkers {
    public static void main(String[] args) {
        Phaser phaser = new Phaser(1); // 1 party: the main thread
        ↪ registers itself first

        for (int i = 1; i <= 3; i++) {
            int workerId = i;
            phaser.register(); // dynamically add a party for this new
            ↪ worker

            new Thread(() -> {
                for (int phase = 0; phase < 2; phase++) {
                    System.out.println("worker " + workerId + " working on
                    ↪ phase " + phase);
                    phaser.arriveAndAwaitAdvance(); // arrive, then wait
                    ↪ for the phase to advance
                }
                phaser.arriveAndDeregister(); // this worker is done,
                ↪ shrink the party count
            }).start();
        }

        phaser.arriveAndDeregister(); // main thread's initial
        ↪ registration is done too
    }
}
```

Phaser also supports **tiered** (hierarchical) structures for very large numbers of parties, and you can override `onAdvance(phase, registeredParties)` to control termination — returning `true` from it terminates the phaser after that phase.

Latch vs Barrier vs Phaser

	CountDownLatch	CyclicBarrier	Phaser
Reusable?	No — one-shot	Yes — resets automatically each round	Yes — advances through phases
Party count fixed?	Fixed at creation (the initial count)	Fixed at creation	Dynamic — register/deregister at runtime
Who signals?	Any thread can call <code>countDown()</code> , independent of <code>await()</code>	Only the parties themselves, via <code>await()</code>	Only the parties themselves, via <code>arrive*()</code>
Optional action on completion?	No	Yes, one barrier action	Yes, override <code>onAdvance()</code>
Typical use	Wait for N independent events (startup, N tasks done)	Wait for N <i>same</i> threads to reach a sync point each round	Multi-phase work with a changing number of participants

Semaphore

A Semaphore guards access to a limited number of **permits** — think of it as a bouncer letting only N people into a room at a time. Unlike a lock, a semaphore has **no ownership**: any thread can `release()` a permit, not just the thread that `acquire()`d it. That makes it useful for resource pools where the “returning” thread isn’t necessarily the “borrowing” thread.

Typical use: limiting how many concurrent requests hit a downstream service, or guarding a fixed-size pool of expensive resources (e.g., database connections).

```
import java.util.concurrent.Semaphore;

public class RateLimitedClient {
    private final Semaphore permits = new Semaphore(5); // at most 5
    ↪ concurrent calls

    public String callExternalApi(String request) throws
    ↪ InterruptedException {
        permits.acquire(); // blocks if 5 calls are already in flight
        try {
            return doHttpCall(request);
        } finally {
            permits.release(); // always release, even on exception —
            ↪ hence try/finally
        }
    }
}
```

```

    }

    private String doHttpCall(String request) {
        return "response for " + request;
    }
}

```

`tryAcquire()` (optionally with a timeout) lets you back off instead of blocking indefinitely — useful for “fail fast if the pool is currently exhausted” behavior. A `Semaphore` can also be constructed as **fair** (`new Semaphore(5, true)`), which guarantees FIFO ordering among waiting threads at some throughput cost, versus the default unfair mode which allows barging for better overall throughput.

A **binary semaphore** (1 permit) looks like a mutex but isn’t one: it has no notion of “owner,” so thread A can acquire it and thread B can release it — a flexible but also easy-to-misuse property.

Exchanger

An `Exchanger<V>` is the simplest of these tools: it lets **exactly two threads** meet at a rendezvous point and atomically swap an object. Each thread calls `exchange(myObject)`, which blocks until the other thread also calls `exchange()`, and each returns the *other* thread’s object.

Typical use: a two-stage pipeline where one thread fills a buffer while another drains/processes a different buffer, and they periodically swap so nobody has to wait for a full read+write cycle on a shared buffer.

```

import java.util.concurrent.Exchanger;
import java.util.ArrayList;
import java.util.List;

public class BufferSwap {
    public static void main(String[] args) {
        Exchanger<List<String>> exchanger = new Exchanger<>();

        Thread producer = new Thread(() -> {
            List<String> buffer = new ArrayList<>();
            try {
                while (true) {
                    buffer.add("item-" + System.nanoTime());
                    if (buffer.size() == 10) {

```

```

        buffer = exchanger.exchange(buffer); // hand off
↪ full buffer, get an empty one back
    }
}
} catch (InterruptedException e) {
    Thread.currentThread().interrupt();
}
});

Thread consumer = new Thread() -> {
    List<String> buffer = new ArrayList<>();
    try {
        while (true) {
            buffer = exchanger.exchange(buffer); // hand off empty
↪ buffer, get the full one back
            System.out.println("processing " + buffer.size() + "
↪ items");
            buffer.clear();
        }
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
});

producer.setDaemon(true);
consumer.setDaemon(true);
producer.start();
consumer.start();
}
}

```

Exchanger is intentionally narrow — exactly two parties, one swap at a time. For more than two participants, reach for a `CyclicBarrier`, `Phaser`, or a `BlockingQueue` instead.

Common Code-Review Interview Pitfalls

1. **Reaching for `Executors.newFixedThreadPool/newCachedThreadPool` without checking their queue/thread limits.** Why it matters: a fixed pool's unbounded queue can grow without limit under sustained load, and a cached pool can spawn unbounded threads — both are a slow-motion `OutOfMemoryError` waiting to happen.

```
// Before
ExecutorService pool = Executors.newFixedThreadPool(10); // unbounded
↳ queue behind the scenes
// After
ExecutorService pool = new ThreadPoolExecutor(10, 10, 0L,
↳ TimeUnit.MILLISECONDS,
    new ArrayBlockingQueue<>(500), new
↳ ThreadPoolExecutor.CallerRunsPolicy());
```

- 2. Calling `submit()` and never checking the returned `Future`.** Why it matters: exceptions thrown inside the task are captured in the `Future` and silently disappear unless you call `get()` (or otherwise inspect it) — a failing task looks like a succeeding one.

```
// Before
pool.submit(() -> riskyWork()); // exception vanishes if thrown
// After
Future<?> f = pool.submit(() -> riskyWork());
f.get(); // or handle it in a wrapper/logging layer
```

- 3. Never shutting down an `ExecutorService`.** Why it matters: a pool with non-daemon threads keeps the JVM alive, and a pool that's simply forgotten leaks threads for the life of the application.

```
// Before
ExecutorService pool = Executors.newFixedThreadPool(4);
// ... used once, never shut down
// After
try (ExecutorService pool = Executors.newFixedThreadPool(4)) {
    pool.submit(this::work);
} // auto shutdown via AutoCloseable (Java 19+)
```

- 4. Using `shutdownNow()` without handling the tasks it returns.** Why it matters: `shutdownNow()` cancels running tasks and hands back the still-queued ones — dropping that list silently discards work that may need to be retried or logged.

```
// Before
pool.shutdownNow(); // returned list ignored, queued work is just gone
// After
List<Runnable> abandoned = pool.shutdownNow();
abandoned.forEach(task -> log.warn("dropped task: {}", task));
```

- 5. Sizing a thread pool for IO-bound work the same way as CPU-bound work.** Why it matters: a pool sized to cores for work that mostly waits on network/disk will badly under-utilize the machine — most threads are blocked, not computing.

```
// Before: 8 threads for an HTTP-call-heavy workload on an 8-core box
ExecutorService pool = Executors.newFixedThreadPool(8);
// After: size based on wait/compute ratio, e.g. cores * (1 +
→ waitTime/computeTime)
ExecutorService pool = Executors.newFixedThreadPool(64);
```

- 6. Choosing a ForkJoinTask threshold that's too small.** Why it matters: forking tiny units of work means the bookkeeping overhead (task objects, scheduling, stealing) outweighs the actual computation, making it slower than sequential code.

```
// Before
private static final int THRESHOLD = 2; // forks constantly, dominated
→ by overhead
// After
private static final int THRESHOLD = 10_000; // benchmark and tune from
→ here
```

- 7. Blocking inside a parallel stream or common-pool task without a ManagedBlocker.**

Why it matters: the common pool is shared JVM-wide; a blocking call ties up one of its few worker threads and can starve unrelated parallel streams elsewhere in the app.

```
// Before
list.parallelStream().forEach(item -> blockingHttpCall(item)); //
→ starves the common pool
// After: run blocking work on a dedicated executor, not the common
→ pool
list.forEach(item -> ioExecutor.submit(() -> blockingHttpCall(item)));
```

- 8. Calling .join()/.get() inside a CompletableFuture callback that shares a bounded pool with its dependency.** Why it matters: it can starve or deadlock the pool — all worker threads end up blocked waiting on each other with none left to make progress.

```
// Before
CompletableFuture.supplyAsync(() -> inner().join()); // blocks a
→ shared-pool worker thread
// After
```



```
CompletableFuture.supplyAsync(this::prepare).thenCompose(x ->
↳ innerAsync(x)); // no blocking
```

9. Not passing an explicit Executor to CompletableFuture's async methods.

Why it matters: it silently defaults to the shared common pool, mixing unrelated workloads together and making capacity and failure modes unpredictable.

```
// Before
CompletableFuture.supplyAsync(this::loadData); // runs on
↳ ForkJoinPool.commonPool()
// After
CompletableFuture.supplyAsync(this::loadData, dedicatedExecutor);
```

10. Using an unbounded LinkedBlockingQueue for a producer-consumer pipeline.

Why it matters: new LinkedBlockingQueue<>() defaults to Integer.MAX_VALUE capacity — there is no backpressure, so a slow consumer just lets memory grow until the process dies.

```
// Before
BlockingQueue<Job> queue = new LinkedBlockingQueue<>(); // effectively
↳ unbounded
// After
BlockingQueue<Job> queue = new LinkedBlockingQueue<>(1000); // bounded
↳ = backpressure
```

11. Using add()/remove() on a queue and expecting blocking behavior.

Why it matters: add() throws IllegalStateException immediately if the queue is full (it doesn't wait), and remove() throws NoSuchElementException if empty — surprising if you meant put()/take() or offer()/poll() with a timeout.

```
// Before
queue.add(job); // throws immediately when full, no waiting
// After
queue.put(job); // blocks until space is available
```

12. Calling back into the same ConcurrentHashMap from inside compute/computeIfAbsent/r

Why it matters: the remapping function runs while the bin's lock is held; recursively touching the same map (especially the same key) can deadlock or throw IllegalStateException.

```

// Before
map.computeIfAbsent(key, k -> map.computeIfAbsent(otherKey, k2 -> 1));
→ // risky recursion
// After
int otherValue = map.computeIfAbsent(otherKey, k2 -> 1);
map.computeIfAbsent(key, k -> otherValue);

```

- 13. Treating `ConcurrentHashMap.size()` as an exact value for correctness-critical logic.** Why it matters: under concurrent modification, `size()`/`mappingCount()` are best-effort estimates from striped counters, not a locked, precise snapshot.

```

// Before
if (cache.size() == 0) { /* assume nothing cached yet */ }
// After: don't rely on size for correctness; check
→ isEmpty()/containsKey() for the specific
// guarantee you actually need, and design around the map's atomic
→ per-key operations instead.

```

- 14. Expecting a `CountDownLatch` to reset for a second round.** Why it matters: `CountDownLatch` is one-shot by design — once it hits zero it's done forever; reusing the same instance for a second wave of work silently does nothing (it's already at zero, so `await()` returns instantly without waiting for anyone).

```

// Before
CountDownLatch latch = new CountDownLatch(3);
// ... used for round 1, then reused for round 2 — await() no longer
→ actually waits
// After: use a new CountDownLatch per round, or use
→ CyclicBarrier/Phaser which are reusable
CyclicBarrier barrier = new CyclicBarrier(3);

```

- 15. Leaving a `CyclicBarrier` broken after a timeout or interruption.** Why it matters: once one waiting thread times out or is interrupted, the barrier becomes broken and throws `BrokenBarrierException` for every other party — the barrier must be explicitly `reset()` before it can be used again, and skipping that hangs or fails the next round.

```

// Before: catch BrokenBarrierException and just log it, never reset
→ the barrier
// After
catch (BrokenBarrierException | TimeoutException e) {
    barrier.reset(); // required before the barrier can be reused
}

```

```
}
```

- 16. Releasing a Semaphore permit without a try/finally.** Why it matters: if the guarded code throws, the permit is never released, permanently shrinking the effective pool size — a slow permit leak that eventually blocks everyone.

```
// Before
permits.acquire();
doWork(); // if this throws, release() below never runs
permits.release();
// After
permits.acquire();
try {
    doWork();
} finally {
    permits.release();
}
```

- 17. Ignoring InterruptedException by swallowing it instead of restoring the interrupt status.** Why it matters: swallowing the exception silently (empty catch block) breaks cooperative cancellation — code further up the call stack that checks `Thread.interrupted()` never finds out the thread was asked to stop.

```
// Before
try { queue.take(); } catch (InterruptedException e) { /* ignored */ }
// After
try { queue.take(); } catch (InterruptedException e) {
    ↪ Thread.currentThread().interrupt(); }
```

- 18. Using invokeAny without realizing it cancels the other in-flight tasks.** Why it matters: reviewers sometimes assume all submitted tasks run to completion; `invokeAny` returns as soon as one succeeds and actively cancels the rest, so any side effects those tasks were relying on to complete may never finish.

```
// Before: assuming all 3 "save to replica" tasks fully complete
String result = pool.invokeAny(saveToReplicaTasks); // the other 2 get
↪ cancelled mid-flight
// After: use invokeAll if every task's side effect must complete,
↪ invokeAny only for
// "first successful result wins, and it's fine to abandon the rest"
List<Future<String>> results = pool.invokeAll(saveToReplicaTasks);
```


Modern Concurrency (Java 21+)

Traditional Java concurrency (Chapters 13-14) is built on **platform threads** — thin wrappers around OS threads — and unstructured coordination primitives like `ExecutorService`. That model works, but it has two long-standing pain points: platform threads are too expensive to create “one per request,” and fan-out/fan-in logic built from raw executors and futures is easy to get wrong (leaked tasks, missed cancellation, unclear ownership). Project Loom, delivered across JDK 19-25, attacks both problems: **virtual threads** make blocking, thread-per-request code cheap again, **structured concurrency** gives fan-out/fan-in a well-defined lifetime and error model, and **scoped values** give virtual threads a safer, cheaper alternative to `ThreadLocal`. This chapter targets JDK 21 (where virtual threads became final) through JDK 25 (where structured concurrency’s API was reshaped), and is explicit everywhere about what is final versus preview in which release.

Virtual Threads (Project Loom)

The Thread-Per-Request Problem

The classic Java server model is thread-per-request: accept a connection, hand it a dedicated thread, let that thread block on I/O (database calls, HTTP calls to other services, disk reads) for as long as it needs. This model is simple to write, simple to debug (one thread = one request, so a thread dump tells you exactly what every request is doing), and works naturally with existing blocking APIs like JDBC, `java.io`, and synchronous HTTP clients.

The problem is that **platform threads are expensive**:

- Each platform thread reserves a large stack (commonly 512 KB-1 MB, configurable via `-Xss`), committed as OS memory.

- Creating a platform thread means an OS-level `clone`/`pthread_create` syscall — not free.
- The OS scheduler multiplexes a limited number of platform threads across a limited number of CPU cores; context switches between them are relatively costly.
- Because of the memory and scheduling cost, practical platform thread pools cap out somewhere in the low thousands, not millions.

So under load, thread-per-request architectures hit a wall: to serve more concurrent *blocked* requests (e.g., waiting on a slow downstream service), you need more threads, but you can't create enough of them without exhausting memory or thrashing the scheduler. Teams historically worked around this with either bounded thread pools (which cap throughput and can deadlock under saturation) or asynchronous/reactive programming (`CompletableFuture` chains, reactive streams), which scales far better but fragments the code into callbacks, breaks stack traces, and makes debugging much harder — you lose the “one thread = one request” story.

```
// Classic thread-per-request server thread pool — caps around a few
↳ thousand
// concurrent in-flight requests no matter how much idle (blocked-on-I/O)
↳ time
// each request spends, because platform threads are the scarce resource.
ExecutorService pool = Executors.newFixedThreadPool(200);
for (Socket client : incomingConnections()) {
    pool.submit(() -> handleRequest(client)); // blocks on I/O inside
↳ handleRequest
}
```

What a Virtual Thread Is

A **virtual thread** is a `Thread` (same `java.lang.Thread` API, same `Runnable`/`interrupt`/`join` semantics) whose stack is **not** a fixed-size native OS stack, but a *continuation* stored on the Java heap that can grow and shrink. Virtual threads are scheduled by the JVM, not the OS, using a small pool of platform threads called **carrier threads** (by default, sized to the number of CPU cores, via a `ForkJoinPool` in FIFO mode).

The key idea is **mounting**: when a virtual thread runs, the JVM *mounts* it onto a carrier thread — the carrier thread executes the virtual thread's code for a while. When the virtual thread does something blocking (e.g., a socket read, `Thread.sleep`, acquiring a `ReentrantLock`), the JVM recognizes this and **unmounts** the virtual thread from its carrier: it parks the continuation (saves just the relevant stack frames to the heap) and

frees the carrier thread to mount and run a *different* virtual thread. When the blocking operation completes, the JVM finds a free carrier thread and **remounts** the virtual thread to resume exactly where it left off.

Carrier thread (platform thread)

```

├── mounts virtual-thread-A — runs until A blocks on I/O — unmounts A
├── mounts virtual-thread-B — runs until B blocks on I/O — unmounts B
├── mounts virtual-thread-C — runs to completion ————— unmounts C
└── mounts virtual-thread-A again (I/O finished) — resumes A

```

Because the “expensive” part — the stack — lives on the heap as a lightweight, resizable object rather than as a pre-allocated OS stack, and because there is no OS-level thread creation involved, you can have **millions** of virtual threads outstanding, most of them parked waiting on I/O, backed by only a handful of carrier threads actually consuming CPU.

Crucially, this is invisible to your code: you still write plain, sequential, blocking-style code. `Thread.sleep(...)`, blocking `Socket` reads, JDBC calls, and synchronous `HttpClient.send(...)` all “just work” and internally cooperate with the scheduler to unmount the virtual thread while waiting, instead of tying up an OS thread.

Creating Virtual Threads

```

// 1. Thread.ofVirtual() builder – full control (name, inheritance of
↳ context, etc.)
Thread vt = Thread.ofVirtual()
    .name("order-worker-", 0)
    .start(() -> processOrder(orderId));
vt.join();

// 2. Thread.startVirtualThread – convenience one-liner, equivalent to
// Thread.ofVirtual().start(task)
Thread vt2 = Thread.startVirtualThread(() -> {
    System.out.println("Running on: " + Thread.currentThread());
});

// 3. Executors.newVirtualThreadPerTaskExecutor() – an ExecutorService
↳ that
// starts a brand-new virtual thread for every submitted task. This is
↳ the
// recommended way to adopt virtual threads inside existing
↳ executor-based code.

```

```

try (ExecutorService executor =
↳ Executors.newVirtualThreadPerTaskExecutor()) {
    List<Future<String>> results = new ArrayList<>();
    for (int i = 0; i < 10_000; i++) {
        int id = i;
        results.add(executor.submit(() -> callDownstreamService(id)));
    }
    for (Future<String> f : results) {
        System.out.println(f.get());
    }
} // executor.close() waits for in-flight tasks, same as shutdown +
↳ awaitTermination

```

`Thread.ofPlatform()` is the analogous builder for platform threads, useful when you need to opt back into a real OS thread deliberately (e.g., for a CPU-bound worker, discussed below).

Final in JDK 21 (JEP 444)

Virtual threads shipped as a **preview feature** in JDK 19 (JEP 425) and JDK 20 (JEP 436), requiring `--enable-preview` to compile and run. They became a **final, standard feature in JDK 21** under **JEP 444** — no preview flag needed from JDK 21 onward. This is one of the headline reasons JDK 21 (an LTS release) is treated as the practical starting point for “modern concurrency” in production Java.

When Virtual Threads Help — and When They Don’t

Virtual threads help enormously for **I/O-bound, high-concurrency, blocking-style code**: web servers handling many concurrent requests that spend most of their time waiting on databases, REST calls to other services, message queues, or file/network I/O. The scheduler can run thousands of such threads on a handful of CPU cores because they’re parked (not consuming CPU) most of the time.

Virtual threads do **not** help — and can actively hurt — for **CPU-bound work**. If a virtual thread is doing tight-loop computation (image processing, cryptography, sorting large in-memory data, JSON parsing of huge payloads), it never yields to the scheduler; it simply occupies its carrier thread until done, exactly like a platform thread would. Since there are only as many carrier threads as CPU cores, spinning up a million virtual threads that are all CPU-bound just serializes them behind the same small carrier pool — you get no more parallelism than you had before, plus the (small) overhead of virtual thread bookkeeping. For CPU-bound work, the right tool is still a **bounded platform thread pool** sized to the number of cores (e.g., `ForkJoinPool` or a fixed

ExecutorService), so you get genuine parallel CPU utilization without oversubscribing cores.

```

    // GOOD fit: thousands of concurrent virtual threads, each mostly blocked
    ↪ on I/O
    try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
        for (String url : urls) {
            executor.submit(() -> httpClient.send(request(url),
    ↪ BodyHandlers.ofString()));
        }
    }

    // BAD fit: CPU-bound matrix multiplication on virtual threads gains
    ↪ nothing —
    // use a bounded platform-thread pool (e.g.,
    ↪ Executors.newFixedThreadPool(cores))
    // or ForkJoinPool.commonPool() instead.
    try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
        for (double[][] matrix : matrices) {
            executor.submit(() -> multiply(matrix)); // pure CPU work — no
    ↪ yield points
        }
    }

```

Pinning

Pinning happens when a virtual thread blocks while it *cannot* be unmounted from its carrier thread — the carrier thread is stuck waiting right along with it, defeating the whole point of virtual threads and reducing the number of usable carriers for everyone else.

In **JDK 21**, pinning is caused by:

1. **synchronized blocks or methods** — if a virtual thread blocks *inside* a synchronized block (e.g., waiting on another lock, or performing blocking I/O while holding the monitor), it cannot be unmounted; the carrier thread is pinned until the virtual thread exits the synchronized region.
2. **Native frames** — if the virtual thread's call stack includes a native method (JNI) between it and the blocking operation, the JVM cannot unmount it, because it cannot relocate native stack frames.

```

// Pinning hazard in JDK 21: blocking I/O while holding a monitor pins the
// carrier thread for the duration of the network call.
private final Object lock = new Object();

void handle() {
    synchronized (lock) {
        // If this blocks (slow socket, slow DB), the carrier thread is
        ↪ pinned –
        // it cannot run any other virtual thread until this call returns.
        String response = callSlowLegacyService();
        cache.put(key, response);
    }
}

```

The fix is usually to replace `synchronized` with `java.util.concurrent.locks.ReentrantLock`, which is pinning-safe, and to keep blocking calls out of any remaining `synchronized` sections:

```

private final ReentrantLock lock = new ReentrantLock();

void handle() {
    lock.lock();
    try {
        String response = callSlowLegacyService(); // no pinning –
        ↪ ReentrantLock
        cache.put(key, response);
        // cooperates with the
        ↪ scheduler
    } finally {
        lock.unlock();
    }
}

```

JDK 24 (JEP 491) removed the `synchronized`-block pinning problem entirely: virtual threads can now be unmounted even while holding a monitor acquired via `synchronized`. This was a significant JVM-internal change (it required reworking how monitors interact with the continuation mechanism) and it means that, from JDK 24 onward, `synchronized` is no longer a pinning hazard — only native frames still pin. This is a common interview trap: candidates who memorized “avoid `synchronized` with virtual threads” as an absolute rule need to know it is version-specific (true for JDK 21-23, no longer the dominant concern from JDK 24 on), though switching to `ReentrantLock` remains harmless and is still good practice for other reasons (e.g., `tryLock`, fairness, interruptible acquisition).

Detecting Pinning

Two supported ways to detect pinning in production or during testing:

```
-Djdk.tracePinnedThreads=full
```

Run with this system property and the JVM prints a stack trace every time a virtual thread parks while pinned, showing exactly which frame (the synchronized block or native call) caused it:

```
Thread[#31,tid=31]
  java.base/java.lang.VirtualThread.parkOnCarrierThread(VirtualThread.j_
↳ ava:661)
  java.base/java.lang.VirtualThread.park(VirtualThread.java:592)
  ...
  <== monitors:1
  OrderService.handle(OrderService.java:42) <-- monitor
```

Use `full` for complete stack traces, or `short` for a one-line-per-occurrence summary.

The second option is **JDK Flight Recorder (JFR)**: virtual thread pinning emits a `jdk.VirtualThreadPinned` event, which you can capture with a JFR recording and inspect in JDK Mission Control (JMC) or via `jfr print --events jdk.VirtualThreadPinned recording.jfr`. JFR is lower-overhead and more suitable for always-on production monitoring than the tracing flag, which is best for local debugging or CI.

```
# Capture a JFR recording and filter for pinning events after the fact
java -XX:StartFlightRecording=filename=app.jfr -jar app.jar
jfr print --events jdk.VirtualThreadPinned app.jfr
```

Why You Should NOT Pool Virtual Threads

Thread pools exist to *reuse* expensive platform threads and to *bound* concurrency. Virtual threads are cheap to create and discard — creating a new one is closer to allocating an object than to an OS syscall — so pooling them buys you nothing on the “reuse” side, and actively hurts on the “bounding” side: pooling caps how many tasks can run concurrently by limiting the pool size, which reintroduces exactly the scalability ceiling virtual threads were meant to remove (and can also create priority inversion / deadlock risk if a pooled task blocks waiting on another task that never gets a pool slot).

The idiomatic pattern is **one virtual thread per task**, unbounded in count, with concurrency to any *scarce downstream resource* controlled separately (see semaphores below) — not by limiting how many virtual threads may exist.

```
// Anti-pattern: don't do this – defeats the purpose of virtual threads
ExecutorService fakeVirtualPool = new ThreadPoolExecutor(
    200, 200, 0L, TimeUnit.MILLISECONDS, new LinkedBlockingQueue<>(),
    Thread.ofVirtual().factory()); // pooling virtual threads:
↪ pointless and limiting

// Idiomatic: unbounded virtual-thread-per-task
ExecutorService executor = Executors.newVirtualThreadPerTaskExecutor();
```

ThreadLocal at Millions of Threads

ThreadLocal was designed under the assumption that thread counts are small (dozens to low thousands) and thread lifetimes are long (pooled and reused), so the per-thread storage cost was negligible and cheap to amortize. Virtual threads invert both assumptions: there can be millions of them, and each is typically short-lived (one per task/request).

Every ThreadLocal variable set on a virtual thread still needs its own storage slot on *that* thread, and Java copies ThreadLocal values into InheritableThreadLocal children on thread creation. At scale:

- Memory cost multiplies by however many concurrent virtual threads exist — a ThreadLocal<byte[]> holding even a modest buffer, times a million virtual threads, is a real amount of heap.
- InheritableThreadLocal inheritance means every child virtual thread created (e.g., inside a StructuredTaskScope) does a copy/snapshot of the parent's inheritable locals, which is wasted work if most children never read them.
- ThreadLocal is **mutable** and has **no defined lifetime bound** — nothing forces you to call remove(), so leaked ThreadLocal entries silently retain memory (and, in pooled-thread contexts, correctness bugs) until the thread itself is discarded. With millions of short-lived virtual threads this is somewhat self-limiting (thread death frees the map), but the per-thread map allocation and lookup overhead still adds up.

This is exactly the gap that **Scoped Values** (later in this chapter) are designed to close.

Semaphores Instead of Thread Pools for Limiting Concurrency

If the goal is not to bound *how many virtual threads exist* but to bound *how many concurrent requests hit a specific downstream resource* (a database connection pool, a rate-limited third-party API, a fixed-capacity in-memory cache), the correct tool is a Semaphore, acquired around just the call to that resource — not a bounded executor that throttles everything indiscriminately.

```
// Bound concurrent calls to a downstream payment gateway to 50, while
↳ still
// allowing unlimited virtual threads for everything else the request
↳ does.
private final Semaphore paymentGatewayLimiter = new Semaphore(50);

String chargeCard(Order order) throws InterruptedException {
    paymentGatewayLimiter.acquire();
    try {
        return paymentGateway.charge(order); // blocking call,
↳ virtual-thread friendly
    } finally {
        paymentGatewayLimiter.release();
    }
}
```

Each caller still runs on its own virtual thread and can make unrelated progress (or block cheaply) while waiting for a permit; only access to the *specific scarce resource* is serialized to the configured limit. This decouples “how many logical tasks can be in flight” (effectively unbounded, one virtual thread each) from “how much load a specific downstream can take” (bounded by the semaphore).

Observability and Thread Dumps

Because a JVM might now host millions of virtual threads, traditional `jstack`-style dumps (designed for thousands of platform threads) don’t scale well as plain text. JDK 21+ added a dedicated, structured thread dump command:

```
jcmd <pid> Thread.dump_to_file -format=json threads.json
```

This produces a JSON dump that groups virtual threads by which carrier thread (if any) they’re currently mounted on, distinguishes platform threads from virtual threads, and is designed to be machine-readable so tooling can visualize or query it (rather than grepping raw text). A plain-text variant is also available:

```
jcmd <pid> Thread.dump_to_file -format=text threads.txt
```

For live, low-level thread dumps of a *specific* thread, `Thread.dump()`-style diagnostics and JFR's `jdk.ThreadDump` and `jdk.VirtualThreadPinned/jdk.VirtualThreadSubmitFailed` events fill in the rest of the observability story: pinning (above), submission failures (e.g., carrier pool exhaustion under a custom scheduler), and thread lifecycle events are all available for continuous monitoring via JFR rather than one-off dumps.

Migration Advice

- **Start at I/O boundaries.** The highest-value, lowest-risk place to introduce virtual threads is swapping a fixed-size platform-thread `ExecutorService` used for blocking I/O (handling requests, calling downstream services) for `Executors.newVirtualThreadPerTaskExecutor()`. Frameworks (Tomcat, Jetty, Spring Boot 3.2+) increasingly support flipping this via configuration rather than hand-written executor code.
- **Audit synchronized usage on JDK 21-23.** Use `-Djdk.tracePinnedThreads=full` in a load test or staging environment before rollout; replace hot, blocking synchronized blocks with `ReentrantLock`. This becomes far less urgent on JDK 24+ thanks to JEP 491, but native-frame pinning (JNI) can still occur on any version.
- **Remove artificial concurrency caps that were sized around thread cost.** A new `FixedThreadPool(200)` sized because “200 platform threads is about all we can afford” should usually become unbounded virtual threads plus a `Semaphore` around the *actual* scarce resource (see above) — don’t just swap the factory and keep the artificial cap, or you gain nothing.
- **Don’t rewrite CPU-bound pools.** Leave `ForkJoinPool`/fixed platform-thread pools alone for CPU-bound work; virtual threads are not a general replacement for all concurrency.
- **Watch `ThreadLocal`-heavy legacy code,** especially framework code (security contexts, MDC/logging context, transaction contexts) that relies on `InheritableThreadLocal`. At virtual-thread scale, either confirm the library has been updated to use scoped values / lightweight inheritance, or measure the actual overhead before assuming it’s fine.
- **Re-baseline connection pool sizing.** Virtual threads can generate far more concurrent *logical* requests than before; connection pools (DB, HTTP) sized for a small number of platform threads may need semaphore-style gating even if they were “big

enough” previously, because now thousands of virtual threads can genuinely try to use them at once.

Structured Concurrency

The Problem with Unstructured Fan-Out

A common pattern is: submit several tasks to an `ExecutorService`, then collect their results with `Future.get()`. This works for the happy path, but it has real structural gaps:

```
// Unstructured fan-out – looks fine, has several latent bugs
ExecutorService executor = Executors.newVirtualThreadPerTaskExecutor();
Future<User> userFuture = executor.submit(() -> fetchUser(userId));
Future<Order> orderFuture = executor.submit(() -> fetchOrder(orderId));

User user = userFuture.get();    // if this throws, orderFuture keeps
→ running –
Order order = orderFuture.get(); // an orphaned task, nobody waits for or
→ cancels it
```

Concretely:

- **Leaked/orphan tasks.** If `userFuture.get()` throws, execution never reaches `orderFuture.get()`. The order-fetching task keeps running in the background — consuming a thread, a downstream connection, CPU — with nothing watching it, joining it, or cancelling it. In a request-handling context this is a silent resource leak that repeats on every failure.
- **No automatic cancellation propagation.** If one subtask fails, the *others* should usually be cancelled (why keep computing a result nobody will use?), but plain futures don’t do this for you — you have to remember to call `cancel()` on every sibling, in every error path, including exceptions thrown from unrelated code between submissions.
- **Unclear ownership and lifetime.** Nothing in the code enforces that all child tasks finish (or are cancelled) before the enclosing method returns. This violates the intuitive nesting of blocks in sequential code, where a child scope’s lifetime is visibly bounded by `{ }`. Concurrent tasks submitted to a shared executor have no such visible boundary — they can outlive the method that created them, which makes reasoning about cancellation, exception propagation, and shutdown far harder than for straight-line code.

Structured concurrency fixes this by treating a group of related concurrent subtasks as a single unit of work: they are forked from one *scope*, and that scope cannot be exited (the `try`-block cannot complete) until all of its forked subtasks have completed, failed, or been cancelled. The concurrent code gets the same nesting and lifetime guarantees as sequential code — a child task’s lifetime is strictly contained within its parent’s block.

StructuredTaskScope Basics

The core type is `java.util.concurrent.StructuredTaskScope`, used in a `try-with-resources` block:

```
try (var scope = new StructuredTaskScope.ShutdownOnFailure()) {
    Subtask<User> userTask = scope.fork(() -> fetchUser(userId));
    Subtask<Order> orderTask = scope.fork(() -> fetchOrder(orderId));

    scope.join();           // wait for both forks to finish (or one to
↪ fail)
    scope.throwIfFailed();  // if either failed, rethrow that failure
↪ here

    // Both succeeded – safe to read results
    User user = userTask.get();
    Order order = orderTask.get();
    return new Profile(user, order);
} // scope.close() guarantees no forked subtask survives past this point
```

- `fork(Callable<T>)` starts a subtask (on its own virtual thread by default) and returns a `Subtask<T>` handle.
- `join()` blocks until all forked subtasks finish, or (for policy-aware scopes like `ShutdownOnFailure`) until the scope’s shutdown condition triggers.
- `throwIfFailed()` (on `ShutdownOnFailure`) rethrows the first captured failure, wrapped, if any subtask failed — this is what makes error propagation explicit and centralized instead of scattered across each `Future.get()` call.
- `close()`, called automatically by `try-with-resources`, **is the structural guarantee**: it waits for any still-running subtasks to finish and, in later API shapes, cancels/interrupts them if the scope is exiting due to an exception. You cannot leave the `try` block with orphaned children — that’s the whole point.

ShutdownOnFailure — All-Must-Succeed

`ShutdownOnFailure` implements the common “fan out, and if *any* subtask fails, cancel the rest and fail fast” policy. This is the right shape when you need every result to build a combined response — e.g., assembling a page that needs a user profile *and* their recent orders *and* their loyalty status; if any one lookup fails, the whole response is incomplete and there’s no point waiting for (or paying for) the others.

```
record DashboardData(User user, List<Order> orders, LoyaltyStatus
↳ loyalty) {}

DashboardData loadDashboard(String userId) throws InterruptedException,
↳ ExecutionException {
    try (var scope = new StructuredTaskScope.ShutdownOnFailure()) {
        Subtask<User> userTask = scope.fork(() ->
↳ userService.fetch(userId));
        Subtask<List<Order>> ordersTask = scope.fork(() ->
↳ orderService.recentOrders(userId));
        Subtask<LoyaltyStatus> loyaltyTask = scope.fork(() ->
↳ loyaltyService.status(userId));

        scope.join();
        scope.throwIfFailed(ExecutionException::new); // custom exception
↳ wrapper

        return new DashboardData(userTask.get(), ordersTask.get(),
↳ loyaltyTask.get());
    }
    // If loyaltyService.status() throws, ShutdownOnFailure immediately
↳ signals
    // shutdown: userTask and ordersTask are cancelled/interrupted if
↳ still
    // running, join() returns promptly instead of waiting for them to
↳ finish
    // naturally, and throwIfFailed() rethrows the loyalty failure.
}
```

ShutdownOnSuccess — First-Wins

`ShutdownOnSuccess` implements the opposite policy: race several subtasks and take the *first* one to succeed, cancelling the rest. This is the natural shape for redundant lookups — e.g., querying a primary and a fallback cache/region simultaneously and using whichever answers first, or trying multiple mirrors for the same resource.

```

String fetchFromFastestMirror(String key) throws InterruptedException,
↳ ExecutionException {
    try (var scope = new StructuredTaskScope.ShutdownOnSuccess<String>())
    ↳ {
        scope.fork(() -> primaryRegionCache.get(key));
        scope.fork(() -> secondaryRegionCache.get(key));
        scope.fork(() -> tertiaryRegionCache.get(key));

        scope.join();
        return scope.result(); // returns the first success; throws if all
    ↳ failed
        // As soon as one fork succeeds, ShutdownOnSuccess cancels the
    ↳ other
        // in-flight forks – no wasted work, no orphaned background
    ↳ lookups.
        }
    }
}

```

Timeouts via `joinUntil`

Instead of (or in addition to) `join()`, `joinUntil(Instant deadline)` waits only until a deadline, throwing `TimeoutException` if the subtasks haven't finished by then — and, combined with a shutdown-triggering policy, still ensures nothing is left running past the scope's `close()`.

```

DashboardData loadDashboardWithTimeout(String userId)
    throws InterruptedException, ExecutionException, TimeoutException
↳ {
    try (var scope = new StructuredTaskScope.ShutdownOnFailure()) {
        Subtask<User> userTask = scope.fork(() ->
    ↳ userService.fetch(userId));
        Subtask<List<Order>> ordersTask = scope.fork(() ->
    ↳ orderService.recentOrders(userId));

        scope.joinUntil(Instant.now().plusSeconds(2)); // give downstreams
    ↳ 2s total
        scope.throwIfFailed();

        return new DashboardData(userTask.get(), ordersTask.get(), null);
    } // close() cancels anything still running if joinUntil timed out
}

```

Custom Scope Policies

Both `ShutdownOnFailure` and `ShutdownOnSuccess` are built on the same extensible mechanism (overriding `handleComplete` in earlier preview shapes, or supplying a custom `Joiner` in the JDK 25 shape — see below). This lets you express policies neither built-in covers, such as “succeed once at least N of M subtasks succeed” (a quorum) or “collect all results, ignoring individual failures, and report which ones failed at the end” instead of failing fast.

```
// Sketch of a quorum policy (conceptual; exact base API differs by JDK
→ version) –
// wait for at least 2 of 3 replica writes to succeed before returning.
try (var scope = new QuorumScope<Boolean>(/* required successes */ 2)) {
    scope.fork(() -> replicaA.write(record));
    scope.fork(() -> replicaB.write(record));
    scope.fork(() -> replicaC.write(record));

    scope.join();
    if (!scope.quorumReached()) {
        throw new WriteFailedException("Could not reach write quorum for
→ " + record.id());
    }
}
```

The exact base class/interface for building custom policies changed between the preview iterations and the JDK 25 API — treat the sketch above as illustrating the *idea* (a policy decides, as subtasks complete, whether the scope should shut down early and what the overall outcome is), not as a copy-paste API.

Preview Status and the JDK 25 API Reshape

Structured concurrency is one of the more actively evolving Loom features, and its exact shape has changed across releases — this is a real trap for anyone reciting API details from memory in an interview:

- **JDK 21:** `StructuredTaskScope` introduced as a **preview API** (JEP 453), requiring `--enable-preview` at both compile and run time. Shape: `new StructuredTaskScope.ShutdownOnFailure() / ShutdownOnSuccess<T>(), fork, join, throwIfFailed, joinUntil` — as shown in the examples above.
- **JDK 22-24:** Continued as a **preview API** (re-previewed, JEP 480/499), with refinements but the same overall `ShutdownOnFailure/ShutdownOnSuccess` subclassing shape. Still requires `--enable-preview`.

- **JDK 25:** The API was **reshaped** around an explicit **Joiner** abstraction (finalized under JEP 505 as a preview-to-standard evolution track; check the exact JEP number against your JDK’s release notes, as this has moved during the feature’s preview lifetime). Instead of subclassing `ShutdownOnFailure/ShutdownOnSuccess`, you open a scope with a joiner strategy:

```
// JDK 25-style shape (illustrative) – StructuredTaskScope.open(...) with
↳ a Joiner
try (var scope =
↳ StructuredTaskScope.open(Joiner.<String>awaitAllSuccessfulOrThrow())) {
    scope.fork(() -> userService.fetch(userId));
    scope.fork(() -> orderService.recentOrders(userId));
    List<Subtask<Object>> results = scope.join();
    // ...
}

// First-success equivalent to the old ShutdownOnSuccess
try (var scope =
↳ StructuredTaskScope.open(Joiner.<String>anySuccessfulResultOrThrow())) {
    scope.fork(() -> primaryRegionCache.get(key));
    scope.fork(() -> secondaryRegionCache.get(key));
    String result = scope.join();
}
```

The `Joiner` shape separates “how subtasks are combined/what triggers shutdown” (the `Joiner`) from “the scope mechanics” (`open`, `fork`, `join`), making custom policies a matter of implementing `Joiner` rather than subclassing `StructuredTaskScope`. **As of JDK 25, structured concurrency is still a preview feature** — `--enable-preview` is required to compile and run code using it, on every JDK version mentioned above, including 25. Do not present it as final/GA without checking the release notes of whatever JDK version is actually in play; at the time of writing it has not exited preview.

```
# Compiling and running structured concurrency code on JDK 21 or JDK 25
javac --release 21 --enable-preview Main.java
java --enable-preview Main
```

Scoped Values

Why ThreadLocal Is a Poor Fit for Virtual Threads

ThreadLocal gives each thread its own independent copy of a variable, which is a fine model when threads are few and long-lived. Three specific properties make it a poor fit once threads are cheap, numerous, and short-lived (i.e., virtual threads):

1. **Mutability.** ThreadLocal.set(...) can be called at any time, by any code that has a reference to the ThreadLocal, changing the value for the rest of the thread's lifetime (or until the next set/remove). This makes it hard to reason about what value is “in scope” at a given point — any called method could have mutated it. It also opens the door to values leaking between logical tasks if a thread is reused (a classic thread-pool bug) or forgotten to be cleared.
2. **Unbounded lifetime.** Nothing forces a ThreadLocal value to be cleared when the logical operation that set it is done; you must remember to call remove(), typically in a finally block, or the value (and whatever memory it retains) lives until the thread itself dies or is reused. Since there is no compiler or runtime enforcement, forgetting to clean up is a common, hard-to-spot leak.
3. **Inheritance cost.** InheritableThreadLocal copies parent values into every child thread at creation time. With platform thread pools this happens rarely (pools don't spawn new threads often). With virtual threads — where creating one per task, and forking many children per StructuredTaskScope, is the *normal* pattern — this copying happens constantly, at a scale (potentially millions of threads) where even a cheap copy operation adds up, and where most children may never actually read the inherited value.

ScopedValue — Immutable, Bounded, Structured

ScopedValue<T> (introduced as a preview feature, JEP 429 in JDK 21, continuing through subsequent JDKs as a preview feature with refinements — check --enable-preview requirements per JDK) addresses all three issues by design:

- **Immutable within a scope.** Once bound via where(...).run(...), the value cannot be changed from inside that block — there is no set() method. Rebinding is only possible by opening a *new*, nested where(...) scope (see below), which is visible and structured, not a silent mutation.
- **Bounded lifetime.** The binding exists only for the duration of the run/call invocation — it is automatically and unconditionally torn down when that method returns, including via exception. There's no remove() to forget.

- **Cheap, structured inheritance.** When a `ScopedValue`-bound thread forks children through a `StructuredTaskScope`, the binding is available to those children without a per-child copy — the runtime shares the immutable binding rather than duplicating it, which is both cheaper and safer at virtual-thread scale.

```
public class RequestContext {
    private static final ScopedValue<String> REQUEST_ID =
↳ ScopedValue.newInstance();

    public static void handleRequest(String requestId, HttpExchange
↳ exchange) {
        ScopedValue.where(REQUEST_ID, requestId)
            .run(() -> processRequest(exchange));
        // REQUEST_ID is unbound again as soon as run() returns — nothing
↳ to clean up
    }

    static void processRequest(HttpExchange exchange) {
        log.info("Handling request {}", REQUEST_ID.get()); // reads the
↳ bound value
        callDownstream(); // REQUEST_ID stays bound for the whole call
↳ tree beneath run()
    }

    static void callDownstream() {
        log.info("Still request {}", REQUEST_ID.get()); // visible several
↳ frames down
    }
}
```

`where(...).call(...)` is the equivalent for code that returns a value or throws a checked exception:

```
String result = ScopedValue.where(REQUEST_ID, requestId)
    .call(() -> fetchAndFormat(exchange));
```

Rebinding

Because bindings are scoped to a block, “changing” a value for a nested piece of work means opening a new `where(...)` inside the existing one — the outer binding is restored automatically once the inner block exits:

```

ScopedValue.where(REQUEST_ID, "req-1").run(() -> {
    log.info(REQUEST_ID.get()); // "req-1"

    ScopedValue.where(REQUEST_ID, "req-1-retry").run(() -> {
        log.info(REQUEST_ID.get()); // "req-1-retry" – rebound for this
        ↪ inner block only
    });

    log.info(REQUEST_ID.get()); // back to "req-1" – outer binding
    ↪ restored
});

```

Inheritance into StructuredTaskScope Forks

ScopedValue bindings established before a StructuredTaskScope is opened are visible inside subtasks forked from that scope, without any explicit copying code — this is the natural pairing of the two features: bind request-scoped context once, fan out safely, and every forked subtask (and anything it calls) can read that context.

```

ScopedValue<String> REQUEST_ID = ScopedValue.newInstance();

DashboardData loadDashboard(String userId, String requestId)
    throws InterruptedException, ExecutionException {
    return ScopedValue.where(REQUEST_ID, requestId).call(() -> {
        try (var scope = new StructuredTaskScope.ShutdownOnFailure()) {
            // Both forked subtasks can call REQUEST_ID.get() and see
            ↪ "requestId",
            // with no manual propagation and no InheritableThreadLocal
            ↪ copy cost.
            Subtask<User> userTask = scope.fork(() -> {
                log.info("[{}] fetching user", REQUEST_ID.get());
                return userService.fetch(userId);
            });
            Subtask<List<Order>> ordersTask = scope.fork(() -> {
                log.info("[{}] fetching orders", REQUEST_ID.get());
                return orderService.recentOrders(userId);
            });

            scope.join();
            scope.throwIfFailed();
            return new DashboardData(userTask.get(), ordersTask.get(),
            ↪ null);
        }
    });
}

```

Preview Status

ScopedValue has been a **preview API since JDK 21** (JEP 429) and remains preview through the JDK versions covered by this chapter, refined along the way (JEP 481/506 in later releases — check the exact JEP number for the JDK you are targeting, since, like structured concurrency, this feature’s finalization timeline has shifted release to release). `--enable-preview` is required to compile and run code that uses it, on every affected JDK version. Do not assume it is final without checking your specific JDK’s release notes.

Before/After: Migrating from ThreadLocal

```
// BEFORE – ThreadLocal: mutable, must remember to clean up, costly to
↳ inherit
// into children at virtual-thread scale.
public class TenantContext {
    private static final ThreadLocal<String> TENANT_ID = new
↳ ThreadLocal<>();

    public static void handle(String tenantId, Runnable task) {
        TENANT_ID.set(tenantId);
        try {
            task.run();
        } finally {
            TENANT_ID.remove(); // easy to forget; leaks the value if
↳ omitted
        }
    }

    public static String currentTenant() {
        return TENANT_ID.get();
    }
}

// AFTER – ScopedValue: immutable for the duration of run(), automatically
// unbound on exit (even via exception), cheap to share with forked
↳ subtasks.
public class TenantContext {
    private static final ScopedValue<String> TENANT_ID =
↳ ScopedValue.newInstance();

    public static void handle(String tenantId, Runnable task) {
        ScopedValue.where(TENANT_ID, tenantId).run(task);
        // no finally, no remove() – unbinding is automatic and guaranteed
    }
}
```



```

    public static String currentTenant() {
        return TENANT_ID.orElse("unknown"); // safe default if unbound
    }
}

```

The AFTER version cannot leak a stale tenant into unrelated work on a reused thread (there's no `set()` outside the scope, and no thread reuse concern for one-virtual-thread-per-task code), cannot be forgotten to be cleaned up (there's no cleanup step at all), and shares its binding cheaply with any `StructuredTaskScope` forks started inside `task.run()`.

Comparison Table: Platform Threads vs Virtual Threads

Aspect	Platform Thread	Virtual Thread
Creation cost	High — OS-level thread creation (syscall), noticeable latency	Very low — heap allocation, no OS thread created per virtual thread
Stack	Fixed-size native stack (commonly 512 KB-1 MB), reserved upfront	Grows/shrinks on the heap as a continuation; starts small
Scheduling	OS scheduler, preemptive, across all platform threads on the machine	JVM scheduler (carrier-thread pool, typically ~one per core), cooperative unmount on blocking calls
Typical count	Hundreds to a few thousand before memory/scheduling cost becomes a problem	Hundreds of thousands to millions
Blocking behavior	Blocks the OS thread; that thread is unusable for anything else while blocked	Unmounts from its carrier on blocking I/O, freeing the carrier for other virtual threads (unless pinned)
Best use	CPU-bound work; long-lived worker/daemon threads; native-frame-heavy work	I/O-bound, high-concurrency, blocking-style code (thread-per-request servers, fan-out to downstreams)
Pooling	Yes — reuse is the point (<code>newFixedThreadPool</code> , etc.)	No — create one per task; never pool

Worked Example: Fan-Out to Three Downstream Calls, Three Ways

Scenario: a `ProfileService` needs to build a page that requires calling three independent downstream services — `UserService`, `OrderService`, and `RecommendationService` — and combining their results. All three calls are independent and can run concurrently; the page can only be built if all three succeed.

```
record Profile(User user, List<Order> orders, List<Recommendation>
↳ recommendations) {}

interface UserService { User fetch(String userId) throws Exception; }
interface OrderService { List<Order> recentOrders(String userId) throws
↳ Exception; }
interface RecommendationService { List<Recommendation>
↳ recommendationsFor(String userId) throws Exception; }
```

Version 1: Raw ExecutorService and Future

```
class ExecutorServiceProfileService {
    private final ExecutorService executor =
↳ Executors.newVirtualThreadPerTaskExecutor();
    private final UserService userService;
    private final OrderService orderService;
    private final RecommendationService recommendationService;

    Profile buildProfile(String userId) throws InterruptedException,
↳ ExecutionException {
        Future<User> userFuture = executor.submit(() ->
↳ userService.fetch(userId));
        Future<List<Order>> ordersFuture = executor.submit(() ->
↳ orderService.recentOrders(userId));
        Future<List<Recommendation>> recsFuture =
            executor.submit(() ->
↳ recommendationService.recommendationsFor(userId));

        try {
            // If userFuture.get() throws, ordersFuture and recsFuture
↳ keep running
            // unattended – we never cancel them. That's the
↳ unstructured-fan-out
            // problem in practice: no automatic cancellation, no bounded
↳ lifetime.
        }
    }
}
```

```

        User user = userFuture.get();
        List<Order> orders = ordersFuture.get();
        List<Recommendation> recs = recsFuture.get();
        return new Profile(user, orders, recs);
    } catch (ExecutionException e) {
        userFuture.cancel(true); // must remember to do this
        ↪ manually,
        ordersFuture.cancel(true); // in every catch block, for every
        ↪ sibling –
        recsFuture.cancel(true); // easy to omit, easy to get wrong
        ↪ under refactors
        throw e;
    }
}
}

```

Version 2: CompletableFuture

```

class CompletableFutureProfileService {
    private final Executor executor =
    ↪ Executors.newVirtualThreadPerTaskExecutor();
    private final UserService userService;
    private final OrderService orderService;
    private final RecommendationService recommendationService;

    Profile buildProfile(String userId) throws ExecutionException,
    ↪ InterruptedException {
        CompletableFuture<User> userFuture =
            CompletableFuture.supplyAsync(() -> unchecked(() ->
            ↪ userService.fetch(userId)), executor);
        CompletableFuture<List<Order>> ordersFuture =
            CompletableFuture.supplyAsync(() -> unchecked(() ->
            ↪ orderService.recentOrders(userId)), executor);
        CompletableFuture<List<Recommendation>> recsFuture =
            CompletableFuture.supplyAsync(
            ↪ recommendationService.recommendationsFor(userId), executor);

        // allOf composes cancellation-unaware futures; if one fails, the
        ↪ others
        // are NOT automatically cancelled by allOf – you'd need to chain
        // .exceptionally / .whenComplete on each one yourself to cancel
        ↪ siblings,
        // which most CompletableFuture code in the wild does not bother
        ↪ doing.
    }
}

```

```

        return CompletableFuture.allOf(userFuture, ordersFuture,
↪ recsFuture)
            .thenApply(v -> new Profile(userFuture.join(),
↪ ordersFuture.join(), recsFuture.join()))
            .get();
    }

    private static <T> T unchecked(Callable<T> callable) {
        try {
            return callable.call();
        } catch (Exception e) {
            throw new CompletionException(e);
        }
    }
}

```

This is more composable than raw futures (chaining, combining, async pipelines) but the exception/cancellation story is still manual and easy to get subtly wrong — `allOf` waits for all of them regardless of failure, and nothing cancels the still-running siblings when one fails unless you add that logic yourself.

Version 3: StructuredTaskScope

```

class StructuredProfileService {
    private final UserService userService;
    private final OrderService orderService;
    private final RecommendationService recommendationService;

    Profile buildProfile(String userId) throws InterruptedException,
↪ ExecutionException {
        try (var scope = new StructuredTaskScope.ShutdownOnFailure()) {
            Subtask<User> userTask = scope.fork(() ->
↪ userService.fetch(userId));
            Subtask<List<Order>> ordersTask = scope.fork(() ->
↪ orderService.recentOrders(userId));
            Subtask<List<Recommendation>> recsTask =
                scope.fork(() ->
↪ recommendationService.recommendationsFor(userId));

            scope.join();
            scope.throwIfFailed(); // rethrows the first failure; other
↪ subtasks are
// already cancelled by
↪ ShutdownOnFailure by this point

```

```

        return new Profile(userTask.get(), ordersTask.get(),
↪ recsTask.get());
    } // close() guarantees zero orphaned subtasks, success or failure
}

```

Comparison: all three versions run the three downstream calls concurrently and produce the same result on the happy path. They diverge sharply on the failure path: Version 1 requires you to remember manual cancellation in every catch block; Version 2's `allOf` does not cancel siblings on failure unless you add explicit exceptionally/whenComplete wiring per future; Version 3 gets fail-fast cancellation of siblings and a guaranteed-no-orphans lifetime *for free*, as a structural property of the scope, with the least code and the clearest error-handling story. This is why structured concurrency, despite still being preview, is the direction the language is pushing fan-out/fan-in code toward.

Common Code-Review Interview Pitfalls

1. Pooling virtual threads (`newFixedThreadPool` with a virtual thread factory).

Why it matters: pooling exists to amortize the cost of expensive platform threads and to bound concurrency; virtual threads are cheap to create, so pooling them buys nothing and reintroduces the same concurrency ceiling virtual threads were meant to remove.

```

// Before
new ThreadPoolExecutor(200, 200, 0, TimeUnit.SECONDS, queue,
↪ Thread.ofVirtual().factory());
// After
Executors.newVirtualThreadPerTaskExecutor();

```

2. Using virtual threads for CPU-bound work and expecting a speedup.

Why it matters: virtual threads only help when threads spend time *blocked*; CPU-bound work occupies its carrier thread the whole time, so a million CPU-bound virtual threads run no faster than a small platform-thread pool sized to the core count — and adds bookkeeping overhead for no benefit.

```

// Before
try (var ex = Executors.newVirtualThreadPerTaskExecutor()) {
    for (var m : matrices) ex.submit(() -> multiply(m)); // pure CPU
↪ work
}

```

```
// After
try (var ex = Executors.newFixedThreadPool(Runtime.getRuntime().getRuntime().
    ↪ availableProcessors())) {
    for (var m : matrices) ex.submit(() -> multiply(m));
}
```

- 3. Blocking inside a synchronized block on JDK 21-23 without realizing it pins the carrier thread.** Why it matters: pinning ties up a carrier thread for the whole blocking call, effectively turning a cheap virtual thread back into an expensive scarce resource, and can silently reduce throughput under load until diagnosed with `-Djdk.tracePinnedThreads`.

```
// Before (JDK 21-23)
synchronized (lock) { String r = callSlowService(); }
// After
lock.lock();
try { String r = callSlowService(); } finally { lock.unlock(); }
```

- 4. Claiming synchronized always pins virtual threads, regardless of JDK version.** Why it matters: JEP 491 (JDK 24) removed synchronized-block pinning; stating this as a universal, version-independent rule in a review or interview is factually outdated and signals stale knowledge. Native-frame pinning is still a concern on every version.

```
// Before: "never use synchronized with virtual threads, period"
// After: "synchronized pins carriers on JDK 21-23; JEP 491 (JDK 24+)"
↪ fixed
//      this for synchronized specifically – native-frame pinning
↪ remains"
```

- 5. Using ThreadLocal/InheritableThreadLocal for per-request context at virtual-thread scale.** Why it matters: mutability plus unbounded lifetime plus per-child inheritance copying becomes expensive and leak-prone once there can be millions of short-lived threads; ScopedValue gives immutability, automatic unbinding, and cheap structured inheritance instead.

```
// Before
private static final ThreadLocal<String> TENANT_ID = new
    ↪ ThreadLocal<>();
// After
private static final ScopedValue<String> TENANT_ID =
    ↪ ScopedValue.newInstance();
```

- 6. Fan-out with raw `ExecutorService/Future` and no cancellation on the failure path.** Why it matters: when one `Future.get()` throws, sibling tasks keep running unattended unless you explicitly cancel every one of them in every catch block — a leak that’s easy to introduce and easy to miss in review.

```
// Before
User u = userFuture.get(); Order o = orderFuture.get(); // orderFuture
↳ orphaned if userFuture throws
// After
try (var scope = new StructuredTaskScope.ShutdownOnFailure()) {
    var uT = scope.fork(() -> fetchUser()); var oT = scope.fork(() ->
    fetchOrder());
    scope.join(); scope.throwIfFailed();
}
```

- 7. Assuming `CompletableFuture.allOf(...)` cancels siblings when one future fails.** Why it matters: `allOf` simply waits for every future regardless of outcome; it does not cancel anything on failure, so “fail fast, stop wasted work” requires extra manual wiring that’s frequently omitted.

```
// Before: assumes cancellation happens automatically
CompletableFuture.allOf(f1, f2, f3).get();
// After: use StructuredTaskScope.ShutdownOnFailure for genuine
↳ fail-fast cancellation
```

- 8. Forgetting `--enable-preview` (or claiming the feature is final) for structured concurrency / scoped values.** Why it matters: both remain preview APIs through JDK 25 at the time of writing; code that compiles for a reviewer without the flag, or a claim in an interview that they are GA, is simply incorrect and easy for an interviewer to catch.

```
# Before
javac Main.java # fails: preview API used without --enable-preview
# After
javac --release 25 --enable-preview Main.java && java --enable-preview
↳ Main
```

- 9. Writing `StructuredTaskScope` code against the wrong API shape for the target JDK (subclassing vs `Joiner`).** Why it matters: JDK 21-24 use new `StructuredTaskScope.ShutdownOnFailure()`-style subclassing; JDK 25 reshapes this around `StructuredTaskScope.open(Joiner...)`. Code (or an answer) that mixes the two shapes reveals unfamiliarity with the feature’s actual evolution.

```
// JDK 21-24
try (var scope = new StructuredTaskScope.ShutdownOnFailure()) { ... }
// JDK 25
try (var scope =
    → StructuredTaskScope.open(Joiner.awaitAllSuccessfulOrThrow())) { ... }
```

- 10. Not bounding concurrency to a genuinely scarce downstream resource just because virtual threads themselves are unbounded.** Why it matters: “virtual threads are free” does not mean the database, connection pool, or third-party API on the other end can handle unbounded concurrent calls; without a Semaphore (or similar) around the call site, unbounded virtual-thread fan-out can overwhelm a downstream that used to be implicitly protected by a small platform-thread pool.

```
// Before: every virtual thread hits the DB directly, no limit
executor.submit(() -> db.query(sql));
// After
limiter.acquire();
try { db.query(sql); } finally { limiter.release(); }
```

- 11. Letting a ScopedValue binding’s lifetime outlive the logical operation by binding too broadly (e.g., at application startup).** Why it matters: the safety benefit of ScopedValue comes from its binding being tightly scoped to one call tree; binding it once at a very high level (defeating the purpose) reintroduces the same “value visible everywhere, for too long” problem ThreadLocal had, just with extra ceremony.

```
// Before: bound once for the whole application run – no real scoping
→ benefit
ScopedValue.where(TENANT_ID, "default").run(Application::run);
// After: bind per request, inside the request-handling method
ScopedValue.where(TENANT_ID, tenantId).run(() ->
    → handleRequest(exchange));
```

- 12. Expecting a StructuredTaskScope’s forked subtasks to survive past the enclosing try-with-resources block.** Why it matters: close() enforces that no subtask outlives the scope — leaking a Subtask handle or its result outside the block, expecting it to keep running in the background, misunderstands the entire point of structured concurrency (bounded, visible lifetimes).

```
// Before: expecting background work to continue after the block
Subtask<String> leaked;
try (var scope = new StructuredTaskScope.ShutdownOnFailure()) {
```



```

        leaked = scope.fork(() -> longRunningJob());
    } // scope.close() already joined/cancelled it – it did not "keep
    ↳ running"
    // After: if it must genuinely outlive this method, don't put it in a
    ↳ scope –
    //          use a plain virtual thread or executor with its own lifecycle

```

- 13. Reusing a single, wide Semaphore (or none at all) for unrelated downstream resources instead of one per resource.** Why it matters: a shared limiter conflates unrelated concurrency budgets — a burst of calls to a fast, high-capacity service can starve permits needed by a slow, low-capacity one (or vice versa) — undermining the whole point of per-resource throttling.

```

// Before: one limiter for every downstream call
private final Semaphore anyDownstream = new Semaphore(100);
// After: one limiter per distinct scarce resource
private final Semaphore paymentGatewayLimiter = new Semaphore(50);
private final Semaphore inventoryServiceLimiter = new Semaphore(200);

```

- 14. Ignoring InterruptedException inside a forked subtask instead of letting cancellation propagate.** Why it matters: StructuredTaskScope's cancellation (on ShutdownOnFailure/ShutdownOnSuccess) works by interrupting running subtasks; a subtask that swallows InterruptedException without restoring the flag or exiting defeats the scope's ability to actually stop wasted work when a sibling fails or wins.

```

// Before
scope.fork(() -> { try { return slowCall(); } catch
↳ (InterruptedException e) { return retry(); } });
// After
scope.fork(() -> {
    try { return slowCall(); }
    catch (InterruptedException e) {
↳ Thread.currentThread().interrupt(); throw e; }
});

```

- 15. Diagnosing “too many threads” performance problems on a virtual-thread-heavy service with jstack-style plain-text dumps instead of the JSON format.** Why it matters: with hundreds of thousands of virtual threads, a flat text dump is impractical to analyze; `jcmd <pid> Thread.dump_to_file -format=json` produces a structured dump (grouped by carrier thread) intended for tooling, and

JFR's `jdk.VirtualThreadPinned` event is the supported way to find pinning at scale rather than eyeballing text.

```
# Before
jstack <pid> > dump.txt    # unwieldy at hundreds of thousands of
↳ virtual threads
# After
jcmd <pid> Thread.dump_to_file -format=json threads.json
```

Reflection & Runtime Features

Most Java code is written to be checked and wired together at **compile time**: you call a method, the compiler verifies it exists, and the JVM runs it. **Reflection** and its related APIs flip that around — they let code inspect and call other code *while the program is running*, even if the exact class or method wasn't known when it was compiled. This is how frameworks like Spring, Hibernate, and Jackson do their “magic”: they look at your classes, fields, and annotations at runtime and build behavior around them. This chapter covers reflection itself, annotations (the metadata reflection often reads), annotation processing (a compile-time alternative), dynamic proxies, the newer and faster method handle and VarHandle APIs, what replaced the old `Unsafe` class, and the Service Provider Interface (SPI) mechanism for plugin-style architectures. We target Java 21+ throughout.

Reflection

Reflection is the ability of code to examine and manipulate classes, fields, methods, and constructors at runtime, using objects instead of source-code syntax. The entry point is almost always a `Class<?>` object, which represents a loaded class inside the JVM. Every object has exactly one `Class` object describing its type.

There are three common ways to get a `Class` object:

```
// 1. From a class literal – known at compile time, checked by the
↪ compiler
Class<String> byLiteral = String.class;

// 2. From an instance you already have
String s = "hello";
Class<?> byInstance = s.getClass();

// 3. By name, at runtime – this is the "dynamic" one, used by frameworks
Class<?> byName = Class.forName("java.lang.String");
```

`Class.forName` is special: it is the classic way to load a class whose name is only known as a `String` (for example, read from a config file or a JDBC driver property). It also triggers static initialization of that class.

Inspecting Members: Fields, Methods, Constructors

Once you have a `Class` object, you can list its members.

```
import java.lang.reflect.*;

public class Inspector {
    static class Person {
        private String name;
        public Person(String name) { this.name = name; }
        public String greet() { return "Hi, " + name; }
        private void secret() { }
    }

    public static void main(String[] args) throws Exception {
        Class<?> clazz = Person.class;

        for (Field f : clazz.getDeclaredFields()) {
            System.out.println("field: " + f.getName() + " : " +
↪ f.getType());
        }
        for (Method m : clazz.getDeclaredMethods()) {
            System.out.println("method: " + m.getName());
        }
        for (Constructor<?> c : clazz.getDeclaredConstructors()) {
            System.out.println("constructor: " + c);
        }
    }
}
```

getDeclaredX vs getX

This is one of the most commonly confused pairs in reflection:

Method	Includes inherited members?	Includes private/protected members?
--------	-----------------------------	-------------------------------------

<code>getFields()</code> / <code>getMethods()</code> / <code>getConstructors()</code>	Yes (public members from superclasses/interfaces too)	No — public only
---	---	------------------

Method	Includes inherited members?	Includes private/protected members?
getDeclaredFields() / getDeclaredMethods() / getDeclaredConstructors()	No — only members declared directly in this class	Yes — all of them, regardless of visibility

Rule of thumb: use `getDeclaredX` when you need to see *everything this exact class wrote*, including private fields. Use `getX` when you want the public API surface, including what it inherited.

Instantiating and Invoking

```
import java.lang.reflect.*;

public class ReflectiveCall {
    public static void main(String[] args) throws Exception {
        Class<?> clazz = Class.forName("java.util.ArrayList");

        // Instantiate via a constructor
        Constructor<?> ctor = clazz.getDeclaredConstructor();
        Object list = ctor.newInstance();

        // Invoke a method reflectively
        Method add = clazz.getMethod("add", Object.class);
        add.invoke(list, "hello");

        Method size = clazz.getMethod("size");
        System.out.println(size.invoke(list)); // 1
    }
}
```

setAccessible(true) and Strong Encapsulation (JDK 16/17+)

Reflection can normally only call public members. To reach private ones, you call `setAccessible(true)` on the `Field`, `Method`, or `Constructor` object, which tells the JVM to skip its usual access checks.

```
Field nameField = Person.class.getDeclaredField("name");
nameField.setAccessible(true);
Person p = new Person("Ada");
```

```
System.out.println(nameField.get(p)); // "Ada" – bypassing private
```

Since JDK 9, the Java Platform Module System (JPMS) introduced **strong encapsulation**: modules can hide their internal packages so that even reflection cannot break in, unless the module explicitly “opens” that package. Starting with **JDK 16 (illegal access denied by default) and fully enforced by JDK 17**, calling `setAccessible(true)` on a member of an internal JDK class (like something in `java.util.internals`) throws an `InaccessibleObjectException` instead of just printing a warning as it did in JDK 9–15.

```
Exception in thread "main" java.lang.reflect.InaccessibleObjectException:
Unable to make field private final byte[] java.lang.String.value
↪ accessible:
module java.base does not "opens java.lang" to unnamed module
```

To allow it anyway, you must explicitly open the package, either in `module-info.java`:

```
module my.app {
    // grants reflective access to internal fields at runtime only
    opens com.internal.pkg to some.other.module;
}
```

or via a JVM flag at launch time, which is the common escape hatch for libraries (like older Mockito or Jackson versions) that still reflect into JDK internals:

```
java --add-opens java.base/java.lang=ALL-UNNAMED -jar app.jar
```

This matters in real projects: upgrading from Java 8/11 to 17+ is a common source of `InaccessibleObjectException` failures in libraries that reflect into JDK internals.

Performance Cost

Reflective calls are slower than direct calls because the JVM must perform access checks, resolve the target dynamically, and (for `invoke`) box/unbox arguments through `Object[]`. The JIT compiler also has a much harder time inlining and optimizing through `Method.invoke`. In hot paths (tight loops, request handling), reflection can be 10-100x slower than a direct call. Frameworks mitigate this by caching `Method/Field` objects (looking them up once, not per call) and by preferring

MethodHandle (covered later) which the JIT can optimize almost as well as direct calls.

Generic Type Info at Runtime

Because of **type erasure** (see the Generics chapter), a `List<String>` and a `List<Integer>` look identical at runtime — the generic type argument is gone from the object. However, reflection can still recover generic type information that was declared on a **field, method, or superclass**, because that information is stored in the class file, not on the runtime object.

```
import java.lang.reflect.*;
import java.util.*;

class StringListHolder {
    private List<String> names;
}

public class GenericInspection {
    public static void main(String[] args) throws Exception {
        Field f = StringListHolder.class.getDeclaredField("names");
        Type genericType = f.getGenericType(); //
        ↪ java.util.List<java.lang.String>
        if (genericType instanceof ParameterizedType pt) {
            System.out.println(pt.getActualTypeArguments()[0]); // class
            ↪ java.lang.String
        }
    }
}
```

This trick — capturing a generic type through a subclass so it survives erasure — is called the **TypeToken pattern** (also called “super type tokens”), and it is how libraries like Gson and Guice let you say “give me a `List<String>` at runtime”:

```
import java.lang.reflect.*;

public abstract class TypeToken<T> {
    private final Type type;

    protected TypeToken() {
        // getGenericSuperclass() sees TypeToken<List<String>> because
        // we captured it in an anonymous subclass below
        ParameterizedType pt = (ParameterizedType)
        ↪ getClass().getGenericSuperclass();
    }
}
```

```

        this.type = pt.getActualTypeArguments()[0];
    }

    public Type getType() { return type; }
}

// usage: an anonymous subclass "bakes in" the type argument into the
↪ class file
TypeToken<List<String>> token = new TypeToken<List<String>>() {};
System.out.println(token.getType()); // java.util.List<java.lang.String>

```

When Reflection Is Legitimate vs a Smell

Legitimate uses:

- **Frameworks and dependency injection** (Spring, Guice) — wiring objects together without every class knowing about every other class.
- **Serialization libraries** (Jackson, Gson) — reading/writing arbitrary object fields generically.
- **Testing tools** (Mockito, JUnit) — creating mocks and invoking test methods discovered by annotation.
- **ORMs** (Hibernate/JPA) — mapping database columns to entity fields.

Code smell / red flag in a code review:

- Application (non-framework) business logic reaching into another class's private fields via reflection instead of adding a getter or constructor parameter. This usually signals a design problem — two classes are too tightly coupled, or an API is missing.
- Reflection used to bypass `final` or visibility rules “just to make the compiler happy” instead of fixing the underlying design.

Annotations

An **annotation** is metadata attached to code — a class, method, field, or parameter — that does nothing by itself. It only has an effect if something (the compiler, a runtime library, or an annotation processor) reads it and acts on it.

Declaring an Annotation

```
import java.lang.annotation.*;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
public @interface Timed {
    String label() default "";
}
```

Usage:

```
public class OrderService {
    @Timed(label = "checkout")
    public void checkout() { /* ... */ }
}
```

Meta-Annotations

Meta-annotations are annotations that describe *other annotations*. The main ones:

- **@Retention** — how long the annotation is kept.
- **@Target** — what kinds of declarations it can be placed on (methods, fields, types, parameters...).
- **@Inherited** — if placed on an annotation, subclasses automatically “inherit” it from their superclass (only applies to class-level annotations, not methods or fields).
- **@Repeatable** — allows the same annotation to be applied more than once to the same element.
- **@Documented** — includes the annotation in generated Javadoc.

```
@Inherited
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.TYPE)
public @interface Auditable { }

@Auditable
class Base { }

class Derived extends Base { } //
↳ Derived.class.isAnnotationPresent(Auditable.class) == true
```

```

import java.lang.annotation.*;

@Retention(RetentionPolicy.RUNTIME)
public @interface Schedules {
    Schedule[] value();
}

@Retention(RetentionPolicy.RUNTIME)
@Repeatable(Schedules.class) // container annotation required for
↔ @Repeatable
public @interface Schedule {
    String cron();
}

class Job {
    @Schedule(cron = "0 0 * * *")
    @Schedule(cron = "0 12 * * *")
    void run() { }
}

```

Reading Annotations at Runtime

Only annotations with `RetentionPolicy.RUNTIME` are visible via reflection.

```

import java.lang.reflect.*;

public class TimedRunner {
    public static void main(String[] args) throws Exception {
        Method m = OrderService.class.getMethod("checkout");
        if (m.isAnnotationPresent(Timed.class)) {
            Timed timed = m.getAnnotation(Timed.class);
            System.out.println("Label: " + timed.label()); // "checkout"
        }
    }
}

```

Retention Policies

Policy	Kept in .class file?	Visible to reflection at runtime?	Typical use
SOURCE	No — discarded by the compiler	No	Compile-time-only checks, e.g. <code>@Override</code> , Lombok's markers, annotation processors that only need source info
CLASS (default if unspecified)	Yes	No — JVM doesn't load it into memory for reflection	Bytecode-level tools (e.g. some bytecode weavers); rarely used directly by application code
RUNTIME	Yes	Yes	Frameworks that inspect annotations via reflection at runtime, e.g. <code>@Autowired</code> , <code>@Test</code> , <code>@Entity</code>

Type Annotations (JDK 8+)

Since Java 8, annotations can also be placed on **type uses**, not just declarations — for example, on a generic type argument, a cast, or a throws clause. This is mainly used by static analysis tools (like the Checker Framework) to catch bugs such as null-pointer risks before runtime.

```
import java.lang.annotation.*;

@Target(ElementType.TYPE_USE)
@interface NonNull { }

public class TypeAnnotationDemo {
    // annotation on the type argument, not the field itself
    private List<@NonNull String> names;

    // annotation on a cast
    Object o = "text";
    String s = (@NonNull String) o;
}
```

Annotation Processing

Annotation processing happens at *compile time*, not runtime. An annotation processor is a plug-in for the compiler (`javac`) that reads annotations in your source code and can generate new source files (or report errors), all before the program ever runs.

The Processor Interface and AbstractProcessor

```
import javax.annotation.processing.*;
import javax.lang.model.SourceVersion;
import javax.lang.model.element.*;
import java.util.Set;

@SupportedAnnotationTypes("com.example.GenerateBuilder")
@SupportedSourceVersion(SourceVersion.RELEASE_21)
public class BuilderProcessor extends AbstractProcessor {

    @Override
    public boolean process(Set<? extends TypeElement> annotations,
↪ RoundEnvironment roundEnv) {
        for (Element element :
↪ roundEnv.getElementsAnnotatedWith(GenerateBuilder.class)) {
            TypeElement classElement = (TypeElement) element;
            generateBuilderClass(classElement);
        }
        return true; // true = "I claimed these annotations, don't let
↪ other processors see them"
    }

    private void generateBuilderClass(TypeElement classElement) {
        // Use processingEnv.getFiler() to write a new .java file (shown
↪ below)
    }
}
```

Rounds

Annotation processing runs in **rounds**. Round 1 processes the original source files. If a processor generates new source files in round 1, `javac` runs another round to process *those* generated files too (since they might themselves have annotations). This repeats until no new files are generated. `RoundEnvironment.processingOver()` tells a processor when the final round has finished, useful for cleanup or validation that should only happen once.

The Filer: Generating Code

The Filer API (`processingEnv.getFiler()`) is how a processor writes new source or resource files during compilation.

```

import javax.annotation.processing.Filer;
import javax.tools.JavaFileObject;
import java.io.Writer;

void writeGeneratedClass(String packageName, String className, String
↪ content) throws Exception {
    Filer filer = processingEnv.getFiler();
    JavaFileObject file = filer.createSourceFile(packageName + "." +
↪ className);
    try (Writer writer = file.openWriter()) {
        writer.write(content);
    }
}

```

Registering a Processor via META-INF/services

For javac to discover your processor automatically, register it as a service (the same SPI mechanism covered later in this chapter):

```
src/main/resources/META-INF/services/javax.annotation.processing.Processor
```

containing one line:

```
com.example.BuilderProcessor
```

Modern projects usually use Google's auto-service library to generate this file automatically via `@AutoService(Processor.class)`.

Real-World Examples

- **Lombok** — generates getters, setters, constructors, equals/hashCode, and builders from annotations like `@Data` and `@Builder`. (Technically it hooks into javac in a slightly unofficial way, modifying the in-memory AST rather than just generating new files, but conceptually it's the same idea.)
- **MapStruct** — generates type-safe mapping code between DTOs and entities from an `@Mapper` interface, avoiding hand-written or reflective mapping code.
- **Dagger** — generates dependency-injection wiring code at compile time instead of using runtime reflection like Spring/Guice.

- **Immutableables** — generates immutable value classes from an `@Value.Immutable`-annotated abstract class or interface.

Why Compile-Time Beats Runtime Reflection

- **Performance:** generated code is plain Java, as fast as hand-written code — no reflective lookups at runtime.
- **Fail fast:** errors are caught at compile time (`javac` fails the build) instead of surfacing as a `RuntimeException` in production.
- **Works with strong encapsulation:** no `setAccessible` calls, so no fights with the module system or native-image tools like GraalVM, which struggle with runtime reflection.
- **IDE support:** generated code is visible, debuggable, and step-through-able, unlike a black-box reflective call.

Dynamic Proxies

A **dynamic proxy** is an object, generated at runtime, that implements one or more interfaces and forwards every method call to a handler you provide. It lets you add cross-cutting behavior (logging, timing, security checks, retries) around any interface without writing a wrapper class by hand for each one.

`Proxy.newProxyInstance` + `InvocationHandler`

```
import java.lang.reflect.*;

interface OrderService {
    void placeOrder(String item);
    int getOrderCount();
}

class OrderServiceImpl implements OrderService {
    private int count = 0;
    public void placeOrder(String item) {
        count++;
        System.out.println("Order placed: " + item);
    }
    public int getOrderCount() { return count; }
}
```

```

class LoggingTimingHandler implements InvocationHandler {
    private final Object target;

    LoggingTimingHandler(Object target) { this.target = target; }

    @Override
    public Object invoke(Object proxy, Method method, Object[] args)
    ⇨ throws Throwable {
        long start = System.nanoTime();
        System.out.println("Calling " + method.getName());
        try {
            return method.invoke(target, args); // forward to the real
    ⇨ object
        } finally {
            long elapsedMicros = (System.nanoTime() - start) / 1000;
            System.out.println(method.getName() + " took " +
    ⇨ elapsedMicros + " microseconds");
        }
    }

    public class ProxyDemo {
        public static void main(String[] args) {
            OrderService real = new OrderServiceImpl();

            OrderService proxy = (OrderService) Proxy.newProxyInstance(
                OrderService.class.getClassLoader(),
                new Class<?>[] { OrderService.class },
                new LoggingTimingHandler(real));

            proxy.placeOrder("Laptop");
            System.out.println("Count: " + proxy.getOrderCount());
        }
    }
}

```

Output looks like:

```

Calling placeOrder
Order placed: Laptop
placeOrder took 12 microseconds
Calling getOrderCount
getOrderCount took 3 microseconds
Count: 1

```

Interface-Only Limitation

`java.lang.reflect.Proxy` can only proxy **interfaces**, not concrete classes. This is a fundamental JDK limitation — the generated proxy class implements the given interfaces, it cannot extend an arbitrary class. If you need to proxy a class with no interface (common with plain Spring `@Service` classes that don't implement one), you need a bytecode-generation library instead.

How Frameworks Use This

- **Spring AOP**: when the target bean implements at least one interface, Spring uses **JDK dynamic proxies** (exactly the mechanism above) to implement things like `@Transactional`, `@Cacheable`, and custom aspects. When the bean has no interface (or `proxyTargetClass=true` is set), Spring falls back to **CGLIB** (bundled, repackaged inside `spring-core`), which generates a real subclass at runtime that overrides methods — this works on concrete classes but can't proxy `final` classes or `final` methods.
- **ByteBuddy** is a modern, actively maintained alternative to CGLIB for generating subclasses/bytecode at runtime; it's used internally by Mockito (for mocking classes, not just interfaces) and newer versions of Hibernate.

Mechanism	Can proxy interfaces?	Can proxy concrete classes?	Notes
JDK Proxy	Yes	No	Built into the JDK, no dependency needed
CGLIB	Yes (via subclass)	Yes, except <code>final</code> classes/methods	Bundled inside Spring; effectively unmaintained upstream
ByteBuddy	Yes	Yes, except <code>final</code> classes/methods	Modern, actively maintained, used by Mockito

Method Handles

A **method handle** is a typed, directly executable reference to an underlying method, constructor, or field access, created through the `java.lang.invoke` package. Method handles were introduced in Java 7 specifically to be a faster, more type-safe alternative to reflection for performance-sensitive dynamic dispatch — and the JVM itself uses them internally to implement lambdas and `invokedynamic`.

Lookup, MethodType, and Finding a Handle

```
import java.lang.invoke.*;

public class MethodHandleDemo {
    public static void main(String[] args) throws Throwable {
        MethodHandles.Lookup lookup = MethodHandles.lookup();

        // MethodType describes the signature: return type first, then
        ↪ parameter types
        MethodType stringLength = MethodType.methodType(int.class);
        MethodHandle lengthHandle = lookup.findVirtual(String.class,
        ↪ "length", stringLength);

        int len = (int) lengthHandle.invokeExact("hello"); // 5

        // findStatic for static methods
        MethodType parseIntType = MethodType.methodType(int.class,
        ↪ String.class);
        MethodHandle parseInt = lookup.findStatic(Integer.class,
        ↪ "parseInt", parseIntType);
        int n = (int) parseInt.invokeExact("42"); // 42

        System.out.println(len + n); // 47
    }
}
```

invokeExact vs invoke

- **invokeExact** requires the argument and return types at the call site to match the handle's MethodType *exactly* — no widening, no boxing conversions. If they don't match, it throws `WrongMethodTypeException`. This is the fast path.
- **invoke** is more forgiving: it performs the same kind of adaptation reflection would (widening primitives, boxing/unboxing, casting), at a small extra cost.

```
MethodHandle mh = MethodHandles.lookup()
    .findStatic(Math.class, "max", MethodType.methodType(int.class,
    ↪ int.class, int.class));

int a = (int) mh.invokeExact(3, 5); // works: types match exactly
// Object r = mh.invoke((Object) 3, (Object) 5); // invoke() would adapt
↪ this automatically
```

Why Method Handles Are Faster Than Reflection

- `Method.invoke` always boxes arguments into an `Object[]` and does a security/access check on every call (unless cached).
- A `MethodHandle`, once resolved, behaves much more like a direct call to the JIT compiler — it can be inlined, and the JVM has special bytecode support (`invokedynamic`) and call-site optimization for it. Repeated calls through the same `MethodHandle` get progressively faster as the JIT specializes them, approaching the speed of a hand-written call.

LambdaMetafactory: How Lambdas Are Actually Implemented

When you write a lambda expression, the compiler does **not** generate an anonymous inner class (unlike pre-Java-8 anonymous classes). Instead, it emits an `invokedynamic` instruction whose bootstrap method is `LambdaMetafactory.metafactory`. At runtime, the first time that lambda expression is executed, the JVM calls the metafactory, which uses method handles to generate a small hidden class implementing the target functional interface, and then caches it. Every subsequent execution of that same lambda reuses the cached class.

```
Runnable r = () -> System.out.println("hi");
```

compiles roughly to bytecode equivalent to:

```
// conceptual illustration – not what you write, what the JVM does
↪ internally
    CallSite site = LambdaMetafactory.metafactory(
        lookup,
        "run", // the functional
    ↪ interface's method name
        MethodType.methodType(Runnable.class), // factory signature
        MethodType.methodType(void.class), // signature of
    ↪ Runnable.run()
        implementationMethodHandle, // handle pointing at the
    ↪ lambda body
        MethodType.methodType(void.class));
    Runnable r = (Runnable) site.getTarget().invokeExact();
```

This is why lambdas are much cheaper than people expect: no class file per lambda at compile time (just one small generated class at runtime, cached), and calling a lambda is close to a direct method call once the JIT has warmed up — much faster than a reflective dispatch.

VarHandles

A `VarHandle` (Java 9+) is a typed reference to a variable — a field, an array element, or an off-heap memory location — that supports fine-grained control over how it's read and written, including atomic and memory-ordering operations. It was introduced to replace two older mechanisms: `sun.misc.Unsafe`'s direct memory-access methods, and the `java.util.concurrent.atomic.AtomicXFieldUpdater` classes (`AtomicIntegerFieldUpdater`, `AtomicLongFieldUpdater`, `AtomicReferenceFieldUpdater`), both of which had awkward or unsafe APIs.

Getting a VarHandle

```
import java.lang.invoke.*;

public class Counter {
    private volatile int count;

    private static final VarHandle COUNT_HANDLE;
    static {
        try {
            COUNT_HANDLE = MethodHandles.lookup()
                .findVarHandle(Counter.class, "count", int.class);
        } catch (ReflectiveOperationException e) {
            throw new ExceptionInInitializerError(e);
        }
    }

    public boolean incrementIfBelow(int max) {
        int current = (int) COUNT_HANDLE.getVolatile(this);
        while (current < max) {
            // atomically set count = current + 1, only if it's still ==
            ↪ current
            if (COUNT_HANDLE.compareAndSet(this, current, current + 1)) {
                return true;
            }
            current = (int) COUNT_HANDLE.getVolatile(this);
        }
        return false;
    }
}
```

Access Modes

`VarHandle` exposes several families of access, each with different performance/visibility trade-offs — from weakest/fastest to strongest/slowest guarantee:

Access mode	Guarantee	Typical method
Plain	No atomicity, no ordering guarantee beyond normal Java semantics — like a plain field read/write	<code>get</code> , <code>set</code>
Opaque	Atomic for that single variable, but no ordering relative to other variables (no happens-before)	<code>getOpaque</code> , <code>setOpaque</code>
Acquire/Release	One-directional memory fence — <code>acquire</code> prevents later reads/writes from moving before it; <code>release</code> prevents earlier ones from moving after it	<code>getAcquire</code> , <code>setRelease</code>
Volatile	Full happens-before ordering in both directions, same as a <code>volatile</code> field	<code>getVolatile</code> , <code>setVolatile</code>

```
// same variable, four different consistency guarantees
int plain    = (int) COUNT_HANDLE.get(counterInstance);
int opaque   = (int) COUNT_HANDLE.getOpaque(counterInstance);
int acquire  = (int) COUNT_HANDLE.getAcquire(counterInstance);
int volatileRead = (int) COUNT_HANDLE.getVolatile(counterInstance);
```

`compareAndSet` and Array Access

```
import java.lang.invoke.*;

public class ArrayHandleDemo {
    public static void main(String[] args) {
        int[] array = new int[10];
        VarHandle arrayHandle =
↪ MethodHandles.arrayElementVarHandle(int[].class);

        arrayHandle.setVolatile(array, 3, 42);
        boolean updated = arrayHandle.compareAndSet(array, 3, 42, 100);
        System.out.println(updated + " " + array[3]); // true 100
    }
}
```

Memory Fences

For advanced lock-free algorithms, `VarHandle` also exposes standalone fence methods that don't touch a specific variable but establish ordering for the surrounding code: `VarHandle.acquireFence()`, `releaseFence()`, `fullFence()`, and `loadLoadFence()/storeStoreFence()`. These map closely to CPU-level memory barrier instructions and are rarely needed outside of writing custom concurrency primitives (most application code should just use `java.util.concurrent` classes instead).

Unsafe Alternatives

`sun.misc.Unsafe` was an internal, undocumented JDK class that gave direct access to raw memory operations: allocating memory outside the heap, reading/writing at arbitrary memory offsets, compare-and-swap on fields by offset, throwing exceptions without declaring them, and even instantiating objects while skipping constructors. It was never part of the official Java API — it existed only because early `java.util.concurrent` and other JDK-internal code needed low-level primitives that didn't exist anywhere else yet, and it “leaked” out because it was technically public.

Why It's Being Removed

- It bypasses the type system and safety checks the JVM is supposed to guarantee, so misuse can crash the JVM instead of throwing a normal exception.
- It made **strong encapsulation** (from JPMS) meaningless for anyone using it, since it can reach into any object's memory layout directly.
- **JEP 471** (targeted for a recent JDK release) deprecates and eventually removes `Unsafe`'s memory-access methods for removal, pushing everyone toward the supported replacements below.
- **JEP 498** relates to further restricting/warning about deep reflective and unsafe memory access as part of the broader multi-release effort to close off “back door” access to internals (following on from JEP 403's strong encapsulation of JDK internals started in JDK 17).

The practical effect for a codebase: any dependency still calling `Unsafe.putInt`, `Unsafe.allocateInstance`, etc., will eventually stop working on newer JDKs, and code reviewers should flag any *new* direct use of `sun.misc.Unsafe` in application code as a serious red flag.

Modern Replacements

Old Unsafe use case	Modern replacement
Atomic field access, CAS on a field by offset	VarHandle (via MethodHandles. lookup().findVarHandle(...))
Fast, checked-exception-free-ish reflective invocation	MethodHandle
Off-heap memory allocation, pointers, native calls	Foreign Function & Memory API (java. lang.foreign, finalized in Java 22, usable as a preview from Java 19+)
Running cleanup logic when an object becomes unreachable (replacing finalize() and ad hoc Unsafe-based tricks)	java.lang.ref.Cleaner
Creating a class at runtime without exposing it as a discoverable, loadable, or reflectively accessible named class (used internally for lambdas)	Hidden classes (MethodHandles.Lookup. defineHiddenClass, Java 15+)

```
import java.lang.foreign.*;

public class ForeignMemoryDemo {
    public static void main(String[] args) {
        try (Arena arena = Arena.ofConfined()) {
            ↪ off-heap
            MemorySegment segment = arena.allocate(4); // 4 bytes,

            ↪ 42
            segment.set(ValueLayout.JAVA_INT, 0, 42);
            System.out.println(segment.get(ValueLayout.JAVA_INT, 0)); //

            } // memory is freed automatically when the arena closes
        }
    }
}
```

```
import java.lang.ref.Cleaner;

public class ResourceHolder implements AutoCloseable {
    private static final Cleaner CLEANER = Cleaner.create();

    private final Cleaner.Cleanable cleanable;

    public ResourceHolder() {
        // register cleanup logic to run if close() is never called and
        ↪ the object is GC'd
        this.cleanable = CLEANER.register(this, () ->
        ↪ System.out.println("cleaning up native resource"));
    }
}
```

```

@Override
public void close() {
    cleanable.clean(); // idempotent – safe to call once explicitly,
    ↪ GC won't run it twice
}
}

```

Service Provider Interface (SPI)

SPI is a pattern built into the JDK for writing pluggable applications: you define an interface (the “service”), other jars provide implementations (the “providers”), and your application discovers those implementations at runtime **without knowing their class names in advance**. This is how JDBC drivers, Charset providers, and many logging bridges plug themselves into applications.

ServiceLoader and META-INF/services

The classic mechanism, working since Java 6, is a text file under `META-INF/services/` named exactly after the fully-qualified interface name, containing one implementation class name per line.

1. Define the service interface:

```

package com.example.spi;

public interface GreetingProvider {
    String greet(String name);
}

```

2. Write two implementations, in the same or different jars:

```

package com.example.spi.impl;
import com.example.spi.GreetingProvider;

public class FormalGreetingProvider implements GreetingProvider {
    public String greet(String name) { return "Good day, " + name + "."; }
}

```

```
package com.example.spi.impl;
import com.example.spi.GreetingProvider;

public class CasualGreetingProvider implements GreetingProvider {
    public String greet(String name) { return "Hey " + name + "!"; }
}
```

3. Register both in a resource file:

```
src/main/resources/META-INF/services/com.example.spi.GreetingProvider
```

containing:

```
com.example.spi.impl.FormalGreetingProvider
com.example.spi.impl.CasualGreetingProvider
```

4. Load and use them:

```
import java.util.ServiceLoader;
import com.example.spi.GreetingProvider;

public class SpiDemo {
    public static void main(String[] args) {
        ServiceLoader<GreetingProvider> loader =
        ↪ ServiceLoader.load(GreetingProvider.class);
        for (GreetingProvider provider : loader) {
            System.out.println(provider.greet("Ada"));
        }
    }
}
```

Output:

```
Good day, Ada.
Hey Ada!
```

Neither the interface nor `SpiDemo` needed to know the implementation class names — adding a third provider jar to the classpath, with its own `META-INF/services` entry, would make it show up automatically, with no code changes.

The Module System Way: provides ... with

If you're using JPMS (the Java Module System), the equivalent declaration goes in `module-info.java` instead of a text file, and is checked at compile time:

```
// in the module that declares the service interface
module com.example.spi {
    exports com.example.spi;
    uses com.example.spi.GreetingProvider;
}
```

```
// in the module that provides an implementation
module com.example.spi.impl {
    requires com.example.spi;
    provides com.example.spi.GreetingProvider
        with com.example.spi.impl.FormalGreetingProvider,
    ↪ com.example.spi.impl.CasualGreetingProvider;
}
```

`ServiceLoader.load(...)` works exactly the same way at the call site — it transparently checks both `module-info.java` declarations and legacy `META-INF/services` files.

Real JDK Uses

- **JDBC drivers:** since JDBC 4.0, drivers register themselves via `META-INF/services/java.sql.Driver`, which is why modern code doesn't need `Class.forName("com.mysql.cj.jdbc.Driver")` anymore — `DriverManager` uses `ServiceLoader` internally to find it just by having the jar on the classpath.
- **java.nio.charset.spi.CharsetProvider:** lets libraries add custom character encodings that `Charset.forName(...)` can then find.
- **Logging bridges:** SLF4J and the Java Logging API use SPI-like discovery to find and bind the actual logging backend (Logback, Log4j2, etc.) at startup, without the application code referencing a specific logging implementation directly.

Common Code-Review Interview Pitfalls

1. **Using `Class.forName(...)` or reflection for a class that's already known at compile time.** Why it matters: this throws away compile-time type safety and IDE

support for no benefit — reflection should be reserved for cases where the target type genuinely isn't known until runtime.

```
// Before
Object svc = Class.forName("com.example.OrderService").j
→ getDeclaredConstructor().newInstance();
// After
OrderService svc = new OrderService();
```

- 2. Calling `setAccessible(true)` on a field or method just to avoid adding a getter/constructor parameter.** Why it matters: it silently breaks encapsulation, is fragile across refactors (renaming the field breaks it with no compile error, just a runtime `NoSuchFieldException`), and will fail outright with `InaccessibleObjectException` on modularized JDK internals starting with Java 17.

```
// Before
Field f = obj.getClass().getDeclaredField("balance");
f.setAccessible(true);
double balance = (double) f.get(obj);
// After
double balance = obj.getBalance();
```

- 3. Not caching Method/Field/Constructor lookups that happen inside a loop or a hot request path.** Why it matters: `getDeclaredMethod`, `getField`, etc. are relatively expensive lookups; doing them on every call instead of once at startup can dominate the cost of an otherwise fast method.

```
// Before
for (Order o : orders) {
    Method m = o.getClass().getMethod("getTotal");
    sum += (double) m.invoke(o);
}
// After
Method m = Order.class.getMethod("getTotal"); // looked up once,
→ outside the loop
for (Order o : orders) sum += (double) m.invoke(o);
```

- 4. Assuming `RetentionPolicy.CLASS` (the default) annotations are readable via reflection.** Why it matters: only `RUNTIME`-retained annotations are visible to `getAnnotation(...)`; forgetting `@Retention(RetentionPolicy.RUNTIME)` makes `isAnnotationPresent` silently return `false` at runtime with no compile error.

```

// Before – no @Retention at all defaults to CLASS, invisible at
↳ runtime
public @interface Auditable { }
// After
@Retention(RetentionPolicy.RUNTIME)
public @interface Auditable { }

```

5. **Trying to create a JDK dynamic proxy for a class instead of an interface.** Why it matters: `java.lang.reflect.Proxy` only supports interfaces; attempting to pass a concrete class to `newProxyInstance` throws `IllegalArgumentException` — the candidate needs to know to reach for CGLIB/ByteBuddy (or Spring's `proxyTargetClass=true`) instead.

```

// Before – throws IllegalArgumentException at runtime
Proxy.newProxyInstance(cl, new Class<?>[]{ OrderServiceImpl.class },
↳ handler);
// After – proxy the interface it implements
Proxy.newProxyInstance(cl, new Class<?>[]{ OrderService.class },
↳ handler);

```

6. **Forgetting that a Spring `@Transactional` (or similar AOP-advised) method call from *within the same class* bypasses the proxy.** Why it matters: AOP proxies (whether JDK or CGLIB) only intercept calls that come in through the proxy object from the *outside*; an internal `this.otherMethod()` call skips the proxy entirely, silently disabling the transaction, caching, or security check.

```

// Before – internal call bypasses the transactional proxy
public void placeOrder() {
    this.chargeCard(); // @Transactional on chargeCard() has no effect
↳ here
}
// After – inject/call through the bean so the proxy intercepts it
public void placeOrder() {
    orderServiceSelfProxy.chargeCard();
}

```

7. **Mixing up `invoke` and `invokeExact` on a `MethodHandle` and expecting automatic type adaptation.** Why it matters: `invokeExact` requires the exact `MethodType`, including matching primitive vs. boxed types; a mismatch throws `WrongMethodTypeException` at runtime instead of a compile error, which is easy to miss in review since the code compiles fine.

```

// Before - mh's MethodType is (int)int, throws
→ WrongMethodTypeException
Object result = mh.invokeExact((Object) 5);
// After
int result = (int) mh.invokeExact(5);

```

- 8. Using AtomicIntegerFieldUpdater/AtomicLongFieldUpdater in new code instead of VarHandle.** Why it matters: the field-updater classes are older, more error-prone (the field must be exactly `volatile` and non-private relative to the updater's caller, with easy-to-miss reflective setup), and `VarHandle` supersedes them with clearer semantics and better performance.

```

// Before
private static final AtomicIntegerFieldUpdater<Counter> UPDATER =
    AtomicIntegerFieldUpdater.newUpdater(Counter.class, "count");
// After
private static final VarHandle COUNT_HANDLE;
static {
    try { COUNT_HANDLE =
→ MethodHandles.lookup().findVarHandle(Counter.class, "count",
→ int.class); }
    catch (ReflectiveOperationException e) { throw new
→ ExceptionInInitializerError(e); }
}

```

- 9. Using `getVolatile`/`setVolatile` on a `VarHandle` when a weaker (and cheaper) access mode like `getOpaque` or plain access would suffice.** Why it matters: full volatile semantics impose a memory fence on every access; if the algorithm doesn't actually need cross-thread ordering guarantees for that particular read, using the strongest mode by default is a needless performance cost, and interviewers look for awareness that access modes are a real trade-off, not just decoration.

```

// Before - full fence on every read, even for a value only this thread
→ touches
int v = (int) handle.getVolatile(obj);
// After - plain access when there's no cross-thread visibility
→ requirement
int v = (int) handle.get(obj);

```

- 10. Any new code calling `sun.misc.Unsafe` directly.** Why it matters: `Unsafe` is an unsupported internal API being deprecated for removal (JEP 471/498); it bypasses

the type system, breaks under strong encapsulation, and has fully supported modern replacements — flagging this in review prevents a maintenance/upgrade landmine.

```
// Before
unsafe.putInt(obj, offset, 42);
// After
intHandle.set(obj, 42); // via VarHandle
```

11. Registering a ServiceLoader provider without a public no-arg constructor.

Why it matters: ServiceLoader instantiates providers reflectively via a public no-arg constructor by default; a provider with only a parameterized constructor (or a private one) fails at load time with ServiceConfigurationError, often discovered only in production when that particular provider is finally needed.

```
// Before – no no-arg constructor, ServiceConfigurationError at load
↳ time
public class DbGreetingProvider implements GreetingProvider {
    public DbGreetingProvider(DataSource ds) { ... }
}
// After – provide a public no-arg constructor, or a static
↳ "provider()" factory method
public class DbGreetingProvider implements GreetingProvider {
    public DbGreetingProvider() { this.ds =
↳ DataSourceRegistry.getDefault(); }
}
```

12. Mismatched or misspelled META-INF/services file name vs. the actual interface's fully-qualified name. Why it matters: the file name must exactly match the service interface's fully-qualified class name; a typo or a stale name after a package rename means ServiceLoader.load(...) silently finds zero providers instead of failing loudly, which is confusing to debug.

```
# Before – file named for the old package after a refactor
META-INF/services/com.example.old.GreetingProvider
# After – matches the current interface location
META-INF/services/com.example.spi.GreetingProvider
```

13. Writing a custom annotation processor (or Lombok-style tool) that mutates behavior at runtime via reflection when compile-time code generation would work and be far faster. Why it matters: reflection-based runtime frameworks pay

a lookup/invoke cost on every use and fight the module system's strong encapsulation; compile-time generation (like MapStruct, Dagger, Immutables) produces plain, fast, debuggable Java with the same errors caught at build time instead of at runtime.

```
// Before – reflective field copy at runtime, every call
for (Field f : Dto.class.getDeclaredFields()) { f.setAccessible(true);
→ ... }
// After – a MapStruct-generated mapper compiled ahead of time
Dto dto = DtoMapper.INSTANCE.toDto(entity);
```

- 14. Assuming @Inherited propagates an annotation across methods or fields, not just class hierarchies.** Why it matters: @Inherited only affects whether a *class-level* annotation is visible on subclasses via `getAnnotation`; it has no effect on method or field annotations, so code that expects an overridden method to “inherit” an annotation from the parent’s method will get `null` back.

```
// Before – expecting method-level @Inherited to work
@Retention(RetentionPolicy.RUNTIME)
@Inherited
@Target(ElementType.METHOD)
@interface Cached { }
// Derived.class.getMethod("compute").getAnnotation(Cached.class) still
→ returns null
// After – re-annotate the overriding method explicitly, or check the
→ superclass method too
class Derived extends Base {
    @Cached
    @Override
    void compute() { ... }
}
```

- 15. Ignoring Proxy.isProxyClass(...) / unwrapping needs when logging, serializing, or type-checking an object that might be a dynamic proxy.** Why it matters: a dynamic proxy’s `getClass()` returns a synthetic proxy class, not the real implementation type; code that does `instanceof SomeConcreteClass` or relies on `getClass().getSimpleName()` for logging can behave unexpectedly when handed a proxied bean (common with Spring AOP-advised beans).

```
// Before – logs a generated proxy class name like "$Proxy42", not the
→ real service
log.info("Handler: {}", handler.getClass().getSimpleName());
```

```
// After – use the interface, or AopUtils.getTargetClass(handler) in  
↳ Spring  
    log.info("Handler: {}",  
    ↳ AopUtils.getTargetClass(handler).getSimpleName());
```


Modules & Native Interoperability

Java code does not live in a vacuum. It is organized into **packages**, packages are (optionally) organized into **modules**, and sometimes Java code needs to reach outside the JVM entirely to call **native code** or squeeze more performance out of the CPU with **vectorized** operations. This chapter walks from the oldest and most universal organizational unit (the package, present since Java 1.0) to the newest, still-incubating performance API (the Vector API). Along the way we cover the Java Platform Module System (JPMS, Java 9+) in depth, and the modern replacement for JNI: the Foreign Function & Memory API (finalized in JDK 22). We target Java 21+ throughout, calling out version-specific behavior where it matters.

- Packages
 - Naming Conventions and Reverse-Domain Rules
 - The package and `import` Statements
 - Static Imports
 - Wildcard Imports
 - The Default (Unnamed) Package
 - Package-Private Visibility as a Design Tool
 - `package-info.java`
 - Split Packages
 - Directory-to-Package Mapping
 - Packages vs Modules
- Java Platform Module System (JPMS)
 - Problems JPMS Solves
 - `module-info.java` Basics

- `requires`, `requires transitive`, `requires static`
- `exports` and `exports ... to`
- `opens` and `opens ... to`
- `uses` and `provides ... with`
- Automatic Modules and the Unnamed Module
- Module Path vs Class Path
- Readability and Accessibility Rules
- `jlink`, `jdeps`, and `jmod`
- Strong Encapsulation and `--add-exports/--add-opens`
- Worked Multi-Module Example
- Migration Strategy
- Why Many Projects Still Don't Use JPMS
- Foreign Function & Memory API
 - From JNI to FFM: Why It Changed
 - Arena and Deterministic Deallocation
 - `MemorySegment`, `MemoryLayout`, `ValueLayout`
 - `VarHandle` Access into Segments
 - Calling a C Function: `Linker`, `SymbolLookup`, `FunctionDescriptor`
 - Upcalls: Java Callback into C
 - Struct Layouts
 - `jextract`
 - `--enable-native-access` and Restricted Methods
 - FFM vs JNI Comparison Table
- Vector API (Incubator)
 - Why It's Still Incubating
 - `VectorSpecies`, Lanes, and Masks
 - Enabling the Incubator Module
 - Worked Example: Vectorized Dot Product vs Scalar
 - The Loop-Plus-Tail Idiom

- [Dependence on Project Valhalla](#)
- [Production Readiness Caveat](#)
- [Common Code-Review Interview Pitfalls](#)

Packages

A **package** is a namespace that groups related classes and interfaces together, and it has been part of Java since version 1.0. Packages solve name collisions (two libraries can each have a `Logger` class as long as they're in different packages) and provide a coarse-grained visibility boundary through package-private access.

Naming Conventions and Reverse-Domain Rules

Package names are conventionally all-lowercase and follow the **reversed internet domain name** of the organization that owns the code, to guarantee global uniqueness without a central registry.

```
// Domain: teampicnic.com -> reversed -> com.teampicnic
package com.teampicnic.orders.billing;

public class InvoiceCalculator {
    // ...
}
```

Rule	Example	Reason
All lowercase	com.teampicnic. orders, not com. TeamPicnic.Orders	Avoids case-sensitivity clashes on some filesystems and mixed-case ambiguity
Reverse domain as prefix	com.example.app for example.com	Guarantees uniqueness without a central registry
No Java reserved words as segments	Never com.example. class	class, int, package etc. are illegal identifiers
Avoid underscores/digits at segment start	v2.api is illegal; use api.v2 or api.v2	Java identifier rules apply to each segment

Rule	Example	Reason
Segments describe increasingly specific scope	<code>com.teampicnic.orders.billing.tax</code>	Mirrors directory structure and narrows responsibility

Since JDK 11, `com.sun.tools.javac` and similar internal packages remind us why the convention matters: the JDK itself uses `java.*` and `javax.*`, reserved prefixes that application code must never use, to avoid colliding with future JDK classes.

The package and import Statements

The package declaration must be the first non-comment line in a source file, and a file can declare at most one package.

```
package com.teampicnic.orders.billing;

import java.math.BigDecimal;
import java.time.LocalDate;
import com.teampicnic.orders.catalog.Product;

public class Invoice {
    private final Product product;
    private final BigDecimal amount;
    private final LocalDate issuedOn;

    public Invoice(Product product, BigDecimal amount, LocalDate
↪ issuedOn) {
        this.product = product;
        this.amount = amount;
        this.issuedOn = issuedOn;
    }
}
```

`import` is purely a compile-time convenience: it lets you use a simple name (`Product`) instead of the fully qualified name (`com.teampicnic.orders.catalog.Product`) everywhere in the file. It has **zero runtime cost** — the compiled bytecode always references fully qualified names, and `import` statements produce nothing in the `.class` file.

Two classes are automatically visible without any `import`:

- Everything in `java.lang` (e.g., `String`, `Object`, `Math`) is imported implicitly.

- Classes in the *same package* as the current file need no import at all.

Static Imports

A **static import** brings static members (fields or methods) into scope so you can use them unqualified, without the enclosing class name.

```
import static java.util.Collections.emptyList;
import static java.lang.Math.PI;
import static java.lang.Math.pow;

public class CircleMath {
    public static double area(double radius) {
        return PI * pow(radius, 2); // no "Math." prefix needed
    }

    public static <T> java.util.List<T> noResults() {
        return emptyList(); // no "Collections." prefix needed
    }
}
```

Static imports are genuinely useful for well-known idioms — `assertEquals` in JUnit tests, `Math` constants, or `Collectors` factory methods in stream pipelines — where the qualifying class name adds noise without adding clarity.

When they hurt readability: overusing static imports removes the reader's ability to tell, at a glance, *where a symbol comes from*. A bare `of(1, 2, 3)` could be `List.of`, `Set.of`, `Stream.of`, or a custom factory — the reader has to scroll up to the imports to find out.

```
// Avoid: which "of" is this? Which "process" is this?
import static com.teampicnic.orders.OrderUtils.*;
import static com.teampicnic.billing.BillingUtils.*;

public class Reconciler {
    void run() {
        var order = of(42);           // OrderUtils.of or BillingUtils.of?
        ↪ Have to check imports.
        process(order);               // ambiguous at a glance
    }
}
```

Rule of thumb: `static-import` only widely recognized, unambiguous helpers (`Collectors.*`, `test assertion libraries`, `Math.*`), and avoid it for your own domain classes where the qualifying name carries meaning.

Wildcard Imports

`import com.teampicnic.orders.*`; imports every top-level type directly in that package (not sub-packages).

```
// Wildcard: convenient, but hides exactly what's being pulled in
import com.teampicnic.orders.*;

public class OrderProcessor {
    Order create() { return new Order(); }
}
```

```
// Explicit: IDE-generated, unambiguous, and diffs cleanly in code review
import com.teampicnic.orders.Order;

public class OrderProcessor {
    Order create() { return new Order(); }
}
```

	Wildcard import pkg.*;	Explicit import pkg.Type;
Compile-time cost	Identical — resolved once at compile time either way	Identical
Readability	Reader must infer which types are actually used	Every used type is listed
Merge conflicts	Fewer lines, but ambiguous diffs	More lines, but precise diffs
Name collisions	Can silently break when the package adds a new type with a name you already use	Immune — each import is explicit
Common tooling default	Most style guides (Google Java Style, and most IDEs by default) disallow it	Preferred by nearly every style guide

Wildcard imports are almost universally discouraged in professional style guides and in code review, precisely because they trade a small amount of typing for a real risk: if

the imported package later adds a class with the same simple name as something else you're using, your code can silently start resolving to the wrong type, or fail to compile with an ambiguity error far from where the real problem is.

The Default (Unnamed) Package

A source file with no package statement lives in the **default (unnamed) package**.

```
// No package declaration at all – lives in the default package
public class QuickTest {
    public static void main(String[] args) {
        System.out.println("Hello from the default package");
    }
}
```

Why to never use it in real code:

1. **It cannot be imported.** Classes in the default package are invisible to any class that belongs to a named package, since `import` requires a package-qualified name.
2. **It cannot participate in JPMS.** A module cannot export the default package, so any type there is entirely inaccessible from a modular application.
3. **Name collisions are almost guaranteed** as soon as the project grows — two unrelated classes both wanting to be called `Utils` will collide immediately with no namespace to separate them.
4. **Build tools and JavaDoc treat it as second class** — many tools assume every meaningful class has a package and behave oddly (or refuse to process the class at all) otherwise.

The default package is acceptable for a one-off scratch file (`Test.java` you compile and delete) and nothing else. Every class that will be checked in should declare a package.

Package-Private Visibility as a Design Tool

When no access modifier is written, a member or top-level class has **package-private** (also called *default*) visibility: accessible only to code in the exact same package.

```

package com.teampicnic.orders.billing;

public class InvoiceService {           // public API entry point
    private final TaxCalculator tax = new TaxCalculator();

    public java.math.BigDecimal totalFor(Invoice invoice) {
        return tax.calculate(invoice); // package-private class used
    }
    ↪ internally
}

class TaxCalculator {                  // package-private: not part of the
    ↪ public API
    java.math.BigDecimal calculate(Invoice invoice) {
        return
    }
    ↪ invoice.amount().multiply(java.math.BigDecimal.valueOf(1.2));
}

```

This is a deliberate design tool, not just “the case where you forgot `public`.” Keeping helper classes package-private lets you refactor their internals freely — rename methods, change constructors, delete the class entirely — without breaking any code outside the package, because the compiler guarantees no outside code could have depended on it. This is the same idea JPMS later formalizes at a coarser grain with `exports`: expose the minimum surface area needed, and hide the rest.

A common pattern: one `public` class or interface per package acting as the facade, and everything else package-private, wired together only within that package.

package-info.java

`package-info.java` is a special file, one per package, that holds package-level Javadoc and package-level annotations. It contains no class body — just an optional Javadoc comment, an optional set of annotations, and the package statement.

```

/**
 * Billing calculations for customer invoices, including tax and currency
 * conversion rules. All monetary values use {@link java.math.BigDecimal}
 * with {@code RoundingMode.HALF_UP} unless documented otherwise.
 *
 * @since 2.3
 */
@NonNullApi // hypothetical package-wide annotation, e.g. from Spring or a
    ↪ custom checker

```



```
package com.teampicnic.orders.billing;

import com.teampicnic.orders.NonNullApi;
```

Two practical uses reviewers should recognize:

- **Documentation** — Javadoc generated for the package overview page comes from here, not from any regular class’s comment.
- **Package-wide annotations** — a `@NonNullApi`-style annotation (like Spring’s `org.springframework.lang.NonNullApi`) applied here means “every parameter and return type in this package is non-null unless explicitly marked `@Nullable`,” saving you from annotating every single method individually.

Split Packages

A **split package** occurs when the *same* package name is provided by more than one JAR (or module) on the classpath/module path. On the plain classpath this “just works” in a fragile way — classes get merged from whichever JAR the classloader finds first, in an order that’s often undocumented and build-tool-dependent.

```
lib-a.jar
  com/example/util/StringHelper.class

lib-b.jar
  com/example/util/StringHelper.class    <- same package AND same class
↳ name, different code!
```

On the classpath, whichever JAR appears first wins silently — no error, no warning, just whichever version happened to load, which is exactly the kind of bug that’s nearly impossible to diagnose from a stack trace alone.

JPMS makes split packages a hard error. Two named modules are not permitted to export the same package; the module system refuses to resolve at launch:

```
Error occurred during initialization of boot layer
java.lang.module.ResolutionException: Module lib.a and module lib.b export
package com.example.util to module myapp
```

This is a deliberate trade-off: JPMS trades classpath flexibility for a hard compile/launch-time guarantee that a package’s contents are unambiguous. It is one of the most common real-world blockers when modularizing an existing multi-JAR project — you often

discover, for the first time, that two of your dependencies quietly ship overlapping packages.

Directory-to-Package Mapping

The Java compiler and JVM require that a class's package name **exactly mirror its directory path** relative to a source or class root.

```
src/
├─ main/
│   └─ java/
│       └─ com/
│           └─ teampicnic/
│               └─ orders/
│                   └─ billing/
│                       └─ Invoice.java          -> package
↳ com.teampicnic.orders.billing;
│                       └─ InvoiceService.java  -> package
↳ com.teampicnic.orders.billing;
```

```
# Compiling from the source root, mirroring the package structure,
↳ produces
# matching directories under the output root:
javac -d out $(find src/main/java -name "*.java")

# out/com/teampicnic/orders/billing/Invoice.class
# out/com/teampicnic/orders/billing/InvoiceService.class
```

This mapping is not a convention the compiler merely prefers — it is enforced. If `Invoice.java` inside a directory called `billing/` declared package `com.teampicnic.orders.tax;`, `javac` would fail to compile it (or, on the classpath at runtime, the class simply couldn't be found where the classloader expects it).

Packages vs Modules

Packages and modules solve related but distinct problems, and mixing them up is a common interview stumbling point.

	Package	Module
Introduced	Java 1.0	Java 9
Unit of	Namespace + package-private visibility	Deployment, strong encapsulation, and dependency declaration
Declared in	package statement at top of each .java file	Single <code>module-info.java</code> per module (a JAR-level concern)
Visibility granularity	Fine-grained (per-class, per-member)	Coarse-grained (per-package, via <code>exports/opens</code>)
Can exist without the other?	Yes — packages predate and work fully without modules	A module is <i>composed of</i> one or more packages; it can't exist without them
Enforces uniqueness across dependencies?	No (split packages silently merge on the classpath)	Yes (split packages are a hard error)
Reflection access to internals	Always allowed (subject to <code>SecurityManager</code> , now removed)	Blocked unless the package is <code>opens</code> -ed

A module *contains* packages; it never replaces them. You still write `package com.teampicnic.orders.billing`; at the top of every file exactly as before — JPMS adds one more file, `module-info.java`, that declares which of those packages the module exposes to the outside world.

Java Platform Module System (JPMS)

JPMS (JSR 376, delivered in Java 9 via Project Jigsaw) introduced modules: named, self-describing units that declare their dependencies and their public surface explicitly, instead of relying on an implicit, all-or-nothing classpath.

Problems JPMS Solves

Problem	Pre-JPMS reality	JPMS fix
Classpath hell	Every JAR's classes are dumped into one flat namespace; duplicate/conflicting versions silently shadow each other; no dependency graph is declared anywhere	Modules declare explicit <code>requires</code> ; the module system builds and validates a real dependency graph at launch, failing fast on missing or conflicting modules

Problem	Pre-JPMS reality	JPMS fix
No strong encapsulation	public meant public to <i>everyone</i> on the classpath, forever, even for classes meant as “internal use only” (sun.misc.Unsafe, com.sun.*)	exports controls which packages are visible outside the module at compile time <i>and</i> runtime; unexported packages are invisible, full stop
Monolithic JDK	The entire JDK shipped as one giant rt.jar, so even a “Hello World” needed the full runtime, and internal JDK classes were reachable via reflection by anyone	The JDK itself is split into ~70 modules (java.base, java.sql, java.desktop, ...); jlink builds a runtime image with only the modules you actually need
Reflection reaching into internals	Any code could call setAccessible(true) and reach into private JDK fields/methods with no declared contract	opens is required for deep reflection; without it, setAccessible throws InaccessibleObjectException

module-info.java Basics

Every module has exactly one module-info.java at the root of its source tree (not inside any package — it sits directly under the module’s source root, alongside the top-level package directories).

```
module com.teampicnic.billing {
    requires java.sql;
    exports com.teampicnic.billing.api;
}
```

- The module name (com.teampicnic.billing) conventionally follows the same reverse-domain convention as packages, but it is a **separate namespace** from package names — a module is not required to contain a package of the exact same name, though it’s a common and readable convention to have one “root” package matching the module name.
- Everything inside the module’s packages is invisible outside the module **unless** explicitly exports-ed.

requires, requires transitive, requires static

requires declares a compile-time and run-time dependency on another module.

```
module com.teampicnic.billing {
    requires java.sql;                // plain dependency
```

```

    requires transitive com.teampicnic.core; // re-exported to consumers
    ↪ of this module
    requires static com.teampicnic.testkit; // compile-time only,
    ↪ optional at runtime
  }

```

Form	Meaning	Typical use
<code>requires X;</code>	This module needs X to compile and run.	The normal case for a real dependency
<code>requires transitive X;</code>	This module needs X, and any module that <code>requires com.teampicnic.billing</code> automatically also gets readability to X without declaring it itself.	Your module's public API exposes types from X (e.g., a method returns <code>java.sql.Connection</code>), so callers need X too
<code>requires static X;</code>	This module needs X only to compile ; it is optional at runtime — if X is absent when running, the module still loads, but any code path that actually touches X's types fails at that point.	Compile-time-only annotations (like <code>org.jetbrains.annotations.Nullable</code>) that aren't needed once compiled

`requires transitive` is the module-system equivalent of a Maven “compile-scope, exposed” dependency: if `Invoice.getConnection()` returns a `java.sql.Connection`, then any caller needs `java.sql` on their module path too — `requires transitive java.sql`; in your module spares every consumer from having to add that requires themselves.

exports and exports ... to

`exports` makes a package's public types accessible (at compile time and runtime) to any module that `requires` this module.

```

module com.teampicnic.billing {
    exports com.teampicnic.billing.api; // visible to
    ↪ everyone
    exports com.teampicnic.billing.internal to
    ↪ com.teampicnic.billing.tests; // visible only to named module(s)
}

```

- Plain exports `pkg`; — anyone who `requires` your module can see `pkg`'s public types.

- **Qualified export** exports pkg to moduleA, moduleB; — only the named modules can see it. This is how you can share internal-but-not-fully-public code with, say, your own test module or a sibling module, without making it part of your general public API.

Packages that are *not* exported at all are invisible outside the module even though their classes may be public — this is the core of JPMS’s strong encapsulation, and it is enforced by the compiler and the runtime, not just convention.

opens and opens ... to

opens is about **reflection**, not compile-time linking. A package can be exported (visible, linkable, callable normally) without being opened, and vice versa.

```
module com.teampicnic.billing {
    exports com.teampicnic.billing.api;
    opens com.teampicnic.billing.model;           // deep reflection
↪ allowed for everyone
    opens com.teampicnic.billing.internal to
↪ com.fasterxml.jackson.databind; // reflection allowed only for Jackson
}
```

	exports	opens
Grants normal compile-time + runtime access (calling public methods, newing public classes)	Yes	No
Grants deep reflection (setAccessible(true) on private members)	No	Yes
Typical need	Library public API	JSON/serialization frameworks (Jackson, Gson), dependency injection (Spring, Hibernate) that reflectively construct and populate your classes, including private fields

Frameworks like Hibernate and Jackson need opens (not just exports) on your model/entity packages, because they use reflection to set private fields directly. A module can also be declared open entirely — open module com.teampicnic.billing { requires java.sql; exports com.teampicnic.billing.api; } — which opens *every* package in the module to reflection at once, a pragmatic

shortcut when a module is mostly framework-managed POJOs and enumerating each package individually isn't worth the ceremony.

uses and provides ... with

These two directives implement the **ServiceLoader** pattern at the module level — a consumer declares it *uses* a service interface, and a provider module declares it *provides* an implementation, without either module needing to know about the other directly.

```
// Module: com.teampicnic.billing.api (declares the service interface)
module com.teampicnic.billing.api {
    exports com.teampicnic.billing.api;
}
```

```
package com.teampicnic.billing.api;

public interface TaxRuleProvider {
    java.math.BigDecimal rateFor(String countryCode);
}
```

```
// Module: com.teampicnic.billing.eu (provides an implementation)
module com.teampicnic.billing.eu {
    requires com.teampicnic.billing.api;
    provides com.teampicnic.billing.api.TaxRuleProvider
        with com.teampicnic.billing.eu.EuTaxRuleProvider;
}
```

```
// Module: com.teampicnic.billing.engine (consumes the service)
module com.teampicnic.billing.engine {
    requires com.teampicnic.billing.api;
    uses com.teampicnic.billing.api.TaxRuleProvider;
}
```

```
package com.teampicnic.billing.engine;

import com.teampicnic.billing.api.TaxRuleProvider;
import java.util.ServiceLoader;

public class TaxEngine {
    public java.math.BigDecimal calculate(String countryCode) {
```

```
        ServiceLoader<TaxRuleProvider> loader =
↪ ServiceLoader.load(TaxRuleProvider.class);
        for (TaxRuleProvider provider : loader) {
            return provider.rateFor(countryCode);
        }
        throw new IllegalStateException("No TaxRuleProvider found on the
↪ module path");
    }
}
```

uses and provides predate JPMS as an idea (ServiceLoader and META-INF/services/ files existed since Java 6), but JPMS makes the wiring declarative and verifiable at compile time: the module system checks that a provides implementation actually implements the declared interface and that consumers legitimately declare uses for what they load, instead of relying on a text file in META-INF/services/ that nothing validates.

Automatic Modules and the Unnamed Module

Not every JAR on your module path has a module-info.class. JPMS handles this with two fallback mechanisms:

	Automatic module	Unnamed module
What it is	A plain JAR (no module-info.class) placed on the module path	Anything placed on the plain classpath when running a modular application
Module name	Derived from the JAR filename (e.g., guava-33.0.0.jar -> guava), or from Automatic-Module-Name in META-INF/MANIFEST.MF if present	Has no name; cannot be required by name
What it exports	Everything (all packages, to everyone) — no encapsulation	Everything, to anyone else in the unnamed module
What it reads	Reads every other module, automatically (requires any named module works without listing it)	Reads all named modules and the rest of the classpath
Typical use	Bridging a legacy, non-modular JAR into a modular application	Legacy applications that haven't adopted JPMS at all

Automatic modules are the “duct tape” that made JPMS adoption practical: the vast majority of the Java ecosystem’s JARs in 2017 had no module descriptor, and automatic modules let a modular application depend on them immediately, with the tradeoff that automatic modules provide none of JPMS’s actual encapsulation guarantees.


```
# guava-33.0.0.jar has no module-info.class, but placing it on the module
↳ path
# turns it into the automatic module "com.google.common" (from its
# Automatic-Module-Name manifest entry), usable via a plain "requires":
java -p libs:out -m com.teampicnic.app/com.teampicnic.app.Main
```

Module Path vs Class Path

	Class path (-cp / -classpath)	Module path (-p / --module-path)
Unit	Flat bag of .class files and JARs	Named modules, each with its own boundary
Duplicate packages	Silently merged/shadowed (split package risk)	Hard resolution error
Encapsulation	None — everything public is visible to everyone	Enforced — only exports-ed packages are visible
Declares dependencies?	No — implicit, whatever happens to be on the path	Yes — explicit requires graph, validated at launch
Can mix the two?	N/A	Yes — a “hybrid” application can use both simultaneously, with classpath JARs landing in the unnamed module

```
# Classic classpath launch — flat, no module resolution at all
java -cp "libs/*:out" com.teampicnic.app.Main

# Modular launch — module path, resolved and validated against
↳ module-info.java
java -p "libs:out" -m com.teampicnic.app/com.teampicnic.app.Main
```

Readability and Accessibility Rules

JPMS access checks layer two independent questions on top of each other:

1. **Readability** — does module A requires module B (directly, or transitively via requires transitive)? If not, A cannot see *any* type in B, no matter how public.
2. **Accessibility** — assuming A reads B, is the specific package inside B exports-ed (for normal use) or opens-ed (for reflection) to A?

```

Module A --requires--> Module B
                        |
                        +-- exports com.b.api           (A can use
↳ com.b.api's public types normally)
                        +-- (com.b.internal not exported -> invisible to
↳ A, even though B "has" it)

```

A useful mental model: readability is “can I even see this module exists,” and accessibility is “given that I can see it, which of its packages am I allowed to touch, and how (normal use vs. reflection).”

```

module com.teampicnic.app {
    requires com.teampicnic.billing; // readability: app can now see
↳ billing's exported packages
}

```

If `com.teampicnic.billing` only exports `com.teampicnic.billing.api`, then `com.teampicnic.app` compiles fine against `Invoice` (which lives in `.api`), but a reference to `com.teampicnic.billing.internal.TaxCalculator` fails to compile with package `com.teampicnic.billing.internal` is not visible — even though app legitimately requires `billing`.

jlink, jdeps, and jmod

Three CLI tools work together around JPMS: `jdeps` for analysis, `jmod` for packaging, and `jlink` for producing a minimal custom runtime.

```

# jdeps: what does this JAR actually depend on, and is it already modular?
jdeps --print-module-deps app.jar           # e.g.
↳ java.base,java.sql,java.logging
jdeps --jdk-internals app.jar               # flags usage of JDK-internal
↳ (encapsulated) APIs

# jmod: package a module into the .jmod format -- mainly used for JDK
↳ modules
# and modules bundling native libraries; regular app modules ship as plain
↳ JARs.
jmod create --class-path out/com.teampicnic.billing --module-version 1.0
↳ billing.jmod

# jlink: assemble a minimal, self-contained runtime image with only the
# modules your app needs -- no full JDK required on the deployment target.
jlink --module-path out:$JAVA_HOME/jmods \

```

```
--add-modules com.teampicnic.app \
--output custom-runtime \
--strip-debug --no-header-files --no-man-pages --compress=2

./custom-runtime/bin/java -m com.teampicnic.app/com.teampicnic.app.Main
```

jlink's output is a real win for container images: instead of a full ~300MB JDK base image, you ship only the ~40-60MB of modules your app actually touches, shrinking both image size and attack surface.

Strong Encapsulation and `--add-exports/--add-opens`

Since Java 16 (JEP 396), strong encapsulation of JDK internals is **on by default** — code can no longer reflectively reach into packages like `jdk.internal.misc` or `sun.nio.ch` unless the module system is explicitly told to allow it. Libraries that relied on these internals (older versions of Lombok, Mockito, various serialization frameworks) had to add explicit flags or be updated.

```
# --add-exports: grants normal (non-reflective) access to an
↪ otherwise-unexported
# package, from a specific module to a specific module.
java --add-exports java.base/jdk.internal.misc=com.teampicnic.app -p out
↪ -m com.teampicnic.app/com.teampicnic.app.Main

# --add-opens: grants deep reflection access to an otherwise-closed
↪ package.
java --add-opens java.base/java.lang=ALL-UNNAMED -p out -m
↪ com.teampicnic.app/com.teampicnic.app.Main
```

ALL-UNNAMED means “grant this to the unnamed module,” i.e., to whatever is on the plain classpath — this is the flag combination you’ll most often see in the wild when a classpath-based library (Mockito, older Jackson, testing frameworks) needs deep reflection into `java.lang` or `java.util` internals to do bytecode manipulation or field injection.

These are explicitly called **escape hatches**: they exist so real-world code that isn’t ready for strong encapsulation can keep running, not as a long-term architectural pattern. Starting in newer JDKs, using them (or performing “restricted” native/reflective operations without them) triggers warnings, and the trend is toward requiring them more strictly with each release — treat every `--add-opens` in a build file as a line item to eventually remove, not a permanent fixture.

Worked Multi-Module Example

Two modules: a library module that exposes a Greeter, and an application module that consumes it.

```
project/
├── com.teampicnic.greetings/
│   ├── module-info.java
│   └── com/
│       └── teampicnic/
│           └── greetings/
│               └── Greeter.java
└── com.teampicnic.app/
    ├── module-info.java
    └── com/
        └── teampicnic/
            └── app/
                └── Main.java
```

```
// project/com.teampicnic.greetings/module-info.java
module com.teampicnic.greetings {
    exports com.teampicnic.greetings;
}
```

```
// project/com.teampicnic.greetings/com/teampicnic/greetings/Greeter.java
package com.teampicnic.greetings;

public class Greeter {
    public String greet(String name) {
        return "Hello, " + name + "!";
    }
}
```

```
// project/com.teampicnic.app/module-info.java
module com.teampicnic.app {
    requires com.teampicnic.greetings;
}
```

```
// project/com.teampicnic.app/com/teampicnic/app/Main.java
package com.teampicnic.app;

import com.teampicnic.greetings.Greeter;
```

```
public class Main {
    public static void main(String[] args) {
        System.out.println(new Greeter().greet("code review"));
    }
}
```

Compile both modules using `--module-source-path`, which lets `javac` resolve inter-module dependencies from source in one pass:

```
mkdir -p out

javac -d out \
    --module-source-path project \
    $(find project -name "*.java")

# out/com.teampicnic.greetings/com/teampicnic/greetings/Greeter.class
# out/com.teampicnic.app/com/teampicnic/app/Main.class
```

Run it directly off the module path:

```
java -p out -m com.teampicnic.app/com.teampicnic.app.Main
# Hello, code review!
```

Build a minimal custom runtime with `jlink`:

```
jlink --module-path out:$JAVA_HOME/jmods \
    --add-modules com.teampicnic.app \
    --launcher greet=com.teampicnic.app/com.teampicnic.app.Main \
    --output custom-runtime

./custom-runtime/bin/greet
# Hello, code review!
```

The `--launcher` flag is a nice `jlink` feature reviewers sometimes miss: it generates a native-feeling launcher script (`custom-runtime/bin/greet`) so end users never need to know or type the module/main-class combination.

Migration Strategy

Two broad strategies exist for turning an existing multi-JAR, classpath-based application into a set of real JPMS modules.

Strategy	Approach	Pros	Cons
Bottom-up	Modularize the lowest-level, most-depended-upon libraries first (utility/core JARs), then work up toward the application	Each step is small and independently verifiable; leaf libraries have the fewest dependencies to reconcile	Slow — the application itself stays non-modular for a long time; benefits (strong encapsulation for the app) arrive last
Top-down	Add a <code>module-info.java</code> to the application first, treating every dependency as an automatic module, then gradually convert dependencies to real modules one at a time	Gets the app onto the module path (and gets <code>jlink</code> working) quickly	Automatic modules provide none of JPMS's actual safety, so early gains are mostly tooling-level (custom runtimes), not encapsulation

In practice, most real migrations are a hybrid: start top-down to get the application itself running on the module path (with `jdeps` used heavily to find hidden internal-API dependencies first), then chip away bottom-up at whichever dependencies are most painful — usually anything relying on `sun.misc.Unsafe`, reflection into `java.lang`, or shading/relocation tricks that conflict with module boundaries.

```
# jdeps is the standard first step of any migration: find out what a JAR
# actually needs before writing a single module-info.java.
jdeps --generate-module-info out-modules/ legacy-app.jar
```

`jdeps --generate-module-info` will even scaffold a best-effort `module-info.java` for you based on observed bytecode dependencies, which is a good starting draft — but always needs manual review, since it can't infer intent (e.g., which packages are truly meant to be public API versus which merely happen to be used across a package boundary today).

Why Many Projects Still Don't Use JPMS

Despite being available since 2017, most application codebases (as opposed to the JDK itself, and a handful of major libraries) never adopt module descriptors, and that is a reasonable, defensible choice:

- **Split packages in the existing dependency tree** are extremely common (two transitive dependencies exporting the same package) and can require upgrading, forking, or shading dependencies just to get past module resolution errors — for zero new feature value.

- **Spring, Hibernate, and most application frameworks** still work perfectly well on the plain classpath, and JPMS's encapsulation benefits matter far less inside a single deployable application than for library authors shipping to unrelated consumers.
- **Build tool support** has historically been fiddly, with friction around test compilation, `--add-opens` requirements for mocking frameworks, and IDE tooling lag.
- **The main payoff for most teams is `jlLink`** (smaller container images), which is achievable alongside simpler tools (slim base images, ArchUnit-style architecture tests) without ever writing a `module-info.java`.

A defensible interview answer: JPMS is essential for JDK maintainers and library authors guaranteeing encapsulation across organizational boundaries; for a typical internal application, the same goals are often reachable more cheaply through build-tool module boundaries and architecture linting — so skipping it is not automatically a red flag.

Foreign Function & Memory API

The Foreign Function & Memory API (FFM API, `java.lang.foreign` package) was finalized in **JDK 22 via JEP 454**, after multiple rounds of incubation and preview (starting as the Foreign-Memory Access API in JDK 14). It lets Java code allocate and manipulate off-heap memory and call native © functions **without writing any native code, without JNI headers, and without `Unsafe`**.

From JNI to FFM: Why It Changed

JNI (Java Native Interface, since Java 1.1) required writing actual C glue code, compiling it into a platform-specific shared library, and loading it with `System.loadLibrary`. `Unsafe` and `ByteBuffer` (direct buffers) let you get *some* off-heap access from pure Java, but both had serious limitations.

```
// The old ByteBuffer / Unsafe world: off-heap memory, but no safe
↪ deallocation
// story, no structured layout description, and no way to call arbitrary
// native functions without JNI.
java.nio.ByteBuffer direct = java.nio.ByteBuffer.allocateDirect(1024);
direct.putInt(0, 42);
int value = direct.getInt(0);
// No explicit "free" -- direct buffers are reclaimed only by GC + a
↪ Cleaner,
// with no deterministic timing and no way to call a C function using this
↪ memory.
```

FFM replaces this with a small set of pure-Java classes: `Arena` for lifecycle-managed allocation, `MemorySegment` for typed, bounds-checked pointers, `MemoryLayout/ValueLayout` for describing native data shapes, and `Linker` for calling and being called by native functions — all without JNI’s C compilation step.

Arena and Deterministic Deallocation

An `Arena` controls the **lifetime** of the off-heap memory it allocates. All memory allocated through an arena becomes invalid the moment the arena is closed — accessing it afterward throws `IllegalStateException` rather than corrupting memory or seg-faulting, which is the safety guarantee JNI/Unsafe never gave you.

Arena kind	Created with	Lifetime	Thread confinement
Confined	<code>Arena.ofConfined()</code>	Explicit <code>close()</code> (use try-with-resources)	Only the owning thread may access or close it
Shared	<code>Arena.ofShared()</code>	Explicit <code>close()</code>	Any thread may access it; any thread may close it
Auto	<code>Arena.ofAuto()</code>	Garbage-collected — no <code>close()</code> at all	Any thread
Global	<code>Arena.global()</code>	Never freed — lives for the whole JVM process	Any thread

```
import java.lang.foreign.Arena;
import java.lang.foreign.MemorySegment;
import java.lang.foreign.ValueLayout;

public class ArenaDemo {
    public static void main(String[] args) {
        try (Arena arena = Arena.ofConfined()) { // single-threaded,
↪ closed like a file handle
            MemorySegment segment =
↪ arena.allocate(ValueLayout.JAVA_INT.byteSize() * 4);
            for (int i = 0; i < 4; i++) {
                segment.setAtIndex(ValueLayout.JAVA_INT, i, i * i);
            }
            for (int i = 0; i < 4; i++) {
↪ System.out.println(segment.getAtIndex(ValueLayout.JAVA_INT, i));
            }
        }
    }
}
```



```

        } // memory is deterministically freed right here

        // Using 'segment' past this point throws IllegalStateException
↪ instead of
        // corrupting memory or segfaulting -- the core safety win over
↪ Unsafe.
    }
}

```

Try-with-resources is the idiomatic way to use a confined arena, mirroring exactly how you'd manage a `FileInputStream`—the “close what you open” instinct from Chapter 6 transfers directly.

MemorySegment, MemoryLayout, ValueLayout

A `MemorySegment` is a typed, bounds-checked view over a contiguous region of memory—either off-heap (native) or backed by an on-heap Java array.

```

import java.lang.foreign.MemorySegment;
import java.lang.foreign.ValueLayout;

public class SegmentDemo {
    public static void main(String[] args) {
        // Wrap an existing Java array as a MemorySegment (heap segment,
↪ no allocation)
        int[] javaArray = {10, 20, 30};
        MemorySegment heapSegment = MemorySegment.ofArray(javaArray);
        System.out.println(heapSegment.getAtIndex(ValueLayout.JAVA_INT,
↪ 1)); // 20

        // Out-of-bounds access throws IndexOutOfBoundsException, not a
↪ crash
        try {
            heapSegment.getAtIndex(ValueLayout.JAVA_INT, 10);
        } catch (IndexOutOfBoundsException e) {
            System.out.println("Bounds-checked: " + e.getMessage());
        }
    }
}

```

`MemoryLayout` describes the *shape* of memory (its size, alignment, and — for structs — named fields), independent of any specific segment. `ValueLayout` is the family of primitive layouts (`JAVA_INT`, `JAVA_LONG`, `JAVA_DOUBLE`, `ADDRESS`, ...) used to describe how raw bytes should be interpreted.

```
import java.lang.foreign.ValueLayout;

// ValueLayout constants describe primitive C types in
↳ platform-independent Java code:
ValueLayout.OfInt    javaInt    = ValueLayout.JAVA_INT;    // 4-byte int
ValueLayout.OfLong   javaLong   = ValueLayout.JAVA_LONG;   // 8-byte long
ValueLayout.OfDouble javaDouble = ValueLayout.JAVA_DOUBLE; // 8-byte
↳ double
AddressLayout    pointer    = ValueLayout.ADDRESS;    // native
↳ pointer (4 or 8 bytes)
```

VarHandle Access into Segments

A `VarHandle` obtained from a layout provides typed, potentially volatile/atomic access into a `MemorySegment` at a computed offset — the FFM equivalent of a field accessor, but for off-heap memory.

```
import java.lang.foreign.Arena;
import java.lang.foreign.MemoryLayout;
import java.lang.foreign.MemorySegment;
import java.lang.foreign.ValueLayout;
import java.lang.invoke.VarHandle;

public class VarHandleDemo {
    public static void main(String[] args) {
        MemoryLayout intArrayLayout = MemoryLayout.sequenceLayout(4,
↳ ValueLayout.JAVA_INT);
        VarHandle elementHandle = intArrayLayout.varHandle(
            MemoryLayout.PathElement.sequenceElement());

        try (Arena arena = Arena.ofConfined()) {
            MemorySegment segment = arena.allocate(intArrayLayout);
            for (int i = 0; i < 4; i++) {
                elementHandle.set(segment, 0L, (long) i, i * 10); //
↳ (segment, base offset, index, value)
            }
            for (int i = 0; i < 4; i++) {
                System.out.println(elementHandle.get(segment, 0L, (long)
↳ i));
            }
        }
    }
}
```

VarHandle access is how the FFM API lets you do fine-grained, layout-aware reads/writes (including atomic operations like compareAndSet on off-heap memory) without ever writing native code — the layout describes the shape once, and the handle gives you type-safe, bounds-checked access forever after.

Calling a C Function: Linker, SymbolLookup, FunctionDescriptor

Three pieces cooperate to call a native function: SymbolLookup finds the function's address, FunctionDescriptor describes its signature, and Linker produces a MethodHandle (a **downcall handle**) that invokes it.

```
import java.lang.foreign.Arena;
import java.lang.foreign.FunctionDescriptor;
import java.lang.foreign.Linker;
import java.lang.foreign.MemorySegment;
import java.lang.foreign.SymbolLookup;
import java.lang.foreign.ValueLayout;
import java.lang.invoke.MethodHandle;

public class StrlenDemo {
    public static void main(String[] args) throws Throwable {
        Linker linker = Linker.nativeLinker();
        SymbolLookup stdlib = linker.defaultLookup(); // the process's
        ↪ standard C library

        // strlen(const char *s) -> size_t    (size_t maps to a native long
        ↪ on 64-bit platforms)
        MethodHandle strlen = linker.downcallHandle(
            stdlib.find("strlen").orElseThrow(() -> new
        ↪ NoSuchElementException("strlen not found")),
            FunctionDescriptor.of(ValueLayout.JAVA_LONG,
        ↪ ValueLayout.ADDRESS)
        );

        try (Arena arena = Arena.ofConfined()) {
            MemorySegment cString = arena.allocateUtf8String("Hello,
        ↪ native world!");
            long length = (long) strlen.invoke(cString);
            System.out.println("Length: " + length); // 20
        }
    }
}
```

A no-argument function follows the same recipe with an empty argument list in the descriptor — getpid() (returns the OS process ID) needs only FunctionDescriptor.

of (`ValueLayout.JAVA_INT`) and `getpid.invoke()` with no arguments. Running either of these requires the native-access flag described below, and both compile/run as ordinary Java — no `javac -h`, no C compiler, no shared library build step, which is the headline improvement over JNI.

Upcalls: Java Callback into C

An **upcall** is the reverse direction: native code calling back into Java, such as passing a Java method as a C comparator function to `qsort`.

```
import java.lang.foreign.*;
import java.lang.invoke.MethodHandle;
import java.lang.invoke.MethodHandles;
import java.lang.invoke.MethodType;
import java.util.Arrays;

public class UpcallDemo {
    // This Java method will be exposed to native code as a C function
    ↪ pointer.
    static int compareInts(MemorySegment a, MemorySegment b) {
        int x = a.get(ValueLayout.JAVA_INT, 0);
        int y = b.get(ValueLayout.JAVA_INT, 0);
        return Integer.compare(x, y);
    }

    public static void main(String[] args) throws Throwable {
        Linker linker = Linker.nativeLinker();
        MethodHandle comparatorHandle = MethodHandles.lookup().findStatic(
            UpcallDemo.class, "compareInts",
            MethodType.methodType(int.class, MemorySegment.class,
            ↪ MemorySegment.class));

        try (Arena arena = Arena.ofConfined()) {
            // Wrap the Java method as a native function pointer ("upcall
            ↪ stub").
            MemorySegment comparatorStub = linker.upcallStub(
                comparatorHandle,
                FunctionDescriptor.of(ValueLayout.JAVA_INT,
            ↪ ValueLayout.ADDRESS, ValueLayout.ADDRESS),
                arena);

            MethodHandle qsort = linker.downcallHandle(
                linker.defaultLookup().find("qsort").orElseThrow(),
                FunctionDescriptor.ofVoid(ValueLayout.ADDRESS,
            ↪ ValueLayout.JAVA_LONG,
```

```

ValueLayout.JAVA_LONG,
↪ ValueLayout.ADDRESS));

    MemorySegment array =
↪ arena.allocateArray(ValueLayout.JAVA_INT, 5, 3, 1, 4, 2);
    qsort.invoke(array, 5L, ValueLayout.JAVA_INT.byteSize(),
↪ comparatorStub);

    int[] sorted = array.toArray(ValueLayout.JAVA_INT);
    System.out.println(Arrays.toString(sorted)); // [1, 2, 3, 4,
↪ 5]
    }
}
}

```

The upcall stub (`comparatorStub`) is itself a `MemorySegment` — a real native function pointer that C code (`qsort`, in this case) can call directly, entirely unaware it's actually calling back into the JVM.

Struct Layouts

`MemoryLayout.structLayout(...)` describes a C struct shape, with named fields accessed via `VarHandles` derived from path elements — no manual offset arithmetic required.

```

import java.lang.foreign.*;
import java.lang.invoke.VarHandle;

public class StructDemo {
    public static void main(String[] args) {
        // Equivalent to: struct Point { int x; int y; };
        StructLayout pointLayout = MemoryLayout.structLayout(
            ValueLayout.JAVA_INT.withName("x"),
            ValueLayout.JAVA_INT.withName("y")
        );

        VarHandle xHandle =
↪ pointLayout.varHandle(MemoryLayout.PathElement.groupElement("x"));
        VarHandle yHandle =
↪ pointLayout.varHandle(MemoryLayout.PathElement.groupElement("y"));

        try (Arena arena = Arena.ofConfined()) {
            MemorySegment point = arena.allocate(pointLayout);
            xHandle.set(point, 0L, 10);
            yHandle.set(point, 0L, 20);
        }
    }
}

```

```

        System.out.println("x=" + xHandle.get(point, 0L) + ", y=" +
↪ yHandle.get(point, 0L));
    }
}

```

`pointLayout.byteSize()` and `pointLayout.byteOffset(PathElement.groupElement("y"))` are computed automatically from the declared fields (respecting natural alignment/padding rules), which is exactly the error-prone bookkeeping JNI forced you to do by hand in C headers.

jextract

Writing every `FunctionDescriptor` and struct layout by hand for a large C library (say, all of `libcurl` or `sqlite3`) would be tedious and error-prone. **jextract** is a separate tool (distributed alongside OpenJDK builds, not part of the JDK proper) that parses a C header file and mechanically generates the Java FFM bindings — `MethodHandles`, layouts, and constants — for every declared function and struct.

```

# jextract reads a C header and generates ready-to-use Java FFM bindings,
# so you don't hand-write FunctionDescriptors for every function in
↪ sqlite3.h.
jextract --output src/generated \
        --target-package com.teampicnic.native_.sqlite \
        /usr/include/sqlite3.h

```

The generated code is ordinary Java calling the same `Linker/MemorySegment` machinery shown above — **jextract** is a code generator for boilerplate, not a different runtime mechanism.

--enable-native-access and Restricted Methods

Native memory access and native calls are **restricted methods**: operations the JVM cannot fully verify are memory-safe, because they cross into native code or raw memory. Since JDK 22, using them prints a runtime warning unless the module (or the unnamed module, for classpath code) is explicitly granted native access; **JDK 24 tightens this further** — by default, restricted native operations throw rather than merely warn, unless access is explicitly enabled.

```

# JDK 22/23: without this flag, restricted FFM operations print a warning
↳ to stderr:
#   WARNING: A restricted method in java.lang.foreign.SymbolLookup has
↳ been called
# JDK 24+: without this flag, restricted operations THROW at the call site
↳ instead.
java --enable-native-access=com.teampicnic.app -p out -m
↳ com.teampicnic.app/com.teampicnic.app.Main

# For classpath (unnamed-module) code:
java --enable-native-access=ALL-UNNAMED -cp out com.teampicnic.app.Main

```

This mirrors the philosophy of `--add-opens/--add-exports` for JPMS internals: native access is powerful enough to corrupt memory or crash the JVM outright (an off-by-one `MemorySegment` write is still possible if you compute an out-of-bounds *but still confined-looking* offset, and an entirely wrong `FunctionDescriptor` for a real native function is undefined behavior), so the JVM requires an explicit, auditable opt-in rather than allowing it silently by default.

FFM vs JNI Comparison Table

	JNI	Foreign Function & Memory API
Requires writing C/C++ glue code	Yes (.c/.cpp files, javah/javac -h header generation)	No — pure Java
Requires a native compiler toolchain	Yes	No
Memory safety	None — raw pointers, manual bookkeeping, easy to crash the JVM	Bounds-checked <code>MemorySegments</code> , arena-scoped lifetimes, <code>IllegalStateException</code> instead of memory corruption on misuse
Deallocation model	Manual, via native code (<code>free</code>) or <code>finalize()</code> -era hacks	Deterministic (<code>Arena.close()</code> /try-with-resources), GC-tied (<code>ofAuto</code>), or process-lifetime (<code>global</code>)
Calling native functions	Requires a pre-built, pre-loaded shared library exposing JNI-specific function signatures	Directly calls existing native libraries (<code>libc</code> , <code>libsqlite3</code> , etc.) with no JNI-specific wrapper needed
Callbacks into Java	Supported, but verbose and error-prone (<code>JNIEnv*</code> juggling)	<code>Linker.upcallStub</code> — a first-class, type-checked Java construct
Struct/layout description	Manual offset math in C headers, kept in sync by hand	<code>MemoryLayout/StructLayout</code> , computed automatically

	JNI	Foreign Function & Memory API
Tooling for large libraries	Hand-written wrappers, or third-party generators	jextract generates bindings from C headers
Finalized in	Java 1.1 (1997)	JDK 22 (2024), JEP 454
Governing flag	None (implicit trust)	--enable-native-access, restricted-method warnings/errors

Vector API (Incubator)

The Vector API (`jdk.incubator.vector`) provides a portable way to express **SIMD** (Single Instruction, Multiple Data) computations in pure Java, letting the JIT compile them down to CPU vector instructions (AVX, SVE, NEON, ...) instead of relying entirely on auto-vectorization heuristics.

Why It’s Still Incubating

The Vector API has been re-incubated in **every JDK release since JDK 16** — as of **JDK 25, it is on its 10th incubation round** — because its final shape depends on language and JVM features that are not yet finished, primarily **Project Valhalla**’s value types. An incubator module (JEP 11) is the JDK’s formal mechanism for shipping an API that is real and usable, but explicitly *not yet a permanent commitment*: its package name, method signatures, and semantics can still change between releases without the usual deprecation cycle.

Signal	What it tells a reviewer
Package is <code>jdk.incubator.vector</code> , not <code>java.util.vector</code> or similar	Not a supported, permanent API yet
Requires <code>--add-modules jdk.incubator.vector</code> explicitly	The JDK itself is telling you “opt in, at your own risk”
Re-incubated ~10 times across ~10 releases	The API shape is still actively changing release to release
No compatibility guarantee across JDK versions	Code compiled against one JDK’s incubator module may not run unchanged against the next

VectorSpecies, Lanes, and Masks

A `VectorSpecies<E>` describes a concrete vector “shape”: the element type and the number of **lanes** (elements processed per vector operation), which the JVM picks based on the actual CPU’s hardware vector width at runtime.

```
import jdk.incubator.vector.FloatVector;
import jdk.incubator.vector.VectorSpecies;

public class SpeciesDemo {
    public static void main(String[] args) {
        // SPECIES_PREFERRED picks the widest vector shape the current CPU
        ↪ supports
        // for float (e.g., 8 lanes on a 256-bit AVX2 machine, 16 on
        ↪ 512-bit AVX-512).
        VectorSpecies<Float> species = FloatVector.SPECIES_PREFERRED;
        System.out.println("Lanes: " + species.length());
    }
}
```

A **mask** is a per-lane boolean that selectively enables or disables lanes for an operation — essential for handling array lengths that aren’t an exact multiple of the vector width, or for conditional (branchless) computation.

```
import jdk.incubator.vector.FloatVector;
import jdk.incubator.vector.VectorMask;
import jdk.incubator.vector.VectorSpecies;

public class MaskDemo {
    static final VectorSpecies<Float> SPECIES =
    ↪ FloatVector.SPECIES_PREFERRED;

    static void clampNegativesToZero(float[] data) {
        int i = 0;
        int upperBound = SPECIES.loopBound(data.length);
        for (; i < upperBound; i += SPECIES.length()) {
            var v = FloatVector.fromArray(SPECIES, data, i);
            VectorMask<Float> negative =
            ↪ v.compare(jdk.incubator.vector.VectorOperators.LT, 0f);
            v.blend(0f, negative).intoArray(data, i); // lanes matching
            ↪ the mask become 0
        }
        for (; i < data.length; i++) { // scalar tail
            if (data[i] < 0f) data[i] = 0f;
        }
    }
}
```

```
}
```

Enabling the Incubator Module

Incubator modules must be explicitly requested at both compile time and run time; they are never on by default.

```
javac --add-modules jdk.incubator.vector --release 21
↳ VectorDotProduct.java
java --add-modules jdk.incubator.vector VectorDotProduct
```

Omitting `--add-modules jdk.incubator.vector` produces a compile error (package `jdk.incubator.vector` is not visible) — this is the same JPMS accessibility machinery from earlier in the chapter, deliberately used here to make incubating APIs opt-in and impossible to depend on by accident.

Worked Example: Vectorized Dot Product vs Scalar

```
// Scalar baseline: processes one float pair per loop iteration.
public class DotProductScalar {
    static float dotProduct(float[] a, float[] b) {
        float sum = 0f;
        for (int i = 0; i < a.length; i++) {
            sum += a[i] * b[i];
        }
        return sum;
    }
}
```

```
// Vectorized version: processes SPECIES.length() float pairs per loop
↳ iteration,
// falling back to a scalar tail loop for the remainder.
import jdk.incubator.vector.FloatVector;
import jdk.incubator.vector.VectorSpecies;
import jdk.incubator.vector.VectorOperators;

public class DotProductVector {
    static final VectorSpecies<Float> SPECIES =
↳ FloatVector.SPECIES_PREFERRED;

    static float dotProduct(float[] a, float[] b) {
```

```

    float sum = 0f;
    int i = 0;
    int upperBound = SPECIES.loopBound(a.length); // largest multiple
    ↪ of the lane count <= a.length

    for (; i < upperBound; i += SPECIES.length()) {
        FloatVector va = FloatVector.fromArray(SPECIES, a, i);
        FloatVector vb = FloatVector.fromArray(SPECIES, b, i);
        sum += va.mul(vb).reduceLanes(VectorOperators.ADD);
    }

    for (; i < a.length; i++) { // tail: leftover elements that don't
    ↪ fill a full vector
        sum += a[i] * b[i];
    }
    return sum;
}

```

The vectorized version does not change the *result* — both compute the same mathematical dot product — it changes how many multiplications the CPU performs per instruction. On a machine with 256-bit AVX2 and `float` (4 bytes), `SPECIES.length()` is typically 8, so each loop iteration does the work of 8 scalar iterations in roughly the same number of cycles.

The Loop-Plus-Tail Idiom

Almost every Vector API loop follows the same two-phase shape: a **vectorized main loop** that processes full lanes, followed by a **scalar tail loop** that handles whatever's left over when the array length isn't an exact multiple of the vector width.

```

static void scaleInPlace(float[] data, float factor) {
    VectorSpecies<Float> species = FloatVector.SPECIES_PREFERRED;
    int i = 0;
    int upperBound = species.loopBound(data.length);

    // Main loop: full vector width each iteration
    for (; i < upperBound; i += species.length()) {
        FloatVector v = FloatVector.fromArray(species, data, i);
        v.mul(factor).intoArray(data, i);
    }

    // Tail loop: whatever's left (0 to species.length() - 1 elements)
    for (; i < data.length; i++) {
        data[i] *= factor;
    }
}

```

```
    }  
}
```

An alternative to a separate tail loop is a **masked final iteration**, using `SPECIES.indexInRange(i, data.length)` to build a mask that disables out-of-bounds lanes so the whole array can be processed with vector operations alone, at the cost of a mask-compare on every iteration (including the ones that don't need it):

```
for (int i = 0; i < data.length; i += species.length()) {  
    var mask = species.indexInRange(i, data.length);  
    FloatVector v = FloatVector.fromArray(species, data, i, mask);  
    v.mul(factor).intoArray(data, i, mask);  
}
```

Both patterns are idiomatic; the explicit tail loop is generally easier to read and slightly faster for the common (non-final) iterations, while the masked version is more compact and avoids duplicating the operation logic.

Dependence on Project Valhalla

The Vector API's classes (`FloatVector`, `IntVector`, etc.) are, today, ordinary heap-allocated objects — every vector operation risks JIT-dependent allocation and indirection overhead that a hand-written intrinsic wouldn't have. **Project Valhalla**'s value types (formerly “inline classes”) are designed to let the JVM represent a `FloatVector` as a flat, register-resident sequence of primitives with no object header and no heap allocation at all, the same way a CPU's SIMD register actually works. The Vector API's final, non-incubating form is expected to be built on top of Valhalla value types once they ship — which is precisely why the API keeps changing shape release over release: its designers are deliberately keeping the door open for a completely different underlying representation.

Production Readiness Caveat

Given the above, the Vector API should be treated, in a code review, the same way you'd treat any other incubator or preview feature:

- It requires `--add-modules jdk.incubator.vector` on every build and run command, forever, until it graduates — a permanent build-configuration burden.
- Its behavior and API surface are not covered by Java's usual strict backward-compatibility guarantees; upgrading the JDK can require source changes.

- Performance-critical vectorization needs (image processing, ML inference, scientific computing) are, today, more commonly delivered via a well-tested native library called through the FFM API described earlier in this chapter, or via a JIT that already does reasonably good auto-vectorization for simple loops without any special API at all.
- **Recommendation for production code:** avoid the Vector API in shipped production systems until it exits incubation; it is excellent to know for interviews and to experiment with, but a reviewer should flag a `jdk.incubator.vector` import in application code as a real risk, not a style nit.

Common Code-Review Interview Pitfalls

1. **Using a wildcard import (`import com.example.*;`) in reviewed code.** Why it matters: it hides exactly which types are in use, risks silent breakage if the package later adds a colliding name, and is rejected by nearly every style guide.

```
// Before
import com.teampicnic.orders.*;
// After
import com.teampicnic.orders.Order;
import com.teampicnic.orders.OrderStatus;
```

2. **Leaving a scratch class in the default (unnamed) package and shipping it.** Why it matters: it cannot be imported by named-package code, cannot participate in JPMS at all, and invites name collisions as the codebase grows.

```
// Before
public class Utils { /* no package statement */ }
// After
package com.teampicnic.common;
public class Utils { }
```

3. **Confusing exports with opens when a framework needs reflection.** Why it matters: Jackson/Hibernate/Spring need deep reflective field access; exports alone grants normal compile-time visibility but still throws `InaccessibleObjectException` on `setAccessible(true)`.

```
// Before (Jackson fails to deserialize private fields at runtime)
module com.teampicnic.billing { exports com.teampicnic.billing.model; }
// After
```

```
module com.teampicnic.billing { opens com.teampicnic.billing.model; }
```

- 4. Forgetting requires transitive when a public API returns a type from another module.** Why it matters: without it, every consumer of your module must separately require the dependency your API leaks, breaking encapsulation and causing confusing “package not visible” errors far from the real cause.

```
// Before: billing.api returns java.sql.Connection but doesn't
→ re-export java.sql
module com.teampicnic.billing { requires java.sql; }
// After
module com.teampicnic.billing { requires transitive java.sql; }
```

- 5. Adding --add-opens/--add-exports flags to “make the error go away” without understanding why they were needed.** Why it matters: these are escape hatches around intentional strong encapsulation; scattering them across build scripts accumulates hidden, unreviewed access to JDK internals that can break on the next JDK upgrade.

```
# Before: blanket, undocumented flag added to silence a warning
java --add-opens java.base/java.lang=ALL-UNNAMED -jar app.jar
# After: scoped to the specific module that actually needs it, with a
# comment explaining which dependency requires it and a plan to remove
→ it
java --add-opens java.base/java.lang=com.teampicnic.legacybridge -jar
→ app.jar
```

- 6. Assuming every project must adopt JPMS “because it’s modern.”** Why it matters: for a typical single-deployable application (as opposed to a widely distributed library), the migration cost (split packages, --add-opens for test/mock framework) often outweighs the benefit; flagging the *absence* of `module-info.java` as a defect, on its own, is not a strong review comment.

```
Review comment to avoid: "This project has no module-info.java, please
→ modularize it."
Better: ask whether the team's actual pain point (image size? internal
encapsulation?) is better solved by jlink + a slimmer base image, or by
architecture-linting tools, before demanding a full JPMS migration.
```

- 7. Calling native functions via the FFM API without wrapping allocation in try-with-resources.** Why it matters: memory allocated by an `Arena.ofConfined()/ofShared()`

is not freed until `close()` runs; forgetting it leaks native memory exactly like forgetting to close a file handle, except it won't show up in heap dumps.

```
// Before
Arena arena = Arena.ofConfined();
MemorySegment s = arena.allocate(1024);
// ... arena never closed, memory leaks for the arena's whole
↳ ("forever") lifetime
// After
try (Arena arena = Arena.ofConfined()) {
    MemorySegment s = arena.allocate(1024);
}
```

- 8. Passing a FunctionDescriptor that doesn't match the real native function signature.** Why it matters: unlike a normal Java type mismatch, this is not caught by the compiler or even reliably at runtime — it produces undefined behavior (corrupted stack, wrong results, or a JVM crash) because the FFM API trusts the descriptor you provide.

```
// Before: strlen actually returns size_t (8 bytes on 64-bit), declared
↳ as 4-byte int
FunctionDescriptor.of(ValueLayout.JAVA_INT, ValueLayout.ADDRESS);
// After
FunctionDescriptor.of(ValueLayout.JAVA_LONG, ValueLayout.ADDRESS);
```

- 9. Running FFM downcalls/upcalls without `--enable-native-access` and treating the resulting warning (or, on JDK 24+, exception) as noise to suppress.** Why it matters: the warning/error exists because restricted operations can crash the JVM or corrupt memory; suppressing it without understanding the risk removes a deliberate safety signal instead of addressing it.

```
# Before: warning ignored / logging turned down to hide it
java -p out -m com.teampicnic.app/com.teampicnic.app.Main 2>/dev/null
# After: explicit, scoped, and documented opt-in
java --enable-native-access=com.teampicnic.app -p out -m
↳ com.teampicnic.app/com.teampicnic.app.Main
```

- 10. Shipping `jdk.incubator.vector` code in a production code path.** Why it matters: incubator modules have no backward-compatibility guarantee and can change or disappear between JDK releases; a production dependency on one means every JDK upgrade is a potential source-breaking event.

```
// Before: production hot path depends on the Vector API directly
import jdk.incubator.vector.FloatVector;
// After: keep vectorization behind a well-tested native library called
// via the finalized FFM API, or rely on JIT auto-vectorization, until
// the Vector API graduates out of incubation
```

- 11. Writing a Vector API loop with no scalar tail (or mask) for leftover elements.** Why it matters: if the array length isn't an exact multiple of `SPECIES.length()`, a naive vectorized loop either throws an out-of-bounds exception or silently skips the trailing elements, producing wrong results.

```
// Before: silently drops elements past the last full vector
for (int i = 0; i < a.length; i += SPECIES.length()) {
    FloatVector.fromArray(SPECIES, a, i).mul(2f).intoArray(a, i);
}
// After
int upperBound = SPECIES.loopBound(a.length);
int i = 0;
for (; i < upperBound; i += SPECIES.length()) { /* vectorized */ }
for (; i < a.length; i++) { a[i] *= 2f; } // tail
```

- 12. Believing opens/exports provide security against a determined attacker.** Why it matters: JPMS encapsulation is a compile-time and linkage-time boundary for well-behaved code, not a security sandbox — `--add-opens`, native access, or an already-loaded reflective handle can bypass it, and it must never be the sole line of defense for genuinely sensitive internals.

```
Review comment: "We don't export the credentials package, so it's
↳ secure."
Correction: JPMS encapsulation prevents accidental coupling between
modules, not deliberate bypass; sensitive data still needs its own
access controls (encryption at rest, proper secret management),
↳ independent
of module boundaries.
```

- 13. Introducing a split package while adding a new dependency to a modularized project.** Why it matters: two modules exporting the same package name fail module resolution at launch with a hard-to-diagnose `ResolutionException`, often only discovered in CI or production, not at compile time on the developer's machine if only one of the two JARs was present locally.


```

Error: Module A and module B export package com.example.util to module
↳ C
Fix: shade/relocate one of the offending JARs, upgrade to a version
↳ that
renamed the package, or exclude the duplicate dependency.

```

- 14. Treating an automatic module (a plain JAR on the module path) as if it had real JPMS encapsulation.** Why it matters: automatic modules export every package to everyone and read every other module, so requires on one gives no actual safety guarantee — code review should not assume “it’s on the module path” implies “its internals are protected.”

```

Review comment to avoid: "guava is modularized now, so its internals
↳ are encapsulated."
Correction: guava (via Automatic-Module-Name) is an automatic module –
all of its packages remain fully open; only a real module-info.class
with explicit exports would encapsulate anything.

```

- 15. Reviewing package-private classes as if they were dead code because “nothing outside the package uses them.”** Why it matters: package-private visibility is a deliberate design boundary, not an accident — flagging an unused-outside-package helper class as “should be deleted” without checking whether it’s the intended internal implementation of a package’s public facade produces a wrong, noisy review comment.

```

// Correct read: TaxCalculator below is intentionally hidden; it's
↳ InvoiceService's
// private implementation detail, not "orphaned" code, even though no
↳ import
// statement anywhere references it directly.
class TaxCalculator { /* used only by InvoiceService in the same
↳ package */ }

```


Networking & Security

Almost every real application talks to something over a network — a database, another service, a browser, a file on another machine. This chapter covers Java’s networking building blocks, from raw sockets to the modern HTTP Client, then moves into running external processes safely, and finishes with security fundamentals and cryptography. Security bugs are some of the most expensive bugs a reviewer can miss, so this chapter leans heavily on **anti-pattern** ↔ **fix** pairs: code that looks fine but is exploitable, next to the version that isn’t. We target Java 21+ throughout.

Networking (java.net)

The `java.net` package is Java’s low-level toolkit for network communication. It has been in the JDK since Java 1.0, so a lot of application code still uses it directly, even though higher-level APIs (like the HTTP Client, covered next) are usually a better choice for talking HTTP.

InetAddress — representing a host

An **IP address** is a numeric address for a machine on a network (like `192.0.2.1`). `InetAddress` represents one, and can resolve a **hostname** (a human-readable name like `example.com`) into an IP address via **DNS** (Domain Name System — the internet’s phonebook that maps names to addresses).

```
import java.net.InetAddress;
import java.net.UnknownHostException;

public class HostLookup {
    public static void main(String[] args) throws UnknownHostException {
        InetAddress address = InetAddress.getByName("example.com");
        System.out.println(address.getHostName());    // example.com
    }
}
```

```

        System.out.println(address.getHostAddress()); // the resolved IP,
↪ e.g. 93.184.216.34
    }
}

```

DNS lookups are **blocking network calls**. Calling `getByName` in a hot loop, or on every request, can silently stall your application if DNS is slow — cache the result if the host rarely changes.

Socket / ServerSocket — TCP communication

A **socket** is one endpoint of a two-way network connection. **TCP** (Transmission Control Protocol) is a reliable, ordered, connection-based protocol — think of it as a phone call: you dial, both sides confirm the connection, then you can talk in either direction until someone hangs up. `ServerSocket` listens for incoming connections; `Socket` represents one established connection (either the server's side of an accepted connection, or the client's side).

Here is a tiny **echo server** — it reads a line of text from a client and sends the same line back — plus a client that talks to it.

```

// EchoServer.java
import java.io.*;
import java.net.ServerSocket;
import java.net.Socket;

public class EchoServer {
    public static void main(String[] args) throws IOException {
        try (ServerSocket serverSocket = new ServerSocket(5000)) {
            System.out.println("Echo server listening on port 5000");
            while (true) {
                try (Socket client = serverSocket.accept();
                    BufferedReader in = new BufferedReader(
↪ InputStreamReader(client.getInputStream()));
                    PrintWriter out = new
↪ PrintWriter(client.getOutputStream(), true)) {

                    String line = in.readLine();
                    if (line != null) {
                        out.println("Echo: " + line);
                    }
                } // client socket closed automatically, connection ends
            }
        }
    }
}

```

```
    }
}
```

```
// EchoClient.java
import java.io.*;
import java.net.Socket;
import java.time.Duration;

public class EchoClient {
    public static void main(String[] args) throws IOException {
        try (Socket socket = new Socket()) {
            socket.connect(new java.net.InetSocketAddress("localhost",
↪ 5000), 3000); // connect timeout: 3s
            socket.setSoTimeout(3000); // read timeout: 3s

            PrintWriter out = new PrintWriter(socket.getOutputStream(),
↪ true);
            BufferedReader in = new BufferedReader(new
↪ InputStreamReader(socket.getInputStream()));

            out.println("Hello, server!");
            System.out.println("Server replied: " + in.readLine());
        }
    }
}
```

Each connected `Socket` gets its own thread (or virtual thread) in a simple server like this; production servers usually use non-blocking I/O or a thread pool to handle thousands of clients without one thread per connection.

DatagramSocket — UDP communication

UDP (User Datagram Protocol) is connectionless and unreliable: you send a **packet** (called a **datagram**) and hope it arrives, with no guarantee of order or delivery. Think of it as mailing postcards instead of making a phone call — cheaper, faster, but some might get lost. It's used where occasional loss is acceptable and low latency matters more, such as video streaming, DNS, or game state updates.

```
import java.net.DatagramPacket;
import java.net.DatagramSocket;
import java.net.InetSocketAddress;

public class UdpSender {
```

```

    public static void main(String[] args) throws Exception {
        try (DatagramSocket socket = new DatagramSocket()) {
            byte[] data = "ping".getBytes();
            DatagramPacket packet = new DatagramPacket(
                data, data.length, new InetAddress("localhost",
↪ 6000));
            socket.send(packet);
        }
    }

    public class UdpReceiver {
        public static void main(String[] args) throws Exception {
            try (DatagramSocket socket = new DatagramSocket(6000)) {
                byte[] buffer = new byte[1024];
                DatagramPacket packet = new DatagramPacket(buffer,
↪ buffer.length);
                socket.receive(packet); // blocks until a packet arrives
                String message = new String(packet.getData(), 0,
↪ packet.getLength());
                System.out.println("Received: " + message);
            }
        }
    }
}

```

URL / URI — and why `URL.equals` is a trap

URI (Uniform Resource Identifier) is just a **syntax** for identifying a resource — it does no network I/O. URL (Uniform Resource Locator) is a URI that also knows how to *fetch* the resource, and that extra behavior is where the trouble starts.

`URL.equals()` and `URL.hashCode()` perform a DNS lookup to compare hosts, because the spec says two URLs are “equal” only if they resolve to the same IP address. This means comparing two URL objects — or worse, putting them in a `HashSet` or using them as `HashMap` keys — can trigger a real network call, block on slow or unreachable DNS, and even give different results at different times if DNS changes.

```

// Anti-pattern: URLs in a HashSet trigger silent DNS lookups on every
↪ add/contains
import java.net.URL;
import java.util.HashSet;
import java.util.Set;

Set<URL> seen = new HashSet<>();
seen.add(new URL("https://example.com/a")); // may block on DNS

```

```
boolean visited = seen.contains(new URL("https://example.com/b")); // may
↳ block on DNS too
```

```
// Fix: use URI, which never touches the network
import java.net.URI;
import java.util.HashSet;
import java.util.Set;

Set<URI> seen = new HashSet<>();
seen.add(URI.create("https://example.com/a"));
boolean visited = seen.contains(URI.create("https://example.com/b")); //
↳ pure string/scheme comparison
```

Rule of thumb: use URI for parsing, comparing, and storing. Only convert to URL (via `uri.toURL()`) at the last moment, right before you actually need to open a connection.

URLConnection / HttpURLConnection — the legacy way

Before the HTTP Client API (next section), Java code opened HTTP connections through `URL.openConnection()`, which returns a `URLConnection` (or, for HTTP, an `HttpURLConnection`). It's still present in the JDK and you'll see it in older codebases, but it's clunky: manual stream handling, awkward redirect and header handling, and no built-in HTTP/2 or async support. Treat it as **legacy** — prefer the HTTP Client API for new code.

```
import java.net.HttpURLConnection;
import java.net.URL;
import java.io.*;

public class LegacyGet {
    public static void main(String[] args) throws IOException {
        URL url = new URL("https://example.com/api/status");
        HttpURLConnection conn = (HttpURLConnection) url.openConnection();
        conn.setRequestMethod("GET");
        conn.setConnectTimeout(3000); // connect timeout: 3s
        conn.setReadTimeout(3000);   // read timeout: 3s

        try (BufferedReader reader = new BufferedReader(
            new InputStreamReader(conn.getInputStream()))) {
            String body = reader.lines().reduce("", (a, b) -> a + b);
            System.out.println("Status: " + conn.getResponseCode() + "
↳ body: " + body);
```

```

        } finally {
            conn.disconnect();
        }
    }
}

```

Timeouts — the setting a missing default punishes you for

Neither `Socket`, `URLConnection`, nor (as we'll see) `HttpClient` has a sane default timeout baked in for every operation. Two timeouts matter:

- **Connect timeout** — how long to wait while establishing the TCP connection.
- **Socket/read timeout** (`soTimeout`) — how long to wait for data once connected, before giving up on a read.

```

// Anti-pattern: no timeouts at all
Socket socket = new Socket("flaky-host", 443); // may hang forever if the
↪ host never responds

// Fix: always set both
Socket socket = new Socket();
socket.connect(new InetSocketAddress("flaky-host", 443), 3000); // connect
↪ timeout
socket.setSoTimeout(5000); // read timeout

```

Why this is a real production incident, not a theoretical concern: if a downstream service hangs (network partition, overloaded server, firewall silently dropping packets), a thread with no timeout can block **forever**. Multiply that by every request thread calling the same downstream service, and the whole application thread pool fills up with stuck threads — a **cascading failure** where one slow dependency takes down an entire service, even though the dependency never returned an error. Always set explicit connect and read timeouts on every network call.

SocketChannel — non-blocking I/O

`SocketChannel` (from `java.nio.channels`) is a **non-blocking** alternative to `Socket`: instead of a thread parking until data arrives, you register interest in an event (readable, writable, connectable) with a `Selector`, and one thread can monitor many channels at once.


```

import java.net.InetSocketAddress;
import java.nio.ByteBuffer;
import java.nio.channels.SocketChannel;

public class NonBlockingClient {
    public static void main(String[] args) throws Exception {
        try (SocketChannel channel = SocketChannel.open()) {
            channel.configureBlocking(false);
            channel.connect(new InetSocketAddress("example.com", 80));

            while (!channel.finishConnect()) {
                // could do other work here instead of spinning
            }

            ByteBuffer buffer = ByteBuffer.wrap("GET /
↪ HTTP/1.0\r\n\r\n".getBytes());
            channel.write(buffer);
        }
    }
}

```

Non-blocking I/O is what lets a server handle thousands of connections with a handful of threads, instead of one thread per connection. It's more complex to write correctly, which is why frameworks (Netty, and Java's own virtual threads for simpler blocking-style code) exist to hide the complexity.

Unix domain sockets (JDK 16+): `SocketChannel` and `ServerSocketChannel` can also bind to a Unix domain socket (a file-path-based socket for fast local inter-process communication, no network stack involved) via `UnixDomainSocketAddress.of(path)` — useful for talking to a sidecar process on the same machine without going through TCP/IP.

HTTP Client API

Since Java 11, the JDK ships a modern `java.net.http.HttpClient` — a proper HTTP/1.1 and HTTP/2 client with synchronous and asynchronous modes, replacing the awkward `URLConnection` for new code.

Building a client

`HttpClient` instances are **immutable and thread-safe** once built, and expensive to create (they manage connection pools). Build one and **reuse it** across your whole application instead of creating a new client per request.

```

import java.net.ProxySelector;
import java.net.http.HttpClient;
import java.time.Duration;
import java.util.concurrent.Executors;

HttpClient client = HttpClient.newBuilder()
    .version(HttpClient.Version.HTTP_2)           // prefer
↳ HTTP/2, falls back to 1.1
    .connectTimeout(Duration.ofSeconds(5))        // connection
↳ timeout
    .followRedirects(HttpClient.Redirect.NORMAL)  // follow
↳ redirects, except HTTPS -> HTTP
    .proxy(ProxySelector.of(new
↳ java.net.InetSocketAddress("proxy.internal", 8080)))
    .executor(Executors.newVirtualThreadPerTaskExecutor()) // custom
↳ executor for async callbacks
    .build();

```

Building requests

`HttpRequest` is also immutable; build one per call with `HttpRequest.newBuilder()`. `BodyPublishers` supplies the request body for methods like POST and PUT.

```

import java.net.URI;
import java.net.http.HttpRequest;
import java.net.http.HttpRequest.BodyPublishers;
import java.time.Duration;

// GET
HttpRequest getRequest = HttpRequest.newBuilder()
    .uri(URI.create("https://api.example.com/users/42"))
    .timeout(Duration.ofSeconds(10)) // per-request timeout, overrides
↳ client default
    .header("Accept", "application/json")
    .GET()
    .build();

// POST with a JSON body
HttpRequest postRequest = HttpRequest.newBuilder()
    .uri(URI.create("https://api.example.com/users"))
    .timeout(Duration.ofSeconds(10))
    .header("Content-Type", "application/json")
    .POST(BodyPublishers.ofString("{\"name\":\"Ada\"}"))
    .build();

```

Sending — sync vs async

send blocks the calling thread until the response arrives. sendAsync returns immediately with a `CompletableFuture<HttpResponse<T>>`, letting you compose follow-up work without blocking.

```
import java.net.http.HttpResponse;
import java.net.http.HttpResponse.BodyHandlers;

// Synchronous
HttpResponse<String> response = client.send(getRequest,
↳ BodyHandlers.ofString());
System.out.println(response.statusCode() + ": " + response.body());
```

```
// Asynchronous, with a CompletableFuture pipeline
client.sendAsync(getRequest, BodyHandlers.ofString())
    .thenApply(HttpResponse::body)
    .thenAccept(System.out::println)
    .exceptionally(ex -> {
        System.err.println("Request failed: " + ex.getMessage());
        return null;
    });
```

BodyHandlers controls how the response body is consumed:

- `BodyHandlers.ofString()` — whole body as a `String`; fine for small JSON responses.
- `BodyHandlers.ofLines()` — a `Stream<String>` of lines; good for line-delimited data.
- `BodyHandlers.ofInputStream()` — raw `InputStream`; lets you stream large responses without loading them fully into memory.
- `BodyHandlers.ofFile(Path)` — writes the body directly to disk; ideal for downloads.

```
// Streaming a large download straight to disk instead of buffering it in
↳ memory
HttpRequest downloadRequest = HttpRequest.newBuilder()
    .uri(URI.create("https://example.com/large-file.zip"))
    .build();

HttpResponse<java.nio.file.Path> saved = client.send(
```

```

        downloadRequest,
        ↪ BodyHandlers.ofFile(java.nio.file.Path.of("large-file.zip")));
        System.out.println("Saved to " + saved.body());

```

HTTP/2 multiplexing

HTTP/2 allows many logical requests to share a single TCP connection at once (**multiplexing**), instead of opening a new connection — or queuing — per request as HTTP/1.1 often does. `HttpClient` negotiates HTTP/2 automatically when the server supports it (via ALPN during the TLS handshake); you don't need to change any request code. This is one reason to prefer a shared `HttpClient` instance: reusing it lets the connection pool actually multiplex requests instead of paying a new handshake cost every time.

Error handling and status codes

`HttpClient` only throws exceptions for things like connection failures, timeouts, or protocol errors. **HTTP error status codes (4xx, 5xx) are not exceptions** — they come back as a normal `HttpResponse` that you must check yourself.

```

// Anti-pattern: assuming a response without an exception means success
HttpResponse<String> response = client.send(getRequest,
↪ BodyHandlers.ofString());
processUser(response.body()); // might be a 404 or 500 error page, not
↪ user JSON!

```

```

// Fix: check the status code explicitly
HttpResponse<String> response = client.send(getRequest,
↪ BodyHandlers.ofString());
int status = response.statusCode();
if (status >= 200 && status < 300) {
    processUser(response.body());
} else if (status == 404) {
    throw new NoSuchElementException("User not found");
} else {
    throw new IllegalStateException("Unexpected status " + status + ": "
↪ + response.body());
}

```

Also handle exceptions from `send/sendAsync`: `java.net.http.HttpTimeoutException` for timeouts, `java.net.ConnectException` for connection failures, and `java.io.IOException` more generally.

Custom SSLContext

By default, `HttpClient` uses the JVM's default `SSLContext`, which trusts the standard public **certificate authorities** (organizations that vouch for a server's identity — see the Security Basics section). To connect to a server with a private/internal certificate authority, supply a custom `SSLContext` built from your own **trust store** (a keystore holding the certificates you trust) — never a trust-all context (shown as an anti-pattern later in this chapter).

```
import javax.net.ssl.SSLContext;
import javax.net.ssl.TrustManagerFactory;
import java.security.KeyStore;

KeyStore trustStore = KeyStore.getInstance("PKCS12");
try (var in = java.nio.file.Files.newInputStream(java.nio.file.Path.of("
→ internal-ca.p12"))) {
    trustStore.load(in, "changeit".toCharArray());
}
TrustManagerFactory tmf = TrustManagerFactory.getInstance(
→ TrustManagerFactory.getDefaultAlgorithm());
tmf.init(trustStore);

SSLContext sslContext = SSLContext.getInstance("TLSv1.3");
sslContext.init(null, tmf.getTrustManagers(), null);

HttpClient secureClient = HttpClient.newBuilder()
    .sslContext(sslContext)
    .build();
```

WebSocket support

`HttpClient` also builds **WebSocket** connections (a persistent, full-duplex connection for real-time, bidirectional messaging — unlike request/response HTTP) via `HttpClient.newWebSocketBuilder()`.

```
import java.net.http.WebSocket;
import java.net.URI;
import java.util.concurrent.CompletionStage;
```

```

WebSocket.Listener listener = new WebSocket.Listener() {
    @Override
    public CompletionStage<?> onText(WebSocket webSocket, CharSequence
↪ data, boolean last) {
        System.out.println("Received: " + data);
        webSocket.request(1);
        return null;
    }
};

WebSocket webSocket = client.newWebSocketBuilder()
    .buildAsync(URI.create("wss://example.com/socket"), listener)
    .join();

webSocket.sendText("hello", true);

```

AutoCloseable in JDK 21+

Since Java 21, `HttpClient` implements `AutoCloseable`, so you can use it in a try-with-resources block to release its resources (connection pools, background threads) deterministically — useful for short-lived clients, though for a long-lived shared client you'd typically just let it live for the application's lifetime and never close it.

```

try (HttpClient shortLivedClient = HttpClient.newHttpClient()) {
    HttpResponse<String> response = shortLivedClient.send(getRequest,
↪ BodyHandlers.ofString());
    System.out.println(response.body());
} // client resources released here

```

Process API

Sometimes your Java application needs to run an external program — a shell script, `ffmpeg`, `git`, a native tool. The **Process API** (`ProcessBuilder`, `Process`, `ProcessHandle`) lets you launch and manage external processes.

ProcessBuilder — launching a process safely

`ProcessBuilder` takes the command as a **list of strings**: the program name, then each argument as a separate list element. This is critical for security — see the anti-pattern below.

```
// Anti-pattern: building a shell command string from user input – COMMAND
↪ INJECTION
String filename = userInput; // attacker-controlled, e.g. "; rm -rf / #"
Process process = Runtime.getRuntime().exec("ls -l " + filename);
```

If filename is "foo; rm -rf /", and this string is ever passed through a shell (e.g. via `bash -c`), the shell happily runs `rm -rf /` as a second command. This class of bug is called **command injection**: attacker input escapes its intended role as *data* and gets interpreted as *code* (here, shell syntax).

```
// Fix: pass arguments as a list – no shell parsing, no injection
import java.util.List;

ProcessBuilder builder = new ProcessBuilder(List.of("ls", "-l",
↪ filename));
Process process = builder.start();
```

Because each argument is a separate list element, filename is passed to the `ls` program as a single literal argument — `;`, `&&`, backticks, and other shell metacharacters have no special meaning here. **Never** build a command by concatenating a string and handing it to a shell (`bash -c "..."`, `sh -c "..."`, or string-based `exec`) when any part of it comes from user input.

Redirecting IO

By default, a child process's `stdin`/`stdout`/`stderr` are separate pipes your Java code must read or write. `redirectOutput`, `redirectError`, and `redirectInput` connect them directly to files, or `inheritIO()` connects them straight to the parent JVM's own console — handy for CLI tools where you just want to see the child's output live.

```
ProcessBuilder builder = new ProcessBuilder("ping", "-c", "3",
↪ "example.com");
builder.inheritIO(); // child's stdout/stderr appear directly in this
↪ program's console
Process process = builder.start();
int exitCode = process.waitFor();
```

```
ProcessBuilder builder = new ProcessBuilder("mytool", "--verbose");
builder.redirectOutput(new java.io.File("output.log"));
```

```
builder.redirectError(ProcessBuilder.Redirect.INHERIT); // errors still
↳ show in console
Process process = builder.start();
```

Reading stdout/stderr without deadlocking

If you don't redirect a stream and don't read it, the child process's output buffer can fill up. Once full, the child blocks trying to write more output — and if your Java code is meanwhile blocked in `waitFor()` waiting for the child to exit, you get a **deadlock**: each side waiting on the other forever.

```
// Anti-pattern: reading stdout only after waitFor() – can deadlock if
↳ stderr fills up first
Process process = new ProcessBuilder("noisy-tool").start();
int exitCode = process.waitFor(); // may hang forever if noisy-tool blocks
↳ on a full stderr buffer
String output = new String(process.getInputStream().readAllBytes());
```

```
// Fix: drain both streams concurrently, in separate threads, before
↳ waiting
Process process = new ProcessBuilder("noisy-tool").start();

Thread outputReader = Thread.ofVirtual().start(() -> {
    try (var in = process.getInputStream()) {
        System.out.write(in.readAllBytes());
    } catch (java.io.IOException e) { /* log */ }
});
Thread errorReader = Thread.ofVirtual().start(() -> {
    try (var err = process.getErrorStream()) {
        System.err.write(err.readAllBytes());
    } catch (java.io.IOException e) { /* log */ }
});

int exitCode = process.waitFor();
outputReader.join();
errorReader.join();
```

An easier fix, when you don't need the streams programmatically, is `ProcessBuilder.redirectErrorStream(true)` (merges stderr into stdout — one stream to read) or `inheritIO()` — both avoid the deadlock entirely by not leaving an unread pipe.

waitFor with a timeout

Waiting forever for a child process is as risky as a network call with no timeout — a hung external tool can hang your JVM's thread indefinitely.

```
import java.util.concurrent.TimeUnit;

Process process = new ProcessBuilder("slow-tool").start();
boolean finished = process.waitFor(30, TimeUnit.SECONDS);
if (!finished) {
    process.destroyForcibly(); // give up and kill it
    throw new IllegalStateException("slow-tool did not finish in time");
}
int exitCode = process.exitValue();
```

ProcessHandle (JDK 9+)

`ProcessHandle` gives you information about *any* process — not just ones you started — by PID (process ID), including the current JVM's own process, without needing native code.

```
ProcessHandle current = ProcessHandle.current();
System.out.println("My PID: " + current.pid());

Process child = new ProcessBuilder("sleep", "60").start();
ProcessHandle handle = child.toHandle();

handle.info().command().ifPresent(cmd -> System.out.println("Command: " +
↳ cmd));
handle.info().startInstant().ifPresent(t -> System.out.println("Started:
↳ " + t));

// List all child processes of the current JVM
ProcessHandle.current().children().forEach(h ->
    System.out.println("Child PID: " + h.pid()));

// React when the process exits, without blocking the current thread
handle.onExit().thenAccept(h ->
    System.out.println("Process " + h.pid() + " exited with status "
↳ + h.exitValue()));
```

destroy vs destroyForcibly

`destroy()` requests a **graceful** termination — on most platforms this sends `SIGTERM` on Unix-like systems, letting the process clean up (close files, flush buffers) before exiting. `destroyForcibly()` requests an **immediate, forceful** kill (`SIGKILL`-like), which the process cannot intercept or clean up after.

```
Process process = new ProcessBuilder("my-daemon").start();
process.destroy(); // ask nicely first
if (!process.waitFor(5, TimeUnit.SECONDS)) {
    process.destroyForcibly(); // it ignored us - force it
}
```

Prefer `destroy()` first and escalate to `destroyForcibly()` only if the process doesn't respond in a reasonable time, so well-behaved processes get a chance to shut down cleanly.

Security Basics

Security in Java isn't a separate skill from writing correct code — most security bugs are just correctness bugs where the attacker controls the input. This section is deliberately defensive: every insecure pattern shown here is labelled **Anti-pattern** and immediately followed by the fix, for recognizing and rejecting these patterns in review — not for building them.

Principle of least privilege

Give code, users, and processes the **minimum** access they need to do their job, nothing more. A reporting service that only reads data should use a database account with read-only permissions, not the same admin account used for migrations. A file-processing service should run as a low-privilege OS user, not `root`. When something does go wrong — and eventually something will — least privilege limits the damage.

Input validation

Never trust input from outside your program's trust boundary — HTTP requests, file uploads, environment variables, even data from another internal service you don't fully control. Validate shape, type, range, and length *before* using the value, ideally as close to the entry point as possible.

```

// Anti-pattern: using a user-supplied value without validation
int pageSize = Integer.parseInt(request.getParameter("pageSize"));
List<Item> items = repository.fetch(pageSize); // attacker sends
↳ pageSize=2000000000

// Fix: validate range before use
int pageSize = Integer.parseInt(request.getParameter("pageSize"));
if (pageSize < 1 || pageSize > 100) {
    throw new IllegalArgumentException("pageSize must be between 1 and
↳ 100");
}
List<Item> items = repository.fetch(pageSize);

```

SQL injection

SQL injection happens when user input is concatenated directly into a SQL query string, letting an attacker change the query's *meaning* instead of just supplying a value.

```

// Anti-pattern: string concatenation builds attacker-controlled SQL
String username = request.getParameter("username"); // e.g. "" OR '1'='1'
String sql = "SELECT * FROM users WHERE username = " + username + "'";
ResultSet rs = statement.executeQuery(sql); // returns ALL users, auth
↳ bypass

```

```

// Fix: PreparedStatement with bound parameters – input is always treated
↳ as data
String sql = "SELECT * FROM users WHERE username = ?";
try (PreparedStatement ps = connection.prepareStatement(sql)) {
    ps.setString(1, username);
    ResultSet rs = ps.executeQuery();
}

```

PreparedStatement sends the query structure and the parameter values separately to the database, so the database never re-parses attacker input as SQL syntax.

Command injection

Covered in detail in the Process API section above: never build a shell command string from untrusted input. Always pass arguments as a list to `ProcessBuilder`, and avoid invoking a shell (`bash -c`, `sh -c`) with concatenated input at all.

Path traversal

Path traversal (or “directory traversal”) happens when user input controls a file path and contains sequences like `../` that escape the intended directory.

```
// Anti-pattern: filename from the client used directly in a file path
String filename = request.getParameter("file"); // e.g. "../../etc/passwd"
File file = new File("/var/app/uploads/" + filename);
byte[] contents = Files.readAllBytes(file.toPath()); // reads outside
↳ uploads/
```

```
// Fix: resolve, normalize, and verify the result stays within the allowed
↳ directory
Path uploadsDir = Path.of("/var/app/uploads").toRealPath();
Path requested = uploadsDir.resolve(filename).normalize();

if (!requested.startsWith(uploadsDir)) {
    throw new SecurityException("Invalid file path: " + filename);
}
byte[] contents = Files.readAllBytes(requested);
```

`resolve+normalize` collapses `..` segments, and the `startsWith` check ensures the final path is still inside the intended base directory before touching the filesystem.

XXE — XML External Entity injection

XML parsers, by default, may resolve **external entities**— references inside an XML document that pull in content from a file or URL. If an attacker controls the XML being parsed, they can use this to read local files or make the server issue outbound requests (**XXE**, XML External Entity injection).

```
// Anti-pattern: default DocumentBuilderFactory resolves external entities
String maliciousXml = """
    <?xml version="1.0"?>
    <!DOCTYPE foo [ <!ENTITY xxe SYSTEM "file:///etc/passwd"> ]>
    <foo>&xxe;</foo>
    """;

DocumentBuilderFactory factory = DocumentBuilderFactory.newInstance();
DocumentBuilder builder = factory.newDocumentBuilder();
Document doc = builder.parse(new
↳ ByteArrayInputStream(maliciousXml.getBytes())); // leaks /etc/passwd
```

```

// Fix: disable DTDs and external entities explicitly
DocumentBuilderFactory factory = DocumentBuilderFactory.newInstance();
factory.setFeature("http://apache.org/xml/features/disallow-doctype-decl"
↪ , true);
factory.setFeature("http://xml.org/sax/features/external-general-entities"
↪ ", false);
factory.setFeature("
↪ http://xml.org/sax/features/external-parameter-entities", false);
factory.setXIncludeAware(false);
factory.setExpandEntityReferences(false);

DocumentBuilder builder = factory.newDocumentBuilder();
Document doc = builder.parse(new
↪ ByteArrayInputStream(maliciousXml.getBytes())); // throws, DTD rejected

```

The same idea applies to `SAXParserFactory`, `XMLInputFactory` (StAX), and `TransformerFactory` — each has equivalent feature flags to disable DTDs and external entities. When parsing XML from an untrusted source, disable them all.

Insecure deserialization

Java's built-in serialization (`ObjectInputStream.readObject()`) reconstructs arbitrary objects from a byte stream, including running code in constructors, `readObject` overrides, and finalizers along the way. Deserializing **untrusted** data can let an attacker construct object graphs that trigger unintended — sometimes arbitrary — code execution (a well-known class of “gadget chain” attacks).

```

// Anti-pattern: deserializing bytes received from a network client
ObjectInputStream in = new ObjectInputStream(socket.getInputStream());
Object payload = in.readObject(); // attacker fully controls the class
↪ graph being built

```

```

// Fix: use a safe, explicit data format instead (e.g. JSON with a known
↪ schema),
// or if Java serialization is unavoidable, filter allowed classes
ObjectInputStream in = new ObjectInputStream(socket.getInputStream());
in.setObjectInputFilter(filterInfo -> {
    Class<?> clazz = filterInfo.serialClass();
    if (clazz != null && !clazz.getName().startsWith("com.myapp.dto.")) {
        return ObjectInputFilter.Status.REJECTED;
    }
    return ObjectInputFilter.Status.UNDECIDED;
});

```

```
Object payload = in.readObject(); // only com.myapp.dto.* classes are
↳ allowed through
```

The best fix is usually to avoid Java's native serialization for untrusted data entirely — use a safe format like JSON with a strict schema and a library that only builds known DTOs, never arbitrary classes.

SSRF — Server-Side Request Forgery

SSRF happens when your server makes an outbound HTTP request to a URL that is (directly or indirectly) controlled by the attacker, letting them reach internal-only systems (metadata endpoints, admin panels, internal services) that would otherwise be unreachable from outside.

```
// Anti-pattern: fetching a user-supplied URL with no restriction
String imageUrl = request.getParameter("avatarUrl"); // e.g.
↳ "http://169.254.169.254/latest/meta-data/"
HttpResponse<byte[]> response = client.send(
    HttpRequest.newBuilder(URI.create(imageUrl)).build(),
↳ BodyHandlers.ofByteArray());
```

```
// Fix: validate against an allow-list of hosts/schemes, and resolve+check
↳ the IP before connecting
private static final Set<String> ALLOWED_HOSTS =
↳ Set.of("images.trusted-cdn.com");

URI uri = URI.create(imageUrl);
if (!"https".equals(uri.getScheme()) ||
↳ !ALLOWED_HOSTS.contains(uri.getHost())) {
    throw new SecurityException("URL not allowed: " + imageUrl);
}
InetAddress resolved = InetAddress.getByName(uri.getHost());
if (resolved.isLoopbackAddress() || resolved.isSiteLocalAddress() ||
↳ resolved.isLinkLocalAddress()) {
    throw new SecurityException("URL resolves to a disallowed internal
↳ address");
}
HttpResponse<byte[]> response = client.send(
    HttpRequest.newBuilder(uri).build(), BodyHandlers.ofByteArray());
```

Note that an allow-list check on the hostname alone isn't quite enough — DNS can be manipulated (**DNS rebinding**) between the check and the actual request, so high-value

systems also pin the resolved IP or route such requests through an isolated network egress proxy.

Hard-coded secrets

Credentials, API keys, and encryption keys committed to source code end up in version control history forever, visible to anyone with repo access (and often, accidentally, on a public GitHub mirror).

```
// Anti-pattern: secret baked into source code
private static final String DB_PASSWORD = "SuperSecret123!";
private static final String API_KEY = "sk_live_51H8x...";
```

```
// Fix: load secrets from the environment or a secrets manager at runtime
String dbPassword = System.getenv("DB_PASSWORD");
String apiKey = secretsManagerClient.getSecret("payments/api-key");
```

Logging sensitive data

Logs are often shipped to third-party aggregators, kept for years, and readable by many engineers — a bad place for passwords, tokens, or full card numbers to end up.

```
// Anti-pattern: logging the full request, including sensitive fields
log.info("Login attempt: {}", loginRequest); // toString() includes the
↳ raw password

// Fix: log only what's needed, and redact the rest
log.info("Login attempt for user: {}", loginRequest.username());
```

V

s String for passwords]char[] vs String for passwords String is **immutable** and Java's string pool (or just normal GC timing) can keep a copy of a password in memory for an unpredictable, possibly long time — there's no way to proactively erase it. A char[] can be overwritten with zeros the moment you're done with it, shrinking the window where the password sits in memory in plaintext. This is why JPasswordField.getPassword() and JDBC's Connection APIs that accept passwords use char[].

```

// Anti-pattern: password as a String – cannot be wiped from memory on
↪ demand
String password = new String(passwordBytes);
authenticate(username, password);
// password may live in memory long after this point, out of your control

// Fix: char[] can be explicitly cleared right after use
char[] password = readPasswordFromInput();
try {
    authenticate(username, password);
} finally {
    java.util.Arrays.fill(password, '\0'); // overwrite in memory
}

```

SecureRandom vs Random

`java.util.Random` is a fast **pseudo-random number generator (PRNG)** seeded predictably — given the seed (or even a handful of outputs), an attacker can often predict every future value. It must never be used for anything security-sensitive: tokens, session IDs, password reset codes, cryptographic keys. `SecureRandom` uses a **cryptographically secure PRNG (CSPRNG)**, designed so past output reveals nothing about future output.

```

// Anti-pattern: predictable token generation
Random random = new Random();
String resetToken = Long.toHexString(random.nextLong()); // guessable by
↪ an attacker

// Fix: SecureRandom for anything security-sensitive
SecureRandom secureRandom = new SecureRandom();
byte[] tokenBytes = new byte[32];
secureRandom.nextBytes(tokenBytes);
String resetToken = java.util.Base64.getUrlEncoder().withoutPadding().
↪ encodeToString(tokenBytes);

```

Constant-time comparison

Comparing two secrets (e.g. an HMAC signature, an API key) with `String.equals` or `Arrays.equals` stops at the **first mismatched byte** — that tiny timing difference, measured over enough repeated attempts, can leak the correct value one byte at a time (a **timing attack**).


```

// Anti-pattern: early-exit comparison leaks timing information
if (!providedSignature.equals(expectedSignature)) {
    throw new SecurityException("Invalid signature");
}

// Fix: constant-time comparison – always examines every byte
byte[] provided = providedSignature.getBytes(StandardCharsets.UTF_8);
byte[] expected = expectedSignature.getBytes(StandardCharsets.UTF_8);
if (!MessageDigest.isEqual(provided, expected)) {
    throw new SecurityException("Invalid signature");
}

```

TLS defaults and certificate validation

TLS (Transport Layer Security, the successor to SSL) provides encryption and — critically — **authentication**: the client verifies the server's identity through its **certificate**, issued and signed by a trusted **certificate authority (CA)**. Skipping that verification defeats the entire point of TLS.

```

// Anti-pattern: a trust-all TrustManager disables certificate validation
→ entirely
TrustManager[] trustAllCerts = new TrustManager[] {
    new X509TrustManager() {
        public void checkClientTrusted(X509Certificate[] chain, String
→ authType) {} // accepts anything
        public void checkServerTrusted(X509Certificate[] chain, String
→ authType) {} // accepts anything
        public X509Certificate[] getAcceptedIssuers() { return new
→ X509Certificate[0]; }
    }
};
SSLContext sslContext = SSLContext.getInstance("TLS");
sslContext.init(null, trustAllCerts, new java.security.SecureRandom());
// Any server, including an attacker performing a man-in-the-middle attack,
→ is now "trusted"

```

```

// Fix: use the platform default trust managers (or a specific, real trust
→ store)
SSLContext sslContext = SSLContext.getInstance("TLSv1.3");
sslContext.init(null, null, null); // null TrustManager[] => use the JVM's
→ default, real CA trust store

```

If you need to trust a private/internal CA, build a real `TrustManagerFactory` from a keystore containing that CA's certificate (as shown in the HTTP Client section's `SSLContext` example) — never disable validation to work around a certificate problem, even “temporarily.”

The Security Manager (JEP 411)

The **Security Manager** was Java's mechanism for sandboxing code — restricting what a piece of code could do (file access, network access, etc.) via fine-grained permissions. As of **JEP 411** (Java 17), it is deprecated for removal, and it has been fully removed in Java 24. If you inherit code that relies on `SecurityManager/AccessController`, plan to migrate away from it — modern isolation is typically done at the process, container, or OS level instead (containers, seccomp profiles, dedicated low-privilege OS users), not inside the JVM.

Dependency vulnerabilities

Most real-world Java vulnerabilities (Log4Shell being the most infamous recent example) come from **third-party libraries**, not code you wrote. Practices that matter in review:

- Keep an inventory of dependencies and their versions (a **Software Bill of Materials**, or SBOM).
- Run a dependency vulnerability scanner (e.g. OWASP Dependency-Check, `mvn versions:display-dependency-updates`, GitHub Dependabot) as part of CI.
- Pin versions and update regularly — being a few major versions behind makes upgrading (and patching a critical CVE) much riskier and slower.
- Don't pull in a whole library for one function if you can avoid the dependency weight and attack surface.

Cryptography APIs

Java's cryptography support is split into the **JCA** (Java Cryptography Architecture — the framework: engine classes like `MessageDigest`, `Cipher`, `Signature`) and the **JCE** (Java Cryptography Extension — historically export-restricted extensions, now merged into the JDK). Both are **provider-based**: you ask for an algorithm by name (e.g. "AES/GCM/NoPadding"), and a registered **Provider** (the built-in `SunJCE`, or a third-party one

like Bouncy Castle) supplies the actual implementation. This lets you swap implementations without changing application code.

```
import java.security.Provider;
import java.security.Security;

for (Provider provider : Security.getProviders()) {
    System.out.println(provider.getName());
}
```

MessageDigest — hashing

A **hash function** takes input of any size and produces a fixed-size fingerprint (**digest**). It's one-way: you can't recover the input from the digest. Use it for integrity checks (e.g. "did this file change?"), never for passwords (see below).

```
import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;
import java.nio.charset.StandardCharsets;
import java.util.HexFormat;

public class HashExample {
    public static String sha256(String input) throws
    ↪ NoSuchAlgorithmException {
        MessageDigest digest = MessageDigest.getInstance("SHA-256");
        byte[] hash =
    ↪ digest.digest(input.getBytes(StandardCharsets.UTF_8));
        return HexFormat.of().formatHex(hash);
    }
}
```

Password hashing — PBKDF2, and why plain hashing isn't enough

Never store passwords in plaintext, and never hash them with a plain, fast hash like MD5 or SHA-1/SHA-256 alone. Fast hashes let an attacker who steals your password database try billions of guesses per second on cheap GPU hardware. Password hashing needs to be **slow and tunable** on purpose, and use a per-password **salt** (random data mixed in) so identical passwords don't produce identical hashes.

```
// Anti-pattern: a fast, unsalted hash for passwords
MessageDigest digest = MessageDigest.getInstance("SHA-256");
byte[] hash = digest.digest(password.getBytes(StandardCharsets.UTF_8));
```

```
// Crackable at billions of attempts/second with commodity GPUs; identical
↳ passwords -> identical hashes
```

```
// Fix: PBKDF2 via SecretKeyFactory – salted, and deliberately slow (many
↳ iterations)
import javax.crypto.SecretKeyFactory;
import javax.crypto.spec.PBEKeySpec;
import java.security.SecureRandom;

public class PasswordHasher {
    public static byte[] hash(char[] password, byte[] salt) throws
↳ Exception {
        PBEKeySpec spec = new PBEKeySpec(password, salt, 210_000, 256); //
↳ iterations, key length in bits
        SecretKeyFactory factory =
↳ SecretKeyFactory.getInstance("PBKDF2WithHmacSHA256");
        return factory.generateSecret(spec).getEncoded();
    }

    public static byte[] newSalt() {
        byte[] salt = new byte[16];
        new SecureRandom().nextBytes(salt);
        return salt;
    }
}
```

Store the salt alongside the hash (it's not secret) and re-derive using the same salt and iteration count to verify a login attempt. In practice, **bcrypt** or **argon2** (via a library, since the JDK doesn't ship them natively) are usually preferred over PBKDF2 for new systems — they're purpose-built for password hashing with better resistance to GPU/ASIC cracking. Whatever you choose: never MD5, never plain SHA-1/SHA-256, never unsalted.

Cipher — AES-GCM done correctly

Cipher performs **encryption** (reversible scrambling with a key) and decryption. **AES-GCM** (Advanced Encryption Standard, Galois/Counter Mode) is the recommended default: it's fast, and it's an **authenticated** mode — it detects tampering, not just confidentiality.

The single most common mistake: **ECB mode**. ECB encrypts each fixed-size block independently, so identical plaintext blocks always produce identical ciphertext blocks — patterns in the input leak straight through, famously visible if you ECB-encrypt an image.

```

    // Anti-pattern: AES in ECB mode – identical plaintext blocks -> identical
    ⇨ ciphertext blocks
    Cipher cipher = Cipher.getInstance("AES/ECB/PKCS5Padding");
    cipher.init(Cipher.ENCRYPT_MODE, secretKey);
    byte[] ciphertext = cipher.doFinal(plaintext); // patterns in plaintext
    ⇨ are visible in ciphertext

```

```

    // Fix: AES-GCM with a fresh random IV (nonce) for every single encryption
    import javax.crypto.Cipher;
    import javax.crypto.SecretKey;
    import javax.crypto.spec.GCMParameterSpec;
    import java.security.SecureRandom;

    public class AesGcmExample {
        private static final int IV_LENGTH_BYTES = 12;
        private static final int TAG_LENGTH_BITS = 128;

        public static byte[] encrypt(byte[] plaintext, SecretKey key) throws
    ⇨ Exception {
            byte[] iv = new byte[IV_LENGTH_BYTES];
            new SecureRandom().nextBytes(iv); // MUST be unique per encryption
    ⇨ with the same key

            Cipher cipher = Cipher.getInstance("AES/GCM/NoPadding");
            cipher.init(Cipher.ENCRYPT_MODE, key, new
    ⇨ GCMParameterSpec(TAG_LENGTH_BITS, iv));
            byte[] ciphertext = cipher.doFinal(plaintext);

            // Prepend the IV so decrypt() can read it back – the IV itself is
    ⇨ not secret
            byte[] result = new byte[iv.length + ciphertext.length];
            System.arraycopy(iv, 0, result, 0, iv.length);
            System.arraycopy(ciphertext, 0, result, iv.length,
    ⇨ ciphertext.length);
            return result;
        }

        public static byte[] decrypt(byte[] ivAndCiphertext, SecretKey key)
    ⇨ throws Exception {
            byte[] iv = java.util.Arrays.copyOfRange(ivAndCiphertext, 0,
    ⇨ IV_LENGTH_BYTES);
            byte[] ciphertext = java.util.Arrays.copyOfRange(ivAndCiphertext,
    ⇨ IV_LENGTH_BYTES, ivAndCiphertext.length);

            Cipher cipher = Cipher.getInstance("AES/GCM/NoPadding");

```

```

        cipher.init(Cipher.DECRYPT_MODE, key, new
↪ GCMParameterSpec(TAG_LENGTH_BITS, iv));
        return cipher.doFinal(ciphertext); // throws AEADBadTagException
↪ if tampered
    }
}

```

The **IV** (Initialization Vector, also called a nonce) must be unique for every encryption performed with the same key — reusing an IV with GCM catastrophically breaks its security guarantees, potentially revealing the plaintext and letting an attacker forge messages. Generating it fresh with `SecureRandom` every time, as above, is the standard safe approach.

KeyGenerator / KeyPairGenerator

`KeyGenerator` creates **symmetric** keys (the same key encrypts and decrypts, used with `Cipher` above). `KeyPairGenerator` creates **asymmetric** key pairs (a public key anyone can use to encrypt or verify, and a private key only the owner holds, to decrypt or sign).

```

import javax.crypto.KeyGenerator;
import javax.crypto.SecretKey;
import java.security.KeyPair;
import java.security.KeyPairGenerator;

// Symmetric AES key
KeyGenerator keyGen = KeyGenerator.getInstance("AES");
keyGen.init(256); // key size in bits
SecretKey aesKey = keyGen.generateKey();

// Asymmetric key pair
KeyPairGenerator keyPairGen = KeyPairGenerator.getInstance("RSA");
keyPairGen.initialize(3072); // modulus size in bits
KeyPair keyPair = keyPairGen.generateKeyPair();

```

Signature — proving authenticity and integrity

A **digital signature** lets the holder of a private key sign data so that anyone with the corresponding public key can verify it came from them and wasn't altered — the asymmetric-key analogue of the `HMAC/MessageDigest.isEqual` comparison shown earlier.

```

import java.security.Signature;
import java.security.PrivateKey;
import java.security.PublicKey;

// Signing
Signature signer = Signature.getInstance("SHA256withRSA");
signer.initSign(privateKey);
signer.update(data);
byte[] signatureBytes = signer.sign();

// Verifying
Signature verifier = Signature.getInstance("SHA256withRSA");
verifier.initVerify(publicKey);
verifier.update(data);
boolean valid = verifier.verify(signatureBytes);

```

KeyStore — storing keys and certificates

A KeyStore is a protected container for keys and certificates, typically backed by a file (PKCS12 format is the modern standard; the older proprietary JKS format still appears in legacy code).

```

import java.security.KeyStore;

KeyStore keyStore = KeyStore.getInstance("PKCS12");
try (var in = java.nio.file.Files.newInputStream(java.nio.file.Path.of("j
→ app-keys.p12"))) {
    keyStore.load(in, "keystorePassword".toCharArray());
}
java.security.cert.Certificate cert = keyStore.getCertificate("my-alias");

```

SecureRandom seeding

SecureRandom seeds itself automatically from the operating system's entropy source (e.g. /dev/urandom on Linux) — you generally should **not** manually reseed it with a predictable value, which would undermine its unpredictability.

```

// Anti-pattern: reseeding with a predictable value defeats the purpose
SecureRandom random = new SecureRandom();
random.setSeed(System.currentTimeMillis()); // now guessable from the
→ approximate time

// Fix: just use the default seeding — it's already good

```

```
SecureRandom random = new SecureRandom();
byte[] output = new byte[32];
random.nextBytes(output);
```

Algorithm do / don't

Purpose	Do use	Don't use	Why
Hashing (integrity)	SHA-256, SHA-3	MD5, SHA-1	MD5 and SHA-1 have known collision weaknesses
Password hashing	bcrypt, argon2, PBKDF2 (many iterations)	MD5, SHA-1, plain SHA-256, unsalted anything	fast hashes are crackable at massive scale; no salt means identical inputs collide
Symmetric encryption	AES-GCM, AES-256	DES, 3DES, RC4, AES-ECB	DES/RC4 are broken; ECB leaks patterns
Asymmetric encryption/signing	RSA (≥3072-bit), ECDSA (P-256/P-384), Ed25519	RSA (<2048-bit)	short keys are brute-forceable with modern hardware
Random values (security use)	SecureRandom	java.util.Random, Math.random()	predictable PRNGs leak future/past values
Transport security	TLS 1.2 / TLS 1.3	SSL 2.0/3.0, TLS 1.0/1.1	old SSL/TLS versions have known protocol flaws
Key comparison	MessageDigest.isEqual	==, Arrays.equals, String.equals	early-exit comparisons leak timing information

Common Code-Review Interview Pitfalls

1. **Opening a socket, HttpURLConnection, or HttpClient request with no connect/read timeout.** Why it matters: a single unresponsive dependency can hang a thread forever, and enough hung threads exhaust the whole pool — a real production outage, not a theoretical risk.

```
// Before
Socket socket = new Socket("host", 443);
// After
Socket socket = new Socket();
socket.connect(new InetSocketAddress("host", 443), 3000);
socket.setSoTimeout(5000);
```


2. Storing URL objects in a HashSet/HashMap or comparing them with equals.

Why it matters: `URL.equals/hashCode` perform a blocking DNS lookup, which can silently stall the application and even give inconsistent results if DNS changes.

```
// Before
Set<URL> cache = new HashSet<>();
// After
Set<URI> cache = new HashSet<>(); // URI never touches the network
```

3. Creating a brand-new HttpClient for every request instead of sharing one instance. Why it matters: `HttpClient` manages connection pools; recreating it per request throws away pooling and HTTP/2 multiplexing benefits and adds unnecessary handshake overhead.

```
// Before
HttpResponse<String> r = HttpClient.newHttpClient().send(req,
↳ BodyHandlers.ofString());
// After
private static final HttpClient CLIENT = HttpClient.newHttpClient(); //
↳ built once, reused everywhere
```

4. Treating any non-thrown HttpClient response as a success. Why it matters: 4xx/5xx responses don't throw exceptions in the HTTP Client API — skipping the status-code check means error bodies get processed as if they were valid data.

```
// Before
processUser(response.body());
// After
if (response.statusCode() / 100 != 2) throw new
↳ IllegalStateException("HTTP " + response.statusCode());
processUser(response.body());
```

5. Building a shell command by string concatenation with untrusted input. Why it matters: this is textbook command injection — shell metacharacters in the input let an attacker run arbitrary commands.

```
// Before
Runtime.getRuntime().exec("ls " + userInput);
// After
new ProcessBuilder("ls", userInput).start(); // arguments never parsed
↳ by a shell
```

6. Calling `process.waitFor()` before draining `stdout/stderr`, risking a deadlock.

Why it matters: if the child's output buffer fills up while your code is blocked waiting for exit, both sides wait on each other forever.

```
// Before
process.waitFor();
String out = new String(process.getInputStream().readAllBytes());
// After
// drain stdout/stderr concurrently in separate threads, then waitFor()
```

7. Calling `Process.waitFor()` with no timeout. Why it matters: a hung external process (bad input, infinite loop, deadlock) can block the calling thread indefinitely.

```
// Before
process.waitFor();
// After
if (!process.waitFor(30, TimeUnit.SECONDS)) process.destroyForcibly();
```

8. Building SQL by string concatenation instead of using `PreparedStatement`.

Why it matters: this is SQL injection — one of the most common and most damaging vulnerabilities, capable of leaking or destroying an entire database.

```
// Before
"SELECT * FROM users WHERE name = '" + name + "'"
// After
PreparedStatement ps = conn.prepareStatement("SELECT * FROM users WHERE
→ name = ?");
ps.setString(1, name);
```

9. Parsing untrusted XML with a default `DocumentBuilderFactory/SAXParserFactory`.

Why it matters: default XML parsers resolve external entities, opening the door to XXE attacks that read local files or trigger SSRF.

```
// Before
DocumentBuilderFactory.newInstance().newDocumentBuilder().parse(
→ untrustedInput);
// After
factory.setFeature("
→ http://apache.org/xml/features/disallow-doctype-decl", true);
// ... plus disabling external general/parameter entities, then parse
```

- 10. Using a trust-all TrustManager (or HostnameVerifier that always returns true) “to get past a certificate error.”** Why it matters: this disables TLS’s core security guarantee, making the connection trivially vulnerable to a man-in-the-middle attack.

```
// Before
public void checkServerTrusted(X509Certificate[] chain, String
↳ authType) {} // trusts anything
// After
// use the JVM's default trust managers, or a real trust store
↳ containing your CA
sslContext.init(null, null, null);
```

- 11. Hashing passwords with a single fast pass of SHA-256 (or MD5/SHA-1) instead of a slow, salted algorithm.** Why it matters: fast hashes let attackers who steal the password table brute-force billions of guesses per second on cheap hardware.

```
// Before
MessageDigest.getInstance("SHA-256").digest(password.getBytes());
// After
SecretKeyFactory.getInstance("PBKDF2WithHmacSHA256").generateSecret(
    new PBEKeySpec(password, salt, 210_000, 256));
```

- 12. Using AES in ECB mode.** Why it matters: ECB leaks structural patterns in the plaintext because identical blocks always encrypt to identical ciphertext — it provides much weaker confidentiality than intended.

```
// Before
Cipher.getInstance("AES/ECB/PKCS5Padding");
// After
Cipher.getInstance("AES/GCM/NoPadding"); // with a fresh random IV per
↳ encryption
```

- 13. Reusing the same IV/nonce for multiple AES-GCM encryptions with the same key.** Why it matters: IV reuse with GCM catastrophically breaks confidentiality and integrity guarantees — it can expose plaintext and allow forged ciphertexts.

```
// Before
static final byte[] FIXED_IV = new byte[12]; // reused every call
// After
byte[] iv = new byte[12];
new SecureRandom().nextBytes(iv); // fresh IV, generated per encryption
```

- 14. Using `java.util.Random` (or `Math.random()`) to generate tokens, session IDs, or keys.** Why it matters: these PRNGs are predictable; an attacker who observes some outputs (or knows the seed) can often predict others, defeating the token's purpose.

```
// Before
new Random().nextLong();
// After
new SecureRandom().nextBytes(new byte[32]);
```

- 15. Comparing secrets (signatures, tokens, MACs) with equals instead of a constant-time comparison.** Why it matters: `equals/Arrays.equals` exit early on the first mismatch, leaking timing information an attacker can use to guess the secret byte by byte.

```
// Before
if (!hmac.equals(expected)) throw new SecurityException();
// After
if (!MessageDigest.isEqual(hmac.getBytes(), expected.getBytes()))
    ↪ throw new SecurityException();
```

- 16. Storing passwords in a `String` instead of a `char []`, or logging request/response objects that contain secrets.** Why it matters: `String` immutability means the password can linger in memory unpredictably with no way to wipe it, and logging full objects routinely leaks credentials into long-lived, widely-read log stores.

```
// Before
log.info("Login attempt: {}", loginRequest); // toString() includes
↪ password field
// After
log.info("Login attempt for user: {}", loginRequest.username());
```

- 17. Fetching a user-supplied URL directly, with no host allow-list (SSRF).** Why it matters: without restriction, an attacker can direct your server to make requests to internal-only endpoints (cloud metadata services, admin panels) it would otherwise never reach.

```
// Before
client.send(HttpRequest.newBuilder(URI.create(userSuppliedUrl)).build(
↪ ), handler);
// After
```

```
if (!ALLOWED_HOSTS.contains(URI.create(userSuppliedUrl).getHost()))  
↳ throw new SecurityException();
```

- 18. Hard-coding API keys, passwords, or encryption keys as string literals in source code.** Why it matters: secrets committed to version control are effectively permanent and visible to anyone with repository access, including in old commits after “removal.”

```
// Before  
private static final String API_KEY = "sk_live_51H8x...";  
// After  
String apiKey = System.getenv("API_KEY"); // or a secrets manager,  
↳ injected at deploy time
```


Software Engineering Best Practices

Writing code that *compiles* is easy. Writing code that a teammate can read, change, and trust six months from now is the actual job. This chapter is a practical tour of the habits, principles, and patterns that Java reviewers look for: clean code rules, SOLID, the highest-value tips from *Effective Java*, the design patterns that keep showing up in interviews, API design guidelines, and the code smells that get flagged in almost every pull request. We target Java 21+ and use records, sealed types, and pattern matching wherever they simplify the classic advice.

Clean Code in Java

“Clean code” means code that is easy to read, easy to change, and does what its name says. It is not about being clever. A reviewer should be able to understand a method in a few seconds, without running it.

Naming

Names are the cheapest form of documentation. A good name removes the need for a comment.

```
// Before: what is d? what is 86400?
int d = 86400;
boolean f = u.getA() > d;

// After: intention is obvious without a comment
int secondsPerDay = 86_400;
boolean accountIsOverdue = user.getAccountAgeInSeconds() > secondsPerDay;
```

Rules of thumb:

- Classes and variables: nouns (`OrderService`, `pendingInvoices`).
- Methods: verbs (`calculateTotal`, `isValid`).
- Booleans: read like a yes/no question (`isEmpty`, `hasPermission`, `canRetry`).
- Avoid abbreviations that only the author understands (`usrRegSvc` → `userRegistrationService`).
- Avoid names that lie. A `List<Order> orderList` that later becomes a `Set` is now a lying name — just call it `orders`.

Function Size and Single Level of Abstraction

A function should do one thing, and every line inside it should be at roughly the same “zoom level.” Mixing high-level steps (“place order”) with low-level details (string parsing, raw loops) in one method forces the reader to context-switch constantly.

```
// Before: one method, three levels of abstraction mixed together
public void placeOrder(Order order) {
    // high-level: validate
    if (order.getItems().isEmpty()) {
        throw new IllegalArgumentException("Order has no items");
    }
    // low-level: compute total by hand
    BigDecimal total = BigDecimal.ZERO;
    for (OrderItem item : order.getItems()) {
        total = total.add(item.getPrice().multiply(BigDecimal.valueOf(
↪ item.getQuantity())));
    }
    // low-level: talk to payment gateway
    HttpURLConnection conn = null;
    try {
        conn = (HttpURLConnection) new
↪ URL("https://pay.example.com").openConnection();
        conn.setDoOutput(true);
        // ... write request body, read response ...
    } catch (IOException e) {
        throw new PaymentException(e);
    }
    // high-level: persist
    orderRepository.save(order);
}
```



```

// After: each method reads at one level of abstraction
public void placeOrder(Order order) {
    validate(order);
    BigDecimal total = order.calculateTotal();
    paymentGateway.charge(order.getCustomerId(), total);
    orderRepository.save(order);
}

private void validate(Order order) {
    if (order.getItems().isEmpty()) {
        throw new IllegalArgumentException("Order has no items");
    }
}

```

The payment call and the total calculation moved into their own well-named units (paymentGateway.charge, order.calculateTotal). placeOrder now reads like a short story: validate, total, charge, save.

Argument Count

More than three or four parameters is a warning sign. Callers can pass arguments in the wrong order, and the call site becomes unreadable.

```

// Before: which boolean is which? easy to swap
createUser("Ana", "ana@example.com", true, false, true);

// After: a small parameter object makes the call self-explanatory
record UserCreationRequest(String name, String email, boolean
→ sendWelcomeEmail,
                                boolean requireEmailVerification, boolean
→ isAdmin) {}

createUser(new UserCreationRequest("Ana", "ana@example.com", true, false,
→ true));

```

Even better, use named construction so the call site itself documents what each flag means:

```

createUser(UserCreationRequest.builder()
    .name("Ana")
    .email("ana@example.com")
    .sendWelcomeEmail(true)
    .requireEmailVerification(false)
    .isAdmin(true)

```

```
.build();
```

Boolean Flag Parameters

A boolean parameter often means the method secretly does two different things. That is a violation of “one thing per function” and it is easy to misread at the call site.

```
// Before: what does "true" mean here? you have to open the method to find
↪ out
void printReport(Report report, boolean detailed) {
    if (detailed) {
        printFullReport(report);
    } else {
        printSummary(report);
    }
}
printReport(report, true);
```

```
// After: split into two clearly named methods
void printDetailedReport(Report report) { ... }
void printSummaryReport(Report report) { ... }

printReport.printDetailedReport(report);
```

If the flag really represents a meaningful choice (not just “two code paths bolted together”), prefer an enum — it is self-documenting and the compiler catches typos that a boolean never would.

```
enum ReportDetail { SUMMARY, DETAILED }

void printReport(Report report, ReportDetail detail) {
    switch (detail) {
        case SUMMARY -> printSummary(report);
        case DETAILED -> printFullReport(report);
    }
}
```

Avoiding Deep Nesting With Guard Clauses

Deeply nested `if` blocks are hard to follow because the reader has to hold every condition in their head at once. A **guard clause**—an early return/throw for the exceptional case—flattens the happy path.

```
// Before: the "real" logic is buried three levels deep
public BigDecimal calculateDiscount(Customer customer, Order order) {
    if (customer != null) {
        if (customer.isActive()) {
            if (order.getTotal().compareTo(BigDecimal.ZERO) > 0) {
                return
    ⇨ order.getTotal().multiply(customer.getDiscountRate());
            } else {
                return BigDecimal.ZERO;
            }
        } else {
            return BigDecimal.ZERO;
        }
    } else {
        return BigDecimal.ZERO;
    }
}
```

```
// After: guard clauses handle the edge cases up front, happy path is flat
public BigDecimal calculateDiscount(Customer customer, Order order) {
    if (customer == null || !customer.isActive()) {
        return BigDecimal.ZERO;
    }
    if (order.getTotal().compareTo(BigDecimal.ZERO) <= 0) {
        return BigDecimal.ZERO;
    }
    return order.getTotal().multiply(customer.getDiscountRate());
}
```

Comments That Explain Why, Not What

A comment that repeats what the code already says is noise. A useful comment explains a decision the code *cannot* express: why an unusual approach was chosen, a business rule, or a workaround for a bug.

```

// Before: comment adds nothing the code doesn't already say
// increment i by 1
i++;

// loop through all users
for (User user : users) { ... }

```

```

// After: the comment carries information the code cannot
// Retry with exponential backoff: the payment provider rate-limits
// bursts and returns 429 without a Retry-After header (see INC-4821).
for (int attempt = 0; attempt < MAX_RETRIES; attempt++) {
    try {
        return paymentClient.charge(request);
    } catch (RateLimitException e) {
        sleep(backoffMillis(attempt));
    }
}

```

Dead Code

Dead code — commented-out blocks, unused methods, unreachable branches — costs nothing to write but confuses every future reader, who has to figure out whether it is safe to delete. Version control already remembers old code; delete it instead of commenting it out.

```

// Before
public int calculateShipping(Order order) {
    // int oldRate = order.getWeight() * 2;
    // return oldRate + BASE_FEE;
    return order.getWeight() * NEW_RATE + BASE_FEE;
}

// After
public int calculateShipping(Order order) {
    return order.getWeight() * NEW_RATE + BASE_FEE;
}

```

Magic Numbers

A magic number is a literal whose meaning is not obvious from context. Give it a name.

```

// Before: why 0.075? why 3?
BigDecimal tax = price.multiply(BigDecimal.valueOf(0.075));
if (failedAttempts >= 3) lockAccount();

// After
private static final BigDecimal SALES_TAX_RATE =
↳ BigDecimal.valueOf(0.075);
private static final int MAX_LOGIN_ATTEMPTS = 3;

BigDecimal tax = price.multiply(SALES_TAX_RATE);
if (failedAttempts >= MAX_LOGIN_ATTEMPTS) lockAccount();

```

Consistent Formatting

Consistent formatting (indentation, brace style, import ordering) is not about aesthetics — it removes noise from diffs and code review. A file that mixes tabs and spaces, or reformats unrelated lines in a “fix a typo” PR, hides the real change.

```

// Before: inconsistent brace style and spacing makes diffs noisy
public void run(){
    if(ready)
    {
        doWork();
    }
    else { skip(); }
}

```

```

// After: one consistent style (enforced by a formatter like
↳ Spotless/google-java-format)
public void run() {
    if (ready) {
        doWork();
    } else {
        skip();
    }
}

```

In real teams this is automated with a formatter and a CI check, not manual discipline — humans are inconsistent, tools are not.

Small Classes

A class should have a short, focused responsibility. If you cannot describe what a class does in one sentence without using “and,” it is probably doing too much.

```
// Before: one class doing validation, persistence, email, and formatting
class UserManager {
    void validate(User u) { ... }
    void save(User u) { ... }
    void sendWelcomeEmail(User u) { ... }
    String formatForDisplay(User u) { ... }
}
```

```
// After: each responsibility gets its own small class
class UserValidator { void validate(User u) { ... } }
class UserRepository { void save(User u) { ... } }
class WelcomeEmailSender { void send(User u) { ... } }
class UserFormatter { String formatForDisplay(User u) { ... } }
```

This is exactly the Single Responsibility Principle, covered next.

SOLID Principles

SOLID is an acronym for five design principles that make object-oriented code easier to maintain and extend. Each one has a classic violation and a classic fix.

Single Responsibility Principle (SRP)

A class should have only one reason to change. If a class mixes unrelated concerns, a change to one concern risks breaking the other.

```
// Violation: Invoice mixes business data, persistence, and printing
class Invoice {
    private List<LineItem> items;

    BigDecimal calculateTotal() {
        return
        ↪ items.stream().map(LineItem::price).reduce(BigDecimal.ZERO,
        ↪ BigDecimal::add);
    }
}
```

```

    void saveToDatabase() {
        // JDBC code directly inside the domain class
    }

    void printToConsole() {
        System.out.println("Invoice total: " + calculateTotal());
    }
}

```

```

// Fixed: each class has exactly one reason to change
class Invoice {
    private final List<LineItem> items;

    BigDecimal calculateTotal() {
        return
        ⇨ items.stream().map(LineItem::price).reduce(BigDecimal.ZERO,
        ⇨ BigDecimal::add);
    }

    List<LineItem> items() { return items; }
}

class InvoiceRepository {
    void save(Invoice invoice) { /* JDBC code lives here */ }
}

class InvoicePrinter {
    void print(Invoice invoice) {
        System.out.println("Invoice total: " + invoice.calculateTotal());
    }
}

```

Now a change to the database schema touches only `InvoiceRepository`; a change to print formatting touches only `InvoicePrinter`. `Invoice` itself only changes when the business rules for an invoice change.

Open/Closed Principle (OCP)

Classes should be open for extension but closed for modification. You should be able to add new behavior without editing existing, already-tested code.

```

// Violation: adding a new shape means editing this method every time
class AreaCalculator {
    double area(Object shape) {

```

```

        if (shape instanceof Circle c) {
            return Math.PI * c.radius() * c.radius();
        } else if (shape instanceof Rectangle r) {
            return r.width() * r.height();
        }
        // every new shape adds another branch here
        throw new IllegalArgumentException("Unknown shape");
    }
}

```

```

// Fixed: new shapes extend the abstraction, AreaCalculator never changes
interface Shape {
    double area();
}

record Circle(double radius) implements Shape {
    public double area() { return Math.PI * radius * radius; }
}

record Rectangle(double width, double height) implements Shape {
    public double area() { return width * height; }
}

class AreaCalculator {
    double area(Shape shape) {
        return shape.area();
    }
}

// Adding Triangle later requires zero changes to AreaCalculator:
record Triangle(double base, double height) implements Shape {
    public double area() { return 0.5 * base * height; }
}

```

Java 21's sealed interfaces give you the best of both worlds when you *do* want the compiler to force you to handle every case (e.g. exhaustive switch), which is useful when the set of variants is meant to be closed rather than open — see the Design Patterns section for a Visitor-style example.

Liskov Substitution Principle (LSP)

Subtypes must be usable anywhere their supertype is expected, without breaking the caller's assumptions. The classic textbook violation is `Square` extends `Rectangle`.


```

// Violation: Square breaks the Rectangle contract (setWidth no longer
→ only
// changes width, it silently changes height too)
class Rectangle {
    protected int width, height;
    void setWidth(int w) { this.width = w; }
    void setHeight(int h) { this.height = h; }
    int area() { return width * height; }
}

class Square extends Rectangle {
    @Override void setWidth(int w) { width = w; height = w; }
    @Override void setHeight(int h) { width = h; height = h; }
}

void resize(Rectangle r) {
    r.setWidth(5);
    r.setHeight(10);
    assert r.area() == 50; // fails for Square! area() is 100
}

```

```

// Fixed: don't force an is-a relationship that doesn't hold behaviorally.
// Model the shared behavior with a common interface instead of
→ inheritance.
interface Shape {
    int area();
}

final class Rectangle implements Shape {
    private final int width, height;
    Rectangle(int width, int height) { this.width = width; this.height =
→ height; }
    public int area() { return width * height; }
}

final class Square implements Shape {
    private final int side;
    Square(int side) { this.side = side; }
    public int area() { return side * side; }
}

```

Square is no longer forced to honor a mutable-rectangle contract it cannot actually satisfy. If you must keep an inheritance hierarchy, the rule is: a subclass must not weaken postconditions, strengthen preconditions, or throw new exceptions the caller of the base type does not expect.

Interface Segregation Principle (ISP)

Clients should not be forced to depend on methods they do not use. Large, “fat” interfaces force implementers to write dummy/unsupported methods.

```
// Violation: a read-only report can't sanely implement print()
interface Machine {
    void print(Document d);
    void scan(Document d);
    void fax(Document d);
}

class OldPrinter implements Machine {
    public void print(Document d) { /* ok */ }
    public void scan(Document d) { throw new
↳ UnsupportedOperationException(); }
    public void fax(Document d) { throw new
↳ UnsupportedOperationException(); }
}
```

```
// Fixed: split into small, focused interfaces
interface Printer { void print(Document d); }
interface Scanner { void scan(Document d); }
interface Fax { void fax(Document d); }

class OldPrinter implements Printer {
    public void print(Document d) { /* ok */ }
}

class AllInOnePrinter implements Printer, Scanner, Fax {
    public void print(Document d) { ... }
    public void scan(Document d) { ... }
    public void fax(Document d) { ... }
}
```

OldPrinter now implements exactly what it can support. No dummy methods, no UnsupportedOperationException traps for callers.

Dependency Inversion Principle (DIP)

High-level code should depend on abstractions, not on concrete low-level implementations. This is what makes classes testable and swappable.

```

// Violation: OrderService is hard-wired to a concrete SMTP sender.
// You cannot test it without sending real emails, and you cannot swap
// providers without editing OrderService.
class SmtEmailSender {
    void send(String to, String body) { /* talks to an SMTP server */ }
}

class OrderService {
    private final SmtEmailSender emailSender = new SmtEmailSender();

    void placeOrder(Order order) {
        // ... business logic ...
        emailSender.send(order.getCustomerEmail(), "Order confirmed");
    }
}

```

```

// Fixed: OrderService depends on an interface; the concrete
// implementation is injected from outside (constructor injection).
interface NotificationSender {
    void send(String to, String body);
}

class SmtEmailSender implements NotificationSender {
    public void send(String to, String body) { /* SMTP */ }
}

class SmsSender implements NotificationSender {
    public void send(String to, String body) { /* SMS gateway */ }
}

class OrderService {
    private final NotificationSender notificationSender;

    OrderService(NotificationSender notificationSender) {
        this.notificationSender = notificationSender;
    }

    void placeOrder(Order order) {
        // ... business logic ...
        notificationSender.send(order.getCustomerEmail(), "Order
⇒ confirmed");
    }
}

// In tests:
OrderService service = new OrderService(new NotificationSender() {

```

```

        public void send(String to, String body) { /* record call, assert on
↳ it */ }
        });

```

`OrderService` no longer knows or cares whether emails are sent over SMTP, SMS, or a mock in a test. This is the foundation dependency-injection frameworks (Spring, Guice, CDI) are built on.

Effective Java Best Practices

Joshua Bloch's *Effective Java* is the most-quoted Java book in code review. Below are roughly twenty of its highest-value items, each with a one-paragraph explanation and a short example.

1. Prefer static factory methods over constructors

A static factory method can have a descriptive name, is not required to create a new object every call, and can return a subtype. A constructor always has the same name as the class and always returns exactly that type.

```

class Connection {
    private Connection() {}

    // Name explains intent; a constructor could not do this.
    public static Connection newTcpConnection(String host, int port) {
↳ ... return new Connection(); }
    public static Connection newInMemoryConnection() { ... return new
↳ Connection(); }
}

```

2. Use a builder when a constructor would need many parameters

Telescoping constructors (many overloads with increasing parameter counts) are error-prone and hard to read at the call site. A builder makes each parameter self-documenting.

```

class Pizza {
    private final int size;
    private final boolean cheese;
    private final boolean pepperoni;

```

```

        private Pizza(Builder b) {
            this.size = b.size; this.cheese = b.cheese; this.pepperoni =
↪ b.pepperoni;
        }

        static class Builder {
            private final int size;
            private boolean cheese;
            private boolean pepperoni;

            Builder(int size) { this.size = size; }
            Builder cheese(boolean value) { this.cheese = value; return this;
↪ }
            Builder pepperoni(boolean value) { this.pepperoni = value; return
↪ this; }
            Pizza build() { return new Pizza(this); }
        }

        Pizza pizza = new Pizza.Builder(12).cheese(true).pepperoni(true).build();

```

3. Use a single-element enum for singletons

An enum singleton gets serialization safety and reflection-attack resistance for free, which a hand-rolled singleton does not.

```

enum AppConfig {
    INSTANCE;
    private final Properties props = loadProperties();
    Properties properties() { return props; }
}

AppConfig.INSTANCE.properties();

```

4. Prefer dependency injection over hardwiring dependencies

Hardwiring a concrete dependency (`new SmtplibEmailSender()`) inside a class, as shown in the DIP example above, makes that class impossible to unit test in isolation. Pass dependencies into the constructor instead.

```
// Hardwired – cannot substitute a fake in tests
class Report { private final Formatter formatter = new PdfFormatter(); }

// Injected – testable and swappable
class Report {
    private final Formatter formatter;
    Report(Formatter formatter) { this.formatter = formatter; }
}
```

5. Avoid finalizers and the old `Object.finalize()`; use `Cleaner` or `try-with-resources`

Finalizers run at an unpredictable time (or never), can hurt performance, and can even resurrect objects. Try-with-resources with `AutoCloseable` is the correct default; `java.lang.ref.Cleaner` is the safety-net alternative when you need last-resort cleanup.

```
class FileHandle implements AutoCloseable {
    private final RandomAccessFile file;
    FileHandle(String path) throws IOException { file = new
↪ RandomAccessFile(path, "r"); }
    public void close() throws IOException { file.close(); }
}

try (FileHandle handle = new FileHandle("data.bin")) {
    // use handle
} // closed deterministically, no finalizer needed
```

6. Obey the `equals/hashCode` contract

If you override `equals`, you must override `hashCode` so that equal objects have equal hash codes — otherwise the object breaks in `HashMap/HashSet`. Since Java 16, a record generates both correctly for you.

```
// Manual, error-prone version
class Point {
    final int x, y;
    Point(int x, int y) { this.x = x; this.y = y; }
    @Override public boolean equals(Object o) {
        return o instanceof Point p && p.x == x && p.y == y;
    }
    @Override public int hashCode() { return Objects.hash(x, y); }
}
```

```
// Modern version: equals/hashCode/toString generated correctly, always
record Point(int x, int y) {}
```

7. Minimize mutability

Immutable objects are simpler to reason about, are automatically thread-safe, and can be freely shared. Make fields `final`, do not provide mutators, and ensure the class cannot be extended in a way that breaks invariants.

```
// Mutable – a caller can corrupt shared state
class Money {
    private BigDecimal amount;
    void setAmount(BigDecimal amount) { this.amount = amount; }
}

// Immutable – every "change" returns a new instance
final class Money {
    private final BigDecimal amount;
    Money(BigDecimal amount) { this.amount = amount; }
    Money plus(Money other) { return new
↪ Money(this.amount.add(other.amount)); }
}
```

8. Favor composition over inheritance

Inheritance couples a subclass to the internal implementation details of its superclass; a change in the superclass can silently break subclasses (“fragile base class” problem). Composition — holding a reference to another object and delegating to it — is more flexible.

```
// Inheritance: ArrayList's internals leak through; size() may not
// behave as expected if addAll() is implemented in terms of add()
class InstrumentedSet<E> extends HashSet<E> {
    int addCount = 0;
    @Override public boolean add(E e) { addCount++; return super.add(e); }
    @Override public boolean addAll(Collection<? extends E> c) {
        addCount += c.size();
        return super.addAll(c); // may double-count if addAll calls add()
↪ internally
    }
}
```

```
// Composition: wrap a Set, forward calls, no fragile-base-class risk
class InstrumentedSet<E> {
    private final Set<E> delegate;
    int addCount = 0;
    InstrumentedSet(Set<E> delegate) { this.delegate = delegate; }
    boolean add(E e) { addCount++; return delegate.add(e); }
    boolean addAll(Collection<? extends E> c) { addCount += c.size();
↵ return delegate.addAll(c); }
}
```

9. Design and document for inheritance, or prohibit it

If a class is not explicitly designed to be extended (documented “self-use” behavior, no calls from constructors to overridable methods), mark it `final` or make its constructors package-private. Otherwise you invite the fragile-base-class problems from item 8.

```
// Dangerous: superclass constructor calls an overridable method;
// subclass fields are not yet initialized when overridden() runs
class Base {
    Base() { overridden(); }
    void overridden() {}
}

// Safe: prevent inheritance entirely when it wasn't designed for it
final class Utility {
    private Utility() {}
    static int square(int x) { return x * x; }
}
```

10. Prefer interfaces to abstract classes for defining types

A class can implement multiple interfaces but extend only one abstract class. Interfaces (with default methods) usually give you enough shared behavior without giving up multiple inheritance of type.

```
interface Flyer { default void takeOff() { System.out.println("Taking
↵ off"); } }
interface Swimmer { default void dive() { System.out.println("Diving"); }
↵ }

// A Duck can be both – impossible with two abstract base classes
class Duck implements Flyer, Swimmer {}
```


11. Prefer lists to arrays

Arrays are covariant and not truly type-safe at runtime (`ArrayStoreException`), and they don't play well with generics. Prefer `List<T>`.

```
// Arrays: compiles, fails at runtime
Object[] objects = new String[1];
objects[0] = 42; // throws ArrayStoreException at runtime

// Lists: caught at compile time instead
List<String> strings = new ArrayList<>();
// strings.add(42); // does not compile
```

12. Favor generics, and use bounded wildcards to increase API flexibility

Generics give compile-time type safety instead of runtime `ClassCastException`. Bounded wildcards (`? extends`, `? super`) let an API accept a wider range of argument types (the “PECS” rule: **P**roducer **E**xtends, **C**onsumer **S**uper).

```
// Without wildcards: only List<Number> works, List<Integer> is rejected
void printAll(List<Number> list) { list.forEach(System.out::println); }

// With a bounded wildcard: any subtype of Number works too (producer ->
→ extends)
void printAll(List<? extends Number> list) {
    → list.forEach(System.out::println); }

// Consumer -> super: this can accept a destination for any supertype of
→ Integer
void addIntegers(List<? super Integer> destination) { destination.add(1);
    → destination.add(2); }
```

13. Prefer enums to int constants

`int` constants (“int enum pattern”) give no type safety and no useful `toString()`. Enums are type-checked, printable, and can carry behavior.

```
// int constants – nothing stops you from passing the wrong "kind" of int
class Season { static final int SPRING = 0, SUMMER = 1, FALL = 2, WINTER
    → = 3; }
void plant(int season) { ... }
plant(Season.SUMMER); // fine
```

```
plant(42);           // compiles, wrong at runtime

// enum – type-checked, self-documenting, switch-friendly
enum Season { SPRING, SUMMER, FALL, WINTER }
void plant(Season season) { ... }
```

14. Return empty collections or arrays, never null

Returning null for “no results” forces every caller to null-check, and someone eventually forgets and gets a `NullPointerException`. Return an empty collection instead.

```
// Before: every caller must remember to null-check
List<Order> findOrders(String customerId) {
    return orders.isEmpty() ? null : orders;
}

// After: caller can always safely iterate
List<Order> findOrders(String customerId) {
    return orders.isEmpty() ? Collections.emptyList() : orders;
}
```

15. Use Optional judiciously — mainly as a return type, not everywhere

`Optional<T>` is great for a method’s return type when “no result” is a normal, expected outcome. It is a poor fit for fields, method parameters, or collection elements — those usages just add wrapping overhead without benefit.

```
// Good use: return type communicates "might be absent"
Optional<User> findByEmail(String email) { ... }

Optional<User> user = findByEmail("a@example.com");
user.ifPresentOrElse(this::greet, () -> log.warn("not found"));

// Bad use: Optional as a field or parameter just adds ceremony
class Order {
    private Optional<String> couponCode; // avoid – use null or a default
    ↪ value/empty string instead
}
```

16. Minimize the scope of local variables

Declare a variable as close as possible to where it is first used, and in the narrowest block possible. This shrinks the window in which a reader has to track its value.

```
// Before: i is declared far from where the loop actually needs it
int i;
// ... 20 lines of unrelated code ...
for (i = 0; i < items.size(); i++) { process(items.get(i)); }

// After: declared right where it's needed, scoped to the loop
for (int i = 0; i < items.size(); i++) { process(items.get(i)); }
```

17. Prefer for-each loops to traditional index-based for loops

for-each removes the off-by-one and index-mutation bugs that plague manual index loops, and it works uniformly across arrays, lists, and any `Iterable`.

```
// Index-based: easy to get the bound wrong
for (int i = 0; i <= items.size(); i++) { // bug: should be <
    process(items.get(i));
}

// for-each: no index to get wrong
for (Item item : items) {
    process(item);
}
```

18. Beware the performance cost of the String concatenation + operator in loops

Each `+` on strings inside a loop creates a new `String` object, giving $O(n^2)$ behavior for n concatenations. Use `StringBuilder` for loops.

```
// Before: O(n^2) - a new String is allocated on every iteration
String result = "";
for (String word : words) {
    result += word + " ";
}

// After: O(n) - one growable buffer
StringBuilder sb = new StringBuilder();
```

```
for (String word : words) {
    sb.append(word).append(' ');
}
String result = sb.toString();
```

19. Refer to objects by their interface, not their implementation type

Declaring variables and parameters using the interface type (`List`, `Map`, `Set`) instead of the concrete class (`ArrayList`, `HashMap`) makes it trivial to swap implementations later.

```
// Before: tied to ArrayList everywhere
ArrayList<String> names = new ArrayList<>();
void printNames(ArrayList<String> names) { ... }

// After: depends only on the List contract
List<String> names = new ArrayList<>();
void printNames(List<String> names) { ... }
// Switching to LinkedList later requires changing only one line
```

20. Prefer primitives to boxed primitives when you don't need null

Boxed types (`Integer`, `Long`, `Double`) allow `null`, incur autoboxing overhead, and `==` compares references (not values) once outside the small cached integer range — a classic bug source. Use primitives unless you specifically need nullability or generics.

```
// Boxed: subtle == bug and unnecessary allocation
Integer a = 1000;
Integer b = 1000;
System.out.println(a == b); // false! outside the -128..127 cache range

// Primitive: no boxing, no reference-equality trap
int a = 1000;
int b = 1000;
System.out.println(a == b); // true, compares values
```

21. Document thread safety

A class's Javadoc should state whether it is thread-safe, conditionally thread-safe, or not thread-safe at all. Without this, callers either over-synchronize (hurting performance) or under-synchronize (causing bugs).

```

/**
 * Thread-safe: all mutating methods are synchronized internally.
 * Callers do not need external locking.
 */
class Counter {
    private int count;
    synchronized void increment() { count++; }
    synchronized int get() { return count; }
}

/**
 * Not thread-safe. Callers must provide external synchronization
 * if this instance is shared across threads.
 */
class Buffer { ... }

```

22. Prefer executors and the `java.util.concurrent` framework to raw threads

Creating raw `Thread` objects by hand gives you no lifecycle management, no bounded resource usage, and no easy way to wait for results. `ExecutorService` (and Java 21's virtual threads) handle pooling, scheduling, and shutdown correctly.

```

// Before: unmanaged raw thread, no pooling, fire-and-forget
new Thread(() -> processOrder(order)).start();

// After: managed executor, bounded resources, results and errors handled
try (ExecutorService executor =
    Executors.newVirtualThreadPerTaskExecutor()) {
    Future<Receipt> future = executor.submit(() -> processOrder(order));
    Receipt receipt = future.get();
}

```

Design Patterns in Java

A **design pattern** is a proven, reusable solution to a common design problem. Patterns are grouped into three families: **creational** (object creation), **structural** (composing classes/objects), and **behavioral** (communication between objects).

Pattern	Family	When to use	Modern Java alternative
Builder	Creational	Many constructor parameters, optional fields	Record with a builder, or named factory methods
Factory Method	Creational	Subclass decides which concrete type to create	Static factory methods (item 1 above)
Abstract Factory	Creational	Create families of related objects	Dependency injection container
Singleton	Creational	Exactly one shared instance	Enum singleton, or DI-managed scope
Strategy	Behavioral	Swap an algorithm at runtime	Lambda / method reference implementing a functional interface
Template Method	Behavioral	Fixed skeleton, customizable steps	Default methods on an interface, or a Consumer/Function parameter
Observer	Behavioral	Notify many listeners of a change	<code>java.util.function</code> callbacks, event bus, Flow/reactive streams
Decorator	Structural	Add behavior without changing the class	Wrapping with lambdas, Stream pipelines
Adapter	Structural	Make an incompatible interface fit	A thin wrapper class or method reference
Facade	Structural	Simplify a complex subsystem behind one API	A service class exposing a narrow API
Proxy	Structural	Control access (lazy load, security, remote)	Dynamic proxies, AOP frameworks (Spring AOP)
Command	Behavioral	Encapsulate a request as an object	Runnable/Callable lambdas
Chain of Responsibility	Behavioral	Pass a request along a chain of handlers	Stream of handlers, servlet filter chains
Iterator	Behavioral	Traverse a collection without exposing internals	Built into Iterable, enhanced for-each
State	Behavioral	Behavior changes with internal state	Sealed interface + switch pattern matching
Visitor	Behavioral	Operate over a closed set of types	Sealed interface + exhaustive switch pattern matching

Builder

```

class Computer {
    private final String cpu;
    private final int ramGb;
    private final boolean hasSsd;

    private Computer(Builder b) {
        this.cpu = b.cpu; this.ramGb = b.ramGb; this.hasSsd = b.hasSsd;
    }

    static class Builder {
        private String cpu = "generic-cpu";
        private int ramGb = 8;
        private boolean hasSsd = true;

        Builder cpu(String cpu) { this.cpu = cpu; return this; }
        Builder ramGb(int ramGb) { this.ramGb = ramGb; return this; }
        Builder hasSsd(boolean hasSsd) { this.hasSsd = hasSsd; return
⇒ this; }

        Computer build() { return new Computer(this); }
    }
}

Computer gamingPc = new
⇒ Computer.Builder().cpu("i9").ramGb(32).hasSsd(true).build();

```

Factory Method

```

interface Notification {
    void send(String message);
}

class EmailNotification implements Notification {
    public void send(String message) { System.out.println("Email: " +
⇒ message); }
}

class SmsNotification implements Notification {
    public void send(String message) { System.out.println("SMS: " +
⇒ message); }
}

class NotificationFactory {
    static Notification create(String type) {

```

```

        return switch (type) {
            case "email" -> new EmailNotification();
            case "sms" -> new SmsNotification();
            default -> throw new IllegalArgumentException("Unknown type:
↪ " + type);
        };
    }
}

Notification n = NotificationFactory.create("email");
n.send("Your order has shipped");

```

Abstract Factory

```

interface Button { void render(); }
interface Checkbox { void render(); }

interface UiFactory {
    Button createButton();
    Checkbox createCheckbox();
}

class DarkThemeFactory implements UiFactory {
    public Button createButton() { return () -> System.out.println("Dark
↪ button"); }
    public Checkbox createCheckbox() { return () ->
↪ System.out.println("Dark checkbox"); }
}

class LightThemeFactory implements UiFactory {
    public Button createButton() { return () -> System.out.println("Light
↪ button"); }
    public Checkbox createCheckbox() { return () ->
↪ System.out.println("Light checkbox"); }
}

// Note: Button/Checkbox declared as functional interfaces here for
↪ brevity via lambdas
void renderUi(UiFactory factory) {
    factory.createButton().render();
    factory.createCheckbox().render();
}

renderUi(new DarkThemeFactory()); // switching the whole family is one
↪ line

```


Singleton

```
// Classic non-enum singleton (shown for comparison – avoid in new code)
class LegacyConfig {
    private static LegacyConfig instance;
    private LegacyConfig() {}
    static synchronized LegacyConfig getInstance() {
        if (instance == null) instance = new LegacyConfig();
        return instance;
    }
}

// Modern replacement: enum singleton – serialization-safe, thread-safe
↳ for free
enum AppConfig {
    INSTANCE;
    private final Map<String, String> settings = new
↳ ConcurrentHashMap<>();
    String get(String key) { return settings.get(key); }
}

AppConfig.INSTANCE.get("timeout");
```

Strategy

```
interface DiscountStrategy {
    BigDecimal apply(BigDecimal price);
}

class PercentageDiscount implements DiscountStrategy {
    private final BigDecimal percent;
    PercentageDiscount(BigDecimal percent) { this.percent = percent; }
    public BigDecimal apply(BigDecimal price) { return
↳ price.subtract(price.multiply(percent)); }
}

class Checkout {
    private final DiscountStrategy strategy;
    Checkout(DiscountStrategy strategy) { this.strategy = strategy; }
    BigDecimal total(BigDecimal price) { return strategy.apply(price); }
}

// Classic: a class implementing the interface
Checkout classic = new Checkout(new
↳ PercentageDiscount(BigDecimal.valueOf(0.10)));
```

```

// Modern replacement: DiscountStrategy is a functional interface, so a
↳ lambda works directly
Checkout modern = new Checkout(price ->
↳ price.subtract(price.multiply(BigDecimal.valueOf(0.10))));

```

Template Method

```

abstract class DataExporter {
    // Template method: fixed skeleton
    final void export() {
        openConnection();
        writeHeader();
        writeRows();
        closeConnection();
    }
    void openConnection() { System.out.println("Opening"); }
    void closeConnection() { System.out.println("Closing"); }
    abstract void writeHeader();
    abstract void writeRows();
}

class CsvExporter extends DataExporter {
    void writeHeader() { System.out.println("id,name"); }
    void writeRows() { System.out.println("1,Ana"); }
}

// Modern alternative for simple cases: pass the customizable steps as
↳ lambdas
void export(Runnable writeHeader, Runnable writeRows) {
    System.out.println("Opening");
    writeHeader.run();
    writeRows.run();
    System.out.println("Closing");
}
export(() -> System.out.println("id,name"), () ->
↳ System.out.println("1,Ana"));

```

Observer

```

interface OrderListener {
    void onOrderPlaced(Order order);
}

class OrderService {
    private final List<OrderListener> listeners = new ArrayList<>();
    void addListener(OrderListener listener) { listeners.add(listener); }
    void placeOrder(Order order) {
        // ... business logic ...
        listeners.forEach(l -> l.onOrderPlaced(order));
    }
}

// Modern replacement: OrderListener is a functional interface, use
↳ lambdas directly
OrderService service = new OrderService();
service.addListener(order -> System.out.println("Send confirmation email
↳ for " + order));
service.addListener(order -> System.out.println("Update analytics for " +
↳ order));

```

Decorator

```

interface Coffee {
    BigDecimal cost();
    String description();
}

class SimpleCoffee implements Coffee {
    public BigDecimal cost() { return BigDecimal.valueOf(2.0); }
    public String description() { return "Coffee"; }
}

abstract class CoffeeDecorator implements Coffee {
    protected final Coffee delegate;
    CoffeeDecorator(Coffee delegate) { this.delegate = delegate; }
}

class MilkDecorator extends CoffeeDecorator {
    MilkDecorator(Coffee delegate) { super(delegate); }
    public BigDecimal cost() { return
↳ delegate.cost().add(BigDecimal.valueOf(0.5)); }

```

```

        public String description() { return delegate.description() + " +
↪ Milk"; }
    }

    Coffee order = new MilkDecorator(new SimpleCoffee());
    System.out.println(order.description() + ": $" + order.cost()); // Coffee
↪ + Milk: $2.5

```

Adapter

```

// Third-party class we cannot change, with an incompatible interface
class LegacyXmlParser {
    String parseXml(String xml) { return "parsed:" + xml; }
}

// The interface our code actually wants to use
interface JsonParser {
    String parseJson(String json);
}

// Adapter bridges the two
class XmlToJsonAdapter implements JsonParser {
    private final LegacyXmlParser legacy = new LegacyXmlParser();
    public String parseJson(String json) {
        String xmlEquivalent = convertJsonToXml(json);
        return legacy.parseXml(xmlEquivalent);
    }
    private String convertJsonToXml(String json) { return "<xml>" + json
↪ + "</xml>"; }
}

```

Facade

```

// A complex subsystem with many moving parts
class InventoryService { boolean reserve(String sku, int qty) { return
↪ true; } }
class PaymentService { void charge(String customerId, BigDecimal amount)
↪ {} }
class ShippingService { void schedule(String orderId) {} }

// Facade: one simple entry point that hides the subsystem's complexity
class OrderFacade {
    private final InventoryService inventory = new InventoryService();
}

```

```

        private final PaymentService payment = new PaymentService();
        private final ShippingService shipping = new ShippingService();

        void placeOrder(String orderId, String sku, int qty, String
⇒ customerId, BigDecimal amount) {
            inventory.reserve(sku, qty);
            payment.charge(customerId, amount);
            shipping.schedule(orderId);
        }
    }

    new OrderFacade().placeOrder("O-1", "SKU-42", 1, "C-9",
⇒ BigDecimal.valueOf(29.99));

```

Proxy

```

interface ImageLoader {
    void display();
}

class RealImage implements ImageLoader {
    private final String path;
    RealImage(String path) { this.path = path; loadFromDisk(); }
    private void loadFromDisk() { System.out.println("Loading " + path); }
    public void display() { System.out.println("Displaying " + path); }
}

// Proxy: defers the expensive load until display() is actually called
class LazyImageProxy implements ImageLoader {
    private final String path;
    private RealImage real;
    LazyImageProxy(String path) { this.path = path; }
    public void display() {
        if (real == null) {
            real = new RealImage(path); // loaded only on first use
        }
        real.display();
    }
}

```

Command

```

interface Command {
    void execute();
}

class LightOnCommand implements Command {
    public void execute() { System.out.println("Light on"); }
}

class RemoteControl {
    private final List<Command> history = new ArrayList<>();
    void submit(Command command) {
        command.execute();
        history.add(command);
    }
}

// Classic: a class implementing Command
new RemoteControl().submit(new LightOnCommand());

// Modern replacement: Command is a functional interface, lambda works
↳ directly
new RemoteControl().submit(() -> System.out.println("Light on"));

```

Chain of Responsibility

```

abstract class SupportHandler {
    protected SupportHandler next;
    SupportHandler setNext(SupportHandler next) { this.next = next;
    ↳ return next; }
    abstract void handle(String issue);
}

class Tier1Support extends SupportHandler {
    void handle(String issue) {
        if (issue.equals("password-reset")) {
            System.out.println("Tier1 handled: " + issue);
        } else if (next != null) {
            next.handle(issue);
        }
    }
}

class Tier2Support extends SupportHandler {

```

```

    void handle(String issue) {
        System.out.println("Tier2 handled: " + issue);
    }
}

SupportHandler chain = new Tier1Support();
chain.setNext(new Tier2Support());
chain.handle("database-outage"); // falls through to Tier2Support

```

Iterator

```

class NameCollection implements Iterable<String> {
    private final String[] names;
    NameCollection(String... names) { this.names = names; }

    public Iterator<String> iterator() {
        return new Iterator<>() {
            private int index = 0;
            public boolean hasNext() { return index < names.length; }
            public String next() { return names[index++]; }
        };
    }
}

for (String name : new NameCollection("Ana", "Bo", "Cy")) {
    System.out.println(name); // for-each works because we implement
    ↪ Iterable
}

```

State

```

// Modern approach: sealed interface + exhaustive switch pattern matching
sealed interface OrderState permits Placed, Shipped, Delivered {}
record Placed() implements OrderState {}
record Shipped(String trackingNumber) implements OrderState {}
record Delivered(LocalDate date) implements OrderState {}

String describe(OrderState state) {
    return switch (state) {
        case Placed p -> "Order placed";
        case Shipped s -> "Shipped, tracking: " + s.trackingNumber();
        case Delivered d -> "Delivered on " + d.date();
    };
}

```

```

        // no default needed – the compiler knows these are the only three
↪ states
        };
    }

```

Visitor — Replaced by Sealed Interfaces + Pattern Matching

The classic Visitor pattern exists to let you add new *operations* over a fixed set of types without editing those types (double-dispatch through `accept/visit` methods). In modern Java, if the set of types is genuinely closed, a sealed hierarchy plus an exhaustive switch achieves the same goal with far less boilerplate — and the compiler enforces exhaustiveness for you.

```

sealed interface Shape permits Circle, Square, Triangle {}
record Circle(double radius) implements Shape {}
record Square(double side) implements Shape {}
record Triangle(double base, double height) implements Shape {}

double area(Shape shape) {
    return switch (shape) {
        case Circle c -> Math.PI * c.radius() * c.radius();
        case Square s -> s.side() * s.side();
        case Triangle t -> 0.5 * t.base() * t.height();
    };
}

// Adding a new operation (e.g. perimeter) is just a new function with its
↪ own switch –
// no accept()/visit() plumbing needed on Shape itself.

```

API Design Best Practices

Designing a public API — a library, a service interface, a set of public classes — is different from writing internal code: once other people depend on it, every change is a potential breaking change.

Naming

Follow existing JDK conventions so your API feels familiar: `getX/isX` for accessors, verbs for actions, plural nouns for collections.


```

// Inconsistent, surprising names
class UserApi {
    User fetchUser(String id) { ... }    // getUser would match
    ↪ convention
    boolean userActive(String id) { ... } // isActive reads as a
    ↪ question
    List<User> userList() { ... }        // users() is more idiomatic
}

// Convention-following names
class UserApi {
    User getUser(String id) { ... }
    boolean isActive(String id) { ... }
    List<User> users() { ... }
}

```

Minimal Surface Area

Every public method, class, and field is a promise to callers. Expose only what is needed; keep everything else package-private or private. A smaller API is easier to learn, test, and evolve.

```

// Before: implementation details leak into the public API
public class OrderProcessor {
    public void validateOrder(Order order) { ... }
    public void calculateTax(Order order) { ... }
    public void chargeCard(Order order) { ... }
    public void process(Order order) {
        validateOrder(order); calculateTax(order); chargeCard(order);
    }
}

// After: only the one operation callers need is public
public class OrderProcessor {
    public void process(Order order) {
        validateOrder(order); calculateTax(order); chargeCard(order);
    }
    private void validateOrder(Order order) { ... }
    private void calculateTax(Order order) { ... }
    private void chargeCard(Order order) { ... }
}

```

Immutability by Default

Return types and parameters exposed by a public API should default to immutable unless there is a specific reason for mutability. This prevents a caller from mutating internal state through a returned reference.

```
// Before: caller can mutate the list backing your internal state
public class Team {
    private final List<String> members = new ArrayList<>();
    public List<String> getMembers() { return members; } // leaks a
↪ mutable reference
}

// After: return an immutable view/copy
public class Team {
    private final List<String> members = new ArrayList<>();
    public List<String> getMembers() { return List.copyOf(members); }
}
```

Nulls at Boundaries

Public API boundaries should be explicit about null: either forbid it (and fail fast) or explicitly document that it is allowed. Silent acceptance of null pushes the bug downstream to a confusing `NullPointerException` far from the real cause.

```
public void registerUser(String email, String referralCode) {
    Objects.requireNonNull(email, "email must not be null");
    // referralCode is allowed to be null – document it clearly
    // referralCode: nullable, meaning "no referral"
    ...
}
```

Optional for Returns Only

As in the Effective Java section, restrict `Optional` to return types. Do not accept `Optional` as a method parameter in a public API — it forces callers to wrap values just to call your method, and it is not `Serializable`.

```
// Avoid: forces every caller to wrap a value in Optional just to call
↪ this
public void setDiscount(Optional<BigDecimal> discount) { ... }
```

```

    // Prefer: an overload, or a sentinel/nullable value with a nullability
    ⇨ contract
    public void setDiscount(BigDecimal discount) { ... } // null = no discount,
    ⇨ documented
    public Optional<BigDecimal> getDiscount() { ... } // Optional is fine
    ⇨ as a return type

```

Parameter Validation and Fail Fast

Validate arguments at the top of a public method and throw immediately with a clear message. Failing fast at the boundary is far easier to debug than a mysterious failure deep inside the call stack.

```

    public BigDecimal calculateInterest(BigDecimal principal, double rate,
    ⇨ int years) {
        if (principal.signum() < 0) {
            throw new IllegalArgumentException("principal must not be negative: "
    ⇨ " + principal);
        }
        if (years < 0) {
            throw new IllegalArgumentException("years must not be negative: "
    ⇨ + years);
        }
        return principal.multiply(BigDecimal.valueOf(rate * years));
    }

```

Overload Sparingly

Too many overloads confuse callers about which one gets called, especially with auto-boxing, varargs, or null arguments. Prefer distinct, clearly named methods when behavior meaningfully differs.

```

    // Confusing: which overload does log(null) call? Ambiguous / surprising.
    void log(String message) { ... }
    void log(String message, Throwable t) { ... }
    void log(Object message) { ... }

    // Clearer: distinct names remove all ambiguity
    void logMessage(String message) { ... }
    void logError(String message, Throwable t) { ... }

```

Exception Contracts

Document exactly which exceptions a public method can throw, and keep that contract stable — it is part of your API. Prefer specific, well-named exceptions over generic ones (see Exception Handling chapter for full details).

```
/**
 * @throws UserNotFoundException if no user exists with the given id
 * @throws IllegalArgumentException if id is blank
 */
public User getUser(String id) throws UserNotFoundException { ... }
```

Backwards Compatibility: Binary vs Source Compatibility

Source compatible means existing client code still *compiles* against the new version. **Binary compatible** means existing *compiled* .class files still run against the new version without recompiling. Adding a new overload is usually both; removing a public method breaks both; changing a method's return type (even to a subtype) can break binary compatibility even though source may still compile.

```
// Adding an overload: source and binary compatible – old callers
↳ unaffected
public void save(Order order) { ... }
public void save(Order order, boolean async) { ... } // new, additive

// Removing/renaming a public method: breaks both source and binary
↳ compatibility
// public void save(Order order) { ... } // deleting this breaks every
↳ existing caller
```

@Deprecated(since, forRemoval)

Deprecate before removing. `since` documents when the replacement became available; `forRemoval = true` tells consumers this really is going away, not just “prefer the alternative.”

```
/**
 * @deprecated use {@link #save(Order, boolean)} instead.
 */
@Deprecated(since = "2.3", forRemoval = true)
public void save(Order order) {
    save(order, false);
}
```

```

}

public void save(Order order, boolean async) { ... }

```

Versioning

Follow semantic versioning (MAJOR.MINOR.PATCH): increment MAJOR for breaking changes, MINOR for backwards-compatible additions, PATCH for backwards-compatible bug fixes. This lets consumers set dependency ranges with confidence.

```

1.4.2 -> 1.5.0    // new feature, backwards compatible (MINOR bump)
1.5.0 -> 1.5.1    // bug fix only (PATCH bump)
1.5.1 -> 2.0.0    // removed a deprecated method (MAJOR bump, breaking)

```

Builders for Wide Constructors

As covered in Clean Code and Effective Java, expose a builder (or static factory with named parameters via a record) instead of a constructor with many parameters — this is doubly important in a public API, where the constructor signature becomes a long-term commitment.

```

public class HttpRequestConfig {
    // A public constructor with 8 parameters would be locked in forever.
    // A builder can add optional settings later without breaking callers.
    public static Builder builder() { return new Builder(); }

    public static final class Builder {
        private Duration timeout = Duration.ofSeconds(30);
        private int maxRetries = 3;
        public Builder timeout(Duration timeout) { this.timeout =
→ timeout; return this; }
        public Builder maxRetries(int maxRetries) { this.maxRetries =
→ maxRetries; return this; }
        public HttpRequestConfig build() { return new
→ HttpRequestConfig(this); }
    }
    // private fields/constructor omitted for brevity
}

```

Documenting Thread Safety and Nullability

Every public class and method should document whether it is thread-safe and whether parameters/return values may be `null`. This is not optional polish — it directly determines how a caller is allowed to use the API.

```
/**
 * Thread-safe cache. Safe to share a single instance across threads.
 *
 * @param key must not be null
 * @return the cached value, or {@code null} if absent (this method never
↳ throws for a cache miss)
 */
public V get(K key) { ... }
```

Common Code Smells

A **code smell** is a surface signal that deeper design trouble might be lurking — not a bug by itself, but a warning sign worth investigating.

Smell	Why it hurts	Refactoring
Long method	Hard to understand, test, and reuse pieces of	Extract method, guard clauses
Large class / God object	Too many responsibilities, high change risk	Split by responsibility (SRP)
Primitive obsession	Loses domain meaning and validation	Introduce small value types / records
Feature envy	A method cares more about another class's data than its own	Move the method to the class it envies
Data clumps	The same group of fields/params travels together everywhere	Extract a parameter object
Shotgun surgery	One logical change requires edits across many classes	Consolidate the responsibility into one place
Anemic domain model	Objects are just data bags; logic lives elsewhere	Move behavior into the domain objects
Boolean parameters	Ambiguous call sites, hidden branching	Split methods, or use an enum
Deep inheritance	Fragile, hard to trace behavior across many levels	Favor composition

Smell	Why it hurts	Refactoring
Utility-class dumping ground	Unrelated static methods with no cohesion	Split into focused, named classes
Stringly-typed code	No compile-time safety for what is really a fixed set of values	Use enums or dedicated types
Exception as control flow	Exceptions are slow and obscure normal logic paths	Use normal conditionals / Optional
Static abuse	Hidden global state, hard to test	Instance fields + dependency injection
Over-mocking	Tests couple to implementation, not behavior	Test through the public behavior/contract

Long Method

```

// Before: one long method mixing many steps
void processOrder(Order order) {
    if (order.getItems().isEmpty()) throw new
    ↪ IllegalArgumentException("empty order");
    BigDecimal total = BigDecimal.ZERO;
    for (OrderItem item : order.getItems()) {
        total = total.add(item.getPrice().multiply(BigDecimal.valueOf(
    ↪ item.getQuantity())));
    }
    if (order.getCoupon() != null) {
        total = total.subtract(total.multiply(order.getCoupon().rate()));
    }
    inventory.reserve(order);
    payment.charge(order.getCustomerId(), total);
    shipping.schedule(order);
}

```

```

// After: extracted into small, named steps
void processOrder(Order order) {
    validate(order);
    BigDecimal total = calculateTotal(order);
    inventory.reserve(order);
    payment.charge(order.getCustomerId(), total);
    shipping.schedule(order);
}

```

Large Class / God Object

```
// Before: one class does everything for the whole application
class ApplicationManager {
    void authenticateUser(String u, String p) { ... }
    void sendEmail(String to, String body) { ... }
    void generateReport() { ... }
    void backupDatabase() { ... }
}
```

```
// After: split by responsibility
class AuthService { void authenticate(String u, String p) { ... } }
class EmailService { void send(String to, String body) { ... } }
class ReportService { void generate() { ... } }
class BackupService { void backupDatabase() { ... } }
```

Primitive Obsession

```
// Before: a raw String can hold any garbage, no validation, no meaning
void sendEmail(String email) {
    if (!email.contains("@")) throw new IllegalArgumentException("bad
↪ email");
    ...
}

// After: a small value type validates itself and carries meaning
↪ everywhere
record EmailAddress(String value) {
    EmailAddress {
        if (!value.contains("@")) throw new IllegalArgumentException("bad
↪ email: " + value);
    }
}

void sendEmail(EmailAddress email) { ... } // impossible to pass an
↪ invalid one
```


Feature Envy

```
// Before: OrderPrinter reaches deep into Order's internals to compute
↪ something
// that arguably belongs on Order itself
class OrderPrinter {
    void print(Order order) {
        BigDecimal total = BigDecimal.ZERO;
        for (OrderItem item : order.getItems()) {
            total = total.add(item.getPrice().multiply(BigDecimal.valueOf
↪ (item.getQuantity())));
        }
        System.out.println("Total: " + total);
    }
}
```

```
// After: the computation moves to the class that owns the data
class Order {
    BigDecimal calculateTotal() {
        return items.stream()
            .map(i ->
↪ i.getPrice().multiply(BigDecimal.valueOf(i.getQuantity()))))
            .reduce(BigDecimal.ZERO, BigDecimal::add);
    }
}

class OrderPrinter {
    void print(Order order) {
        System.out.println("Total: " + order.calculateTotal());
    }
}
```

Data Clumps

```
// Before: street/city/zip always travel together as separate params
void shipOrder(String street, String city, String zip, Order order) { ...
↪ }
void billCustomer(String street, String city, String zip, Customer
↪ customer) { ... }
```

```
// After: extracted into a single cohesive type
record Address(String street, String city, String zip) {}

void shipOrder(Address address, Order order) { ... }
void billCustomer(Address address, Customer customer) { ... }
```

Shotgun Surgery

```
// Before: "tax rate" logic is duplicated in three unrelated classes.
// Changing the tax rule means hunting down and editing all three.
class InvoiceService { BigDecimal tax(BigDecimal amount) { return
↳ amount.multiply(BigDecimal.valueOf(0.08)); } }
class ReceiptService { BigDecimal tax(BigDecimal amount) { return
↳ amount.multiply(BigDecimal.valueOf(0.08)); } }
class ReportService { BigDecimal tax(BigDecimal amount) { return
↳ amount.multiply(BigDecimal.valueOf(0.08)); } }
```

```
// After: one authoritative place to change
class TaxCalculator {
    private static final BigDecimal RATE = BigDecimal.valueOf(0.08);
    BigDecimal tax(BigDecimal amount) { return amount.multiply(RATE); }
}
// InvoiceService, ReceiptService, ReportService all delegate to
↳ TaxCalculator
```

Anemic Domain Model

```
// Before: Order is just a data bag; all logic lives in a separate
↳ "service"
class Order {
    List<OrderItem> items;
    List<OrderItem> getItems() { return items; }
    void setItems(List<OrderItem> items) { this.items = items; }
}

class OrderService {
    BigDecimal calculateTotal(Order order) {
        return order.getItems().stream()
            .map(i ->
↳ i.getPrice().multiply(BigDecimal.valueOf(i.getQuantity())))
            .reduce(BigDecimal.ZERO, BigDecimal::add);
}
```

```
    }
}
```

```
// After: behavior moves into the domain object it belongs to
class Order {
    private final List<OrderItem> items;
    Order(List<OrderItem> items) { this.items = items; }

    BigDecimal calculateTotal() {
        return items.stream()
            .map(i ->
→ i.getPrice().multiply(BigDecimal.valueOf(i.getQuantity())))
            .reduce(BigDecimal.ZERO, BigDecimal::add);
    }
}
```

Boolean Parameters

```
// Before
void search(String query, boolean caseSensitive) { ... }
search("Java", true); // true meaning what, exactly, at a glance?
```

```
// After: an enum documents itself at the call site
enum CaseMode { SENSITIVE, INSENSITIVE }
void search(String query, CaseMode caseMode) { ... }
search("Java", CaseMode.SENSITIVE);
```

Deep Inheritance

```
// Before: five levels deep, unclear which ancestor defines what
class Animal {}
class Mammal extends Animal {}
class Carnivore extends Mammal {}
class Feline extends Carnivore {}
class Cat extends Feline {}
```

```
// After: flatten using composition – combine behaviors instead of
→ stacking classes
interface Diet { void eat(); }
```

```

interface Movement { void move(); }

class Cat {
    private final Diet diet;
    private final Movement movement;
    Cat(Diet diet, Movement movement) { this.diet = diet; this.movement =
↪ movement; }
}

```

Utility-Class Dumping Ground

```

// Before: unrelated static methods dumped into one "Utils" class
class Utils {
    static String formatDate(LocalDate d) { ... }
    static BigDecimal calculateTax(BigDecimal amount) { ... }
    static boolean isValidEmail(String s) { ... }
}

```

```

// After: each concern gets its own focused, named home
class DateFormatting { static String format(LocalDate d) { ... } }
class TaxCalculator { static BigDecimal calculate(BigDecimal amount) {
↪ ... } }
class EmailValidator { static boolean isValid(String s) { ... } }

```

Stringly-Typed Code

```

// Before: a "status" that is really a fixed set of values, stored as a
↪ raw String
void updateStatus(String status) {
    if (status.equals("PENDING")) { ... }
    else if (status.equals("Pending")) { ... } // typo/case variant
↪ silently does nothing
}

```

```

// After: an enum makes invalid values impossible to represent
enum OrderStatus { PENDING, SHIPPED, DELIVERED, CANCELLED }
void updateStatus(OrderStatus status) {
    switch (status) {
        case PENDING -> ...;
        case SHIPPED -> ...;

```

```

        default -> ...;
    }
}

```

Exception as Control Flow

*// Before: uses an exception to detect "not found" during normal, expected
 ↪ flow*

```

int findIndex(List<String> list, String target) {
    try {
        for (int i = 0; i < list.size(); i++) {
            if (list.get(i).equals(target)) throw new FoundException(i);
        }
    } catch (FoundException e) {
        return e.index;
    }
    return -1;
}

```

// After: a normal conditional; exceptions are reserved for actual errors

```

int findIndex(List<String> list, String target) {
    for (int i = 0; i < list.size(); i++) {
        if (list.get(i).equals(target)) return i;
    }
    return -1;
}

```

Static Abuse

*// Before: hidden global mutable state via static fields – hard to test in
 ↪ isolation*

```

class OrderCounter {
    static int count = 0;
    static void increment() { count++; }
}

```

// After: instance state, injected where needed, easy to isolate in tests

```

class OrderCounter {
    private int count = 0;
    void increment() { count++; }
}

```

```

    int get() { return count; }
}

```

Over-Mocking

```

// Before: test mocks every internal collaborator and asserts on call
↪ order,
// so it breaks the moment implementation details change even if behavior
↪ doesn't
@Test
void placeOrder_callsCollaboratorsInOrder() {
    OrderService service = new OrderService(mockInventory, mockPayment,
↪ mockShipping);
    service.placeOrder(order);
    InOrder inOrder = inOrder(mockInventory, mockPayment, mockShipping);
    inOrder.verify(mockInventory).reserve(order);
    inOrder.verify(mockPayment).charge(any(), any());
    inOrder.verify(mockShipping).schedule(order);
}

```

```

// After: test the observable behavior/contract, not the internal wiring
@Test
void placeOrder_reservesInventoryAndChargesCustomer() {
    OrderService service = new OrderService(realInventory, fakePayment,
↪ fakeShipping);
    service.placeOrder(order);
    assertThat(realInventory.isReserved(order)).isTrue();
    assertThat(fakePayment.chargedAmountFor(order)).isEqualTo(order.
↪ calculateTotal());
}

```

Common Code-Review Interview Pitfalls

- 1. Method does more than one thing, mixing abstraction levels.** Why it matters: it forces the reader to jump between “what” and “how,” making the method hard to skim and hard to test in isolation.

```

// Flag: validation, calculation, and I/O all inline in one method
void placeOrder(Order o) { /* validate + compute + call HTTP + save */
↪ }

```

2. **Boolean parameter with no context at the call site.** Why it matters: `foo(x, true)` tells the reader nothing about what `true` means without opening the method.

```
printReport(report, true); // true = ??? – prefer an enum or split
↳ methods
```

3. **`equals()` overridden without `hashCode()`.** Why it matters: breaks `HashMap`/`HashSet` — objects that are “equal” end up in different buckets and lookups silently fail.

```
class Point { boolean equals(Object o) { ... } /* no hashCode()! */ }
```

4. **Public method returns `null` instead of an empty collection.** Why it matters: pushes a mandatory null-check onto every caller; someone eventually forgets and gets an NPE.

```
List<Order> findOrders(String id) { return orders.isEmpty() ? null :
↳ orders; }
```

5. **New class extends a concrete class purely for code reuse.** Why it matters: couples the subclass to the superclass’s internals, which is exactly the fragile-base-class problem LSP and “favor composition” warn about.

```
class InstrumentedList extends ArrayList<String> { /* leaks ArrayList
↳ internals */ }
```

6. **Singleton implemented with a lazy-checked static field instead of an enum.** Why it matters: hand-rolled singletons are vulnerable to reflection and serialization attacks that recreate a second instance; enum singletons are not.

```
class Config { private static Config instance; static Config
↳ getInstance() { ... } }
```

7. **`Optional` used as a field or method parameter.** Why it matters: `Optional` was designed as a return type; using it for fields/parameters adds wrapping overhead and is not serializable.

```
class Order { Optional<String> couponCode; } // flag: should be a plain
↳ nullable String
```

- 8. String concatenation with + inside a loop.** Why it matters: quietly $O(n^2)$ — each iteration allocates a brand-new String; use StringBuilder.

```
String s = "";
for (String w : words) s += w; // flag
```

- 9. Magic numbers or literals with no named meaning.** Why it matters: the reader cannot tell if 3 is a retry count, a threshold, or an index without digging through history.

```
if (attempts >= 3) lockAccount(); // what does 3 mean, and why 3?
```

- 10. Catching a broad exception and swallowing it, or catching Throwable.** Why it matters: hides real bugs, and catching Throwable also swallows JVM-level Errors that should never be caught by application code.

```
try { risky(); } catch (Exception e) { /* nothing */ } // flag
```

- 11. New public API with a constructor that has more than four parameters.** Why it matters: callers can silently swap two same-typed arguments (e.g. two ints) with no compiler warning; a builder or record removes the ambiguity.

```
new User("Ana", "ana@x.com", true, false, 30, "US"); // flag: use a
→ builder
```

- 12. Exposing a mutable internal collection through a public getter.** Why it matters: any caller can mutate the object's internal state from outside, breaking encapsulation and invariants.

```
public List<String> getMembers() { return members; } // flag: return
→ List.copyOf(members)
```

- 13. Removing or renaming a public method without deprecation.** Why it matters: breaks both source and binary compatibility for every downstream consumer with no warning period.

```
// Flag if this method existed in a previous public release and just
→ disappears
// public void save(Order order) { ... }
```


- 14. Deep nested if blocks instead of guard clauses.** Why it matters: the reader must hold multiple nested conditions in their head at once to find the actual logic.

```
if (a != null) { if (a.isValid()) { if (b > 0) { doWork(); } } } //
↳ flag
```

- 15. A “Manager”/“Helper”/“Utils” class that keeps growing with unrelated static methods.** Why it matters: it becomes a God object with no cohesion, and every unrelated change collides in the same file.

```
class Utils { static String formatDate(...) {} static void
↳ chargeCard(...) {} } // flag
```

- 16. Design pattern boilerplate (Strategy/Command/Observer as full classes) where a lambda would do.** Why it matters: in modern Java, a one-line lambda implementing a functional interface is clearer than a whole extra class file for simple cases — flag unnecessary ceremony.

```
class AlwaysApprove implements DiscountStrategy { public BigDecimal
↳ apply(BigDecimal p) { return p; } }
// Prefer: DiscountStrategy noDiscount = price -> price;
```

- 17. Using a raw String/int where a fixed set of values exists (stringly-typed code).** Why it matters: no compile-time safety — typos like "Pending" vs "PENDING" compile fine and fail silently at runtime.

```
void setStatus(String status) { ... } // flag: should be an enum
↳ OrderStatus
```

- 18. Test mocks every collaborator and asserts call order instead of testing behavior.** Why it matters: the test breaks whenever internal wiring changes, even if the externally observable behavior is unchanged — a maintenance burden with no correctness benefit.

```
verify(mockA).method(); verify(mockB).method(); // flag if this is the
↳ entire assertion
```


Testing

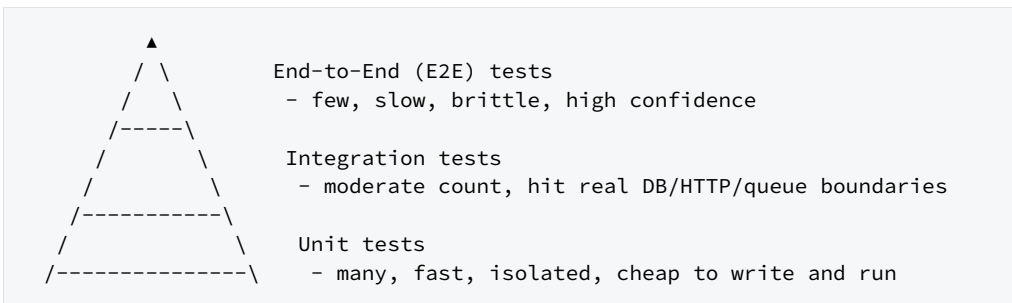
Code review is not just about the production code in the diff — it is at least as much about the tests that accompany it. A reviewer who cannot tell *what* a test proves, or who spots a test that mocks its way around the actual bug, will (and should) block the PR. This chapter covers JUnit 5 in depth — the framework almost every Java shop uses today — and Mockito, the de-facto mocking library, with an emphasis on the patterns and anti-patterns that come up constantly in review. We target JUnit Jupiter 5.10+ and Mockito 5.x on Java 21+.

Testing Fundamentals (JUnit 5)

Why We Test, and the Test Pyramid

Tests exist to answer one question cheaply and repeatedly: **does the system still behave correctly after this change?** Without automated tests, that question can only be answered by manual verification (slow, inconsistent, easily skipped) or by production incidents (expensive, embarrassing). A good test suite turns “did I break anything?” into a command that runs in seconds and reports the answer precisely.

The **test pyramid** is a mental model for how to allocate testing effort:



- **Unit tests** exercise a single class or a small cluster of classes in isolation (often with collaborators mocked or faked). They should be the vast majority of your suite: fast (milliseconds), deterministic, and pinpoint the exact failure.
- **Integration tests** verify that your code talks correctly to a real (or realistic) database, message broker, HTTP client, etc. Fewer of these — they are slower and more fragile — but essential because unit tests alone cannot catch a wrong SQL query or a misconfigured serializer.
- **End-to-end tests** drive the whole system as a black box (e.g., through its public API or UI). Very few of these: they are slow, flaky, and expensive to maintain, but they catch wiring problems nothing else can.

A reviewer's rule of thumb: if a PR adds a new branch of logic and the diff has no matching unit test, that is a legitimate request-changes comment. If a PR tries to verify business logic exclusively through a slow end-to-end test, that is also worth challenging — push the assertion down the pyramid.

AAA / Given-When-Then Structure

Every well-written test should have three clearly separated parts, whether or not you label them:

- **Arrange / Given** — set up the object under test and its inputs.
- **Act / When** — perform the one action being tested.
- **Assert / Then** — check the outcome.

```
@Test
void discount_appliesTenPercentForPremiumCustomer() {
    // Arrange (Given)
    Customer premiumCustomer = new Customer("alice",
    ↪ CustomerTier.PREMIUM);
    PricingService pricingService = new PricingService();

    // Act (When)
    BigDecimal finalPrice = pricingService.priceFor(premiumCustomer, new
    ↪ BigDecimal("100.00"));

    // Assert (Then)
    assertEquals(new BigDecimal("90.00"), finalPrice);
}
```

Keep each test to **one logical action** and, ideally, **one reason to fail**. A test with five unrelated `assertEquals` calls checking five unrelated behaviors is a maintenance headache: when it fails, you have to read the whole body to figure out which behavior broke. Splitting it into five small tests (or using `assertAll`, covered below, when the assertions really are about a single outcome) gives you a failure message that already tells you what broke.

Naming Tests So Failures Read Like Sentences

A test name is documentation that never goes stale — if it does, the test breaks and tells you. Favor names that describe **behavior**, not implementation, and that read naturally when a report lists a failing test.

Common conventions:

Style	Example
<code>methodName_condition_expectedResult</code>	<code>withdraw_insufficientFunds_throwsIllegalStateException</code>
Given-When-Then in the name	<code>givenEmptyCart_whenCheckout_thenThrowsEmptyCartException</code>
Sentence-like with backticks (@DisplayName)	<code>"withdraw() throws when balance is insufficient"</code>

```
// Weak: tells you nothing when it's in a failure list
@Test
void test1() { ... }

// Better: reads like a specification
@Test
void withdraw_insufficientFunds_throwsIllegalStateException() { ... }

// Best when paired with @DisplayName for human-readable reports
@Test
@DisplayName("withdraw() throws IllegalStateException when balance is
↳ insufficient")
void withdraw_insufficientFunds_throwsIllegalStateException() { ... }
```

Avoid names like `testWithdraw` or `shouldWork` — they force the reader to open the method body to learn anything, defeating the purpose of a descriptive test suite.

JUnit 5 Architecture: Platform, Jupiter, Vintage

JUnit 5 is not one library but three:

- **JUnit Platform** — the foundation. It discovers and runs tests via the `TestEngine` API, and is what build tools (Maven Surefire, Gradle) and IDEs actually talk to.
- **JUnit Jupiter** — the new programming model and extension API (`@Test`, `@BeforeEach`, `assertEquals`, extensions, etc.). This is “JUnit 5” as most people mean it.
- **JUnit Vintage** — a `TestEngine` that runs old JUnit 3/4 tests on the JUnit Platform, so you can migrate incrementally instead of rewriting everything at once.

```
JUnit Platform (test discovery + execution, IDE/build tool integration)
├── JUnit Jupiter Engine → runs @Test-annotated JUnit 5 tests
└── JUnit Vintage Engine → runs legacy JUnit 3/4 tests
```

This separation is why you can mix legacy `@org.junit.Test` (JUnit 4) tests and modern `@org.junit.jupiter.api.Test` (JUnit 5) tests in the same module during a migration — both engines plug into the same platform.

JUnit 4 → JUnit 5 Annotation Mapping

JUnit 4	JUnit 5 (Jupiter)	Notes
<code>@Test</code>	<code>@Test</code>	JUnit 5’s <code>@Test</code> no longer supports <code>expected=</code> / <code>timeout=</code> attributes
<code>@Before</code>	<code>@BeforeEach</code>	
<code>@After</code>	<code>@AfterEach</code>	
<code>@BeforeClass</code>	<code>@BeforeAll</code>	Must be <code>static</code> unless <code>@TestInstance(PER_CLASS)</code>
<code>@AfterClass</code>	<code>@AfterAll</code>	Must be <code>static</code> unless <code>@TestInstance(PER_CLASS)</code>
<code>@Ignore</code>	<code>@Disabled</code>	
<code>@Category({Foo.class})</code>	<code>@Tag("foo")</code>	String-based, no marker interface needed
<code>@RunWith(...)</code>	<code>@ExtendWith(...)</code>	Composable — you can stack multiple extensions
<code>@Rule / @ClassRule</code>	Extension implementations	More granular lifecycle hooks (see Extensions below)

JUnit 4	JUnit 5 (Jupiter)	Notes
<code>@Test(expected = X. class)</code>	<code>assertThrows(X. class, () -> ...)</code>	Returns the exception so you can assert on its message too
<code>Assume. assumeTrue(...)</code>	<code>Assumptions.assume True(...)</code>	Same idea, new package

Maven and Gradle Setup

Maven (`pom.xml`) — the BOM keeps all Jupiter artifacts on matching versions:

```
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>org.junit</groupId>
      <artifactId>junit-bom</artifactId>
      <version>5.10.2</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>

<dependencies>
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>org.mockito</groupId>
    <artifactId>mockito-junit-jupiter</artifactId>
    <version>5.11.0</version>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>org.assertj</groupId>
    <artifactId>assertj-core</artifactId>
    <version>3.25.3</version>
    <scope>test</scope>
  </dependency>
</dependencies>
```

```

<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-surefire-plugin</artifactId>
      <version>3.2.5</version>
    </plugin>
  </plugins>
</build>

```

Gradle (build.gradle.kts):

```

dependencies {
    testImplementation(platform("org.junit:junit-bom:5.10.2"))
    testImplementation("org.junit.jupiter:junit-jupiter")
    testImplementation("org.mockito:mockito-junit-jupiter:5.11.0")
    testImplementation("org.assertj:assertj-core:3.25.3")
}

tasks.test {
    useJUnitPlatform()
}

```

Forgetting `useJUnitPlatform()` in Gradle (or the Surefire dependency alignment in Maven) is a classic “why are my tests not running at all” bug — the build reports success with zero tests executed, which is worse than a failure because it hides silently.

Core Annotations and Lifecycle

```

import org.junit.jupiter.api.*;
import static org.junit.jupiter.api.Assertions.*;

class OrderCalculatorTest {

    private OrderCalculator calculator;

    @BeforeAll
    static void setUpClass() {
        System.out.println("Runs once before any test in this class");
    }

    @BeforeEach
    void setUp() {
        calculator = new OrderCalculator();
    }
}

```



```

        System.out.println("Runs before every single test method");
    }

    @Test
    @DisplayName("total() sums line item prices")
    void total_sumsLineItemPrices() {
        calculator.addLine("widget", 2, new BigDecimal("5.00"));
        calculator.addLine("gadget", 1, new BigDecimal("12.50"));

        assertEquals(new BigDecimal("22.50"), calculator.total());
    }

    @AfterEach
    void tearDown() {
        System.out.println("Runs after every single test method");
    }

    @AfterAll
    static void tearDownClass() {
        System.out.println("Runs once after all tests in this class");
    }
}

```

By default, JUnit creates a **new instance of the test class for every test method** (`@Test Instance(Lifecycle.PER_METHOD)`), which is why `@BeforeAll`/`@AfterAll` must be static — there is no single instance to hold state across tests. This also means each test starts from a completely fresh object graph, preventing one test's mutated state from leaking into another — a common source of order-dependent flakiness in older frameworks.

You can opt into one shared instance per test class:

```

@TestInstance(TestInstance.Lifecycle.PER_CLASS)
class ExpensiveSetupTest {

    private ExpensiveResource resource;

    @BeforeAll
    void setUp() { // no longer needs to be static
        resource = new ExpensiveResource();
    }

    @Test
    void usesSharedResource() {
        assertNotNull(resource);
    }
}

```

```
}
```

Use `PER_CLASS` sparingly — it is convenient for genuinely expensive, read-only setup, but it reintroduces the risk of tests sharing mutable state, the exact problem `PER_METHOD` was designed to avoid. If you use it, make sure the shared object is either immutable or reset in `@BeforeEach`.

Assertions

Jupiter's built-in `Assertions` class covers the common cases:

```
import static org.junit.jupiter.api.Assertions.*;

@Test
void assertionShowcase() {
    assertEquals(4, 2 + 2);
    assertTrue(List.of(1, 2, 3).contains(2));
    assertFalse(List.of(1, 2, 3).isEmpty());
    assertNull(findUser("missing"));
    assertNotNull(findUser("alice"));

    assertIterableEquals(List.of(1, 2, 3), computeSequence());

    IllegalArgumentException ex = assertThrows(
        IllegalArgumentException.class,
        () -> new Account(new BigDecimal("-1"))
    );
    assertEquals("initial balance must not be negative", ex.getMessage());

    assertTimeout(Duration.ofMillis(200), () ->
        ↪ slowButBoundedOperation());

    // Groups related checks: ALL run even if one fails, and all failures
    ↪ are reported together
    Order order = buildOrder();
    assertAll("order invariants",
        () -> assertEquals("ORD-1", order.id()),
        () -> assertEquals(OrderStatus.PENDING, order.status()),
        () -> assertFalse(order.lineItems().isEmpty())
    );
}
```

`assertAll` is the key one reviewers look for: without it, three separate `assertEquals` calls stop at the first failure, hiding whether the second and third would also have failed.

`assertAll` runs every supplied executable and reports every failure together — much more useful when triaging a broken test.

`assertTimeout` runs the code on the *same* thread and waits for it to finish (so a hang will still block the test thread); `assertTimeoutPreemptively` runs it on a separate thread and aborts at the deadline, but can leave background work running and has subtle interactions with `ThreadLocals` — prefer `assertTimeout` unless you specifically need preemptive cancellation.

AssertJ as a Fluent Alternative

JUnit's static assertions are fine, but they read backwards (`assertEquals(expected, actual)`) and don't chain well. **AssertJ** provides a fluent, chainable, and far more readable API, and is the de-facto standard paired with JUnit 5 in most professional code-bases:

```
import static org.assertj.core.api.Assertions.assertThat;

@Test
void total_sumsLineItemPrices_assertj() {
    calculator.addLine("widget", 2, new BigDecimal("5.00"));
    calculator.addLine("gadget", 1, new BigDecimal("12.50"));

    assertThat(calculator.total()).isEqualToComparingTo("22.50");

    assertThat(calculator.lineItems())
        .hasSize(2)
        .extracting("name")
        .containsExactly("widget", "gadget");

    assertThatThrownBy(() -> calculator.addLine("bad", -1,
    ↪ BigDecimal.TEN))
        .isInstanceOf(IllegalArgumentException.class)
        .hasMessageContaining("quantity");
}
```

AssertJ's chained assertions produce much richer failure messages out of the box (e.g., showing exactly which elements differ in a collection), and `assertThatThrownBy` combined with `.hasMessageContaining(...)` is more expressive than `assertThrows` + a separate `assertEquals` on the message.

Nested Tests

@Nested groups related tests inside inner classes, which is useful for expressing “given this context, these things should be true” structures and produces much clearer test reports:

```
import org.junit.jupiter.api.*;
import static org.junit.jupiter.api.Assertions.*;

class BankAccountTest {

    private BankAccount account;

    @BeforeEach
    void setUp() {
        account = new BankAccount("acc-1", new BigDecimal("100.00"));
    }

    @Nested
    @DisplayName("when the account has a positive balance")
    class WithPositiveBalance {

        @Test
        @DisplayName("withdraw() succeeds for amounts within balance")
        void withdraw_withinBalance_succeeds() {
            account.withdraw(new BigDecimal("40.00"));
            assertEquals(new BigDecimal("60.00"), account.balance());
        }

        @Test
        @DisplayName("withdraw() throws for amounts above balance")
        void withdraw_aboveBalance_throws() {
            assertThrows(InsufficientFundsException.class,
                () -> account.withdraw(new BigDecimal("500.00")));
        }
    }

    @Nested
    @DisplayName("when the account is frozen")
    class WhenFrozen {

        @BeforeEach
        void freeze() {
            account.freeze();
        }

        @Test
        @DisplayName("withdraw() throws regardless of balance")
```

```

        void withdraw_whenFrozen_throws() {
            assertThrows(AccountFrozenException.class,
                () -> account.withdraw(new BigDecimal("1.00")));
        }
    }
}

```

Each `@Nested` class gets its own `@BeforeEach` chain (outer then inner), so `WhenFrozen.freeze()` runs after the outer `setUp()`, letting you build up scenario-specific state incrementally.

Tags and Filtering

`@Tag` lets you categorize tests (e.g., `slow`, `integration`, `smoke`) and then include or exclude categories at build time without touching test code:

```

@Tag("integration")
class OrderRepositoryIntegrationTest {

    @Test
    @Tag("slow")
    void savesAndReloadsOrder() {
        // hits a real (test) database
    }
}

```

Maven Surefire:

```

<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-surefire-plugin</artifactId>
  <configuration>
    <excludedGroups>slow, integration</excludedGroups>
  </configuration>
</plugin>

```

Gradle:

```

tasks.test {
    useJUnitPlatform {
        excludeTags("slow", "integration")
    }
}

```

This is how most CI pipelines run a fast “unit only” stage on every push and reserve a slower “integration” stage for merge queues or nightly runs.

Disabling Tests

```
@Test
@Disabled("Flaky due to JDK-XXXX clock resolution bug on CI; see
↪ TICKET-1234")
void timeSensitiveTest() { ... }
```

A reviewer should always ask **why** a test is disabled and whether there is a tracked ticket — `@Disabled` with no reason or follow-up is a silent hole in coverage that tends to be forgotten forever.

Assumptions

`Assumptions.assertTrue(...)` aborts (not fails) a test when a precondition isn’t met — useful for environment-dependent tests you don’t want counted as failures:

```
import static org.junit.jupiter.api.Assumptions.*;

@Test
void nativeLibraryTest() {
    ↪ assertTrue(System.getProperty("os.name").toLowerCase().contains("linux"),
        "This test only runs on Linux");

    // ... test that depends on a Linux-only native library
}
```

If the assumption fails, JUnit marks the test as **skipped**, not failed — it will not turn a CI build red, but it also will not silently pass, so it still shows up in the report.

Parameterized Tests

Parameterized tests let you run the same test logic against many inputs without copy-pasting the method.

```

import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.*;
import static org.junit.jupiter.api.Assertions.*;

class PasswordValidatorTest {

    private final PasswordValidator validator = new PasswordValidator();

    @ParameterizedTest
    @ValueSource(strings = { "abc", "12345", "      ", "" })
    void isValid_rejectsShortOrBlankPasswords(String candidate) {
        assertFalse(validator.isValid(candidate));
    }

    @ParameterizedTest
    @CsvSource({
        "Password1!, true",
        "password1!, false",    // no uppercase
        "PASSWORD1!, false",    // no lowercase
        "Password!, false",     // no digit
        "Password1, false"      // no symbol
    })
    void isValid_enforcesComplexityRules(String candidate, boolean
    ⇨ expected) {
        assertEquals(expected, validator.isValid(candidate));
    }

    @ParameterizedTest
    @MethodSource("complexPasswordCases")
    void isValid_matchesMethodSourceCases(String candidate, boolean
    ⇨ expected) {
        assertEquals(expected, validator.isValid(candidate));
    }

    static Stream<Arguments> complexPasswordCases() {
        return Stream.of(
            Arguments.of("Correct1!Horse", true),
            Arguments.of("nope", false),
            Arguments.of("Tr0ub4dor&3", true)
        );
    }

    @ParameterizedTest
    @EnumSource(value = PasswordStrength.class, names = "WEAK", mode =
    ⇨ EnumSource.Mode.EXCLUDE)
    void isValid_acceptsNonWeakStrengths(PasswordStrength strength) {
        assertTrue(validator.isValid(strength.samplePassword()));
    }
}

```

```

    @ParameterizedTest
    @ArgumentsSource(SqlInjectionPayloadsProvider.class)
    void isValid_rejectsSqlInjectionLikePayloads(String payload) {
        assertFalse(validator.isValid(payload));
    }

    static class SqlInjectionPayloadsProvider implements
↪ ArgumentsProvider {
        @Override
        public Stream<? extends Arguments>
↪ provideArguments(ExtensionContext context) {
            return Stream.of("' OR '1'='1", "; DROP TABLE users; --")
                .map(Arguments::of);
        }
    }
}
```

Source	Best for
@ValueSource	A flat list of one primitive/String argument each
@CsvSource	A handful of inline multi-argument rows
@MethodSource	Complex objects, or cases shared across multiple test methods
@EnumSource	Iterating over some or all values of an enum
@ArgumentsSource	A reusable, custom provider you want to share across test classes

Parameterized tests are one of the highest-leverage tools in the whole framework: a reviewer seeing five nearly identical `@Test` methods that differ only in input/output values should almost always suggest collapsing them into one `@ParameterizedTest`.

Repeated Tests

`@RepeatedTest` runs the same test multiple times — useful for surfacing flakiness in something that has an element of non-determinism (e.g., relies on system time, randomness, or concurrency) during investigation, though it is not a substitute for fixing the non-determinism.


```

@RepeatedTest(10)
void idGenerator_neverProducesDuplicateIds(RepetitionInfo info) {
    String id = IdGenerator.next();
    assertTrue(seenIds.add(id), "Duplicate on repetition " +
    ↪ info.getCurrentRepetition());
}

```

Dynamic Tests with @TestFactory

Regular @Test methods are fixed at compile time. @TestFactory lets you *generate* tests at runtime, returning a Stream/Collection/Iterable of DynamicTest:

```

import org.junit.jupiter.api.DynamicTest;
import org.junit.jupiter.api.TestFactory;
import static org.junit.jupiter.api.DynamicTest.dynamicTest;

class FizzBuzzDynamicTest {

    @TestFactory
    Stream<DynamicTest> fizzBuzzCases() {
        Map<Integer, String> expectations = Map.of(
            1, "1", 3, "Fizz", 5, "Buzz", 15, "FizzBuzz"
        );

        return expectations.entrySet().stream()
            .map(entry -> dynamicTest(
                "fizzBuzz(" + entry.getKey() + ") == " + entry.getValue(),
                () -> assertEquals(entry.getValue(),
    ↪ FizzBuzz.of(entry.getKey()))
            ));
    }
}

```

Dynamic tests are less common than parameterized tests in day-to-day code but show up when test cases themselves need to be computed (e.g., generated from a spec file or a database of fixtures).

Extensions

Jupiter replaces JUnit 4's @Rule/@RunWith with a single, composable Extension model, activated with @ExtendWith.

```

class LoggingTest {
    // Third-party or shared extensions plug in the same way Mockito's
    ↪ extension does
}

```

A custom extension that times every test and logs slow ones:

```

import org.junit.jupiter.api.extension.*;

public class TimingExtension implements BeforeTestExecutionCallback,
    ↪ AfterTestExecutionCallback {

    private static final ExtensionContext.Namespace NAMESPACE =
        ExtensionContext.Namespace.create(TimingExtension.class);

    @Override
    public void beforeTestExecution(ExtensionContext context) {
        getStore(context).put("start", System.nanoTime());
    }

    @Override
    public void afterTestExecution(ExtensionContext context) {
        long start = getStore(context).remove("start", long.class);
        long durationMs = (System.nanoTime() - start) / 1_000_000;
        if (durationMs > 100) {
            System.out.printf("SLOW TEST [%s]: %d ms%n",
    ↪ context.getDisplayName(), durationMs);
        }
    }

    private ExtensionContext.Store getStore(ExtensionContext context) {
        return context.getStore(NAMESPACE);
    }
}

@ExtendWith(TimingExtension.class)
class ReportGeneratorTest {

    @Test
    void generatesMonthlyReport() {
        // ...
    }
}

```

@TempDir is a built-in extension for tests that need a real filesystem directory that is automatically created before the test and deleted after:

```

import org.junit.jupiter.api.io.TempDir;
import java.nio.file.*;

class CsvExporterTest {

    @Test
    void export_writesCsvFileWithHeader(@TempDir Path tempDir) throws
    ↳ IOException {
        Path outputFile = tempDir.resolve("export.csv");

        new CsvExporter().export(List.of(new Row("a", 1)), outputFile);

        List<String> lines = Files.readAllLines(outputFile);
        assertEquals("name,value", lines.get(0));
        assertEquals("a,1", lines.get(1));
    }
}

```

@TempDir removes the classic “tests that write to /tmp and never clean up, eventually filling the CI disk” problem entirely.

Testing Exceptions

Always assert on both the exception **type** and enough of the **message/state** to know the right branch was hit — asserting only the type can pass even if the wrong condition threw it.

```

@Test
void transfer_negativeAmount_throwsWithDescriptiveMessage() {
    Account source = new Account("A", new BigDecimal("100"));
    Account target = new Account("B", new BigDecimal("0"));

    IllegalArgumentException ex =
    ↳ assertThrows(IllegalArgumentException.class,
        () -> transferService.transfer(source, target, new
    ↳ BigDecimal("-10")));

    assertEquals("transfer amount must be positive: -10",
    ↳ ex.getMessage());
}

```

Testing Time with an Injected Clock

Code that calls `Instant.now()`, `LocalDate.now()`, or `System.currentTimeMillis()` directly is untestable without sleeping or mocking static methods. The fix is to depend on `java.time.Clock` and inject it, so tests can supply a fixed clock.

```
public class SubscriptionService {

    private final Clock clock;

    public SubscriptionService(Clock clock) {
        this.clock = clock;
    }

    public boolean isExpired(Subscription subscription) {
        return subscription.expiresAt().isBefore(LocalDate.now(clock));
    }
}
```

```
class SubscriptionServiceTest {

    @Test
    void isExpired_returnsTrue_whenExpiryIsInThePast() {
        Clock fixedClock = Clock.fixed(
            Instant.parse("2026-08-07T00:00:00Z"), ZoneOffset.UTC);
        SubscriptionService service = new SubscriptionService(fixedClock);

        Subscription expired = new
↳ Subscription(LocalDate.parse("2026-08-01"));

        assertTrue(service.isExpired(expired));
    }

    @Test
    void isExpired_returnsFalse_whenExpiryIsInTheFuture() {
        Clock fixedClock = Clock.fixed(
            Instant.parse("2026-08-07T00:00:00Z"), ZoneOffset.UTC);
        SubscriptionService service = new SubscriptionService(fixedClock);

        Subscription active = new
↳ Subscription(LocalDate.parse("2026-12-31"));

        assertFalse(service.isExpired(active));
    }
}
```

This one pattern eliminates an entire category of flaky, date-dependent tests (the ones that mysteriously fail once a year around New Year's Eve, or that pass locally but fail on CI in a different timezone).

Testing Concurrency — and Why `Thread.sleep` Is a Smell

A test that does this is a ticking time bomb:

```
// Bad: racy and slow
@Test
void taskRunsAsynchronously() throws InterruptedException {
    AtomicBoolean ran = new AtomicBoolean(false);
    executor.submit(() -> ran.set(true));

    Thread.sleep(500); // hope 500ms is enough... until CI is under load

    assertTrue(ran.get());
}
```

`Thread.sleep` in a concurrency test encodes a guess about timing. It is either wasteful (sleeping far longer than necessary, slowing down the whole suite) or unreliable (too short under CI load, causing intermittent failures — flakiness). The fix is to wait for the actual event, not for a fixed amount of wall-clock time.

CountDownLatch — signal exactly when the work is done:

```
@Test
void taskRunsAsynchronously_withLatch() throws InterruptedException {
    CountDownLatch latch = new CountDownLatch(1);
    AtomicBoolean ran = new AtomicBoolean(false);

    executor.submit(() -> {
        ran.set(true);
        latch.countDown();
    });

    boolean completedInTime = latch.await(2, TimeUnit.SECONDS);

    assertTrue(completedInTime, "task did not complete within timeout");
    assertTrue(ran.get());
}
```

Awaitility — for polling a condition that eventually becomes true (e.g., an async cache being populated), which reads far better than a hand-rolled polling loop:

```

import static org.awaitility.Awaitility.await;
import java.time.Duration;

@Test
void cache_eventuallyContainsWarmedEntries() {
    cacheWarmer.warmAsync();

    await().atMost(Duration.ofSeconds(3))
        .pollInterval(Duration.ofMillis(50))
        .until(() -> cache.get("key") != null);

    assertEquals("value", cache.get("key"));
}

```

Both approaches wait for the *minimum* necessary time and fail fast with a clear timeout message, instead of guessing a sleep duration that is either too short (flaky) or too long (slow suite).

Test Doubles Vocabulary

“Mock” is used loosely in everyday speech, but in precise testing vocabulary there are five distinct kinds of test doubles:

Term	What it does	Verifies behavior?	Typical use
Dummy	A placeholder passed to satisfy a parameter list; never actually used	No	<code>new Service(null, dummyLogger)</code> when <code>dummyLogger</code> is never called
Stub	Returns canned answers to calls made during the test	No	<code>when(repo.findById(1)).thenReturn(user)</code>
Fake	A lightweight <i>working</i> implementation, not production-grade (e.g., in-memory DB)	Indirectly, through real behavior	<code>InMemoryUserRepository</code> implements <code>UserRepository</code>
Spy	A real object with some methods overridden/recorded, or a mock that records calls on a real instance	Yes (records calls, can also stub)	<code>Mockito.spy(realService)</code>
Mock	A dynamically generated object programmed with expectations, used to verify interactions	Yes (that's its purpose)	<code>verify(emailSender).send(any())</code>

The line between stub and mock is about **intent**: a stub just feeds data into the test (state verification — you check the *result*), while a mock is used to verify that specific interactions happened (behavior verification — you check the *calls*). Mockito's `mock()` can be used as either, depending on whether you call `verify(...)` on it.

Code Coverage — and Why 100% Is Not the Goal

Coverage tools (JaCoCo, for example) report the percentage of lines/branches executed by the test suite. Coverage is a **necessary but not sufficient** signal:

- Low coverage on business-critical code is a real red flag — it means there is a good chance that logic has never been exercised by a test.
- High coverage tells you code *ran*, not that it was *checked*. A test with no assertions can hit 100% of a method's lines and prove nothing.

```
// Executes every line (100% "covered") but asserts nothing meaningful
@Test
void processOrder_doesNotThrow() {
    orderService.process(order); // "coverage" achieved, but no behavior
    ↪ verified
}
```

Chasing 100% coverage as a target also incentivizes testing trivial code (getters, `toString()`, generated code) at the expense of time that should go to edge cases and error paths. A better review question than “what’s the coverage number?” is “does this test actually assert the behavior that matters, including the failure paths?”

Flaky Tests

A **flaky test** passes and fails intermittently with no code change — the single most corrosive thing to trust in a test suite, because a team that has learned to click “rerun” on red builds has effectively stopped trusting CI.

Common causes:

- **Time dependence** — using `LocalDate.now()/Instant.now()` directly instead of an injected `Clock` (see above).
- **Shared mutable state** — static fields, singletons, or a shared database row mutated by tests running in parallel or in a different order than expected.

- **Real concurrency/timing assumptions** — `Thread.sleep` races, or asserting order between two async operations without synchronization.
- **External dependencies** — hitting a real network service, real clock-based expiry, or an unseeded random number generator.
- **Test order dependence** — a test that only passes because an earlier test happened to leave the system in the right state; JUnit does not guarantee method execution order, so this can flip at any time.
- **Resource leaks across tests** — a thread pool, file handle, or connection left open by one test that starves the next one.

The fix is almost always to remove the non-determinism (inject a clock, replace `sleep` with a latch, isolate state per test, seed random generators) rather than to add retries — retries hide the bug, they don't fix it.

A Full Realistic Test Class

Putting several of the above together for a small service that computes shipping fees:

```
// Production code
public class ShippingFeeCalculator {

    private static final BigDecimal FREE_SHIPPING_THRESHOLD = new
↳ BigDecimal("50.00");
    private static final BigDecimal STANDARD_FEE = new BigDecimal("4.99");

    public BigDecimal feeFor(BigDecimal orderTotal, ShippingSpeed speed) {
        if (orderTotal == null) {
            throw new IllegalArgumentException("orderTotal must not be
↳ null");
        }
        if (orderTotal.compareTo(BigDecimal.ZERO) < 0) {
            throw new IllegalArgumentException("orderTotal must not be
↳ negative: " + orderTotal);
        }

        if (orderTotal.compareTo(FREE_SHIPPING_THRESHOLD) >= 0 && speed
↳ == ShippingSpeed.STANDARD) {
            return BigDecimal.ZERO;
        }

        return switch (speed) {
            case STANDARD -> STANDARD_FEE;
            case EXPRESS -> STANDARD_FEE.multiply(new BigDecimal("2"));
        }
    }
}
```



```

        case OVERNIGHT -> STANDARD_FEE.multiply(new BigDecimal("4"));
    };
}
}

```

```

import org.junit.jupiter.api.*;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvSource;
import java.math.BigDecimal;

import static org.junit.jupiter.api.Assertions.*;

@DisplayName("ShippingFeeCalculator")
class ShippingFeeCalculatorTest {

    private final ShippingFeeCalculator calculator = new
↳ ShippingFeeCalculator();

    @Nested
    @DisplayName("input validation")
    class InputValidation {

        @Test
        @DisplayName("throws when orderTotal is null")
        void feeFor_nullOrderTotal_throws() {
            IllegalArgumentException ex =
↳ assertThrows(IllegalArgumentException.class,
                () -> calculator.feeFor(null, ShippingSpeed.STANDARD));
            assertEquals("orderTotal must not be null", ex.getMessage());
        }

        @Test
        @DisplayName("throws when orderTotal is negative")
        void feeFor_negativeOrderTotal_throws() {
            assertThrows(IllegalArgumentException.class,
                () -> calculator.feeFor(new BigDecimal("-1.00"),
↳ ShippingSpeed.STANDARD));
        }
    }

    @Nested
    @DisplayName("standard shipping")
    class StandardShipping {

        @Test
        @DisplayName("is free at or above the free-shipping threshold")
        void feeFor_atThreshold_isFree() {

```

```

        BigDecimal fee = calculator.feeFor(new BigDecimal("50.00"),
↪ ShippingSpeed.STANDARD);
        assertEquals(BigDecimal.ZERO, fee);
    }

    @Test
    @DisplayName("charges the standard fee below the threshold")
    void feeFor_belowThreshold_chargesStandardFee() {
        BigDecimal fee = calculator.feeFor(new BigDecimal("49.99"),
↪ ShippingSpeed.STANDARD);
        assertEquals(new BigDecimal("4.99"), fee);
    }

    @ParameterizedTest(name = "{0} order with {1} shipping costs {2}")
    @CsvSource({
        "10.00, EXPRESS, 9.98",
        "10.00, OVERNIGHT, 19.96",
        "60.00, EXPRESS, 9.98", // free-shipping discount only applies
↪ to STANDARD
        "60.00, OVERNIGHT, 19.96"
    })
    @DisplayName("express and overnight fees ignore the free-shipping
↪ threshold")
    void feeFor_expressAndOvernight_neverFree(BigDecimal total,
↪ ShippingSpeed speed, BigDecimal expectedFee) {
        assertEquals(expectedFee, calculator.feeFor(total, speed));
    }
}

```

This class demonstrates: @Nested grouping by scenario, @DisplayName for readable reports, a parameterized test for the combinatorial cases, and assertions that check both the *type* and the *content* of thrown exceptions.

Mocking (Mockito)

What to Mock and What NOT to Mock

Mocking is a tool for isolating the class under test from its **collaborators with real-world side effects**: databases, HTTP clients, message queues, clocks, random number generators, email senders. It is not a tool for isolating a class from everything it touches.

Do mock:

- Repositories/DAOs (avoid hitting a real database in a unit test)

- External service clients (payment gateway, email provider, third-party API)
- Anything with side effects you don't want during a test (file system, network)

Do NOT mock:

- **Value objects / DTOs** (Money, Address, records, simple POJOs). These have no behavior worth faking — just construct a real instance. Mocking a value object produces an object whose `equals()`/`getters` are unpredictable stubs instead of real values, which actively makes the test harder to trust.

```
// Bad: mocking a plain value object
Money mockMoney = mock(Money.class);
when(mockMoney.amount()).thenReturn(new BigDecimal("10.00"));

// Good: it's just data – construct it
Money money = new Money(new BigDecimal("10.00"), "USD");
```

- **Types you don't own** (JDK classes, third-party library classes like `HttpClient`, `Connection`, `File`). Mocking framework internals couples your tests to implementation details of a library that can change between versions, and often the type isn't even designed to be mocked (`final` classes/methods, complex internal state). Instead, **wrap it** in a small interface you own, and mock *that* interface.

```
// Bad: mocking java.time.Clock's cousin, a third-party HttpClient
↳ directly everywhere
HttpClient mockClient = mock(HttpClient.class);
when(mockClient.send(any(), any())).thenReturn(fakeResponse());

// Good: define a narrow seam you control
public interface WeatherClient {
    WeatherReport fetch(String city);
}
// production implementation wraps the real HttpClient/SDK
// test code mocks WeatherClient, a one-method interface you own
WeatherClient mockClient = mock(WeatherClient.class);
when(mockClient.fetch("Berlin")).thenReturn(new WeatherReport(18,
↳ "cloudy"));
```

- **The class under test itself.** If you find yourself mocking half the methods on the very class you're testing, that's a sign the class needs to be split, not mocked around.

Creating Mocks: `mock()`, `@Mock`, `@InjectMocks`, `@Spy`

The programmatic form:

```
UserRepository userRepository = mock(UserRepository.class);
```

The annotation form, wired up by `MockitoExtension` (Mockito's JUnit 5 integration):

```
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.Spy;
import org.mockito.junit.jupiter.MockitoExtension;

import static org.mockito.Mockito.*;
import static org.junit.jupiter.api.Assertions.*;

@ExtendWith(MockitoExtension.class)
class UserRegistrationServiceTest {

    @Mock
    private UserRepository userRepository;

    @Mock
    private EmailSender emailSender;

    @Spy
    private AuditLog auditLog = new AuditLog(); // real object,
    ↪ selectively overridable

    @InjectMocks
    private UserRegistrationService registrationService; // Mockito
    ↪ injects the mocks above via constructor

    @Test
    void register_newEmail_savesUserAndSendsWelcomeEmail() {
    ↪ when(userRepository.existsByEmail("alice@example.com")).thenReturn(false);

        registrationService.register("alice@example.com", "Alice");

        verify(userRepository).save(any(User.class));
        verify(emailSender).sendWelcomeEmail("alice@example.com");
    }
}
```

`@InjectMocks` tries constructor injection first (preferred — it fails loudly at compile time if the constructor shape changes), then setter injection, then field injection. If `UserRegistrationService` has a single constructor taking `UserRepository`, `EmailSender`, and `AuditLog`, Mockito matches the `@Mock/@Spy` fields to those parameters by type.

`@Spy` wraps a **real** object: calls go to the real implementation unless you explicitly stub them, and Mockito still records every call for `verify(...)`. Use a spy when you want most of the real behavior but need to intercept one troublesome method (e.g., a legacy class that does real I/O in one method you want to fake out for the test).

Stubbing

`when().thenReturn(...)` is the everyday form for stubbing non-void methods on a mock:

```
when(userRepository.findById(1L)).thenReturn(Optional.of(new User(1L,
    ↪ "Alice")));
```

`thenThrow` stubs an exception:

```
when(paymentGateway.charge(any())).thenThrow(new
    ↪ PaymentDeclinedException("insufficient funds"));
```

`thenAnswer` for computed/dynamic responses based on the actual arguments:

```
when(userRepository.save(any(User.class))).thenAnswer(invocation -> {
    User user = invocation.getArgument(0);
    return user.withId(42L); // simulate the DB assigning an id
});
```

Consecutive returns — different answers on successive calls, useful for simulating retry logic:

```
when(flakyClient.fetch()
    .thenThrow(new IOException("timeout"))
    .thenReturn("success")); // first call throws, second call (and beyond)
    ↪ returns "success"
```

doReturn/doThrow/doNothing().when(...) — and When You Must Use It

The `when(mock.method()).thenReturn(...)` form has a hard limitation: it actually *calls* the real method on the mock first to figure out what to stub. That is invisible for a plain mock (Mockito intercepts the call before any real code runs), but it breaks down in two situations:

1. **Spies** — because a spy *does* run real code, `when(spy.riskyMethod()).thenReturn(...)` would execute the real (possibly side-effecting or exception-throwing) method before Mockito even gets to stub it.
2. **Void methods** — `when(...)` needs a return value to chain off of; a void method call has none, so there is nothing to pass to `when(...)`.

The `do...().when(mock)` family avoids this by stubbing without invoking the real method first:

```
@Spy
private OrderValidator validator = new OrderValidator(); // real object;
↳ validate() does real, expensive checks

@Test
void skipsExpensiveValidationInThisTest() {
    doReturn(true).when(validator).isPreValidated(any()); // safe on a
↳ spy; when(validator...) would run the real check first

    // ...
}

@Mock
private NotificationService notificationService;

@Test
void notify_whenServiceUnavailable_doesNothingInsteadOfThrowing() {
    doThrow(new ServiceUnavailableException()).when(notificationService).
↳ notifyUser(any()); // void method

    orderService.completeOrder(order); // should swallow the notification
↳ failure gracefully

    verify(notificationService).notifyUser(order.customerId());
}

@Test
void suppressesAuditLoggingDuringThisScenario() {
```

```
doNothing().when(auditLog).record(any()); // void method, explicit
↳ no-op
    // ...
}
```

Rule of thumb for reviewers: `when(...).thenReturn(...)` for normal mocks and non-void methods; `doX().when(...)` whenever the target is a **spy** or the method is **void**.

Argument Matchers

Matchers let you stub/verify based on argument *shape* rather than an exact instance:

```
when(userRepository.findByEmail(anyString())).thenReturn(Optional.empty())
↳ );
when(userRepository.findById(eq(1L))).thenReturn(Optional.of(alice));
when(pricingEngine.priceFor(argThat(order -> order.items().size() > 5)))
    .thenReturn(new BigDecimal("0.00")); // free for large orders,
↳ expressed as a custom predicate
```

The critical rule: you cannot mix raw values and matchers in the same call. If *any* argument uses a matcher, *every* argument in that call must use a matcher (use `eq(...)` to wrap literal values).

```
// Throws InvalidUseOfMatchersException at runtime
when(userService.updateEmail(1L, anyString())).thenReturn(true);

// Correct: wrap the literal in eq(...) once any() is used elsewhere
when(userService.updateEmail(eq(1L), anyString())).thenReturn(true);
```

This is a very common copy-paste mistake in review — a matcher gets added to one argument while a neighboring literal is left bare, and the test blows up with a confusing runtime exception instead of a compile error.

Verification

Stubbing controls what a mock *returns*; verification checks what was *called*.

```
verify(emailSender).sendWelcomeEmail("alice@example.com"); // called
↳ exactly once, with this argument
```

```

    verify(emailSender, times(2)).sendReminder(any());           // called
    ↪ exactly twice
    verify(emailSender, never()).sendWelcomeEmail("blocked@example.com");
    verify(emailSender, atLeast(1)).flush();
    verify(emailSender, atMost(3)).retry();

    InOrder inOrder = inOrder(userRepository, emailSender);
    inOrder.verify(userRepository).save(any(User.class));
    inOrder.verify(emailSender).sendWelcomeEmail(anyString()); // save()
    ↪ must happen before the email is sent

    verify(userRepository).save(any(User.class));
    verify(emailSender).sendWelcomeEmail(anyString());
    verifyNoMoreInteractions(userRepository, emailSender);    // fails
    ↪ if any OTHER call happened on these mocks

```

`verifyNoMoreInteractions` is a strong assertion — it fails the test if the code under test called anything on that mock beyond what you’ve already verified. It’s a good way to lock down that a method does *exactly* what you expect and nothing extra, but overusing it on every mock in every test makes tests brittle against harmless refactors (e.g., adding a debug-log call). Use it selectively, on the interactions that genuinely matter.

ArgumentCaptor

When you need to inspect the *value* an argument had, not just that a call happened, capture it:

```

@Captor
private ArgumentCaptor<User> userCaptor;

@Test
void register_savesUserWithNormalizedEmail() {
    registrationService.register(" Alice@example.com ", "Alice");

    verify(userRepository).save(userCaptor.capture());
    User savedUser = userCaptor.getValue();

    assertEquals("alice@example.com", savedUser.email()); // trimmed and
    ↪ lower-cased
}

```

`ArgumentCaptor` is essential when a mock’s method returns `void` (or nothing you need) but you still want to assert on exactly what was passed in — verification alone tells you a call happened; the captor tells you what it looked like.

RETURNS_DEEP_STUBS and Why It's a Smell

Mockito can auto-generate stubs for chained calls:

```
// Works, but...
OrderContext context = mock(OrderContext.class,
    ↳ Mockito.RETURNS_DEEP_STUBS);
when(context.getCustomer().getAddress().getCountry()).thenReturn("DE");
```

This “solves” a symptom while hiding the real problem: a chain like `context.getCustomer().getAddress().getCountry()` violates the **Law of Demeter** — the code under test is reaching three objects deep into someone else’s internals. `RETURNS_DEEP_STUBS` makes that chain *mockable*, but it does not make it *good design*. The test passing this way is a strong signal that the production code should instead depend directly on a `Country` (passed in, or obtained through a single well-named method like `context.getCustomerCountry()`), which would need no deep stub at all.

Strict Stubs and UnnecessaryStubbingException

`MockitoExtension` runs in **strict stubbing** mode by default: if you stub something that the test never actually calls, the test fails with `UnnecessaryStubbingException` (or, depending on version/settings, a warning that fails on Mockito. `validateMockitoUsage()`), rather than silently passing.

```
@ExtendWith(MockitoExtension.class)
class DiscountServiceTest {

    @Mock
    private CouponRepository couponRepository;

    @Test
    void appliesDiscount() {

        ↳ when(couponRepository.findByCode("SAVE10")).thenReturn(Optional.of(new
        ↳ Coupon(10)));
        // If discountService never actually calls
        ↳ couponRepository.findByCode("SAVE10")
        // (e.g., because the test forgot to pass a coupon code), this
        ↳ line is flagged
        // as an UnnecessaryStubbingException – a leftover stub from a
        ↳ copy-pasted test.
        ...
    }
}
```

```
}
```

This is a deliberately helpful failure: unused stubs accumulate in copy-pasted tests and quietly rot, giving a false sense of what's actually being exercised. Treat `UnnecessaryStubbingException` as a signal to delete the stub or fix the test, never to silence it with `lenient()` unless there's a genuinely good reason (e.g., a shared `@BeforeEach` stub that only some tests in the class need).

```
lenient().when(couponRepository.findByCode(any())).thenReturn(Optional.  
↳ empty()); // opt out of strict checking, deliberately
```

Mocking Static Methods

Mockito 3.4+ supports mocking static methods via `mockStatic`, using inline mocking:

```
try (MockedStatic<Instant> mockedInstant =  
↳ Mockito.mockStatic(Instant.class)) {  
    mockedInstant.when(Instant::now).thenReturn(Instant.parse("2026-08-07T00:00:00Z"));  
↳ 2026-08-07T00:00:00Z");  
  
    Report report = reportGenerator.generateDailyReport();  
  
    assertEquals(LocalDate.parse("2026-08-07"), report.date());  
}
```

This works, and the try-with-resources scoping (the static mock is only active inside the block) is important to prevent it from leaking into other tests. But **needing this at all is usually a design smell**: it means the production code called a static method directly (`Instant.now()`, a static utility, a singleton accessor) instead of depending on an injectable abstraction. The preferred fix, shown earlier in this chapter, is to inject a `Clock` (or wrap the static call behind an interface you own) — `mockStatic` should be a last resort for code you cannot easily refactor (e.g., a legacy static utility from a third-party library), not a first choice for new code.

MockedConstruction

Similarly, `mockConstruction` intercepts `new SomeType(...)` calls made *inside* the code under test, redirecting them to a mock — useful when legacy code directly instantiates a collaborator instead of receiving it via dependency injection:

```

    try (MockedConstruction<PdfRenderer> mocked =
↳ Mockito.mockConstruction(PdfRenderer.class,
        (mockRenderer, context) ->
↳ when(mockRenderer.render(any())) .thenReturn(new byte[] { 1, 2, 3 }))) {

        byte[] pdf = invoiceService.generatePdf(invoice); // internally does
↳ `new PdfRenderer()`

        assertEquals(new byte[] { 1, 2, 3 }, pdf);
    }

```

Like `mockStatic`, treat this as a stopgap while working with un-refactorable legacy code, not a pattern to reach for in new code — the sustainable fix is to inject `PdfRenderer` (or a factory for it) so tests can simply pass in a mock or fake.

BDDMockito

BDDMockito provides `given/willReturn/then` aliases that read more naturally in Given-When-Then style tests — purely stylistic, functionally identical to the plain Mockito API:

```

import static org.mockito.BDDMockito.*;

@Test
void register_newEmail_savesUserAndSendsWelcomeEmail_bdd() {
    // Given
    given(userRepository.existsByEmail("alice@example.com")).willReturn(
↳ false);

    // When
    registrationService.register("alice@example.com", "Alice");

    // Then
    then(userRepository).should().save(any(User.class));
    then(emailSender).should().sendWelcomeEmail("alice@example.com");
}

```

Teams pick one style (plain Mockito or BDDMockito) and stay consistent — mixing `when(...)` and `given(...)` in the same codebase is a minor but real readability nit reviewers flag.

A Full Realistic Example: Service with a Repository and a Clock

```
// Production code
public class LoanEligibilityService {

    private final LoanRepository loanRepository;
    private final Clock clock;

    public LoanEligibilityService(LoanRepository loanRepository, Clock
↪ clock) {
        this.loanRepository = loanRepository;
        this.clock = clock;
    }

    public EligibilityResult checkEligibility(String customerId) {
        List<Loan> activeLoans =
↪ loanRepository.findActiveLoansFor(customerId);

        boolean hasOverdueLoan = activeLoans.stream()
            .anyMatch(loan ->
↪ loan.dueDate().isBefore(LocalDate.now(clock)));

        if (hasOverdueLoan) {
            return EligibilityResult.denied("has an overdue loan");
        }
        if (activeLoans.size() >= 3) {
            return EligibilityResult.denied("too many active loans");
        }
        return EligibilityResult.approved();
    }
}
```

```
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

import java.time.*;
import java.util.List;

import static org.mockito.Mockito.*;
import static org.junit.jupiter.api.Assertions.*;

@ExtendWith(MockitoExtension.class)
class LoanEligibilityServiceTest {
```

```

    private static final Clock FIXED_CLOCK =
        Clock.fixed(Instant.parse("2026-08-07T00:00:00Z"),
↳ ZoneOffset.UTC);

    @Mock
    private LoanRepository loanRepository;

    private LoanEligibilityService service;

    @org.junit.jupiter.api.BeforeEach
    void setUp() {
        service = new LoanEligibilityService(loanRepository, FIXED_CLOCK);
    }

    @Test
    void checkEligibility_noActiveLoans_isApproved() {
↳ when(loanRepository.findActiveLoansFor("cust-1")).thenReturn(List.of());

        EligibilityResult result = service.checkEligibility("cust-1");

        assertTrue(result.approved());
    }

    @Test
    void checkEligibility_hasOverdueLoan_isDenied() {
        Loan overdue = new Loan("loan-1", LocalDate.parse("2026-08-01"));
↳ // before fixed "now"
        when(loanRepository.findActiveLoansFor("cust-1")).thenReturn(List
↳ .of(overdue));

        EligibilityResult result = service.checkEligibility("cust-1");

        assertFalse(result.approved());
        assertEquals("has an overdue loan", result.reason());
    }

    @Test
    void checkEligibility_threeActiveLoans_isDenied() {
        List<Loan> loans = List.of(
            new Loan("loan-1", LocalDate.parse("2026-12-01")),
            new Loan("loan-2", LocalDate.parse("2026-12-01")),
            new Loan("loan-3", LocalDate.parse("2026-12-01"))
        );
↳ when(loanRepository.findActiveLoansFor("cust-1")).thenReturn(loans);

        EligibilityResult result = service.checkEligibility("cust-1");

```

```

        assertFalse(result.approved());
        assertEquals("too many active loans", result.reason());
    }

    @Test
    void
    ↪ checkEligibility_queriesRepositoryExactlyOnceForTheGivenCustomer() {
        when(loanRepository.findActiveLoansFor(anyString())).thenReturn(
    ↪ List.of());

        service.checkEligibility("cust-42");

        verify(loanRepository, times(1)).findActiveLoansFor("cust-42");
        verifyNoMoreInteractions(loanRepository);
    }
}

```

Notice the fixed `Clock` makes the “overdue” scenario completely deterministic — no reliance on the real system date, so this test will pass identically today and five years from now.

Over-Mocking as an Anti-Pattern: Prefer a Fake

Mocking every collaborator, including simple, deterministic ones, produces tests that verify *wiring* instead of *behavior* — they pass even when the real logic is subtly wrong, because every answer was hand-fed by `when(...)`.

Before — over-mocked, brittle, and barely tests anything real:

```

@ExtendWith(MockitoExtension.class)
class ShoppingCartServiceOverMockedTest {

    @Mock
    private InventoryRepository inventoryRepository; //
    ↪ in-memory-friendly; doesn't need mocking

    @Mock
    private PricingRepository pricingRepository; // same

    @InjectMocks
    private ShoppingCartService cartService;

    @Test
    void addItem_computesTotalCorrectly() {
        when(inventoryRepository.hasStock("sku-1", 2)).thenReturn(true);
    }
}

```

```

        when(pricingRepository.priceOf("sku-1")).thenReturn(new
↪ BigDecimal("9.99"));

        cartService.addItem("sku-1", 2);

        // This assertion mostly re-proves the mock's own stubbed values,
↪ not the real
        // multiplication logic inside ShoppingCartService – a bug in the
↪ multiplication
        // could still pass if the stubs happen to line up.
        assertEquals(new BigDecimal("19.98"), cartService.total());
    }
}

```

Both `InventoryRepository` and `PricingRepository` are simple, deterministic, side-effect-free lookups — exactly the kind of collaborator better served by a real (if simplified) in-memory implementation than a mock.

After — a real in-memory fake exercises the actual logic:

```

class InMemoryInventoryRepository implements InventoryRepository {
    private final Map<String, Integer> stock = new HashMap<>();

    void setStock(String sku, int quantity) { stock.put(sku, quantity); }

    @Override
    public boolean hasStock(String sku, int quantity) {
        return stock.getDefault(sku, 0) >= quantity;
    }
}

class InMemoryPricingRepository implements PricingRepository {
    private final Map<String, BigDecimal> prices = new HashMap<>();

    void setPrice(String sku, BigDecimal price) { prices.put(sku, price);
↪ }

    @Override
    public BigDecimal priceOf(String sku) {
        return prices.getDefault(sku, BigDecimal.ZERO);
    }
}

class ShoppingCartServiceFakeTest {

    private final InMemoryInventoryRepository inventoryRepository = new
↪ InMemoryInventoryRepository();
}

```

```
private final InMemoryPricingRepository pricingRepository = new
↳ InMemoryPricingRepository();
private final ShoppingCartService cartService =
    new ShoppingCartService(inventoryRepository, pricingRepository);

@Test
void addItem_computesTotalCorrectly() {
    inventoryRepository.setStock("sku-1", 5);
    pricingRepository.setPrice("sku-1", new BigDecimal("9.99"));

    cartService.addItem("sku-1", 2);

    assertEquals(new BigDecimal("19.98"), cartService.total());
}

@Test
void addItem_insufficientStock_throws() {
    inventoryRepository.setStock("sku-1", 1);

    assertThrows(InsufficientStockException.class, () ->
↳ cartService.addItem("sku-1", 2));
}
}
```

The fake-based test is no more verbose, needs zero mocking annotations, reads like plain Java, and — crucially — actually exercises real lookup logic (`getOrDefault`, `map` storage) instead of blindly returning whatever was stubbed. Reserve mocks for the collaborators that genuinely have side effects (network, disk, email, payment) worth avoiding in a unit test; reach for a small fake for everything else.

	Mock	Stub	Fake	Spy
Real logic runs?	No	No	Yes (simplified)	Yes, unless overridden
Primary purpose	Verify interactions	Feed canned data	Realistic lightweight substitute	Observe + selectively override
Created with	<code>mock()</code> / <code>@Mock</code>	<code>mock()</code> + <code>when()</code>	Hand-written class implementing the interface	<code>spy()</code> / <code>@Spy</code>
Review red flag	Excessive <code>verifyNoMoreInteractions</code> on trivial calls	Stubbing something never exercised	None — often <i>reduces</i> smell	Spying on the class under test itself

Common Code-Review Interview Pitfalls

1. **Tests with no assertions (“coverage theater”).** Why it matters: the test executes code and reports green, but proves nothing — a regression can slip through untouched.

```
// Before
@Test
void processOrder_works() { orderService.process(order); }
// After
@Test
void processOrder_marksOrderAsShipped() {
    orderService.process(order);
    assertEquals(OrderStatus.SHIPPED, order.status());
}
```

2. **Meaningless test names (test1, testFoo, shouldWork).** Why it matters: a failing test in a CI report should be diagnosable from its name alone; vague names force everyone to open the source to understand what broke.

```
// Before
@Test void test1() { ... }
// After
@Test void withdraw_insufficientFunds_throwsIllegalStateException() {
    ... }
```

3. **Using Thread.sleep to wait for async work instead of a latch or Awaitility.** Why it matters: fixed sleeps are either wastefully slow or flaky under load — a frequent source of “works on my machine, fails on CI.”

```
// Before
Thread.sleep(500); assertTrue(flag.get());
// After
assertTrue(latch.await(2, TimeUnit.SECONDS)); assertTrue(flag.get());
```

4. **Calling LocalDate.now()/Instant.now() directly in production code instead of injecting a Clock.** Why it matters: makes date/time logic untestable deterministically, leading to tests that only fail once a year or in a different timezone.

```
// Before
boolean expired = expiry.isBefore(LocalDate.now());
// After
```

```
boolean expired = expiry.isBefore(LocalDate.now(clock)); // clock
↳ injected in constructor
```

- 5. Mixing raw argument values and Mockito matchers in the same call.** Why it matters: throws `InvalidUseOfMatchersException` at runtime instead of failing at compile time — a confusing failure mode for anyone unfamiliar with the rule.

```
// Before
when(userService.updateEmail(1L, anyString())).thenReturn(true);
// After
when(userService.updateEmail(eq(1L), anyString())).thenReturn(true);
```

- 6. Using `when(...).thenReturn(...)` on a spy for a method with real side effects.** Why it matters: the real method runs first (before Mockito applies the stub), triggering whatever expensive/exceptional behavior you were trying to avoid.

```
// Before
when(spyValidator.expensiveCheck(order)).thenReturn(true);
// After
doReturn(true).when(spyValidator).expensiveCheck(order);
```

- 7. Mocking value objects/DTOs instead of just constructing them.** Why it matters: produces an object whose getters return arbitrary stubbed values instead of real data, weakening the test's connection to reality and adding boilerplate for no benefit.

```
// Before
Money mockMoney = mock(Money.class);
↳ when(mockMoney.amount()).thenReturn(TEN);
// After
Money money = new Money(TEN, "USD");
```

- 8. Over-mocking simple, deterministic collaborators instead of using a real in-memory fake.** Why it matters: the test ends up verifying that stubs return what they were told to return, not that the real logic (calculations, branching) is correct.

```
// Before
when(pricingRepository.priceOf("sku-1")).thenReturn(new
↳ BigDecimal("9.99"));
// After
pricingRepository.setPrice("sku-1", new BigDecimal("9.99")); //
↳ InMemoryPricingRepository fake
```

9. **@Disabled with no reason string or tracked follow-up ticket.** Why it matters: skipped tests are easy to forget forever, silently eroding coverage without anyone noticing.

```
// Before
@Disabled
@Test void flakyTest() { ... }
// After
@Disabled("Flaky under load; see TICKET-1234")
@Test void flakyTest() { ... }
```

10. **Reaching for RETURNS_DEEP_STUBS instead of fixing a Law-of-Demeter violation.** Why it matters: it makes a bad dependency chain (`a.getB().getC().getD()`) testable without questioning why the code reaches three objects deep into someone else's internals.

```
// Before
when(context.getCustomer().getAddress().getCountry()).thenReturn("DE");
// After
when(context.getCustomerCountry()).thenReturn("DE"); // narrower,
↳ single-hop dependency
```

11. **Asserting only the exception type, ignoring the message/state.** Why it matters: the wrong code path can throw the same exception type for a different reason, and the test would still pass, masking a real bug.

```
// Before
assertThrows(IllegalArgumentException.class, () ->
↳ service.transfer(-10));
// After
var ex = assertThrows(IllegalArgumentException.class, () ->
↳ service.transfer(-10));
assertEquals("transfer amount must be positive: -10", ex.getMessage());
```

12. **Chasing 100% line coverage instead of testing edge cases and failure paths.** Why it matters: coverage percentage measures execution, not verification; time spent testing trivial getters is time not spent on the branch that actually breaks in production.

```
// Before: test exists purely to hit the line
@Test void getName_doesNotThrow() { assertNotNull(user.getName()); }
// After: test the behavior that matters
```

```
@Test void withdraw_exactBalance_leavesZeroBalance() { ... }
```

- 13. Ignoring UnnecessaryStubbingException by sprinkling lenient() everywhere.** Why it matters: strict stubbing exists to catch leftover, copy-pasted stubs that no longer reflect what the test exercises; blanket `lenient()` defeats that safety net.

```
// Before
lenient().when(repo.findAll()).thenReturn(List.of()); // added to
→ silence a warning, never investigated
// After
// delete the unused stub, or confirm and document why it's
→ legitimately conditional
```

- 14. One giant test method covering five unrelated scenarios instead of @ParameterizedTest or separate tests.** Why it matters: a failure only says “somewhere in this 80-line test something broke,” instead of pinpointing which scenario regressed.

```
// Before
@Test void allDiscountScenarios() { /* 5 unrelated asserts */ }
// After
@ParameterizedTest
@CsvSource({"REGULAR,0", "PREMIUM,10", "VIP,20"})
void discount_variesByTier(CustomerTier tier, int expectedPercent) {
→ ... }
```

- 15. Depending on test execution order (state leaking between tests via static/shared fields).** Why it matters: JUnit does not guarantee method order; a suite that only passes in one order is a latent flaky-test time bomb waiting for a JUnit version bump or parallel execution to expose it.

```
// Before
static User sharedUser; // set in test A, read in test B
// After
@BeforeEach void setUp() { user = new User(...); } // fresh per test
```

- 16. Mocking a type you don't own (JDK/third-party class) instead of wrapping it behind your own seam.** Why it matters: couples tests to a library's internal shape, breaks across library upgrades, and often fights `final` classes/methods that resist mocking cleanly.

```
// Before
HttpClient mockClient = mock(HttpClient.class); // fragile, tightly
↳ coupled to the SDK
// After
WeatherClient mockClient = mock(WeatherClient.class); // your own
↳ one-method interface
```

- 17. mockStatic/mockConstruction used as a first choice for new code instead of dependency injection.** Why it matters: static mocking is a workaround for code that can't be injected; reaching for it in new code perpetuates hard-to-test designs instead of fixing them.

```
// Before
try (MockedStatic<Instant> m = mockStatic(Instant.class)) {
↳ m.when(Instant::now).thenReturn(fixed); ... }
// After
// inject java.time.Clock into the constructor and use Clock.fixed(...)
↳ in the test
```

- 18. Blanket verifyNoMoreInteractions on every mock in every test, making refactors brittle.** Why it matters: adding an unrelated, harmless call (e.g., a new debug log) breaks unrelated tests, training the team to treat verification failures as noise instead of signal.

```
// Before
verifyNoMoreInteractions(userRepository, emailSender, auditLog,
↳ metricsClient); // on every test
// After
verify(emailSender).sendWelcomeEmail(anyString()); // verify only the
↳ interactions that matter to this test
```


Build & Documentation

A reviewer rarely gets to pick the build tool on an existing project, but they are expected to read a `pom.xml` or `build.gradle.kts` fluently, spot a dependency-scope mistake, and know when a Javadoc comment is padding rather than documentation. This chapter covers Maven and Gradle in depth — enough to read, modify, and debug real build files — and Javadoc, the API-contract documentation every reviewer is expected to enforce. We target Java 21+, Maven 3.9+, and Gradle 8.x throughout.

Maven

Maven is a **declarative, convention-over-configuration** build tool. You describe *what* your project is (its coordinates, dependencies, packaging) in an XML file called `pom.xml` (Project Object Model), and Maven figures out *how* to build it by running a fixed, well-known **lifecycle**. Compare this to Gradle, which is closer to a scripting/task-graph model — see the [Maven vs Gradle](#) table later in this chapter.

Convention Over Configuration and the Standard Directory Layout

Maven assumes a standard project layout so that a plain `pom.xml` with no extra configuration already knows where to find source, tests, and resources:

```

my-app/
├── pom.xml
├── src/
│   ├── main/
│   │   ├── java/           # application source (.java)
│   │   └── resources/      # files copied onto the classpath as-is
│   └── test/
│       ├── java/           # test source
│       └── resources/      # test-only resources
└── target/                 # generated output – never commit this

```

```

├── classes/
├── test-classes/
└── my-app-1.0.0.jar

```

Because this layout is a convention, plugins do not need to be told where anything is. Deviating from it (e.g., putting sources in `source/`) means every plugin that touches source directories must be reconfigured — almost never worth it.

A Full Annotated pom.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
↳ http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <!-- Coordinates: uniquely identify this artifact in any repository -->
  <groupId>com.example.orders</groupId>
  <artifactId>orders-service</artifactId>
  <version>1.4.0-SNAPSHOT</version>
  <packaging>jar</packaging>      <!-- jar (default), war, pom, ear ...
↳ -->

  <name>Orders Service</name>
  <description>REST API for managing customer orders.</description>

  <!-- Properties: reusable values, also read by many plugins -->
  <properties>
    <maven.compiler.release>21</maven.compiler.release>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <junit.version>5.10.2</junit.version>
  </properties>

  <!-- dependencyManagement declares *versions/scopes* without adding the
       dependency to the build. Children/modules inherit the version but
       must still declare the dependency themselves. This is how BOMs work.
↳ -->
  <dependencyManagement>
    <dependencies>
      <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-dependencies</artifactId>
        <version>3.3.0</version>

```



```

        <type>pom</type>
        <scope>import</scope>
    </dependency>
</dependencies>
</dependencyManagement>

<!-- Actual dependencies used by this module -->
<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
        <!-- no <version> needed: it comes from the imported BOM above -->
    </dependency>

    <dependency>
        <groupId>org.projectlombok</groupId>
        <artifactId>lombok</artifactId>
        <version>1.18.32</version>
        <scope>provided</scope>
    </dependency>

    <dependency>
        <groupId>org.junit.jupiter</groupId>
        <artifactId>junit-jupiter</artifactId>
        <version>${junit.version}</version>
        <scope>test</scope>
    </dependency>
</dependencies>

<build>
    <finalName>${project.artifactId}</finalName>
    <plugins>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-compiler-plugin</artifactId>
            <version>3.13.0</version>
            <configuration>
                <release>${maven.compiler.release}</release>
            </configuration>
        </plugin>

        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-surefire-plugin</artifactId>
            <version>3.2.5</version>
        </plugin>
    </plugins>
</build>

```

```
<!-- Profiles: conditionally activated configuration -->
<profiles>
  <profile>
    <id>ci</id>
    <activation>
      <property><name>env.CI</name></property>
    </activation>
    <properties>
      <maven.test.failure.ignore>false</maven.test.failure.ignore>
    </properties>
  </profile>
</profiles>
</project>
```

Coordinates: groupId / artifactId / version, SNAPSHOT vs Release

Every artifact in a Maven repository is uniquely addressed by three (sometimes four) coordinates:

Coordinate	Meaning	Example
groupId	Organization/namespace, usually a reversed domain	com.example.orders
artifactId	The specific module/artifact name	orders-service
version	The version of that artifact	1.4.0, 1.4.0-SNAPSHOT
packaging	Artifact type (optional, defaults to jar)	jar, war, pom
classifier	Distinguishes artifacts with the same GAV	sources, javadoc

SNAPSHOT vs release: a version ending in `-SNAPSHOT` (e.g., `1.4.0-SNAPSHOT`) is a mutable, in-progress version — Maven will re-check the remote repository for a newer snapshot on every build (or per the repository’s update policy) because the artifact behind that version string can change. A version without `-SNAPSHOT` (e.g., `1.4.0`) is a **release**: immutable, published once, never overwritten. Deploying the same release version twice to a repository that enforces immutability will fail — that’s intentional, it protects reproducibility. Application `pom.xml` files should never depend on a `-SNAPSHOT` artifact when released to production; it means the build is not reproducible.

The Build Lifecycle: Phases, Goals, and Plugins

Maven has three built-in **lifecycles**: default (build and deploy the artifact), clean (remove target/), and site (generate documentation site). The default lifecycle is the one developers use constantly, and it's a fixed, ordered sequence of **phases**:

```

validate → initialize → generate-sources → process-sources →
→ generate-resources
  → process-resources → compile → process-classes → generate-test-sources
  → process-test-sources → generate-test-resources → process-test-resources
  → test-compile → test → prepare-package → package → pre-integration-test
  → integration-test → post-integration-test → verify → install → deploy

```

The phases most people actually type at the command line:

Phase	What it does
validate	Checks the project is correct and all information is available
compile	Compiles main source code
test	Runs unit tests (via Surefire) — does not package anything
package	Packages compiled code into a jar/war
verify	Runs checks on the package, including integration tests (via Failsafe)
install	Installs the package into the local repository (~/.m2/repository), for use by other local projects
deploy	Copies the package to a remote repository, for sharing with other developers/CI

Key mental model: **running a phase runs every phase before it too**. `mvn install` runs `validate` → ... → `package` → `verify` → `install`, in order.

A **goal** is the actual unit of work — a specific piece of functionality provided by a **plugin**, written as `plugin:goal` (e.g., `compiler:compile`, `surefire:test`, `dependency:tree`). Phases don't *do* anything by themselves; they are just named hooks that plugin goals are **bound** to.

Phase	Bound goal (default binding)	Plugin
-----	-----	
→ -----		
compile	→ compiler:compile	→ maven-compiler-plugin
test	→ surefire:test	→ maven-surefire-plugin

package	→	jar:jar	→	maven-jar-plugin
install	→	install:install	→	maven-install-plugin
deploy	→	deploy:deploy	→	maven-deploy-plugin

You can also invoke a goal directly without running a whole phase chain, useful for diagnostics:

```
mvn dependency:tree
mvn compiler:compile
mvn versions:display-dependency-updates
```

Plugin vs goal vs phase, in one sentence each:

- A **phase** is a named step in the lifecycle (compile, test, package).
- A **plugin** is a jar containing one or more executable goals (maven-surefire-plugin).
- A **goal** is the actual task a plugin performs, optionally bound to a phase (surefire:test is bound to the test phase).

Dependency Scopes

Scope	On compile class-path	On test class-path	On run-time class-path	Packag in artifact	Typical use
compile (default)	Yes	Yes	Yes	Yes	Normal library dependency
provided	Yes	Yes	No	No	Servlet API, Lombok — supplied by the container/JDK at runtime
runtime	No	Yes	Yes	Yes	JDBC drivers — needed at runtime, not to compile against
test	No	Yes	No	No	JUnit, Mockito, AssertJ
system	Yes	Yes	No	No	Local jar via explicit <systemPath> — avoid, not portable

Scope	On com- pile class- path	On test class- path	On run- time class- path	Packag in ar- tifact	Typical use
import	n/a	n/a	n/a	n/a	Only valid in <dependencyManagement>, type=pom — pulls in a BOM's managed versions

```

<dependency>
  <groupId>org.postgresql</groupId>
  <artifactId>postgresql</artifactId>
  <version>42.7.3</version>
  <scope>runtime</scope>
</dependency>

```

A common review flag: marking a dependency compile when it's only needed at runtime (bloats the compile classpath and leaks transitively to consumers), or forgetting provided for a servlet container API that will already be on the classpath when deployed, causing a duplicate/incompatible class at deploy time.

Transitive Dependencies, Conflict Resolution, and `dependency:tree`

Maven pulls in **transitive dependencies** automatically — if your dependency depends on something, you get that something too, without declaring it. This is convenient but is also the #1 source of “it works on my machine” classpath bugs.

When two versions of the same artifact are pulled in transitively at different depths, Maven uses **nearest-wins** (also called “nearest definition”) resolution: the dependency declared closest to your project in the dependency graph wins, regardless of version number. If two dependencies are at the *same* depth, the first one declared in the POM wins.

```

your-app
├── lib-a:1.0 → depends on commons-lang3:3.9
└── lib-b:1.0 → depends on commons-lang3:3.14

```

Here commons-lang3 is at the same depth (both direct children of lib-a/lib-b, which are depth 1, so commons-lang3 is depth 2 either way) — the version resolved is

whichever is declared first in your POM's dependency list. This is exactly why nearest-wins can silently pick an old, buggy, or incompatible version. Never trust it blindly — always inspect the tree:

```
mvn dependency:tree
mvn dependency:tree -Dincludes=commons-lang3
mvn dependency:tree -Dverbose    # shows the versions that lost, and why
```

To force a specific version regardless of what transitive resolution would pick, either declare the dependency directly (direct declarations always win over transitive ones) or use `dependencyManagement` (see below), or exclude the unwanted transitive dependency:

```
<dependency>
  <groupId>com.example</groupId>
  <artifactId>lib-a</artifactId>
  <version>1.0</version>
  <exclusions>
    <exclusion>
      <groupId>org.apache.commons</groupId>
      <artifactId>commons-lang3</artifactId>
    </exclusion>
  </exclusions>
</dependency>
```

dependencyManagement and BOMs

`dependencyManagement` centralizes version (and scope/exclusion) declarations **without adding the dependency to the build**. A child module or a plugin still needs a bare `<dependency>` (no version) to actually pull the artifact in; the version comes from the management block. This is how you avoid version drift across a multi-module project.

A **BOM** (Bill of Materials) is a pom-packaged artifact whose sole purpose is a big `dependencyManagement` block of mutually-tested versions (Spring Boot's, Jackson's, and the AWS SDK's BOMs are common examples). Importing one gives you a consistent, tested set of versions for an entire ecosystem in one line:

```
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>com.fasterxml.jackson</groupId>
```

```

        <artifactId>jackson-bom</artifactId>
        <version>2.17.1</version>
        <type>pom</type>
        <scope>import</scope>
      </dependency>
    </dependencies>
  </dependencyManagement>

  <dependencies>
    <!-- version omitted on purpose: comes from the imported BOM -->
    <dependency>
      <groupId>com.fasterxml.jackson.core</groupId>
      <artifactId>jackson-databind</artifactId>
    </dependency>
  </dependencies>

```

Parent POMs and Multi-Module Builds

A **parent POM** lets multiple modules share configuration (properties, dependencyManagement, plugin versions) via inheritance:

```

<!-- child module's pom.xml -->
<parent>
  <groupId>com.example</groupId>
  <artifactId>orders-parent</artifactId>
  <version>1.0.0</version>
  <relativePath>../pom.xml</relativePath>
</parent>
<artifactId>orders-service</artifactId>

```

A **multi-module** (aggregator) build is a parent pom.xml with packaging=pom and a <modules> list; building the parent builds every listed module in dependency order:

```

<groupId>com.example</groupId>
<artifactId>orders-parent</artifactId>
<version>1.0.0</version>
<packaging>pom</packaging>

<modules>
  <module>orders-api</module>
  <module>orders-service</module>
  <module>orders-persistence</module>
</modules>

```

```

mvn install                                # builds all modules, in dependency
↪ order
mvn -pl orders-service -am install         # build only orders-service + its
↪ dependents

```

A module can be both a *child* (inherits from a parent) and independently list its own dependencies — parent and aggregator are separate, orthogonal concepts, even though the same `pom.xml` often plays both roles.

Profiles

Profiles activate additional configuration conditionally — by an explicit flag, an OS, a JDK version, a file's presence, or a system/environment property. Common uses: switching database connection properties for local vs CI, enabling static analysis only on CI, or activating a native-image build only on demand.

```

<profiles>
  <profile>
    <id>integration-tests</id>
    <build>
      <plugins>
        <plugin>
          <groupId>org.apache.maven.plugins</groupId>
          <artifactId>maven-failsafe-plugin</artifactId>
          <executions>
            <execution>
↪ <goals><goal>integration-test</goal><goal>verify</goal></goals>
              </execution>
            </executions>
          </plugin>
        </plugins>
      </build>
    </profile>
  </profiles>

```

```
mvn verify -Pintegration-tests
```


Properties

Properties are `${key}`-substitutable values, declared in `<properties>`, usable anywhere in the POM (versions, plugin config) and readable by many plugins directly (e.g., `maven.compiler.release`, `project.build.sourceEncoding`). Maven also exposes implicit properties like `${project.version}` and `${project.basedir}`.

```
<properties>
  <maven.compiler.release>21</maven.compiler.release>
  <spring.version>6.1.8</spring.version>
</properties>
```

Common Plugins

Plugin	Purpose
maven-compiler-plugin	Compiles Java; use <code><release>21</release></code> (not the deprecated source/target pair) so both the language level <i>and</i> the bootclasspath match Java 21 exactly
maven-surefire-plugin	Runs unit tests during the test phase
maven-failsafe-plugin	Runs integration tests during <code>integration-test/verify</code> ; failures don't stop the build until <code>verify</code> , so post-integration-test cleanup still runs
maven-shade-plugin	Builds an uber/fat jar by unpacking dependency classes and merging them into one jar; supports relocating packages to avoid classpath collisions
maven-assembly-plugin	Also builds fat jars/distribution archives, but merges by concatenation, not by understanding class content — less safe for merging <code>META-INF/services</code> files than shade's transformers
jacoco-maven-plugin	Code coverage instrumentation and reporting
maven-enforcer-plugin	Fails the build if rules are violated — e.g., banned dependencies, minimum Maven/JDK version, no duplicate classes
versions-maven-plugin	Reports and can update outdated dependency/plugin versions (<code>versions:display-dependency-updates</code>)

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-compiler-plugin</artifactId>
  <configuration>
    <release>21</release>
```

```

        <parameters>true</parameters>    <!-- keep parameter names for
↪ reflection -->
    </configuration>
</plugin>

<plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-shade-plugin</artifactId>
    <version>3.6.0</version>
    <executions>
        <execution>
            <phase>package</phase>
            <goals><goal>shade</goal></goals>
            <configuration>
                <transformers>
↪      <transformer implementation=
"org.apache.maven.plugins.shade.resource.ManifestResourceTransformer">
                    <mainClass>com.example.orders.Main</mainClass>
                </transformer>
            </transformers>
        </configuration>
    </execution>
</executions>
</plugin>

<plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-enforcer-plugin</artifactId>
    <executions>
        <execution>
            <goals><goal>enforce</goal></goals>
            <configuration>
                <rules>

↪      <requireMavenVersion><version>[3.9,)</version></requireMavenVersion>

↪      <requireJavaVersion><version>[21,)</version></requireJavaVersion>
                <bannedDependencies>
                    <excludes><exclude>log4j:log4j</exclude></excludes>
                </bannedDependencies>
            </rules>
        </configuration>
    </execution>
</executions>
</plugin>

```

The Local Repository and Offline Mode

Maven caches every artifact it downloads under `~/.m2/repository`, keyed by GAV coordinates. `mvn install` places your own module's artifact there too, so sibling projects can depend on it without a remote repository round-trip.

Reproducible builds: pin exact versions (avoid version ranges like `[1.0,2.0)`, which resolve differently over time), pin plugin versions explicitly (an unpinned plugin resolves to “latest” the first time it's used and then locks to whatever was cached — inconsistent across machines/CI), and prefer `-SNAPSHOT`-free dependencies for anything released. Maven also supports a reproducible build mode via `<project.build.outputTimestamp>` so two builds of the same source produce byte-identical jars (same file timestamps inside the archive).

```
mvn -o package           # offline: fail fast if anything is missing from
↳ ~/.m2, don't hit the network
mvn -o dependency:tree # useful to sanity-check nothing is silently
↳ redownloaded
```

Skipping Tests: `-DskipTests` vs `-Dmaven.test.skip`

These are **not** the same thing, and mixing them up is a classic review flag:

Flag	Compiles test sources?	Runs tests?
<code>-DskipTests</code>	Yes	No
<code>-Dmaven.test.skip=true</code>	No	No

```
mvn install -DskipTests           # tests are compiled (catches compile
↳ errors) but not run
mvn install -Dmaven.test.skip=true # test sources aren't even compiled –
↳ fastest, but hides broken tests
```

Using `-Dmaven.test.skip=true` in CI can let a broken test file merge unnoticed, since it never even compiles until someone runs tests locally.

The Maven Wrapper

The **Maven Wrapper** (`mvnw` / `mvnw.cmd`) pins the exact Maven version a project needs, generated once with `mvn wrapper:wrapper` and committed to the repository. Anyone cloning the repo runs `./mvnw` and gets a reproducible Maven version automatically downloaded — no “works with Maven 3.9 but the CI box has 3.6” surprises.

```
mvn wrapper:wrapper -Dmaven=3.9.6    # generate the wrapper, pin version
↪ 3.9.6
./mvnw clean verify                  # anyone can build without installing
↪ Maven at all
```

Commit `mvnw`, `mvnw.cmd`, and `.mvn/wrapper/maven-wrapper.properties` — never `.gitignore` them.

Command Cheat-Sheet

```
mvn clean                            # delete target/
mvn compile                          # compile main sources
mvn test                             # run unit tests
mvn package                          # produce jar/war
mvn verify                           # run integration tests + checks
mvn install                          # install into ~/.m2
mvn deploy                           # publish to remote repository

mvn dependency:tree                   # show resolved dependency graph
mvn dependency:analyze                # find unused/undeclared direct
↪ dependencies
mvn versions:display-dependency-updates # find outdated dependency
↪ versions
mvn help:effective-pom                # print the fully resolved/merged POM

mvn -pl module-a -am install          # build module-a plus everything it
↪ depends on
mvn -pl module-a -amd install          # build module-a plus everything that
↪ depends on it
mvn -T 4 install                      # build with 4 threads (parallel
↪ modules)
mvn -X verify                         # debug/verbose logging
mvn -q verify                         # quiet output, errors only
```

Gradle

Gradle is a general-purpose build tool built around a **task graph** rather than a fixed lifecycle: you (or a plugin) define **tasks** and their dependencies, and Gradle figures out which tasks need to run and in what order, skipping anything whose inputs haven't changed. Build scripts are code (Groovy or Kotlin), not declarative XML, which makes Gradle more flexible but also easier to write build logic that's hard to reason about — a common review concern.

Groovy DSL vs Kotlin DSL

Both DSLs configure the same underlying model. Kotlin DSL (`build.gradle.kts`) gets full IDE type-checking and autocompletion; Groovy DSL (`build.gradle`) is more concise and was the historical default. Since Gradle 8.x, Kotlin DSL is the recommended default for new projects.

```
// build.gradle (Groovy DSL)
plugins {
    id 'java'
    id 'application'
}

repositories {
    mavenCentral()
}

dependencies {
    implementation 'com.google.guava:guava:33.2.1-jre'
    testImplementation platform('org.junit:junit-bom:5.10.2')
    testImplementation 'org.junit.jupiter:junit-jupiter'
}

java {
    toolchain {
        languageVersion = JavaLanguageVersion.of(21)
    }
}

application {
    mainClass = 'com.example.App'
}

test {
    useJUnitPlatform()
}
```

```
// build.gradle.kts (Kotlin DSL) – same build, statically typed
plugins {
    java
    application
}

repositories {
    mavenCentral()
}

dependencies {
    implementation("com.google.guava:guava:33.2.1-jre")
    testImplementation(platform("org.junit:junit-bom:5.10.2"))
    testImplementation("org.junit.jupiter:junit-jupiter")
}

java {
    toolchain {
        languageVersion.set(JavaLanguageVersion.of(21))
    }
}

application {
    mainClass.set("com.example.App")
}

tasks.test {
    useJUnitPlatform()
}
```

An Annotated build.gradle.kts

```
plugins {
    id("java-library")                // produces api()/implementation()
    ↪ id("distinction")                // coverage
    id("jacoco")                      // coverage
    id("com.github.spotbugs") version "6.0.18"
}

group = "com.example.orders"
version = "1.4.0-SNAPSHOT"

repositories {
    mavenCentral()
}
```

```

dependencies {
    // api: exposed to consumers of THIS library on their compile
    → classpath
    api("com.fasterxml.jackson.core:jackson-databind:2.17.1")

    // implementation: internal detail, NOT leaked to consumers
    implementation("org.apache.commons:commons-lang3:3.14.0")

    // compileOnly: needed to compile, but must be supplied at runtime by
    → the caller
    compileOnly("org.projectlombok:lombok:1.18.32")
    annotationProcessor("org.projectlombok:lombok:1.18.32")

    // runtimeOnly: needed at runtime, not visible at compile time
    runtimeOnly("org.postgresql:postgresql:42.7.3")

    testImplementation(platform("org.junit:junit-bom:5.10.2"))
    testImplementation("org.junit.jupiter:junit-jupiter")
    testImplementation("org.assertj:assertj-core:3.26.0")
}

java {
    toolchain {
        languageVersion.set(JavaLanguageVersion.of(21))
    }
    withSourcesJar()
    withJavadocJar()
}

tasks.test {
    useJUnitPlatform()
    maxParallelForks = Runtime.getRuntime().availableProcessors() / 2
}

tasks.jacocoTestReport {
    dependsOn(tasks.test)
}

```

The `plugins {}` Block

The `plugins {}` block applies **binary plugins** by ID, resolved from the Gradle Plugin Portal (or `mavenCentral()` for some). It replaces the older, discouraged `apply plugin: 'java' / buildscript-classpath` style, because it lets Gradle resolve and version-check plugins before any script code runs, enabling better IDE support and the plugin version-management shown below.

```
plugins {
    java
    id("org.springframework.boot") version "3.3.0"
    id("io.spring.dependency-management") version "1.1.5"
}
```

The Task Graph and Incremental Builds

Every unit of work in Gradle is a **task** (`compileJava`, `test`, `jar`, ...). Tasks declare **inputs** and **outputs**; Gradle hashes them, and if neither changed since the last run, the task is reported UP-TO-DATE and skipped entirely — this is **incremental build**. Running `./gradlew build` twice in a row with no source changes does almost nothing the second time.

```
$ ./gradlew build
> Task :compileJava
> Task :processResources UP-TO-DATE
> Task :classes
> Task :test
> Task :jar
> Task :build

BUILD SUCCESSFUL
```

You can inspect why a task ran (or didn't) and visualize the dependency graph:

```
./gradlew build --dry-run           # print the task plan without executing
↳ anything
./gradlew test --rerun              # force a task to rerun even if
↳ UP-TO-DATE
```

Configuration Cache and Build Cache

These are two distinct, complementary optimizations, and mixing them up is a common misunderstanding:

- **Configuration cache:** caches the *result of evaluating the build scripts themselves* (the task graph), so subsequent invocations skip re-running the configuration phase. Speeds up every single build, even a clean one, because it avoids re-parsing/re-executing Groovy/Kotlin build logic. Enable with `org.gradle.configuration-cache=true` in `gradle.properties`.

- **Build cache:** caches *task outputs* keyed by task input hash, and can restore an already-computed output on a *different* machine (with a shared remote cache) or a different branch, skipping execution entirely — not just “up to date” within one workspace, but genuinely retrieved from cache. Enable with `org.gradle.caching=true`.

```
# gradle.properties
org.gradle.configuration-cache=true
org.gradle.caching=true
org.gradle.parallel=true
```

```
./gradlew build --build-cache      # explicitly opt in to the build cache
↪ for this invocation
```

Dependency Configurations

Configuration	Compile classpath (consumer sees it?)	Runtime classpath	Typical use
api	Yes — leaks to consumers	Yes	Types that appear in your public method signatures
implementation	No — internal only	Yes	Everything else; the default, safest choice
compileOnly	Yes (this module only)	No	Annotation-only or container-supplied deps (Lombok, servlet API)
runtimeOnly	No	Yes	JDBC drivers, logging backends
testImplementation	Yes (compile only)	Test runtime	JUnit, Mockito
annotationProcessor	Yes (processor classpath)	No	Lombok, MapStruct, Dagger processors

Why implementation beats the old compile: the deprecated `compile` configuration put every dependency on the *consumer’s* compile classpath transitively, whether or not the consumer’s code actually referenced it. That meant bumping an internal-only dependency’s version in a library forced every downstream module to recompile, even though nothing in the public API changed. `implementation` hides internal dependencies from consumers entirely, which (a) shrinks consumers’ compile classpaths,

(b) means changing an internal dependency no longer forces downstream recompilation, and © makes accidental leakage of internal types through public APIs a compile error for consumers, rather than something that silently works until the dependency changes.

```
dependencies {
    // Consumer of this module can call methods that return ObjectMapper
    ↪ directly:
        api("com.fasterxml.jackson.databind:jackson-databind:2.17.1")

    // Used internally only, e.g., for a private helper class – invisible
    ↪ to consumers:
        implementation("org.apache.commons:commons-lang3:3.14.0")
}
```

Version Catalogs (libs.versions.toml)

A **version catalog** centralizes dependency coordinates and versions in one TOML file, shared across all subprojects, with type-safe accessors generated for both DSLs:

```
# gradle/libs.versions.toml
[versions]
junit = "5.10.2"
jackson = "2.17.1"

[libraries]
junit-jupiter = { module = "org.junit.jupiter:junit-jupiter", version.ref
    ↪ = "junit" }
jackson-databind = { module =
    ↪ "com.fasterxml.jackson.core:jackson-databind", version.ref = "jackson" }

[bundles]
testing = ["junit-jupiter"]

[plugins]
spring-boot = { id = "org.springframework.boot", version = "3.3.0" }
```

```
// build.gradle.kts – type-safe accessors generated from the catalog above
plugins {
    alias(libs.plugins.spring.boot)
}

dependencies {
```

```

    implementation(libs.jackson.databind)
    testImplementation(libs.bundles.testing)
}

```

This is the Gradle equivalent of a Maven BOM plus centralized `<properties>`: one file, one place to bump a version, autocomplete in the IDE, and no version drift between subprojects.

Multi-Project Builds and `settings.gradle.kts`

`settings.gradle.kts` (at the repo root) declares which subprojects participate in the build — this is what makes a directory a “project” at all, before any `build.gradle.kts` is even read.

```

// settings.gradle.kts
rootProject.name = "orders-platform"

include("orders-api", "orders-service", "orders-persistence")

dependencyResolutionManagement {
    repositories {
        mavenCentral()
    }
}

```

A subproject depends on another subproject with the `project()` accessor, not a version string:

```

// orders-service/build.gradle.kts
dependencies {
    implementation(project(":orders-api"))
}

```

```

./gradlew :orders-service:build    # build a single subproject
./gradlew build                    # build everything from the root

```

Custom Tasks

```
tasks.register("printVersion") {
    doLast {
        println("Building version ${project.version}")
    }
}

tasks.register<Copy>("copyConfig") {
    from("src/main/resources/config")
    into(layout.buildDirectory.dir("config"))
}

tasks.named("build") {
    dependsOn("printVersion")
}
```

Custom tasks with typed inputs/outputs (rather than `doLast {}` closures) can participate correctly in incremental build and the build cache — a plain `doLast` task has no declared inputs/outputs and Gradle can never mark it UP-TO-DATE.

The Gradle Wrapper

Exactly like Maven's wrapper, `gradlew/gradlew.bat` pin the exact Gradle version a project was built and tested with, downloaded on demand.

```
gradle wrapper --gradle-version 8.8 # (re)generate the wrapper for
↳ version 8.8
./gradlew build # anyone can build without
↳ installing Gradle
```

Always commit the wrapper (`gradlew`, `gradlew.bat`, `gradle/wrapper/gradle-wrapper.jar`, `gradle/wrapper/gradle-wrapper.properties`). Without it, a teammate or CI runner using a different globally-installed Gradle version can produce a different, non-reproducible build — or fail outright on a script feature that only exists in a newer/older Gradle.

Diagnostics: --scan, --dry-run, dependencies

```

./gradlew build --scan           # publish a shareable, detailed build
→ report (timings, deprecations, task graph)
./gradlew build --dry-run       # show which tasks WOULD run, without
→ running them
./gradlew dependencies          # print the resolved dependency tree for
→ all configurations
./gradlew dependencies --configuration runtimeClasspath # scope it to
→ one configuration
./gradlew :orders-service:dependencyInsight --dependency commons-lang3 #
→ why is this version chosen?

```

Maven vs Gradle Comparison

Aspect	Maven	Gradle
Config style	Declarative XML	Code (Groovy or Kotlin DSL)
Build model	Fixed lifecycle of phases	Flexible task graph
Incremental builds	Limited (plugin-dependent)	First-class, built into every task
Build/config caching	No built-in equivalent	Configuration cache + (remote) build cache
Multi-module	Parent POM + <modules>	settings.gradle.kts + subprojects
Dependency management	dependencyManagement + BOM import	Version catalogs (libs.versions.toml)
Learning curve	Lower — one way to do things	Higher — scripting flexibility cuts both ways
Ecosystem	Extremely mature, huge plugin base, Java/Enterprise-centric	Dominant for Android; strong in polyglot/large monorepos
Performance on large builds	Slower on very large multi-module builds	Generally faster due to caching/incrementality
Debuggability	Very predictable, easy to reason about from the POM alone	Custom Groovy/Kotlin logic can be hard to review/debug

When to pick which: choose Maven for straightforward, mostly-declarative enterprise Java projects where predictability and a large, stable plugin ecosystem matter more than raw build speed, or when the team values “one obvious way to configure things.”

Choose Gradle for large monorepos, Android (mandatory), polyglot builds, or when build performance (incremental/cached builds) matters enough to justify the extra scripting flexibility and the review overhead that comes with it.

Javadoc

Javadoc is the documentation format baked into the Java language itself: specially-formatted comments (`/** ... */`) immediately before a declaration, parsed by the javadoc tool (and IDEs) into structured API documentation. In code review, Javadoc quality is judged on one criterion above all others: **does it describe the contract, not the implementation?**

The Comment Format

A Javadoc comment starts with `/**` (two asterisks), ends with `*/`, and sits directly above the declaration it documents — a class, interface, method, constructor, or field. Every other `*`-prefixed line is conventional but not required by the parser.

```
/**
 * Converts a monetary amount between currencies using the exchange rate
 * in effect at the time this method is called.
 *
 * @param amount the amount to convert; must be non-negative
 * @param from the source currency
 * @param to the target currency
 * @return the converted amount, rounded to the target currency's
 *         standard number of decimal places
 * @throws IllegalArgumentException if {@code amount} is negative
 * @throws CurrencyUnavailableException if no exchange rate is available
 *         for the given currency pair
 */
public BigDecimal convert(BigDecimal amount, Currency from, Currency to) {
    ...
}
```

Standard Tags

Tag	Where used	Purpose
@param	Methods, constructors	Describes a parameter — name plus meaning/constraints
@return	Methods (non-void)	Describes the return value and its meaning
@throws (or @exception)	Methods, constructors	Describes when/why an exception is thrown
@see	Anywhere	Cross-reference to a related class/method/URL
@since	Classes, methods	Version this API first appeared in
@deprecated	Classes, methods, fields	Marks the API as obsolete; should always explain the replacement
@author	Classes	Author name(s) — increasingly omitted in favor of VCS history
@serial	Fields (of Serializable classes)	Documents the serialized form of a field for the serialization contract

```

/**
 * Legacy price calculator.
 *
 * @deprecated Use {@link PricingEngine#calculate(Order)} instead; this
 *             class does not account for regional tax rules and will be
 *             removed in 3.0.
 */
@Deprecated(since = "2.4", forRemoval = true)
public class LegacyPriceCalculator {
    ...
}

```

Note the pairing convention: `@deprecated` in the Javadoc comment (human-readable explanation) should always accompany the `@Deprecated` annotation (machine-readable, drives compiler warnings) — one without the other is an incomplete deprecation.

Inline Tags

Inline tags are embedded inside the flowing text of a description, wrapped in `{@ ... }`.

Inline tag	Purpose
<code>{@link Type#member}</code>	Clickable cross-reference, rendered as a link
<code>{@linkplain Type#member}</code>	Same as <code>{@link}</code> but rendered as plain text, not code font
<code>{@code text}</code>	Renders as code font, and — critically — HTML-escapes its content, so <code><</code> , <code>></code> , <code>&</code> are safe
<code>{@literal text}</code>	Escapes HTML like <code>{@code}</code> but does <i>not</i> apply code font
<code>{@value #FIELD}</code>	Inlines the value of a static final constant
<code>{@inheritDoc}</code>	Pulls the Javadoc text from the overridden/implemented method
<code>{@snippet ...}</code>	(JDK 18+) Embeds a validated, syntax-highlighted code example

```
/**
 * The default timeout, in milliseconds: {@value #DEFAULT_TIMEOUT_MS}.
 *
 * <p>Use {@code Duration.ofMillis(timeout)} to convert to a {@link
↪ java.time.Duration}
 * if you need a {@link java.time.Duration}-based API instead.
 *
 * <p>Generic type bound example (needs {@literal @literal}, not {@code
↪ @code},
 * because {@code List<String>} would be parsed as an HTML tag otherwise):
 * a raw {@literal List<String>} is unsafe.
 */
public static final int DEFAULT_TIMEOUT_MS = 5000;
```

```
/**
 * {@inheritDoc}
 *
 * <p>This implementation additionally logs every call at DEBUG level.
 */
@Override
public void process(Order order) {
    ...
}
```

The `{@snippet}` tag (JEP 413, finalized JDK 18) replaces the old `<pre>{@code ...}</pre>` idiom with something the javadoc tool can actually validate and highlight:


```

/**
 * Builds an immutable list.
 *
 * {@snippet lang = "java" :
 * List<String> names = List.of("Ada", "Grace", "Linus");
 * names.forEach(System.out::println);
 * }
 */
public static <T> List<T> immutableListOf(T... items) { ... }

```

The First-Sentence Summary Rule

The **first sentence** of a Javadoc comment (up to the first period followed by whitespace or a line break, in the classic algorithm — recent javadoc also respects an explicit `{@summary ... }` tag) becomes the **summary**: what shows up in package/class overview tables, method-summary tables, and search results. This means:

- Front-load the single most important fact about the method in that first sentence — assume it's the *only* sentence some readers will ever see.
- Avoid a stray period inside the first sentence (e.g., in “e.g.” or a versioned class name) — it truncates the summary at the wrong place.

```

/**
 * Returns the customer's preferred shipping address, or the billing
 * address if none is set. Falls back further to {@code null} if the
 * customer record itself has no addresses at all.
 */
public Address preferredShippingAddress() { ... }

```

```

// Bad: the period after "e.g." truncates the rendered summary to
// "Formats a duration, e." — nonsensical in the summary table.
/**
 * Formats a duration, e.g. "3h 12m", using the default locale.
 */

```

Documenting the Contract, Not the Implementation

The most important review distinction in this chapter: Javadoc documents the **contract** — what callers can rely on — not *how* the current implementation happens to work.

Implementation details belong in inline `//` comments inside the method body, if anywhere, because they can change without breaking any caller; contract statements cannot.

```
// Bad: describes the current implementation, which callers should never
↪ depend on
/**
 * Looks up the user by scanning a HashMap keyed by user ID.
 */
public User findById(long id) { ... }

// Good: describes the contract – inputs, outputs, and failure behavior
/**
 * Returns the user with the given ID.
 *
 * @param id the user ID; must be positive
 * @return the matching user
 * @throws NoSuchElementException if no user with this ID exists
 */
public User findById(long id) { ... }
```

If the class later switches from a `HashMap` to a database call, the first comment becomes a lie that nobody remembers to fix; the second comment remains true regardless of the storage mechanism.

Documenting Nullability, Thread Safety, and Side Effects

These three properties are exactly the ones that cannot be inferred from a method signature alone in Java (pre-nullability-annotations, and even with them, the *reasoning* still benefits from prose), and they are exactly what reviewers look for missing:

```
/**
 * Finds a cached session for the given token.
 *
 * <p><b>Nullability:</b> returns {@code null} if no session is cached
 * for the token; never throws for an unknown token.
 *
 * <p><b>Thread safety:</b> this method is safe to call concurrently
 * from multiple threads without external synchronization.
 *
 * <p><b>Side effects:</b> refreshes the token's last-access timestamp
 * as a side effect of a successful lookup, which resets its expiry.
 *
 * @param token the session token
 * @return the cached {@link Session}, or {@code null} if not found
```

```
*/
public Session findSession(String token) { ... }
```

Where the project uses JSR-305 / `org.jspecify` annotations (`Nullable`, `NonNull`), the annotation is machine-checkable and should be preferred over prose alone — but prose explaining *why* something can be null (e.g., “null before the account is activated”) still adds value an annotation cannot.

package-info.java and module-info.java Docs

Package-level and module-level Javadoc live in dedicated files, not attached to any class, because a package/module is not itself a class.

```
// package-info.java
/**
 * Order pricing and discount calculation.
 *
 * <p>Classes in this package are responsible for computing the final
 * price of an {@link com.example.orders.model.Order}, including tax and
 * promotional discounts. Nothing in this package performs persistence
 * or network I/O.
 *
 * @since 1.0
 */
package com.example.orders.pricing;
```

```
// module-info.java
/**
 * Defines the order-processing service and its public API.
 *
 * <p>Only {@code com.example.orders.api} is exported; all other packages
 * are internal implementation details and not accessible to other
 * modules.
 */
module com.example.orders {
    requires java.sql;
    exports com.example.orders.api;
}
```

HTML in Javadoc and Markdown Javadoc (JEP 467, JDK 23+)

Classic Javadoc comments are parsed as HTML fragments, so structuring text requires literal HTML tags — `<p>` for paragraphs, `<ul>/<li>` for lists, `<b>` for bold. This is verbose and easy to get wrong (unclosed tags silently break rendering, or worse, get swallowed by `-Xdoclint`, discussed next).

```
/**
 * Represents an order line item.
 *
 * <p>Each line item has:
 * <ul>
 *   <li>a {@code productId}</li>
 *   <li>a {@code quantity}, always positive</li>
 *   <li>a {@code unitPrice}</li>
 * </ul>
 */
public class LineItem { ... }
```

JEP 467 (finalized in JDK 23) allows Javadoc comments to be written in **Markdown** instead, using `///` line comments rather than `/** */` blocks:

```
/// Represents an order line item.
///
/// Each line item has:
/// - a `productId`
/// - a `quantity`, always positive
/// - a `unitPrice`
///
/// See [LineItem#total()] for how the extended price is computed.
public class LineItem {
    /// Returns the extended price: `unitPrice * quantity`.
    public BigDecimal total() { ... }
}
```

Markdown Javadoc is substantially easier to read in raw source form (in an IDE or a diff) and eliminates most of the HTML-escaping foot-guns, at the cost of requiring JDK 23+ tooling to render.

Generating Javadoc

```
# Directly with the javadoc tool
javadoc -d docs -sourcepath src/main/java -subpackages com.example.orders

# Maven
mvn javadoc:javadoc          # generates target/site/apidocs
mvn javadoc:jar              # packages it as a javadoc.jar (needed to
→ publish to Maven Central)

# Gradle
./gradlew javadoc            # generates build/docs/javadoc
```

```
<!-- pom.xml: enforce doc quality and produce a javadoc jar for publishing
→ -->
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-javadoc-plugin</artifactId>
  <version>3.8.0</version>
  <configuration>
    <doclint>all,-missing</doclint> <!-- validate HTML/tags; don't
→ require every element documented -->
    <failOnWarning>true</failOnWarning>
  </configuration>
  <executions>
    <execution>
      <id>attach-javadocs</id>
      <goals><goal>jar</goal></goals>
    </execution>
  </executions>
</plugin>
```

```
// build.gradle.kts
tasks.javadoc {
  (options as StandardJavadocDocletOptions).apply {
    addStringOption("Xdoclint:all,-missing", "-quiet")
  }
}
```

-Xdoclint and Treating Doc Warnings as Errors

-Xdoclint is the javadoc tool's built-in linter: it flags malformed HTML, {@link} references to symbols that don't exist, missing @param/@return/@throws tags, and

other structural problems, without needing a separate static-analysis tool. Since JDK 8, `javac` itself also runs a subset of doclint checks and can emit warnings during **compilation**, not just during Javadoc generation.

```
javadoc -Xdoclint:all -d docs -sourcepath src/main/java com.example.orders
javac -Xdoclint:all -d out src/main/java/com/example/orders/*.java
```

Common `-Xdoclint` groups you can enable/disable independently: `accessibility`, `html`, `missing`, `reference`, `syntax`. A pragmatic default that reviewers commonly ask for is `all`, `-missing` — validate everything that’s present, but don’t force every single public element to have a comment (that requirement tends to produce exactly the restating-the-name comments discussed below).

Treating doc warnings as build errors (`failOnWarning` in Maven, or `-Werror`-equivalent doclint options in Gradle) prevents doc rot: a broken `{@link}` to a renamed method, or a `<p>` tag that was never closed, stays broken forever unless something makes the build fail on it.

Publishing Javadoc

For a library, Javadoc is typically published alongside the release artifact as a `-javadoc.jar` classifier (required by Maven Central), and/or as a static HTML site (e.g., via GitHub Pages) generated by CI on release. `javadoc.io` can render any artifact already on Maven Central without any extra publishing step, by resolving the `-javadoc.jar` classifier directly.

```
mvn javadoc:jar deploy    # produces and deploys my-lib-1.4.0-javadoc.jar
↪ alongside the main jar
```

When a Javadoc Comment Is Worse Than No Comment

A Javadoc comment that merely restates the method/parameter name in English adds visual noise, gives the false impression that the method is documented, and — worst of all — will drift out of sync with the real behavior over time because nobody feels the need to update “obvious” prose. It is strictly worse than no comment, because a missing comment at least invites the reader to go read the code.

```
// Bad: pure noise, restates the signature
/**
 * Gets the name.
```

```

    * @return the name
    */
    public String getName() { return name; }

    // Bad: restates behavior visible from the method body one line below,
    // adds nothing about the contract (can it return null? is it idempotent?)
    /**
     * Sets the value.
     * @param value the value
     */
    public void setValue(int value) { this.value = value; }

```

```

    // Good: worth documenting, because the contract isn't obvious from the
    ↪ signature
    /**
     * Returns the display name shown in the UI, falling back to the account's
     * email address if the user has never set one.
     *
     * @return the display name; never {@code null}
     */
    public String getName() { return name != null ? name : email; }

```

A simple review heuristic: if you deleted the Javadoc comment and could still write the exact same sentence just by reading the method name and signature, the comment is not documenting anything and should either be deleted or expanded to cover the actual contract (nullability, exceptions, thread-safety, or a non-obvious side effect).

Common Code-Review Interview Pitfalls

1. **Committing target/ or build/ output directories to version control.** Why it matters: generated artifacts bloat the repository, go stale relative to source, and cause merge noise; they should always be build-tool-generated, never hand-edited or tracked.

```

# .gitignore
target/
build/

```

2. **Not committing the Maven/Gradle Wrapper (mvnw/gradlew).** Why it matters: without a pinned wrapper, “works on my machine” becomes literal — a teammate or CI runner with a different globally installed build-tool version can produce a different, sometimes-broken build.

```
mvn wrapper:wrapper -Dmaven=3.9.6 # generate and commit mvnw,
↪ mvnw.cmd, .mvn/
```

- 3. Using a version range ([1.0,2.0)) instead of a pinned version for a dependency.** Why it matters: the resolved version can silently change between builds as new releases are published, breaking reproducibility — the exact opposite of what a lockfile-free ecosystem needs to stay predictable.

```
<!-- Before -->
<version>[1.0,2.0)</version>
<!-- After -->
<version>1.4.2</version>
```

- 4. Declaring a dependency as compile/api when it's only used internally.** Why it matters: it leaks the dependency onto every consumer's compile classpath, forcing consumers to recompile when an internal-only dependency changes version, and can leak internal types through the public API by accident.

```
// Before
api("org.apache.commons:commons-lang3:3.14.0")
// After – used only inside this module's private helpers
implementation("org.apache.commons:commons-lang3:3.14.0")
```

- 5. Using -Dmaven.test.skip=true in CI instead of -DskipTests (or, worse, skipping tests routinely at all).** Why it matters: maven.test.skip doesn't even compile the test sources, so a broken test file can merge without anyone noticing until someone runs tests locally — a much worse failure mode than merely not running passing tests.

```
# Before (hides compile errors in tests)
mvn install -Dmaven.test.skip=true
# After (at least verifies tests compile)
mvn install -DskipTests
```

- 6. Relying on Maven's nearest-wins transitive resolution without checking dependency:tree.** Why it matters: nearest-wins picks a version based on graph depth, not correctness — it can silently select an old or vulnerable version of a transitive dependency, and the only way to know is to actually look.

```
mvn dependency:tree -Dverbose
```


- 7. Writing a Javadoc comment that restates the method name instead of documenting the contract.** Why it matters: it looks like documentation without providing any information beyond the signature, and gives false confidence that behavior (nullability, exceptions, thread-safety) is documented when it isn't.

```
// Before
/** Gets the name. @return the name */
public String getName() { ... }
// After
/** Returns the display name, or the email address if none was set. */
public String getName() { ... }
```

- 8. Javadoc describing the current implementation instead of the caller-visible contract.** Why it matters: implementation-describing comments become lies the moment the implementation changes (e.g., swapping a HashMap for a database call), and nobody remembers to update prose that isn't checked by the compiler.

```
// Before
/** Looks up the user by scanning an in-memory HashMap. */
// After
/** Returns the user with the given ID. @throws NoSuchElementException
↳ if not found */
```

- 9. A broken {@link} reference to a renamed or deleted method/class.** Why it matters: it silently degrades documentation quality (a dead link in generated docs) and usually indicates the comment wasn't updated alongside a refactor; `-Xdoclint:reference` catches this automatically if enabled and enforced in CI.

```
javadoc -Xdoclint:reference -d docs -sourcepath src/main/java
↳ com.example.orders
```

- 10. Mixing up provided/compileOnly with runtime/runtimeOnly, causing a NoClassDefFoundError at deploy time or a bloated fat jar.** Why it matters: provided/compileOnly dependencies (e.g., servlet API, Lombok) are intentionally excluded from the packaged artifact because the runtime environment supplies them — packaging them anyway (wrong scope) can cause classloading conflicts; the reverse mistake (using provided for something actually needed at runtime) causes a missing-class crash in production.

```

    <!-- Before: servlet-api is needed only to compile against, container
    ↪ supplies it -->
    <scope>compile</scope>
    <!-- After -->
    <scope>provided</scope>

```

- 11. Using maven-assembly-plugin (or an unconfigured fat jar) for a Spring Boot / service-loader-heavy app instead of maven-shade-plugin with transformers.** Why it matters: assembly's naive concatenation can silently drop or corrupt merged META-INF/services files (used by ServiceLoader, JDBC drivers, logging back-ends), producing runtime failures that only show up when the fat jar is actually run, not when it's built.

```

    <transformer implementation=
    ↪ "org.apache.maven.plugins.shade.resource.ServicesResourceTransformer"/>

```

- 12. A custom Gradle task using doLast {} with no declared inputs/outputs, breaking incremental build and the build cache.** Why it matters: Gradle can only skip a task as UP-TO-DATE or restore it from cache if it knows what the task's inputs and outputs are; an undeclared doLast closure always reruns, silently making every build slower than it needs to be.

```

// Before: always reruns, never cacheable
tasks.register("generateDocs") { doLast { /* ... */ } }
// After: typed task with declared @InputFiles/@OutputDirectory
tasks.register<GenerateDocsTask>("generateDocs")

```

- 13. Manually keeping dependency versions in sync across Maven modules or Gradle subprojects instead of using a BOM / version catalog.** Why it matters: without a single source of truth for versions, modules drift apart over time, producing exactly the kind of version conflicts dependency:tree/dependencyInsight are needed to untangle later — the bug is prevented far more cheaply than it's debugged.

```

# gradle/libs.versions.toml — one place to bump a version for every
↪ subproject
[versions]
jackson = "2.17.1"

```

- 14. Declaring <release> inconsistently across modules (or using the old source/target pair instead of release).** Why it matters: source/target alone can compile

against a *newer* JDK's bootclasspath while claiming an older language level, letting code accidentally reference APIs unavailable on the actual target runtime; `<release>` locks both together and fails the build if that happens.

```
<!-- Before -->
<source>17</source>
<target>17</target>
<!-- After -->
<release>21</release>
```

- 15. Suppressing all Javadoc/doclint warnings globally instead of fixing or narrowly excusing them.** Why it matters: a blanket `-Xdoclint:none` (or `doclint=none`) hides genuinely broken documentation (bad HTML, dead `{@link}`s, missing `@throws` for a checked exception) right alongside harmless style nitpicks, defeating the entire purpose of running the linter in the first place.

```
<!-- Before: disables everything -->
<doclint>none</doclint>
<!-- After: validates structure/links, only relaxes the "every element
↳ must have a comment" rule -->
<doclint>all,-missing</doclint>
```


Code-Review Interview Focused Topics

A code-review interview hands you a snippet — sometimes a whole small class, sometimes a diff — and asks: “what’s wrong with this?” It is a different skill from solving an algorithm puzzle on a whiteboard. Nobody is grading whether you can invent clever code; they are grading whether you can **read someone else’s code critically**, recognize the same handful of mistakes that break production systems every day, and talk about them the way a helpful senior engineer would in a real pull request.

Interviewers (and good reviewers in general) judge findings against a rough priority order. Learn this order and use it to decide what to say first:

1. **Correctness** — does the code do what it claims? Off-by-one errors, wrong boolean logic, unhandled edge cases, silent data corruption.
2. **Safety** — will it corrupt shared state, leak a resource, deadlock, or blow up under concurrent/production load, even if the “happy path” looks fine on a single thread?
3. **Design** — is it maintainable and testable? Right abstraction, right exception type, right collection, sensible API shape?
4. **Performance** — is it needlessly slow or memory-hungry, once correctness and safety are already settled?
5. **Style** — naming, formatting, and other things a linter could catch.

If you spend the first three minutes discussing a variable name (style) while a `HashMap` is being mutated from two threads two lines above it (safety), you have failed the interview even if every word about naming was correct. Everything in this chapter is organized around that ordering — find the highest-severity issue first, say it first, and only get to nits if there’s time left.

This chapter assumes you have already read the earlier chapters on the underlying mechanics — it does not re-derive *why* `HashMap` isn't thread-safe or *why* checked exceptions exist. It focuses purely on the reviewer's lens: what to look for, how to phrase it, and worked examples.

The Review Checklist

Run through this whenever you're handed a snippet, top to bottom, in this order. It mirrors the correctness → safety → design → performance → style priority from the intro.

Correctness

Does every branch (including the ones with no `else`) do the right thing? What happens on empty input, `null`, zero, negative numbers, max values?

Are loop bounds right? Off-by-one on `<` vs `<=`, first/last index, empty collections.

Is `==` used where `.equals()` was needed (boxed types, `String`, custom objects)?

Does the method actually do what its name and signature promise?

Are there unhandled edge cases the tests (if any) don't cover?

Thread Safety & Concurrency

Is any state **shared** across threads (static fields, singletons, fields on a class handed to an executor)?

If shared and mutable — is access synchronized, or is it a concurrent-safe type (`ConcurrentHashMap`, `AtomicInteger`, `volatile`, immutable value)?

Does construction fully finish before the object is published to another thread (no leaking `this`)?

Are there lazy-initialized singletons, caches, or `SimpleDateFormat`-style non-thread-safe fields stored `static`?

Is every `ExecutorService`/thread pool shut down somewhere? Is every lock released on all paths, including exceptions?

Any risk of deadlock from inconsistent lock ordering, or of double-checked locking without `volatile`?

Resource & Exception Management

Is every `Closeable`/`AutoCloseable` (stream, connection, statement, lock) closed on **every** path, including exceptions? Prefer try-with-resources — see [Chapter 6, *Exception Handling*](#).

Is any catch block empty, or does it just log-and-swallow without rethrowing or handling?

Is the catch scope too wide — wrapping code that can throw for unrelated reasons?

Is the original cause preserved when wrapping an exception (`new Foo("msg", e)`, not `new Foo("msg")`)?

Are checked vs. unchecked exceptions used appropriately at the API boundary?

Collections & Data Structures

Is the chosen collection right for the access pattern (lookup-heavy vs. order-sensitive vs. dedupe vs. FIFO)? See [Chapter 7, *Collections Framework*](#).

Are keys used in a `HashMap`/`HashSet` immutable, with a consistent `equals`/`hashCode` pair?

Any `equals()` override missing a matching `hashCode()` override?

Any unbounded growth — cache, list, map that is only ever added to and never trimmed or evicted?

Any `computeIfAbsent`/`merge` call whose lambda has side effects or re-enters the same map?

API Design

Can any public parameter or return value legally be `null`? Is that documented, or should it be `Optional`?

Are parameters validated at the boundary (fail fast) instead of failing deep inside with a confusing error?

Is the class needlessly mutable when it could be immutable?

Do overloads behave consistently, and is the naming unambiguous about side effects (`getX` vs `computeX`)?

Would this change break existing callers (removed method, changed semantics, widened exceptions)?

Performance & Memory

Any $O(n^2)$ work disguised as “just a loop inside a loop” (nested lookups, repeated linear scans)?

Any string concatenation with `+` inside a loop instead of `StringBuilder`?

Any object allocation, regex compilation, or reflection happening inside a hot loop that could be hoisted out?

Is autoboxing happening in a tight numeric loop (`Integer` instead of `int`, boxed streams)?

Any premature synchronized/parallelism added without evidence it’s needed, at the cost of readability?

Style (only after everything above is clean)

Naming clear and consistent with the domain?

Method length / single responsibility reasonable?

Are modern idioms used well (records, `var`, streams, switch expressions) without overuse — see [Modern Java Feature Usage](#)?

How to Communicate in a Code-Review Interview

Finding the bug is half the exercise. How you talk about it is the other half — interviewers are evaluating whether you’d be a good teammate in an actual review, not just a bug detector.

- **Lead with the highest-severity issue.** Don’t narrate top-to-bottom in file order; scan the whole snippet first, rank what you found, and open with the worst one. “Before anything else — this singleton isn’t thread-safe” beats starting with a variable-naming nit.
- **Say why, not just what.** “This will break” is a claim; “this will break *because* two threads can both pass the null-check before either assigns the instance, so you can get two singletons” is a review. Explaining the mechanism is what proves you understand it rather than pattern-matching a lint rule.
- **Propose a concrete fix.** Don’t stop at “this is unsafe.” Say what you’d change it to — an initialization-on-demand holder, `ConcurrentHashMap`, a try-with-resources block — ideally in a sentence or two of code, not just an adjective.

- **Separate blocking issues from nits.** Explicitly label things: “this one’s a blocker, it will corrupt data in production” vs. “this is a nit, purely a naming preference — not worth blocking on.” Interviewers want to see you can triage, not just that you can spot fifteen things.
- **Ask questions instead of accusing.** “Was this meant to run in a single-threaded context, or could multiple requests hit it concurrently?” reads as collaborative. “You clearly didn’t think about thread safety” reads as hostile — and might be wrong if there’s context you don’t have (maybe it really is confined to one thread by contract).
- **Say what’s good, too.** If the error handling is solid or the naming is clear, say so briefly. It shows judgment (you’re not fault-finding for its own sake) and it’s simply what a real review looks like — real PRs aren’t 100% criticism.

A reasonable spoken structure: *“The biggest problem is X, because Y — I’d fix it by Z. There’s also a smaller issue with A. Everything else — B, C — are just nits, not blockers. One thing I like here is D.”*

Common Java Interview Pitfalls

These are the recurring, easy-to-miss patterns interviewers plant because they show up constantly in real codebases. Recognize the *shape* of each one instantly — you shouldn’t need to trace through logic to spot them.

1. **== on boxed types or String.** `Integer a = 1000, b = 1000; a == b` is `false` outside the `-128..127` cache range. Always `.equals()` for objects.
2. **Autoboxing NPE.** `Integer count = map.get(key); if (count > 0)` throws `NullPointerException` when `count` is `null` — the unboxing happens invisibly at `>`.
3. **Mutable static fields shared across requests/threads** — the classic `SimpleDateFormat`, lazily-initialized singleton, or cache-as-`HashMap` pattern (see Scenarios below).
4. **`equals()` without `hashCode()`**, or vice versa — breaks every hash-based collection silently (no compiler error, just wrong behavior).
5. **Using a mutable object as a `Map/Set` key**, then mutating it after insertion — the entry becomes unreachable by lookup.
6. **String concatenation with `+` in a loop** — quadratic behavior because each `+` builds a new `String`.

7. **Catching Exception (or Throwable) far wider than needed**, and/or doing nothing with it — see [Chapter 6, Exception Handling](#) for the full treatment.
8. **finally with a return/throw** that silently discards the exception that was in flight.
9. **Off-by-one and empty-collection bugs** — `for (int i = 0; i <= list.size(); i++)`, or forgetting a collection can be empty before calling `.get(0)`.
10. **Integer overflow from int arithmetic** — `int total = a * b`; overflows silently where `long` wouldn't; no exception, just a wrong answer.
11. **Comparing floating point with ==** instead of an epsilon, or using `double/float` for money instead of `BigDecimal`.
12. **Modifying a collection while iterating it** with a plain `for-each`, throwing `ConcurrentModificationException` — should use an `Iterator.remove()`, `removeIf`, or a copy.
13. **Ignoring the return value of an immutable operation** — `str.trim()`; on its own line does nothing, because `String` is immutable and `trim()` returns a new value.
14. **Leaking this from a constructor** — starting a thread, registering a listener, or passing `this` to another object before the constructor finishes (see Scenario 15).
15. **Unbounded recursion or unbounded collection growth** with no base case / no eviction, eventually a `StackOverflowError` or `OutOfMemoryError` under load rather than at compile time.
16. **Checked exception swallowed and converted to a generic, cause-less RuntimeException**, losing the original stack trace.
17. **Using Optional as a field type, method parameter, or inside a collection** instead of only as a return type — it's designed for one purpose, and misusing it is itself a common interview trap (see [API Design Review](#)).
18. **Relying on HashMap/HashSet iteration order**, which is unspecified and can silently change between JDK versions or after resizing.

These patterns recur throughout the scenarios below — as you read each one, try to name which pitfall(s) from this list it is before reading the hidden answer.

Code Review Scenarios

Each scenario is realistic production-style code with a real, plantable bug (sometimes more than one). Read the snippet, form your own opinion, rank the issues by severity, then expand the review to check yourself.

Scenario 1: The Configuration Singleton

```
public class ConfigManager {
    private static ConfigManager instance;
    private final Map<String, String> settings = new HashMap<>();

    private ConfigManager() {
        settings.put("timeout", "30");
        settings.put("retries", "3");
    }

    public static ConfigManager getInstance() {
        if (instance == null) {
            instance = new ConfigManager();
        }
        return instance;
    }

    public String get(String key) {
        return settings.get(key);
    }

    public void set(String key, String value) {
        settings.put(key, value);
    }
}
```

REVIEW

- **Blocker — non-thread-safe lazy singleton.** `getInstance()` has a classic race: two threads can both see `instance == null`, and both construct a `ConfigManager`, handing out two different instances to different callers. This is the textbook double-checked-locking setup, minus the locking. Under low concurrency it “works” in testing and then fails intermittently in production — the worst kind of bug to debug.

- **Major** — **settings map is a mutable HashMap with no synchronization**, and `set()/get()` can be called concurrently once multiple threads hold references (even to the same instance). Concurrent writes to a plain HashMap can corrupt its internal structure, not just race on values.
- **Minor** — **no way to remove/reset settings** for tests, which tends to leak state between test cases sharing the singleton. Not a functional bug, but worth flagging as a design smell for testability.

Fixed code — initialization-on-demand holder for lazy, thread-safe singleton construction, plus a concurrent-safe backing map:

```
public class ConfigManager {
    private final Map<String, String> settings = new
↳ ConcurrentHashMap<>();

    private ConfigManager() {
        settings.put("timeout", "30");
        settings.put("retries", "3");
    }

    private static final class Holder {
        static final ConfigManager INSTANCE = new ConfigManager();
    }

    public static ConfigManager getInstance() {
        return Holder.INSTANCE; // JVM class-loading guarantees are the
↳ "locking"
    }

    public String get(String key) {
        return settings.get(key);
    }

    public void set(String key, String value) {
        settings.put(key, value);
    }
}
```

The holder idiom is thread-safe without explicit locking because the JVM guarantees a class is initialized exactly once, under a lock, the first time it's actively used — see [Chapter 13, Concurrency \(Core\)](#) for the Java Memory Model guarantees behind this.

Scenario 2: The Request Counter

```
public class RequestCounterService {
    private final Map<String, Integer> countsByEndpoint = new HashMap<>();

    // Called concurrently from many servlet request threads
    public void recordRequest(String endpoint) {
        Integer current = countsByEndpoint.get(endpoint);
        if (current == null) {
            countsByEndpoint.put(endpoint, 1);
        } else {
            countsByEndpoint.put(endpoint, current + 1);
        }
    }

    public int getCount(String endpoint) {
        return countsByEndpoint.getOrDefault(endpoint, 0);
    }

    public Map<String, Integer> snapshot() {
        return countsByEndpoint;
    }
}
```

REVIEW

- **Blocker** — **plain HashMap mutated from multiple threads.** The comment even says “called concurrently,” so this isn’t a hypothetical: concurrent `put()` calls on a `HashMap` can corrupt internal bucket structure (infinite loops during resize in older JDKs, or silently dropped entries), not just lose a counter increment.
- **Major** — **read-then-write is not atomic** even with a concurrent map: `get` then `put(current + 1)` is a classic lost-update race — two threads can both read the same `current` and both write `current + 1`, losing one increment. Switching the map type alone does not fix this.
- **Minor** — **`snapshot()` returns the live internal map**, not a copy, so callers can mutate internal state, and any iteration over it concurrently with a write is still unsafe even with `ConcurrentHashMap` (iteration is weakly consistent, which is fine for a snapshot but callers should not assume they can safely mutate what they get back).

Fixed code — `ConcurrentHashMap` plus an atomic update method (`merge`, or an `AtomicInteger` per key) instead of `get-then-put`, and a defensive copy for the snapshot:

```
public class RequestCounterService {
    private final Map<String, AtomicInteger> countsByEndpoint = new
↳ ConcurrentHashMap<>();

    public void recordRequest(String endpoint) {
        countsByEndpoint.computeIfAbsent(endpoint, e -> new
↳ AtomicInteger())
                            .incrementAndGet();
    }

    public int getCount(String endpoint) {
        AtomicInteger count = countsByEndpoint.get(endpoint);
        return count == null ? 0 : count.get();
    }

    public Map<String, Integer> snapshot() {
        Map<String, Integer> copy = new HashMap<>();
        countsByEndpoint.forEach((k, v) -> copy.put(k, v.get()));
        return copy;
    }
}
```

Scenario 3: The Order Deduplicator

```
public class OrderKey {
    private String customerId;
    private LocalDate orderDate;

    public OrderKey(String customerId, LocalDate orderDate) {
        this.customerId = customerId;
        this.orderDate = orderDate;
    }

    public void setOrderDate(LocalDate orderDate) {
        this.orderDate = orderDate;
    }

    @Override
    public boolean equals(Object o) {
        if (!(o instanceof OrderKey other)) return false;
        return customerId.equals(other.customerId) &&
↳ orderDate.equals(other.orderDate);
    }
}
```

```

@Override
public int hashCode() {
    return Objects.hash(customerId, orderDate);
}

public class OrderDeduplicator {
    private final Map<OrderKey, Order> seen = new HashMap<>();

    public boolean isDuplicate(OrderKey key, Order order, LocalDate
→ correctedDate) {
        key.setOrderDate(correctedDate); // apply a late correction before
→ checking
        boolean duplicate = seen.containsKey(key);
        seen.put(key, order);
        return duplicate;
    }
}

```

REVIEW

- **Blocker** — **mutable object used as a hash-based map key**. `OrderKey` has a setter (`setOrderDate`) and participates in `hashCode()`. Mutating a key after it's already stored in a `HashMap` changes its hash code, so the entry ends up in the wrong bucket — it becomes permanently unreachable via `get/containsKey` even though it's still sitting in the map, silently leaking memory *and* breaking the dedup logic it exists to implement.
- **Major** — **the mutation happens right before the lookup**, so even a *new* key that's mutated before first insertion could match against a stale bucket layout if the same key object is reused/cached elsewhere. Any key type used in a hash-based collection must be effectively immutable for its whole lifetime.
- **Minor** — **`OrderKey` fields aren't `final`**, which invites exactly this kind of mistake in the future even if today's only caller happens to use it “safely” for a single mutation before insertion.

Fixed code — make `OrderKey` immutable; if a corrected date is needed, build a new key instead of mutating the old one:

```

public final class OrderKey {
    private final String customerId;
    private final LocalDate orderDate;
}

```

```

    public OrderKey(String customerId, LocalDate orderDate) {
        this.customerId = Objects.requireNonNull(customerId);
        this.orderDate = Objects.requireNonNull(orderDate);
    }

    public OrderKey withDate(LocalDate correctedDate) {
        return new OrderKey(customerId, correctedDate); // new key, old
↪ one untouched
    }

    @Override
    public boolean equals(Object o) {
        if (!(o instanceof OrderKey other)) return false;
        return customerId.equals(other.customerId) &&
↪ orderDate.equals(other.orderDate);
    }

    @Override
    public int hashCode() {
        return Objects.hash(customerId, orderDate);
    }
}

public class OrderDeduplicator {
    private final Map<OrderKey, Order> seen = new HashMap<>();

    public boolean isDuplicate(OrderKey key, Order order, LocalDate
↪ correctedDate) {
        OrderKey correctedKey = key.withDate(correctedDate);
        boolean duplicate = seen.containsKey(correctedKey);
        seen.put(correctedKey, order);
        return duplicate;
    }
}

```

A record ([Chapter 3, *Modern Java Language Features*](#)) would have made this mistake impossible by construction, since record components can't have setters.

Scenario 4: The Report Exporter

```
public class ReportExporter {

    public String loadTemplate(String templatePath) throws IOException {
        FileInputStream fis = new FileInputStream(templatePath);
        byte[] data = fis.readAllBytes();
        return new String(data, StandardCharsets.UTF_8);
    }

    public void exportToFile(String content, String outputPath) throws
    ↪ IOException {
        FileOutputStream fos = new FileOutputStream(outputPath);
        Connection conn = dataSource.getConnection();
        PreparedStatement stmt = conn.prepareStatement("INSERT INTO
    ↪ exports(path) VALUES (?)");
        stmt.setString(1, outputPath);
        stmt.executeUpdate();
        fos.write(content.getBytes(StandardCharsets.UTF_8));
        fos.close();
    }

    private DataSource dataSource;
}
```

REVIEW

- **Blocker** — **loadTemplate never closes fis**. Every call leaks a file descriptor; under load this exhausts the process's file descriptor limit and every subsequent I/O call in the JVM starts failing, not just this method.
- **Blocker** — **exportToFile leaks conn and stmt unconditionally**, and additionally leaks fos if stmt.executeUpdate() throws (e.g., a SQL error), because fos.close() is only reached on the successful path. Database connections are a scarcer, more expensive resource than file handles — this will exhaust the connection pool quickly.
- **Major** — **mixing an I/O write and a DB write with no transactional/ordering thought**. If the DB insert succeeds but the file write fails (or vice versa), the export record and the exported file disappear out of sync with no rollback or compensation. Not the focus of this exercise, but worth a one-line flag.

Fixed code — try-with-resources for every AutoCloseable, guaranteeing cleanup on every path including exceptions (see [Chapter 6, Exception Handling](#)):

```

public class ReportExporter {

    public String loadTemplate(String templatePath) throws IOException {
        try (FileInputStream fis = new FileInputStream(templatePath)) {
            return new String(fis.readAllBytes(), StandardCharsets.UTF_8);
        }
    }

    public void exportToFile(String content, String outputPath) throws
    ↳ IOException, SQLException {
        try (Connection conn = dataSource.getConnection();
            PreparedStatement stmt = conn.prepareStatement("INSERT INTO
    ↳ exports(path) VALUES (?)")) {
            stmt.setString(1, outputPath);
            stmt.executeUpdate();
        }
        try (FileOutputStream fos = new FileOutputStream(outputPath)) {
            fos.write(content.getBytes(StandardCharsets.UTF_8));
        }
    }

    private DataSource dataSource;
}

```

Scenario 5: The Payment Processor

```

public class PaymentProcessor {

    public PaymentResult process(PaymentRequest request) {
        try {
            validate(request);
            ChargeResult charge = gateway.charge(request.amount(),
    ↳ request.cardToken());
            ledger.record(request.orderId(), charge.transactionId());
            return PaymentResult.success(charge.transactionId());
        } catch (Exception e) {
            return PaymentResult.failure("Payment could not be
    ↳ processed");
        }
    }

    private void validate(PaymentRequest request) {
        if (request.amount().compareTo(BigDecimal.ZERO) <= 0) {
            throw new IllegalArgumentException("amount must be positive");
        }
    }
}

```

```

        }
    }
}

```

REVIEW

- **Blocker — the exception is swallowed with no logging and no rethrow.** `catch (Exception e) { return PaymentResult.failure(...); }` discards every detail about *why* a payment failed — network timeout, declined card, a `NullPointerException` bug in `validate`, or the ledger write failing after the charge already succeeded. Support will get “payment could not be processed” tickets with zero ability to diagnose them, and real bugs will hide behind the same generic message as legitimate declines.
- **Blocker — the charge can succeed but the ledger write can fail**, and both end up reported identically as a generic failure to the caller. That’s a customer charged with no record of it — a money-losing (or trust-losing) bug, not just a logging annoyance. This needs to be handled as a distinct case (compensating action, alerting, retry-safe ledger write), not lumped into the same catch.
- **Major — catch (Exception e) is too wide.** It will also catch bugs (`NullPointerException`, `ClassCastException`) from anywhere in this call chain and report them as ordinary payment failures instead of surfacing them as defects to be fixed.

Fixed code — narrow the catch to expected failure modes, log with full context, and treat “charged but not recorded” as its own alertable case:

```

public class PaymentProcessor {
    private static final Logger log =
↳ LoggerFactory.getLogger(PaymentProcessor.class);

    public PaymentResult process(PaymentRequest request) {
        validate(request); // let IllegalArgumentException propagate —
↳ it's a caller bug, fail fast

        ChargeResult charge;
        try {
            charge = gateway.charge(request.amount(),
↳ request.cardToken());
        } catch (GatewayException e) {
            log.warn("Charge declined for order {}: {}",
↳ request.orderId(), e.getMessage());

```

```

        return PaymentResult.failure("Payment declined: " +
↳ e.getMessage());
    }

    try {
        ledger.record(request.orderId(), charge.transactionId());
    } catch (LedgerException e) {
        // Money moved but we failed to record it – this must page
↳ someone, not just log.
        log.error("CRITICAL: charge {} succeeded but ledger record
↳ failed for order {}",
            charge.transactionId(), request.orderId(), e);
        alerting.pageOnCall("Unreconciled charge " +
↳ charge.transactionId());
        throw new UnreconciledChargeException(request.orderId(),
↳ charge.transactionId(), e);
    }

    return PaymentResult.success(charge.transactionId());
}

private void validate(PaymentRequest request) {
    if (request.amount().compareTo(BigDecimal.ZERO) <= 0) {
        throw new IllegalArgumentException("amount must be positive");
    }
}
}

```

Scenario 6: The Order Summary Builder

```

public class OrderSummaryBuilder {

    public List<OrderSummary> buildSummaries(List<Order> orders) {
        List<OrderSummary> summaries = new ArrayList<>();
        for (Order order : orders) {
            Customer customer =
↳ customerRepository.findById(order.customerId());
            List<LineItem> items =
↳ lineItemRepository.findById(order.id());
            BigDecimal total = BigDecimal.ZERO;
            for (LineItem item : items) {
                for (LineItem discountRule : discountRules) {
                    if (discountRule.appliesTo(item)) {
                        item = discountRule.apply(item);
                    }
                }
            }
            OrderSummary summary = new OrderSummary(customer, items, total);
            summaries.add(summary);
        }
    }
}

```

```

        }
    }
    total = total.add(item.price());
}
summaries.add(new OrderSummary(order.id(), customer.name(),
    ↪ total));
}
return summaries;
}

private CustomerRepository customerRepository;
private LineItemRepository lineItemRepository;
private List<LineItem> discountRules;
}

```

REVIEW

- **Blocker — classic N+1 query pattern.** For every single order, `customerRepository.findById(...)` and `lineItemRepository.findByOrderId(...)` each issue a separate round trip to the database. For 1,000 orders that's 2,000+ queries where 2-3 would do. This is the single most common real-world performance bug found in code review, and it degrades gracefully in dev (small data) and catastrophically in production.
- **Major — $O(n \times m)$ discount-rule loop nested inside the per-order loop**, itself nested inside the per-line-item loop — for orders $\times$ items $\times$ rules this is cubic in the worst case relative to the input sizes, when the rules could be pre-indexed or applied via a single pass per item without needing to be recomputed identically for every order.
- **Minor — `discountRule.apply(item)` reassigns the loop variable `item`**, which is legal but confusing: it looks like line items and discount rules are the same type (`LineItem`), which they likely aren't conceptually — this is either a copy-paste type mismatch or a naming/API smell worth a question to the author.

Fixed code — batch-load customers and line items once, outside the per-order loop:

```

public class OrderSummaryBuilder {

    public List<OrderSummary> buildSummaries(List<Order> orders) {
        List<String> customerIds =
    ↪ orders.stream().map(Order::customerId).distinct().toList();
        List<Long> orderIds = orders.stream().map(Order::id).toList();
    }
}

```

```

        Map<String, Customer> customersById =
        ↪ customerRepository.findAllByIds(customerIds)
            .stream().collect(Collectors.toMap(Customer::id, c -> c));
        Map<Long, List<LineItem>> itemsByOrderId =
        ↪ lineItemRepository.findByIds(orderIds)

        ↪ .stream().collect(Collectors.groupingBy(LineItem::orderId));

        List<OrderSummary> summaries = new ArrayList<>();
        for (Order order : orders) {
            Customer customer = customersById.get(order.customerId());
            List<LineItem> items =
        ↪ itemsByOrderId.getOrDefault(order.id(), List.of());
            BigDecimal total = BigDecimal.ZERO;
            for (LineItem item : items) {
                LineItem discounted = applyDiscounts(item);
                total = total.add(discounted.price());
            }
            summaries.add(new OrderSummary(order.id(), customer.name(),
        ↪ total));
        }
        return summaries;
    }

    private LineItem applyDiscounts(LineItem item) {
        for (DiscountRule rule : discountRules) {
            if (rule.appliesTo(item)) {
                item = rule.apply(item);
            }
        }
        return item;
    }

    private CustomerRepository customerRepository;
    private LineItemRepository lineItemRepository;
    private List<DiscountRule> discountRules;
}

```

Two round trips total, regardless of how many orders are in the list. See [Performance Optimization](#) for more on spotting this pattern.

Scenario 7: The Audit Log Formatter

```
public class AuditLogFormatter {

    public String formatEntries(List<AuditEntry> entries) {
        String report = "";
        report += "Audit Report\n";
        report += "=====\n";
        for (AuditEntry entry : entries) {
            report += entry.timestamp() + " | " + entry.user() + " | " +
            ↪ entry.action();
            report += "\n";
        }
        report += "Total entries: " + entries.size() + "\n";
        return report;
    }
}
```

REVIEW

- **Major — string concatenation with += inside a loop.** String is immutable, so every += allocates a brand-new String and copies everything seen so far into it. For n entries, that's roughly $O(n^2)$ total character copying. For a handful of audit entries this is invisible; for a report over tens of thousands of entries it becomes a real, measurable slowdown and a burst of short-lived garbage for the GC to clean up.
- **Nit — mixing string-building style** (some lines start fresh, some append) makes the method harder to scan; consolidating onto one StringBuilder also reads more clearly as “building one thing incrementally.”

Fixed code — a single StringBuilder for the whole method:

```
public class AuditLogFormatter {

    public String formatEntries(List<AuditEntry> entries) {
        StringBuilder report = new StringBuilder();
        report.append("Audit Report\n");
        report.append("=====\n");
        for (AuditEntry entry : entries) {
            report.append(entry.timestamp()).append(" | ")
                .append(entry.user()).append(" | ")
                .append(entry.action()).append('\n');
        }
        report.append("Total entries: ")
            .append(entries.size())
            .append("\n");
        return report.toString();
    }
}
```

```

        }
        report.append("Total entries:
↪ ").append(entries.size()).append('\n');
        return report.toString();
    }
}

```

Note: a single + expression made of several pieces on one line (a + " " + b) is fine — javac compiles that to one `StringBuilder` chain automatically. The bug is specifically repeated concatenation *across loop iterations*, where each iteration starts from an already-large accumulated string.

Scenario 8: The Invoice Date Parser

```

public class InvoiceDateParser {

    private static final SimpleDateFormat FORMAT = new
↪ SimpleDateFormat("yyyy-MM-dd");

    public Date parse(String rawDate) throws ParseException {
        return FORMAT.parse(rawDate);
    }

    public String format(Date date) {
        return FORMAT.format(date);
    }
}

```

REVIEW

- **Blocker** — `SimpleDateFormat` is not thread-safe, and here it's a shared static final field. Its internal `Calendar` field is mutated during both `parse()` and `format()`. Two threads calling either method concurrently can interleave their mutations of that shared `Calendar`, producing wrong dates, garbage results, or a thrown exception — silently and intermittently, which makes it brutal to reproduce in a bug report.
- **Major** — even ignoring the field being static, sharing *any single* `SimpleDateFormat` instance across threads (static or not) has this problem; the fix is either “don’t share it” or “don’t use `SimpleDateFormat`.”

Fixed code — use `java.time`'s `DateTimeFormatter`, which is immutable and thread-safe by design (see [Chapter 9, Date, Time & Localization](#)):

```
public class InvoiceDateParser {

    private static final DateTimeFormatter FORMAT =
    ↪ DateTimeFormatter.ofPattern("yyyy-MM-dd");

    public LocalDate parse(String rawDate) {
        return LocalDate.parse(rawDate, FORMAT);
    }

    public String format(LocalDate date) {
        return date.format(FORMAT);
    }
}
```

If `SimpleDateFormat` truly cannot be avoided (some legacy API demands `Date`), the safe fallback is a fresh instance per call, or one instance per thread via `ThreadLocal<SimpleDateFormat>` — never a shared mutable field.

Scenario 9: The Translation Cache

```
public class TranslationService {

    private static final Map<String, String> cache = new HashMap<>();

    public String translate(String key, String locale) {
        String cacheKey = key + ":" + locale;
        return cache.computeIfAbsent(cacheKey, k ->
    ↪ callTranslationApi(key, locale));
    }

    private String callTranslationApi(String key, String locale) {
        // expensive network call
        return translationClient.fetch(key, locale);
    }

    private TranslationClient translationClient;
}
```

REVIEW

- **Blocker — unbounded cache.** Every distinct (key, locale) pair ever requested stays in cache for the lifetime of the process — there is no eviction, no size cap, no TTL. In a long-running service with many keys/locales (or with any attacker- or user-influenced key), this is a slow, guaranteed memory leak that eventually produces an `OutOfMemoryError` days or weeks after deployment, long after the code that caused it has been forgotten.
- **Major — plain `HashMap` shared as a static field with no synchronization,** read and written from what is presumably a multi-threaded service (translation lookups happening per request). Same category of bug as Scenario 2 — needs a concurrent-safe map at minimum, independent of the eviction problem.
- **Minor — string-concatenated composite key (key + ":" + locale)** works but is fragile (a key containing `:` could collide with a different locale split) and allocates a new `String` per lookup; a small record `TranslationKey(String key, String locale)` is clearer and avoids the delimiter-collision risk.

Fixed code — bounded, size- and time-limited cache (e.g., `Caffeine`), or at minimum a bounded `LinkedHashMap`-based LRU, plus a proper composite key:

```
public class TranslationService {

    private record TranslationKey(String key, String locale) {}

    private final Cache<TranslationKey, String> cache =
↳ Caffeine.newBuilder()
        .maximumSize(10_000)
        .expireAfterWrite(Duration.ofHours(6))
        .build();

    public String translate(String key, String locale) {
        return cache.get(new TranslationKey(key, locale),
            k -> callTranslationApi(k.key(), k.locale()));
    }

    private String callTranslationApi(String key, String locale) {
        return translationClient.fetch(key, locale);
    }

    private TranslationClient translationClient;
}
```

See [Memory & GC Review](#) for the general “unbounded cache” pattern and how to spot it without a library like Caffeine available.

Scenario 10: The Duplicate-Free Product List

```
public class Product {
    private final String sku;
    private String displayName;

    public Product(String sku, String displayName) {
        this.sku = sku;
        this.displayName = displayName;
    }

    public void rename(String newDisplayName) {
        this.displayName = newDisplayName;
    }

    @Override
    public boolean equals(Object o) {
        if (!(o instanceof Product other)) return false;
        return sku.equals(other.sku);
    }

    public String getSku() { return sku; }
}

public class ProductCatalog {
    private final Set<Product> products = new HashSet<>();

    public void add(Product product) {
        products.add(product);
    }

    public boolean contains(String sku) {
        return products.contains(new Product(sku, ""));
    }
}
```

REVIEW

- **Blocker** — `equals()` is overridden without overriding `hashCode()`. `Product` still uses `Object`'s identity-based `hashCode()`, so two `Product` instances with the same `sku` are `equals()` but almost certainly land in different `HashSet` buckets. `contains(String sku)` builds a throwaway `Product` with a different identity hash than any stored one, so it will essentially always return `false` — the exact opposite of what this class exists to do. This is a silent, no-compiler-warning correctness bug: the `equals/hashCode` contract (“equal objects must have equal hash codes”) is broken.
- **Major** — **`Product` is mutable (rename) but is stored in a `HashSet`.** Even after fixing `hashCode()`, `sku` is the only field participating in equality, so mutating `displayName` is actually safe here — but this is fragile by accident, not by design; a future field added to `equals/hashCode` that is also mutable would reintroduce Scenario 3's mutable-key bug.
- **Nit** — **`contains(String sku)` constructs a throwaway `Product("", sku)` just to do a lookup** — legal once `equals/hashCode` are fixed, but a `Map<String, Product>` keyed directly by `sku` is more direct and avoids the wasted allocation per lookup.

Fixed code — add the missing `hashCode()`, and prefer keying directly by the identity field:

```
public class Product {
    private final String sku;
    private String displayName;

    public Product(String sku, String displayName) {
        this.sku = sku;
        this.displayName = displayName;
    }

    public void rename(String newDisplayName) {
        this.displayName = newDisplayName;
    }

    @Override
    public boolean equals(Object o) {
        if (!(o instanceof Product other)) return false;
        return sku.equals(other.sku);
    }

    @Override
    public int hashCode() {
        return sku.hashCode();
    }

    public String getSku() { return sku; }
}
```

```

public class ProductCatalog {
    private final Map<String, Product> productsBySku = new HashMap<>();

    public void add(Product product) {
        productsBySku.put(product.getSku(), product);
    }

    public boolean contains(String sku) {
        return productsBySku.containsKey(sku);
    }
}

```

Scenario 11: The Per-User Session Cache

```

public class SessionCache {

    private final Map<String, List<String>> activityByUser = new
    ⇨ ConcurrentHashMap<>();

    public void recordActivity(String userId, String activity) {
        activityByUser.computeIfAbsent(userId, id -> {
            List<String> history = loadHistoryFromDb(id); // side effect:
            ⇨ DB call inside the lambda
            history.add(activity);
            return history;
        });
    }

    public void addRelatedUser(String userId, String relatedUserId) {
        activityByUser.computeIfAbsent(userId, id -> new ArrayList<>())
            .add("related:" + relatedUserId);
        activityByUser.computeIfAbsent(relatedUserId, id ->
            ⇨ reference
            activityByUser.get(userId)); // reuse the same list
    }

    private List<String> loadHistoryFromDb(String userId) { /* ... */
    ⇨ return new ArrayList<>(); }
}

```

REVIEW

- **Blocker** — **recordActivity** only adds the new activity on the *first* call for a user. `computeIfAbsent`'s mapping function only runs when the key is absent; on every subsequent call for a user who's already in the map, the lambda is skipped entirely and activity is silently dropped. This looks like it works in a quick manual test (first call always “works”) and then quietly loses data for every repeat visitor — the most dangerous kind of bug because the obvious test case passes.
- **Blocker** — **loadHistoryFromDb** (a blocking, potentially slow DB call) runs *inside* the `computeIfAbsent` mapping function, which `ConcurrentHashMap` executes while holding an internal lock on that key's bin. If it's slow, or if it happens to (directly or transitively) call back into the same map for the same key, you can get very long stalls for every other thread touching that bin, or in some JDK versions an actual deadlock/`IllegalStateException` (recursive update). `computeIfAbsent`'s mapping function must be fast and must never touch the same map.
- **Major** — **addRelatedUser** aliases two users to the *same mutable List instance*. After this call, `activityByUser.get(userId)` and `activityByUser.get(relatedUserId)` are the identical list object; recording activity for one silently mutates the “history” of the other. This is almost certainly not intended and is very hard to spot from the call site.

Fixed code — separate the DB load from the update, keep the mapping function cheap and side-effect-free with respect to the map itself, and never alias mutable collections between keys:

```
public class SessionCache {

    private final Map<String, List<String>> activityByUser = new
    ↪ ConcurrentHashMap<>();

    public void recordActivity(String userId, String activity) {
        List<String> history = activityByUser.computeIfAbsent(userId,
            id -> new CopyOnWriteArrayList<>(loadHistoryFromDb(id)));
        ↪ // fast to construct, DB call happens once, outside any recursive risk
        history.add(activity); // always runs, regardless of whether this
        ↪ was the first call
    }

    public void addRelatedUser(String userId, String relatedUserId) {
        activityByUser.computeIfAbsent(userId, id -> new
        ↪ CopyOnWriteArrayList<>())
            .add("related:" + relatedUserId);
        activityByUser.computeIfAbsent(relatedUserId, id -> new
        ↪ CopyOnWriteArrayList<>())
            .add("related-to:" + userId); // own list, not a shared
        ↪ reference
    }
}
```

```

    }

    private List<String> loadHistoryFromDb(String userId) { /* ... */
    ↪ return new ArrayList<>(); }
    }

```

Scenario 12: The Invalid Order Filter

```

public class OrderValidationReport {

    public List<String> findInvalidOrderIds(List<Order> orders) {
        List<String> invalidIds = new ArrayList<>();
        int[] invalidCount = {0};

        orders.parallelStream().forEach(order -> {
            if (!isValid(order)) {
                invalidIds.add(order.id());
                invalidCount[0]++;
            }
        });

        System.out.println("Found " + invalidCount[0] + " invalid
    ↪ orders");
        return invalidIds;
    }

    private boolean isValid(Order order) {
        return order.total().compareTo(BigDecimal.ZERO) >= 0 &&
    ↪ order.customerId() != null;
    }
}

```

REVIEW

- **Blocker**—`invalidIds.add(...)` from a `parallelStream().forEach`. `ArrayList` is not thread-safe, and `parallelStream()` runs the lambda on multiple threads from the common `ForkJoinPool`. Concurrent, unsynchronized `add()` calls on an `ArrayList` can throw `ArrayIndexOutOfBoundsException`, silently drop elements, or corrupt the list's internal array—a data race hiding behind a one-line stream call that looks innocuous.

- **Major** — `invalidCount[0]++` is also a race, for the same reason: the array-boxing trick to mutate a captured variable from a lambda doesn't make the increment atomic. Multiple threads can read-increment-write the same stale value and lose counts.
- **Major** — reaching for `parallelStream()` at all here is questionable ([Performance Optimization](#) covers this pattern): `isValid` is a trivial, non-blocking check, so the overhead of splitting work across the shared `ForkJoinPool` easily outweighs any benefit, and it puts load on a JVM-wide shared pool that other unrelated parallel streams also depend on.
- **Nit** — mixing a raw `System.out.println` for what looks like log-worthy operational information instead of a logger.

Fixed code — collect via a proper stream `Collector` (thread-safe by construction) and drop the unnecessary parallelism:

```
public class OrderValidationReport {
    private static final Logger log =
↳ LoggerFactory.getLogger(OrderValidationReport.class);

    public List<String> findInvalidOrderIds(List<Order> orders) {
        List<String> invalidIds = orders.stream()
            .filter(order -> !isValid(order))
            .map(Order::id)
            .toList();

        log.info("Found {} invalid orders", invalidIds.size());
        return invalidIds;
    }

    private boolean isValid(Order order) {
        return order.total().compareTo(BigDecimal.ZERO) >= 0 &&
↳ order.customerId() != null;
    }
}
```

If this really needs to run in parallel because `isValid` does expensive, CPU-bound, side-effect-free work over a very large list, `parallelStream()` is fine—but then `.filter(...).map(...).toList()` (a proper collector) is still the right way to gather results, never a shared mutable list touched from inside `forEach`.

Scenario 13: The Batch Email Sender

```
public class BatchEmailSender {

    public void sendBatch(List<Email> emails) {
        ExecutorService executor = Executors.newFixedThreadPool(10);
        for (Email email : emails) {
            executor.submit(() -> smtpClient.send(email));
        }
    }

    private SmtplibClient smtpClient;
}
```

REVIEW

- **Blocker — the executor is never shut down.** A brand-new `ExecutorService` (with 10 live platform threads) is created on every call to `sendBatch` and simply abandoned once the method returns. `ExecutorServices` hold real threads that are not garbage-collected just because the reference goes out of scope — each call leaks 10 threads that live forever (or until the JVM exits), and a service that calls `sendBatch` repeatedly will exhaust available threads/memory and eventually fail with `OutOfMemoryError: unable to create new native thread`.
- **Major — no way to know when the batch actually finished,** and no error handling for individual sends: `smtpClient.send(email)` inside the submitted task can throw, and since nothing calls `Future.get()` on the submitted tasks, that exception is silently swallowed by the executor (visible only if you dig through logs for an “uncaught exception in thread pool” trace, if that’s even configured).
- **Minor — a fresh fixed pool per batch is also wasteful even if shut down correctly;** a single shared, appropriately-sized executor reused across calls (owned by the surrounding component’s lifecycle, not created ad hoc per method call) is both cheaper and easier to reason about.

Fixed code — own the executor as a long-lived field with a proper lifecycle (shutdown), and track task outcomes:

```
public class BatchEmailSender implements AutoCloseable {
    private static final Logger log =
        ↳ LoggerFactory.getLogger(BatchEmailSender.class);
```

```

    private final ExecutorService executor =
↳ Executors.newFixedThreadPool(10);
    private final SmtClient smtpClient;

    public BatchEmailSender(SmtClient smtpClient) {
        this.smtpClient = smtpClient;
    }

    public void sendBatch(List<Email> emails) {
        List<Future<?>> futures = emails.stream()
            .map(email -> executor.submit(() ->
↳ smtpClient.send(email)))
            .toList();

        for (Future<?> future : futures) {
            try {
                future.get();
            } catch (ExecutionException e) {
                log.warn("Failed to send an email in batch",
↳ e.getCause());
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                throw new RuntimeException("Interrupted while sending
↳ batch", e);
            }
        }
    }

    @Override
    public void close() {
        executor.shutdown(); // let in-flight sends finish; see
↳ 14-concurrency-advanced.md for shutdown patterns
    }
}

```

Scenario 14: The Import Job Runner

```

public class ImportJobRunner {

    public ImportResult runImport(String filePath) {
        try {
            List<String> lines = Files.readAllLines(Path.of(filePath));
            List<Record> records = new ArrayList<>();
            for (String line : lines) {

```

```

        Record record = parseLine(line);
        validate(record);
        records.add(record);
    }
    repository.saveAll(records);
    notifyStakeholders(records.size());
    return ImportResult.success(records.size());
} catch (Exception e) {
    return ImportResult.failure("Import failed");
}
}

private Record parseLine(String line) { /* throws
    ↪ NumberFormatException on bad rows */ return null; }
private void validate(Record record) { /* throws ValidationException
    ↪ */ }
private void notifyStakeholders(int count) { /* sends an email, can
    ↪ throw MessagingException */ }
}

```

REVIEW

- **Major — one giant try wraps five completely different failure domains:** file I/O (IOException), per-line parsing (NumberFormatException), business validation (ValidationException), the DB save (repository.saveAll — any runtime exception it throws), and notification (MessagingException). All of them collapse into the same generic "Import failed" message, so a caller (or a human on call) cannot tell “the file didn’t exist” apart from “row 4,502 had a wrong data type” apart from “the DB write itself failed after everything else succeeded” apart from “the import fully succeeded but the confirmation email failed to send.” Those require completely different responses.
- **Major — a failure in notifyStakeholders (just sending an email) reports the entire import as failed,** even though repository.saveAll(records) already committed. The caller will likely re-run the import, potentially double-importing records, because of a problem in an unrelated, non-critical side effect.
- **Minor — no logging at all inside the catch,** generic or otherwise — whatever the real exception was is discarded completely, not even preserved for later diagnosis.

Fixed code — scope each try to the failure domain that’s actually being handled, and don’t let a non-critical side effect (notification) mark an otherwise-successful core operation as failed:

```

public class ImportJobRunner {
    private static final Logger log =
↳ LoggerFactory.getLogger(ImportJobRunner.class);

    public ImportResult runImport(String filePath) {
        List<String> lines;
        try {
            lines = Files.readAllLines(Path.of(filePath));
        } catch (IOException e) {
            log.error("Could not read import file {}", filePath, e);
            return ImportResult.failure("Import file could not be read: "
↳ + e.getMessage());
        }

        List<Record> records = new ArrayList<>();
        for (int i = 0; i < lines.size(); i++) {
            try {
                Record record = parseLine(lines.get(i));
                validate(record);
                records.add(record);
            } catch (NumberFormatException | ValidationException e) {
                log.error("Bad row {} in {}: {}", i + 1, filePath,
↳ e.getMessage());
                return ImportResult.failure("Row " + (i + 1) + " is
↳ invalid: " + e.getMessage());
            }
        }

        repository.saveAll(records); // let a DB failure propagate as-is –
↳ it's a real defect, not an expected outcome

        try {
            notifyStakeholders(records.size());
        } catch (MessagingException e) {
            // Import already succeeded; a failed notification shouldn't
↳ undo that fact.
            log.warn("Import of {} records succeeded but notification
↳ failed", records.size(), e);
        }

        return ImportResult.success(records.size());
    }

    private Record parseLine(String line) { return null; }
    private void validate(Record record) { }
    private void notifyStakeholders(int count) throws MessagingException
↳ { }
}

```

Scenario 15: The Live Price Ticker

```

public class LivePriceTicker {

    private final List<PriceListener> listeners = new ArrayList<>();
    private volatile double lastPrice;

    public LivePriceTicker(PriceFeed feed) {
        feed.subscribe(this::onPriceUpdate); // registers 'this' before
        ⇒ construction finishes
        this.startBackgroundRefresh();
    }

    private void startBackgroundRefresh() {
        Thread refresher = new Thread(() -> {
            while (true) {
                refreshFromFeed();
                sleepQuietly();
            }
        });
        refresher.start(); // thread started from inside the constructor
    }

    public void addListener(PriceListener listener) {
        listeners.add(listener);
    }

    private void onPriceUpdate(double price) {
        this.lastPrice = price;
        for (PriceListener listener : listeners) {
            listener.onPrice(price);
        }
    }

    private void refreshFromFeed() { /* ... */ }
    private void sleepQuietly() { try { Thread.sleep(1000); } catch
    ⇒ (InterruptedException e) { Thread.currentThread().interrupt(); } }
}

```

REVIEW

- **Blocker — this escapes the constructor twice, before construction has finished.** `feed.subscribe(this::onPriceUpdate)` hands out a reference to the object (via the method reference, which captures `this`) to another object (`feed`) *while the constructor is still running*. If `feed` calls back synchronously, or another thread reaches `this` through `feed` before the constructor returns, it can observe a half-initialized `LivePriceTicker` — for example, listeners might still be `null` if this call happened before the field initializer ran, or fields set later in the constructor simply won't be visible yet. This is the “leaking this” pitfall, and it's especially dangerous combined with...
- **Blocker — starting a background thread from inside the constructor.** `refresher.start()` hands the *same partially-constructed* object to a new thread of execution, with no memory barrier guaranteeing the new thread sees a fully-initialized object — the Java Memory Model does not guarantee that a background thread started during construction sees writes made later in that same constructor, unless those fields are `final` or otherwise safely published. This is a subtle, hard-to-reproduce race that depends on JIT/CPU reordering and timing.
- **Major — the refresher thread runs while (true) with no way to stop it**, and it's never tracked anywhere (no field holding the `Thread`). There is no way to shut this down cleanly — same lifecycle problem as Scenario 13's abandoned executor, but worse, because there isn't even a reference to call `interrupt()` on later.
- **Minor — listeners (an `ArrayList`) is read and written without synchronization**, and `onPriceUpdate` (which iterates it) can be invoked concurrently with `addListener` (which mutates it) from different threads (the `feed`'s callback thread vs. whatever thread calls `addListener`) — another `ConcurrentModificationException`/corruption risk layered on top of the construction issue.

Fixed code — finish construction fully before publishing `this` anywhere, use a static factory to start the background work only after the object is safely built, keep a handle to the thread for shutdown, and use a concurrent-safe listener list:

```
public class LivePriceTicker {

    private final List<PriceListener> listeners = new
↳ CopyOnWriteArrayList<>();
    private final Thread refresher;
    private volatile double lastPrice;
    private volatile boolean running = true;

    private LivePriceTicker() {
        this.refresher = new Thread(this::refreshLoop);
    }
}
```

```

    // Factory method: object is fully constructed before anything
    ↪ external can see it.
    public static LivePriceTicker start(PriceFeed feed) {
        LivePriceTicker ticker = new LivePriceTicker();
        feed.subscribe(ticker::onPriceUpdate); // 'this' only escapes
    ↪ after the constructor returned
        ticker.refresher.start();
        return ticker;
    }

    private void refreshLoop() {
        while (running) {
            refreshFromFeed();
            sleepQuietly();
        }
    }

    public void shutdown() {
        running = false;
        refresher.interrupt();
    }

    public void addListener(PriceListener listener) {
        listeners.add(listener);
    }

    private void onPriceUpdate(double price) {
        this.lastPrice = price;
        for (PriceListener listener : listeners) {
            listener.onPrice(price);
        }
    }

    private void refreshFromFeed() { /* ... */ }
    private void sleepQuietly() { try { Thread.sleep(1000); } catch
    ↪ (InterruptedException e) { Thread.currentThread().interrupt(); } }
    }

```

See [Chapter 13, *Concurrency \(Core\)*](#) for the Java Memory Model rules behind “safe publication,” and [Chapter 2, *Object-Oriented Programming*](#) for constructor discipline in general.

Performance Optimization

A checklist first, then the wins that show up over and over in review.

Checklist

Is there actual evidence (a profile, a benchmark, a production metric) that this code is a bottleneck, or is this optimization speculative?

Is the algorithmic complexity (Big-O) obviously worse than it needs to be for the data sizes involved?

Is the right data structure being used for the access pattern (see [Collections Selection](#))?

Is boxing/unboxing happening in a numeric hot path that could use primitives?

Is a Pattern being compiled repeatedly (e.g., inside a loop or via `String.matches()`) instead of once?

Is I/O unbuffered, or done byte-by-byte/line-by-line in a way that causes excessive system calls?

Are database calls batched, or issued one-by-one in a loop ($N+1$)?

Is expensive work done eagerly at startup/construction when it could be lazy, or vice versa — lazily recomputed on every call when it could be cached once?

Is parallelism (`parallelStream`, manual thread pools) applied to trivial or I/O-bound work where the overhead outweighs the benefit?

Measure First

The single biggest mistake in performance review is optimizing before measuring. “This *looks* slow” is not evidence; a profiler, a JFR recording ([Chapter 12, Memory Management](#)), or a microbenchmark is. For method-level questions (“is `StringBuilder` actually faster here, and by how much”), the standard tool is **JMH** (Java Microbenchmark Harness) — it handles JIT warm-up, dead-code elimination by the optimizer, and statistical noise that a naive `System.nanoTime()` loop gets wrong. In an interview, you won’t run JMH, but saying “I’d confirm this with a JMH benchmark rather than guessing” is exactly the right instinct to voice.

Big-O Awareness

Recognize complexity by shape, without doing formal analysis:

- A loop calling `.contains()` or `.get()` on a `List` inside another loop $\rightarrow O(n^2)$ (should be a `Set/Map` lookup, $O(n)$).

- A loop making a DB/network call per iteration → $O(n)$ round trips that should be $O(1)$ (Scenario 6).
- Nested loops over independent dimensions that don't need to be recomputed per outer iteration → hoist the inner computation out or precompute it once (Scenario 6's discount rules).
- String-building with + across iterations → $O(n^2)$ character copying (Scenario 7).

```
// Flagged:  $O(n * m)$  – .contains() on a List is a linear scan, done once
→ per outer element
List<String> blockedIds = loadBlockedIds(); // a List
for (Order order : orders) {
    if (blockedIds.contains(order.customerId())) { ... }
}
```

```
// Fixed:  $O(n + m)$  – one linear pass to build the Set, then  $O(1)$  lookups
Set<String> blockedIds = new HashSet<>(loadBlockedIds());
for (Order order : orders) {
    if (blockedIds.contains(order.customerId())) { ... }
}
```

Common Real Wins

```
// Flagged: Pattern compiled on every call – String.matches() compiles
→ internally, every time
public boolean isValidSku(String sku) {
    return sku.matches("[A-Z]{3}-\\d{4}");
}
```

```
// Fixed: compile once, reuse the compiled Pattern
private static final Pattern SKU_PATTERN =
→ Pattern.compile("[A-Z]{3}-\\d{4}");

public boolean isValidSku(String sku) {
    return SKU_PATTERN.matcher(sku).matches();
}
```

```
// Flagged: boxed Integer accumulator in a tight numeric loop –
→ boxes/unboxes a million times
Integer total = 0;
```

```

for (int i = 0; i < prices.length; i++) {
    total += prices[i]; // unbox total, add, re-box into a new Integer,
↪ every iteration
}

```

```

// Fixed: primitive accumulator, no boxing at all
int total = 0;
for (int i = 0; i < prices.length; i++) {
    total += prices[i];
}

```

```

// Flagged: unbuffered, one-byte-at-a-time reads — one native call per
↪ byte
try (FileInputStream in = new FileInputStream(path)) {
    int b;
    while ((b = in.read()) != -1) { process(b); }
}

```

```

// Fixed: wrap with a BufferedInputStream (or use readAllBytes/NIO for
↪ whole-file reads)
try (BufferedInputStream in = new BufferedInputStream(new
↪ FileInputStream(path))) {
    int b;
    while ((b = in.read()) != -1) { process(b); }
}

```

Premature Optimization Is Also a Finding

Flag the opposite mistake too: adding a cache, a custom data structure, or manual parallelism for code that isn't a bottleneck adds real cost — more moving parts, more places to get thread-safety wrong (see Scenario 9, Scenario 12), harder-to-read code — for a performance gain nobody asked for or measured. “I'd leave this as a simple loop unless we have evidence it's hot” is a legitimate, senior-sounding review comment, not a cop-out.

Thread Safety Review

Checklist

What state does this object hold, and is any of it mutable?

Can more than one thread reach this object at the same time (is it a singleton, a field on a shared service, handed to an executor, stored in a servlet-scoped or application-scoped context)?

If shared and mutable: is every access to that state properly synchronized, `volatile`, or backed by a concurrent-safe type?

Does the object fully finish constructing before any reference to it (or to `this` via a lambda/method reference) is handed to another thread?

Is the class's thread-safety contract documented, so callers don't have to read the implementation to find out?

Reasoning About Shared Mutable State

The question to ask about *every* field is: **who else can see this, and can they see it while I'm changing it?** Three outcomes:

1. **Not shared** — a local variable, or an object fully confined to one thread. No synchronization needed. This is the best outcome; prefer designs that keep state confined.
2. **Shared but immutable** — the value never changes after construction (`final` fields, no setters, defensively-copied collections). Safe to share freely with no locking, because there's nothing to race on. This is the second-best outcome — see [Chapter 5, Object Class & Common APIs](#) on immutability.
3. **Shared and mutable** — now you need an explicit safety mechanism: a lock (`synchronized`, `ReentrantLock`), `volatile` for single-variable visibility, an atomic class, or a concurrent collection. This is the case that needs the most scrutiny in review, and where most real bugs (Scenarios 1, 2, 9, 15) live.

Prefer immutability first. Before reaching for a lock, ask whether the mutable field even needs to be mutable — could it instead be replaced wholesale (a `volatile` reference to a new immutable object) rather than mutated in place? This sidesteps entire classes of bugs instead of managing them.

Confinement is the other escape hatch: if an object is only ever touched by one thread (e.g., a per-request object never handed off, a `ThreadLocal`), it doesn't need synchronization regardless of whether the class itself is thread-safe in general — but this must be true structurally, not “true in practice today,” because it silently breaks the moment someone hands the object to an executor.

The Escape/Publication Question

Ask, for every object: **how does a reference to this object first reach another thread, and has construction fully finished by then?** This is exactly what broke in Scenario 15 — this was captured by a lambda and handed to `feed.subscribe(...)` before the constructor returned. Safe publication options, cheapest first:

- Return the fully-constructed object from a `static` factory method instead of doing any handoff inside the constructor itself.
- Store the reference in a `volatile` or `final` field, or into a properly locked structure, before any other thread can see it.
- Use a concurrent collection (`ConcurrentHashMap`, etc.) to publish it — insertion into those collections has the necessary memory-visibility guarantees built in.

Documenting Thread Safety

A class's thread-safety contract is part of its API, not an implementation detail — callers need to know it without reading the source. State it explicitly, ideally in the class Javadoc: “this class is thread-safe; all methods may be called concurrently” or “this class is not thread-safe; instances must be confined to a single thread” or “safe for concurrent reads, writes must be externally synchronized.” See [Chapter 13, *Concurrency \(Core\)*](#) for the Java Memory Model vocabulary (visibility, atomicity, ordering, happens-before) to use precisely when writing or reviewing such statements.

Collections Selection

Decision Table

Need	Prefer	Avoid	Why
Index-based random access, mostly reads	<code>ArrayList</code>	<code>LinkedList</code>	$O(1)$ indexed get vs. $O(n)$ traversal
Frequent insert/remove at both ends	<code>ArrayDeque</code>	<code>LinkedList</code> , <code>Stack</code>	Better cache locality, no legacy synchronized overhead
Fast key lookup, order doesn't matter	<code>HashMap</code> / <code>HashSet</code>	<code>TreeMap</code> / <code>TreeSet</code> (unless sorted order needed)	$O(1)$ average vs. $O(\log n)$

Need	Prefer	Avoid	Why
Insertion order must be preserved	LinkedHashMap / LinkedHashSet	HashMap (order unspecified)	Predictable iteration order
Sorted key order needed	TreeMap / TreeSet	Manually sorting a HashMap's keys repeatedly	Maintains order incrementally, $O(\log n)$ per op
Multiple threads read/write a map concurrently	ConcurrentHashMap	HashMap, or Collections. synchronizedMap for hot paths	Lock-stripped/segment-free reads, far less contention
Mostly-read, occasionally-written list shared across threads	CopyOnWriteArrayList	synchronized(List) with manual locking, or plain ArrayList	Read without locking at all; write cost is acceptable when reads dominate
Producer/consumer handoff between threads	BlockingQueue (LinkedBlockingQueue, ArrayBlockingQueue)	A List polled in a loop (“busy wait”)	Built-in blocking wait, no CPU-spinning
A fixed, never-changing collection	List.of(...) / Map. of(...)	Collections. unmodifiableList(new ArrayList<>(...)) for something that's genuinely constant	Simpler, truly immutable, slightly more memory-efficient
Enum-keyed map/set	EnumMap / EnumSet	HashMap<SomeEnum, V>	Backed by an array indexed by ordinal — faster and more memory-compact
Cache that shouldn't pin entries in memory once nothing else references the key	WeakHashMap (or better, a real cache library)	HashMap	Entries can be GC'd once the key is otherwise unreachable

See [Chapter 7, Collections Framework](#) for the full breakdown of each type's internals.

“They Used X, Should Be Y” Cases

```
// Flagged: LinkedList for pure random access by index
LinkedList<Transaction> transactions = loadTransactions();
Transaction tenth = transactions.get(9); // O(n) traversal from the head
↪ every time
```

Should be: `ArrayList<Transaction>` — $O(1)$ indexed access, and `LinkedList` rarely wins in practice even for its supposed strengths, because of pointer-chasing and per-node allocation overhead.

```
// Flagged: HashMap iterated in a context that assumes stable, predictable
↪ order (e.g., building a UI list or a diff)
Map<String, BigDecimal> balancesByAccount = new HashMap<>();
// ... populate ...
for (var entry : balancesByAccount.entrySet()) { render(entry); } // order
↪ can change silently between runs/resizes
```

Should be: `LinkedHashMap` if insertion order matters for the output, or `TreeMap` if sorted-by-key order matters — never rely on `HashMap`'s iteration order, which is an implementation detail, not a contract.

```
// Flagged: Vector or Collections.synchronizedList for a hot, contended,
↪ mostly-read list
List<Listener> listeners = new Vector<>(); // synchronizes every single
↪ method call, even reads
```

Should be: `CopyOnWriteArrayList` if reads vastly outnumber writes (typical for listener lists) — no locking at all on reads.

```
// Flagged: a List used to check membership repeatedly
List<String> allowedRoles = List.of("ADMIN", "OWNER", "BILLING");
if (allowedRoles.contains(user.role())) { ... } // fine for 3 elements, a
↪ trap if this list grows or is called in a hot loop
```

Should be, once the list could grow or the check is on a hot path: `Set.of("ADMIN", "OWNER", "BILLING")` — $O(1)$ lookup instead of $O(n)$ linear scan, and it also documents “this is a membership set,” not an ordered sequence.

Exception Design

Checklist

Is the exception type at this API boundary checked (caller must handle/declare) or unchecked (caller may ignore)? Is that the right choice for whether callers can realistically recover?

Does a custom exception type add real domain meaning, or is it one more type to remember with no behavioral difference from an existing one?

Is any exception ever caught and silently discarded (`catch (X e) {}`) or logged with no rethrow and no handling)?

Is any exception logged **and** rethrown at the same layer (double reporting up the call stack)?

When wrapping a lower-level exception, is the original passed as the cause (new Foo(msg, e)), never dropped?

Does validation happen at the boundary (fail fast, clear message) rather than deep inside, producing a confusing downstream NullPointerException instead?

Full mechanics — finally/return interactions, suppressed exceptions, multi-catch, NullPointerException messages — are covered in [Chapter 6, Exception Handling](#); this section is the *design* lens specifically for review.

Checked vs. Unchecked at API Boundaries

Situation	Choice
Caller can genuinely do something different on failure (retry, fallback, prompt again)	Checked, or a documented unchecked type callers are expected to catch
Failure represents a bug in the calling code (bad argument, wrong state)	Unchecked (IllegalArgumentException, IllegalStateException)
Code composed with streams/lambdas, where checked exceptions don't fit the functional interfaces	Unchecked, or wrap with UncheckedIOException-style adapters
A public library boundary where you want to force callers to consciously acknowledge a failure mode	Checked

Never Swallow, Never Log-and-Rethrow

```
// Flagged: swallowed – the failure vanishes with no trace
try {
    inventoryService.reserve(sku, qty);
} catch (Exception e) {
    // ignored
}
```

```
// Flagged: logged AND rethrown – the same failure gets logged again by
↳ every caller up the stack
try {
    inventoryService.reserve(sku, qty);
} catch (InventoryException e) {
```

```

    log.error("Reservation failed", e);
    throw e; // caller will likely log this again
}

```

```

    // Fixed: handle it here, or propagate it – pick one. If propagating,
    ↪ don't also log here;
    // log once, at the boundary that actually decides what to do about it.
    try {
        inventoryService.reserve(sku, qty);
    } catch (InventoryException e) {
        throw new OrderFulfillmentException("Could not reserve " + sku, e); //
    ↪ wrapped, cause preserved, not logged yet
    }

```

Wrap With Cause, Fail Fast

```

    // Flagged: cause dropped, and validation happens deep inside where the
    ↪ error is confusing
    public void applyDiscount(Order order, String code) {
        DiscountRule rule = ruleRepository.findByCode(code); // returns null
    ↪ if not found
        order.applyDiscount(rule.percentage()); // NPE here if code was
    ↪ invalid – confusing stack trace
    }

```

```

    // Fixed: fail fast with a clear message, and preserve any real cause when
    ↪ wrapping
    public void applyDiscount(Order order, String code) {
        Objects.requireNonNull(order, "order must not be null");
        DiscountRule rule = ruleRepository.findByCode(code)
            .orElseThrow(() -> new InvalidDiscountCodeException("No
    ↪ discount rule for code: " + code));
        order.applyDiscount(rule.percentage());
    }

```

Custom Exception Hierarchies

Keep them shallow and purposeful — one or two levels, each type adding real meaning a caller might act on differently:


```

public class OrderException extends RuntimeException {
    public OrderException(String message, Throwable cause) {
↪ super(message, cause); }
    public OrderException(String message) { super(message); }
}

public class InsufficientInventoryException extends OrderException {
    public InsufficientInventoryException(String sku) {
        super("Insufficient inventory for SKU: " + sku);
    }
}

public class InvalidDiscountCodeException extends OrderException {
    public InvalidDiscountCodeException(String message) { super(message);
↪ }
}

```

A reviewer should flag a hierarchy that's either too flat (everything is a bare `RuntimeException`, so callers can't distinguish failure modes without string-matching messages — pitfall #14 from the [Common Java Interview Pitfalls](#) list) or too deep (five levels of subclassing with no behavioral difference, adding ceremony with no payoff).

API Design Review

Checklist

Can any parameter or return value be `null`? If so, is that documented, defended against with `Objects.requireNonNull`, or better, redesigned to avoid `null` altogether?

Is `Optional` used only as a return type, never as a field, a parameter, or inside a collection?

Are parameters validated at the top of the method (fail fast) with a clear message, rather than failing confusingly several calls deep?

Is the type unnecessarily mutable — could fields be `final` and setters removed in favor of a constructor or a “with”-style copy method?

Do method names accurately signal cost and side effects (`getX` should be cheap and side-effect-free; `computeX`/`fetchX` signals real work)?

Do overloads behave consistently with each other, and is it clear which one a given call site will resolve to?

Would a currently-reasonable call site break if this method's signature, exceptions, or semantics changed — i.e., is this API easy to evolve later?

See [Chapter 19, *Software Engineering Best Practices*](#) for the broader API-design and clean-code principles this section specializes for review purposes.

Nulls and Optional

```
// Flagged: null used as "not found," undocumented, and Optional used as a
↪ field type
public class UserLookupResult {
    private Optional<User> user; // don't do this – Optional as a field
    public User find(String id) {
        return repository.findById(id); // returns null if missing, no
↪ Javadoc says so
    }
}
```

```
// Fixed: Optional only as a return type; the field itself is a plain,
↪ possibly-null-free reference
public class UserLookupResult {
    /**
     * @return the user, or empty if no user exists with the given id.
     */
    public Optional<User> find(String id) {
        return repository.findById(id); // repository itself now returns
↪ Optional<User>
    }
}
```

Parameter Validation and Naming

```
// Flagged: no validation; a bad call fails deep inside, far from the
↪ actual mistake
public void scheduleShipment(String orderId, int daysFromNow) {
    LocalDate shipDate = LocalDate.now().plusDays(daysFromNow);
    warehouse.schedule(orderId, shipDate); // throws something cryptic if
↪ orderId is blank
}
```

```

// Fixed: fail fast, at the boundary, with a message that names the actual
↪ problem
public void scheduleShipment(String orderId, int daysFromNow) {
    if (orderId == null || orderId.isBlank()) {
        throw new IllegalArgumentException("orderId must not be blank");
    }
    if (daysFromNow < 0) {
        throw new IllegalArgumentException("daysFromNow must not be
↪ negative: " + daysFromNow);
    }
    warehouse.schedule(orderId, LocalDate.now().plusDays(daysFromNow));
}

```

Mutability and Backward Compatibility

```

// Flagged: needlessly mutable value type, and a public setter that could
↪ break invariants
public class Money {
    private BigDecimal amount;
    private String currency;
    public void setAmount(BigDecimal amount) { this.amount = amount; } //
↪ no validation, no currency re-check
}

```

```

// Fixed: immutable, validated at construction, changes produce a new
↪ instance
public final class Money {
    private final BigDecimal amount;
    private final String currency;

    public Money(BigDecimal amount, String currency) {
        if (amount.scale() > 2) throw new IllegalArgumentException("amount
↪ must have at most 2 decimal places");
        this.amount = amount;
        this.currency = Objects.requireNonNull(currency);
    }

    public Money withAmount(BigDecimal newAmount) {
        return new Money(newAmount, currency);
    }
}

```

For backward compatibility: widening what a method accepts (e.g., `List<? extends T>` instead of `List<T>`) or narrowing what it promises to throw is generally safe; removing a public method, changing a return type, or *narrowing* accepted input is a breaking change and should be flagged as a compatibility risk even if it “looks like a small tidy-up.”

Memory & GC Review

Checklist

Any `static` collection/field that only ever grows (cache, registry, listener list) with no eviction or removal path?

Any allocation happening inside a hot loop that could be hoisted outside it (a new for an object that could be reused, a boxed wrapper, a formatter)?

Any long-lived object holding a reference to something that should have a short lifetime (e.g., an outer class implicitly held by a non-static inner class registered as a long-lived listener)?

Any `ThreadLocal` used inside a thread-pool-backed executor, without being explicitly cleared when the task finishes?

Would a heap dump plausibly show one dominant retained-size culprit here, or is this a death-by-a-thousand-small-allocations concern instead?

Deep GC algorithm mechanics (G1, ZGC, Shenandoah, tuning flags) live in [Chapter 12, *Memory Management*](#); this section is about spotting the *code patterns* that create memory pressure or leaks in the first place, regardless of which collector is running.

Leak-Prone Patterns

```
// Flagged: static, ever-growing collection – a permanent, unbounded leak
public class SessionRegistry {
    private static final Map<String, Session> ACTIVE_SESSIONS = new
↳ HashMap<>();
    public static void register(Session s) { ACTIVE_SESSIONS.put(s.id(),
↳ s); } // never removed
}
```

```

    // Fixed: an explicit removal path tied to the session's actual lifecycle,
    ↪ or a bounded/expiring cache
    public class SessionRegistry {
        private static final Map<String, Session> ACTIVE_SESSIONS = new
    ↪ ConcurrentHashMap<>();
        public static void register(Session s) { ACTIVE_SESSIONS.put(s.id(),
    ↪ s); }
        public static void unregister(String sessionId) {
    ↪ ACTIVE_SESSIONS.remove(sessionId); } // called on logout/expiry
    }

```

ThreadLocal in a Pool

```

    // Flagged: ThreadLocal set but never removed, on a thread that will be
    ↪ reused by the pool
    public class RequestContext {
        private static final ThreadLocal<User> CURRENT_USER = new
    ↪ ThreadLocal<>();
        public void handle(Request req) {
            CURRENT_USER.set(req.user());
            process(req);
            // no CURRENT_USER.remove() – the User object (and everything it
    ↪ references) stays
            // reachable from this pool thread until some *unrelated* future
    ↪ task overwrites it
        }
    }

```

```

    // Fixed: always remove in a finally, so the pool thread doesn't retain
    ↪ the value between tasks
    public class RequestContext {
        private static final ThreadLocal<User> CURRENT_USER = new
    ↪ ThreadLocal<>();
        public void handle(Request req) {
            CURRENT_USER.set(req.user());
            try {
                process(req);
            } finally {
                CURRENT_USER.remove();
            }
        }
    }

```

This matters specifically *because* pool threads are long-lived and reused — a `ThreadLocal` left set silently extends an object's lifetime far beyond the request that created it, and can even leak one user's data into logic that (mistakenly) reads the `ThreadLocal` during a later, different request on the same pooled thread.

Heap Dump Basics (What a Reviewer Should Know to Ask For)

When a leak is suspected but not obvious from reading the code, the review-worthy question is “have we taken a heap dump (`jcmd <pid> GC.heap_dump`, or `-XX:+HeapDumpOnOutOfMemoryError`) and looked at retained size by class/dominator tree?” A handful of instances of one class holding a disproportionate amount of retained memory is the signature of exactly the patterns above — an unbounded map, a listener list nobody ever removes from, or an accidental reference chain keeping something alive. See [Chapter 12, *Memory Management*](#) for the tools (`jmap`, `jcmd`, JFR, JMC) in depth.

Concurrency Review

Checklist

Is every `ExecutorService` shut down somewhere on every relevant path (normal completion *and* error/shutdown paths)?

Any double-checked locking pattern — is the field `volatile`? Without it, the second thread can see a partially-constructed object (see [Chapter 13, *Concurrency \(Core\)*](#)'s Java Memory Model section).

Is `volatile` used where an actual atomic compound operation (`i++`, check-then-act) was needed instead? `volatile` only guarantees visibility, not atomicity of anything beyond a single read or write.

If multiple locks are acquired, is the acquisition order consistent everywhere, avoiding a lock-ordering deadlock?

Does any `CompletableFuture` chain swallow exceptions (no `.exceptionally().handle()`), or block on `.get()/.join()` on a thread that shouldn't block (e.g., an event-loop thread)?

If virtual threads are used, does any code hold a lock (`synchronized`) or otherwise pin the platform thread across a blocking call, defeating the point of virtual threads?

Double-Checked Locking Without `volatile`

```
// Flagged: classic double-checked locking bug – missing volatile
public class ExpensiveResource {
    private static ExpensiveResource instance;
    public static ExpensiveResource getInstance() {
        if (instance == null) {
            synchronized (ExpensiveResource.class) {
                if (instance == null) {
                    instance = new ExpensiveResource(); // can be observed
↪ half-built without volatile
                }
            }
        }
        return instance;
    }
}
```

```
// Fixed: volatile field restores the needed happens-before edge, OR use
↪ the holder idiom (Scenario 1) instead
public class ExpensiveResource {
    private static volatile ExpensiveResource instance;
    public static ExpensiveResource getInstance() {
        if (instance == null) {
            synchronized (ExpensiveResource.class) {
                if (instance == null) {
                    instance = new ExpensiveResource();
                }
            }
        }
        return instance;
    }
}
```

Without `volatile`, another thread can see `instance` as non-`null` while still observing the *partially initialized* object it points to, because the write to `instance` and the writes inside the constructor can be reordered from that thread's point of view. In review, the holder idiom (Scenario 1) is almost always the simpler recommendation over fixing double-checked locking by hand.

Deadlock Ordering

```
// Flagged: two methods acquire the same two locks in opposite order –
↳ classic deadlock
public class AccountTransfer {
    public void transfer(Account from, Account to, BigDecimal amount) {
        synchronized (from) {
            synchronized (to) {
                from.debit(amount);
                to.credit(amount);
            }
        }
    }
    // if some other code path calls transfer(to, from, ...) concurrently,
↳ thread A can hold
    // from's lock waiting for to's, while thread B holds to's lock
↳ waiting for from's – deadlock
}
```

```
// Fixed: always acquire locks in a consistent, global order (e.g., by
↳ account id)
public class AccountTransfer {
    public void transfer(Account from, Account to, BigDecimal amount) {
        Account first = from.id().compareTo(to.id()) < 0 ? from : to;
        Account second = first == from ? to : from;
        synchronized (first) {
            synchronized (second) {
                from.debit(amount);
                to.credit(amount);
            }
        }
    }
}
```

CompletableFuture Pitfalls

```
// Flagged: exception in the async stage is silently lost; nothing
↳ observes the completed future
CompletableFuture.supplyAsync(() -> paymentGateway.charge(request))
    .thenAccept(result -> ledger.record(result));
// if charge() or record() throws, this failure disappears – nobody calls
↳ get()/join(),
// and there's no .exceptionally().handle to observe it
```



```

// Fixed: handle the failure explicitly
CompletableFuture.supplyAsync(() -> paymentGateway.charge(request))
    .thenAccept(result -> ledger.record(result))
    .exceptionally(ex -> {
        log.error("Async charge/record pipeline failed", ex);
        alerting.pageOnCall("Payment pipeline failure: " +
↪ ex.getMessage());
        return null;
    });

```

Virtual Thread Pinning

```

// Flagged: synchronized block around a blocking call, run on a virtual
↪ thread – pins the carrier platform thread
public synchronized String fetchAndCache(String key) {
    String cached = cache.get(key);
    if (cached != null) return cached;
    String value = blockingHttpCall(key); // blocks while holding the
↪ monitor
    cache.put(key, value);
    return value;
}

```

A synchronized block that's holding its monitor while making a blocking call prevents the virtual thread from unmounting from its carrier platform thread during that block — under load, this can exhaust the small pool of carrier threads and stall unrelated virtual threads, defeating the scalability virtual threads exist to provide.

```

// Fixed: use a java.util.concurrent.locks.ReentrantLock, which does allow
↪ unmounting, or restructure
// to avoid holding any lock across the blocking call in the first place
private final ReentrantLock lock = new ReentrantLock();

public String fetchAndCache(String key) {
    String cached = cache.get(key);
    if (cached != null) return cached;
    String value = blockingHttpCall(key); // no lock held here at all
    lock.lock();
    try {
        cache.putIfAbsent(key, value);
    } finally {
        lock.unlock();
    }
    return cache.get(key);
}

```

```
}
```

See [Chapter 15, *Modern Concurrency \(Java 21+\)*](#) for virtual threads and structured concurrency in depth, and [Chapter 14, *Concurrency \(Advanced\)*](#) for the executor/CompletableFuture framework this section leans on.

Modern Java Feature Usage

Modern syntax is not automatically good style — review both directions: missed opportunities to simplify, *and* overuse that hurts readability. Background on each feature is in [Chapter 3, *Modern Java Language Features*](#); this section is specifically about judging usage.

Records — Good Fit

```
// Flagged: a hand-rolled immutable value class with all the
↪ equals/hashCode/toString boilerplate
public final class Point {
    private final int x, y;
    public Point(int x, int y) { this.x = x; this.y = y; }
    public int getX() { return x; }
    public int getY() { return y; }
    @Override public boolean equals(Object o) { /* ... */ return false; }
    @Override public int hashCode() { /* ... */ return 0; }
    @Override public String toString() { return "Point(" + x + ", " + y +
↪ ")"; }
}
```

```
// Fixed: a record generates the constructor, accessors,
↪ equals/hashCode/toString for free
public record Point(int x, int y) {}
```

Records — Bad Fit

```
// Flagged: a record with a mutable component defeats the entire point of
↪ using a record
public record OrderBatch(List<Order> orders) {} // caller can still mutate
↪ the passed-in list from outside
```

```

    // Fixed: defensively copy in the compact constructor, and expose an
    ↪ unmodifiable view
    public record OrderBatch(List<Order> orders) {
        public OrderBatch(List<Order> orders) {
            this.orders = List.copyOf(orders); // immutable copy, breaks the
    ↪ aliasing hazard
        }
    }

```

A record whose only mutable-looking component is an array or a mutable collection is a very common planted bug — the record's own fields are `final`, but that says nothing about whether the *referenced* object can still be mutated by the caller.

Pattern Matching vs. Polymorphism

```

    // Flagged: pattern-matching switch reimplementing what virtual dispatch
    ↪ already does better
    public double area(Shape shape) {
        return switch (shape) {
            case Circle c -> Math.PI * c.radius() * c.radius();
            case Square s -> s.side() * s.side();
            case Rectangle r -> r.width() * r.height();
            default -> throw new IllegalStateException();
        };
    }

```

```

    // Fixed: if this switch would need a new case every time a new Shape type
    ↪ is added,
    // and Shape is a type you own, plain polymorphism is simpler and can't
    ↪ forget a case
    sealed interface Shape { double area(); }
    record Circle(double radius) implements Shape { public double area() {
    ↪ return Math.PI * radius * radius; } }
    record Square(double side) implements Shape { public double area() {
    ↪ return side * side; } }
    record Rectangle(double width, double height) implements Shape { public
    ↪ double area() { return width * height; } }

```

Pattern matching earns its keep when the operation is genuinely external to the type (e.g., a serializer that needs to know about every type but the types themselves shouldn't know about serialization), or when combined with a sealed hierarchy so the compiler enforces exhaustiveness. It's a smell when it's just reinventing a method that `area()` on

the interface would express more simply — and a sealed hierarchy makes the compiler catch a missing case either way, so exhaustiveness isn't a deciding factor between the two styles.

var — Good and Bad

```
// Flagged: var hides the type where the type is exactly the information
↳ the reader needs
var result = process(data); // what does process() return? Not obvious
↳ from this line alone
```

```
// Fixed: keep the explicit type when it's not obvious from the right-hand
↳ side
ValidationResult result = process(data);
```

```
// Good use of var: the type is already obvious from the constructor call
↳ — no information lost
var orders = new ArrayList<Order>();
var connection = DriverManager.getConnection(url);
```

The rule of thumb: `var` is good when the type is redundant with something already visible on the same line (a constructor call, a well-named factory method); it's a readability regression when it hides the type of something returned by a method whose return type isn't obvious from its name.

Streams — Good and Bad

```
// Flagged: a stream pipeline that's harder to read than the equivalent
↳ loop, with a side-effecting peek
List<String> ids = orders.stream()
    .peek(o -> auditLog.add(o.id())) // side effect hidden inside peek
↳ — easy to miss, and peek isn't guaranteed to run for every element in all
↳ pipelines
    .filter(o -> o.total().compareTo(BigDecimal.TEN) > 0)
    .map(Order::id)
    .collect(Collectors.toList());
```

```

// Fixed: keep the side effect explicit and separate from the pipeline;
→ let the stream just transform data
List<String> ids = orders.stream()
    .filter(o -> o.total().compareTo(BigDecimal.TEN) > 0)
    .map(Order::id)
    .toList();
ids.forEach(auditLog::add); // or better, log alongside the filtering with
→ a plain loop if the two concerns are related

```

Streams earn their keep for a clear filter/map/collect pipeline over a collection. They're a smell when: the lambda has side effects (mutating something outside the stream — see Scenario 12), the pipeline is nested or nearly unreadable compared to an equivalent loop, or exceptions need awkward wrapping because the functional interfaces involved don't declare throws (see [Chapter 6, Exception Handling](#) and [Chapter 8, Functional Programming](#)).

Switch Expressions and Text Blocks

```

// Flagged: old-style switch statement with fall-through risk and repeated
→ assignment
String tier;
switch (score) {
    case 0: case 1: case 2: tier = "BRONZE"; break;
    case 3: case 4: tier = "SILVER"; break;
    default: tier = "GOLD";
}

```

```

// Fixed: switch expression – no fall-through risk, no repeated variable,
→ exhaustiveness is clearer
String tier = switch (score) {
    case 0, 1, 2 -> "BRONZE";
    case 3, 4 -> "SILVER";
    default -> "GOLD";
};

```

```

// Flagged: multi-line SQL/JSON built with escaped string concatenation
String query = "SELECT id, name\n" +
    "FROM customers\n" +
    "WHERE status = 'ACTIVE'\n";

```

```
// Fixed: a text block reads exactly like the text it represents
String query = """
    SELECT id, name
    FROM customers
    WHERE status = 'ACTIVE'
    """;
```

These two are close to unconditional wins — flag their *absence* as a missed simplification, not just misuse.

Common Code-Review Interview Pitfalls

These are mistakes candidates make *in the interview itself* — separate from the Java-language pitfalls earlier in this chapter. Interviewers see these constantly, and each one costs real points even when the candidate's technical instincts are otherwise sound.

1. **Bikeshedding a style nit while a data race sits three lines away.** Why it matters: it signals you can't triage severity, which is the actual skill being tested — finding bugs is only half the job.
2. **Not asking about the concurrency model before reviewing.** Why it matters: “is this called from multiple threads?” changes almost every finding in a snippet; guessing wrong wastes the whole review and can make you miss the one bug that mattered.
3. **Missing the concurrency bug entirely and only flagging surface-level issues (naming, formatting).** Why it matters: concurrency bugs are usually the actual point of the exercise; if the snippet has a shared mutable `HashMap` or an unsynchronized singleton, that has to be the headline finding.
4. **Listing every issue in file order with no prioritization.** Why it matters: a review that treats a missing `@Override` annotation with the same weight as a resource leak tells the interviewer you don't understand production risk.
5. **Being rude or accusatory about the (fictional) author's competence.** Why it matters: real code review is collaborative; interviewers are explicitly evaluating whether they'd want you reviewing a teammate's PR, not just whether you can spot bugs.
6. **Not verifying assumptions before declaring something broken.** Why it matters: jumping to “this is definitely a bug” about, say, an overload's resolution without checking the actual types involved can make you confidently state something wrong — worse than not spotting it, because it damages credibility on the things you got right.

7. **Over-engineering the fix — proposing a distributed lock, a message queue, or a rewrite for a five-line bug.** Why it matters: it shows you can't scope a fix to the actual problem, and real reviewers are wary of suggestions that would balloon the size and risk of a PR.
8. **Not asking about requirements or constraints before assuming the “obvious” fix is right.** Why it matters: what looks like an unbounded cache bug might be intentional if the process is short-lived and restarted daily — the right fix depends on context you don't have unless you ask.
9. **Silence — reading the code for a long time without narrating your thought process.** Why it matters: interviewers are assessing *how* you think, not just your final answer; silent reading gives them nothing to evaluate and often reads as being stuck.
10. **Fixating on one bug and running out of time to scan the rest of the snippet.** Why it matters: most review snippets have 2-4 issues by design; spending the entire session on the first one you spot means you miss the ones planted specifically to test breadth.
11. **Proposing a fix without explaining the mechanism behind the bug.** Why it matters: “just make it a `ConcurrentHashMap`” without explaining *why* the current code races sounds like memorized pattern-matching rather than understanding — interviewers probe follow-ups precisely to distinguish the two.
12. **Ignoring what's actually good about the code and only producing criticism.** Why it matters: real reviews acknowledge good decisions; an interview review that's 100% negative, even of intentionally-buggy code, reads as a habit rather than a calibrated judgment for this specific snippet.
13. **Getting defensive or argumentative when the interviewer pushes back or asks “are you sure?”** Why it matters: pushback is often a genuine probe to see if you'll re-examine your reasoning, not a challenge to win — treat it as an invitation to double-check, not an attack.
14. **Confusing “I would add a unit test for this” with actually explaining what the bug is.** Why it matters: proposing tests is good practice, but it doesn't substitute for identifying and explaining the defect itself — interviewers will ask “so what's actually wrong?” if you deflect into testing talk.
15. **Not distinguishing “this will definitely break” from “this could theoretically be an issue under conditions we haven't confirmed.”** Why it matters: conflating certain bugs with speculative ones misleads the listener about urgency — say which is which.

- 16. Trying to rewrite the entire snippet from scratch instead of making the minimal fix.** Why it matters: real reviews respect the existing design and diff size where possible; a wholesale rewrite in an interview eats your limited time and often introduces new bugs of its own.
- 17. Forgetting to mention severity labels or any triage vocabulary at all.** Why it matters: explicit labels (blocker/major/minor/nit) are what make a review actionable for a team lead deciding whether to merge — omitting them makes your feedback harder to act on even if it's technically correct.
- 18. Not asking whether this code is even meant to run in production as-is, versus being a simplified teaching example.** Why it matters: some interview snippets are deliberately minimal and not every “issue” (e.g., missing logging, missing tests) is worth raising with the same weight as an actual correctness or safety bug — read the intent of the exercise.
- 19. Spending so long on the review checklist ritual that you never get to the actual snippet.** Why it matters: the checklist in this chapter is a tool to internalize beforehand, not a script to recite out loud during the interview — using it as a mental scan, not a spoken monologue, keeps the conversation moving.
- 20. Treating the interviewer as an adversary to be defeated rather than a colleague to collaborate with.** Why it matters: the entire point of the exercise is simulating a real code review conversation — the candidates who do best treat it exactly like one, asking questions, taking suggestions, and building toward the same goal as the interviewer: correct, safe, maintainable code.